

Colección
Recursos Informáticos

www.FreeLibros.me

C# 5

Los fundamentos del lenguaje

Desarrollar con

Visual Studio 2012

Descarga

www.ediciones-eni.com

Thierry GROUSSARD

 **INFORMÁTICA TÉCNICA**



C# 5

Los fundamentos del lenguaje - Desarrollar con Visual Studio 2012

Este libro sobre C# se dirige a los **desarrolladores**, incluso principiantes, que desean dominar el lenguaje C# en su versión 5.

Después de una descripción del entorno de desarrollo (**Visual Studio 2012**), el lector descubrirá las bases de la **programación orientada a objetos** con C#. Evolucionará gradualmente hacia su puesta en marcha con el desarrollo **de aplicaciones Windows Form**. Las novedades que presenta este lenguaje en lo relativo a la **programación asíncrona** le permitirán mejorar el rendimiento y la reactividad de sus aplicaciones. Los numerosos ejemplos y consejos de uso de las **herramientas de depuración** le proporcionarán una gran ayuda para la implementación de una aplicación.

Se dedica un capítulo al acceso a las bases de datos con la ayuda de **ADO.NET** y de **SQL**, lo que le permitirá evolucionar hacia el desarrollo de aplicaciones cliente-servidor. También se detallan las potentes **funcionalidades de LINQ** para facilitar el acceso a los datos y el trabajo con ellos. Igualmente se presenta el uso del **lenguaje XML**, ya que facilita **el intercambio de datos** con otras aplicaciones.

Los usuarios de las versiones anteriores descubrirán las **novedades y mejoras** de esta versión 2012 para desarrollar aún más rápida y fácilmente aplicaciones para el **framework .NET 4.5**.

Se presenta la distribución de una aplicación utilizando **Windows Installer** y la tecnología **Click Once**.

Los capítulos del libro:

Prólogo – Presentación de la plataforma .NET – Presentación de Visual Studio – Organización de una aplicación – Fundamentos del lenguaje – Programación orientada a objetos – Gestión de los errores y depuración del código – Aplicaciones de Windows – Acceso a las bases de datos – Presentación de LINQ – Utilización de XML – Despliegue de componentes y aplicaciones

Thierry GROUSSARD

Después de más de 10 años como analista y desarrollador, **Thierry Groussard** se orientó a la formación, particularmente en el campo del desarrollo. Sus profundos conocimientos de las necesidades de la empresa y sus cualidades pedagógicas hacen que sus libros estén especialmente adaptados al aprendizaje y a la puesta en práctica del desarrollo en C#.

Índice

Información

Título, autor...

Prólogo

Prólogo

Presentación de la plataforma .NET

Introducción

Escritura, compilación y ejecución de una aplicación

Presentación de Visual Studio

Organización de una aplicación

Fundamentos del lenguaje

Programación orientada a objetos

Gestión de los errores y depuración del código

Aplicaciones de Windows

Acceso a las bases de datos

Presentación de LINQ

Utilización de XML

Despliegue de componentes y aplicaciones

Prólogo

Desde la primera versión aparecida con Visual Studio en 2002, el lenguaje C# siguió una evolución constante hasta esta versión 5.0. Actualmente es el lenguaje de referencia de Microsoft. Para convencerse de ello, basta consultar los numerosos recursos disponibles en Internet referentes a la plataforma .NET y darse cuenta de que la mayoría de los ejemplos propuestos se desarrollan con este lenguaje.

El objetivo de este libro consiste en presentar las bases de este lenguaje para permitirle aprovechar lo mejor posible las funcionalidades de la versión 4.5 del Framework .NET. Después del aprendizaje de estas bases, usted tendrá todas las cartas en la mano para tratar el diseño de aplicaciones gráficas.

Sus futuras aplicaciones necesitarán trabajar seguramente con información ubicada en una base de datos. Los dos capítulos dedicados a este tema le aportarán una ayuda preciosa para llevar a cabo esta tarea. El primero le familiarizará con la utilización de ADO.NET, que es la tecnología clásica de Microsoft para la gestión del acceso a una base de datos. El segundo presentará el lenguaje LINQ, cuyo principal objetivo consiste en uniformizar los accesos a los datos de una aplicación, y ello, sea cual sea el origen de estos datos (base de datos, archivos XML, objetos...).

El despliegue es por supuesto la última etapa de la elaboración de una aplicación, pero no por ello se debe desatender. Las dos tecnologías de despliegue disponibles se tratan en el último capítulo de este libro para permitirle simplificar la instalación de sus aplicaciones en los puestos clientes.

Este libro no tiene como vocación sustituir la documentación del Framework .NET, que debe seguir siendo su referencia para obtener datos como la lista de los métodos o propiedades presentes en una clase.

Subir

[Best Cell Phone](#)

[Direct Tv Offers](#)

[Gmail Account](#)

[New Cell Phone](#)

[Crossover SUV](#)

[Online Payroll](#)

Índice

Información

Título, autor...

Prólogo

Prólogo

Presentación de la plataforma .NET

Introducción

Escritura, compilación y ejecución de una aplicación

Presentación de Visual Studio

Organización de una aplicación

Fundamentos del lenguaje

Programación orientada a objetos

Gestión de los errores y depuración del código

Aplicaciones de Windows

Acceso a las bases de datos

Presentación de LINQ

Utilización de XML

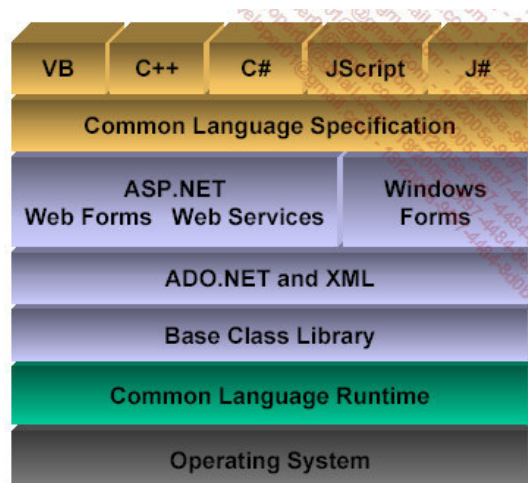
Despliegue de componentes y aplicaciones

Introducción

La plataforma .NET pone a su disposición un conjunto de tecnologías y herramientas que simplifican el desarrollo de aplicaciones y propone una solución para casi cualquier tipo de aplicaciones:

- aplicaciones Windows clásicas;
- aplicaciones Web;
- servicios Windows;
- servicios Web.

Todas estas aplicaciones se pueden realizar gracias a un elemento esencial: el Framework .NET. Este Framework se encarga, por medio de numerosas capas de software superpuestas, de la integridad de la vida de una aplicación, desde el desarrollo hasta la ejecución. El sistema operativo, con el que va a interactuar, debe albergar el framework. El primer sistema que permite acogerlo es, por supuesto, Windows, pero hay otras versiones disponibles que permiten la adaptación de la plataforma .NET a sistemas tales como Linux o Unix.



El framework contiene dos elementos principales: el Common Language Runtime y la librería de clases

[Best Cell Phone](#)

[Direct Tv Offers](#)

[Gmail Account](#)

[New Cell Phone](#)

[Crossover SUV](#)

[Online Payroll](#)

Escritura, compilación y ejecución de una aplicación

En este capítulo, vamos a detallar el ciclo de vida de una aplicación desde la redacción del código hasta la ejecución de la aplicación, estudiando en detalle los mecanismos puestos en marcha.

1. Escritura del código

La inmensa mayoría de las aplicaciones se desarrollan gracias a un entorno integrado que agrupa las principales herramientas necesarias, a saber:

- un editor de texto;
- un compilador;
- un depurador.

Este enfoque es, de lejos, el más cómodo. Sin embargo necesita una pequeña fase de aprendizaje para familiarizarse con la herramienta. Para nuestra primera aplicación, vamos a utilizar una manera de hacer un poco diferente, ya que vamos a utilizar herramientas individuales: el bloc de notas de Windows para la escritura del código y el compilador en línea de comandos para Visual C#.

Nuestra primera aplicación será muy sencilla, ya que visualizará simplemente el mensaje «Hola» en una ventana de comando. A continuación se presenta el código de nuestra primera aplicación, que luego explicaremos línea por línea. Se debe introducir usando el bloc de notas de Windows o cualquier otro editor de texto siempre y cuando éste no añada ningún código de formato en el interior del documento, como sí hacen por ejemplo programas de tratamiento de texto.

Ejemplo

```
using System;
class Program
{
    static String mensaje = "Hola";
    static void Main(String[] args)
    {
        Console.WriteLine(mensaje);
    }
}
```

Se debe guardar este código en un archivo con la extensión .cs. Esta extensión no es obligatoria, pero permite respetar las convenciones utilizadas por Visual Studio. Detallamos ahora algunas líneas de nuestra primera aplicación.

```
using System
```

Esta línea permite dejar directamente accesibles los elementos presentes en el namespace System. Sin ella, habría que utilizar los nombres plenamente cualificados para todos los elementos contenidos en el namespace. En nuestro caso, deberíamos utilizar entonces: `System.Console.WriteLine("Hola");`

```
class Program
```

En Visual C#, cualquier porción de código debe estar contenida en una clase.

```
static String mensaje= "Hola";
```

Esta línea declara una variable. Se debe declarar todas las variables antes de poder utilizarlas. La declaración permite especificar el tipo de información que la variable va a contener: aquí, una cadena de caracteres y eventualmente un valor inicial, «hola» en nuestro caso.

```
static void Main (String[]args)
```

Todas las instrucciones, aparte de las declaraciones, deben estar ubicadas en un procedimiento o una función. La mayor parte del código se sitúa entonces entre los caracteres { y } , delimitando cada procedimiento o función. Entre todos los procedimientos y funciones, se designa a uno de ellos como el punto de entrada en la aplicación. A través de la ejecución de este procedimiento arranca la aplicación. Este procedimiento se debe llamar Main y debe ser estático. Se debe declarar en el interior de una clase o estructura. El tipo de retorno puede ser `void` o `int`. Los parámetros son optativos y, si se utilizan, representan los argumentos pasados en la línea de comando.

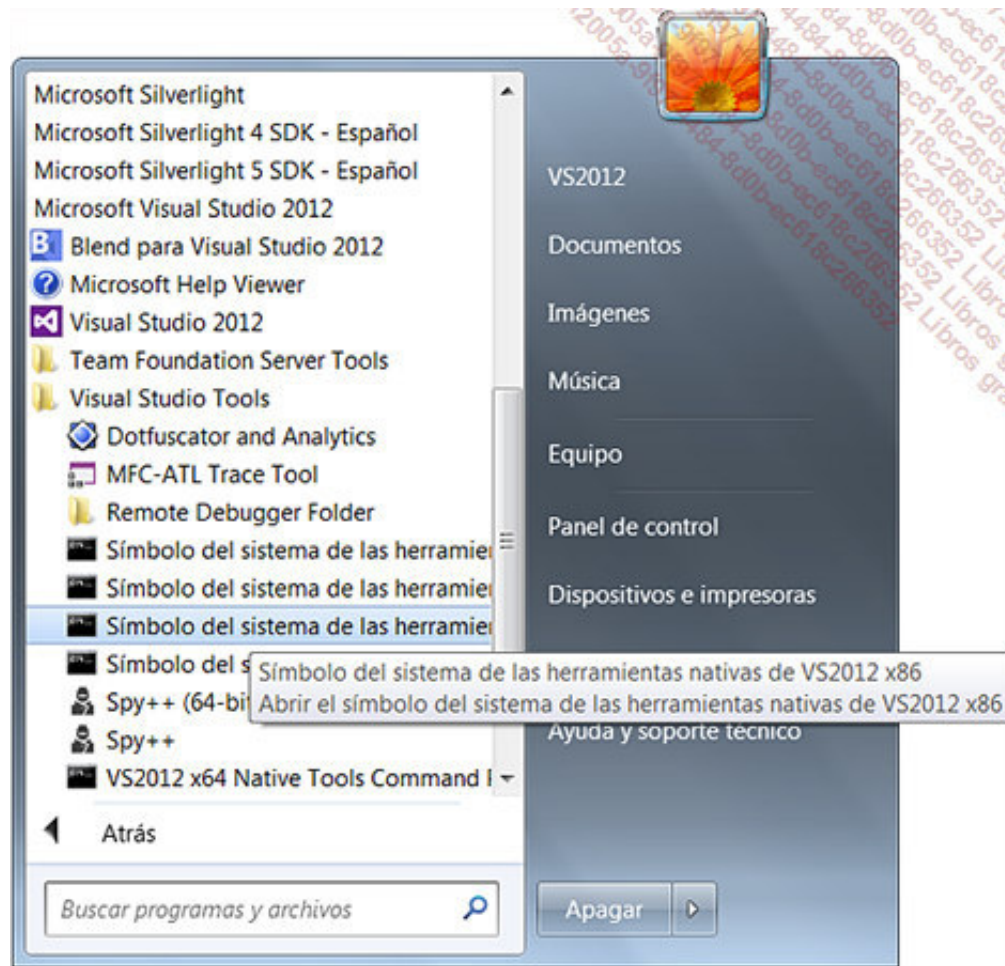
```
Console.WriteLine("Hola");
```

La clase Console definida en el espacio de nombres System provee un conjunto de métodos que permite la visualización de datos en la consola o la lectura de datos desde la consola. El procedimiento WriteLine permite la visualización de una cadena de caracteres en la consola.

Cabe destacar también que Visual C# distingue entre las minúsculas y las mayúsculas en las instrucciones. Si usted utiliza el editor de Visual Studio para redactar su código, éste le guiará para evitar errores (IntelliSense).

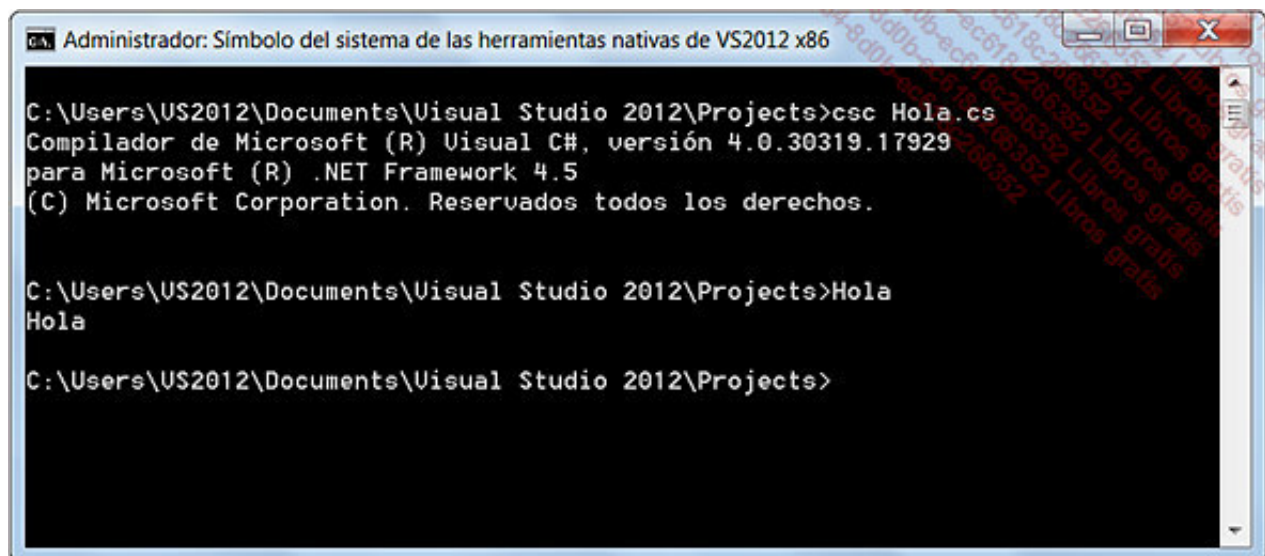
2. Compilación del código

El Framework .NET incluye un compilador en línea de comando para Visual C#. Para compilar el código fuente de nuestro ejemplo, debemos abrir una ventana de comando DOS para poder lanzar el compilador. Para ello la instalación creó un atajo en el menú Inicio. Este atajo lanza la ejecución de un archivo .bat que posiciona algunas variables de entorno necesarias para el correcto funcionamiento de las herramientas Visual Studio en línea de comando.



Desde la ventana de comandos abierta, conviene situarse en el directorio en el cual se encuentra el archivo fuente. Se lanza la compilación con el comando `csc Hola.cs`.

Después de un breve instante, el compilador nos devuelve el control. Podemos comprobar la presencia del archivo ejecutable y comprobar su correcto funcionamiento.



Nuestra primera aplicación es realmente muy sencilla. Para aplicaciones más complejas, será útil a veces especificar algunas opciones para el funcionamiento del compilador. El conjunto de las opciones disponibles se puede obtener con el comando `csc / ?`.

Las principales opciones son:

`/out:archivo.exe`

Esta opción permite especificar el nombre del archivo resultado de la compilación. Por defecto, es el nombre del archivo fuente en curso de compilación que se utiliza.

`/target:exe`

Esta opción pide al compilador la generación de un archivo ejecutable para una aplicación en modo consola.

`/target:winexe`

Esta opción pide al compilador la generación de un archivo ejecutable de aplicación de Windows.

`/target:library`

Esta opción pide al compilador la generación de un archivo librería dll.

`/referencia:lista de archivos`

Esta opción indica al compilador la lista de los archivos referenciados en el código y necesarios para la compilación. Los nombres de los archivos se deben separar con una coma.

3. Análisis de un archivo compilado

Ahora que se ha creado nuestro archivo ejecutable, intentemos ver lo que contiene.

Primera solución: abrirlo con el bloc de notas de Windows

Hola.txt - Bloc de notas

Archivo Edición Formato Ver Ayuda

VZ0 0 0 yy @ € 00°0
! !,OLi!This program cannot be run in DOS mode.
\$ PE L00 G<AC à 00000 0 0 0 . @ @ 0 W
@ 0 0 0 0 0 0 0 0 0 0 Ä-
0 0 H .text \$0 0 0
.rsrc 0 @ 0 0 @ @.reloc 0 0
0 B . H 0 0 ð! 00 0 0 0
00(0 0*00(0 0*000 0
0 0~0 000 00+ 0* 000 0 0 0~0 000 00+ 0* 000 0 0 0~0
000 00+ 0* 000 0 0 0~0 000 00+ 0* !s0 0€0 0s 0€0 0s0 0
€0 0s0 0€0 0* 00(0 0*000 0 0 000(0 0(0 00+ 0* 000 0 0
00(0 00+ 0* 000 0 0 000 0(0 00+ 0* 000 0 0 00(0 00+ 0* 000 0
00€0 0-0(0 +0+0+000+ 0*000 0 0 000 p00 0000
0*00(0 0*000 0 0~0 0€0 0-0(0 +€0 0~0 00+ 0*.ro p(0 0
*BSJB0 0 0 v2.0.50215 0 l È0 #~ 40 €0 #Strings A
0 #US Ö 0 #GUID ä ð0 #Blob 0 0w04
0 ú03 0
0 0 0 }00 0 \$ } 0 0½ 0 è ä 0 <0} 0 Åä0 T0>00 m0>00
00,00 Å00 0 00U00 00000 V0600 T0ä 0 ~0ä 0 ±00 0 è0100 Y0ä 0 00600 00ä
0 B0ä 0 U060 0 0) 7 0 0 : 7 0 0 E 7 0
0 0 00 O 0 0 0 00] 0 0 0 00 x 0 0 0 1 Ÿ 0 1 (0) 1 J06 1 00C
0 000 P 00. 0 0 X 00. 0 0 00ð 0 0 | 0000\$ 0
00A01 0 00_0> 0 Ø 0000K 0 ü 00. 0 0 0! F0~0c 0 \$!
F0µ0h 0 <! f Å0l 0 X! F0f0q 0 p! 0 x0x 0 "! 0 ð0€ 0 *!
00. 0 0 ! 00 0< 0 ä! 0 90K 0 0 "0 0 i0 0 i0 . 0 1
. "0 . 0 \$ 0< , 0< 4 0< < 0< \$. 0 , . 0 4 . 0 < . 0 A .
0 I . 0 Q . Ÿ Y . 0 0 . 0 a eo(00 ~0c 0 µ0h) t0500 f0q q 40E0y . U00
000 0 . 00% . 0 . 0 " 8000j . 0 @ . à0) s 00. ó å0c 0 Ÿ I s ä { Ø
C 0 Ÿ i s ö € { Ø f k Ø f c Ø % s 00 { Ø i Ø Ø ; 0 Ø é Ø Ÿ é } »0A {
Ø Å É Ø0Å 0 Ÿ ä { Ø ä c Ø 00 Ÿ 0{ Ø 000 Ÿ 00{ Ø '0{ Ø '00 Ÿ €0{ Ø
€00 Ÿ 0{ Ø Å0{ Ø 0{ Ø 0è Ø ~ » Ç Ó -010<0A0P0P0É00 0 0 0 0 O
"0T <0Y 0^ -0" 0 0 0 0 0 0 0 0 0 0 0 0 0 u u ^ " A i
¿00€ u0
0 ä 0 0 0 0 0 0 0 ê0 0 0 ê0 0 ê0- K0- Å0 <Module>
mscorlib Microsoft.VisualBasic MyApplication My MyComputer MyProject
MywebServices ThreadSafeObjectProvider`1 test
Microsoft.VisualBasic.ApplicationServices ApplicationBase .ctor
Microsoft.VisualBasic.Devices.Computer System.Object get_Computer
m_ComputerObjectProvider get_Application m_AppObjectProvider User
get_User m_UserObjectProvider get_WebServices
m_MywebServicesObjectProvider .cctor Application webServices Equals o

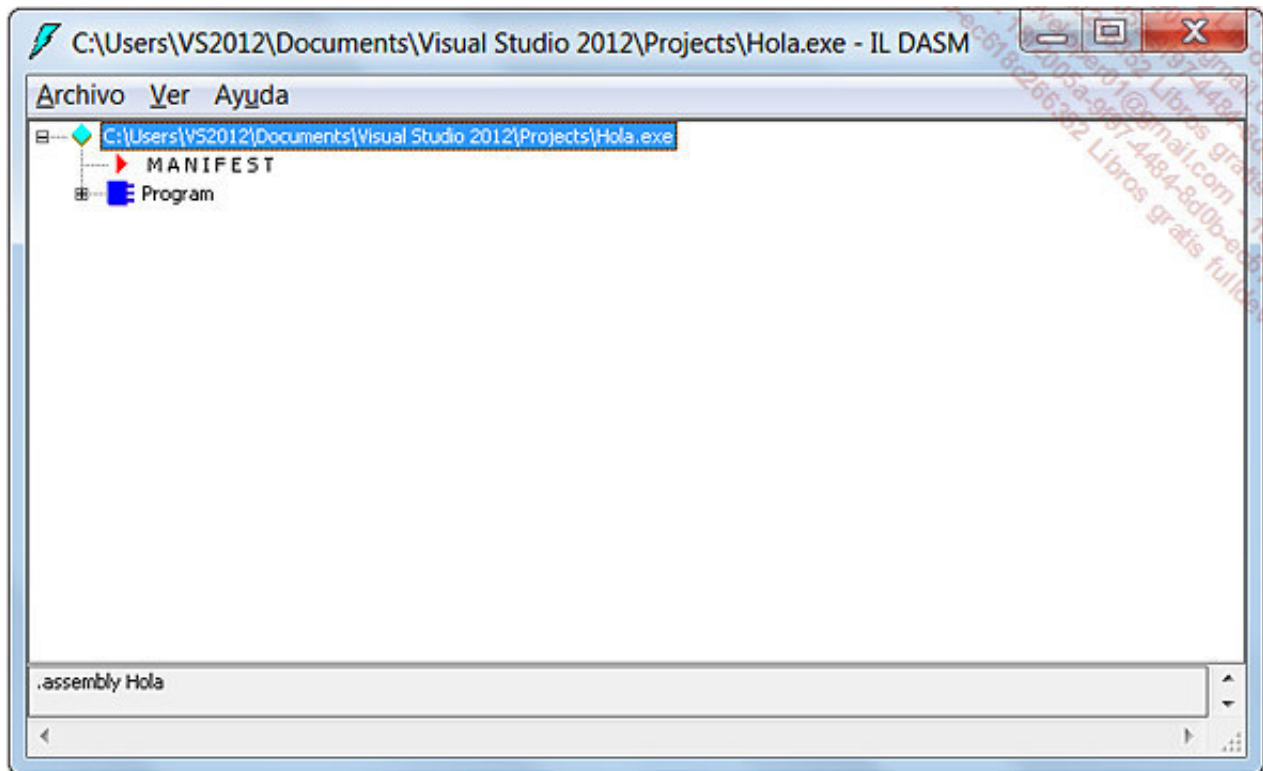
El resultado no es muy elocuente, ies lo menos que puede decirse!

Hemos dicho que el compilador genera código MSIL. Por lo tanto es este código lo que visualizamos en el bloc de notas. Para visualizar el contenido de un archivo MSIL, el Framework .NET propone una herramienta mejor adaptada.

Segunda solución: utilizar un desensamblador

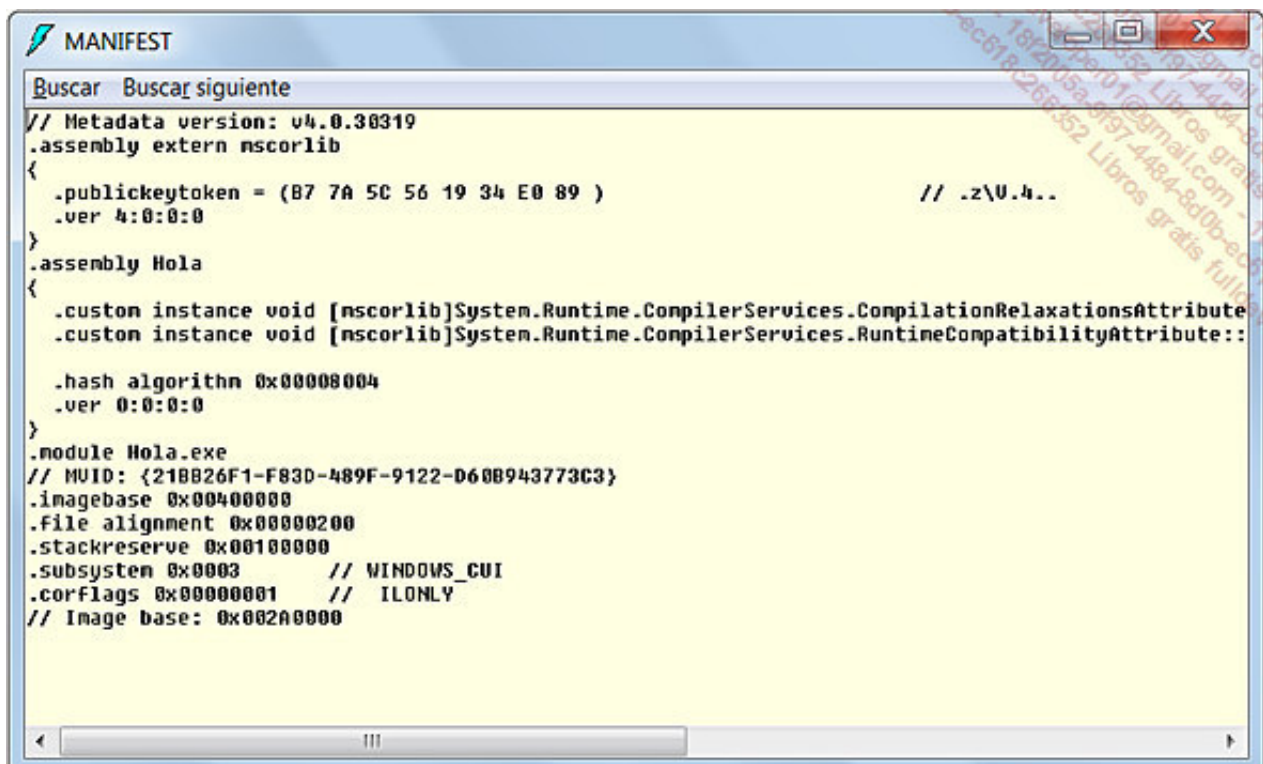
Esta herramienta se ejecuta a partir de la línea de comando con la instrucción `ildasm`.

Permite visualizar un archivo generado por el compilador, más claramente que con el bloc de notas. Conviene indicar el archivo que se desea examinar por el menú **Archivo - Abrir**. El desensamblador visualiza entonces su contenido.



La información presente en el archivo se puede separar en dos categorías: el manifiesto y el código MSIL. El manifiesto contiene los metadatos que permiten describir el contenido del archivo y los recursos que necesita. Hablamos en este caso de archivo autodescriptivo. Esta técnica es muy interesante, ya que en cuanto el Common Language Runtime lee el archivo, dispone de toda la información necesaria para su ejecución.

Ya no es necesario utilizar una grabación en el registro de la máquina. Se puede visualizar el manifiesto con un doble clic en su nombre.

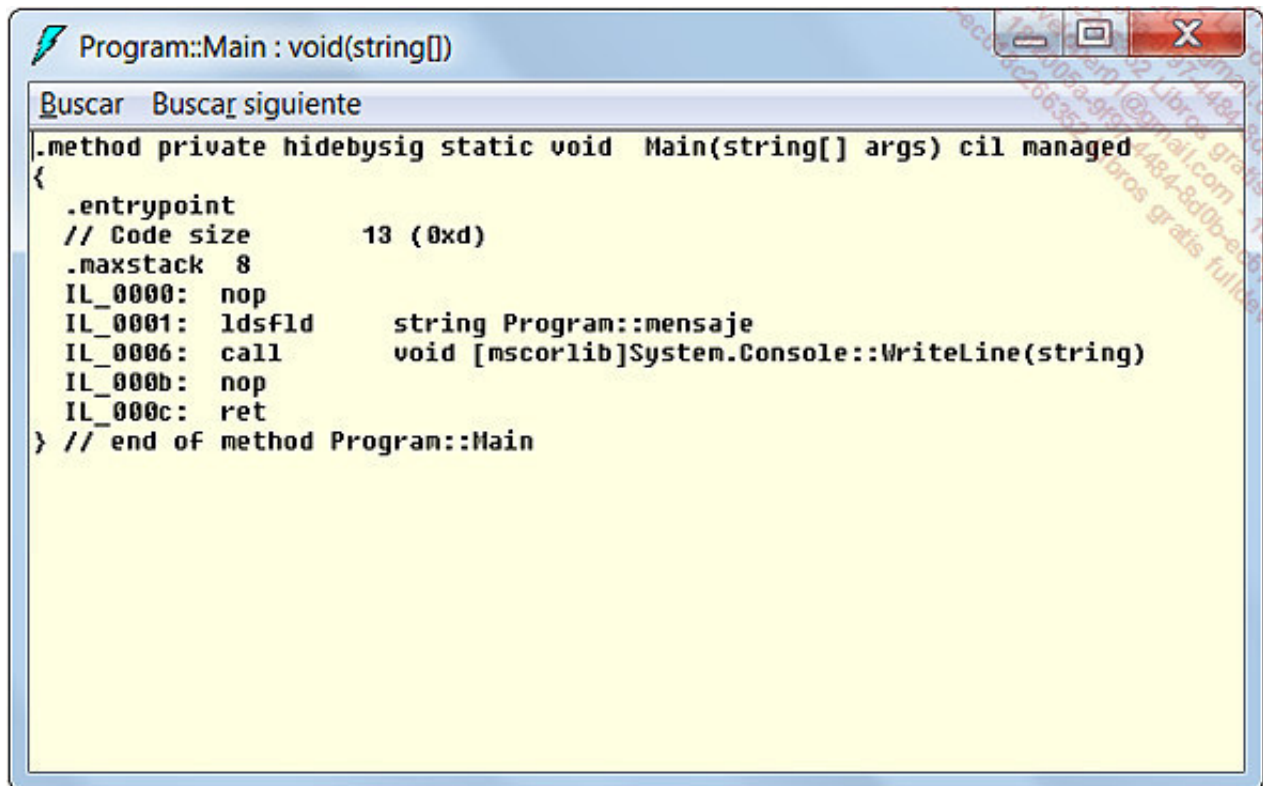


Encontramos en este manifiesto datos que indican que, para poder funcionar, la aplicación necesita el ensamblado externo mscorlib.

La segunda parte corresponde realmente al código MSIL. Un conjunto de iconos se utiliza para facilitar la visualización de los datos.

Símbolo	Significado
	Más información
	Espacio de nombres
	Clase
	Interfaz
	Clase de valores
	Enumeración
	Método
	Método estático
	Campo
	Campo estático
	Evento
	Propiedad
	Elemento de manifiesto o de información de clase

Como en el caso del manifiesto, un doble clic en un elemento permite obtener más detalles. Así podemos, por ejemplo, visualizar la traducción de nuestro procedimiento Main.

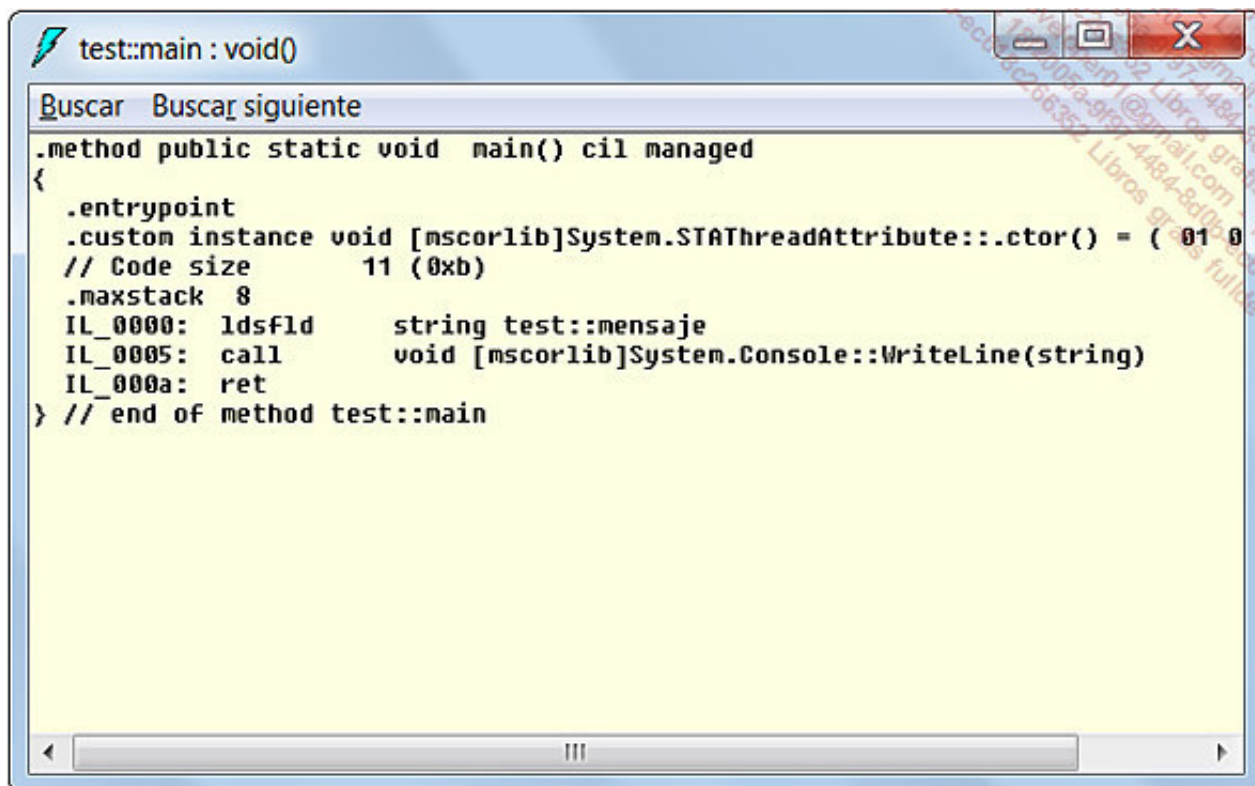


En un ejemplo de código tan sencillo, es fácil relacionar el código Visual C# y su traducción en código MSIL. Para las personas entusiasmadas por el código MSIL, existe un ensamblador MSIL: `ilasm`. Esta herramienta acepta como parámetro un archivo de texto que contiene código MSIL y lo transforma en formato binario.

Ya que somos capaces visualizar el código MSIL, podemos verificar que es realmente independiente del lenguaje fuente utilizado para desarrollar la aplicación. A continuación veamos el código Visual Basic que realiza lo mismo que nuestro código Visual C#.

```
using System
Imports System
Public Module test
    Dim mensaje As String = "Hola"
    Public Sub main()
        console.writeline(mensaje)
    End Sub
End Module
```

Tras la compilación y desensamblaje por `ildasm`, veamos lo que nos presenta para el método Main.



No hay ninguna diferencia con respecto a la versión Visual C# del método Main.

También es posible dar los pasos inversos al transformar un archivo texto que contiene código MSIL en archivo binario correspondiente. Esta transformación se hace gracias al ensamblador ilasm. La única dificultad consiste en crear un archivo texto que contiene el código MSIL, ya que incluso si la sintaxis es comprensible, no es intuitiva. Una solución puede consistir en pedir a la herramienta ildasm (el desensamblador) que genere este archivo de texto. Para ello, después de haber abierto el archivo ejecutable o la librería dll con ildasm, usted debe utilizar la opción **Volcar** del menú **Archivo**. Se le invita entonces a elegir el nombre del archivo que hay que generar (extension .il).

Este archivo se puede modificar con un simple editor de texto. Sustituya, por ejemplo, el contenido de la variable mensaje con la cadena «Hello».

```
.method private hidebysig specialname rtspecialname static
    void .cctor() cil managed
{
    // Code size          11 (0xb)
    .maxstack 8
    IL_0000: ldstr        "Hello"
    IL_0005: stsfld        string Program::mensaje
    IL_000a: ret
} // end of method Program::.cctor
```

Guarde luego el archivo. Ahora sólo queda volver a generar el archivo ejecutable gracias al ensamblador ilasm. Para ello, introduzca la línea de comando siguiente:

```
ilasm Hola.il /output=Hello.exe
```

La opción /output=Hello permite indicar el nombre del archivo generado. Si no se especifica esta opción, se utilizará el nombre del archivo fuente. Usted puede ahora lanzar el nuevo ejecutable y verificar el mensaje visualizado. Todas estas operaciones se pueden hacer en cualquier archivo ejecutable o librería dll. La única dificultad reside en el volumen de información facilitado por la

descompilación. Sin embargo, esto crea un problema: cualquier persona que dispone de los archivos ejecutables o librerías dll de una aplicación puede modificar la aplicación.

Por supuesto las modificaciones pueden resultar peligrosas, pero se puede considerar la modificación de un valor que representa una información importante para la aplicación (contraseña, clave de licencia...) Un remedio posible a este tipo de operación consiste en hacer lo más incomprensible posible el código generado por el descompilador. Para ello, hay que actuar a nivel del archivo ejecutable o de la librería dll con la modificación de los datos que contienen sin, por supuesto, perturbar el funcionamiento. Hay herramientas llamadas ofusadores que son capaces de realizar esta operación. Visual Studio se suministra con una herramienta de la empresa PreEmptive Solutions llamada Dotfuscator Community Edition. Esta versión permite realizar las operaciones básicas para «embrollar» un archivo. El principal tratamiento efectuado en el archivo consiste en renombrar los identificadores contenidos en él (nombre de las variables, nombre de los procedimientos y funciones...) con valores muy poco explícitos, en general a carácter único. Ahí tenemos un extracto de la descompilación del archivo Hola.exe tras su tratamiento por Dofuscator Community Edition.

```
.class public auto ansi sealed beforefieldinit DotfuscatorAttribute
    extends [mscorlib]System.Attribute
{
    .custom instance void [mscorlib]System.AttributeUsageAttribute::.ctor(value-
type [mscorlib]System.AttributeTargets) = ( 01 00 01 00 00 00 00 00 )
    .field private string a
    .method public hidebysig specialname rtspecialname
        instance void .ctor(string a) cil managed
    {
        // Code size          14 (0xe)
        .maxstack 2
        IL_0000: ldarg.0
        IL_0001: dup
        IL_0002: call          instance void [mscorlib]System.Attribute::.ctor()
        IL_0007: ldarg.1
        IL_0008: stfld          string DotfuscatorAttribute::a
        IL_000d: ret
    } // end of method DotfuscatorAttribute::.ctor
    .method public hidebysig string
        a() cil managed
    {
        // Code size          7 (0x7)
        .maxstack 1
        IL_0000: ldarg.0
        IL_0001: ldfld          string DotfuscatorAttribute::a
        IL_0006: ret
    } // end of method DotfuscatorAttribute::a
    .property instance string A()
    {
        .get instance string DotfuscatorAttribute::a()
    } // end of property DotfuscatorAttribute::A
} // end of class DotfuscatorAttribute

.class private auto ansi beforefieldinit a
    [mscorlib]System.Object
{
    .field private static string a
    .method private hidebysig static void a(string[] A_0) cil managed
    {
        .entrypoint
        // Code size          13 (0xd)
        .maxstack 8
        IL_0000: nop
    }
}
```

```

IL_0001: ldsfld      string a::a
IL_0006: call       void [mscorlib]System.Console::WriteLine(string)
IL_000b: nop
IL_000c: ret
} // end of method a::a
.method public hidebysig specialname rtspecialname
    instance void .cil managed
{
    // Code size          7 (0x7)
    .maxstack 8
    IL_0000: ldarg.0
    IL_0001: call     instance void [mscorlib]System.Object::.ctor()
    IL_0006: ret
} // end of method a::.ctor
.method private hidebysig specialname rtspecialname static
    void .cctor() cil managed
{
    // Code size          (0xb)
    .maxstack 8
    IL_0000: ldstr     "Hola"
    IL_0005: stsfld     string a::a
    IL_000a: ret
} // end of method a::.cctor
} // end of class a

```

En este archivo, no queda rastro de los nombres utilizados en el código. La clase se llama a, el procedimiento Main se llama ahora «a», la variable mensaje se llama también ahora «a». ¡Imagínese el resultado de tal tratamiento en un archivo que contiene varias decenas de variables y procedimientos!

La versión Professional Edition permite también la encriptación de las cadenas de caracteres, la modificación y el añadido de código inútil para complicar las estructuras de controles (bucles, condiciones...).

A continuación presentamos un ejemplo de transformación de la documentación de Dotfuscator.

El código original:

```

public int CompareTo(Object o)
{
    int n = occurrences - ((WordOccurrence)o).occurrences;
    if (n == 0)
    {
        n = String.Compare(word, ((WordOccurrence)o).word);
    }
    return(n);
}

```

El código generado:

```

public virtual int _a(Object A_0) {
    int local0;
    int local1;
    local0 = this.a - (c) A_0.a;
    if (local0 != 0) goto i0;
    goto il;
    while (true) {
        return local1;
    i0: local1 = local0;
    }
    il: local0 = System.String.Compare(this.b, (c) A_0.b);
}

```

```
goto i0;  
}
```

¡El análisis de miles de líneas de código de este tipo puede provocar algunas migrañas! Por lo tanto, es preferible conservar el código original para las modificaciones posteriores. Dispone de más información en el sitio <http://www.preemptive.com/>

4. Ejecución del código

Cuando un usuario ejecuta una aplicación gestionada, el cargador de código del sistema operativo carga el Common Language Runtime que luego lanza la ejecución del código gestionado. Como el procesador de la máquina en la cual se ejecuta la aplicación no puede encargarse directamente del código MSIL, el Common Language Runtime debe convertirlo a código nativo.

Esta conversión no incluye la totalidad del código de la aplicación. Convierte el código según las necesidades. Los pasos adoptados son los siguientes:

- Al cargar una clase, el Common Language Runtime sustituye cada método de la clase con un trozo de código que requiere al compilador JIT que lo compile en lenguaje nativo.
- Luego, cuando se utiliza el método en el código, la porción de código generado en la carga entra en acción y compila el método en código nativo.
- El fragmento de código que requiere la compilación del método es sustituido luego por el código nativo generado.
- Las futuras llamadas de este método se harán directamente en el código nativo generado.

Instalación y primer arranque

1. Configuración necesaria

Para permitir un correcto funcionamiento, Visual Studio necesita una configuración mínima. Microsoft aconseja los siguientes valores:

Componente	Mínimo recomendado	Prestaciones óptimas
Procesador	Pentium 1,6 GHz o equivalente	Pentium 2,2 GHz o equivalente
RAM	1.024 MB	2.048 MB o más
Espacio en disco	1 GB en el disco del sistema y de 2,8 a 5 GB en otro disco	
Vídeo	1.024 x 768	1.280 x 1.024 o superior
Lector de DVD	Indispensable	Indispensable
Sistema operativo	Windows 7 Microsoft Windows Server 2008	Cualquier versión posterior (Windows 8, Windows Server 2012)

Procedimiento de instalación

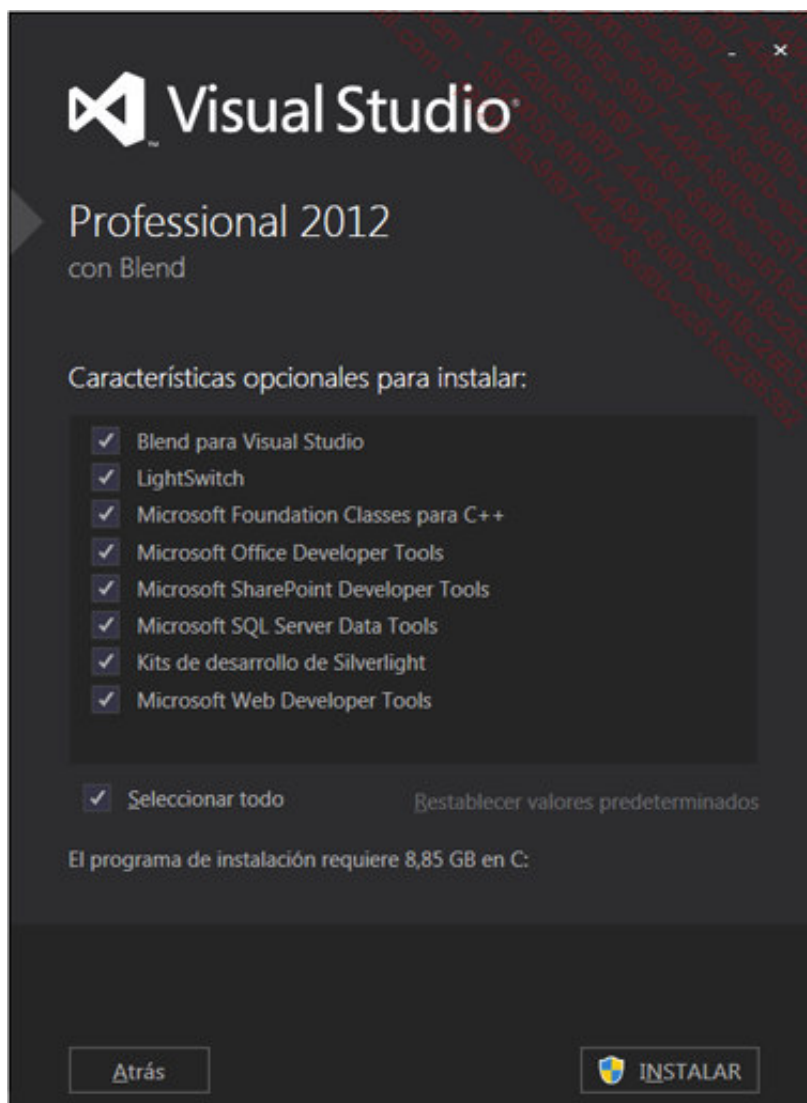
Los elementos necesarios son:

- el DVD de Visual Studio.NET;
- espacio disponible en su disco duro (de 5 a 9 GB en función de las herramientas instaladas);
- y sobre todo paciencia, ya que la instalación es larga...

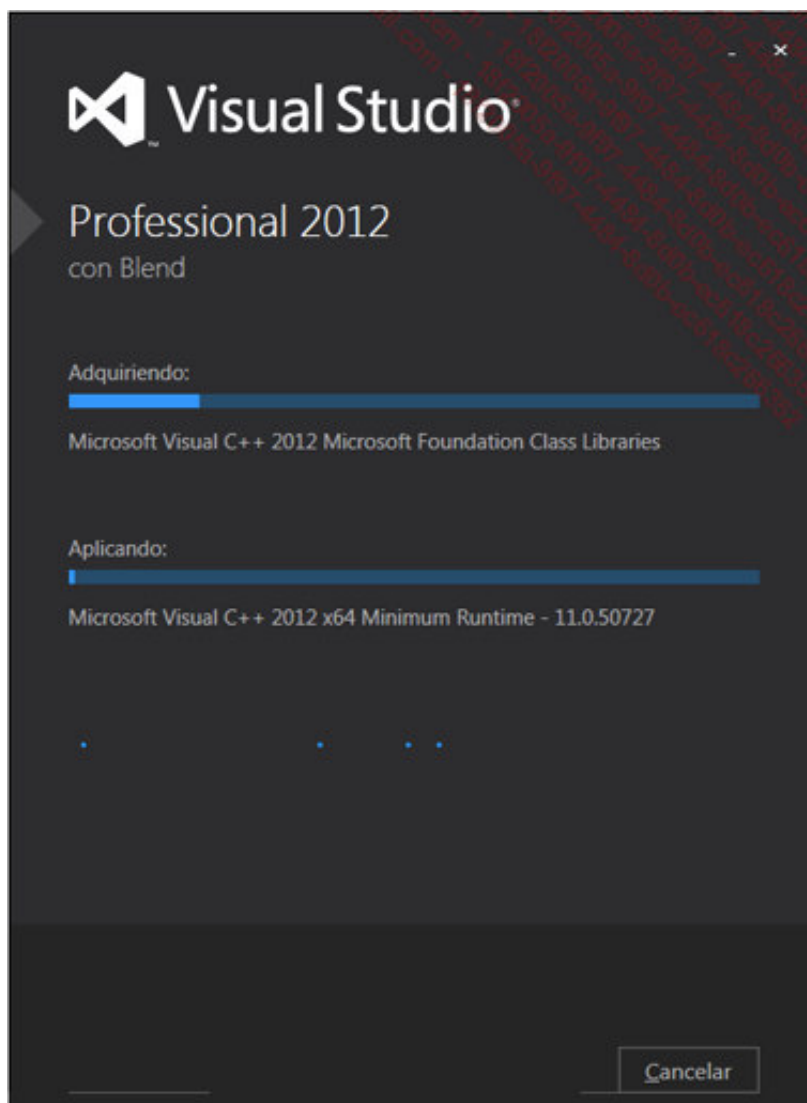
Después de insertar el DVD y tras algunos segundos de carga, se muestra la siguiente pantalla:



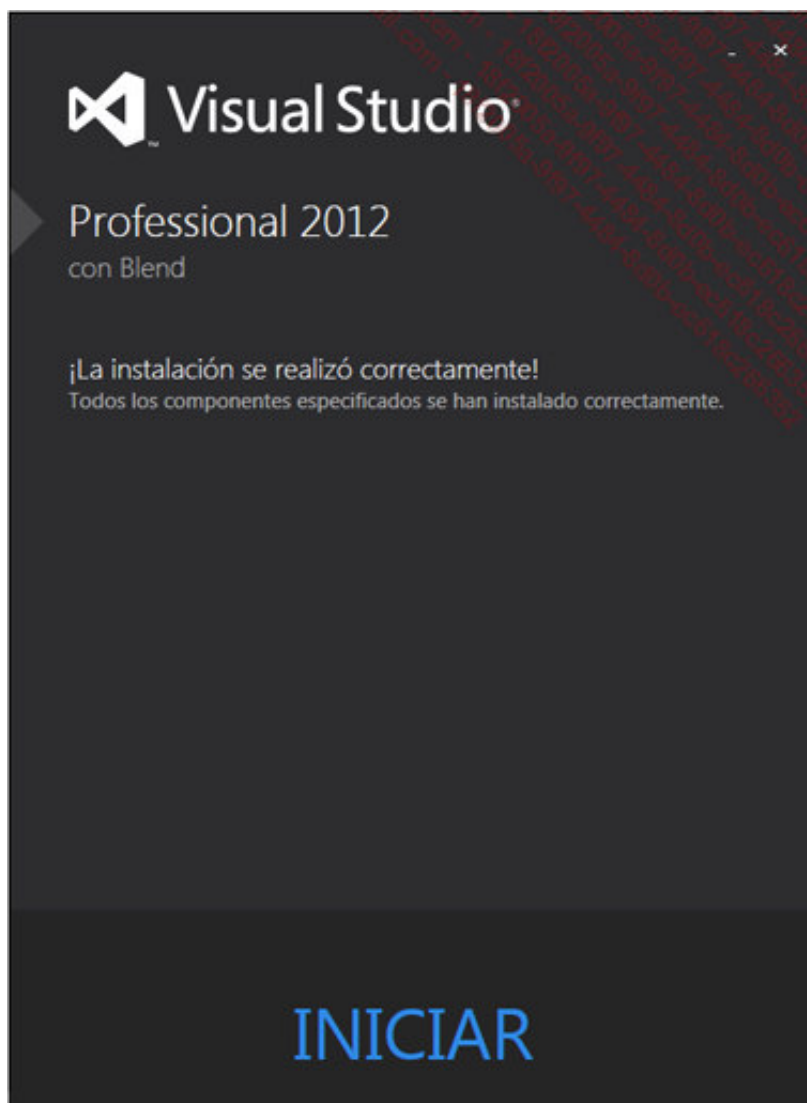
Esta pantalla le permite escoger la carpeta de instalación del producto y le indica el espacio de disco necesario para esta instalación. Para seguir con la instalación, debe aceptar el contrato de licencia. La siguiente etapa le permite escoger las funcionalidades suplementarias que desea instalar e iniciar la instalación del producto.



La siguiente pantalla le informa del progreso de la instalación.

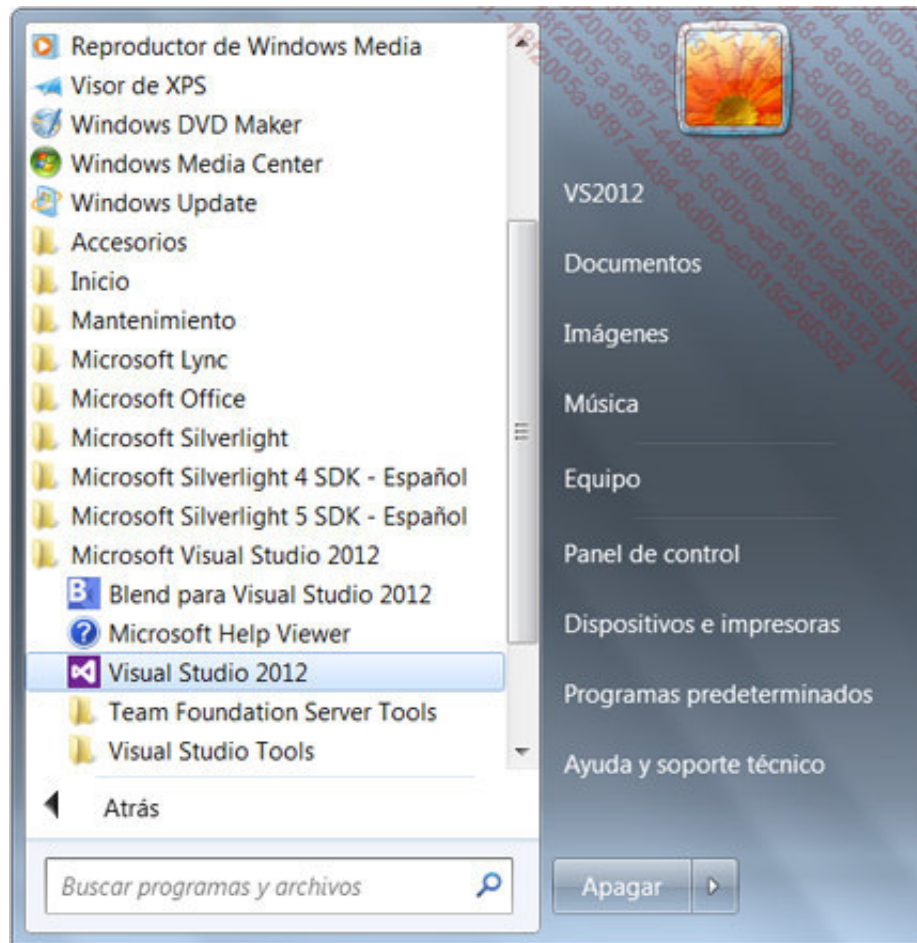


Hay que tener paciencia, pues la instalación puede ser bastante larga en función de las opciones marcadas. A este efecto, la siguiente pantalla le informa del éxito de la instalación y le permite ejecutar directamente el producto.

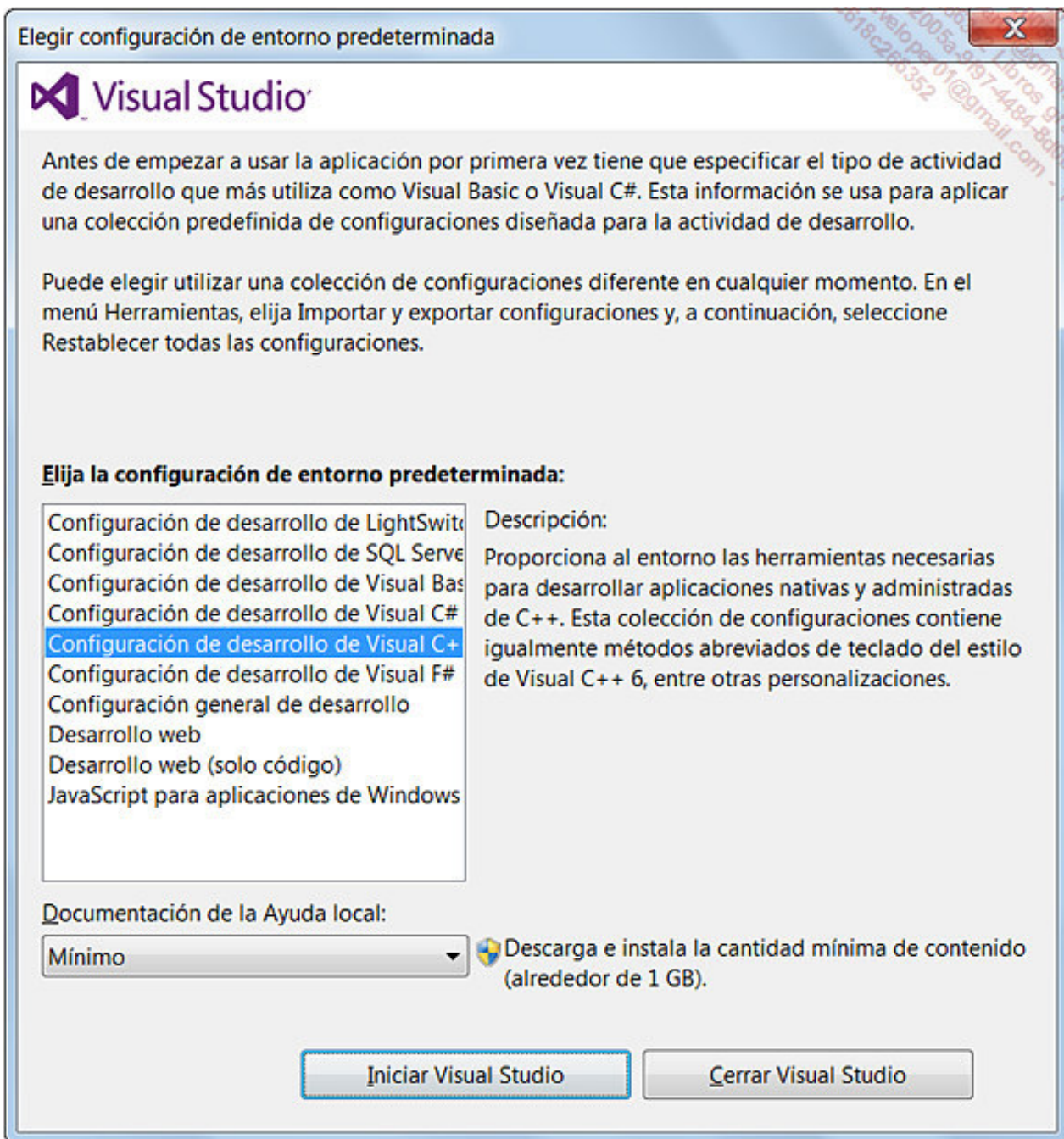


2. Primera ejecución

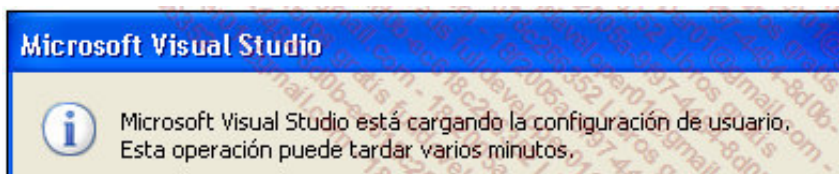
Un acceso directo creado automáticamente por el programa de instalación le permite ejecutar Visual Studio.



La primera vez que lo use, Visual Studio le propondrá personalizar el entorno de trabajo. En función de su preferencia por un lenguaje particular, Visual Studio configura el entorno con las herramientas adaptadas. Se puede modificar más tarde esta configuración con el menú **Herramientas - Importar y exportar configuraciones**.



Visual Studio aplica la configuración elegida antes de arrancar.

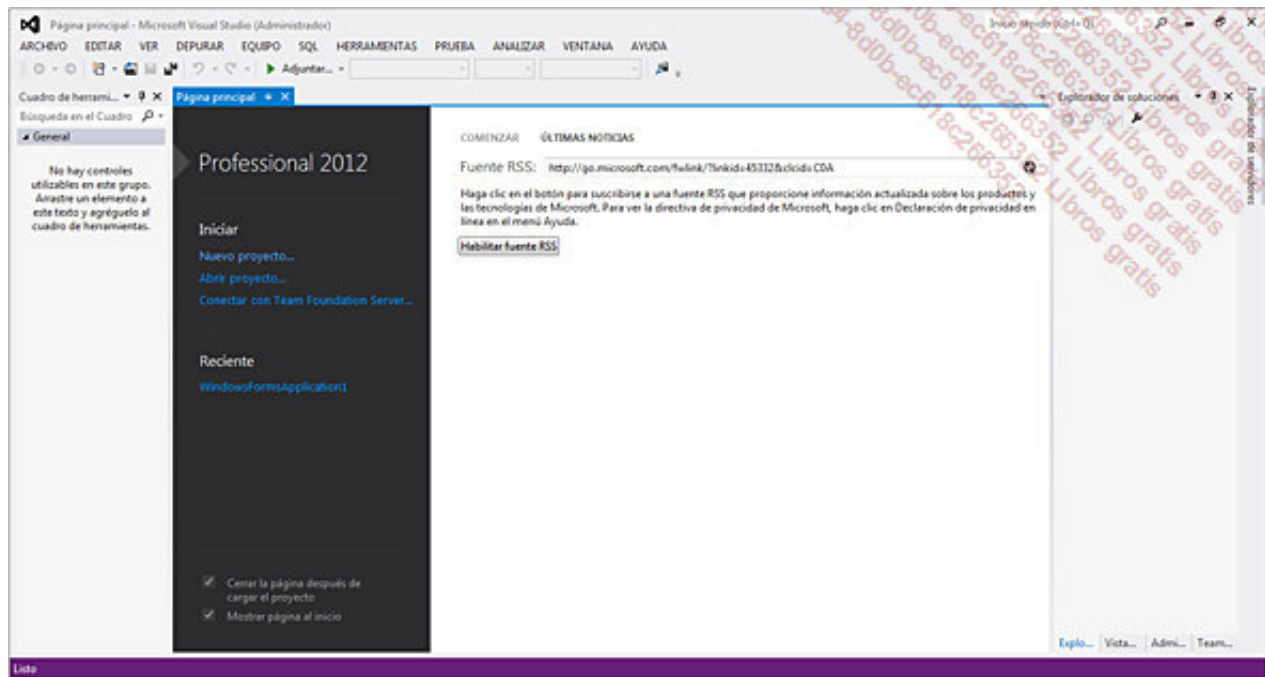


Descubrimiento del entorno

1. Página de inicio

Esta página se visualiza cada vez que invoca a Visual Studio. Le permite acceder rápidamente a los últimos proyectos en los cuales ha trabajado, crear un nuevo proyecto o abrir un proyecto existente.

La pestaña **Últimas noticias** permite activar un flujo RSS que facilita información de las actualizaciones disponibles.



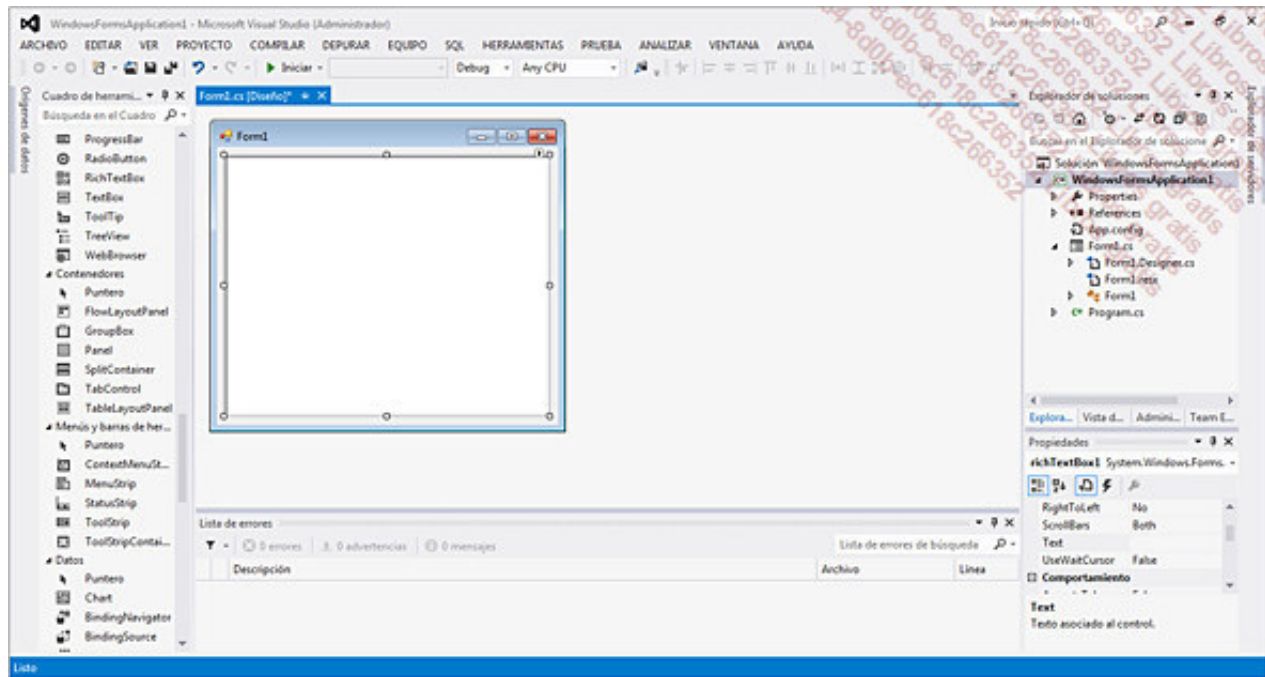
Después de la creación de un nuevo proyecto o la apertura de un proyecto existente, se arranca el entorno Visual Studio.

2. Entorno Visual Studio

El entorno se compone de tres tipos de elementos:

- una zona de barra de menús y de barras de herramientas;
- una zona central de trabajo;
- una multitud de ventanas que constituyen las diferentes herramientas a nuestra disposición.

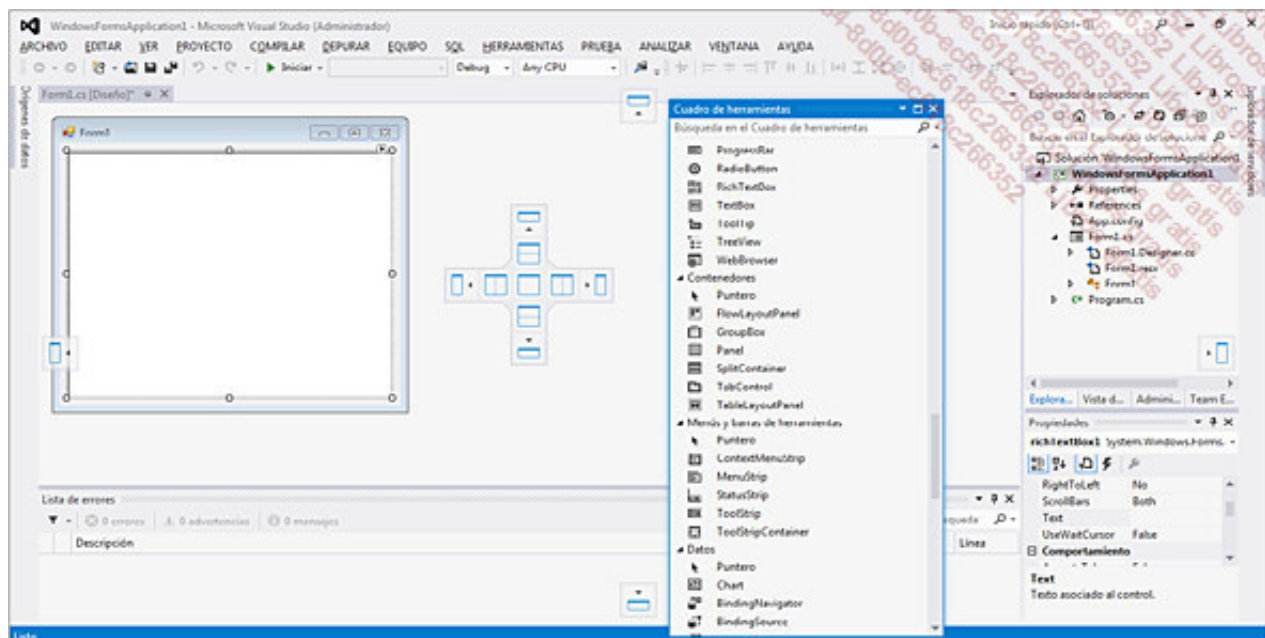
El conjunto presenta, a pesar de todo, un aspecto cargado, y tras añadir una o dos barras de herramientas y la aparición de algunas ventanas adicionales, la zona de trabajo queda más restringida, sobre todo en una pantalla de tamaño reducido.



Afortunadamente hay varias soluciones disponibles para gestionar nuestro espacio de trabajo:

- el anclaje de las ventanas;
- la ocultación automática de las ventanas;
- la utilización de pestañas.

El anclaje de ventanas no permite ganar espacio en la pantalla, pero sí colgar en un borde de la pantalla o de una ventana una ventana determinada. También es posible convertir cada ventana en flotante haciendo doble clic en su barra de título o utilizando el menú contextual. Luego se puede desplazar o anclar esta ventana en otro borde. Para guiarnos en el anclaje de una ventana, Visual Studio muestra, durante el desplazamiento de una ventana, guías que permiten elegir el borde donde efectuar el anclaje.



Los iconos



situados en la periferia de la pantalla facilitan el anclaje en el borde

correspondiente de la pantalla. Los iconos



aparecen en el centro de la ventana que se

está moviendo controlan el anclaje en sus bordes o bajo la forma de una pestaña adicional para la ventana.

Más interesante para ganar espacio en la pantalla, las ventanas ocultables sólo son visibles si el cursor del ratón se encuentra encima. Si no, sólo una zona de pestañas, ubicada en el borde del entorno de desarrollo, permite hacer que aparezca su contenido. Para conservar una ventana siempre visible,

basta con bloquearla utilizando la chincheta presente en su barra de título  .

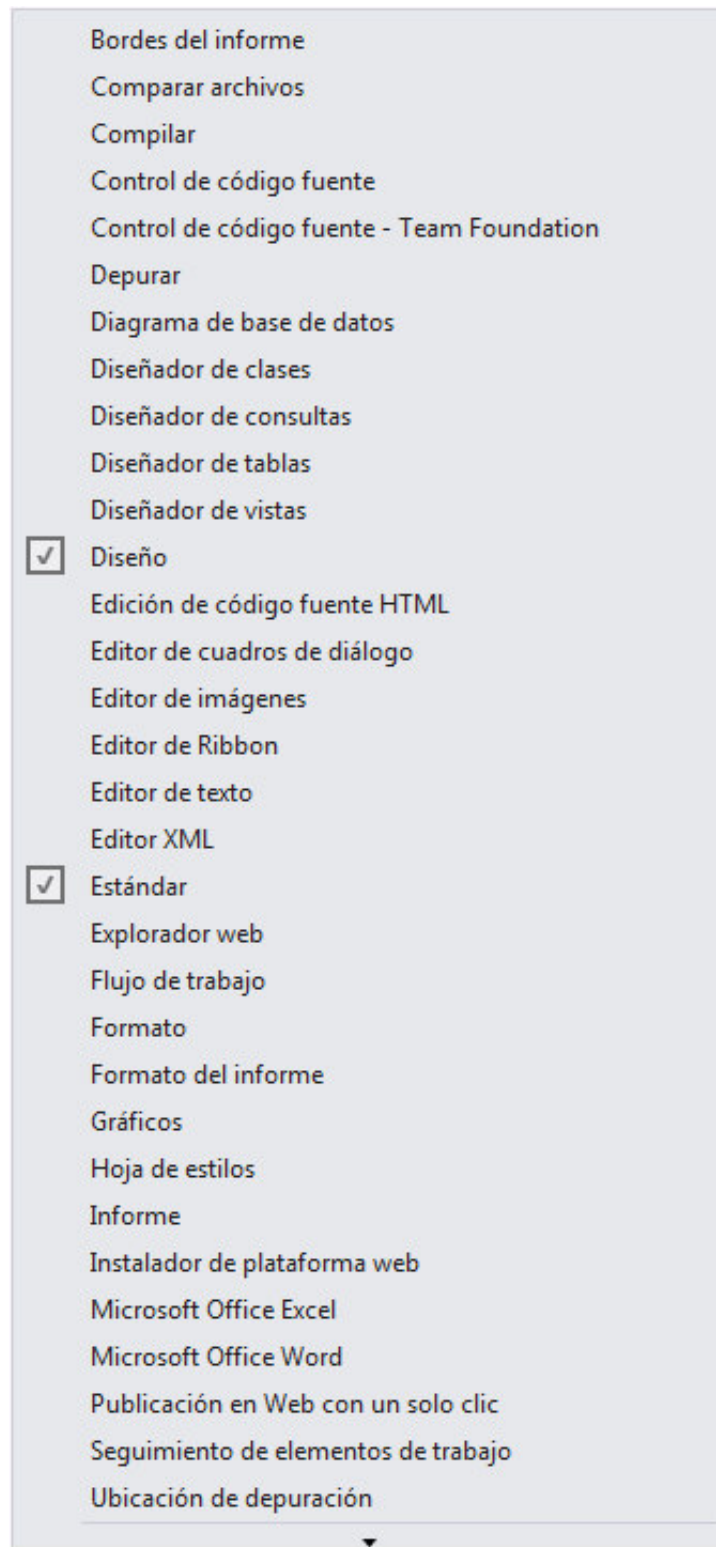
Finalmente, la utilización de pestañas permite compartir una misma zona de pantalla entre diferentes ventanas; a este nivel, los diseñadores de Visual Studio las han utilizado sin moderación.

Las herramientas disponibles

Miremos más en detalle las diferentes barras de herramientas y ventanas que están a nuestra disposición.

1. Las barras de herramientas

No menos de treinta barras de herramientas diferentes están disponibles en Visual Studio. La visualización de cada una de ellas se puede controlar con el menú contextual, accesible haciendo doble clic en la barra principal de menús.

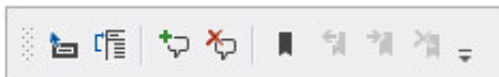


Por supuesto, es inútil visualizar el conjunto de las barras de herramienta de manera simultánea; conviene mostrar sólo las más útiles.

Estándar



Editor de texto



Editor de cuadros de diálogo



Disposición



Depurar

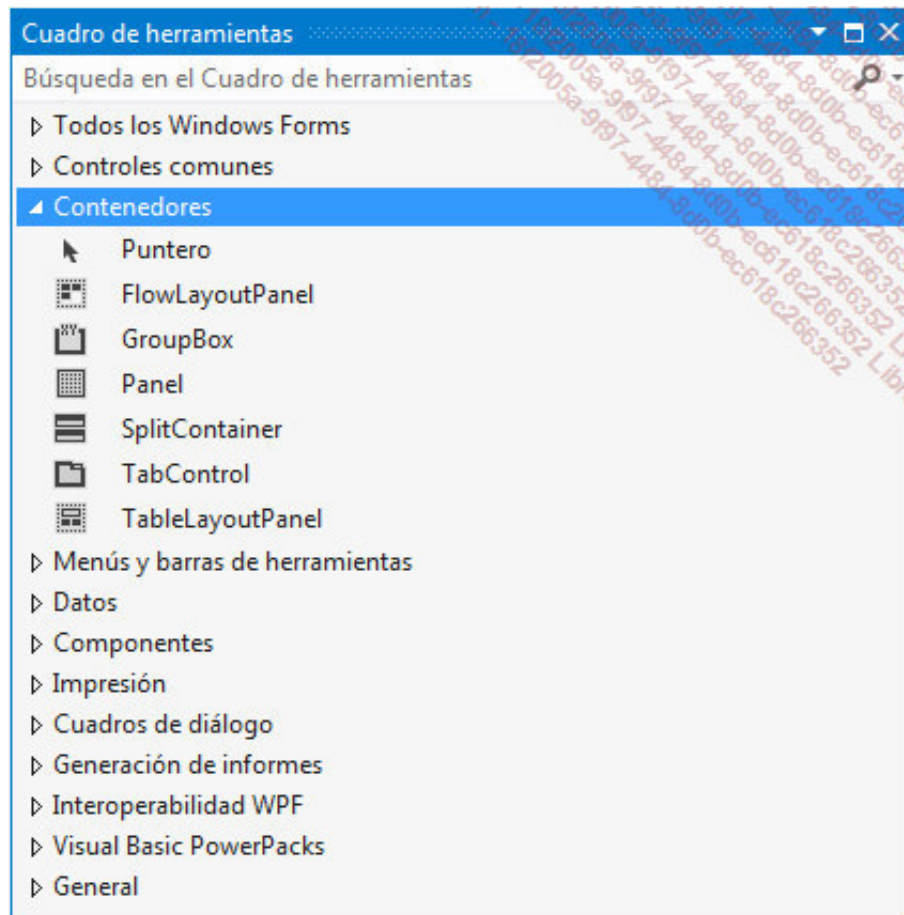


Las otras barras disponibles se visualizarán bajo demanda, en función de sus necesidades, con el fin de evitar sobrecargar su pantalla.

Las ventanas disponibles son también bastante numerosas y vamos a descubrir las más corrientes.

2. El cuadro de herramientas

A partir del cuadro de herramientas vamos a elegir los elementos utilizados para el diseño de la interfaz de la aplicación.



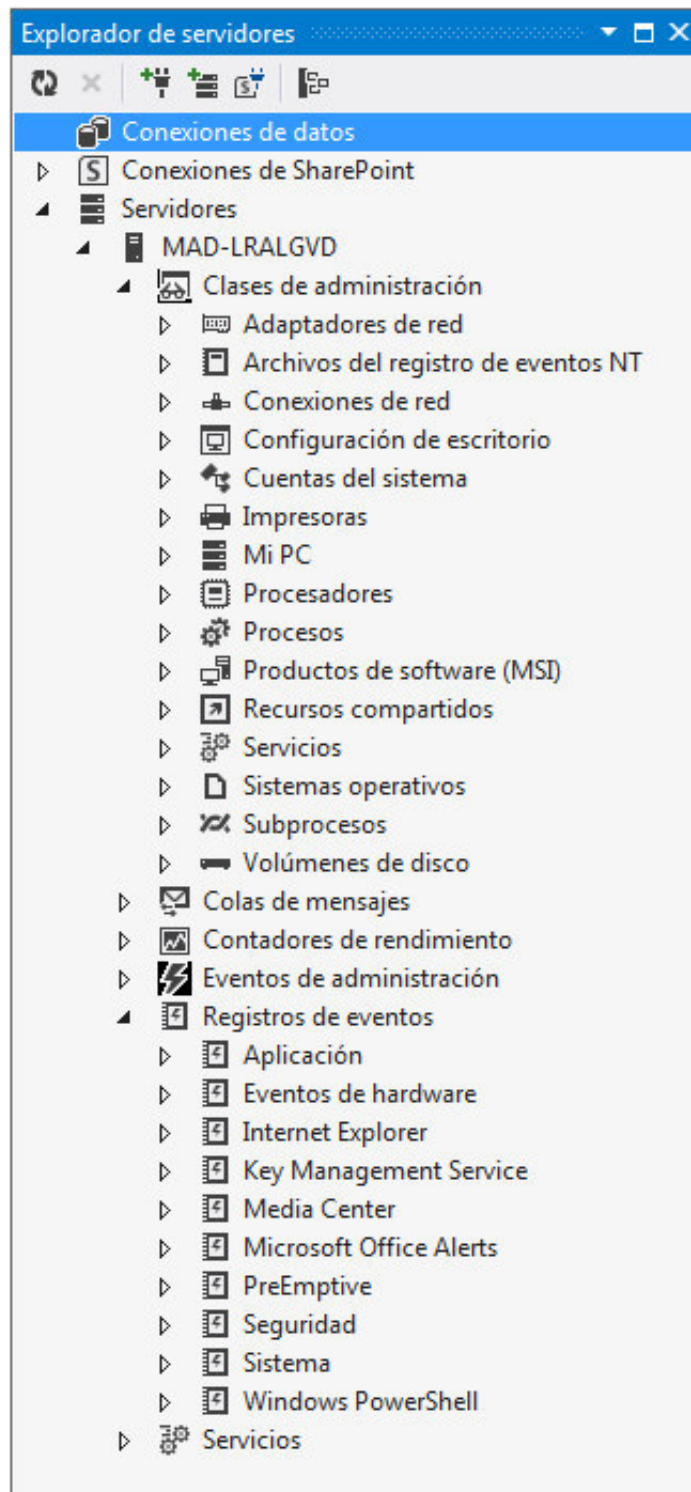
El cuadro de herramientas, organizado por secciones, permite encontrar los controles fácilmente.

Cada uno podrá personalizar su cuadro de herramientas al añadirle por ejemplo controles no disponibles por defecto. Puede ser juicioso, antes de añadir controles a su cuadro de herramientas, crear una nueva sección para albergarla. Para ello, abra el menú contextual del cuadro de herramientas (haciendo clic con el botón derecho del ratón en el cuadro de herramientas), elija la opción **Agregar ficha**, luego dé un nombre a la nueva sección que acaba de crear. Después de haber seleccionado esta nueva sección, puede añadirle controles. Visualice de nuevo el menú contextual del cuadro de herramientas, luego elija la opción **Elegir elementos**.

Se presenta entonces la lista de los controles (COM o .NET), disponibles en la máquina, que le permite seleccionar los controles que hay que añadir en esta sección del cuadro de herramientas. La configuración del cuadro de herramientas no está relacionada con el proyecto activo sino con el propio entorno (el cuadro de herramientas será idéntica sea cual sea el proyecto abierto).

3. El explorador de servidores

El explorador de servidores está disponible con el menú **Ver - Explorador de servidores** o por el atajo [Ctrl][Alt] **S**. Se visualiza en una nueva pestaña de la ventana asociada al cuadro de herramientas.



La mayoría de las aplicaciones requieren otras máquinas presentes en la red para poder funcionar. Por lo tanto es necesario tener, durante la fase de desarrollo de una aplicación, la posibilidad de acceder a los recursos disponibles en otras máquinas.

El elemento de la ventana del explorador de servidores utilizado de manera más frecuente será la sección **Conexiones de datos**.

Permite en particular la gestión de los objetos disponibles en el servidor SQL (tablas, vistas, procedimientos almacenados).

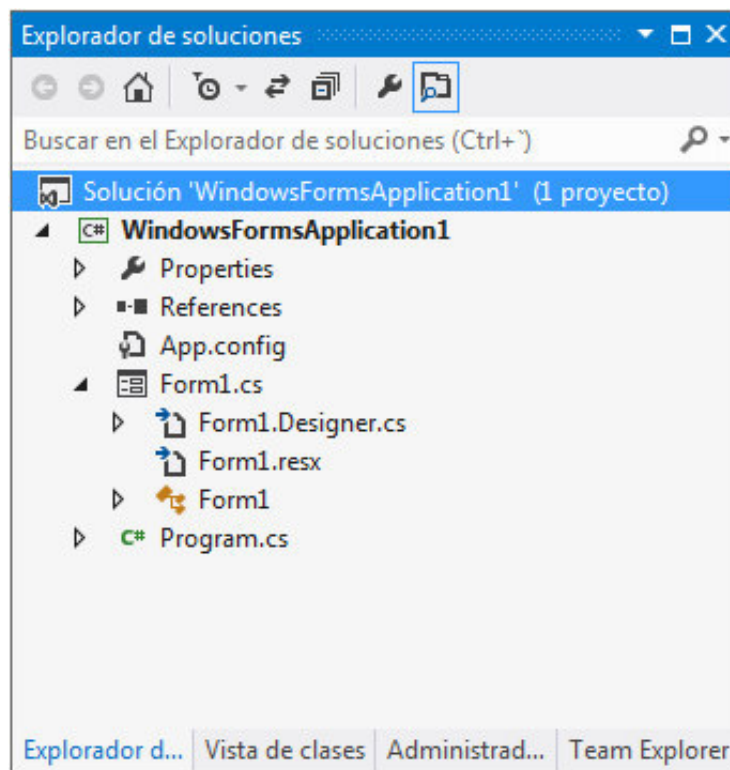
El explorador de servidores también permite gestionar servicios operativos en las máquinas tanto a través de la interfaz gráfica como de código. Ofrece la posibilidad de visualizar la actividad de las

máquinas analizando los contadores de rendimiento o recuperando datos guardados en los diferentes registros de eventos. Un sencillo arrastrar y soltar entre el explorador de servidores y una ventana que se está diseñando genera automáticamente el código que permite trabajar con este elemento en la aplicación. Por ejemplo, el desplazamiento de un contador de rendimiento encima de una ventana genera el código siguiente:

```
private System.Diagnostics.PerformanceCounter performanceCounter1;  
this.performanceCounter1 = new System.Diagnostics.PerformanceCounter();  
this.performanceCounter1.CategoryName = "Memoria"  
this.performanceCounter1.CounterName = "Kilo-bytes disponibles"  
this.performanceCounter1.MachineName = "portátil TG"
```

4. El explorador de soluciones

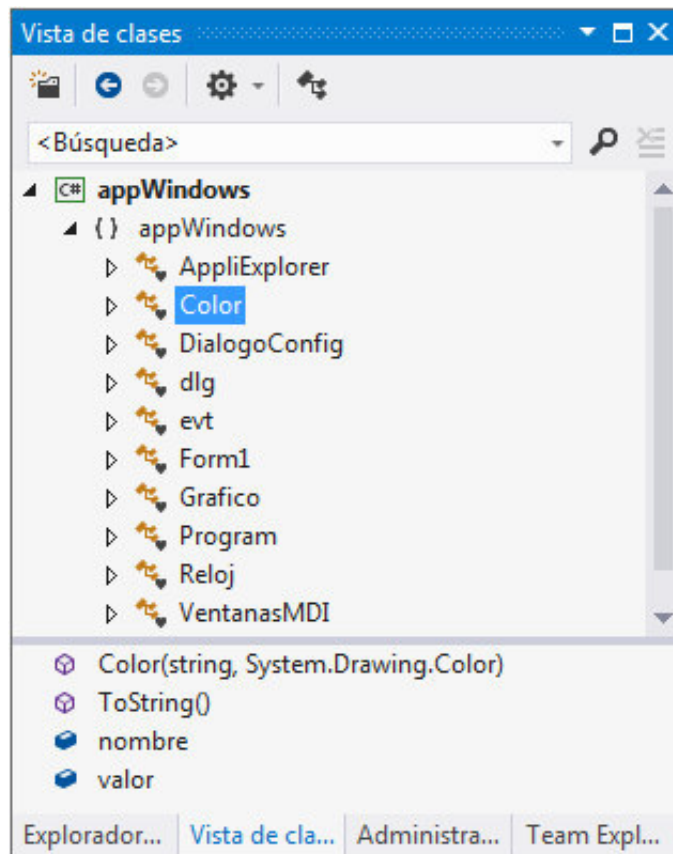
El explorador de soluciones permite ver los elementos que constituyen una solución y modificar sus propiedades.



- La utilización del explorador de soluciones se presenta en detalle en el capítulo dedicado a la organización de una aplicación.

5. El visor de clases

El visor de clases es accesible mediante el menú **Ver - Vista de clases** o con la combinación de teclas [Ctrl][Shift] **C**. Comparte su zona de pantalla con el explorador de soluciones.

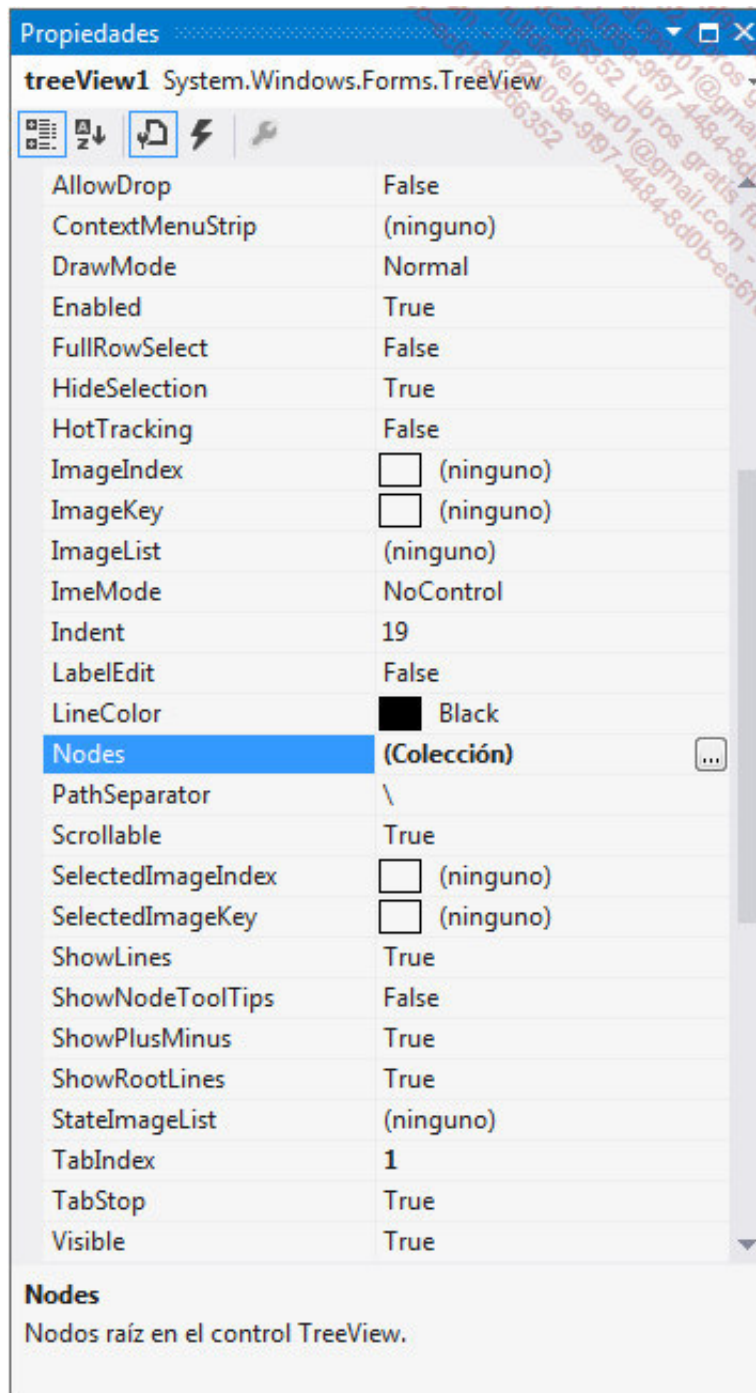


La visualización de clases permite tener una visión lógica de una solución presentando las diferentes clases utilizadas en esa solución.

6. La ventana de propiedades

Se puede visualizar la ventana de propiedades usando cualquiera de estos tres métodos:

- Utilizando el menú **Ver - Ventana propiedades**.
- Con la tecla de función [F4].
- Con la opción **Propiedades** del menú contextual disponible al hacer clic con el botón derecho en uno de los elementos que constituye un proyecto (elemento gráfico de la interfaz de usuario, fichero o archivo del proyecto...). La ventana de propiedades adapta automáticamente su contenido en función del elemento seleccionado y permite modificar estas características.



Los elementos cuyas características puede modificar se pueden seleccionar directamente en la lista desplegable o en la interfaz de la aplicación.

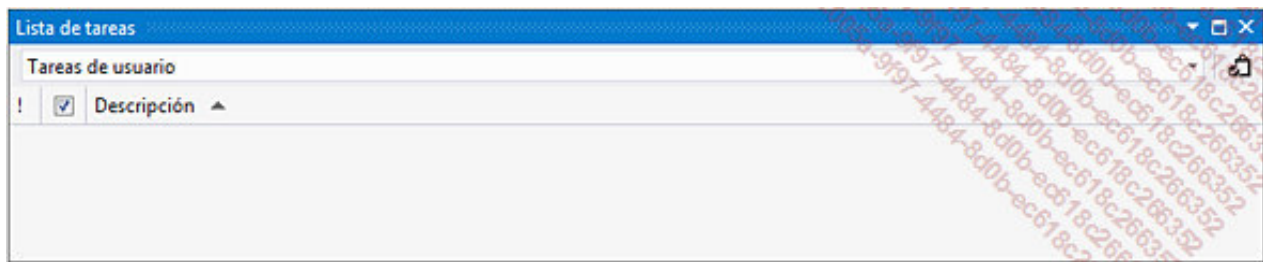
Hay dos presentaciones disponibles para la lista de propiedades:

El modo **Alfabético**, que se activa al hacer clic en el icono .

El modo **Por categoría**, que se activa al hacer clic en el icono .

7. La lista de las tareas

Esta ventana le permitirá sustituir decenas de post-it pegados en el borde de su pantalla. En efecto, Usted puede gestionar lo que queda por hacer en su proyecto teniendo en cuenta una lista de las modificaciones que es preciso aportar en su código.



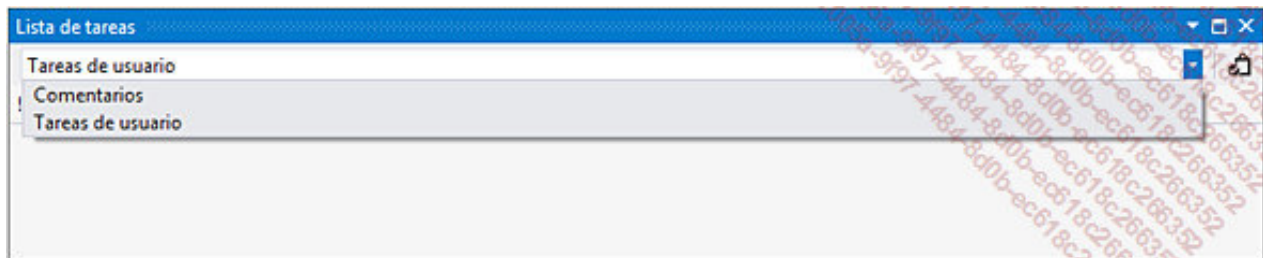
La información presente en la lista puede tener dos orígenes:

- Los comentarios insertados en su código.
- La información introducida directamente en la ventana.

Usted puede ubicar en su código los comentarios que aparecerán luego en la lista de las tareas. Esta técnica le permite, por ejemplo, indicar una modificación que es preciso efectuar más tarde en su código.

Basta con que el comentario empiece con **ToDo**, para luego retomarlo automáticamente en la lista de las tareas.

También puede introducir directamente los datos en la lista de las tareas. Para ello seleccione la opción **Tareas de usuario** que se muestra si despliega la zona de lista disponible en la barra de título de la lista de las tareas.



La adición de una tarea se ejecuta luego con el botón  , disponible en la lista de las tareas.

Es posible especificar ya una descripción y una prioridad para la nueva tarea haciendo clic en la columna de izquierda en la lista de las tareas. Hay tres niveles de prioridad disponibles:

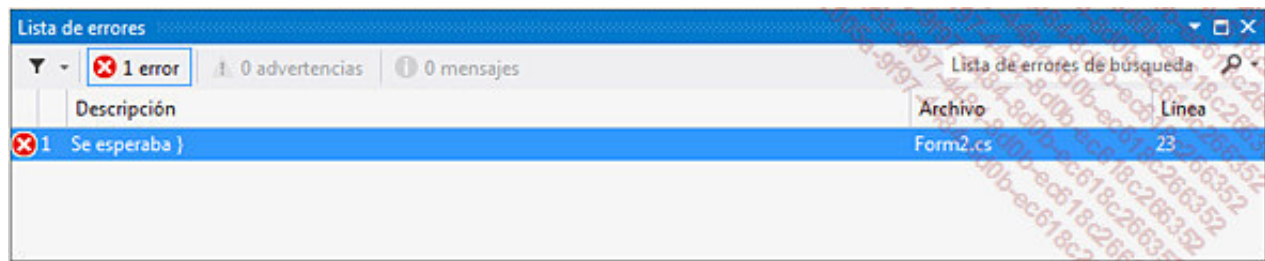
- Alta.
- Normal.
- Baja.

Para cada tarea, una casilla de selección permite indicar que se ha realizado. Su descripción aparece entonces tachada en la lista de las tareas. Para las tareas de usuario, no hay enlace automático con un fragmento cualquiera de código.

8. La lista de los errores

El código que va introduciendo es analizado en tiempo real por Visual Studio y los posibles errores de

sintaxis son retomados por Visual Studio en la ventana **Lista de errores**.



Para ir directamente a la línea donde haya aparecido un error de sintaxis, basta con hacer doble clic en la lista del elemento correspondiente (en el ejemplo anterior, doble clic en **Se esperaba }** para alcanzar la línea 23). No es necesario en absoluto pedir la compilación completa del código para rastrear todos los errores de sintaxis. En cuanto el error está corregido, desaparece automáticamente de la lista de errores.

Los botones de error, alerta, mensaje activan un filtro sobre los mensajes visualizados en la lista de los errores.

9. La ventana de edición de código

Vamos a dedicar más tiempo a esta ventana. Propone muchas funcionalidades que permiten automatizar las acciones más corrientes.

a. Los Snippets

Los Snippets son fragmentos de código que se pueden incorporar muy fácilmente a un archivo fuente. Permiten escribir muy rápidamente porciones de código correspondiente a situaciones corrientes. Visual Studio propone una multitud de Snippets. Hay dos soluciones disponibles para insertar un Snippet:

- Utilizar la opción **Insertar fragmento de código** del menú contextual del editor de código.
- Utilizar las combinaciones de teclas [Ctrl] **K**, luego [Ctrl] **X**.

Para estos dos métodos, Visual Studio le propone elegir en una lista el Snippet que le interesa. Se pueden personalizar estas porciones de código. En principio están en azul claro. La modificación de una de estas porciones de código repercute en todas las instancias en el Snippet.

En el ejemplo siguiente, se empleó un Snippet para añadir un bucle `for` en una función.

```
public static void calculo()  
{  
    for (int i = 0; i < length; i++)  
    {  
    }  
}
```

Se efectuará la modificación de los valores `i` y `length` en cascada en el conjunto del código del Snippet.

Puede también diseñar sus propios Snippets. Para ello, debe crear el archivo XML que va a contener el código del Snippet. Este archivo debe tener la extensión `.snippet`.

Para ayudarle en la creación de un Snippet, Microsoft tiene previsto un Snippet. Usted puede incorporarlo en su archivo XML con el menú contextual **Insertar fragmento de código**.

Debe obtener el documento siguiente:

```
<?xml version="1.0" encoding="utf-8" ?>
<CodeSnippet Format="1.0.0" xmlns="http://schemas.microsoft.com/
VisualStudio/2005/CodeSnippet">
  <Header>
    <Title>título</Title>
    <Author>autor</Author>
    <Shortcut>atajo</Shortcut>
    <Description>descripción</Description>
    <SnippetTypes>
      <SnippetType>SurroundsWith</SnippetType>
      <SnippetType>Expansion</SnippetType>
    </SnippetTypes>
  </Header>
  <Snippet>
    <Declarations>
      <Literal>
        <ID>nombre</ID>
        <Default>valor</Default>
      </Literal>
    </Declarations>
    <Code Language="XML">
      <![CDATA[<test>
        <name>$nombre$</name>
        $selected$ $end$</test>]]>
    </Code>
  </Snippet>
</CodeSnippet>
```

Luego puede personalizar su Snippet. En un primer momento, debe modificar la sección Header sustituyendo los valores de las diferentes etiquetas.

```
<Header>
  <Title>Recorrer un array</Title>
  <Author>Thierry</Author>
  <Shortcut>tablo</Shortcut>
  <Description>este fragmento añade un bucle que permite recorrer
un array</Description>
  <SnippetTypes>
    <SnippetType>Expansion</SnippetType>
  </SnippetTypes>
</Header>
```

La sección Declaraciones permite crear parámetros utilizados en el Snippet. Para cada parámetro, debe crear una sección <Literal> y facilitar un nombre para el parámetro y un valor por defecto.

```
<Declarations>
  <Literal>
    nombreTabla</ID>
    <Default>laTabla</Default>
  </Literal>
  <Literal>
    <ID>tipoTabla</ID>
    <Default>tipoDeLaTabla</Default>
  </Literal>
</Declarations>
```

```
</Literal>
<Literal>
  <ID>tamañoTabla</ID>
  <Default>tamañoDeLaTabla</Default>
</Literal>
</Declarations>
```

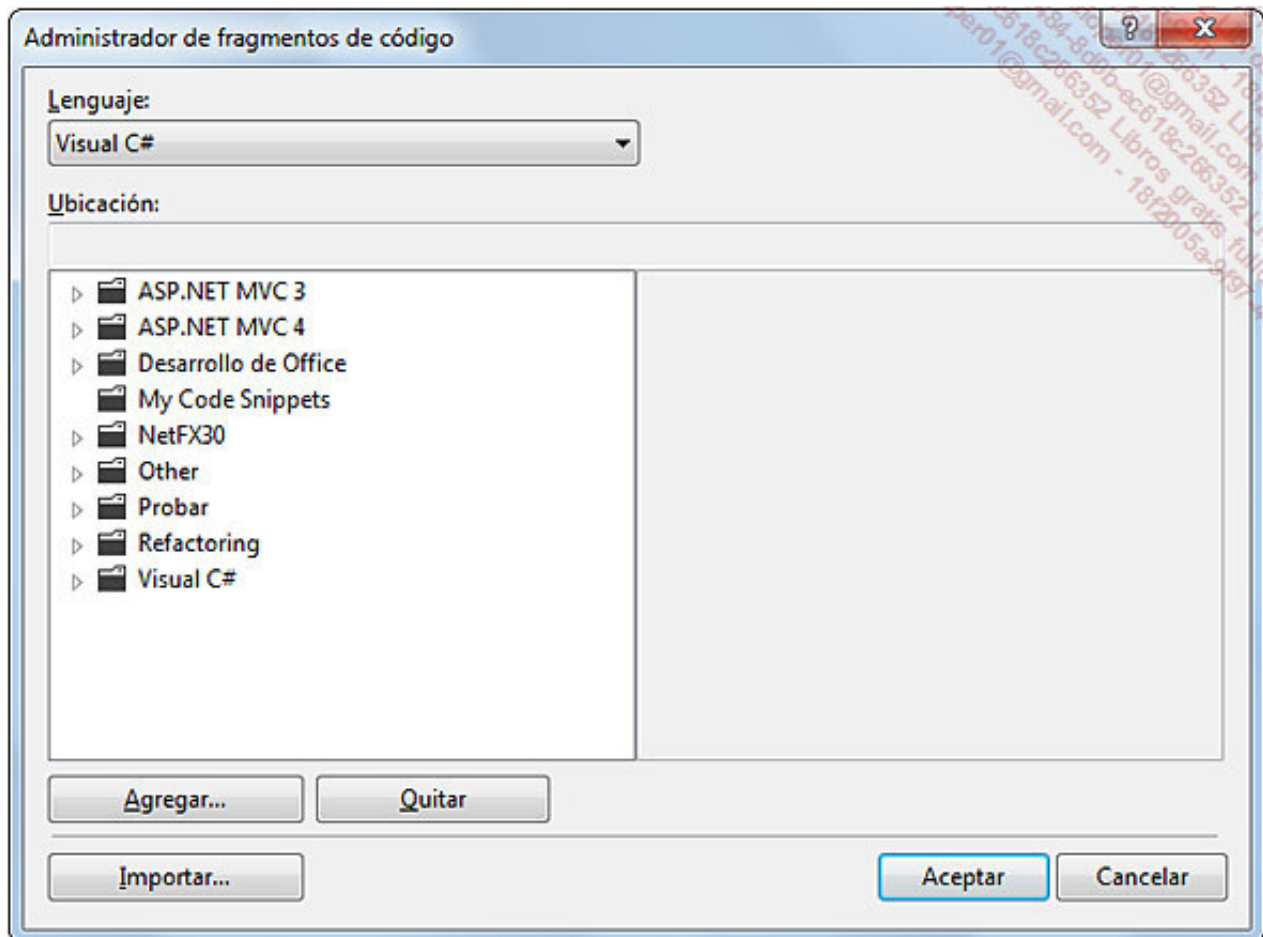
Luego debe indicar para qué lenguaje está previsto su Snippet.

```
<Code Language="CSharp">
```

Y finalmente definir en la etiqueta CDATA el código Snippet. En este código, puede utilizar los parámetros del Snippet enmarcándolos entre dos caracteres \$.

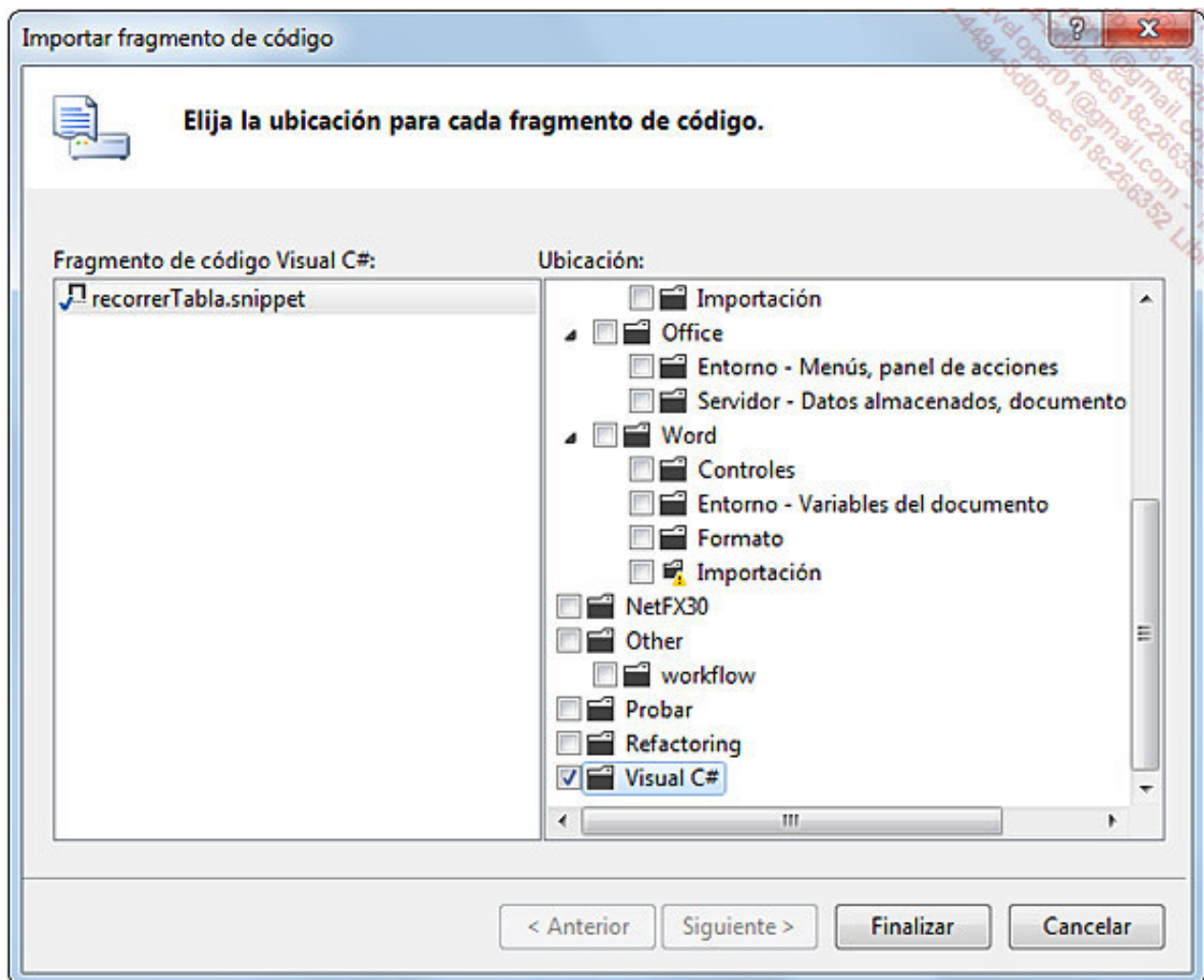
```
<![CDATA[
    $tipoTabla$[] $nombreTabla$;
    $nombreTabla$ = new $tipoTabla$ [$tamañoTabla$];
    int index;
    for (index = 0; index < $nombreTabla$.Length; index++)
    {
        // insertar el código de tratamiento de la tabla
    }
]]>
```

Luego puede guardar el archivo y su Snippet está listo. Conviene ahora integrarlo en Visual Studio. Para ello, active el gestor de Snippet usando el menú **Herramientas - Administrador de fragmentos de código**.

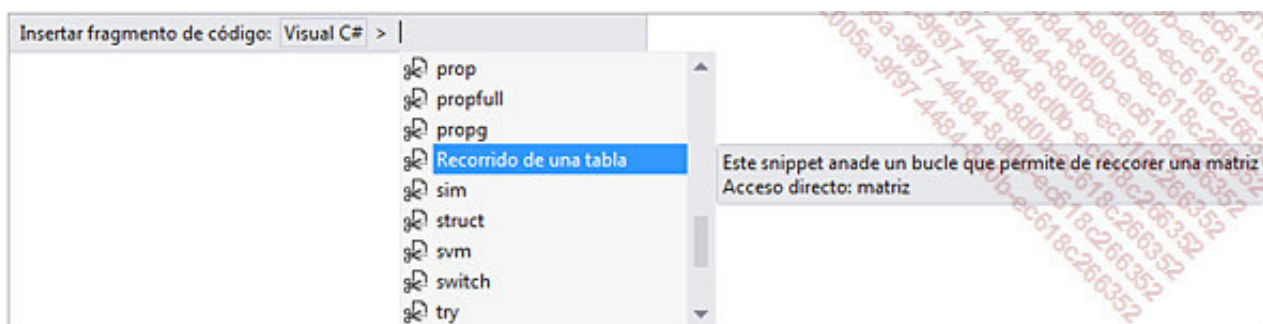


El botón **Importar** permite añadir su Snippet a los ya disponibles en Visual Studio.

Después de haber seleccionado el archivo que contiene el Snippet, debe elegir la sección en la cual se guardará.



Su Snippet está ahora disponible en el editor de código.



Sólo le queda personalizar el código generado.


```

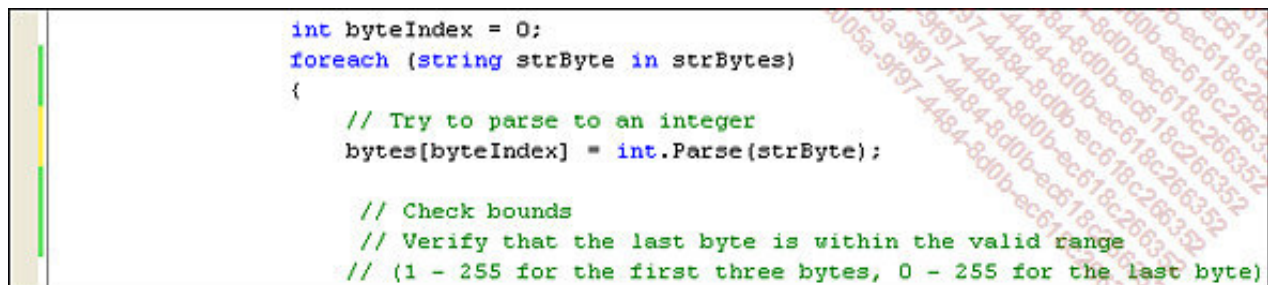
        tipoDeLaTabla[] laTabla;
        laTabla = new tipoDeLaTabla [tamanoDeLaTabla];
        int index;
        for (index = 0; index < laTabla.Length; index++)
        {
            // insertar el codigo de tratamiento de la tabla
        }
    }

```

b. Seguimiento de las modificaciones

Es posible visualizar las porciones de código que ya han sido modificadas desde la ejecución de Visual Studio. Se identifican las modificaciones con un borde de color que aparece en el margen del editor de código.

- Un borde amarillo indica que se ha modificado el código pero que aún no ha sido guardado.
- Un borde verde indica que se ha modificado y guardado el código.



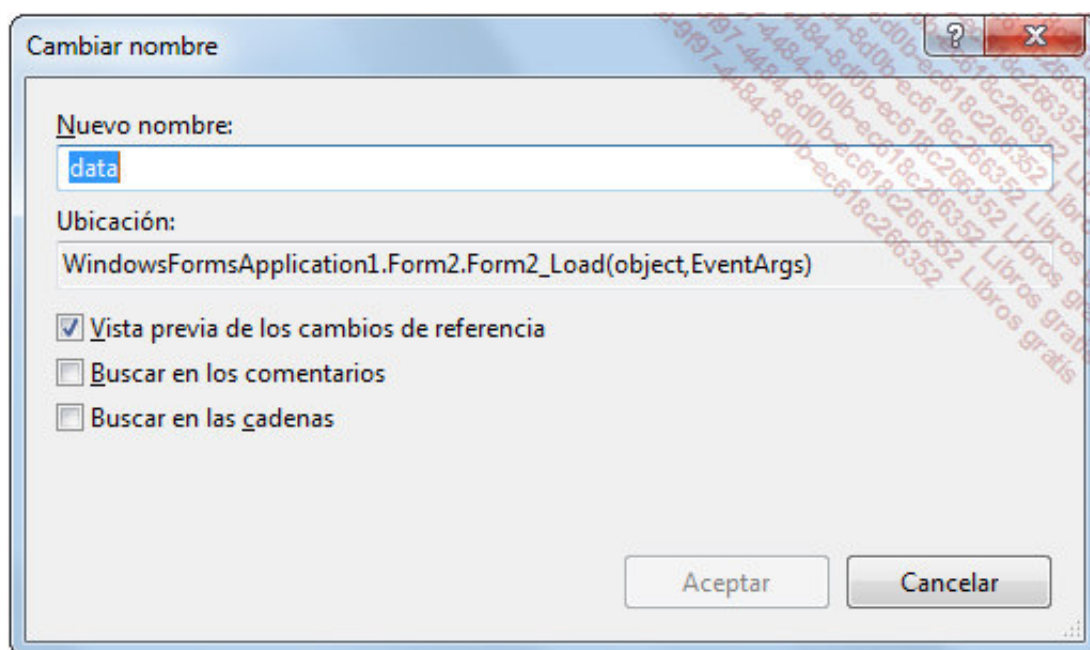
```

        int byteIndex = 0;
        foreach (string strByte in strBytes)
        {
            // Try to parse to an integer
            bytes[byteIndex] = int.Parse(strByte);

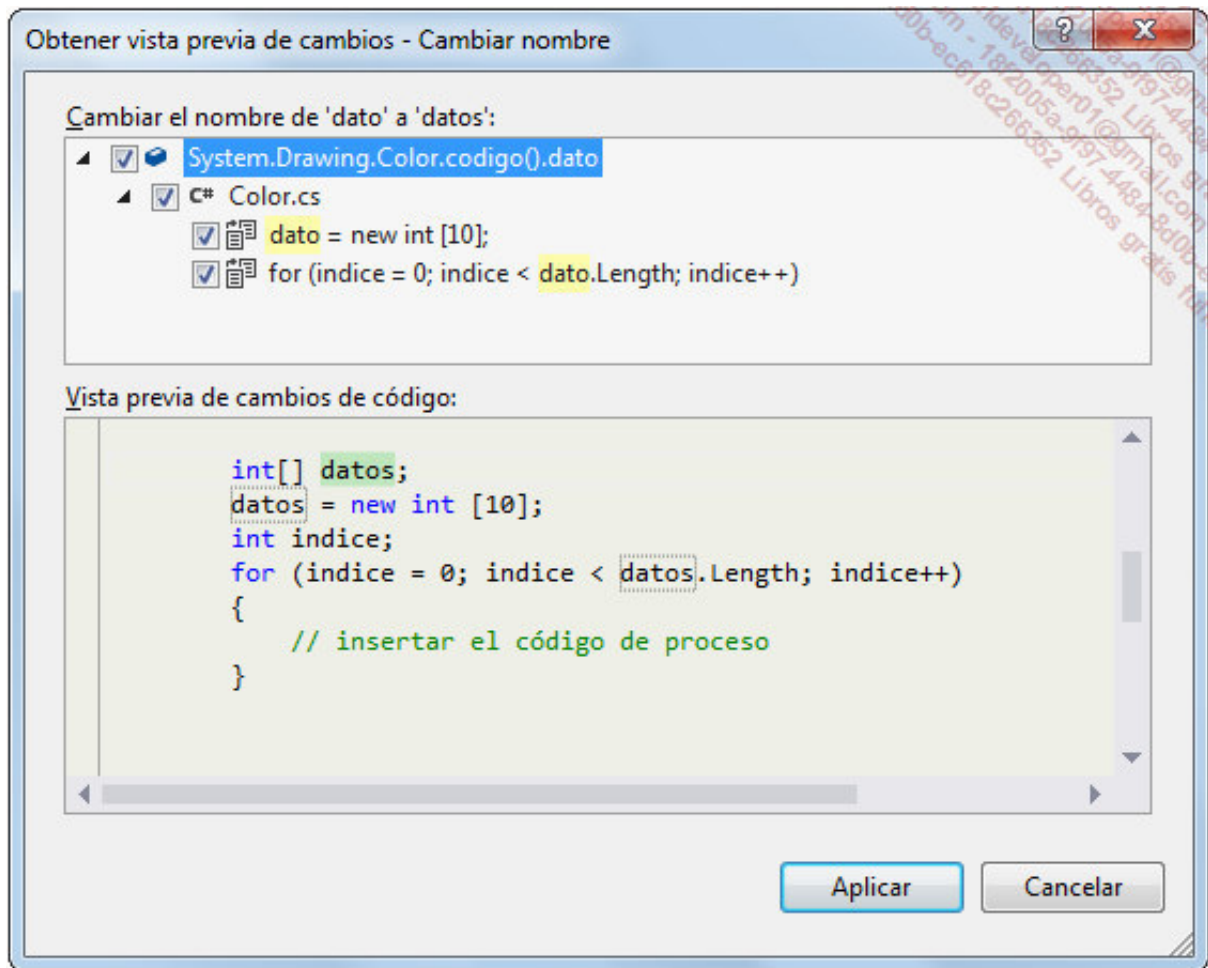
            // Check bounds
            // Verify that the last byte is within the valid range
            // (1 - 255 for the first three bytes, 0 - 255 for the last byte)
        }
    }

```

También puede renombrar un elemento y propagar automáticamente la modificación al resto del código. El uso típico consiste en cambiar el nombre de una variable o clase. Usted no debe renombrar la variable directamente en el código, sino utilizar el cuadro de diálogo visualizado utilizando la opción **Cambiar nombre** del menú contextual del editor de código sobre el nombre actual de la variable.



Se puede extender la búsqueda para efectuar las sustituciones en los comentarios y en las cadenas de caracteres activando las opciones correspondientes. Por defecto se muestra una vista previa de todas las modificaciones previstas antes de que se efectúen realmente.



Se puede cancelar algunas de ellas desmarcando la casilla correspondiente en la lista.

- La modificación realizada mediante este cuadro de diálogo repercute sobre el conjunto del código donde se utiliza la variable.

c. Las herramientas de edición de código

Los editores de texto de Visual Studio disponen de muchas funcionalidades que permiten facilitar las operaciones efectuadas con frecuencia durante la escritura del código de una aplicación.

Selección de texto

Como complemento de las funciones clásicas de selección de texto y de copiar/pegar, el editor de Visual Studio permite la selección de zonas rectangulares de texto manteniendo apretada la tecla [Alt] durante la selección. Cuando se introduce luego algo de código en la selección, se duplica sobre cada línea de la selección.

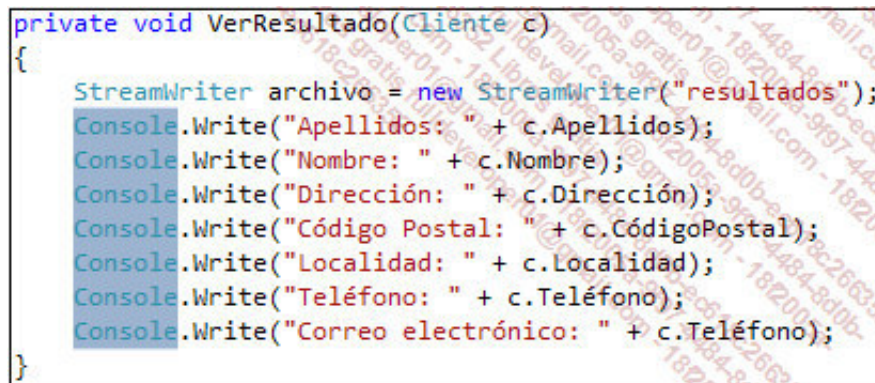
Si, por ejemplo, utiliza el método siguiente, que visualiza en la consola los datos de una persona:

```
private void visualizaciónResultados(Cliente c)
{
    Console.WriteLine("apellido:" + c.apellido);
    Console.WriteLine("nombre:" + c.nombre);
    Console.WriteLine("calle:" + c.calle);
    Console.WriteLine("código postal:" + c.códigoPostal);
    Console.WriteLine("ciudad:" + c.ciudad);
    Console.WriteLine("tél:" + c.tél);
    Console.WriteLine("email:" + c.email);
}
```

Para modificar este método y escribir estos datos en un archivo en vez de visualizarlos en la consola, sólo debe crear el archivo y luego modificar todas las instrucciones **.Write** para que se apliquen al archivo creado. Para ello, añada simplemente la línea siguiente para la creación del archivo:

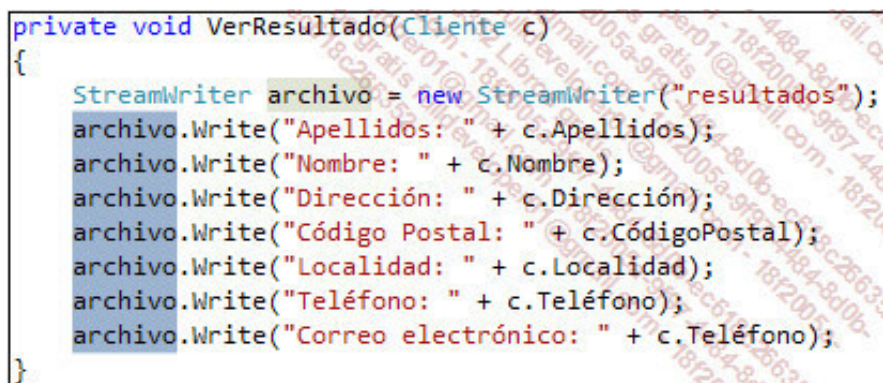
```
StreamWriter archivo=new StreamWriter("resultados");
```

Luego debe modificar cada instrucción **Write** para escribir hacia el archivo, y no hacia la consola. Seleccione para ello una zona rectangular que contenga todas las palabras **consola** e introduzca la palabra **archivo**.



```
private void VerResultado(Cliente c)
{
    StreamWriter archivo = new StreamWriter("resultados");
    Console.WriteLine("Apellidos: " + c.Apellidos);
    Console.WriteLine("Nombre: " + c.Nombre);
    Console.WriteLine("Dirección: " + c.Dirección);
    Console.WriteLine("Código Postal: " + c.CódigoPostal);
    Console.WriteLine("Localidad: " + c.Localidad);
    Console.WriteLine("Teléfono: " + c.Teléfono);
    Console.WriteLine("Correo electrónico: " + c.Teléfono);
}
```

Se sustituye entonces la palabra **Consola** en todas las líneas de la selección.



```
private void VerResultado(Cliente c)
{
    StreamWriter archivo = new StreamWriter("resultados");
    archivo.WriteLine("Apellidos: " + c.Apellidos);
    archivo.WriteLine("Nombre: " + c.Nombre);
    archivo.WriteLine("Dirección: " + c.Dirección);
    archivo.WriteLine("Código Postal: " + c.CódigoPostal);
    archivo.WriteLine("Localidad: " + c.Localidad);
    archivo.WriteLine("Teléfono: " + c.Teléfono);
    archivo.WriteLine("Correo electrónico: " + c.Teléfono);
}
```

También es posible insertar texto simultáneamente en varias líneas creando una zona de selección rectangular de cero caracteres de ancho en todas las líneas donde se debe efectuar la inserción.

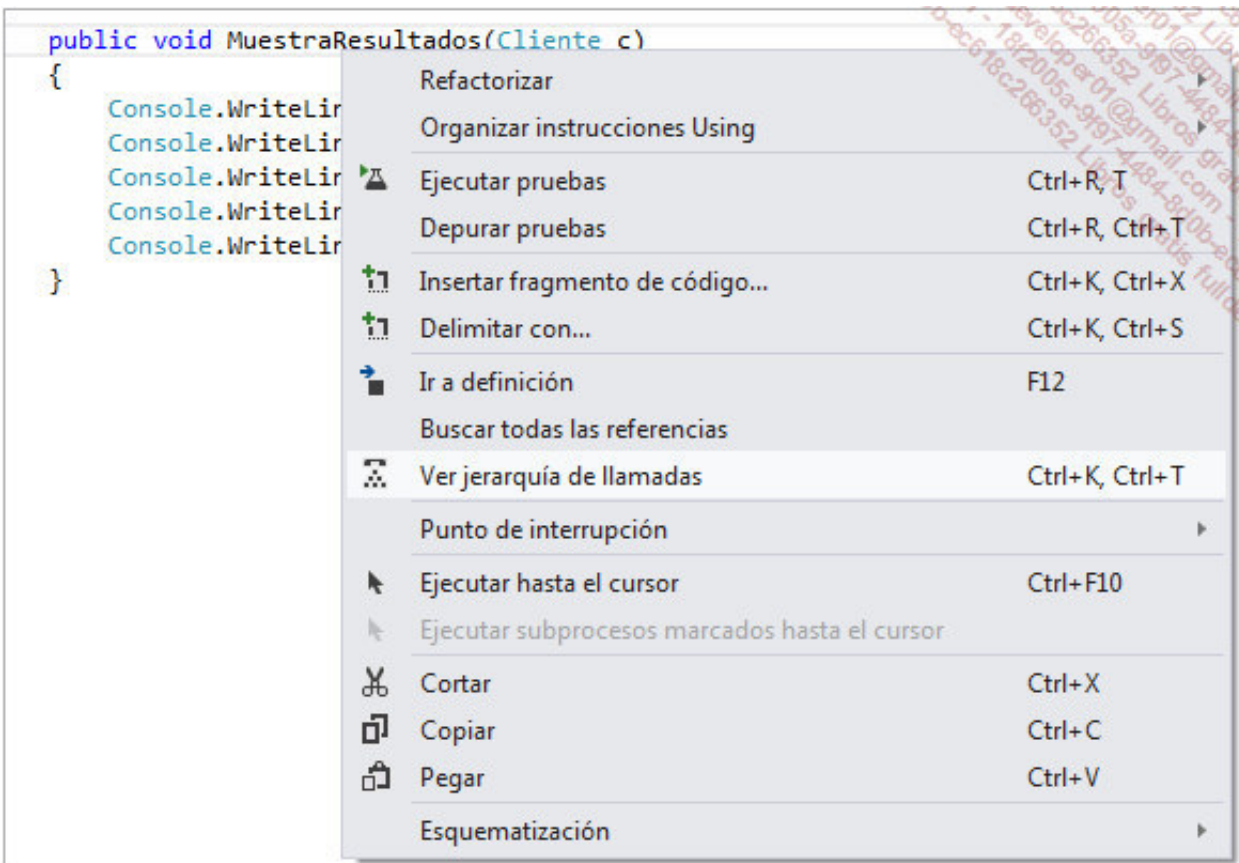

```
private void VerResultado(Cliente c)
{
    StreamWriter archivo = new StreamWriter("resultados");
    archivo.Write("Apellidos: " + c.Apellidos);
    archivo.Write("Nombre: " + c.Nombre);
    archivo.Write("Dirección: " + c.Dirección);
    archivo.Write("Código Postal: " + c.CódigoPostal);
    archivo.Write("Localidad: " + c.Localidad);
    archivo.Write("Teléfono: " + c.Teléfono);
    archivo.Write("Correo electrónico: " + c.Teléfono);
}
```

Luego se inserta el texto introducido en todas las líneas de la selección.

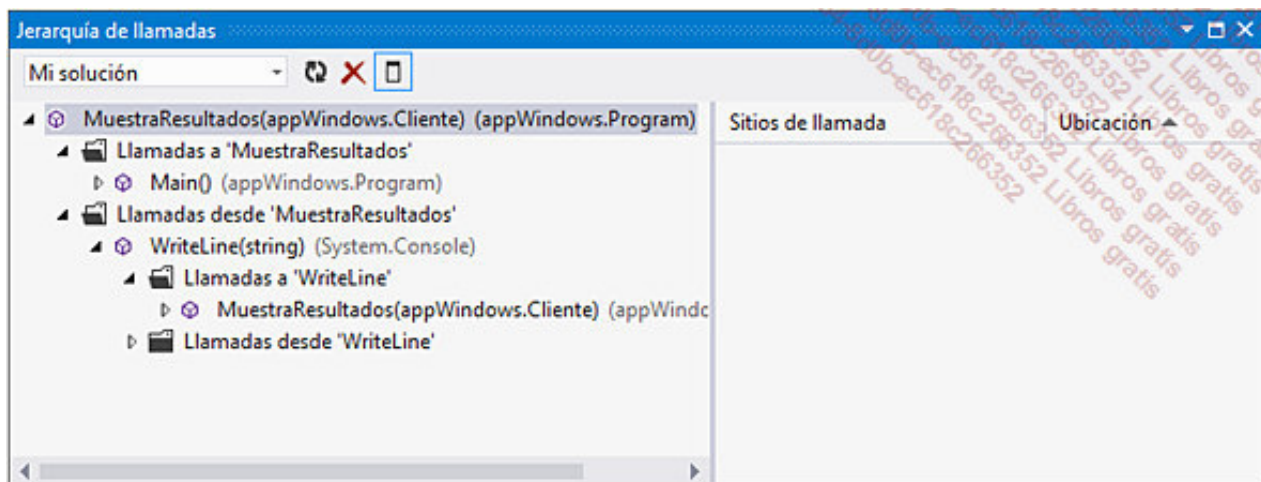
```
private void VerResultado(Cliente c)
{
    StreamWriter archivo = new StreamWriter("resultados");
    archivo.WriteLine("Apellidos: " + c.Apellidos);
    archivo.WriteLine("Nombre: " + c.Nombre);
    archivo.WriteLine("Dirección: " + c.Dirección);
    archivo.WriteLine("Código Postal: " + c.CódigoPostal);
    archivo.WriteLine("Localidad: " + c.Localidad);
    archivo.WriteLine("Teléfono: " + c.Teléfono);
    archivo.WriteLine("Correo electrónico: " + c.Teléfono);
}
```

Jerarquía de llamadas

La jerarquía de llamadas permite visualizar todas las llamadas hacia un método, una propiedad o un constructor, así como las efectuadas desde este método, propiedad o constructor. Se activa con la opción **Ver jerarquía de llamadas** del menú contextual disponible en el elemento concerniente.



Se visualiza entonces la ventana siguiente.



Resaltado de las referencias

Cuando hace clic en un símbolo en el código fuente, el editor resalta todas las instancias de este símbolo.

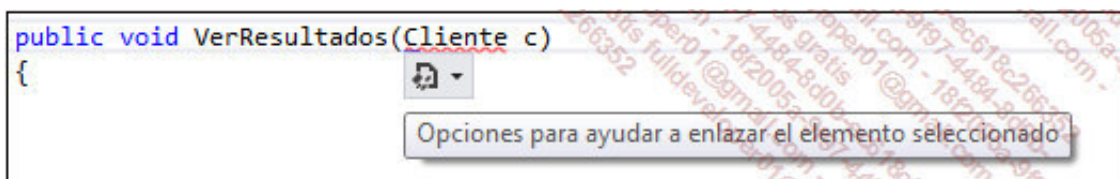
```
private void VerResultado(Cliente c)
{
    c.Nombre = c.Nombre.ToUpper();
    StreamWriter archivo = new StreamWriter("resultados");
    archivo.WriteLine("Apellidos: " + c.Apellidos);
    archivo.WriteLine("Nombre: " + c.Nombre);
    archivo.WriteLine("Dirección: " + c.Dirección);
    archivo.WriteLine("Código Postal: " + c.CódigoPostal);
    archivo.WriteLine("Localidad: " + c.Localidad);
    archivo.WriteLine("Teléfono: " + c.Teléfono);
    archivo.WriteLine("Correo electrónico: " + c.CorreoElectrónico);
}
```

Funcionalidad Generar a partir de la utilización

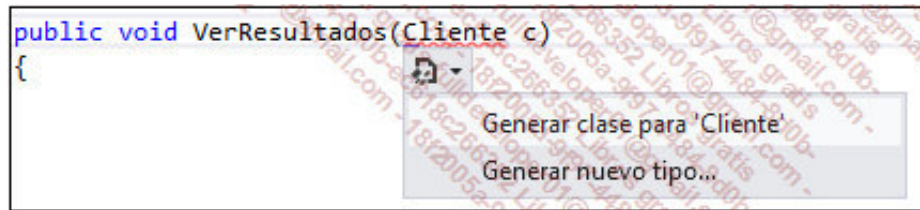
Durante el desarrollo de una aplicación, ocurre a veces que se intenta utilizar un elemento antes de su declaración posponiendo ésta para más tarde. Sin embargo, esta solución tiene el inconveniente de no permitir realizar pruebas hasta que todos los elementos utilizados hayan sido definidos. También es frustrante para el desarrollador ver decenas de líneas de código subrayadas en rojo.

```
private void VerResultado(Cliente c)
{
    c.Nombre = c.Nombre.ToUpper();
    StreamWriter archivo = new StreamWriter("resultados");
    archivo.WriteLine("Apellidos: " + c.Apellidos);
    archivo.WriteLine("Nombre: " + c.Nombre);
    archivo.WriteLine("Dirección: " + c.Dirección);
    archivo.WriteLine("Código Postal: " + c.CódigoPostal);
    archivo.WriteLine("Localidad: " + c.Localidad);
    archivo.WriteLine("Teléfono: " + c.Teléfono);
    archivo.WriteLine("Correo electrónico: " + c.CorreoElectrónico);
}
```

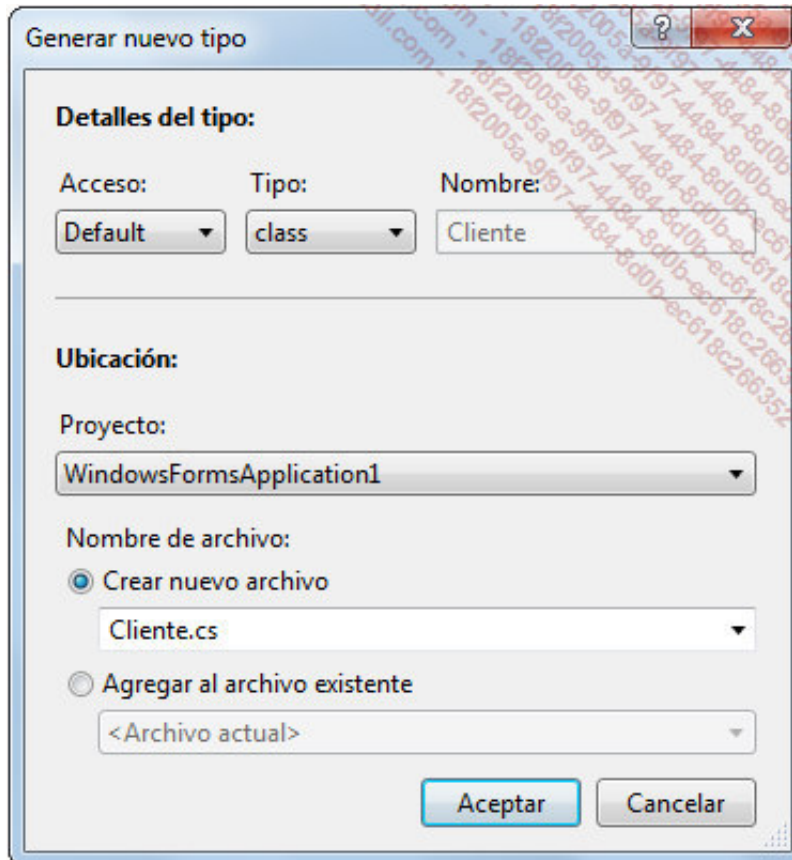
El editor de Visual Studio es capaz de generar el código necesario para los elementos que faltan. Cuando el ratón pasa por encima del elemento referido, aparece un botón bajo este elemento.



Al hacer clic en este botón aparece un menú contextual con las opciones que permiten generar el código que puede resolver los problemas detectados.



Las opciones disponibles en este menú contextual se adaptan según la ubicación del elemento en el que éste está activado. En el ejemplo anterior, el término Cliente puede corresponder a un nombre de clase, enumeración, estructura o interfaz. Sólo hace falta completar el cuadro de diálogo siguiente para que el esqueleto de código se genere.



Zoom

Esta funcionalidad permite efectuar un zoom hacia delante o hacia atrás sobre una ventana de texto. Se puede acceder a ella accionando la rueda del ratón mientras se mantiene pulsada la tecla [Ctrl].

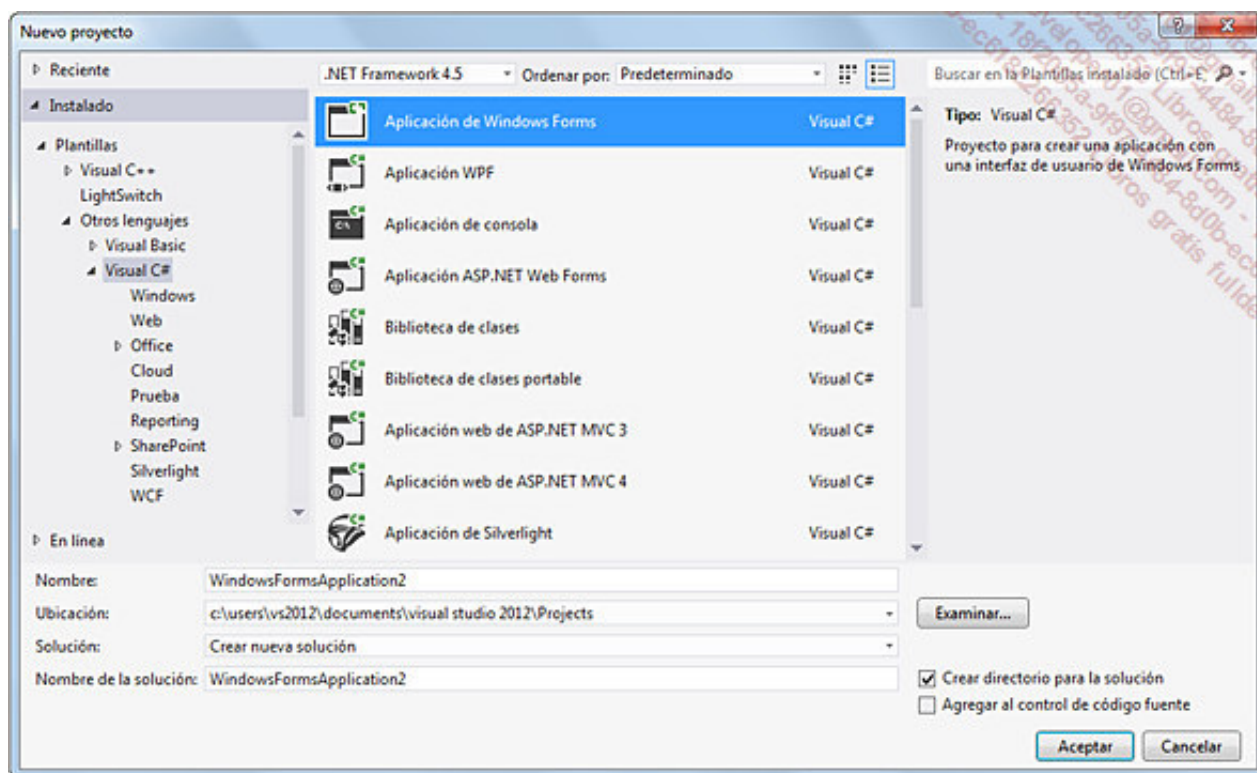
Las soluciones

1. Presentación

Con objeto de ayudarle en la creación de aplicaciones, Visual Studio le propone varios elementos que sirven para agrupar los componentes de una aplicación. El contenedor de más alto nivel es la solución en la cual podrá ubicar uno o varios proyectos. Estos proyectos contendrán, a su vez, todos los elementos para que el compilador sea capaz de generar el archivo ejecutable o dll del proyecto. El explorador de soluciones nos va a permitir manejar todos estos elementos.

2. Creación de una solución

La creación de una solución es automática cuando lanza un nuevo proyecto en Visual Studio. Durante la creación del nuevo proyecto, se le pedirá información al respecto.



A través del cuadro de diálogo, facilitará los datos siguientes:

- la versión del Framework necesario para utilizar la aplicación,
- el lenguaje utilizado para desarrollar el proyecto,
- el tipo de proyecto que hay que crear,
- el nombre del proyecto,
- el directorio raíz donde estarán almacenados los archivos,
- el nombre de la solución,
- la creación de un directorio para la solución.

Después de validar este cuadro de diálogo, el explorador de soluciones le presenta la nueva solución en la cual usted va a poder trabajar. Todos los archivos de su solución ya están creados y guardados en la ubicación del disco que usted ha especificado.

Una solución contendrá al menos los archivos siguientes:

- Un archivo con la extensión .sln, que es el archivo de configuración de la solución. Este archivo contiene entre otros la lista de todos los proyectos que componen la solución. Se completa al mismo tiempo que usted añade nuevos proyectos a la solución.
- Un archivo con la extensión .suo, en el que se guardan las opciones asociadas a la solución. Este archivo permite encontrar estas opciones.
- Un archivo para el proyecto que lleva la extensión .csproj. Este archivo contiene toda la información de configuración del proyecto: en particular, la lista de los archivos que constituyen el proyecto, la lista de referencias utilizadas por este proyecto, las opciones que hay que utilizar para la compilación del proyecto, etc.
- Numerosos archivos con la extensión .cs que van a contener el código fuente de todas las clases, hojas, módulos que constituyen el proyecto.
- Un archivo .resx asociado a cada hoja de su aplicación. Este archivo en formato XNL contiene entre otras la lista de los recursos utilizados en este proyecto.
- Al final, una solución contiene otros numerosos archivos en función de los elementos utilizados en su proyecto (acceso a una base de datos, archivos html...).

3. Modificación de una solución

Las soluciones son contenedores y, por ello, es posible gestionar todos sus elementos. Puede añadir, suprimir, renombrar elementos en la solución.

a. Agregar un proyecto

Hay varias posibilidades para añadir un proyecto:

Si desea crear un nuevo proyecto, elija la opción **Nuevo Proyecto** del menú **Archivo - Agregar**. Un cuadro de diálogo le propone configurar entonces las características del nuevo proyecto. Este cuadro de diálogo le propone un directorio por defecto para guardar el proyecto. Si este directorio no corresponde a la ubicación donde desea grabar el proyecto, puede seleccionar una nueva ubicación. Esta operación se deberá realizar para cada proyecto que quiera añadir. Puede ser interesante modificar la ruta propuesta por defecto para guardar los proyectos. Para ello, abra el menú **Herramientas - Opciones**, en el cuadro de diálogo elija la opción **Proyectos y soluciones** y modifique la sección **Ubicación de los proyectos** de Visual Studio.

Si desea añadir un proyecto ya existente, elija la opción **Proyecto existente** del menú **Archivo - Agregar**. Un cuadro de diálogo de selección de archivos le permite elegir entonces el archivo .csproj del proyecto que desea añadir a la solución.

Tenga en cuenta que el proyecto se mantiene en su ubicación original en el disco.

b. Suprimir un proyecto

Para suprimir un proyecto, utilice el menú contextual del explorador de soluciones efectuando un clic en el nombre del proyecto que desea suprimir dentro de la solución.

Se ha eliminado el proyecto de la solución, pero queda grabado en el disco. Para suprimirlo de manera definitiva, utilice el explorador de Windows para suprimir los archivos de este proyecto. Si no borra los archivos, se puede añadir luego de nuevo el proyecto a una solución.

c. Renombrar un proyecto

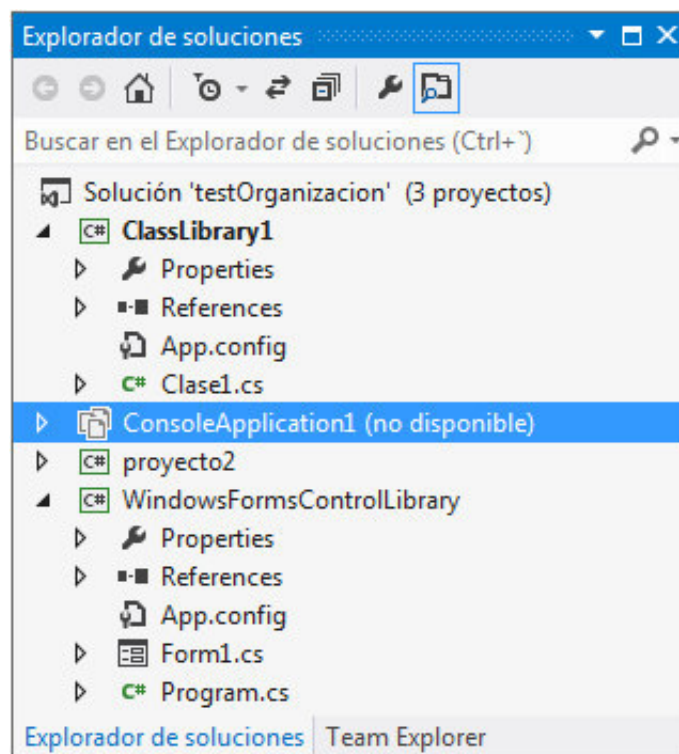
Para renombrar un proyecto, utilice el menú contextual del explorador de soluciones efectuando un clic derecho en el nombre del proyecto que desea renombrar.

El nombre del proyecto puede modificarse en el explorador de soluciones. Esta modificación sólo tiene efecto en el nombre del archivo .csproj asociado al proyecto. No modifica en ningún caso el nombre del directorio en el cual se encuentran los archivos del proyecto.

d. Descargar un proyecto

Si desea excluir de manera temporal un proyecto del proceso de generación o impedir la edición de sus componentes, puede descargar el proyecto de la solución gracias a la opción **Descargar el proyecto**.

- No se elimina un proyecto descargado de la solución, sino que simplemente queda marcado como no disponible.



Por supuesto, se puede rehabilitar el proyecto en la solución utilizando la opción **Volver a cargar el proyecto** del menú contextual.

4. Organización de una solución

Si está trabajando con una solución que contiene numerosos proyectos, puede añadir un nuevo nivel de jerarquía creando carpetas de soluciones. Éstas permiten la agrupación lógica de proyectos dentro

de una solución.

Para ello, cree primero las carpetas en la solución, luego organice los proyectos en estas carpetas.

 Las soluciones no crean carpetas físicas en un disco, sólo son contenedores lógicos en el interior de la solución.

a. Crear una carpeta de solución

Se puede crear una carpeta de solución con dos métodos diferentes.

Para ambos métodos, seleccione la solución en el explorador de soluciones.

Luego, utilice el menú **Proyecto - Agregar nueva carpeta de soluciones**, o incluso el menú contextual disponible con un clic derecho en el nombre de la solución.

Sea cual sea el método utilizado, debe facilitar un nombre para el archivo creado.

b. Crear un proyecto en una carpeta

La creación de un proyecto en una carpeta de solución es idéntica a la creación de un proyecto directamente en la solución.

Seleccione simplemente la carpeta en la que desea crear el proyecto.

c. Desplazar un proyecto a una carpeta

Ocurre a menudo que es necesario organizar una solución con archivos cuando ya existen proyectos en la solución.

En este caso, cree los archivos y arrastre los proyectos a las carpetas correspondientes.

5. La carpeta Elementos de solución

Las soluciones contienen principalmente proyectos; sin embargo es posible tener, en una solución, archivos gestionados de manera independiente de un proyecto particular, pero asociados a la solución. Es el caso, por ejemplo, de un archivo icono que desea utilizar en varios proyectos de la solución. Estos archivos se llaman elementos de solución y se encuentran en una carpeta específica de la solución.

Para añadir un nuevo elemento de solución, abra el menú contextual sobre el nombre de la solución y seleccione la opción **Agregar - Nuevo elemento** o la opción **Agregar - Elemento existente**.

Se añade entonces el nuevo elemento en la carpeta **Elementos de solución**. Debe tener en cuenta que, por defecto, esta carpeta no existe en la solución, sino que se crea automáticamente durante la adición del primer elemento de solución. Luego se puede modificar los elementos de solución con un editor específico al tipo de archivo creado.

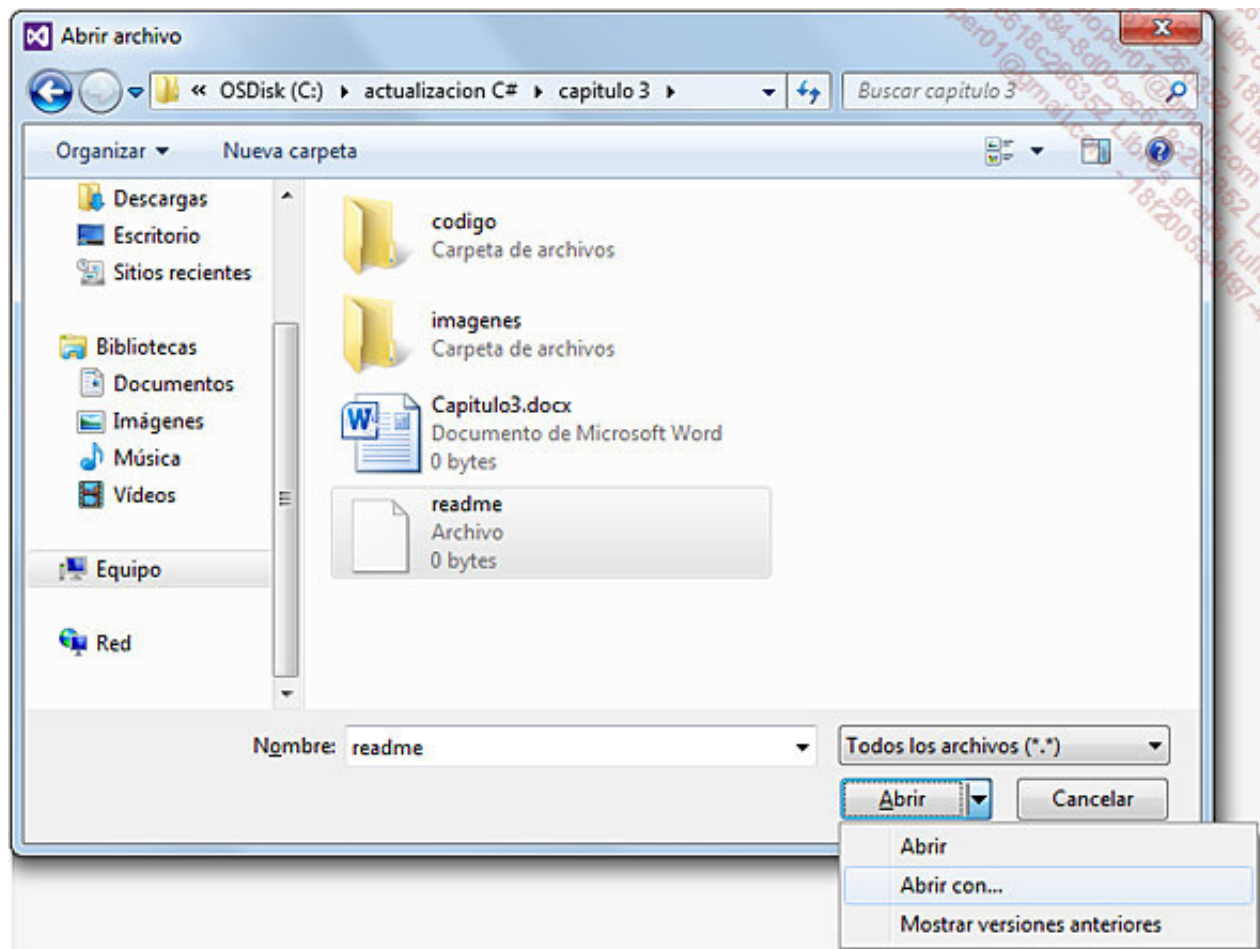
6. La carpeta Archivos varios

A veces puede desear visualizar el contenido de un archivo mientras está trabajando en una solución,

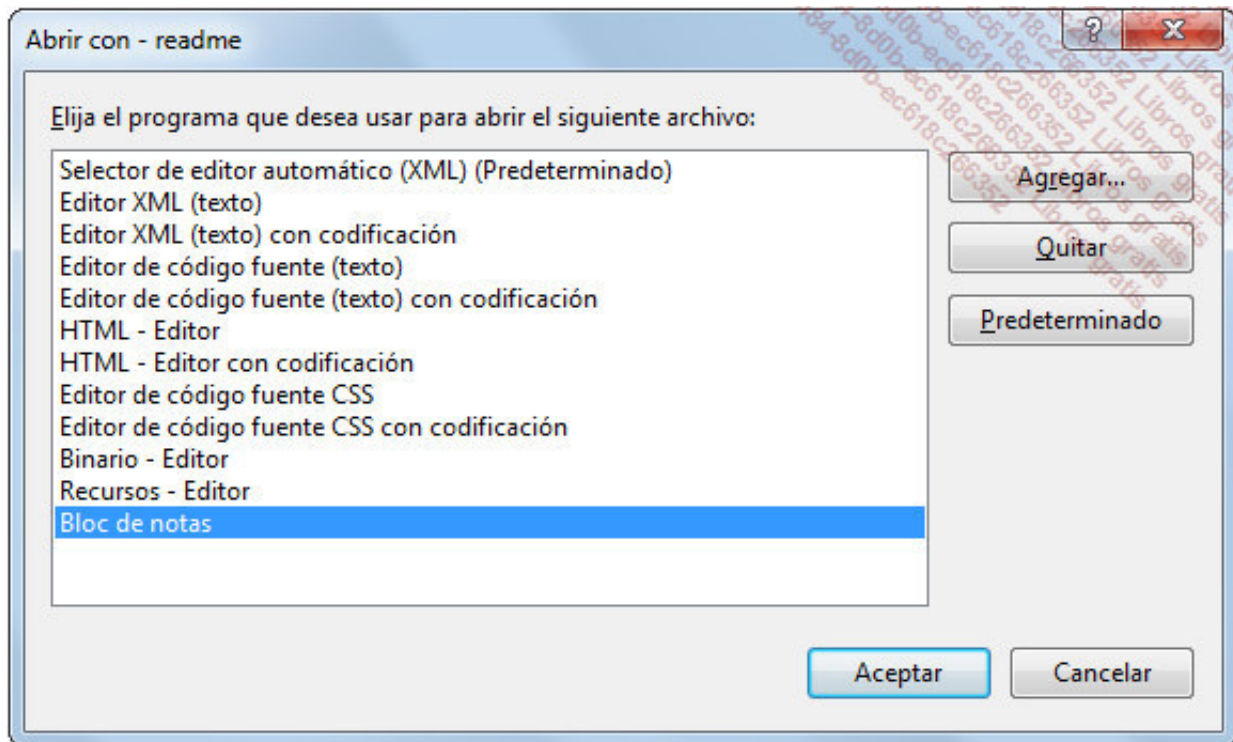
como por ejemplo el acta de una reunión. Este archivo no debe pertenecer a la solución de manera permanente. Puede abrirlo con un editor externo y guardar tanto con Visual Studio como con este editor externo. Pero resulta más práctico visualizar el archivo directamente en el entorno Visual Studio.

Utilice la opción **Abrir - Archivo** del menú **Archivo**.

El cuadro de diálogo le permite elegir el archivo que desea abrir. Según el tipo de archivo, un editor por defecto le será asociado automáticamente para permitir su modificación. Puede resultar útil a veces elegir el editor asociado a un archivo. Para ello, el botón **Abrir** del cuadro de diálogo dispone de un menú que propone la opción **Abrir con** que permite la elección del editor asociado al archivo.



El cuadro de diálogo siguiente le propone la lista de los editores disponibles.



Seleccione el editor asociado al archivo con el que desea trabajar, luego acepte.

El archivo está ahora disponible en la carpeta **Archivos varios** de la solución. De la misma manera que con la carpeta **Elementos de solución**, la carpeta **Archivos varios** no existe por defecto en la solución, sino que se crea automáticamente durante la creación de un archivo.

Sólo será visible en el explorador de soluciones si se activa la opción correspondiente en el entorno Visual Studio. Para ello, abra el menú **Herramientas - Opciones**. Luego, en el cuadro de diálogo, elija la opción **Entorno - Documentos** y active la opción **Mostrar archivos varios en el explorador de soluciones**. Como la carpeta **Elementos de solución**, éste es una carpeta «lógica» y no corresponde a ninguna ubicación en el disco.

7. Configuración de una solución

Las soluciones disponen de propiedades que permiten configurar su comportamiento durante la generación o ejecución de la aplicación. Dichas propiedades están agrupadas en un cuadro de diálogo accesible con la opción **Propiedades** del menú contextual de una solución. Hay cuatro categorías de propiedades disponibles:

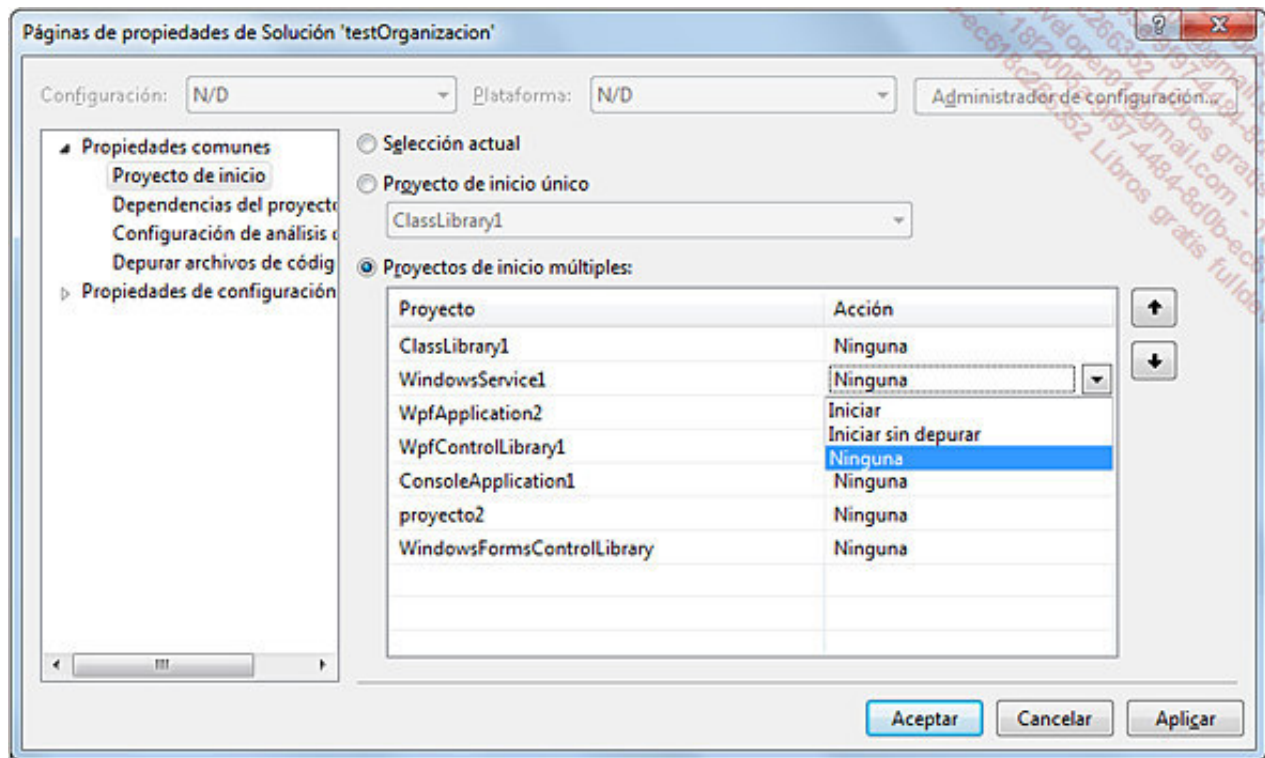
- Proyecto de inicio.
- Dependencias del proyecto.
- Configuración de análisis de código.
- Depurar archivos de código fuente.
- Propiedades de configuración.

Veamos con más detalle cada una de ellas.

a. Configuración del proyecto de inicio

Esta página de propiedades de la solución determina, entre los proyectos disponibles, cuál o cuáles

se inician al ejecutar la solución.



Hay tres opciones disponibles:

Selección actual

Esta opción indica que el proyecto seleccionado en el explorador de soluciones se ejecutará cuando se inicie la solución.

Proyecto de inicio único

Un combo le propone la lista de los proyectos disponibles en la solución, entre los cuales debe elegir el que será ejecutado al abrir la solución. Se marca este proyecto en el explorador de solución con su nombre en negrita. Esta selección también se puede hacer con el menú contextual del explorador de soluciones eligiendo la opción **Establecer como proyecto de inicio**.

Proyectos de inicio múltiples

Hay una tabla que muestra la lista de todos los proyectos disponibles en la solución. Para cada uno de ellos, puede indicar la acción que se debe ejecutar al inicio de la aplicación. Las opciones posibles son:

- Ninguna
- Iniciar
- Iniciar sin depurar.

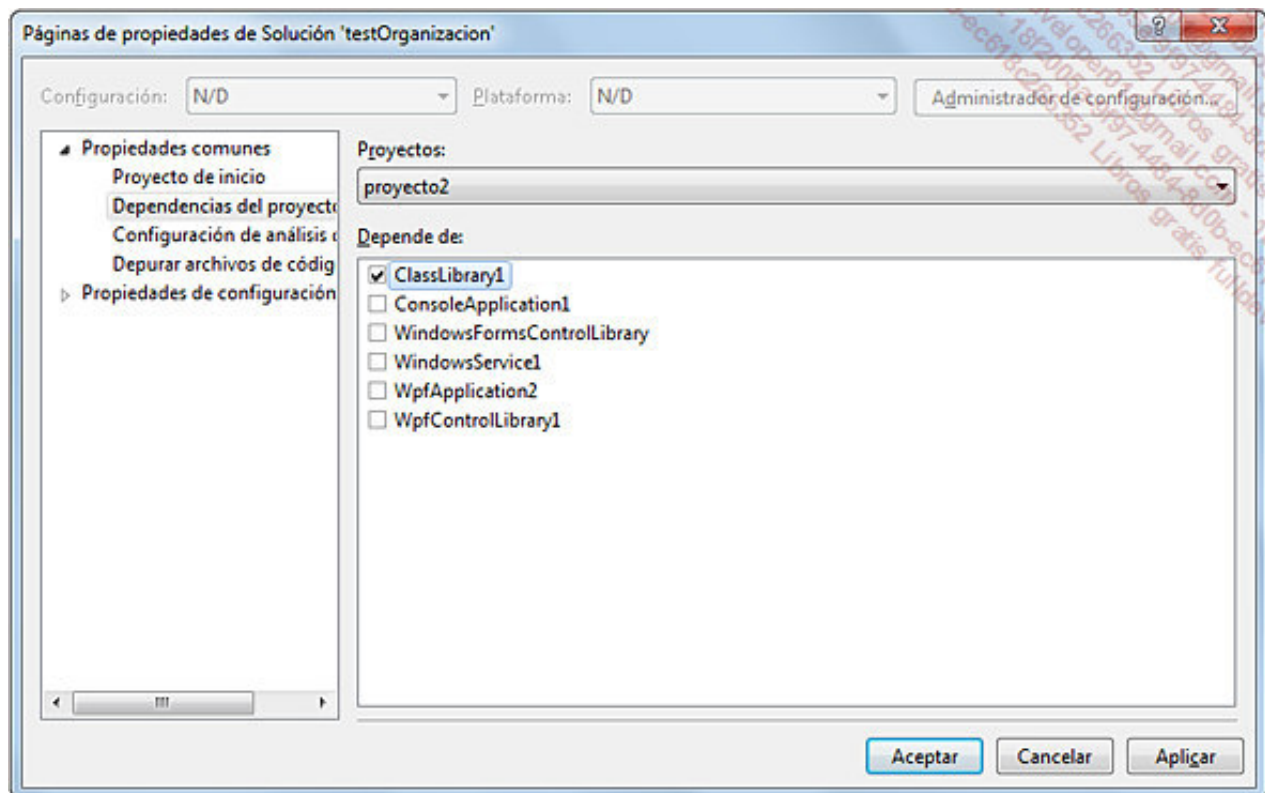
Si elige iniciar varios proyectos a la vez en el lanzamiento de la solución, también debe indicar el orden en el cual se iniciarán estos proyectos. Este orden corresponde en realidad al orden de los

proyectos en la tabla. Los botones  y  permiten modificar este orden.

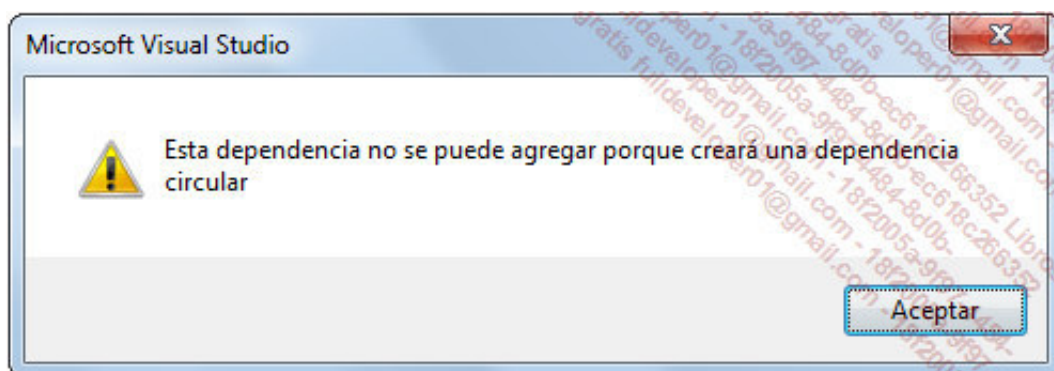
b. Dependencias del proyecto

La generación de algunos proyectos requiere la generación previa de otros proyectos. Es el caso, por ejemplo, de la generación de un proyecto que utiliza una referencia hacia otro: éste se convierte entonces en una dependencia del proyecto inicial.

La página de propiedades siguiente permite configurar estas dependencias.



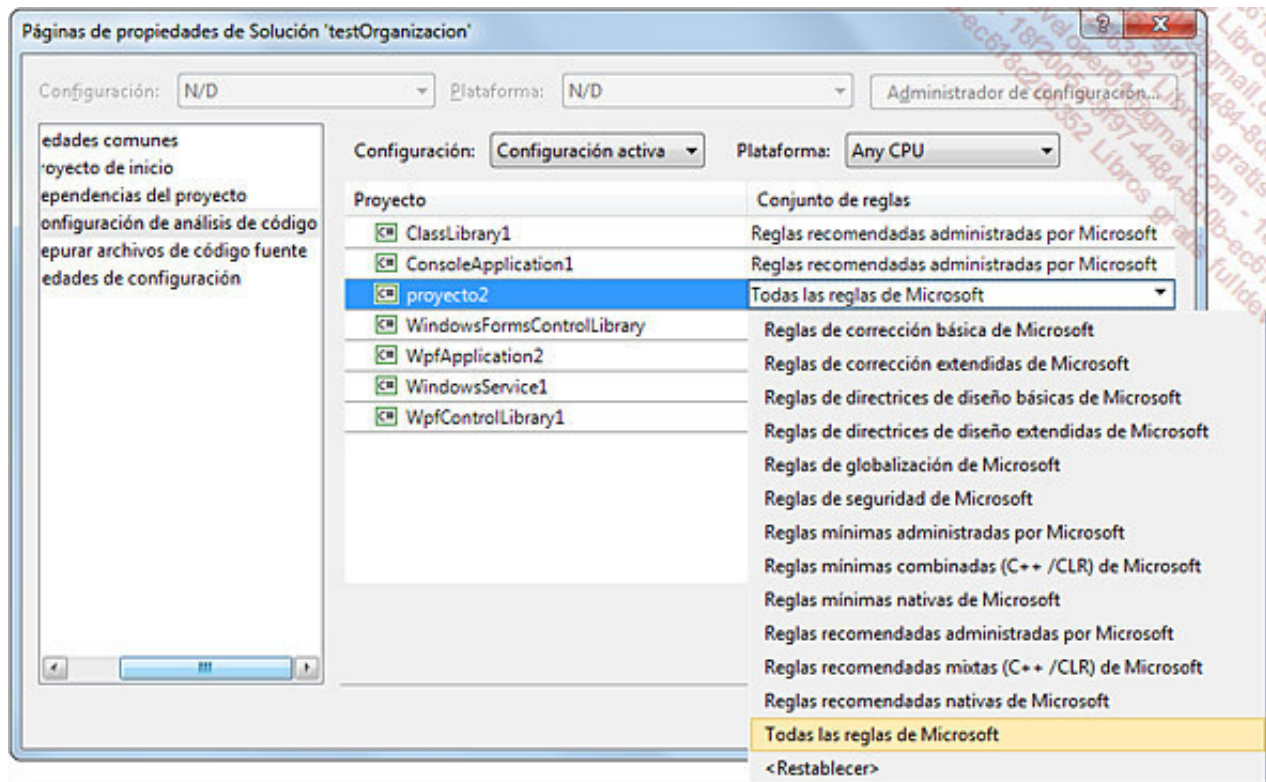
En la lista de los proyectos, seleccione el proyecto cuyas dependencias desea configurar. Los otros proyectos de la solución aparecen entonces en una lista con una casilla de verificación para cada uno. Durante la generación del proyecto, todos los proyectos de los cuales depende serán regenerados automáticamente si han sido modificados desde la última generación o si nunca han sido generados. Algunas dependencias no pueden ser modificadas; por esa razón la casilla de opción aparece en gris. Suele ser el caso cuando un proyecto posee una referencia a otro proyecto o cuando la adición de una dependencia corre peligro de crear un bucle. Por ejemplo, el proyecto1 depende del proyecto2, y a la inversa.



También se puede configurar las dependencias de proyecto con el menú contextual del explorador de soluciones mediante la opción **Dependencias del proyecto**.

c. Configuración de análisis de código

Esta pantalla le permite configurar las reglas utilizadas durante el análisis del código de los distintos elementos de la solución.



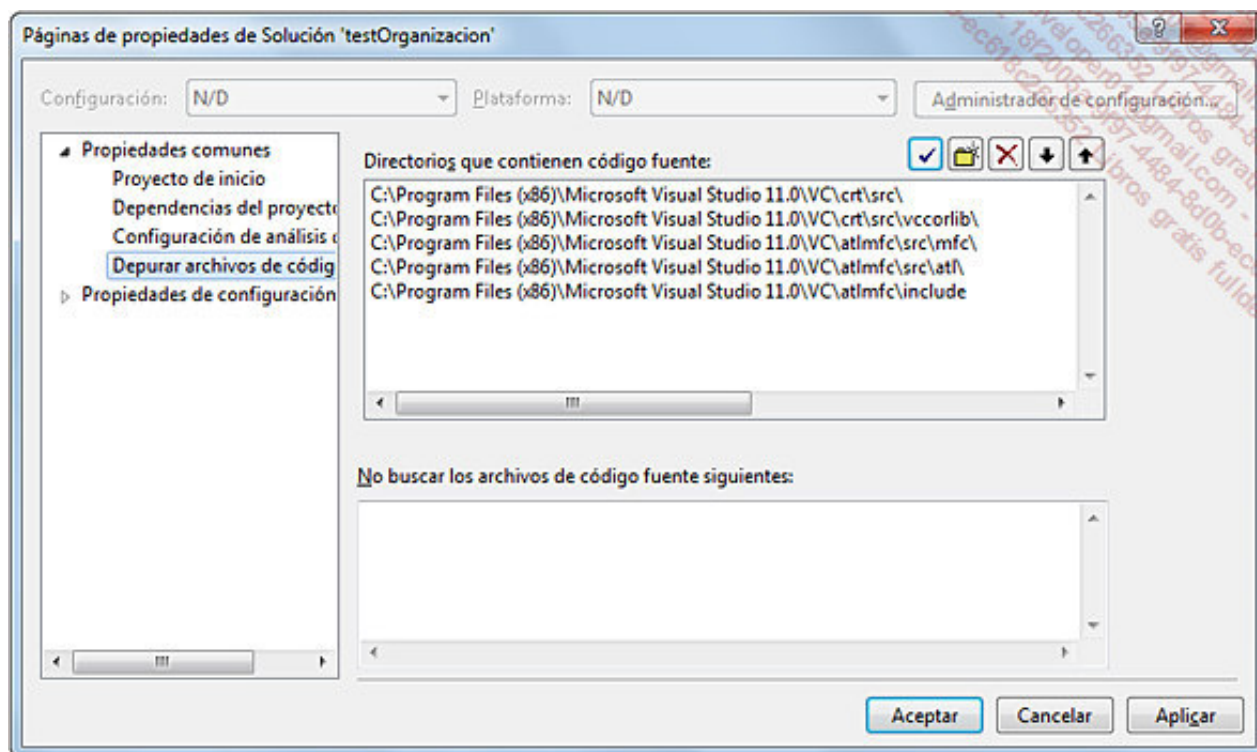
Para cada proyecto de la solución, puede indicar qué configuración utilizarán las herramientas de análisis.

La opción **Todas las reglas de Microsoft** es la más estricta y detecta la más mínima anomalía, en particular:

- Parámetros que se pasan a la función y no se utilizan en el interior de la misma.
- Variables locales que no se utilizan.
- Nombres de parámetros poco explícitos.
- Si no se respeta las convenciones respecto a las mayúsculas y minúsculas de los identificadores.

d. Depurar archivos de código fuente

Durante la depuración de una aplicación, el entorno de Visual Studio necesita acceder al archivo fuente del código que está depurando. Esta página de propiedad permite especificar los directorios que serán analizados durante la búsqueda del código fuente.



La lista **Directorios que contienen código fuente** muestra el nombre de los directorios que serán abiertos durante la búsqueda de código fuente. Se puede gestionar esta lista gracias a la barra de herramientas cuyos botones permiten:

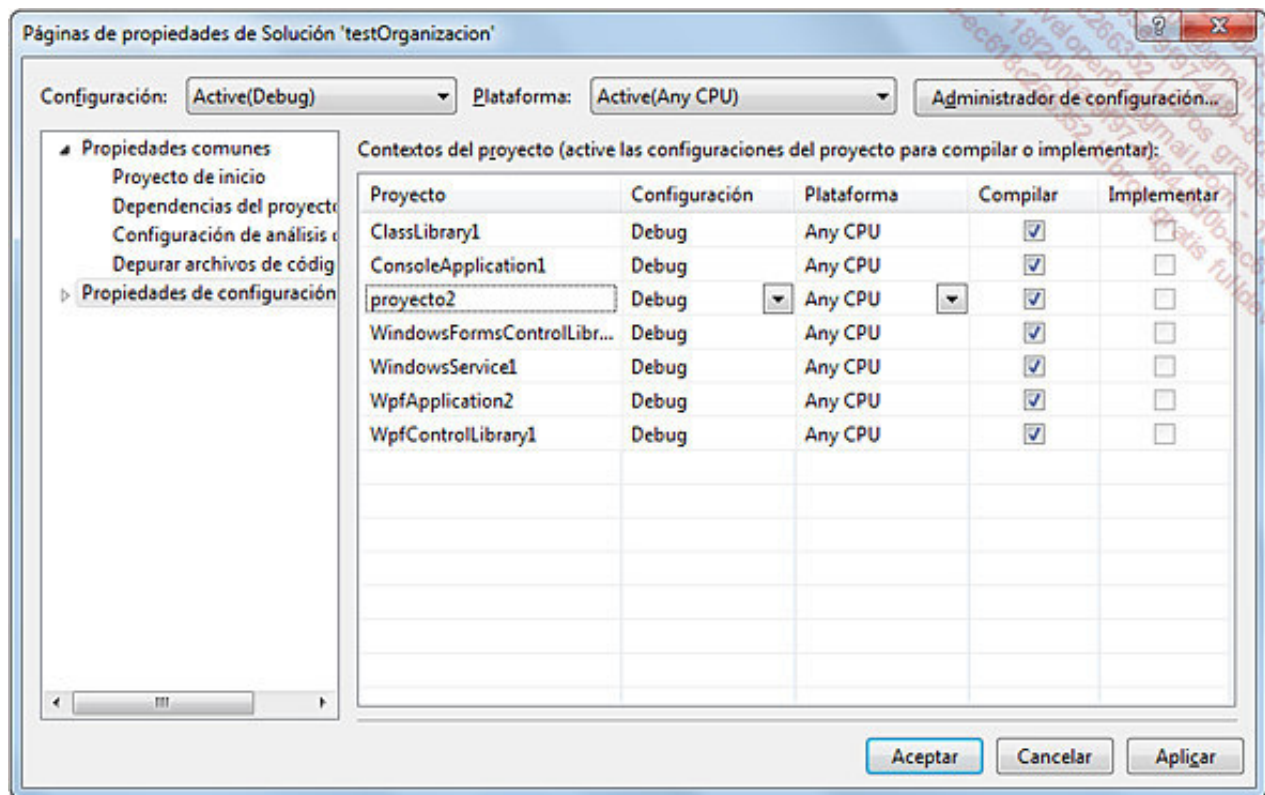
-  Comprobar la existencia del directorio.
-  Añadir un nuevo directorio.
-  Suprimir el directorio seleccionado de la lista.
-  Desplazar el directorio hacia abajo en la lista.
-  Desplazar el directorio hacia arriba en la lista.

La lista **No buscar los archivos de código fuente siguientes** excluye algunos archivos de la búsqueda.

e. Propiedades de configuración

Las opciones de configuración permiten definir cómo se generan varias versiones de una solución y de los proyectos que la componen. Por defecto, hay dos configuraciones disponibles para una solución en Visual Studio: la configuración Debug y la configuración Release.

Para cada uno de los proyectos presentes en la solución, las dos configuraciones también estarán disponibles. A nivel de proyecto, las configuraciones permiten definir opciones de compilaciones. Se utiliza la configuración Debug durante el desarrollo y las pruebas del proyecto. Se utiliza la configuración Release para la generación final del proyecto.



En realidad, tenemos un sistema de tres niveles: para cada configuración de solución, se indica qué configuración utilizar en cada proyecto, y para cada configuración de proyecto, se especifica opciones de compilación. Se pueden modificar las opciones de compilación a nivel de las propiedades del proyecto.

Los proyectos

Los proyectos son los contenedores de segundo nivel en una aplicación. Se utilizan para organizar lógicamente, gestionar, generar y depurar los componentes de una aplicación. La generación de un proyecto suele producir un archivo ejecutable o una librería dll. Un proyecto puede ser muy simple y sólo contener dos elementos, un archivo fuente (.cs) y el archivo de proyecto (.csproj). Más comúnmente, los proyectos contienen numerosos archivos fuente, script básicos de datos, referencias hacia servicios Web, recursos gráficos, etc.

Visual Studio propone por defecto un conjunto de plantillas de proyectos. Estas plantillas representan un punto de partida para la mayoría de las necesidades en el desarrollo de una aplicación. Para casos más específicos, puede crear sus propias plantillas de proyecto.

1. Creación de un proyecto

Para activar la creación de un proyecto, active el menú **Archivo - Nuevo proyecto**. Un cuadro de diálogo le propone entonces elegir las características del nuevo proyecto.

Elija primero la versión del Framework para la cual desea desarrollar el proyecto. La versión elegida influye en los tipos de proyectos que puede crear.

Elija luego el lenguaje con el cual desea desarrollar el proyecto. Las elecciones disponibles dependen de los lenguajes instalados en Visual Studio. En nuestro caso, elegimos naturalmente Visual C#.

Luego elija el tipo de proyecto que desea desarrollar. El cuadro de diálogo propone entonces las diferentes plantillas de proyectos disponibles según el tipo de proyecto elegido.

Después de haber hecho su elección, dé un nombre al proyecto, una ubicación para los archivos del proyecto y un nombre para la solución. El asistente utiliza la plantilla seleccionada para crear los elementos del proyecto.

Después de unos instantes, el proyecto estará disponible en el explorador de soluciones.

Ahora personalice la plantilla creada.

a. Las plantillas de proyectos

Hay numerosas plantillas de proyectos disponibles en Visual Studio. Estas plantillas facilitan los elementos básicos necesarios para desarrollar cada tipo de proyecto. Siempre contienen al menos el archivo de proyecto, más un ejemplar del elemento más utilizado para el tipo de proyecto correspondiente. Por ejemplo, para un proyecto de librería clase, se crea un archivo fuente que contiene un boceto de clase. Las plantillas proveen también referencias e importaciones por defecto para las librerías y los espacios de nombres más útiles en función del tipo de proyecto.

Aplicación Windows Forms

Esta plantilla de proyecto es seguramente la más utilizada. Permite el desarrollo de aplicación de Windows estándar. La plantilla añade los elementos siguientes al proyecto:

- Un archivo AssemblyInfo.cs utilizado para la descripción de la aplicación con la información relativa a la versión.
- Un formulario básico con su archivo fuente form1.cs.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Deployment
- System.Drawing
- System.Windows.Forms
- System.Xml
- System.Xml.Linq

Librería de clases

Esta plantilla de proyecto se puede utilizar para crear clases y componentes que luego podrán ser compartidos con otros proyectos. Los elementos siguientes se añaden automáticamente al proyecto:

- Un archivo AssemblyInfo.cs utilizado para la descripción del proyecto con la información relativa a la versión.
- Una clase básica con su archivo fuente class1.cs.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Xml
- System.Xml.Linq

Librería de controles Windows Forms

Como la plantilla anterior, este tipo de proyecto permite crear una librería de clases utilizable en otros proyectos. Esta librería es más específica, ya que está dedicada a la creación de controles, utilizables luego en una aplicación de Windows. Estos controles amplían el cuadro de herramientas disponible en las aplicaciones de Windows. Los elementos siguientes se añaden automáticamente al proyecto:

- Un archivo AssemblyInfo.cs utilizado para la descripción del proyecto con la información relativa a la versión.
- Una clase UserControl1 que hereda de la clase System.Windows.Forms.UserControl que facilita las funcionalidades básicas para un control de Windows, con su archivo fuente UserControl1.cs.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp

- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Drawing
- System.Windows.Forms
- System.Xml
- System.Xml.Linq

Aplicación de consola

Este tipo de aplicación está destinado a ejecutarse desde la línea de comandos. Por supuesto está diseñada sin interfaz gráfica, y las entradas y salidas van y vienen desde y hacia la consola.

Este tipo de aplicación es muy práctica para realizar pruebas con Visual C#, ya que permite concentrarse en un punto particular sin tener que preocuparse del aspecto presentación de la aplicación.

Muchos ejemplos de este libro se basan en una aplicación de consola. Sin embargo, hay que admitir que, aparte de la sencillez de su creación, este tipo de aplicación se ha vuelto obsoleta.

Los elementos siguientes se incorporan por defecto al proyecto:

- Un archivo `AssemblyInfo.cs` utilizado para la descripción del proyecto con la información relativa a la versión.
- Una clase básica con su archivo fuente `Program.cs`.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Xml
- System.Xml.Linq

Servicio Windows

Se usa este tipo de plantilla para la creación de aplicaciones que se ejecutan en segundo plano en el sistema. El inicio de este tipo de aplicaciones puede asociarse al del propio sistema y no necesita que haya una sesión de usuario abierta para poder ejecutarse.

Este tipo de aplicación está desprovisto de interfaz de usuario. Si se debe comunicar información al usuario, deberá transitar por los diarios sistema disponibles en el visor de sucesos. Los elementos siguientes se añaden al proyecto:

- Un archivo `AssemblyInfo.cs` utilizado para la descripción del proyecto con la información relativa a la versión.

- Una clase básica con el esqueleto de procedimientos `OnStart` y `OnStop` llamada automáticamente en el inicio y la parada del servicio.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.ServiceProcess
- System.Xml
- System.Xml.Linq

Aplicación WPF

Esta plantilla de proyecto permite beneficiarse del nuevo sistema de visualización gráfica de Windows, utilizado en Windows Vista.

Los elementos siguientes se añaden automáticamente al proyecto:

- Un archivo `AssemblyInfo.cs` utilizado para la descripción de la aplicación con la información relativa a la versión.
- Un archivo `App.Xaml` y su archivo de código asociado, `App.Xaml.cs`, permite la gestión de eventos desactivados a nivel de aplicación.
- Una ventana básica `Window1.Xaml` y su archivo de código asociado, `Window1.Xaml.cs`.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- PresentationCore
- PresentationFramework
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Xaml
- System.Xml
- System.Xml.Linq
- WindowsBase

Librería de controles usuario WPF

Como la librería de controles Windows, este tipo de proyecto permite ampliar el cuadro de herramientas ya disponible en las aplicaciones WPF. Se añaden los elementos siguientes al proyecto:

- Un archivo `AssemblyInfo.cs` utilizado para la descripción de la aplicación con la información relativa a la versión.
- Un archivo `UserControl1.xaml` para la definición del aspecto gráfico del control.
- Un archivo `UserControl1.xaml.cs` para el código asociado a este control.

Las referencias siguientes se añaden e importan automáticamente:

- Microsoft.CSharp
- PresentationCore
- PresentationFramework
- System
- System.Core
- System.Data
- System.Data.DataSetExtensions
- System.Xaml
- System.Xml
- System.Xml.Linq
- WindowsBase

Librería de controles WPF personalizados

Este tipo de proyecto también tiene por vocación extender el cuadro de herramientas disponible para las aplicaciones WPF. A diferencia del tipo de proyecto anterior, los controles no han sido creados completamente, sino que están basados en controles existentes cuyas características extienden.

Las referencias e importaciones son idénticas al tipo de proyecto anterior.

Proyecto vacío

Debe utilizar esta plantilla cuando desee crear su propio tipo de proyecto. Sólo crea un archivo de proyecto. A cambio, no se añade ningún otro elemento automáticamente ni crea o importa referencia alguna.

b. Creación de una plantilla de proyecto

Puede crear su propia plantilla de proyecto según sus costumbres de desarrollo y hacerlo de tal manera que aparezca entre las plantillas predefinidas.

Debe diseñar los elementos siguientes:

- Un archivo de definición que contiene los metadatos de la plantilla. Visual Studio utiliza este archivo para la visualización del proyecto en el entorno de desarrollo y para la asignación de propiedades por defecto al proyecto. Estos datos están contenidos en un archivo XML con la extensión `.vstemplate`.
- Un archivo para el proyecto (`.csproj`).
- Los archivos fuentes y recursos incluidos por defecto durante la creación de un proyecto a partir de esta plantilla.



Se debe comprimir estos archivos en un archivo zip. El archivo zip debe contener los archivos individualmente, y no el directorio en el que están ubicados.

El archivo .vstemplate debe tener el formato siguiente:

```
<VSTemplate Version="2.0.0"
xmlns="http://schemas.microsoft.com/developer/vstemplate/2005" Type="Project">
  <TemplateData>*
    <Name>AppliPerso</Name>
    <Description>Creacion de un proyecto con una configuracion personalizada
  </Description>
    <ProjectType>CSharp</ProjectType>
    <DefaultName>AppliPerso</DefaultName>
  </TemplateData>
  <TemplateContent>
    <Project File="AppliPerso.csproj">
      <ProjectItem>AssemblyInfo.cs</ProjectItem>
      <ProjectItem>Hojal.cs</ProjectItem>
      <ProjectItem>Hojal.Designer.cs</ProjectItem>
      <ProjectItem>Hojal.resx</ProjectItem>
    </Project>
  </TemplateContent>
</VSTemplate>
```

En este archivo encontramos:

En la sección Name

El nombre visualizado por el cuadro de diálogo de creación de un nuevo proyecto.

En la sección Description

Una descripción detallada del proyecto.

En la sección ProjectType

El nombre del archivo en el cual este proyecto será clasificado en el cuadro de diálogo de creación de proyecto.

En la sección DefaultName

El nombre utilizado por defecto para todos los proyectos creados desde esta plantilla. Se completa este nombre con un sufijo numérico en la creación del proyecto.

En la sección Project File

El nombre del archivo proyecto asociado a la plantilla. Este archivo debe estar presente en el archivo zip de la plantilla.

En las secciones ProjectItem

Los elementos que forman parte del proyecto. También estos elementos deben estar disponibles en el archivo zip.

c. Modificación de una plantilla existente

La modificación de una plantilla consiste en utilizar un archivo zip existente que contiene los elementos necesarios al proyecto y añadir elementos adicionales. Si se añaden archivos a la plantilla, se les debe ubicar en el archivo zip y también referenciarlos en el archivo .vstemplate. Las plantillas predefinidas de Visual Studio están ubicadas en el directorio C:\Program Files\Microsoft Visual Studio 11.0\Common7\IDE\ProjectTemplates\CSharp. Para que se tengan en cuenta las modificaciones, debe actualizar la caché utilizada por Visual Studio.

Para ello:

Abra una ventana de comando Visual Studio.

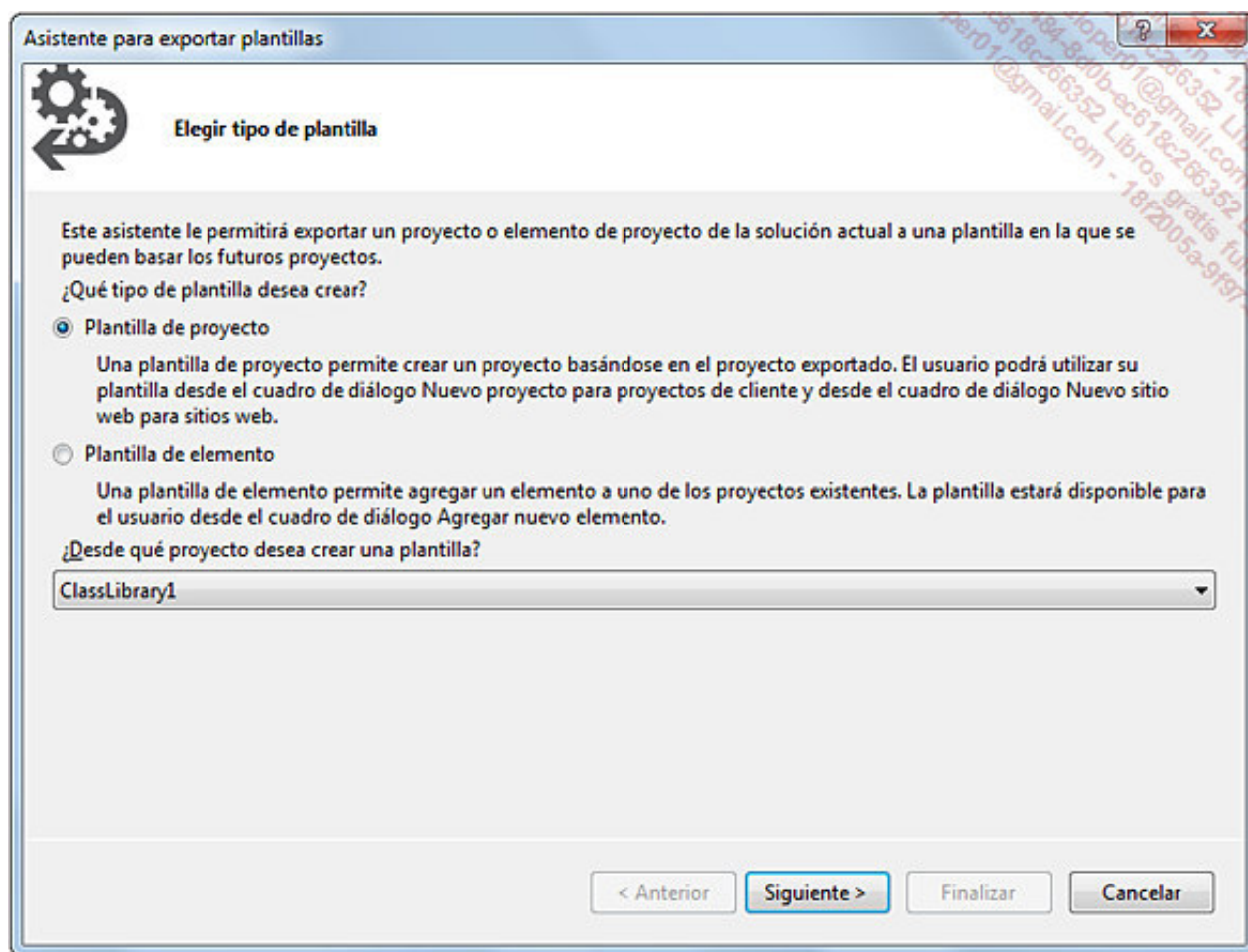
Introduzca el comando `devenv /setup`. Sea paciente, ya que este comando tarda bastante en ejecutarse. Después de la ejecución del comando, sus modificaciones están disponibles en la plantilla de proyecto.

d. Utilización de un proyecto existente como plantilla

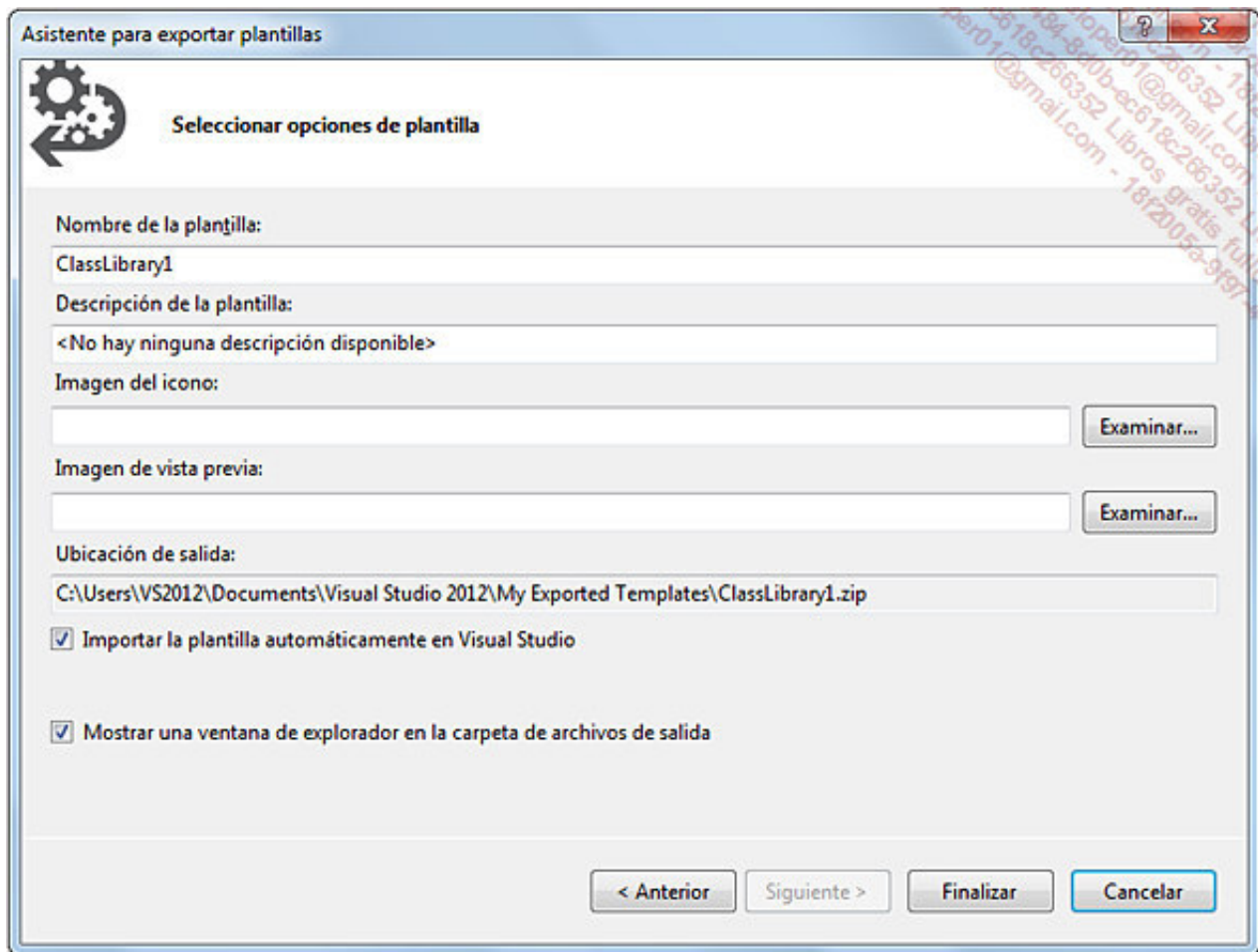
Puede que sea la solución más simple para construir una plantilla de proyecto.

En una primera fase cree la plantilla como un proyecto ordinario.

Una vez finalizado su proyecto, expórtelo como plantilla. El menú **Archivo - Exportar plantilla** inicia un asistente para guiarle durante la creación de la plantilla.



Este primer cuadro de diálogo le propone elegir el proyecto que desea exportar.



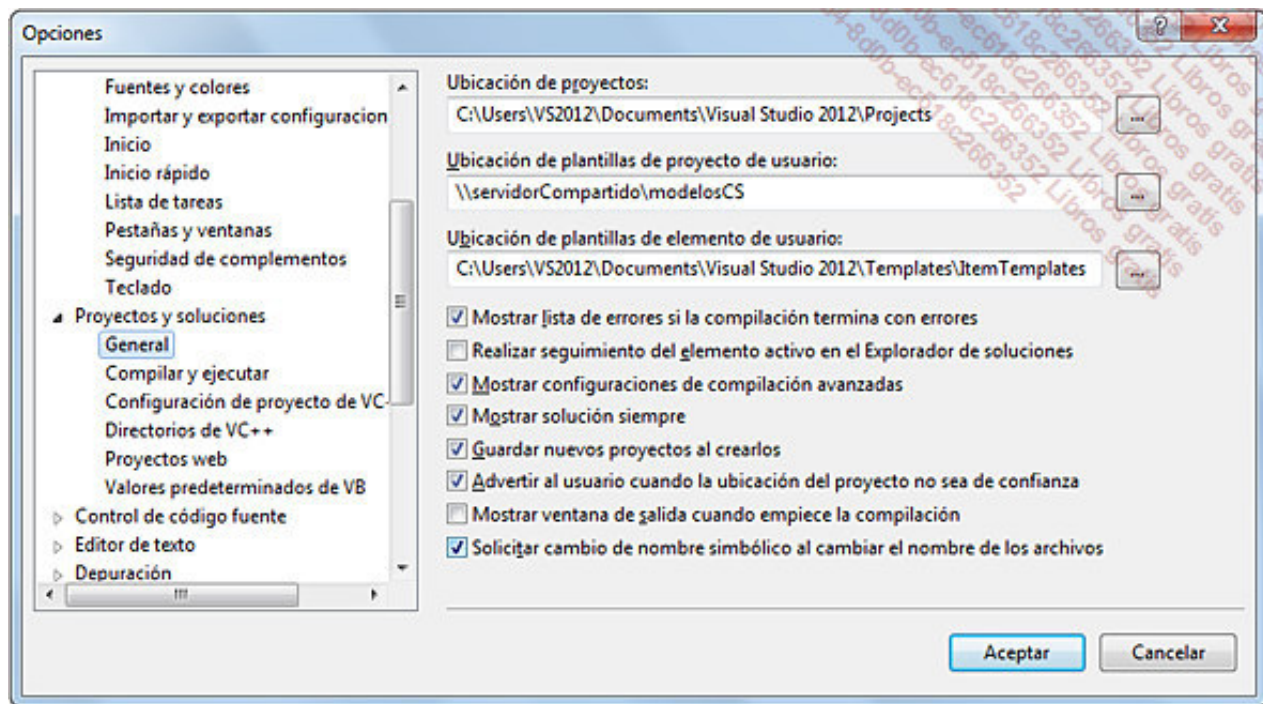
Este segundo cuadro de diálogo le invita a elegir un icono para su plantilla de proyecto, un nombre para la plantilla y una descripción. Hay dos opciones adicionales que le permiten tener en cuenta inmediatamente la nueva plantilla en Visual Studio y presentarle el resultado de la generación mostrándole el contenido del archivo zip creado. Después de validar este último cuadro de diálogo, la nueva plantilla de proyecto está disponible en Visual Studio.

- Este método es muy simple para construir una nueva plantilla de proyecto y evita enredarse con la sintaxis del archivo `.vstemplate`.

En el marco de un desarrollo en equipo, puede resultar interesante compartir las plantillas personalizadas entre todos los miembros del equipo.

Copie otra vez los archivos zip en una red compartida.

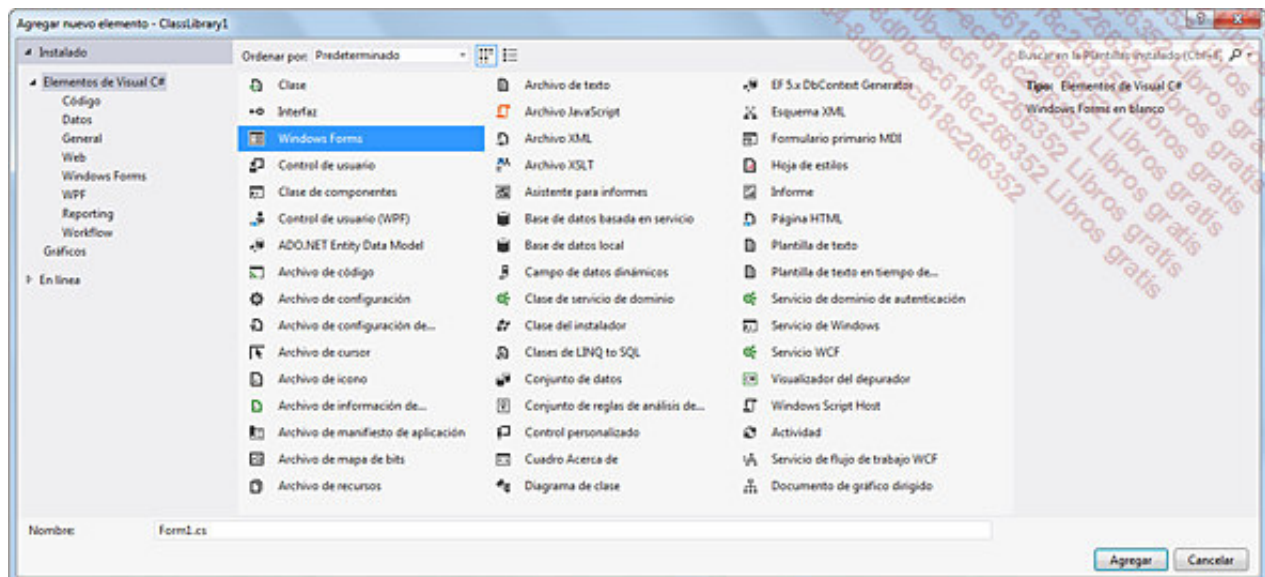
Configure el entorno Visual Studio para permitirle acceder a las plantillas. Esta modificación se efectúa gracias al cuadro de diálogo disponible en el menú **Herramientas - Opciones**.



2. Modificación de un proyecto

Las plantillas de proyectos son muy útiles para crear rápidamente las bases de una aplicación, pero a menudo necesitarán el añadido de nuevos elementos al proyecto. Estos añadidos se hacen por medio del menú contextual del explorador de proyecto.

Active la opción **Agregar - Nuevo elemento** a fin de elegir el tipo de elemento que desea añadir al proyecto. El cuadro de diálogo propone un número impresionante de elementos que se pueden añadir a un proyecto.

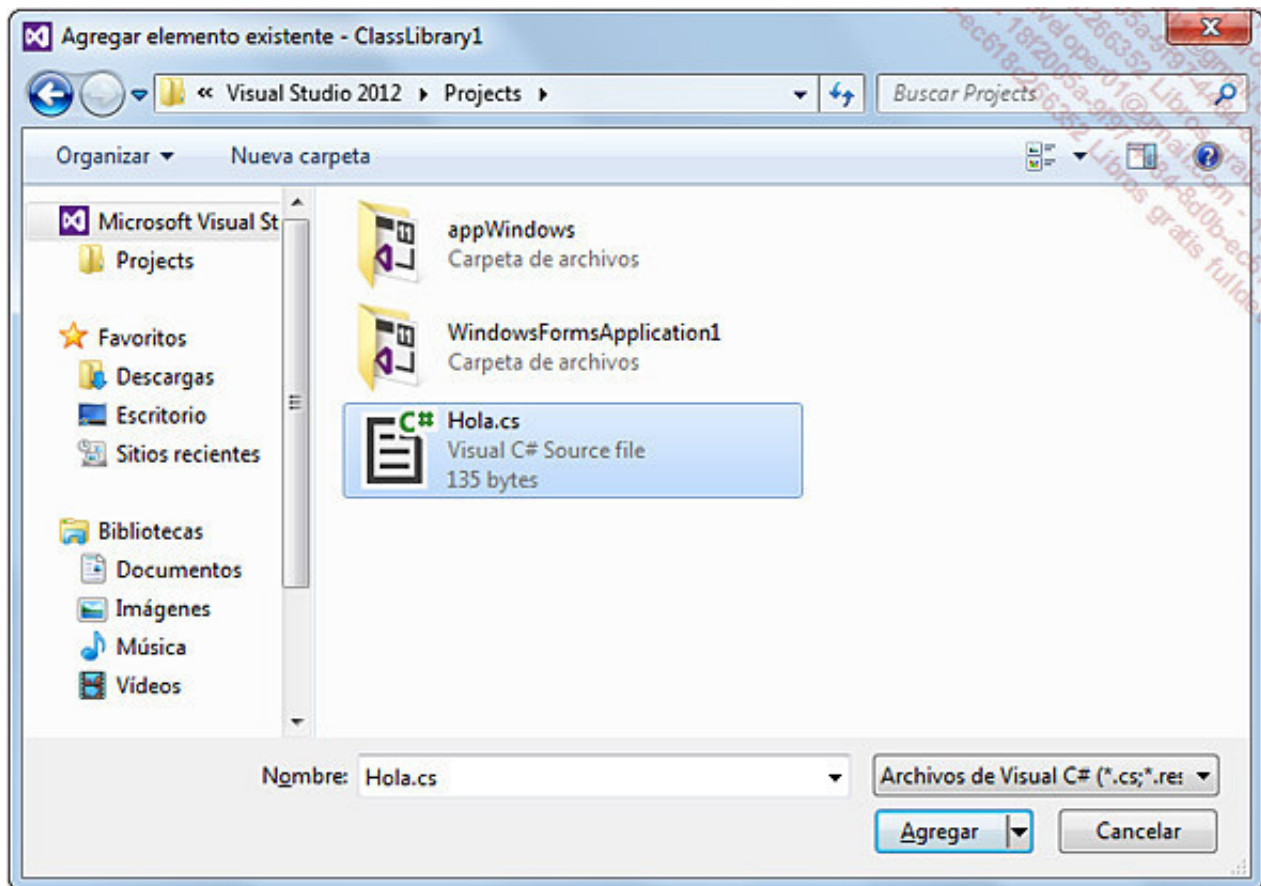


Indique luego un nombre para el archivo que contiene el nuevo elemento.

➤ En función de los tipos de proyecto, hay opciones adicionales disponibles en el menú contextual que permiten añadir rápidamente un nuevo elemento. Se visualizan simplemente en

el cuadro de diálogo anterior con el tipo de elemento correspondiente ya preseleccionado.

También es posible retomar un elemento existente en otro proyecto y añadirlo a un proyecto. Utilice en este caso la opción **Agregar - Elemento existente** del menú contextual del explorador de proyectos. Un cuadro de diálogo le propone la selección del archivo que hay que incluir en el proyecto.



El botón **Agregar** de este cuadro de diálogo comporta un menú que permite añadir el archivo de forma normal (se realiza una copia local del archivo) o crear un vínculo en el archivo (se utiliza el archivo original). Hay que ser prudente con esta posibilidad, ya que el archivo «no pertenece» realmente a la aplicación, pero se puede compartir entre varias aplicaciones. Si se suprime el archivo del disco, ninguna de las aplicaciones que lo utilizan se podrán compilar.

➤ La gestión de los archivos en el explorador de soluciones es idéntica a la gestión de los archivos en el explorador de Windows. Se puede copiar y pegar, o desplazar los archivos arrastrando una carpeta a otra. El uso de las teclas [Ctrl], [Shift] y [Ctrl][Shift] modifica la acción realizada durante el arrastre. Si el arrastre se produce dentro del mismo proyecto, efectúa un desplazamiento de archivo. Si se realiza entre dos proyectos, se efectúa entonces una copia de archivo. Se puede modificar este compartamiento mediante la utilización de la tecla [Shift] durante el arrastre. Para realizar una copia de archivo dentro de un proyecto, se utilizará la tecla [Ctrl] con el arrastre. La creación de un vínculo se efectúa con la combinación de teclas [Ctrl][Shift] durante el arrastre.

Para quitar un elemento de un proyecto, dos opciones están accesibles con el menú contextual del explorador de soluciones:

- La opción **Eliminar** suprime el archivo del proyecto, pero también del disco.

- La opción **Excluir del proyecto** quita el archivo del proyecto, pero no lo suprime del disco. Esta opción es útil si otros proyectos utilizan este archivo por medio de un vínculo.

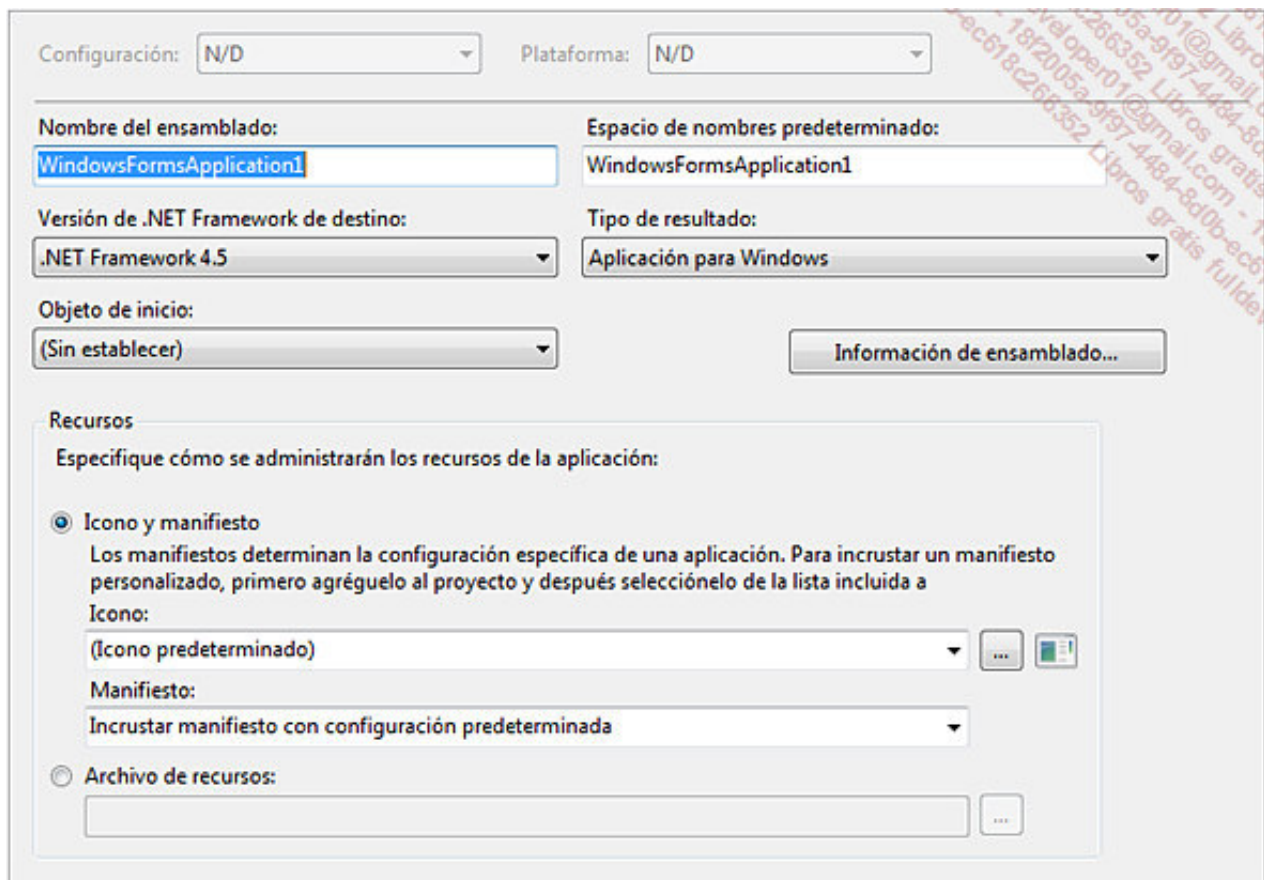
3. Propiedades de los proyectos

Los proyectos son elementos fundamentales del diseño de una aplicación con Visual C#. Poseen muchas propiedades que permiten modificar sus comportamientos en el momento de diseñar o ejecutar la aplicación. El conjunto de las propiedades están accesibles a través de un cuadro de diálogo que presenta mediante pestañas las diferentes secciones de configuración de un proyecto.

Active este cuadro de diálogo con la opción **Propiedades** del menú contextual del explorador de soluciones o con el botón  de la barra de herramientas del explorador de soluciones.

a. Aplicación

Las propiedades presentes en esta pestaña van a permitir configurar el comportamiento de la aplicación.



Configuración: N/D Plataforma: N/D

Nombre del ensamblado: Espacio de nombres predeterminado:

Versión de .NET Framework de destino: .NET Framework 4.5 Tipo de resultado: Aplicación para Windows

Objeto de inicio: (Sin establecer) Información de ensamblado...

Recursos

Especifique cómo se administrarán los recursos de la aplicación:

☒ **Icono y manifiesto**
Los manifiestos determinan la configuración específica de una aplicación. Para incrustar un manifiesto personalizado, primero agréguelo al proyecto y después selecciónelo de la lista incluida a

Icono:  

Manifiesto:

☐ **Archivo de recursos:** 

Nombre del ensamblado

Esta propiedad determina el nombre utilizado para el archivo resultante de la compilación de la aplicación. Por defecto, este archivo lleva el mismo nombre que el proyecto, pero se pueden modificar los dos de manera independiente el uno del otro. La extensión asociada al archivo depende del tipo del proyecto.

Framework de destino

Esta propiedad indica la versión del framework necesaria para la ejecución de la aplicación. Por defecto, este valor es idéntico al indicado durante la creación del proyecto.

Objeto de inicio

Esta propiedad determina el punto de entrada en la aplicación durante su ejecución. Corresponde al nombre de la clase que contiene la función `Main`. Esta función suele ser la encargada de crear la instancia de la ventana principal de la aplicación y asegurar su visualización. Esta propiedad sólo está disponible para los proyectos que se pueden ejecutar de manera autónoma. En el caso de una librería de clase, por ejemplo, contiene el valor '(no definido)'.

Espacio de nombres predeterminado

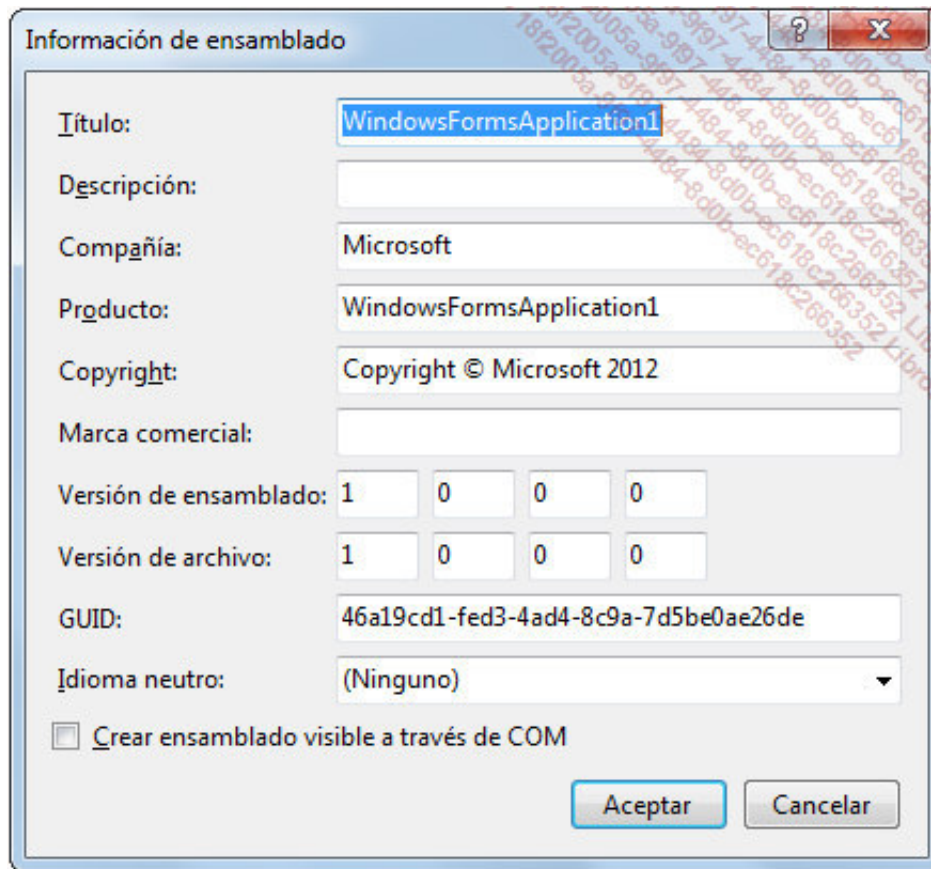
Todos los elementos del proyecto accesibles a partir de otro proyecto pertenecen al espacio de nombres definido por esta propiedad. Ésta viene a añadirse a los posibles espacios de nombres definidos a nivel del propio código. Por defecto, esta propiedad corresponde al nombre del proyecto, pero se puede modificar de manera independiente de éste. Incluso puede estar vacía, lo que le permite generar espacios de nombres directamente en el código.

Tipo de resultado

Esta propiedad determina el tipo de aplicación generada por la compilación del proyecto. Por regla general, esta propiedad viene determinada por el modelo escogido durante la creación del proyecto. Esta propiedad raramente se modifica puesto que de ella depende mucha parte del código del proyecto (si se ha creado la aplicación como una aplicación Windows y quiere considerarla como una aplicación de consola, se tiene el riesgo de tener bastante código inútil!).

Información del ensamblado

Esta opción permite facilitar información sobre el código generado por la compilación del proyecto. Un cuadro de diálogo permite informar distintas secciones relativas a la descripción del proyecto.



Información de ensamblado

Título: WindowsFormsApplication1

Descripción:

Compañía: Microsoft

Producto: WindowsFormsApplication1

Copyright: Copyright © Microsoft 2012

Marca comercial:

Versión de ensamblado: 1 0 0 0

Versión de archivo: 1 0 0 0

GUID: 46a19cd1-fed3-4ad4-8c9a-7d5be0ae26de

Idioma neutro: (Ninguno)

☐ Crear ensamblado visible a través de COM

Aceptar Cancelar

El usuario de su código podrá consultar la información visualizando las propiedades del archivo compilado en el explorador de Windows.

Opción Icono y Manifiesto

Esta opción permite tener acceso a las opciones para configurar el icono y el manifiesto de la aplicación.

Icono

Esta propiedad configura el icono asociado al archivo compilado del proyecto cuando se visualiza en el explorador de Windows o cuando la aplicación aparece sobre la barra de tareas de Windows.

Manifiesto

Se utiliza el manifiesto durante la ejecución de la aplicación bajo Windows Vista para determinar el nivel de ejecución requerido para la aplicación (UAC: *User Account Control*). Hay tres opciones disponibles:

- Incrustar manifiesto con configuración predeterminada: con esta opción, se genera automáticamente un archivo manifiesto durante la compilación. Este archivo determina que la aplicación debe ejecutarse con la identidad actual del usuario y no requiere aumento de privilegios (asInvoker).
- Crear una aplicación sin archivo manifiesto: esta opción activa la virtualización durante la ejecución de la aplicación bajo Windows Vista.
- Facilitar su propio archivo manifiesto cuyo nombre debe aparecer en este caso como tercera opción.

Archivo de recursos

Debe seleccionar esta opción cuando indica un archivo de recursos personalizado para el proyecto. La selección de esta opción desactiva las opciones Icono y Manifiesto.

b. Generar

Se utiliza esta página de propiedades para configurar las diferentes opciones de generación.

Primero hay que elegir a qué configuración (Debug o Release) y a qué plataforma se van a aplicar los parámetros.

Símbolos de compilación condicional

Esta zona de grabación de datos se utiliza para definir constantes que se chequean durante la compilación. Por ejemplo, puede definir la constante DEMO y utilizarla como en el ejemplo siguiente para modificar el título de una ventana.

```
#if (DEMO)
Text="version de demo";
#else
Text="version completa";
#endif
```

Si se deben definir varias constantes, hay que separarlas con un espacio.

Definir constante DEBUG

Define automáticamente una constante de compilación condicional llamada DEBUG.

Definir constante TRACE

Define automáticamente una constante de compilación condicional llamada TRACE.

Destino de la plataforma

Esta opción especifica el procesador para el cual se debe generar el código. Hay cuatro opciones disponibles:

x86	para los procesadores de 32 bits compatibles con Intel.
Itanium	para los procesadores Intel Itanium de 64 bits.
x64	para los otros procesadores de 64 bits.
Any CPU	CPU para todos los procesadores.
Preferencia de 32 bits	Esta opción indica que la aplicación siempre se ejecuta como una aplicación de 32 bits incluso sobre un sistema de 64 bits. Sólo está disponible si se selecciona la opción Any CPU.

Permitir código no seguro

Autoriza la compilación del código utilizando la palabra clave unsafe. Se utiliza la palabra clave cuando el código debe manejar directamente punteros.

Optimizar código

Activa o desactiva las optimizaciones efectuadas por el compilador para generar código más eficiente.

Nivel de advertencia

Durante su trabajo, el compilador puede encontrarse situaciones que no le parecen normales. En este caso genera una advertencia. Esta opción permite configurar los tipos de advertencias generadas.

0: Desactiva la emisión de todos los mensajes de advertencia.

1: Visualiza los mensajes de advertencia grave.

2: Visualiza las advertencias de nivel 1, así como algunas advertencias menos graves.

3: Visualiza las advertencias de nivel 2, así como algunas advertencias menos graves, como por ejemplo para señalar expresiones que siempre toman el valor true o false.

4: Visualiza todas las advertencias de nivel 3 más las advertencias de información.

Suprimir advertencias

Esta opción permite la generación de algunas advertencias para el compilador. Las advertencias deben ser indicadas por su número, separándolas con comas o con punto y coma.

Tratar advertencias como errores

Determina cuáles son las advertencias del compilador que serán tratadas como errores y que bloquearán la compilación. Se proponen los valores siguientes:

Ninguno	No considera ninguna advertencia como error.
Advertencias específicas	Considera las advertencias específicas como errores. Como para la sección Suprimir las advertencias, se deben separar los números de advertencias con una coma o un punto y coma.
Todo	Tratar todas las advertencias como errores.

Ruta de acceso de los resultados

Esta opción indica el directorio donde se copiarán los archivos generados por el compilador.

Archivo de documentación XML

Indica el nombre del archivo en el cual se copiará la documentación generada a partir de los comentarios ubicados en el código.

Registrar para interoperabilidad COM

Esta opción indica al compilador que debe generar código compatible con el entorno COM. Esta opción sólo está disponible para los proyectos de tipo librería de clases.

Generar ensamblados de serialización

Pide al compilador que optimice el código para las operaciones de serialización y deserialización de las instancias de las clases del proyecto.

c. Eventos de compilación

Este cuadro de diálogo permite configurar un comando que se puede lanzar automáticamente antes o después de la compilación del proyecto.

Configuración: N/D Plataforma: N/D

Línea de comandos del evento anterior a la compilación:

Edición anterior a la compilación...

Línea de comandos del evento posterior a la compilación:

Edición posterior a la compilación...

Ejecutar el evento posterior a la compilación:

Si la compilación es correcta

Cada uno de los comandos se puede introducir en la zona de texto correspondiente. Los botones **Edición anterior a la compilación** y **Edición posterior a la compilación** abren una ventana de edición que facilita la introducción del comando.

Línea de comandos del evento anterior a la compilación

xcopy "\$(ProjectPath)*.*" c:\copiasseguridad /s

Macro	Valor
TargetName	WindowsFormsApplicati...
TargetPath	c:\users\vs2012\docume...
ProjectPath	c:\users\vs2012\docume...
ProjectFileName	WindowsFormsApplicati...
TargetExt	.exe

<< Macros

Insertar Aceptar Cancelar

También propone este cuadro de diálogo una lista de macros que permiten la recuperación y el uso por parte de su comando de ciertos parámetros del proyecto. El ejemplo presentado en la figura anterior efectúa una copia completa del directorio de la aplicación en el directorio C:\guardar, antes de cada generación.

La ejecución del comando después de la generación puede ser condicional y ocurrir sólo en caso de generación exitosa o si la generación actualizó la salida del proyecto.

- Si un comando debe ejecutar un archivo .bat después de la generación, la llamada a éste debe venir precedida de la palabra clave call.

d. Propiedades de depuración

Las propiedades presentes en esta página determinan el comportamiento del proyecto durante su depuración.

Configuración: (Debug) activa Plataforma: (Any CPU) activa

Acción de inicio

☒ Proyecto de inicio

☐ Programa externo de inicio: ...

☐ Iniciar explorador con la dirección URL:

Opciones de inicio

Argumentos de la línea de comandos:

Directorio de trabajo: ...

☐ Usar máquina remota

Habilitar depuradores

☐ Habilitar depuración de código nativo

☐ Habilitar depuración de SQL Server

☒ Habilitar el proceso de hospedaje de Visual Studio

Acción de inicio

Esta propiedad determina el comportamiento del proyecto durante el inicio de la depuración. Hay tres opciones posibles:

- **Proyecto de inicio** indica que el propio proyecto debe ser ejecutado. Sólo se puede utilizar para los proyectos de aplicación de Windows o los proyectos de aplicación de consola.
- **Programa externo de inicio** permite provocar la ejecución de una aplicación externa que se va a encargar de realizar llamadas al código de nuestro proyecto. Se utiliza esta opción para la depuración de librería de clases.
- **Iniciar explorador con la dirección URL** es idéntica a la opción anterior, excepto que la aplicación ejecutada es una aplicación Web.

Opciones de inicio

Argumentos de la línea de comandos precisa los argumentos pasados a la aplicación durante su ejecución por Visual Studio. El código puede utilizar estos argumentos para determinar la acción que hay que acometer: por ejemplo, iniciar la aplicación en modo mantenimiento.

Directorio de trabajo permite especificar el directorio activo durante la ejecución de la aplicación.

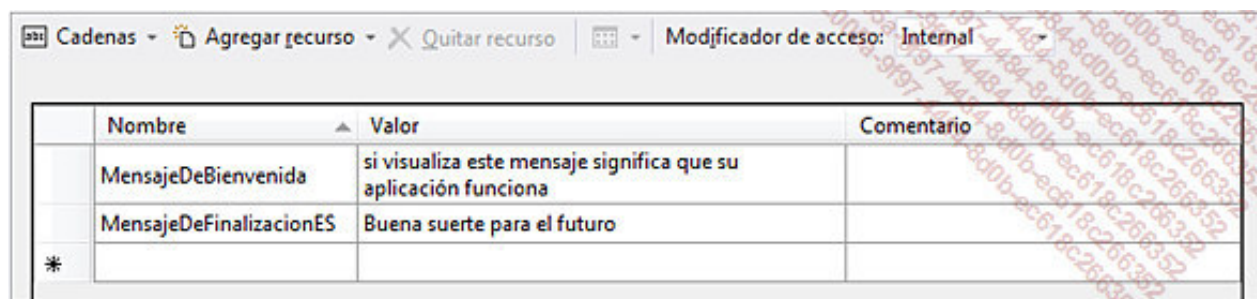
Usar máquina remota autoriza la depuración de una aplicación que se ejecuta en otra máquina. En este caso, se debe indicar el nombre de la máquina remota en la cual se va a ejecutar el código.

Habilitar depuradores

Estas opciones determinan los diferentes tipos de depuradores activos, en complemento del depurador de código gestionado de Visual Studio.

e. Recursos

Se utilizan los recursos para externalizar ciertos elementos de una aplicación. Permiten realizar rápidamente modificaciones sencillas en una aplicación, sin tener que buscar entre miles de líneas de código. La utilización más clásica consiste en separar del código las constantes en forma de cadena de caracteres. También puede crear recursos de iconos, imágenes, archivo de texto o audio. Este cuadro de diálogo gestiona todos los recursos.



Para cada recurso, indique un nombre y un valor. Por supuesto, el nombre será utilizado en el código para poder recuperar el valor.

En función del tipo de recurso, tiene a su disposición un editor adaptado para modificar el recurso. Los recursos pueden ser relacionados o incorporados, en función de su tipo. Un recurso relacionado está almacenado en su propio archivo y el archivo **Resources.resx** contiene simplemente un vínculo hacia el archivo original. Un recurso incorporado está almacenado directamente en el archivo **Resources.resx** de la aplicación. En todos los casos, se compilarán los recursos en el ejecutable de la aplicación.

Veamos ahora cómo acceder a los recursos a partir del código de la aplicación. Todos los recursos son accesibles a través de la propiedad **Resources** del objeto **My**. El ejemplo siguiente utiliza:

- Un recurso de cadena de caracteres (MensajeBienvenidaEs).
- Un recurso de icono (IconApli).
- Un recurso de imagen bitmap (ImagenFondo).
- Un archivo de sonido (Música).

```
private void Form1_Load(object sender, EventArgs e)
{
    this.Icon=WindowsFormsApplication1.Properties.Resources.IconApli;
    this.BackgroundImage = WindowsFormsApplication1.Properties.Resources.Imagen
    Fondo;
```



```

new SoundPlayer(WindowsFormsApplication1.Properties.Resources.Música).Play
Looping();
    MessageBox.Show(WindowsFormsApplication1.Properties.Resources.Mensaje
BienvenidaEs);
}

```

f. Configuración de la aplicación

Se suelen utilizar los parámetros de aplicación para almacenar y cargar dinámicamente los parámetros de configuración de una aplicación, como por ejemplo las preferencias del usuario o los últimos archivos utilizados en la aplicación.

Primero se deben crear los parámetros en la página de propiedades siguiente.

La configuración de la aplicación le permite almacenar y recuperar la configuración de la propiedad y otra información para su aplicación dinámicamente. Por ejemplo, la aplicación puede guardar las preferencias de color del usuario y, a continuación, recuperarlas la próxima vez que se ejecute. [Aprender más acerca de la configuración de la aplicación...](#)

	Nombre	Tipo	Ámbito	Valor
	ColorFondo	System.Dra...	Usuario	<input type="color"/> White
	ColorTexto	System.Dra...	Usuario	<input type="color"/> Yellow
	UltimoUso	System.Dat...	Usuario	12/07/2012
	Titulo	string	Usuario	

Presione Entrar para comenzar una nueva línea.
Presione Ctrl+Entrar para aceptar el texto.

Para cada parámetro, debe proveer un nombre que se utilizará para manejar el parámetro en el código, así como un tipo para el parámetro.

También debe facilitar un ámbito para el parámetro. Hay dos opciones posibles:

Usuario

Se puede modificar el parámetro durante el funcionamiento de la aplicación.

Aplicación

El parámetro es de sólo lectura durante la ejecución y sólo puede modificarse mediante este cuadro de diálogo.

La última cosa por hacer consiste en especificar un valor para el parámetro.

Vamos a estudiar ahora cómo manejar los parámetros en el código. Debemos realizar tres operaciones.

- Al iniciar la aplicación, debemos cargar los parámetros. El acceso a los parámetros se hace a través de la propiedad Default del objeto Settings.

```
WindowsFormsApplication1.Properties.Settings.Default.Reload();
```

- Durante la ejecución de la aplicación, tenemos también acceso a los parámetros con esta propiedad Default del objeto Settings, a la cual añadimos el nombre del parámetro. Esto nos permite la lectura del valor del parámetro o la asignación de un valor al parámetro.

```
this.BackColor = WindowsFormsApplication1.Properties.Settings.Default.ColorFondo;  
WindowsFormsApplication1.Properties.Settings.Default.UltimaUtilizacion =  
Date-Time.UtcNow;
```

- Al cerrar la aplicación, debemos guardar los parámetros utilizando el método Save:

```
WindowsFormsApplication1.Properties.Settings.Default.Save();
```

Ruta de acceso de las referencias

Cuando referencia un ensamblado en su proyecto, Visual Studio empieza buscándolo directamente en el directorio del proyecto. Si no lo encuentra en este directorio, entonces buscará en el/los directorios que usted configuró en el cuadro de diálogo ruta de acceso de las referencias.

Las variables, constantes y enumeraciones

1. Las variables

Las variables le permitirán almacenar, durante la ejecución de su aplicación, diferentes valores útiles para el funcionamiento de su aplicación. Se debe declarar una variable obligatoriamente antes de su uso en el código. Mediante la declaración de una variable, usted fija sus características.

a. Nombre de las variables

Veamos las reglas que se deben respetar para nombrar las variables:

- El nombre de una variable empieza obligatoriamente con una letra.
- Puede estar formada por letras, cifras o por el carácter subrayado (_).
- Puede contener un máximo de 1.023 caracteres (por razones prácticas, es preferible limitarse a un tamaño razonable).
- Hay una distinción entre minúsculas y mayúsculas (la variable EdadDelCapitan es diferente de la variable edaddelcapitan).
- No se deben usar las palabras reservadas del lenguaje (a pesar de lo dicho, sí que es posible, pero en este caso el nombre de la variable debe ir precedido del carácter @. Por ejemplo, una variable nombrada `if` se utilizará en el código con esta forma `@if=56;`).

b. Tipo de variables

Al especificar un tipo para una variable, indicamos qué datos vamos a poder almacenar en esta variable.

Hay dos categorías de tipos de variables disponibles:

- Los tipos valor: la variable contiene realmente los datos.
- Los tipos referencia: la variable contiene la dirección de la memoria donde se encuentran los datos.

Los diferentes tipos de variables disponibles están definidos a nivel del propio Framework. Usted también puede utilizar los alias definidos a nivel de Visual C#, quizá más explícitos. Así, el tipo **System.Int32** definido a nivel del Framework se puede sustituir con el tipo **int** en Visual C#.

Se pueden clasificar los diferentes tipos en seis categorías.

Los tipos numéricos enteros

Tipos enteros con signo			
sbyte	-128	127	8 bits
short	-32768	32767	16 bits
int	-2147483648	2147483647	32 bits
long	-9223372036854775808	9223372036854775807	64 bits

Tipos enteros sin signo			
byte	0	255	8 bits
ushort	0	65535	16 bits
uint	0	4294967295	32 bits
ulong	0	18446744073709551615	64 bits

Cuando elige un tipo para sus variables enteras, debe tener en cuenta los valores mínimo y máximo que piensa almacenar en la variable con el fin de optimizar la memoria utilizada por la variable. En efecto, es inútil utilizar un tipo Long para una variable cuyo valor no supera 50; un tipo byte es suficiente en este caso.



El ahorro de memoria parece irrisorio para una sola variable, pero se vuelve interesante en caso de uso de tablas de gran dimensión.

En caso contrario, si desea optimizar la velocidad de ejecución de su código, es preferible utilizar el tipo int.

Los tipos decimales

float	-3.40282347E+38	3.40282347E+38	4 bytes
double	-1.7976931348623157E+308	1.7976931348623157E+308	8 bytes
decimal	-79228162514264337593543950335	79228162514264337593543950335	16 bytes

Se debe tener en cuenta las mismas consideraciones de optimización que para las variables enteras. En este caso, se obtiene una rapidez de ejecución máxima con el tipo double. Se recomienda el tipo decimal para los cálculos financieros, en los cuales los errores de redondeo están prohibidos, pero en detrimento de la rapidez de ejecución del código.

Los tipos caracteres

El tipo char (carácter) se utiliza para almacenar un carácter único. Una variable de tipo char utiliza dos bytes para almacenar el código Unicode del carácter. En un juego de caracteres Unicode, los primeros 128 caracteres son idénticos al juego de caracteres ASCII, los caracteres siguientes hasta 255 corresponden a los caracteres especiales del alfabeto latino (por ejemplo, los caracteres acentuados); el resto se utiliza para símbolos o para los caracteres de otros alfabetos.

La asignación de un valor a una variable de tipo char se debe efectuar enmarcando el valor con caracteres ". Algunos caracteres tienen un significado especial para el lenguaje y por esa razón se deben utilizar precedidos de una secuencia de escape. Esta secuencia siempre empieza con el carácter \. La tabla siguiente resume las diferentes secuencias disponibles.

Secuencia de escape	Carácter
\'	Comilla simple
\"	Comilla doble

\\	Barra inversa
\0	Carácter nulo
\a	Alerta
\b	Backspace
\f	Salto de página
\n	Salto de línea
\r	Retorno de carro
\t	Tabulación horizontal
\v	Tabulación vertical

También se pueden utilizar estas secuencias de escape en una cadena de caracteres. Cada una de ellas representa un carácter único.

Para poder almacenar cadenas de caracteres, conviene utilizar el tipo string, que representa una serie de cero a 2.147.483.648 caracteres. Las cadenas de caracteres son invariables ya que, durante la asignación de un valor a una cadena de carácter, algo de espacio se reserva en memoria para el almacenamiento. Si luego esta variable recibe un nuevo valor, el sistema le asigna una nueva ubicación en memoria. Afortunadamente, este mecanismo es transparente para nosotros y la variable siempre hará automáticamente referencia al valor que le está asignado. Con este mecanismo, las cadenas de caracteres pueden tener un tamaño variable. El espacio ocupado en memoria se ajusta automáticamente a la longitud de la cadena de caracteres.

Para asignar una cadena de caracteres a una variable, el contenido de la cadena se debe introducir entre " ", como en el ejemplo siguiente:

Ejemplo

```
NombreDelCapitan = "Garfio";
```

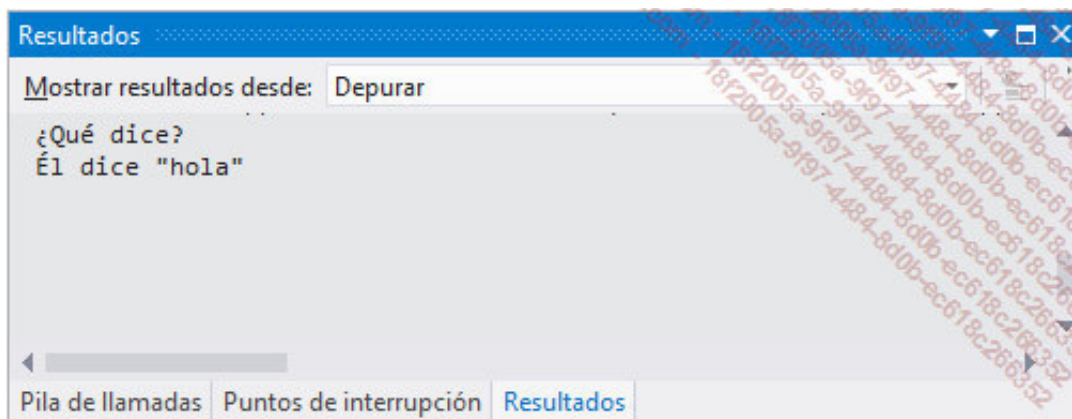
Si algunos caracteres especiales deben aparecer en una cadena, se deben especificar con una secuencia de escape. Sin embargo, existe otra posibilidad que permite a veces hacer el código más legible. Esta solución consiste en hacer que la cadena de caracteres vaya precedida del símbolo @. El compilador considera entonces que se deben utilizar todos los caracteres contenidos en las comillas dobles tal cual, incluidos los posibles retornos de carro. La única limitación es relativa al carácter " que, si debe formar parte de la cadena, se debe doblar.

Las dos declaraciones de cadenas siguientes son idénticas:

```
cadena = "¿Qué dice?\rÉl dice \"hola\"";
```

```
cadena = @"¿Qué dice?  
Él dice ""hola""";
```

Cuando se visualizan en la consola, dan el resultado siguiente:



- Hay numerosas funciones de la clase string que permiten el manejo de las cadenas de caracteres y que serán detalladas más adelante en este capítulo.

El tipo bool

El tipo bool permite utilizar una variable que puede tener dos estados verdadero/falso, si/no, on/off.

La asignación se hace directamente con los valores true o false, como en el ejemplo siguiente:

```
Disponible=true;  
Modificable=false;
```

El tipo Object

Quizá sea el tipo más universal de Visual C#. En una variable de tipo Object, usted puede almacenar cualquier cosa. En realidad, este tipo de variable no almacena nada. La variable no contendrá el propio valor, sino la dirección, en la memoria de la máquina, donde se podrá encontrar el valor de la variable. Tranquilícese, todo este mecanismo es transparente, y nunca tendrá que manejar las direcciones de memoria directamente.

- Una variable de tipo Object podrá por lo tanto hacer referencia a cualquier otro de tipo de valor, incluidos tipos numéricos simples. Sin embargo, el código será menos rápido debido al uso de una referencia.

El tipo dynamic

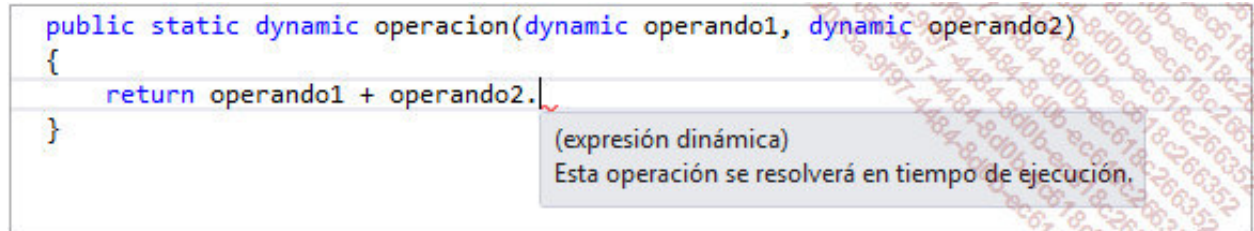
Desde su primera versión, el lenguaje C# es un lenguaje estáticamente tipado. Se debe declarar cada variable utilizada con un tipo definido. Esta exigencia permite al compilador comprobar que usted sólo realiza con esta variable operaciones compatibles con su tipo. Esto impone por supuesto conocer el tipo de variable en el momento de diseñar de la aplicación. Sin embargo, a veces ocurre que sólo se puede conocer el tipo de la variable en el momento de ejecutar la aplicación. En este caso, es posible utilizar la palabra reservada **dynamic** como tipo para la variable afectada. Para las variables declaradas con este tipo, el compilador no hace ninguna verificación de compatibilidad relativa a las operaciones ejecutadas con esta variable. Estas operaciones de verificación se efectúan sólo en el momento de ejecutar la aplicación. Si estas operaciones no son compatibles con el tipo de la variable, se lanza una excepción.

La función que mostramos más abajo espera dos parámetros cuyo verdadero tipo no se conoce

durante el diseño de la función. Por eso son declarados con **dynamic**. El tipo devuelto por la función, dependiendo del tipo de parámetros que se le pasan en el momento de la llamada, también es declarado con **dynamic**. Esta función utiliza el operador + en los dos parámetros que se le pasan.

```
public static dynamic operacion(dynamic operando1,dynamic operando2)
{
    return operando1 + operando2;
}
```

El tipo de los parámetros `operando1` y `operando2` es desconocido en tiempo de diseño y por ello Visual Studio es incapaz hacer la mínima propuesta en los diferentes métodos que se puedan utilizar en estas variables.

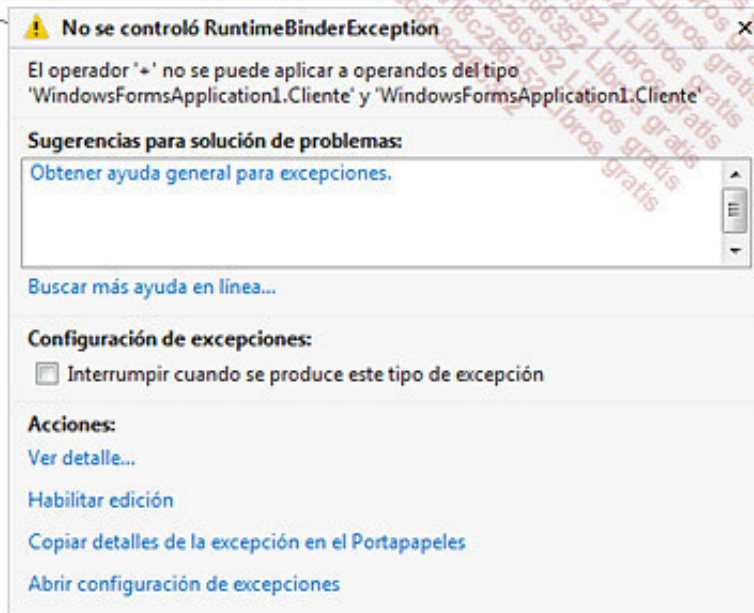


De la misma manera, acepta sin problema que la función sea utilizada en los diferentes ejemplos de la siguiente captura:

```
int i1;
int i2;
i1 = 2;
i2 = 4;
Console.WriteLine(operacion(i1,i2));
String s1;
String s2;
s1 = "2";
s2 = "4";
Console.WriteLine(operacion(s1, s2));
Cliente c1;
c1 = new Cliente();
Cliente c2;
c2 = new Cliente();
Console.WriteLine(operacion(c1, c2));
```

En el momento de la ejecución, las primeras dos llamadas de la función se realizan sin problema. La primera efectúa una suma de los dos enteros, la segunda efectúa una concatenación de las dos cadenas de caracteres. Por el contrario, la tercera llamada que utiliza instancias de clase `Cliente` lanza una excepción, ya que el operador + no es aplicable con este tipo de datos.

```
public static dynamic operacion(dynamic operando1, dynamic operando2)
{
    return operando1 + operando2;
}
```



Este ejemplo demuestra que hay que ser prudente con el uso del tipo **dynamic** y siempre prever la recuperación de la excepción que se puede producir en caso de utilización inadaptada del tipo real de datos.

Se utiliza principalmente esta funcionalidad para manejar elementos obtenidos desde un lenguaje dinámico (IronRuby o IronPython) o desde una API COM.

Los tipos Nullables

Ocurre a veces que en algunas circunstancias una variable no tenga un valor bien definido. Es, por ejemplo, el caso que se produce durante la recuperación de información procedente de una base de datos cuando para un campo no hay valor en la fila. ¿Cómo representar esta situación con variables en Visual C#? Una solución consiste en utilizar un valor que no tiene ningún significado para la aplicación.

Por ejemplo, para una variable numérica que representa un código postal en la aplicación, se puede considerar asignar a esta variable un valor negativo en el caso en el cual el código no está indicado. El resto del código debe tener en cuenta por supuesto esta convención. Para cierto tipo de datos, esta solución no se puede considerar. Tomemos el caso de una variable de tipo bool para la cual sólo hay dos valores admitidos, «true» o «false», ¿cómo representar el hecho de que el contenido de la variable sea nulo?

Para resolver este problema, Visual C# propone los tipos Nullables. Permiten a las variables de tipo valor no contener ninguna información. Para activar esta funcionalidad en una variable, sólo hay que utilizar el carácter '?' después del tipo de la variable, como en el ejemplo siguiente.

```
int? CodigoPostal
```

En cambio, hay que ser prudente durante la utilización de una variable de este tipo y verificar antes de utilizarla si contiene realmente un valor. Para ello, hay que probar la propiedad `HasValue` de la variable para determinar si contiene realmente un valor. Si es el caso, este valor está disponible gracias a la propiedad `Value` de la variable. Esta propiedad es de sólo lectura, ya que la asignación de un valor se hace directamente en la variable.

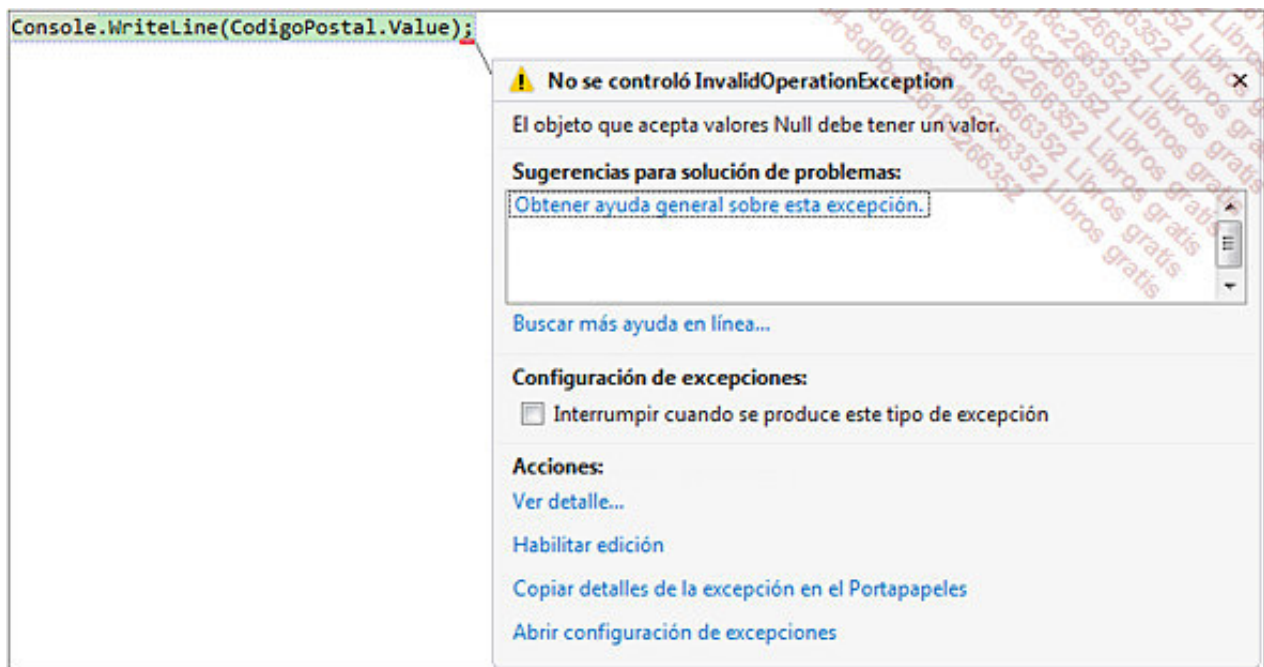
```

CodigoPostal = 17000;
if (CodigoPostal.HasValue)
{
    Console.WriteLine(CodigoPostal.Value);
}
else
{
    Console.WriteLine("Código postal vacío");
}

```

Es indispensable probar la propiedad `HasValue` antes de la utilización de la propiedad `value`, ya que si la variable no contiene ningún valor, se activa una excepción.

Es el caso de el ejemplo siguiente, ya que una variable nullable, frente a una variable normal, no contiene ningún valor por defecto.



Una variable que contiene un valor puede volver al estado «nulo» si se le asigna el valor `null`.

El uso de variables de tipo boolean nullable con los operadores lógicos `&` y `|` puede ser a veces problemático. A continuación, se muestra la tabla de la verdad de estos dos operadores con variables nullables.

B1	B2	B1 & B2	B1 B2
null	null	null	null
null	true	null	true
null	false	false	null
true	null	null	true
true	true	true	true
true	false	false	true
false	null	false	null
false	true	false	true

false	false	false	false
-------	-------	-------	-------

c. Conversiones de tipos

Las conversiones de tipos consisten en transformar una variable de un tipo en otro tipo. Las conversiones se pueden efectuar hacia un tipo superior o inferior.

Si se convierte hacia un tipo inferior, hay riesgo de pérdida de información. Por ejemplo, la conversión de un tipo `double` hacia un tipo `long` hará perder la parte decimal del valor.

Para limitar este riesgo, el compilador vigila las conversiones realizadas en su código y activa un error cuando se encuentra con tal situación.

```
double x;  
long y;  
x = 21.234323;  
y = x;  
(variable local) double x  
Error:  
No se puede convertir implícitamente el tipo 'double' en 'long'. Ya existe una conversión explícita (compruebe si le falta una conversión)
```

Este tipo de conversión no está totalmente prohibido. Sólo tiene que avisar al compilador de su intención utilizando una operación de conversión explícita. En realidad, no hay un operador específico para la conversión explícita; es el tipo de datos hacia el cual desea hacer la conversión lo que se debe utilizar como operador. Sólo basta con prefijar la variable que desea convertir con el tipo, nombre del tipo de datos deseado, teniendo cuidado de colocarlo entre paréntesis. Por lo tanto, nuestro ejemplo anterior cambia a:

```
double x;  
long y;  
x = 21.234323;  
y = (long) x;  
Console.WriteLine("valor de x:" + x);  
Console.WriteLine("valor de y:" + y);
```

➤ El uso de esta sintaxis no provoca errores de compilación si usted intenta una conversión restrictiva, ya que el compilador considera entonces que usted la realiza con pleno conocimiento de causa.

Las conversiones desde cadenas de caracteres y hacia cadenas de caracteres son más específicas.

Conversión hacia una cadena de caracteres

La función `format` de la clase `string` permite elegir la forma del resultado de la conversión de un valor cualquiera en cadena de caracteres. Esta función espera como primer parámetro una cadena de caracteres que representa el formato en el cual desea obtener el resultado. El segundo parámetro corresponde al valor que se debe convertir.

Algunos formatos estándares están predefinidos, pero también es posible personalizar el resultado de la función `format`. Se presentan los parámetros de esta función a continuación.

Formateo de valores numéricos

Currency

Formato monetario tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:c}",12.35);`

Resultado: 12,35 €

Fixed

Utiliza al menos un carácter para la parte entera y al menos dos caracteres para la parte decimal de un nombre. El separador decimal, tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:f}",0.2);`

Resultado: 0,20

Percent

Multiplica el valor indicado por cien y añade el símbolo « % » después.

Ejemplo: `String.format("{0:p}",0.2);`

Resultado: 20,00%

Standard

Formato numérico tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:n}",245813.5862);`

Resultado: 245.813,59

Scientific

Notación científica.

Ejemplo: `String.format("{0:c}",245813.58);`

Resultado: 2,458136e+005

Hexadecimal

Formato hexadecimal. Utilizable únicamente para los valores enteros.

Ejemplo: `String.format("{0:x}",245813);`

Resultado: 3C035

Cadena de formateo personalizada para valores numéricos

0

Reserva una ubicación para un carácter numérico. Los ceros no significativos se visualizan.

Ejemplo: `String.format("{0:000000000000.0000}", 245813.12);`

Resultado: 00000245813,1200

#

Reserva una ubicación para un carácter numérico. Los ceros no significativos no se visualizan.

Ejemplo: `String.format("{0:#####.###}", 245813.12);`

Resultado: 245813,12

.

Reserva una ubicación para el separador decimal. El carácter realmente utilizado en el resultado depende de la configuración de las opciones regionales y lingüísticas del panel de configuración del sistema.

,

Reserva una ubicación para el separador de millares. El carácter realmente utilizado en el resultado depende de la configuración de las opciones regionales y lingüísticas del panel de configuración del sistema.

Formatos de fecha y hora

G

Formato Fecha corta y formato Hora tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:G}", DateTime.now);`

Resultado 25/03/2008 11:10:42

D

Formato Fecha larga tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:D}", DateTime.now);`

Resultado martes 25 marzo del 2008

d

Formato Fecha corta tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:d}", DateTime.now);`

Resultado 25/03/2008

T

Formato Hora tal como está definido en las opciones regionales y lingüísticas del panel de configuración del sistema.

Ejemplo: `String.format("{0:T}", DateTime.now);`

Resultado 11:45:30

s

Formato «ordenable».

Ejemplo: `String.format("{0:s}", DateTime.now);`

Resultado 2008-03-25T11:47:30

Cadena de formateo personalizado para valores de fecha y hora

d	Día del mes sin cero no significativo
dd	Día del mes con cero no significativo
ddd	Nombre del día de la semana abreviado
dddd	Nombre del día de la semana completo
M	Número del mes sin cero no significativo
MM	Número del mes con cero no significativo
MMM	Nombre del mes abreviado
MMMM	Nombre del mes completo
h	Hora sin cero no significativo (formato 12H)
hh	Hora con cero no significativo (formato 12H)
H	Hora sin cero no significativo (formato 24H)
HH	Hora con cero no significativo (formato 24H)

m	Minuto sin cero no significativo
mm	Minuto con cero no significativo
s	Segundo sin cero no significativo
ss	Segundo con cero no significativo
y	Año en una cifra. Si es el único carácter de la cadena de formateo, en este caso se debe utilizar %y
yy	Año en dos cifras
yyyy	Año en cuatro cifras
zzz	Desfase respecto al tiempo universal (GMT).

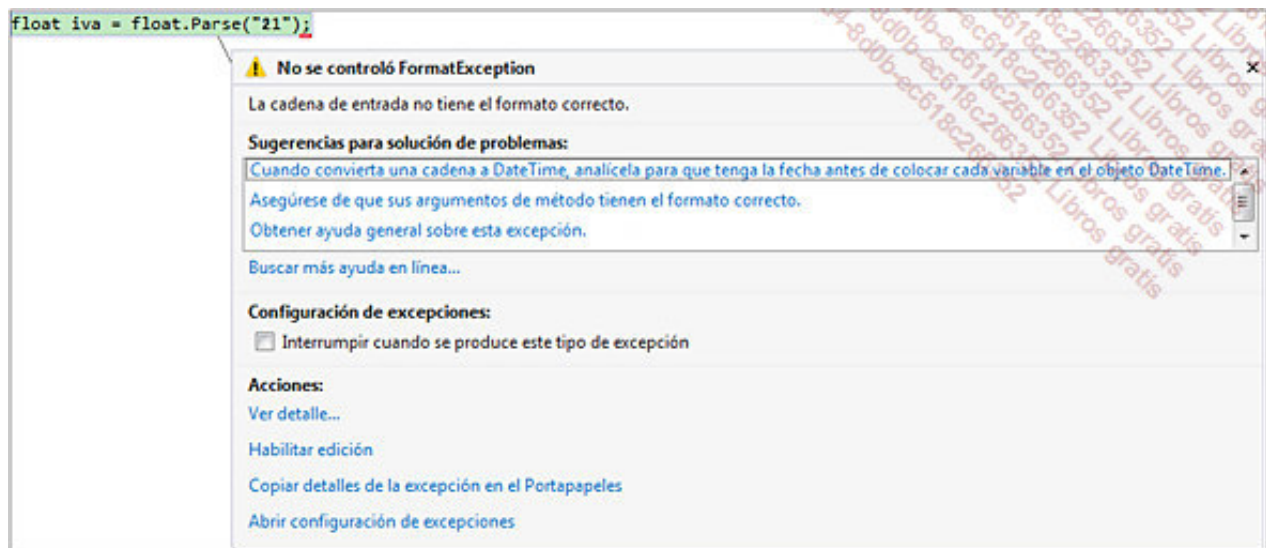
Conversión desde una cadena de caracteres

La conversión inversa, desde una cadena de caracteres hacia un tipo numérico, se hace con la función Parse. Esta función está disponible en las principales clases que representan los diferentes tipos numéricos. Por lo tanto, hay que utilizar el método Parse de la clase correspondiente al tipo de datos que deseamos obtener.

El ejemplo siguiente convierte una cadena de caracteres a tipo float.

```
float iva=float.Parse("21");
```

Durante la llamada, debe estar seguro de que la conversión se podrá efectuar sin problema. En caso contrario, se lanzará una excepción. Será por ejemplo el caso en la expresión siguiente, ya que el separador decimal no corresponde al configurado en el puesto de trabajo.



Por lo tanto, se recomienda gestionar las excepciones durante la ejecución de la función Parse.

Una alternativa más rápida consiste en utilizar la función TryParse. Esta función espera como primer parámetro la cadena de caracteres a partir de la cual desea efectuar la conversión. El segundo parámetro corresponde a la variable en la cual estará disponible el resultado de la conversión. A diferencia de la función Parse, esta función no genera excepción si la conversión fracasa: la función devuelve simplemente un valor false y la variable que debe contener el resultado se inicializa a cero. Si la conversión se efectúa correctamente, la función devuelve un valor true y la variable se inicializa con el resultado de la conversión.

```
if (float.TryParse("21", out iva))
{
    Console.WriteLine("Conversión OK");
}
else
{
    Console.WriteLine("Problema durante la conversión");
}
```

d. Declaración de las variables

El compilador considera que toda variable que aparece en una aplicación debe haber sido declarada.

La sintaxis general de declaración de una variable es la siguiente:

```
Tipo de la variable nombreVariable[=valor inicial][,nombreVariable2]
```

Los parámetros entre corchetes son opcionales.

En caso de que se omita el valor inicial, la variable será inicializada a cero si corresponde a un tipo **numérico**; a una cadena de carácter vacío si es del tipo **String**; al valor null si es del tipo **Object**, y a false si es del tipo **bool**.

Estas reglas no se aplican a las variables declaradas en el interior de una función que deben ser inicializadas antes de poder utilizarse. Esta inicialización puede ocurrir en el momento de la declaración o con posterioridad, pero obligatoriamente antes de que una instrucción utilice el contenido de la variable.



Si se especifican varios nombres, las variables correspondientes serán todas del tipo indicado.

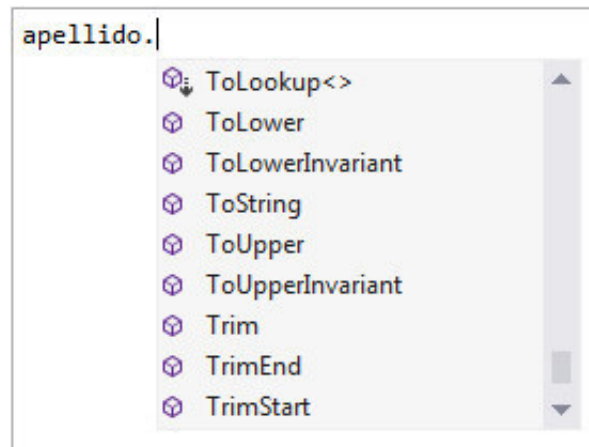
e. Inferencia de tipo

Vimos en el párrafo anterior que es obligatorio siempre declarar las variables antes de su utilización. Sin embargo, en algunos casos, se puede considerar dejar que el compilador realice una parte del trabajo. Gracias a la inferencia de tipo, el compilador puede determinar el tipo que se ha utilizar para una variable local. Para ello, se basa en el tipo de la expresión utilizada para inicializar la variable. El nombre de la variable debe venir precedido en este caso de la palabra reservada **var**. En el ejemplo siguiente la variable se considera como una cadena de caracteres.

```
var apellido = "García";
```

Para asegurarse de que esta variable se considera realmente como una cadena de caracteres, basta

con pedir a IntelliSense lo que nos propone para utilizar esta variable.



En efecto, disponemos de los métodos y propiedades del tipo String.

- La inferencia de tipo no es equivalente a la utilización del tipo de datos **dynamic**. Con la inferencia, se pide al compilador que adivine el tipo de la variable, y por lo tanto en el momento de la ejecución la variable dispone de un tipo. Con la utilización de la palabra reservada **dynamic**, el descubrimiento del tipo de la variable se hace en el momento de la ejecución.

Para que la inferencia de tipo funcione correctamente, es imperativo respetar algunas reglas:

- La inferencia sólo funciona para las variables locales, es decir, las declaradas en una función.
- La inicialización debe hacerse en la misma línea de código que la declaración.

f. Ámbito de las variables

El ámbito de una variable es la porción de código en la cual se puede trabajar con dicha variable. Depende de la ubicación

Los operadores

Los operadores son palabras reservadas del lenguaje que permiten la ejecución de operaciones en el contenido de ciertos elementos, en general variables, constantes, valores literales o devoluciones de funciones. La combinación de uno o varios operadores y elementos en los cuales los operadores van a apoyarse se llama una expresión. Estas expresiones se valoran en el momento de su ejecución, en función de los operadores y valores que son asociados.

Los operadores se pueden repartir en seis categorías.

1. Los operadores de asignación

El único operador disponible en esta categoría es el operador `=`. Permite asignar un valor a una variable. Se usa siempre el mismo operador, sea cual sea el tipo de variable (numérico, cadena de caracteres...).

2. Los operadores aritméticos

Los operadores aritméticos permiten efectuar cálculos en el contenido de las variables:

Operador	Operación realizada	Ejemplo	Resultado
+	Suma	6+4	10
-	Sustracción	12-6	6
*	Multiplicación	3*4	12
/	División	25/3	8.3333333333
%	Módulo (resto de la división entera)	25 % 3	1

3. Los operadores binarios

Estos operadores efectúan operaciones sobre enteros únicamente (Byte, Short, Integer, Long). Trabajan a nivel del bit en las variables que manejan.

Operador	Operación realizada	Ejemplo	Resultado
&	Y Binario	45 & 255	45
	O Binario	99 46	111
^	O exclusivo	99 ^ 46	77
~	Negación	~ 23	-24

4. Los operadores de comparación

Los operadores de comparación se utilizan en las estructuras de control de una aplicación (if, do loop...). Devuelven un valor de tipo boolean en función del resultado de la comparación efectuada. Luego este valor será utilizado por la estructura de control.

Operador	Operación realizada	Ejemplo	Resultado
==	Igualdad	2 == 5	False

!=	Desigualdad	2 != 5	True
<	Inferior	2 < 5	True
>	Superior	2 > 5	False
<=	Inferior o igual	2 <= 5	True
>=	Superior o igual	2 >= 5	False
is	Comparación del tipo de la variable con el tipo dado	O1 is Cliente	True si la variable O1 referencia un objeto creado a partir del tipo Cliente

5. Operador de concatenación

El operador se utiliza para la concatenación de cadenas de caracteres. Es el mismo operador que se utiliza para la suma. Sin embargo, no hay riesgo de confusión, ya que Visual C# no hace conversión implícita de las cadenas de caracteres en numérico. Determina por lo tanto que, si uno de los dos operandos es una cadena de caracteres, se debe ejecutar una concatenación, incluso si una de las cadenas representa un valor numérico.

El código siguiente

```
string cadena = "123";
Console.WriteLine(cadena + 456);
```

visualiza

```
123456
```

El inconveniente del operador + es que no resulta muy rápido para la concatenación. Si dispone de numerosas concatenaciones para ejecutar en una cadena, es preferible utilizar la clase `StringBuilder`.

Ejemplo

```
long duracion;
string liebre;
string tortuga="";
DateTime principio, fin;
principio = DateTime.Now;
for (int i = 0; i <= 100000; i++)
{
    tortuga = tortuga + " " + i;
}
fin = DateTime.Now;
duracion = new TimeSpan(fin.Ticks - principio.Ticks).Seconds;
Console.WriteLine("duración para la tortuga: " + duracion + "s");
principio = DateTime.Now;
StringBuilder sb = new StringBuilder();
for (int i = 0; i <= 100000; i++)
{
    sb.Append(" ");
    sb.Append(i);
}
liebre = sb.ToString();
in = DateTime.Now;
duracion = new TimeSpan(fin.Ticks - principio.Ticks).Seconds;
Console.WriteLine("duración para la liebre: " + duracion + "s");
if (liebre.Equals(tortuga))
```



```
{
    Console.WriteLine("las dos cadenas son idénticas");
}
```

Resultado de la carrera:

```
duración para la tortuga: 21 segundos
duración para la liebre: 0 segundos
las dos cadenas son idénticas.
```

¡Este resultado no necesita comentario!

6. Los operadores lógicos

Los operadores lógicos permiten combinar las expresiones en estructuras condicionales o de bucle.

Operador	Operación	Ejemplo	Resultado
&	Y lógico	If (test1) & (test2)	verdadero si test1 y test2 es verdadero
	O lógico	If (test1) (test2)	verdadero si test1 o test2 es verdadero
^	O exclusivo	If (test1) ^ (test2)	verdadero si test1 o test2 es verdadero, pero no si los dos son verdaderos simultáneamente
!	Negación	If Not test	Invierte el resultado del test
&&	Y lógico	If (test1) && (test2)	Idem «y lógico» pero test2 sólo será evaluado si test1 es verdadero
	O lógico	If (test1) (test2)	Idem «o lógico» pero test2 sólo será evaluado si test1 es falso

Conviene ser prudente con los operadores && y || ya que la expresión que prueba en segundo término (test2 en nuestro caso) puede no llegar a ser ejecutada. Si esta segunda expresión modifica una variable, ésta se modificará sólo en los siguientes casos:

- primer test verdadero en el caso del &&,
- primer test falso en el caso del ||.

7. Orden de evaluación de los operadores

Cuando varios operadores se combinan en una expresión, son valorados en un orden muy preciso. En primer lugar se resuelven las operaciones aritméticas, luego las operaciones de comparación y entonces los operadores lógicos.

Los operadores aritméticos tienen también entre ellos un orden de evaluación en una expresión. El orden de evaluación es el siguiente:

- Negación (-)
- Multiplicación y división (*, /)

- Módulo (%)
- Suma y sustracción (+, -), concatenación de cadenas (+)

Si necesita un orden de evaluación diferente en su código, dé prioridad a las porciones que se han de evaluar primero colocándolas entre paréntesis, como en la siguiente expresión:

$X = (z * 4) + (y * (a + 2));$

 Usted puede utilizar tantos niveles de paréntesis como desee en una expresión. Es importante, sin embargo, que la expresión contenga tantos paréntesis cerrados como paréntesis abiertos; si no el compilador generará un error.

Las estructuras de control

Las estructuras de control permiten modificar el orden de ejecución de las instrucciones en su código. Hay dos tipos de estructuras disponibles:

- Las estructuras de decisión: orientarán la ejecución de su código en función de los valores que pueda tener una expresión de test.
- Las estructuras de bucle: harán ejecutar una porción de su código un cierto número de veces hasta que se cumpla una condición o mientras una condición sea cumplida.

1. Estructuras de decisión

Hay dos tipos de estructuras de decisión:

a. Estructura if

Cuatro sintaxis están a su disposición para la instrucción `if`.

```
if (condición) instrucción;
```

Si la condición es verdadera, entonces la instrucción se ejecuta; en este caso, «condición» debe ser una expresión que, una vez evaluada, debe devolver una booleana `true` o `false`. Con esta sintaxis, sólo la instrucción colocada después del `if`, se ejecutará si la condición es verdadera.

Para poder ejecutar varias instrucciones en función de una condición, la sintaxis que hay que utilizar es:

```
if (condición)
{Instrucción 1;
...
Instrucción n;}
```



En este caso, el grupo de instrucciones ubicado en las llaves será ejecutado si la condición es verdadera.

También puede especificar una o varias instrucciones que se ejecutarán si la condición es falsa.

```
if (condición)
{Instrucción 1;
...
Instrucción n;}
else
{Instrucción 1;
...
Instrucción n;}
```

b. Estructura switch

La estructura `switch` permite un funcionamiento equivalente, pero ofrece una mejor legibilidad del código. La sintaxis es la siguiente:

```

switch (variable)
{
    case valor1:
        Bloque de código 1
    case valor2:
        Bloque de código 2
    case valor3:
        Bloque de código 3
    default:
        Bloque de código 4
}

```

El valor de la variable se evalúa al principio de la estructura (por el `switch`). Luego el valor obtenido se compara con el valor especificado en el primer `case` (`valor1`).

Si los dos valores son iguales, entonces el bloque de código 1 se ejecuta.

Si no, el valor obtenido se compara con el valor del `case` siguiente; si hay correspondencia, el bloque de código se ejecuta y así sucesivamente hasta el último `case`.

Si ningún valor concordante se encuentra en los diferentes `case`, entonces el bloque de código especificado en el `default` se ejecuta. Cada uno de los bloques debe terminarse con la instrucción `break`.

El valor que hay que probar puede estar contenido en una variable, pero también puede ser el resultado de un cálculo. En este caso, el cálculo sólo se efectúa una vez al principio del `switch`. El tipo del valor probado puede ser numérico o cadena de caracteres. El tipo de la variable probada debe corresponder por supuesto al tipo de los valores en los diferentes `case`.

```

String respuesta;
Console.WriteLine("¿su respuesta?");
respuesta=Console.ReadLine();
switch (respuesta)
{
    case "si":
        Console.WriteLine("respuesta positiva");
        break;
    case "no":
        Console.WriteLine("respuesta negativa");
        break;
    default:
        Console.WriteLine("respuesta de gallego");
        break;
}

```

2. Las estructuras de bucle

Cuatro estructuras están a nuestra disposición:

```

while
do ... while
for
foreach

```

Todas tienen como objetivo ejecutar un bloque de código cierto número de veces en función de una condición.

a. Estructura while

```
while (condición) {Bloque de código}
```

Esta sintaxis permite ejecutar el bloque de código mientras la condición es verdadera. Se evalúa la condición incluso antes del primer paso en el bucle. Por lo tanto, el bloque de código podrá no ejecutarse nunca si la condición es falsa desde el principio. En caso de que la condición sea verdadera en el primer paso, el bloque de código se ejecuta. La condición se prueba otra vez y, si es verdadera, se vuelve a ejecutar el bloque de código. En el caso contrario, la próxima instrucción ejecutada será la que sigue al bloque de código. Sin embargo es posible prever una salida «prematura» del bucle utilizando la instrucción `break`. La ejecución se retoma, por lo tanto, en la línea que sigue inmediatamente al bloque de código.

b. Estructura do ... while

La estructura `do while` utiliza la sintaxis siguiente:

```
do
    {Bloque de código}
    while (condition);
```

Esta sintaxis nos permite garantizar que el bloque de código se ejecutará al menos una vez, ya que la condición se probará al final del bloque de código.

c. Estructura for

Cuando conoce el número de iteraciones que se deben realizar en un bucle, es preferible utilizar la estructura `for`. Para poder utilizar esta instrucción, debe declarar previamente una variable que actúe de contador.

Esta variable puede declararse en la estructura `for` o fuera. En este caso, se debe declarar antes de la estructura `for`.

La sintaxis general es la siguiente:

```
for(inicialización del contador ;condición ;instrucción de iteración)
{
    Bloque de código
}
```

La parte de inicialización se ejecuta una sola vez en el momento de la entrada en el bucle. La parte de condición se evalúa en el momento de entrar en el bucle, y luego en cada iteración. El resultado de la evaluación de la condición determina si el bloque de código se ejecuta. Para ello, hace falta que la condición sea evaluada como `true`. Después de la ejecución del bloque de código se ejecuta a su vez la instrucción de iteración. Luego se prueba de nuevo la condición, y así sucesivamente hasta que la condición se evalúa como `false`.

A continuación, dos bucles `for` en acción para visualizar una tabla de multiplicar.

```
int k;
for(k=1;k<10;k++)
{
    for (int l = 1; l < 10; l++)
    {
        Console.Write(k * l + "\t");
    }
    Console.WriteLine();
}
```

Obtenemos el siguiente resultado:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

La instrucción `break` puede utilizarse para provocar una salida prematura del bucle. La instrucción `continue` permite volver inmediatamente a la evaluación de la condición. Por supuesto, se deben ejecutar estas dos instrucciones de manera condicional; si no, las líneas de código ubicadas después no se ejecutarán nunca.

d. Estructura `foreach`

Otra sintaxis del bucle `for` permite ejecutar un bloque de código para cada elemento contenido en una matriz o en una colección. La sintaxis general de esta instrucción es la siguiente:

```
foreach (elemento in matriz)
{Bloque de código}
```

No hay noción de contador en esta estructura, ya que efectúa ella misma las iteraciones en todos los elementos presentes en la matriz o la colección.

La variable `elemento` sirve para extraer los elementos de la matriz o de la colección para que el bloque pueda manejarla. El tipo de la variable `elemento` debe ser compatible con el tipo de elementos almacenados en la matriz o la colección. Por el contrario, no debe preocuparse del número de elementos, ya que la instrucción `foreach` es capaz de gestionar ella misma el desplazamiento en la matriz o la colección. ¡A continuación se muestra un ejemplo para aclarar la situación!

Con un bucle clásico:

```
string[] matriz={"rojo","verde","azul","blanco"};
int contador;
for (contador = 0; contador < matriz.Length; contador++)
{
    Console.WriteLine(matriz[contador]);
}
```

Con el bucle `foreach`:

```
string[] matriz={"rojo","verde","azul","blanco"};
foreach (string s in matriz)
{
    Console.WriteLine(s);
}
```



La variable utilizada para recorrer la matriz debe ser declarada obligatoriamente en la instrucción `foreach`, y no fuera.

e. Otras estructuras

Hay otras dos estructuras disponibles destinadas más bien a simplificar el desarrollo:

Estructura using

Esta estructura se dedica a acoger un bloque de código utilizando un recurso externo, como por ejemplo un archivo. Esta estructura se encarga automáticamente de la liberación del recurso al final del bloque de código. El recurso se puede crear en la estructura o bien existir previamente y pasarse bajo el control de la estructura. Al final de la estructura, el recurso es liberado llamando al método `Dispose`.

Ejemplo

Subir

Condiciones generales de uso

Copyright - ©Editions ENI



Los procedimientos y funciones

En una aplicación Visual C#, se deben ubicar todas las instrucciones de manera obligatoria en un procedimiento o una función. Estos procedimientos o funciones nos permiten crear bloques de código que podrán ser llamados luego en otras porciones de su aplicación. La llamada al procedimiento o función se hará simplemente utilizando su identificador.

Para que sean más fácilmente reutilizables, tendrá la posibilidad de usar parámetros. Los valores de estos parámetros se especificarán en el momento de la llamada.

Durante el desarrollo, no dude en crear numerosos procedimientos y funciones. La división de su aplicación en muchos procedimientos y funciones facilitará la depuración (una decena de bloques de código de una quincena de líneas es más fácil de probar que un «tocho» de ciento cincuenta líneas). Incluso se pueden reutilizar ciertos procedimientos varias veces en su aplicación.

En Visual C#, hay cuatro tipos disponibles:

- Los procedimientos que ejecutan simplemente un bloque de código a petición, sin devolver un resultado.
- Las funciones que ejecutan un bloque de código y devuelven el resultado de su cálculo al código que las llamó.
- Los procedimientos de propiedades que permiten manejar las propiedades de los objetos creados en la aplicación.
- Los procedimientos de operador utilizados para modificar el funcionamiento de un operador cuando se aplica a una clase o una estructura.

Veamos ahora cómo declarar procedimientos y funciones.

1. Procedimiento

El código de un procedimiento se debe ubicar en un bloque de código delimitado por llaves. Para poder identificar este bloque de código, hay que hacerlo preceder de un nombre que se utilizará luego para llamar al procedimiento. Por defecto, Visual C# sólo sabe utilizar funciones, es decir, un bloque de código que ejecuta un código y devuelve un resultado. Para poder crear un procedimiento hay que indicar que nuestro bloque no devuelve ninguna información usando la palabra reservada `void`. La sintaxis general de declaración de un procedimiento es la siguiente:

```
void VisualizacionResultado()  
{  
    Console.WriteLine("¡¡funciona!!!");  
}
```

Los paréntesis después del nombre se utilizan para especificar los parámetros que se pasarán durante la llamada. Los paréntesis son obligatorios en la declaración incluso si no se requiere ningún parámetro para el procedimiento.

Hay muchas otras palabras reservadas utilizables en la declaración de un procedimiento que modifican las posibilidades de reutilización de este procedimiento. La mayoría de ellas están relacionadas con la programación orientada a objetos y se estudiarán en otro capítulo. Por el contrario, para modificar la visibilidad de su procedimiento, puede utilizar los operadores que ya hemos usado para la declaración de las variables (*private*, *public*, *internal*). Sin especificación, se considerará público un procedimiento.

Para pedir la ejecución de su procedimiento en el código, basta con especificar su nombre.

Incluso si su procedimiento no espera parámetros, la utilización de los paréntesis es obligatoria durante la llamada.

2. Función

Una función se declara según el mismo principio que un procedimiento. Sin embargo, como la función debe devolver un resultado al código invocante, usted debe indicar el tipo de dato que la función debe devolver. Este tipo de devolución debe preceder al nombre de la función (en sustitución de la palabra reservada `void` utilizada para los procedimientos). Se puede utilizar cualquier tipo de datos como devolución de una función. La sintaxis de declaración es la siguiente:

```
int calculo()
{
    Instrucción1
    ... Instrucción n
}
```

En el código de su función, debe especificar qué valor será devuelto por su función. Para ello, debe utilizar la instrucción `return` indicando el valor que quiere devolver por la función. La ejecución de la instrucción `return` provoca inmediatamente la salida de la función, incluso si no es la última instrucción.

Además, una función puede utilizarse en el código principal en lugar de una variable del mismo tipo que la devuelto por la función. También se puede utilizar como un procedimiento. En este último caso, el valor devuelto simplemente se ignorará.

3. Procedimientos de propiedades

Los procedimientos de propiedades van a permitirnos añadir una propiedad a una clase, un módulo o una estructura. Estos procedimientos se llaman «encapsuladores». Se utilizarán cuando se modifica (`Set`) o se recupera (`Get`) el valor de la propiedad que encapsulan. Su utilización parece similar al uso de una variable: se puede asignar un valor a una propiedad o leer el valor de una propiedad. Sin embargo, existen numerosas diferencias importantes entre las variables y las propiedades:

- Las variables necesitan una sola línea de código para la declaración.
- Las propiedades requieren un bloque de código para la declaración.
- El acceso a una variable se efectúa directamente.
- El acceso a una propiedad implica la ejecución de una porción de código.
- El contenido de una variable siempre se recupera tal cual.
- El contenido de una propiedad se puede modificar con el código durante el acceso a la propiedad.

La sintaxis general de creación de una propiedad es la siguiente:

```
public tipoDeLaPropiedad nombrePropiedad
{
    get
    {
        ...
    }
    set
```

```

    {
        ...
    }
}

```

En esta declaración

- `nombrePropiedad` corresponde al nombre con el cual se puede manejar la propiedad en el código.
- `TipoDeLaPropiedad` corresponde al tipo de datos asociado a la propiedad. Usted puede utilizar cualquier tipo de datos para una propiedad (los tipos básicos del lenguaje o un tipo personalizado, como por ejemplo una clase).

La declaración de una propiedad se parece mucho a la declaración de una función. La pequeña diferencia reside en los dos bloques de código `get` y `set` ubicados en el interior.

- El bloque `get` contiene el código ejecutado durante la lectura de la propiedad. Debe contener obligatoriamente una instrucción `return` para proveer el valor de la propiedad.
- El bloque `set` contiene el código ejecutado durante la asignación de un valor a la propiedad. En este bloque de código, hay una variable local llamada `value` creada implícitamente y contiene el valor que se debe asignar a la propiedad.

Como para cualquier elemento declarado en Visual C#, puede especificar un modificador de nivel de acceso para una propiedad. Se aplica al bloque `get` y `set`. También puede especificar un modificador de nivel de acceso para cada uno de los bloques `get` y `set`. En este caso, deben ser más restrictivos que aquel indicado a nivel de la propiedad.

```

protected int tasa
{
    public get
    {
        El modificador de accesibilidad del descriptor de acceso 'ConsoleApplication3.Program.Cliente.tasa.get' debe ser más restrictivo que la propiedad o el indicador 'ConsoleApplication3.Program.Cliente.tasa'.
        ret
    }

    public set
    {
    }
}

```

Las propiedades pueden ser también de sólo lectura o en sólo escritura. En este caso, debe eliminar el bloque de código `set` en el caso de una propiedad en sólo lectura, y el bloque `get` en el caso de una propiedad en sólo escritura.

Se puede implementar automáticamente la encapsulación cuando no haya tratamiento alguno en los bloques `get` y `set`. La propiedad se declara entonces de la siguiente manera:

```
public int tasa { get; set; }
```

Cuando declara así una propiedad, el compilador crea un espacio de almacenamiento privado y anónimo al que puede accederse únicamente a través de los encapsuladores `get` y `set` de la propiedad.

4. Los procedimientos de operador

Este tipo de procedimiento permite la redefinición de un operador estándar del lenguaje para utilizarlo en tipos personalizados (clase o estructura). Tomemos un ejemplo con la estructura cliente ya utilizada.

```
struct Cliente
```

```
{
    public int codigo;
    public string apellido;
    public string nombre;
}
```

Probamos el siguiente código:



```
Cliente c1, c2, c3;

c1.codigo = 200;
c1.nombre = "cliente1";
c1.apellido = "apellido1";

c2.codigo = 125;
c2.nombre = "cliente2";
c2.apellido = "apellido2";

c3 = c1 + c2;
```

(variable local) Cliente c1

Error:
El operador '+' no se puede aplicar a operandos del tipo 'ConsoleApplication3.Program.Cliente' y 'ConsoleApplication3.Program.Cliente'

Visiblemente, el compilador no se muestra cooperativo con la idea de sumar dos clientes.

Para que este código funcione, debemos indicarle el procedimiento que debe seguir con objeto de realizar esta operación. Por lo tanto, debemos redefinir el operador «+» para utilizarlo con dos clientes.

```
struct Cliente
{
    public int codigo;
    public string apellido;
    public string nombre;
    public static Cliente operator +(Cliente c11, Cliente c12)
    {
        Cliente c;
        c.codigo = c11.codigo + c12.codigo;
        c.apellido = c11.apellido + c12.apellido;
        c.nombre = c11.nombre + c12.nombre;
        return c;
    }
}
```

Después de esta modificación, el compilador se muestra más cooperativo y la ejecución del procedimiento anterior `test` visualiza el siguiente resultado:

```
325
cliente1cliente2
nombre1nombre2
```

5. Los argumentos de los procedimientos y funciones

Para que el código sea más fácilmente reutilizable, los valores empleados por los procedimientos y funciones pueden pasarse como parámetros en el momento de la llamada al procedimiento o función. Durante la declaración del procedimiento, debe especificar la lista de los parámetros

esperados. Esta lista se sitúa entre los paréntesis de la declaración del procedimiento. Debe indicar, para cada parámetro, su nombre y su tipo. Si se espera varios parámetros, conviene separarlos con una coma.

En el código del procedimiento, los parámetros se consideran como variables declaradas localmente.

Durante la llamada al procedimiento, se deberá indicar un valor para cada uno de los parámetros esperados. Tomemos un ejemplo de declaración y de utilización:

```
public static double CalculoNETO(double Pbruto,double Tasa)
{
    return Pbruto * (1 + (Tasa / 100));
}

double PrecioBruto = 100;
double PrecioNeto;
PrecioNeto = TestEstructura.CalculoNETO(PrecioBruto, 5.5);
Console.WriteLine(PrecioNeto);
```

Para pasar una variable como parámetro a un procedimiento (el PrecioBruto del ejemplo anterior), existen dos posibilidades:

- El paso por valor : en este caso, la información transmitida al procedimiento será simplemente el contenido de la variable pasada como parámetro.
- El paso por referencia : en este caso, la información transmitida al procedimiento ya no es el contenido de la variable, sino la ubicación donde está almacenada la variable, en la memoria de la máquina. El código del procedimiento va a buscar en esta ubicación el valor que necesita. El código del procedimiento puede también modificar el contenido de la variable y, en este caso, las modificaciones serán visibles en el código que llamó a su procedimiento.

Por defecto, es el tipo del parámetro el que determina la técnica utilizada. Los siguientes tipos: numéricos enteros, numéricos como flotante, decimal, bool, estructuras definidas por el usuario se pasan por valor. Los demás tipos siempre se pasan por referencia.

Sin embargo, es posible forzar el paso por referencia de uno o varios parámetros utilizando la palabra reservada `ref` o `out` durante la declaración del parámetro en la función. Se utiliza esta solución para que cualquier modificación que realice el método en el parámetro sea reflejada en el código invocante cuando recupera el control.

El siguiente ejemplo de función calcula un importe NETO a partir de un precio bruto y de una tasa de IVA. El importe NETO está disponible como valor de devolución de la función, el importe del IVA es recuperado por un parámetro pasado como referencia.

```
public static double CalculoNETO(double Pbruto, double Tasa,ref double iva)
{
    iva = Pbruto * (Tasa / 100);
    return Pbruto+iva;
}
```

El uso de la palabra reservada `ref` en la declaración de una función impone dos exigencias durante la llamada de la función:

- También se debe utilizar esta palabra reservada durante la llamada.
- Se debe inicializar la variable obligatoriamente antes de la llamada.

```
double PrecioBruto = 100;
double PrecioNeto;
double importeIva=0;
```

```
PrecioNeto = TestEstructura.CalculoNETO(PrecioBruto, 5.5, ref importeIva);  
Console.WriteLine("Precio neto: {0}", PrecioNeto);  
Console.WriteLine("Importe iva: {0}", importeIva);
```

La palabra reservada `out` presenta un funcionamiento similar, excepto en cuanto a la exigencia de inicialización obligatoria, que no se aplica.

El paso por referencia no funciona si la información pasada a la función es una propiedad o un valor literal no considerados como variables.

Otra posibilidad permite crear un procedimiento que podrá coger un número cualquiera de parámetros. En este caso, utilice la palabra reservada `params` para declarar una matriz de parámetro.

En el siguiente ejemplo, vamos a crear una función que calcula la media de todos los parámetros que se le pasan.

```
public static double media(params double[] notas)  
{  
    double suma=0;  
    foreach (double nota in notas)  
    {  
        suma = suma + nota;  
    }  
    return suma / notas.Length;  
}
```

Luego se puede llamar a la función con un número cualquiera de parámetros.

```
Resultado=media(1,6,23,45);
```

o

```
Resultado=media(12,78);
```

Parámetros opcionales

También puede indicar, en la lista de los parámetros de un procedimiento o de una función, que ciertos parámetros son opcionales asignando un valor por defecto al parámetro en la declaración del procedimiento o función.

```
double calculoNETO(double Pbruto, double Tasa = 21)
```

Cuando un parámetro es declarado opcional en un procedimiento o una función, todos los siguientes deben ser declarados también opcionales. La siguiente declaración no es válida, ya que el tercer parámetro también debe ser opcional.

```
double calculoNeto(double Pbruto, double Tasa = 21, String divisa)  
{  
    return Pbruto * Tasa;  
}
```

Los parámetros opcionales deben aparecer después de todos los parámetros necesarios.

Hay que utilizar la siguiente sintaxis:

```
double calculoNETO(double Pbruto, double Tasa = 21, String  
divisa="€")
```

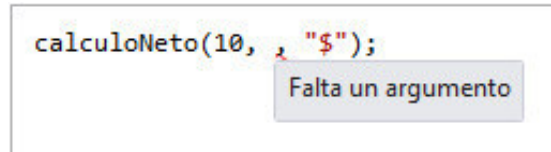

Se puede llamar a esta función con la siguiente sintaxis con, en este caso, el uso del valor por defecto para los parámetros tasa y divisa.

```
calculoNETO(10);
```

La utilización de la siguiente sintaxis también es posible con, en este caso, el uso del valor por defecto para el parámetro divisa.

```
calculoNETO(10, 5.5);
```

En cambio, la siguiente sintaxis de llamada está prohibida, ya que si usted especifica un valor para un parámetro opcional, todos los parámetros opcionales anteriores deben definirse.



En este caso, hay que utilizar la siguiente sintaxis:

```
calculoNETO(10, 21, "$");
```

Parámetros nominados

Durante la llamada al procedimiento, tiene dos opciones para indicar el valor utilizado para cada parámetro:

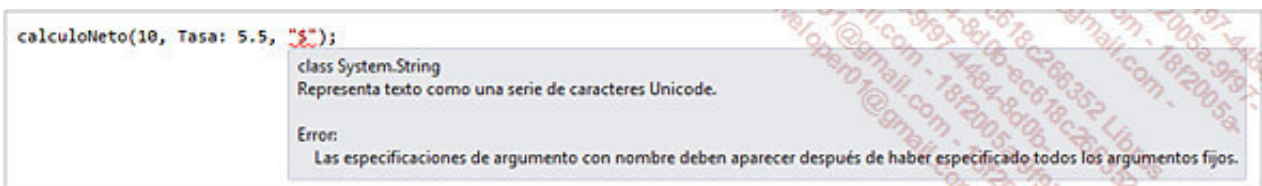
- Utilizar el paso por posición con el cual los valores de los parámetros deben aparecer en el mismo orden que en la declaración del procedimiento.
- Utilizar el paso por nombre indicando durante la llamada del procedimiento o de la función el nombre de cada parámetro y el valor que desea asignarle y separando estos dos datos con el carácter :. El orden de los parámetros no tiene importancia en este caso, pero debe especificar obligatoriamente un valor para los parámetros que no son opcionales.

```
calculoNETO(divisa: "$", Pbruto: 250);
```

Estas dos soluciones se pueden combinar en la misma llamada de procedimiento o función. Se puede usar la siguiente sintaxis:

```
calculoNETO(10, divisa: "$");
```

Por el contrario, un parámetro nombrado sólo puede ser utilizado después de los parámetros por posición.



6. Funciones asíncronas

Las funciones asíncronas permiten mejorar la reactividad de una aplicación. Es frecuente tener que efectuar, en una aplicación, procesamiento relativamente largos. Con un modelo de desarrollo clásico, la aplicación completa se bloquea esperando a que termine el procesamiento. Esta situación es molesta para el usuario. No sabe, realmente, qué está haciendo la aplicación. Si quiere detener la aplicación durante este tiempo de bloqueo, no tiene más opción que utilizar el administrador de tareas de Windows. Para evitar esta situación, ahora es posible definir funciones asíncronas. La palabra reservada `async` incluida en la firma de una función hace que su ejecución se produzca de forma asíncrona. Para que este mecanismo sea realmente eficaz hace falta, además, indicar en el interior de este tipo de función al menos una ubicación donde pueda suspenderse la ejecución y esperar a que el procesamiento termine. La palabra reservada `await` situada delante de una expresión indica estos puntos de interrupción. Cuando termina el procesamiento, se evalúa la expresión y se retoma la ejecución de la función. Para que este mecanismo funcione, es preciso que la expresión genere un tipo `Task<...>`. Veamos a continuación varios ejemplos prácticos. Vamos a partir de la base de una función que verifica si un número es primo o no.

```
public static bool esPrimo(int nb)
{
    if (nb < 2)
    {
        return false;
    }
    if (nb == 2)
    {
        return true;
    }
    if (nb % 2 == 0)
    {
        return false;
    }
    int i;
    i = 3;
    while ((i * i <= nb))
    {
        if (nb % i == 0)
        {
            return false;
        }
        else
        {
            i = i + 1;
        }
    }
    return true;
}
```

Esta función no tiene nada de particular, salvo que es susceptible de requerir una cantidad de tiempo importante para ejecutarse, si se invoca con un número entero con un valor muy elevado. El fenómeno se amplifica, además, si se invoca en repetidas ocasiones. Esto es lo que vamos a poner de relieve con la siguiente aplicación, que permite realizar la búsqueda de cuántos números primos existen entre 0 y un número especificado.

Agregamos una nueva función que cuenta cuántos números primos existen entre 0 y el valor que se recibe como parámetro.

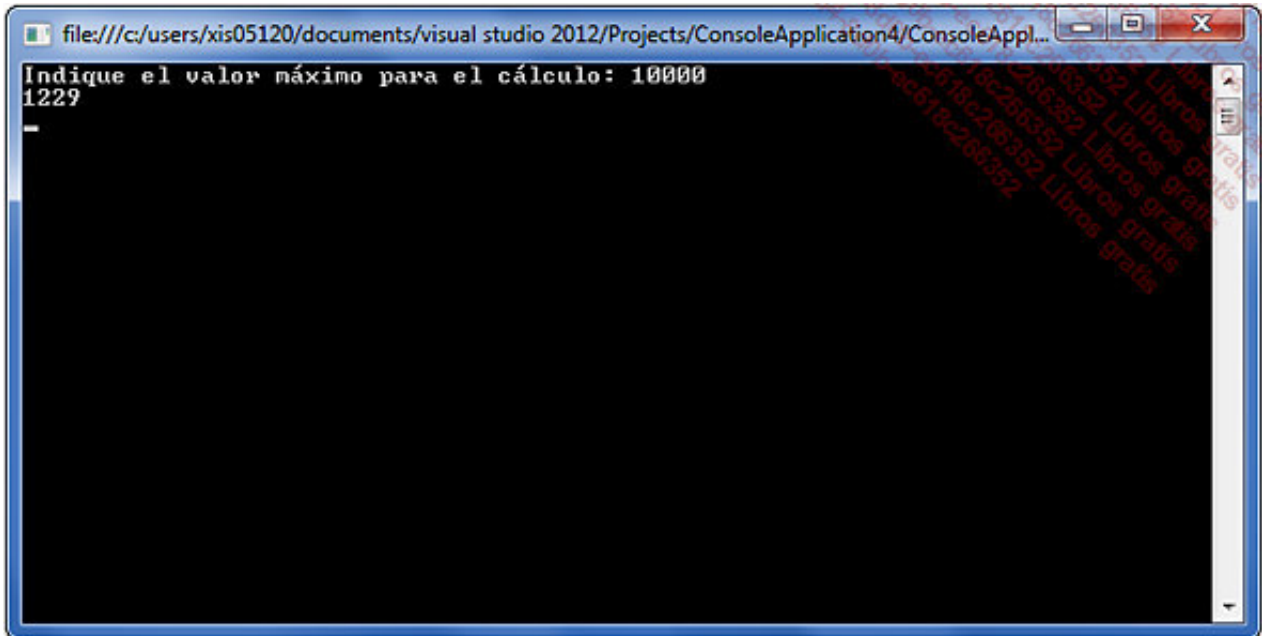
```
public static int cuentaPrimos(int maxi)
{
    int i;
```

```
int nb;
{
    nb = 0;
    for (i = 0; i <= maxi; i++)
    {
        if (esPrimo(i))
        {
            nb = nb + 1;
        }
    }
    return nb;
}
```

Sólo nos queda agregar un método `main` para completar nuestra aplicación.

```
static void Main(string[] args)
{
    Console.WriteLine("Indique el valor máximo para el cálculo: ");
    string maxi;
    maxi = Console.ReadLine();
    int resultado;
    resultado = cuentaPrimos(int.Parse(maxi));
    Console.WriteLine(resultado);
    Console.ReadKey();
}
```

Verifiquemos, a continuación, que funciona correctamente.



No hay problema, obtenemos rápidamente el resultado. Por el contrario, si intentamos realizar esta operación con un límite mucho más elevado (100000000, por ejemplo), va a hacer falta que tengamos paciencia puesto que los cálculos van a requerir una cantidad considerable de tiempo.

Si no tiene paciencia para esperar al resultado, deberá detener la aplicación cerrando de manera brusca la consola, o utilizando el administrador de tareas de Windows. Esto no es propio de una buena solución.

Podemos mejorar este comportamiento proporcionando al usuario la posibilidad de terminar la

ejecución de la aplicación. Para ello, agregamos a nuestro método `main` un simple bucle `do while` en el que realizamos nuestro procesamiento y solicitamos al usuario que introduzca un carácter 's' para detener la ejecución.

```
static void Main(string[] args)
{
    Console.Write("Indique el valor máximo para el cálculo: ");
    string maxi;
    maxi = Console.ReadLine();
    int resultado;
    char respuesta;
    do
    {
        respuesta = cuentaPrimos(int.Parse(maxi));
        Console.WriteLine(resultado);
        respuesta = Console.ReadKey().KeyChar;
    } while (respuesta != 's');
}
```

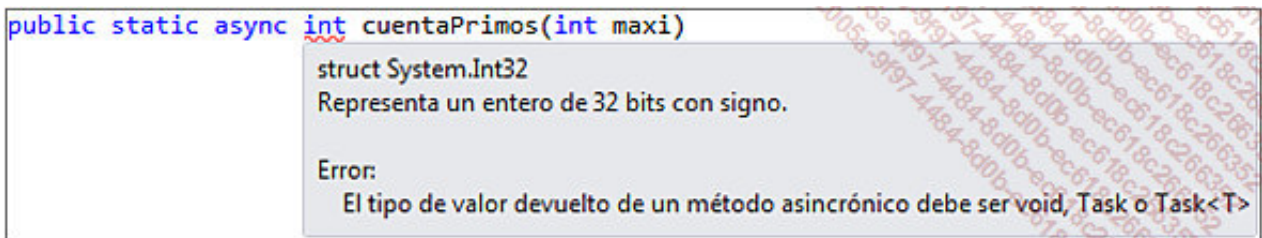
Durante la prueba, no se apreciaba una verdadera mejora.

De hecho, las instrucciones presentes en el bucle `do while` se ejecutan de forma secuencial y en este caso la instrucción `respuesta = Console.ReadKey().KeyChar;` sólo se ejecutará tras haber mostrado el resultado y, por tanto, tras haber finalizado el cálculo. La función `cuentaPrimos`, bloqueante, no deja que se ejecute ninguna otra acción en la aplicación mientras no haya terminado. La solución consiste en transformar la función `cuentaPrimos` en una función asíncrona.

Para ello, basta con agregar la palabra reservada `async` en la firma:

```
public static async int cuentaPrimos(int maxi)
```

El hecho de agregar esta palabra reservada impone otra modificación a la firma.

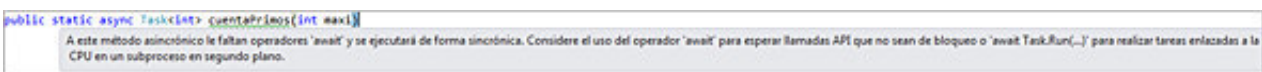


Hace falta, por tanto, modificar la función para que devuelva un tipo `Task<int>`.

```
public static async Task<int> cuentaPrimos(int maxi)
```

Nuestra función puede, ahora, convertirse en una función asíncrona, aunque todavía no está lista.

Es lo que nos indica Visual Studio:



El procesamiento que se ejecuta en la función asíncrona debe pasarse como parámetro, bajo la forma de una expresión lambda, al método `Run` de la clase `Task`. Es, a continuación, el resultado de la ejecución de esta expresión lambda el que se utiliza como valor de retorno de la función

asíncrona.

```
public static async Task<int> cuentaPrimos(int maxi)
{
    int resultado;
    resultado=await Task.Run<int>(() =>
    {
        int i;
        int nb;
        {
            nb = 0;
            for (i = 0; i <= maxi; i++)
            {
                if (esPrimo(i))
                {
                    nb = nb + 1;
                }
            }
        }
        return nb;
    });
    return resultado;
}
```

La última etapa consiste en utilizar esta función de forma asíncrona.

Para ello, vamos a crear una nueva función que va a llamar a la función `cuentaPrimos`. Para que la función `cuentaPrimos` se ejecute en modo asíncrono, debemos utilizar la palabra reservada `await` durante su llamada.

```
static async Task procesamiento(int maxi)
{
    int resultado;
    resultado = await cuentaPrimos(maxi);
    Console.WriteLine(resultado);
    Console.WriteLine("*****");
}
```

Para terminar, modifiquemos la función `Main` para poder interrumpir el cálculo.

```
static void Main(string[] args)
{
    string maxi;
    int mx;
    Console.Write("Indique el valor máximo para el
cálculo: ");
    maxi = Console.ReadLine();
    mx = int.Parse(maxi);
    procesamiento(mx);
    Console.WriteLine("presione una tecla cualquiera
para detener la aplicación antes de que realice el cálculo ");
    Console.ReadKey();
}
```

Visual Studio nos indica que se produce una anomalía durante el procesamiento.

```
procesamiento(mx);  
(admite await) Task Program.procesamiento(int max)  
  
Use:  
await procesamiento(...);
```

Subir

[Condiciones generales de uso](#)

Copyright - ©Editions ENI



Ensamblados, espacios de nombres y atributos

1. Los ensamblados

Visual C# ha sido diseñado con el Framework .NET, lo que le permite disfrutar de muchas ventajas, en particular en términos de seguridad durante la ejecución y la gestión de la memoria. También permite esta imbricación asegurar la compatibilidad entre código escrito en los diferentes lenguajes disponibles. Así puede utilizar en Visual C# elementos diseñados con otros lenguajes (e inversamente), de manera totalmente transparente sin que tenga ni siquiera que preocuparse del lenguaje en el cual ha sido desarrollado el elemento.

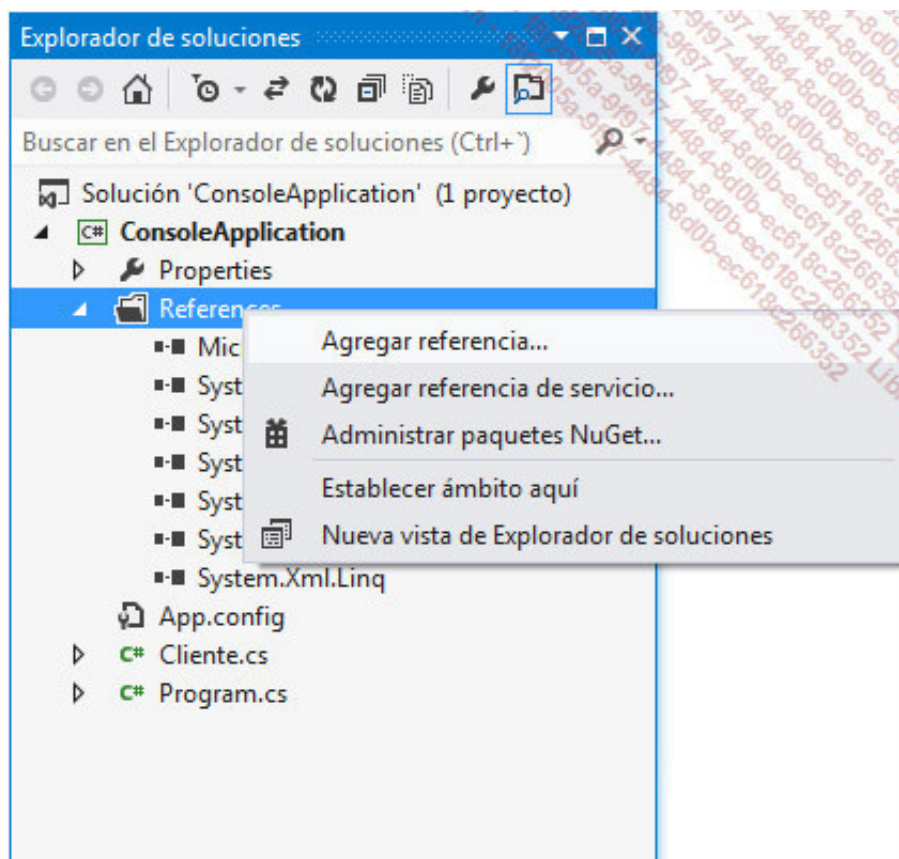
El elemento básico de esta reutilización en el Framework .NET es el ensamblado. Se puede considerar como la agrupación de tipos, recursos y funcionalidades diseñados para funcionar conjuntamente.

Los ensamblados se almacenan en archivos .exe o .dll, según el tipo. Son generados simplemente por la compilación del proyecto correspondiente.

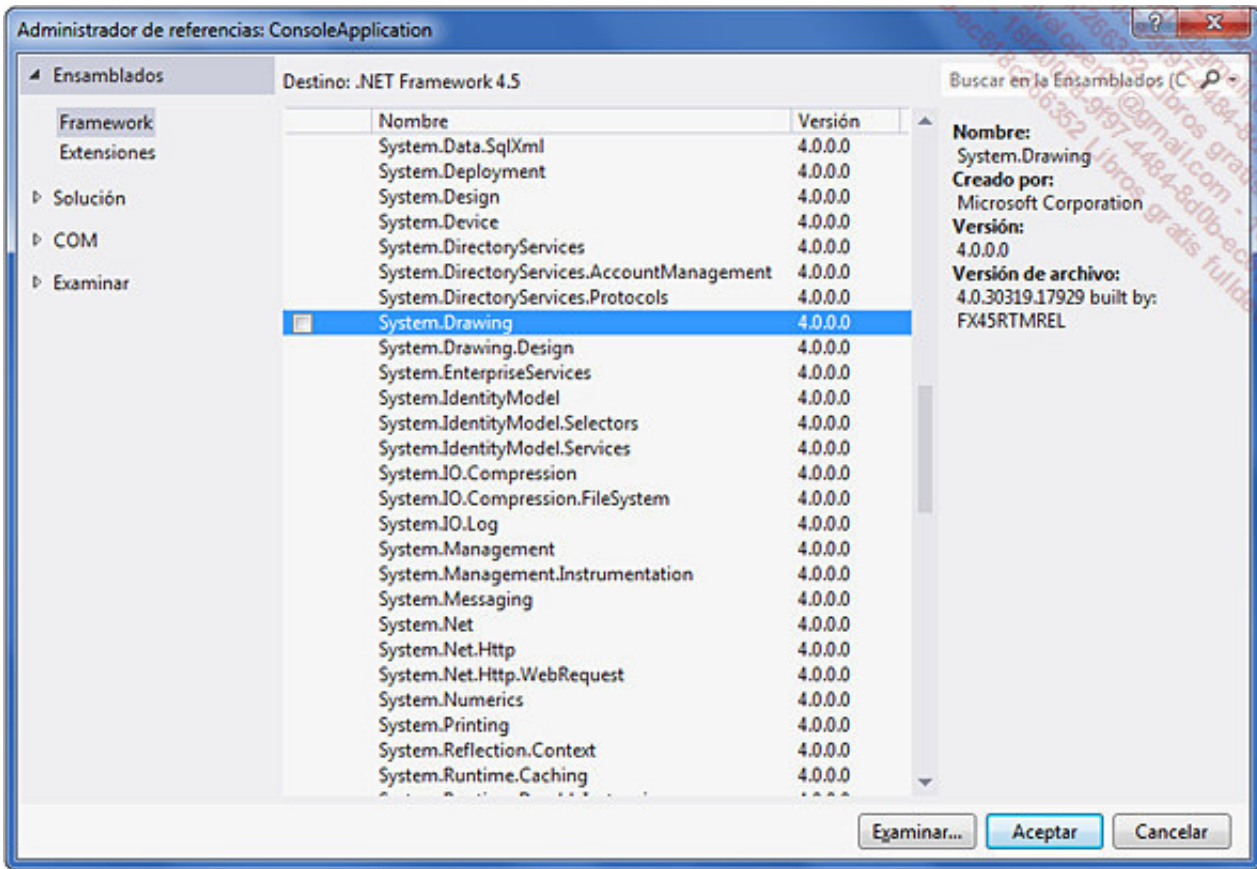
Son autodescriptivos, ya que contienen los datos necesarios para su utilización en otro proyecto. Estos datos están contenidos en el **manifiesto** del ensamblado. El **manifiesto** contiene, entre otras cosas:

- la identidad del ensamblado (su nombre y su versión),
- una lista de los archivos utilizados por el ensamblado (por ejemplo, los otros ensamblados utilizados por éste, los recursos de mapa de bits, etc.).

Para poder utilizar un ensamblado en un proyecto, añada simplemente una referencia hacia el ensamblado. Para ello, utilice el menú contextual del archivo de referencia del proyecto.



El siguiente cuadro de diálogo permite entonces elegir las referencias que hay que añadir al proyecto.



Las diferentes pestañas permiten elegir, según la categoría, el tipo de referencia que hay que añadir al proyecto:

.NET

El conjunto de los componentes del Framework .NET disponibles.

Solución

Los otros proyectos de la solución actual.

COM

Los componentes COM y ActiveX registrados en el sistema.

Examinar

Búsqueda de un archivo (dll, ocx...) que contiene los recursos.

2. Los espacios de nombres

Los namespaces o espacios de nombres organizan de manera lógica los objetos disponibles en un ensamblado. Se utilizan para evitar las ambigüedades cuando en un proyecto se añaden referencias

en unos ensamblados que contienen elementos que tienen nombres idénticos.

Por ejemplo, la clase `ListBox` existe en los ensamblados `System.Web` y `System.Windows.Forms`. Si se añaden referencias en un proyecto hacia estos ensamblados, el compilador se arriesga a no poder determinar cuál de estas clases desea realmente utilizar.

La utilización del nombre plenamente cualificado, incluyendo el espacio de nombres en el cual se define la clase, permite resolver este tipo de problema.

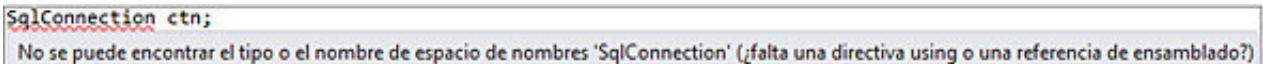
Usted puede utilizar, por ejemplo, el siguiente código:

Ejemplo

```
System.Windows.Forms.ListBox listaWindows;  
System.Web.UI.WebControls.ListBox listaWeb;
```

Sin embargo, la utilización del nombre plenamente cualificado puede hacer pesada la escritura del código. Es posible utilizar la palabra reservada `using` para aligerar el código. Indica al compilador que ciertos espacios de nombres están sobreentendidos.

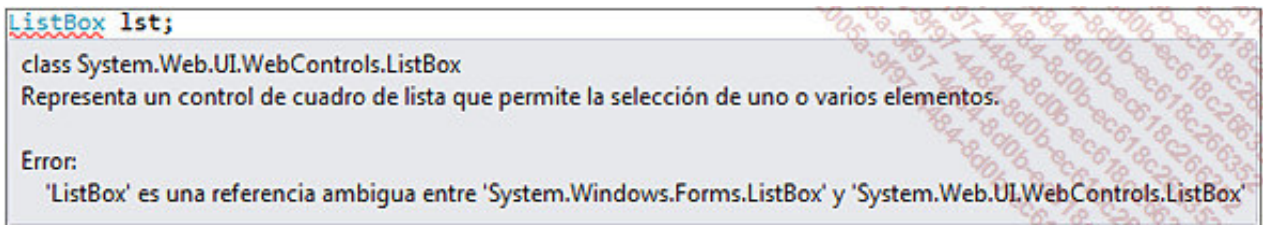
Por ejemplo, la instrucción `using System.Data.SqlClient;` autoriza la utilización de la siguiente declaración: `SqlConnection ctn`, que sin importación del namespace hubiera provocado un error de compilación:



```
SqlConnection ctn;  
No se puede encontrar el tipo o el nombre de espacio de nombres 'SqlConnection' (¿falta una directiva using o una referencia de ensamblado?)
```

Las instrucciones `using` deben ser las primeras líneas de código de un archivo fuente Visual C#.

Sin embargo, permanezca atento para no volver a caer en el problema anterior.



```
ListBox lst;  
class System.Web.UI.WebControls.ListBox  
Representa un control de cuadro de lista que permite la selección de uno o varios elementos.  
Error:  
'ListBox' es una referencia ambigua entre 'System.Windows.Forms.ListBox' y 'System.Web.UI.WebControls.ListBox'
```

La instrucción `using` propone una solución elegante creando un alias durante la importación del espacio de nombres.

```
using ctrlWin=System.Windows.Forms;  
using ctrlWeb=System.Web.UI.WebControls;  
ctrlWin.ListBox listaWindow;  
ctrlWeb.ListBox listaWeb;
```

Esta solución autoriza la utilización de nombres de una longitud razonable evitando los conflictos.

Cabe observar también que, según el tipo de proyecto en el cual está trabajando, se realizan referencias e importaciones por defecto.

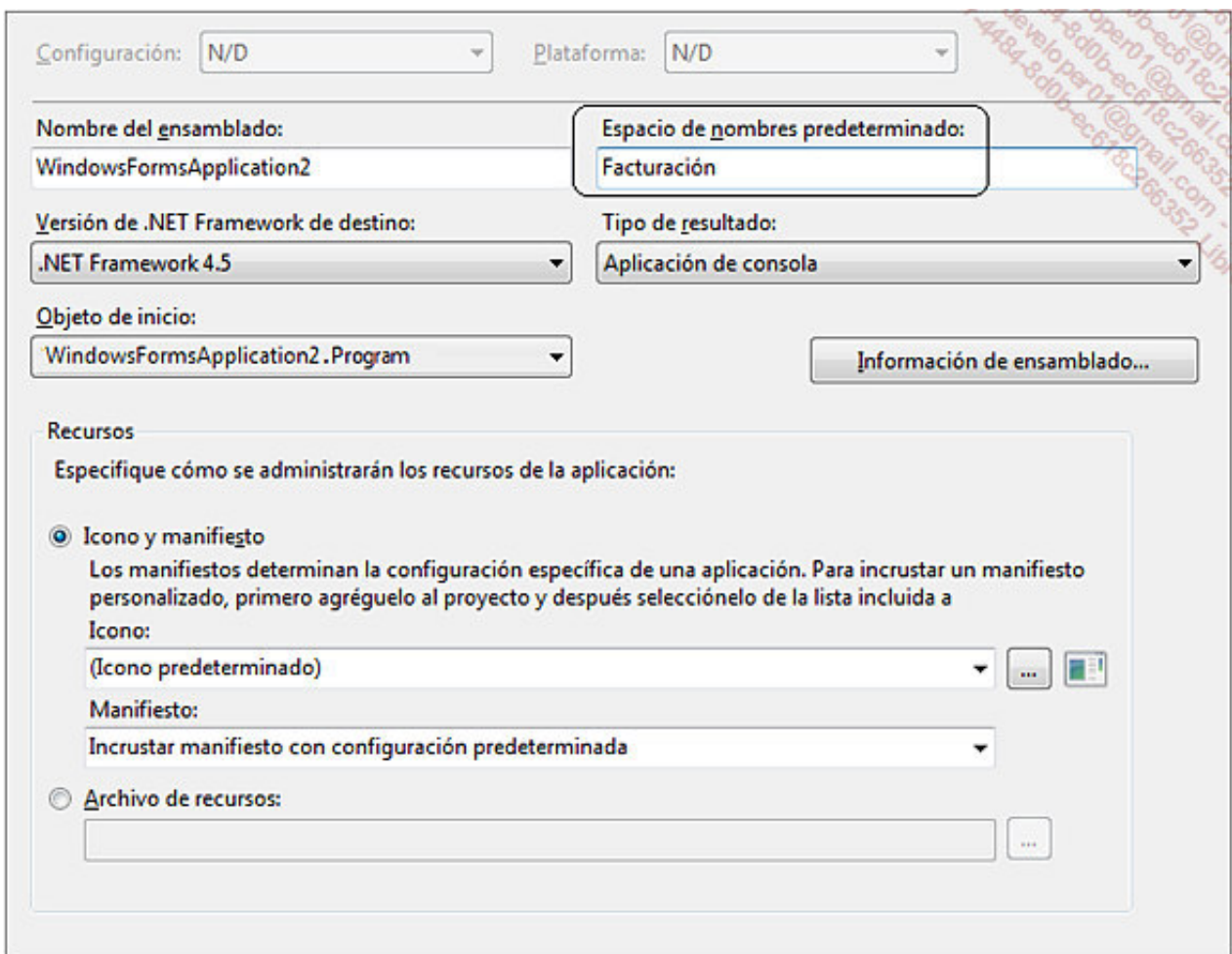
Los namespaces se declararán en el código usando la palabra reservada `namespace` seguida el nombre del namespace y de un bloque de código.

Todos los elementos declarados en este bloque de código serán accesibles al darles un prefijo con

el nombre del namespace.

```
namespace Facturacion
{
    class Tarificacion
    {
        public static double CalculoNETO(double Pbruto, double Tasa)
        {
            return Pbruto * (1 + (Tasa / 100));
        }
    }
}
```

En el ejemplo anterior, la función `calculoNETO` definida en la clase `Tarificacion` es accesible al darle un prefijo con el nombre del namespace. También hay que observar que Visual Studio añade automáticamente una instrucción 'namespace' en el código de todos los elementos que puede añadir a un proyecto. Utiliza como nombre los datos indicados en las propiedades del proyecto.



En nuestro ejemplo, la función `calculoNETO` es, por lo tanto, accesible con el siguiente código:

```
public static void main()
{
    double precioNETO=Tarificacion.CalculoNETO(100, 5.5);
}
```

Utilice la misma técnica en el caso de espacios de nombres anidados; como en el siguiente ejemplo:

```
namespace Gestion
{
    namespace Paga
    {
        public class Salario
        {
        }
    }
    namespace Facturacion
    {
        public class Factura
        {
        }
    }
}
```

La clase `Salario` será, por lo tanto, accesible con el nombre `Gestion.Paga. Salario`.

3. Los atributos

Los atributos son marcas que puede colocar en su código con el fin de añadir datos adicionales a los elementos de su aplicación.

Se guardan en los metadatos del ensamblado durante la compilación del proyecto. El runtime utiliza los metadatos para gestionar la depuración, el seguimiento de las versiones, la compilación y otros datos relativos a la utilización de su código. Los atributos pueden aplicarse a un ensamblado, un módulo o una porción de código más pequeña, como un procedimiento o una función. A veces podrán aceptar argumentos para modificar su significado.

Los atributos están ubicados en el código entre los símbolos [y]. Si se utilizan varios atributos, deben ir separados con comas. Los posibles parámetros de un atributo estarán ubicados entre paréntesis.

El alcance de un atributo puede extenderse también con las palabras reservadas `Assembly:` o `Module:` ubicadas antes del atributo. La sintaxis de utilización de un atributo es, por lo tanto:

```
[alcance:Atributo1(parámetro1,...),Atributo2,...]
```

a. Atributos más habituales en Visual C#

Entre los atributos disponibles, algunos se usan muy a menudo en el desarrollo con Visual C#. Vamos a estudiar su utilización e ilustrarlo con un ejemplo.

Serializable, NonSerialized

Estos dos atributos controlan la serialización de una clase y de sus miembros. La serialización permite el registro de una instancia de clase en un archivo, con lo que asegura la persistencia de los datos. El archivo generado puede estar en formato binario o XML. En este caso, facilita el intercambio de datos entre aplicaciones. Para que una clase sea utilizable por el mecanismo de serialización, ésta debe ser marcada con el atributo `SerializableAttribute`. Durante la operación de serialización, el contenido de cada uno de los miembros de la instancia de la clase se guarda en el archivo. Si algunos de ellos no deben guardarse en el archivo, se deben marcar con el atributo `NonSerializedAttribute`.

El ejemplo siguiente define la clase `Persona` con dos miembros (`Apellido` y `Nombre`) que se serializarán y un miembro (`Edad`) que no se serializará. Una instancia de la clase se crea y se guarda en un archivo con formato XML.

Ejemplo

```
using System;
using System.Runtime.Serialization;
using System.Runtime.Serialization.Formatters.Soap;
using System.IO;
namespace Contab
{
    [Serializable()] public class Persona
    {
        public String apellido;
        public string nombre;
        [NonSerialized()] public int edad;
        public Persona()
        {
        }
    }
    static class Serializacion
    {
        public static void main()
        {
            Persona unaPersona;
            unaPersona = new Persona();
            unaPersona.apellido = "García";
            unaPersona.nombre = "Pablo";
            unaPersona.edad = 25;
            Stream flujo;
            flujo = File.Open("c:\\datos.xml", FileMode.Create);
            SoapFormatter formador;
            formador = new SoapFormatter();
            formador.Serialize(flujo, unaPersona);
            flujo.Close();
        }
    }
}
```

La ejecución de este código genera el siguiente archivo XML:

```
<SOAP-ENV:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:SOAP-ENC=
"http://schemas.xmlsoap.org/soap/encoding/" xmlns:SOAP-ENV=
"http://schemas.xmlsoap.org/soap/envelope/" xmlns:clr="http://schemas.
microsoft.com/soap/encoding/clr/1.0" SOAP-ENV:encodingStyle=
"http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<a1:Persona id="ref-1" xmlns:a1="http://schemas.microsoft.com/clr/nsassem/
Contab/testFunciones%2C%20Version%3D1.0.0.0%2C%20Culture%3Dneutral%2C%20Public-
KeyToken%3Dnull">
<apellido id="ref-3">García</apellido>
<nombre id="ref-4">Pablo</nombre>
</a1:Persona>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Nuestra instancia de la clase `Persona` se encuentra guardada en este archivo con sus dos miembros `Apellido` y `Nombre` y, como hemos indicado en la definición de la clase, el miembro `Edad` no se ha guardado.

DllImport

Se utiliza este atributo para indicar que una función es importada desde una librería de código no gestionado. Permite en particular la utilización de funciones definidas en una librería del sistema. En el siguiente ejemplo, la función `MoveFile` se puede utilizar como una función clásica.

Ejemplo

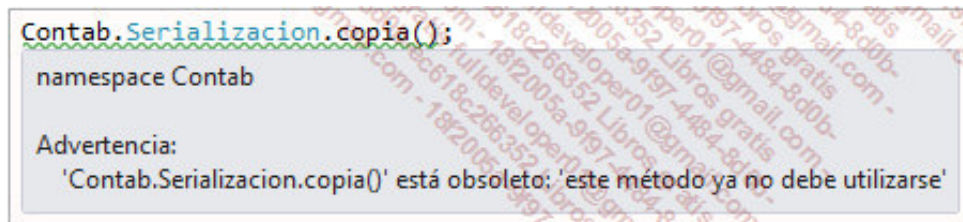
```
[DllImport("KERNEL32.DLL")] public static extern bool
MoveFile(string src,string dst);
```

Obsolete

Se puede utilizar este atributo para indicar que un elemento, clase o método o propiedad no se debe utilizar más. Si a pesar de todo se utiliza este elemento en una aplicación, el compilador genera una advertencia o un error en función de la configuración del atributo. Es posible pasar a este atributo una cadena de caracteres como parámetro para representar el mensaje visualizado por el compilador. Un segundo parámetro de tipo booleano permite especificar si la utilización del elemento, marcado con este atributo, genera una advertencia o un error de compilación.

```
[Obsolete ("No se debe utilizar más este método",false)]
public static void copia()
{
    Persona unaPersona;
    unaPersona = new Persona();
    unaPersona.apellido = "García";
    unaPersona.elNombre = "Pablo";
    unaPersona.edad = 25;
    Stream flujo;
    flujo = File.Open("c:\\datos.xml", FileMode.Create);
    SoapFormatter formateador;
    formateador = new SoapFormatter();
    formateador.Serialize(flujo, unaPersona);
    flujo.Close();
    Persona.MoveFile("c:\\datos.xml", "c:\\data.xml");
}
```

La utilización de este método en una aplicación provoca la siguiente advertencia durante la compilación.



Si este atributo viene definido con un segundo parámetro igual a `true`, el compilador activa un error cuando se utiliza el elemento.

```
[Obsolete("no se debe utilizar más esta propiedad", true)]  
public String nombre  
{  
    get  
    {  
        return elnombre;  
    }  
    set  
    {  
        elnombre= value;  
    }  
}
```

```
Persona unaPersona;  
unaPersona = new Persona();  
unaPersona.nombre = "Pablo";
```

Subir

[Condiciones generales de uso](#)

Copyright - ©Editions ENI



Introducción

Con Visual C#, la noción de objeto es omnipresente y requiere un mínimo de aprendizaje. Vamos a ver primero los principios de la programación orientada a objetos y el vocabulario asociado, luego veremos cómo poner eso en una aplicación con Visual C#.

En un lenguaje de procedimientos clásico, el funcionamiento de una aplicación se basa en una sucesión de llamadas a los diferentes procedimientos y funciones disponibles en el código. No hay ningún enlace entre los datos y los procedimientos que los manejan. Por el contrario, en un lenguaje orientado a objetos, vamos a intentar agrupar al máximo los datos y el código para manejarlos. Las clases son la representación simbólica de los objetos. Describen los campos, propiedades, métodos y eventos de la misma manera que un plano de arquitecto describe las diferentes partes de un edificio.

Prosigamos nuestra analogía entre una clase y el plano de un edificio. Sabemos que es posible construir varios edificios a partir del mismo plano. De la misma manera, se pueden construir varios objetos a partir de la misma clase. Se puede utilizar una clase para crear tantas instancias como sea necesario.

En un plano de edificio, algunas zonas pueden ofrecer un acceso limitado a ciertas personas. De la misma manera, en una clase, ciertos elementos pueden disponer de un acceso restringido. Es el principio de encapsulación.

Los términos clase y objeto se confunden a menudo, pero se trata en realidad de elementos muy distintos. Una clase representa la estructura de un elemento, mientras el objeto es un ejemplar creado a partir del modelo de esta estructura. La modificación de un elemento en un objeto no cambia en absoluto los otros objetos creados a partir del mismo modelo (clase). En nuestro ejemplo de plano de edificio, el añadido de una nueva habitación en un edificio existente no cambia para nada los otros edificios construidos según el mismo plano. Por el contrario, la modificación del plano (de la clase) conlleva modificaciones para todos los nuevos edificios (todos los nuevos objetos).

Las clases están compuestas de campos, propiedades, métodos y eventos. Los campos y propiedades representan los datos contenidos en los objetos. Se consideran los campos como variable y es posible leer su contenido o asignarles un valor directamente. Por ejemplo, si tiene una clase que representa un cliente, puede guardar su nombre en un campo.

Las propiedades se manejan de la misma manera que los campos, pero se activan a partir de procedimientos de propiedad `Get` y `Set`. Esto permite más control sobre la forma en la que los valores son leídos o asignados y permite validar los datos antes de su utilización.

Los métodos representan las acciones que un objeto puede realizar. Se activan gracias a la creación de procedimientos o funciones en una clase.

Los eventos son datos que un objeto recibe o transmite desde o hacia otro objeto o aplicación. Los eventos permiten a los objetos ejecutar acciones cuando se da una situación particular. Como Windows es un sistema operativo de eventos, los eventos pueden provenir de otros objetos, del sistema o de las acciones del usuario sobre el ratón y el teclado.

Esto sólo es una faceta de la programación orientada a objetos. Hay otros tres elementos fundamentales:

- La encapsulación.
- La herencia.
- El polimorfismo.

La encapsulación es la capacidad que permite crear y controlar el acceso a un grupo de elementos. Las clases proveen el medio más fiable de asegurar la encapsulación. Si tomamos el ejemplo de una cuenta bancaria, en una programación clásica, nos harían falta muchas variables y procedimientos o funciones para manejar los datos. La situación sería aún más compleja si tuviéramos que gestionar de

manera simultánea varias cuentas bancarias. Habría que trabajar entonces con tablas y hacer malabarismo con los índices. La encapsulación permite agrupar los datos y el código al manejarlos en una clase. Si debe trabajar con varias cuentas bancarias de manera simultánea, entonces tendrá varias instancias de la misma clase, limitando así el riesgo de errores. La encapsulación asegura también un control sobre la utilización de datos y procedimientos o funciones. Usted puede utilizar los modificadores de acceso, tales como `private` o `protected`, para restringir el acceso a ciertos métodos, propiedades o campos. Una regla fundamental de la encapsulación estipula que los datos de una clase sólo deben ser manipulados por el código de la clase (procedimientos de propiedades o métodos). A veces esta técnica se llama ocultación de datos. Asegura el funcionamiento de su código al enmascarar los detalles internos de la clase y evita así que sean utilizados de manera inadecuada. Autoriza también la modificación de una parte del código sin alterar el funcionamiento del resto de la aplicación.

La herencia permite la creación de una nueva clase basada en una clase existente. La clase que sirve de modelo para la creación de otra clase se llama clase base. La clase así creada hereda los campos, propiedades, métodos y eventos de la clase base. La nueva clase puede personalizarse añadiéndole campos, propiedades, métodos y eventos. Las clases creadas a partir de una clase base se llaman clases derivadas. Usted puede definir una clase base y reutilizarla varias veces para crear clases derivadas.

El polimorfismo es otra noción importante de la programación orientada a objetos. Gracias al polimorfismo es posible utilizar varias clases de manera intercambiable incluso si estas clases implementan sus propiedades y métodos de manera diferente. Estas propiedades y métodos son utilizables por el mismo nombre, con independencia de la clase a partir de la cual se ha construido el objeto.

Hay tres conceptos más asociados al polimorfismo. La sobrecarga, la sobrescritura y la ocultación de miembros permiten la definición de miembros de una clase que llevan el mismo nombre. Sin embargo hay algunas diferencias entre estas tres técnicas.

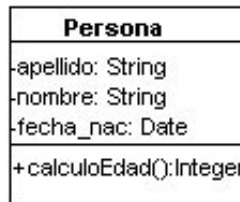
Se utiliza la sobrecarga para diseñar propiedades o métodos que llevan el mismo nombre pero que tienen un número de parámetros diferentes o tipos de parámetros diferentes.

La sobrescritura permite la redefinición de métodos o propiedades heredadas de una clase base. Los miembros sobrescritos pueden aceptar el mismo número y tipo de parámetros que el método o

Subir

Aplicación con Visual C#

En el resto de este capítulo, vamos a trabajar en la clase `Persona`, cuya representación UML (*Unified Modeling Language*) está disponible a continuación.



UML es un lenguaje gráfico especializado en la representación de los conceptos de programación orientada a objetos. Para más información sobre este lenguaje, puede consultar el libro UML2 en esta misma colección.

1. Creación de una clase

La creación de una clase pasa por la declaración de la propia clase y de todos los elementos que la constituyen.

a. Declaración de la clase

La declaración de una clase se hace utilizando la palabra reservada `class` seguida de un bloque delimitado por los caracteres `{` y `}`. En este bloque de código se encuentran las declaraciones de variables, que serán los campos de la clase, y los procedimientos, que serán los métodos de la clase.

La sintaxis general de definición de una clase es, por lo tanto, la siguiente:

```
[atributos] [modificadores] [parcial] class nombreDeLaClase
[ : clase base] [, interfaz1, interfaz2,...]
{
    Código de la clase
}
```

Hay muchas palabras clave que permiten personalizar una clase. En el momento de su declaración, se puede especificar la visibilidad de la clase. Para determinar la visibilidad el lenguaje cuenta con las siguientes palabras clave:

`public`

Podrá utilizar la clase en todo su proyecto, pero también en otros proyectos.

`internal`

El acceso a la clase es limitado al proyecto en el cual está definida.

`private`

La clase sólo puede ser utilizada en el módulo en el cual está definida.

`protected`

La clase sólo puede ser utilizada en una subclase en la cual aquélla esté definida. Sólo se puede utilizar esta palabra reservada para una clase declarada en otra clase.

`protected internal`

Idéntica a la unión de los alcances `protected` y `internal`.

También puede indicar cómo su clase se va a comportar con respecto a la herencia. Hay dos opciones posibles:

`abstract`

Indica que la clase sirve de clase base en una relación de herencia. Usted no podrá crear instancias de esta clase. En general, en este tipo de clase, sólo las declaraciones de los métodos están definidas. Hará falta escribir en las clases derivadas el contenido de estos métodos.

`sealed`

Esta clase será la última de la jerarquía. Por lo tanto, no será posible utilizar esta clase como clase base de otra clase.

Para indicar que su clase recupera las características de otra clase por una relación de herencia, debe utilizar el carácter `:` seguido del nombre de la clase base. Puede implementar en su clase una o varias interfaces. Más adelante en este capítulo se verán estas dos nociones con más detalle.

El inicio de la declaración de nuestra clase `Persona` es, por lo tanto, el siguiente:

```
public class Persona
{
    string apellido;
    string nombre;
    DateTime fecha_naci;
}
```

b. Clase parcial

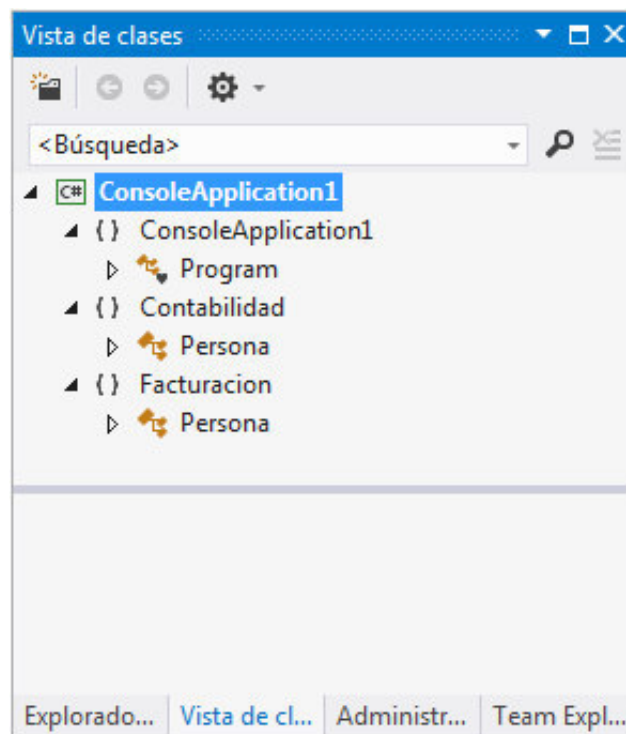
La definición de una clase se puede repartir entre varias declaraciones utilizando la palabra reservada `partial`. Esta técnica permite la definición de la clase en varios archivos fuente. Se utiliza mucho en Visual Studio para permitir la personalización de clases generadas de manera automática. El código generado suele colocarse en un archivo llamado `.designer.cs` que, en principio, no debe modificarse directamente.

Durante la compilación, el compilador agrupa todas las definiciones parciales para obtener el código fuente de la clase. Las diferentes partes de la definición de una clase en cambio deben estar en el mismo proyecto y formar parte del mismo namespace. Probamos el siguiente código:

```
namespace Contab
{
    public partial class Persona
    {
        string apellido;
```

```
        string nombre;
        DateTime fecha_nac;
        int calculoEdad()
        {
            return DateTime.Now.Year - fecha_nac.Year;
        }
    }
} namespace Facturacion
{
    public partial class Persona
    {
        int calculoEdad()
        {
            return DateTime.Now.Year - fecha_naci.Year;
        }
    }
}
```

A primera vista, nada ilegal, ya que el compilador genera correctamente el código. Sin embargo, no tiene la misma visión de las cosas que nosotros. Veamos lo que nos presenta el explorador de clases.



Hay dos clases `Persona` disponibles. El compilador determinó en realidad que nuestras dos definiciones de clase no forman parte del mismo namespace.

c. Creación de propiedades

Usted puede crear variables simples para almacenar los datos de su clase, pero los procedimientos de propiedad proporcionan más flexibilidad y control sobre el almacenamiento de los datos en una clase. Permiten a la clase proteger y validar sus propios datos. Una propiedad es similar a una función con dos bloques de código en el interior. Estos dos bloques están definidos por las palabras reservadas `get` y `set`; el bloque de código `get` se ejecuta durante la lectura de la propiedad. El bloque de código `set` se ejecuta durante la asignación de un valor a la propiedad.

Nuestra clase `Persona` se puede mejorar de la siguiente manera:

```
public class Persona
{
    string elApellido;
    string elNombre;
    DateTime laFecha_naci;
    public string apellido
    {
        get
        {
            return elApellido;
        }
        set
        {
            elApellido=value;
        }
    }
    public string nombre
    {
        get
        {
            return elNombre;
        }
        set
        {
            elNombre = value;
        }
    }
    public DateTime fecha_naci
    {
        get
        {
            return laFecha_naci;
        }
        set
        {
            laFecha_naci = value;
        }
    }
}
```

La creación de las propiedades permite ahora acceder de manera directa a los campos de la clase. Podemos permitirnos modificar la visibilidad de los campos de la clase y convertirlos en privados. De hecho se recomienda esta práctica para respetar el principio de encapsulación. Así tenemos la posibilidad de ser más exigentes en cuanto a los datos registrados en nuestra clase. Vamos a poner en práctica algunas de las siguientes reglas de gestión:

- El apellido se almacenará en mayúsculas.
- El nombre se almacenará en minúsculas.
- La fecha de nacimiento no será inferior a 1900.

Los procedimientos de encapsulación serán los encargados de la aplicación de estas reglas.

```
public class Persona
{
    private string elApellido;
```

```
private string elNombre;  
private DateTime laFecha_naci;  
public string apellido  
{  
    get  
    {  
        return elApellido;  
    }  
    set  
    {  
        elApellido=value.ToUpper();  
    }  
}  
public string nombre  
{  
    get  
    {  
        return elNombre;  
    }  
    set  
    {  
        elNombre= value.ToLower();  
    }  
}  
public DateTime fecha_naci  
{  
    get  
    {  
        return laFecha_naci;  
    }  
    set  
    {  
        if (value.Year >= 1900)  
        {  
            laFecha_naci = value;  
        }  
    }  
}  
}
```



Cabe observar que los procedimientos de encapsulamiento tienen un acceso completo a los campos de la clase, incluso los declarados privados.

Sólo lectura y sólo escritura

A veces puede ser interesante restringir los posibles accesos a una propiedad. También pueden ser definidas de sólo lectura o de sólo escritura.

El bloque de código `get` debe ser omitido para una propiedad de sólo escritura. Para una propiedad de sólo lectura, es el bloque de código `set` el que debe omitirse. Para poner esto en un ejemplo, vamos a añadir a la clase `Persona` una propiedad `contraseña` de sólo escritura y una propiedad `edad` de sólo lectura. La edad se puede deducir directamente de la fecha de nacimiento y la contraseña no debe ser accesible a la lectura desde el exterior de la clase.

```
public int edad  
{  
    get
```



```
        {  
            return DateTime.Now.Year - laFecha_naci.Year;  
        }  
    }  
    public string contraseña  
    {  
        set  
        {  
            laContraseña = value;  
        }  
    }  
}
```

Propiedades indexadas

Las propiedades indexadas permiten un acceso de tipo matriz a grupos de elementos. Las propiedades indexadas, llamadas indexores o propiedades por defecto, difieren ligeramente de las propiedades normales, ya que esperan un parámetro que indique el elemento del grupo al cual hay que acceder. Esta propiedad no tiene nombre (se trata de la propiedad por defecto de la clase). Sin embargo, es posible especificarle uno al añadir el atributo `IndexerName` a la definición de la propiedad. Este nombre no será utilizado por Visual C#, pero sí por otro lenguaje de la plataforma .NET (VB por ejemplo).

Apliquemos esto a nuestro ejemplo añadiendo a la clase `Persona` la lista de los hijos de esta persona y definiendo esta propiedad como propiedad indexada.

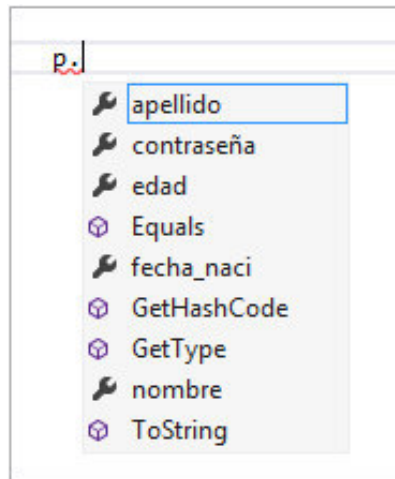
El código de nuestra clase `Persona` se convierte, por lo tanto, en:

```
public class Persona  
{  
    private string elApellido;  
    private string elNombre;  
    private DateTime laFecha_naci;  
    private string laContraseña;  
    private Persona[] losHijos = new Persona[10];  
    public string apellido  
    {  
        get  
        {  
            return elApellido;  
        }  
        set  
        {  
            elApellido=value.ToUpper();  
        }  
    }  
    public string nombre  
    {  
        get  
        {  
            return elNombre;  
        }  
        set  
        {  
            elNombre = value.ToLower(); ;  
        }  
    }  
    public DateTime fecha_naci  
    {  
        get
```

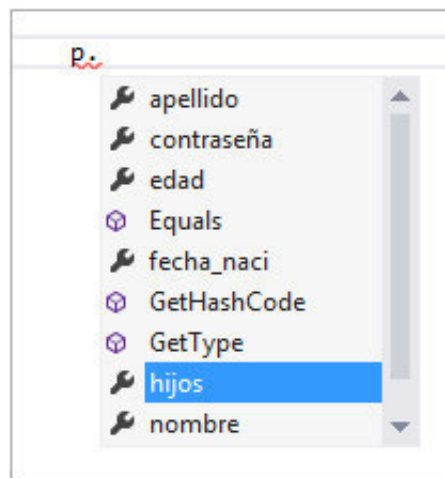
```
        {
            return laFecha_naci;
        }
        set
        {
            if (value.Year >= 1900)
            {
                laFecha_naci = value;
            }
        }
    }
    public int edad
    {
        get
        {
            return DateTime.Now.Year - laFecha_naci.Year;
        }
    }
    public string contraseña
    {
        set
        {
            laContraseña = value;
        }
    }
    public Persona this[int index]
    {
        get
        {
            return losHijos[index];
        }
        set
        {
            losHijos[index] = value;
        }
    }
}
}
```

Cabe observar que tenemos la obligación de crear un nuevo campo en la clase `Persona` con el fin de asegurar el almacenamiento de la lista de los hijos. De momento, este campo está constituido por una matriz de persona, pero podría ser sustituido de manera ventajosa por una estructura más flexible de gestionar, como por ejemplo una colección. La propiedad por defecto espera entonces como parámetro un índice que permite especificar el hijo con el cual deseamos trabajar.

La clase `Persona` vista por Visual C#:



La misma clase vista por Visual Basic:



El siguiente código nos permite probar el correcto funcionamiento de nuestra clase:

```
public static void Main()
{
    Persona p= new Persona();
    Persona hijo1 =new Persona();
    Persona hijo2=new Persona();
    p.apellido = "garcía";
    p.nombre = "pablo";
    p.fecha_naci = new DateTime(1954,12,23);
    hijo1.apellido = "garcía";
    hijo1.nombre = "pascual";
    hijo1.fecha_naci = new DateTime(1979,10,5);
    // también podemos utilizar el apellido del padre para
    // inicializar el nombre del hijo
    hijo2.apellido = p.apellido;
    hijo2.nombre = "marco";
    hijo2.fecha_naci = new DateTime(1982,4,18);
    // podemos asignar un hijo a una persona
    p[0] = hijo1;
    p[1] = hijo2;
    // verifiquemos que nuestros datos son correctos
    Console.WriteLine("Sr {0} {1} nacido el {2} tiene 2 hijos",
        p.apellido,p.nombre,p.fecha_naci);
}
```

```

        Console.WriteLine("{0} {1}", p[0].apellido, p[0].nombre);
        Console.WriteLine("{0} {1}", p[1].apellido, p[1].nombre);
        Console.WriteLine("pulsar una tecla para salir");
        Console.ReadLine();
    }

```

Obtenemos en la consola el siguiente resultado:

```

Sr GARCÍA pablo nacido el 23/12/1954 00:00:00 tiene 2 hijos
GARCÍA pascual
GARCÍA marco
pulsar una tecla para salir

```



Podemos aprovechar para verificar que nuestras reglas relativas al apellido y al nombre se tienen en cuenta: el apellido está en mayúsculas, el nombre está en minúsculas.

d. Creación de métodos

Los métodos son procedimientos o funciones definidos en el interior de una clase. Suelen usarse para manejar los campos y los propiedades de la clase. Para poder utilizar un método de una instancia de clase, basta prefijarlo con el nombre de la instancia en cuya clase ha definido el método.

Añadamos a la clase `Persona` la función `calculo_edad()` y el procedimiento `visualización` insertando el siguiente código:

```

public int calculoEdad()
{
    return DateTime.Now.Year - laFecha_naci.Year;
}
public void visualización()
{
    Console.WriteLine("Sr {0} {1} nacido el {2}", apellido, nombre,
laFecha_naci);
}

```



Debemos observar que, en estas líneas de código, podemos manejar los campos de la clase incluso si se declaran privados, ya que estamos en el interior de la clase. También es posible acceder a los datos de la clase utilizando las propiedades. En este caso, se aplicarán las reglas de gestión relativas al apellido y al nombre.

Podemos modificar nuestro código de prueba para utilizar el procedimiento y la función añadidos a la clase.

```

p.visualización();
Console.WriteLine("tiene 2 hijos", p.apellido, p.nombre, p.fecha_naci);
Console.WriteLine("{0} {1} que tiene {2} años", p[0].apellido, p[0].nombre,
p[0].calculoEdad());
Console.WriteLine("{0} {1} que tiene {2} años", p[1].apellido, p[1].nombre,
p[1].calculoEdad());
Console.WriteLine("teclear una tecla para salir");
Console.ReadLine();

```

Sobrecarga de método

La sobrecarga de método es la creación, dentro de una clase, de un grupo de métodos que tienen un nombre idéntico pero un número de parámetros o tipos de parámetros diferentes. Esto nos permite conservar un nombre coherente para varios métodos cuya meta es similar, pero que para algunos detalles cambian. Los siguientes parámetros no se tienen en cuenta para distinguir dos métodos sobrecargados:

- El nombre de los parámetros.
- El tipo de devolución de una función.
- Los modificadores `out` o `ref` aplicados a los parámetros del método.

Por ejemplo, podemos sobrecargar el método `visualización` de la clase `Persona` para tener en cuenta el idioma en el cual se debe hacer la visualización. El parámetro esperado por el procedimiento permite elegir el idioma.

```
public void visualización(string idioma)
{
    switch (idioma)
    {
        case "es":
            Console.WriteLine("Sr {0} {1} nacido el {2}", apellido, nombre, laFecha_naci);
            break;
        case "en":
            Console.WriteLine("Mr {0} {1} was born {2}", apellido, nombre, laFecha_naci);
            break;
    }
}
```

Sobrescritura de métodos

Las clases derivadas heredan de las propiedades y métodos de su clase base. Usted los puede reutilizar a partir de una subclase sin ninguna modificación. Por el contrario, si el funcionamiento de esta propiedad o método no está adaptado a la nueva clase, tiene la posibilidad de sobrescribirla por una nueva implementación en la clase derivada. En este caso, hay que utilizar la palabra reservada `override` durante la sobrescritura en la clase derivada. También es imperativo que la clase base haya autorizado esta sobrescritura por el uso de la palabra reservada `virtual`. Sin indicación particular, un método o una propiedad no es sobrescribible. En general, la sobrescritura se utiliza para asegurar el polimorfismo entre clases. Por supuesto, los métodos sobrescritos deben tener el mismo nombre, pero también el mismo número y tipo de parámetros que los métodos de la clase base a los cuales se sustituye. Así podemos sustituir en la clase `Asalariado` el método `visualización`.

```
public override void visualización()
{
    Console.WriteLine("Sr {0} {1} nacido el {2} cobra {3} euros al mes",
        apellido, nombre, laFecha_naci, salario);
}
```

Con esta declaración, el método `visualización` de la clase `Persona` ya no es visible para los usuarios de la clase `Asalariado`. Sólo el método `visualización` de la clase `Asalariado` será accesible. No obstante, el código del método `visualización` de la clase `Asalariado` puede tener acceso a este método utilizando la palabra reservada `base`. Por lo tanto, hubiéramos podido escribir para el método `visualización` de la clase `Asalariado`:

```
public override void  visualización()
{
    // llamada del método visualización de la clase Persona
    base.visualización();
    // añadido de las funcionalidades específicas a la clase Asalariado
    Console.WriteLine("cobra {0} euros al mes", elSalario);
}
```

En cuanto un método es declarado como sobrescribible en una clase, lo será para todas sus subclases, sea cual sea el grado de parentesco (clase hija, nieta...). La palabra clave `sealed` se puede utilizar para bloquear esta funcionalidad a partir de un nivel dado. Por ejemplo, en la clase `Asalariado` hubiéramos podido escribir:

```
public sealed override void  visualización()
{
    // llamada del método visualización de la clase Persona
    base.visualización();
    // añadido de las funcionalidades específicas a la clase Asalariado
    Console.WriteLine("cobra {0} euros al mes", elSalario);
}
```

Esta sintaxis cancela, para las subclases de la clase `Asalariado`, la autorización de sobrescritura que estaba definida en la clase `Persona`. Si intentamos sustituir este método en una clase `Jefe` que hereda de `Asalariado`, obtenemos el siguiente mensaje:

```
public override void visualizacion()
{
    'Contabilidad.Jefe.visualizacion()': no puede invalidar el miembro heredado 'Contabilidad.Asalariado.visualizacion()' porque no está marcado como virtual, abstract u override
}
```

Y al revés, podemos exigir en una clase base que una clase heredada sustituya un método definido por aquélla. Este método debe marcarse con la palabra reservada `abstract`. Para tal método, no debe haber implementación sino sólo definición.

```
public abstract string estado_civil();
```

Tal método se llama método abstracto. Exige que la clase en la cual está definida se marque igualmente como abstracta con la palabra reservada `abstract`.

```
public abstract string estado_civil();
'Contabilidad.Jefe.estado_civil()' es abstract pero está incluido en la clase non-abstract 'Contabilidad.Jefe'
```

Ocultación de método

Si en un programa varios elementos comparten nombre, uno de ellos puede ocultar al otro. En tal caso, el que quede oculto no será accesible y el compilador utilizará en su lugar el elemento ocultador. Esta ocultación puede hacerse entre elementos de diferente tipo. Sólo el nombre del elemento se utiliza para asegurar la ocultación. En el momento de ocultar, conviene utilizar la palabra reservada `new`, delante del nombre del miembro que va a realizar la ocultación. Por ejemplo, podemos enmascarar la propiedad `edad` en una clase derivada de la clase `Persona`.

```
public new int edad()
{
    return DateTime.Now.Year - laFecha_naci.Year;
```

}

Para esta clase sólo habrá en adelante un elemento llamado `edad`. Todo elemento de nombre `edad` que pueda existir en la clase base o superiores queda oculto e inaccesible. El único elemento visible es la función `edad` declarada en la clase. La propiedad `edad` heredada de la clase `persona` queda oculta.



Esta técnica se debe utilizar con precaución, ya que en función de la ubicación donde se encuentra una instrucción, el mismo nombre puede hacer referencia a elementos de diferente naturaleza.

Método parcial

Se utilizan los métodos parciales para permitirnos personalizar el código de una clase parcial generada por una herramienta de Visual Studio. Se utilizan principalmente para proveer una notificación de cambio. La herramienta genera únicamente el esqueleto del método y lo llama cuando la notificación debe producirse. El usuario de la clase puede eventualmente definir su propia versión del método y, en este caso, éste será llamado en el lugar de aquel generado automáticamente. Veamos cómo podemos aplicar esto con la clase `Persona`. Primero debemos definir la clase como clase parcial y luego incluir en el interior de la clase un método parcial respetando las siguientes reglas:

- el método debe ser un procedimiento, y no una función,
- el cuerpo del método debe estar vacío (nada de bloque de código),
- el método no debe disponer de modificador de acceso.

```
public string apellido
{
    get
    {
        return elNombre;
    }
    set
    {
        elApellido=value.ToUpper();
        apellidoChanged();
    }
}
...
...
partial void apellidoChanged();
```

Ahora nos queda personalizar esta clase en otro archivo fuente y probar el resultado. Para ello, en un nuevo archivo, añadamos el siguiente código:

```
namespace Contab
{
    partial class Persona
    {
        partial void apellidoChanged()
        {
            Console.WriteLine("se asigna un nuevo apellido");
        }
    }
}
```



```
}
```

Luego intentemos crear una persona y modificar su apellido.

```
Persona p= new Persona();  
p.apellido = "garcía";  
p.nombre = "pablo";  
p.fecha_naci = new DateTime(1954,12,23);  
Console.WriteLine(p.apellido);
```

Al ejecutarlo, tenemos el siguiente resultado:

```
se asigna un nuevo apellido  
García
```

Se trata, en efecto, de nuestra versión del método `apellidoChanged`, que acaba de ejecutarse y, sin embargo, no hemos tocado el código original de la clase `Persona`.

Métodos de extensión

Los métodos de extensión permiten añadir funcionalidades a una clase ya definida sin tener que modificar el código de esta clase. Sólo están escritos en el exterior de la clase y luego se llaman exactamente de la misma manera que los métodos disponibles directamente en la clase. Sin embargo, se deben respetar algunas reglas:

- Pueden ser de tipo procedimiento o función, pero no de tipo propiedad.
- El primer parámetro debe venir precedido de la palabra reservada `this`.
- El tipo del primer parámetro del método determina el tipo extendido por este método.
- En el momento de la ejecución, este primer parámetro representa la instancia de la clase sobre la cual se llama el método.
- Se deben definir en una clase `static`.
- Ellos mismos deben ser `static`.

En el ejemplo siguiente, añadimos un método a la clase `Persona`.

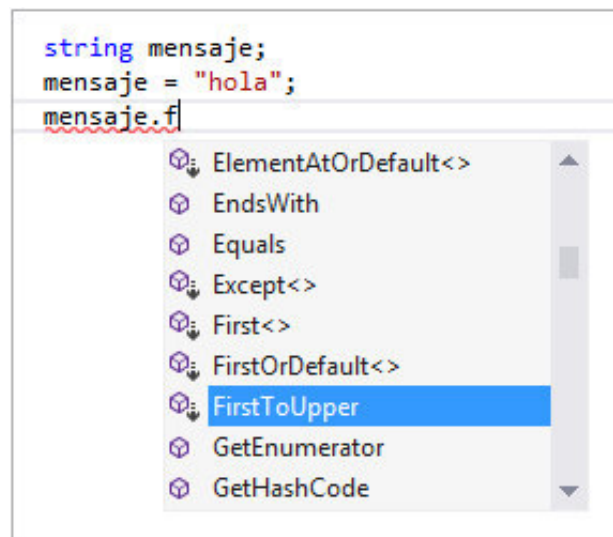
```
static class Extensions  
{  
    public static void presentacion(this Persona p)  
    {  
        Console.WriteLine("apellido: {0}", p.apellido);  
        Console.WriteLine("nombre: {0}", p.nombre);  
        Console.WriteLine("fecha de nacimiento: {0}", p.fecha_naci);  
    }  
}
```

Los métodos de extensión también se pueden definir para los tipos básicos del Framework, como por ejemplo la clase `string`. El siguiente código añade a la clase `string` un método que permite convertir el primer carácter de una cadena en mayúscula.

```
public static string FirstToUpper(this String s)  
{  
    if ((s == null) || (s.Length == 0))  
    {
```

```
        return s;
    }
    else if (s.Length == 1)
    {
        return s.ToUpper();
    }
    else
    {
        return s.Substring(0, 1).ToUpper() + s.Substring(1, s.Length - 1);
    }
}
```

Si utilizamos luego una variable de tipo `string`, nuestro método se hace disponible e incluso es propuesto por IntelliSense.



Observe el icono diferente utilizado para diferenciar un método de extensión de un método normal de la clase.

e. Constructores y destructores

Los constructores son métodos particulares de una clase por diferentes aspectos. El constructor es un método que siempre lleva el mismo nombre que la propia clase. No devuelve ningún tipo, ni siquiera `void`. Nunca se le llama de manera explícita en el código, sino implícita, en la creación de una instancia de la clase. Como para un método clásico, un constructor puede esperar parámetros. El constructor de una clase que no espera parámetro alguno es designado como el constructor por defecto de la clase. El papel del constructor consiste principalmente en la inicialización de los campos de una instancia de clase. Los constructores también pueden ser sobrecargados.

Añadamos a la clase `Persona` unos constructores.

```
public Persona()
{
    elApellido = "";
    elNombre = "";
    laContraseña = "";
}
public Persona(string apellido, string nombre, string pwd)
{
    elApellido = apellido;
```

```
        elNombre = nombre;
        laContraseña = pwd;
    }
```

Cuando creamos una clase derivada, también puede disponer de sus propios constructores. Si añadimos en la clase derivada un constructor por defecto, debemos seguir algunas reglas:

- Si el constructor de una clase derivada no invoca de forma explícita al constructor de su clase base (con la ayuda de la palabra reservada `base`), el constructor por defecto, si existe, lo hará de manera implícita.
- Si una clase base no ofrece constructor por defecto, la clase derivada debe hacer una llamada explícita al constructor de la clase base usando la palabra reservada `base`.
- En nuestro caso, el constructor por defecto de la clase `Asalariado` puede tener la siguiente forma.

```
public Asalariado():base()
{
    elSalario = 0;
}
```

El comportamiento será el mismo si el constructor está definido de la siguiente manera.

```
public Asalariado()
{
    elSalario = 0;
}
```

Añadir un constructor sobrecargado en la clase `Asalariado` también se puede hacer de la siguiente forma.

```
public Asalariado(string apellido, string nombre, string pwd, decimal salario)
{
    elApellido = apellido;
    elNombre = nombre;
    laContraseña = pwd;
    elSalario = salario;
}
```

También se puede optimizar utilizando la siguiente sintaxis, que llama a un constructor de la clase base (`Persona`).

```
public Asalariado(string apellido, string nombre, string pwd, decimal salario)
    :base(apellido,nombre,pwd)
{
    elSalario = salario;
}
```

Los destructores son otros métodos particulares de una clase. Como los constructores, se llaman de manera implícita, pero únicamente durante la destrucción de una instancia de clase. La firma del destructor se impone. El destructor lleva el mismo nombre que la clase pero va precedido del signo `~` y no toma ningún parámetro. Debido a esta firma impuesta, sólo puede haber un único destructor para esta clase, y por lo tanto ninguna sobrecarga posible para los destructores.

La declaración de un destructor es, entonces, la siguiente:

```
~Asalariado()
{
}
}
```

El código presente en el destructor debe permitir la liberación de los recursos utilizados por la clase. Por ejemplo, se puede encontrar en ella código que cierra un archivo abierto por la clase o el cierre de una conexión a un servidor de base de datos.

➤ Veremos en detalle en el apartado Destrucción de una instancia las circunstancias en las cuales se llama al destructor.

f. Miembros compartidos

Los miembros compartidos son campos, propiedades o métodos a los que pueden acceder todas las instancias de una clase. Se habla también de miembros estáticos.

Son muy útiles cuando tiene que gestionar, en una clase, datos que no son específicos de una instancia de clase, sino de la propia clase. Por oposición a los miembros de instancia, para los cuales existe un ejemplar por instancia de la clase, los miembros compartidos existen en un ejemplar único. La modificación del valor de un miembro de instancia sólo modifica el valor para esta instancia de clase, mientras que la modificación del valor de un miembro compartido modifica el valor para todas las instancias de la clase. Los miembros compartidos son asimilables a variables globales en una aplicación. Sólo se pueden utilizar en el código haciendo referencia a ellos con el nombre de la clase.

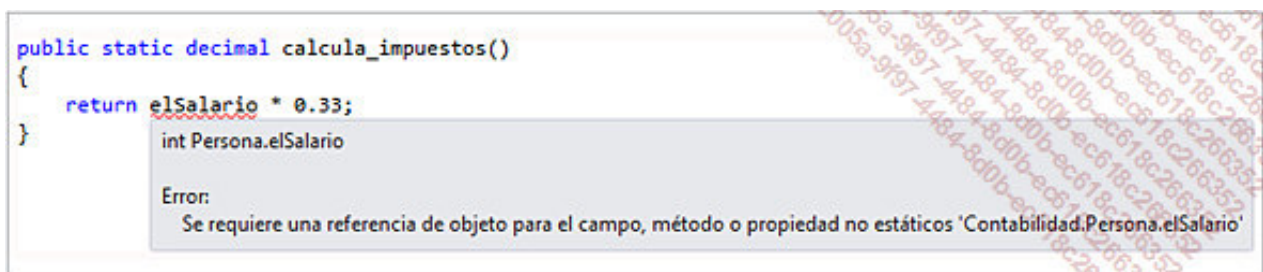
La utilización de un miembro compartido mediante una instancia de clase está prohibido.



```
Persona p;
p = new Persona();
p.test = 1;
```

No se puede obtener acceso al miembro 'ConsoleApplication1.Persona.test' con una referencia de instancia; califíquelo con un nombre de tipo en su lugar

Los métodos compartidos siguen las mismas reglas y pueden servir para la creación de librerías de funciones. El ejemplo clásico es la clase `Math`, que contiene numerosas funciones compartidas. Los métodos compartidos poseen, no obstante, una limitación, ya que sólo pueden utilizar variables locales u otros miembros compartidos de la clase. Nunca deben utilizar miembros de instancia de una clase, ya que es posible que el método sea utilizado sin que exista una instancia de la clase. El compilador verificará este tipo de error.



```
public static decimal calcula_impuestos()
{
    return elSalario * 0.33;
}
```

Error:
Se requiere una referencia de objeto para el campo, método o propiedad no estáticos 'Contabilidad.Persona.elSalario'

Los miembros compartidos deben declararse con la palabra reservada `static`. Como con cualquier otro miembro de clase, puede definir la visibilidad. En cambio, una variable local a un procedimiento o función no se puede compartir.

2. Utilización de una clase

La utilización de una clase en una aplicación pasa por dos etapas.

- La declaración de una variable que permite el acceso al objeto.
- La creación del objeto.

a. Creación de una instancia

Las instancias son variables del tipo referencia. Difieren de las variables clásicas por el hecho de que la instancia no contiene directamente datos, sino una referencia de la ubicación en la memoria de la máquina donde se encuentran los datos. Al igual que el resto de las variables, debe ser declarada antes de su utilización. La declaración se efectúa de manera idéntica a la de una variable clásica (int u otra).

```
Persona p;
```

A esta altura la variable existe, pero no hace referencia a una ubicación válida. Contiene el valor `null`.

La segunda etapa consiste realmente en crear la instancia de la clase. La palabra reservada `new` se utiliza a este efecto. Espera como parámetro el nombre de la clase de la cual está encargado de crear una instancia. El operador `new` hace una petición al sistema para obtener la memoria necesaria para almacenar la instancia de la clase; luego inicializa la variable con esta dirección de memoria. El constructor de la clase es llamado para inicializar la nueva instancia creada.

```
p = new Persona ();
```

Las dos operaciones pueden ser combinadas en una sola línea.

```
Persona p = new Persona();
```

En este caso se llama al constructor por defecto. Para utilizar otro constructor, debe especificar una lista de parámetros y, en función del número y del tipo de parámetros, el operador `new` llama al constructor correspondiente.

```
Persona p1 = new Persona("garcía", "pedro", "secreto");
```

b. Inicialización de una instancia

Después de haber creado una instancia de clase, puede inicializar los miembros de ésta por medio de propiedades de clase. Es posible combinar estas dos etapas en una sola. Para ello, durante la creación de la instancia, hay que facilitar una lista de propiedades y valores que hay que asignar a estas propiedades. A continuación tenemos la sintaxis exacta que se debe utilizar:

```
Persona p2 = new Persona
{
    apellido = "García",
    nombre = "Pedro",
    contraseña = "secreto"
};
```

No hay limitación sobre el número de propiedades inicializadas ni tampoco en el orden de aparición

de las propiedades en la lista de inicialización. Esta única línea de código es el equivalente de la siguiente sintaxis, menos condensada y más tradicional:

```
Persona p3;  
p3 = new Persona();  
p3.apellido = "García";  
p3.nombre = "Pedro";  
p3.contraseña = "secreto";
```

c. Destrucción de una instancia

La destrucción de una instancia de clase es automática en una aplicación. El Common Language Runtime vigila a intervalos regulares que todas las instancias de clases creadas en la aplicación sean accesibles. Es decir, que todavía exista en la aplicación una variable o una propiedad que permita el acceso a esta instancia. Si no se encuentra ningún medio de acceder a esta instancia, entonces el objeto queda marcado como huérfano. Cuando la memoria libre de la aplicación mengua, el Garbage Collector (recolector de basura) interviene y elimina los objetos huérfanos. Es durante esta eliminación cuando los destructores de cada uno de los objetos son llamados. No existe manera de precipitar las cosas pidiendo la eliminación inmediata de la memoria de una instancia particular de clase. Sin embargo, es posible forzar el garbage collector a intervenir con la siguiente línea de código.

```
GC.Collect();
```

En este caso, el Garbage Collector interviene para todas las instancias huérfanas. El inconveniente de esta solución es que es relativamente costosa en recursos para recuperar a veces sólo algunas decenas de bytes de memoria, incluso ninguna si no hay instancia de clase por suprimir.

A veces esta situación es problemática cuando el objeto utiliza un recurso externo, como por ejemplo una conexión hacia un servidor de base de datos. Si el cierre de la conexión está previsto en el destructor de la clase, puede pasar mucho tiempo entre el momento en el que el objeto se hace inaccesible y la llamada a su destructor.

Para paliar este problema, es posible poner en marcha otra solución. El código encargado de la liberación de los recursos está ubicado en otro método, y este método se llama de manera explícita en el código. Este método se suele llamar `Dispose`. Para asegurarse de que los recursos están efectivamente liberados, también puede prever una llamada a este método en el destructor de la clase.

Otro problema puede surgir entonces: si el método fue llamado explícitamente en el código de la aplicación, lo será de nuevo de manera implícita cuando el Garbage Collector entre en acción. Por lo tanto, debe hacer de tal manera que el código de este método `Dispose` pueda ejecutarse dos veces sin causar errores. También puede indicar al Garbage Collector que no debe ejecutar el destructor de esta instancia de clase. Para ello, en el método `Dispose`, debe avisarle de que el trabajo de «limpieza» ya está realizado llamando al método `SuppressFinalize`. El código del método `Dispose` y del destructor debe tener, pues, la siguiente forma:

```
public void Dispose()  
{  
    //insertar el código cargado de la liberación de los recursos  
    //  
    //pide al garbage collector no llamar al destructor  
    GC.SuppressFinalize(this);  
}  
~Asalariado()  
{  
    Dispose();  
}
```

}

d. Enlace tardío, enlace temprano

El compilador Visual C# efectúa una operación llamada enlace cuando se asigna un objeto a una variable. Este enlace se llama temprano cuando se crea la variable a partir de una clase específica. Esta funcionalidad permite al compilador efectuar optimizaciones sobre el código generado. La asignación de un objeto también puede realizarse a una variable de tipo `Object`. Este tipo de variable es capaz de referenciar a cualquier otro tipo de clase. En este caso, el enlace se llama tardío, ya que el tipo real del objeto sólo se descubrirá en el momento de la ejecución de la aplicación. Se debe evitar esta técnica, ya que genera un código menos eficaz y sobre todo no permite beneficiarse de la complementación automática del código en el editor ni tampoco de la ayuda dinámica. En efecto, en este caso Visual C# no puede determinar el tipo real del objeto con el que se trabaja.

Sin embargo, algunas funciones devuelven un tipo `Object`, pero para poder manejarlo, conviene tomar algunas precauciones. La primera solución consiste en utilizar sólo miembros de la clase `Object` con el objeto devuelto por la función. Esta solución es relativamente limitada en cuanto a las funcionalidades disponibles.

La segunda solución consiste en asignar a una variable de un tipo particular el valor devuelto por la función. Esta solución permite utilizar todas las funcionalidades del objeto devuelto por la función. Sin embargo, hay que estar seguro de que el objeto devuelto es realmente una instancia de la clase con la que se desea trabajar. De hecho, el compilador se encargará de recordárnoslo.

```
public Object getHijo(int index)
{
    return this.losHijos[index];
}
```



Por lo tanto, debemos asegurarnos del tipo del objeto devuelto y pedir la conversión explícita. Podemos obtener el nombre del tipo del objeto y efectuar una comparación de cadena de caracteres.

```
Persona hijo;
if (p.getHijo(0).GetType().Name.Equals("Persona"))
{
    hijo = (Persona)p.getHijo(0);
}
```

Esta solución funciona, pero comporta el riesgo de ortografiar mal el nombre de la clase durante la comparación. El operador `is` ... está más adaptado a esta situación.

```
if (p.getHijo(0) is Persona)
{
    hijo = (Persona)p.getHijo(0);
}
```


Observe que la conversión explícita no cambia el tipo del objeto en memoria, sino que permite simplemente verlo de otra manera. Si por ejemplo tenemos en memoria una instancia de la clase `Asalariado`, la conversión explícita nos permite verla como un `Object`, una `Persona` o un `Asalariado`, pero seguirá siendo una instancia de la clase `Asalariado`.

3. Herencia

La herencia es una potente funcionalidad de un lenguaje orientado a objetos, pero a veces puede usarse mal. Hay dos tipos de relaciones que se pueden establecer entre dos clases. Podemos tener la relación «es un tipo de» y la relación «conciene a». Se debe considerar la relación de herencia cuando la relación «es un tipo de» se puede aplicar entre dos clases. Tomemos un ejemplo con tres clases: `Persona`, `Cliente`, `Pedido`.

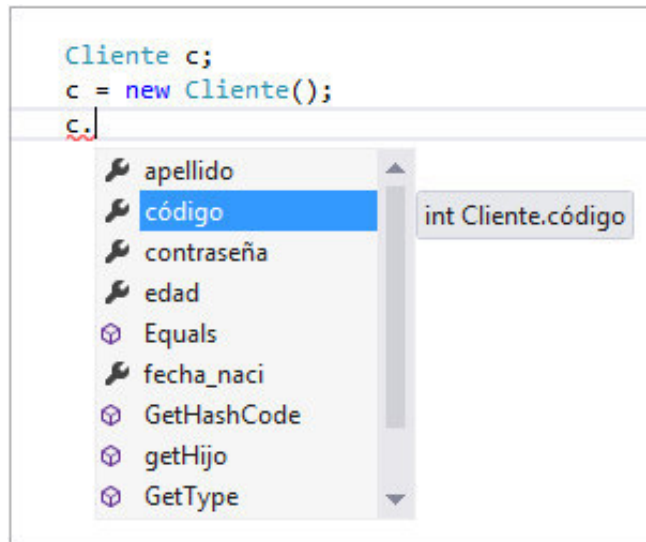
Probemos las relaciones para cada una de las clases.

- Un pedido es un tipo de cliente.
- Un pedido es un tipo de persona.
- Un cliente es un tipo de pedido.
- Un cliente es un tipo de persona.
- Una persona es un tipo de cliente.
- Una persona es un tipo de pedido.

Entre todos los intentos, sólo uno nos parece lógico: un cliente es un tipo de persona. Por lo tanto, podemos considerar una relación de herencia entre estas dos clases. La puesta en práctica es muy simple a nivel del código, ya que en la declaración de la clase basta con especificar el carácter : seguido del nombre de la clase de la cual se desea heredar. Al no aceptar Visual C# la herencia múltiple, sólo usted puede especificar un único nombre de la clase base.

```
class Cliente:Persona
{
    protected int elcodigo;
    public int codigo
    {
        get
        {
            return elcodigo;
        }
        set
        {
            elcodigo = value;
        }
    }
}
```

Luego se puede utilizar la clase, y ésta propone todas las funcionalidades definidas en la clase `Cliente` más las heredadas de la clase `Persona`.



a. base y this

Es legítimo querer modificar a continuación el funcionamiento de ciertos métodos heredados para adaptarlos a la clase `Cliente`. Por ejemplo, se puede sustituir el método `visualización` para tener en cuenta los nuevos campos disponibles en la clase.

```
public void visualizacion()
{
    Console.WriteLine("Sr {0} {1} nacido el {2}", apellido, nombre, laFecha_naci);
    Console.WriteLine("Código cliente: {0}", elcodigo);
}
```

Este código funciona muy bien, pero no respeta uno de los principios de la programación orientada a objetos que requiere que se reutilice al máximo lo que ya existe. En nuestro caso, ya tenemos una porción de código encargada de la visualización del apellido, nombre y fecha de nacimiento de

Subir

Los tipos genéricos

Los tipos genéricos son elementos de un programa que se adaptan sobre la marcha para realizar la misma funcionalidad que otros tipos de datos. Cuando crea un elemento genérico no necesita diseñar una versión diferente para cada tipo de dato con el cual desea realizar una funcionalidad.

Para realizar una analogía con un objeto corriente, vamos a tomar el ejemplo de un destornillador. En función del tipo de tornillo que quiera utilizar, puede emplear un destornillador específico para este tipo de tornillo (plano, cruciforme...). Una técnica a menudo utilizada por un manitas avezado consiste en adquirir un destornillador universal con múltiples extremos.

En función del tipo de tornillo, elige el extremo adaptado. El resultado final es el mismo que si dispone de varios destornilladores: puede atornillar y desatornillar.

Cuando utiliza un tipo genérico, lo configura con un tipo de datos. Esto permite al código adaptarse automáticamente y realizar la misma acción independientemente del tipo de datos. Una alternativa podría ser la utilización del tipo universal `Object`. La utilización de los tipos genéricos presenta varias ventajas respecto a esta solución:

- Impone la verificación de los tipos de datos en el momento de la compilación y evita las verificaciones que deben efectuarse manualmente con la utilización del tipo `Object`.
- Evita las operaciones de conversión del tipo `Object` hacia un tipo más específico y a la inversa, ya que son consumidoras de recursos.
- Evita la utilización del enlace `tardío`, ineludible con el tipo `Object`.
- La escritura del código es facilitada por el entorno de desarrollo gracias a IntelliSense.
- Favorece la escritura de algoritmos independientes de los tipos de datos.

Los tipos genéricos pueden imponer, sin embargo, ciertas restricciones relativas al tipo de dato utilizado. Por ejemplo, pueden imponer que el tipo utilizado implemente una o varias interfaces, que sea un tipo de referencia o posea un constructor por defecto.

Es importante entender correctamente algunos términos utilizados con los genéricos:

El tipo genérico

Es la definición de una clase, estructura, interfaz o procedimiento para el cual puede especificar al menos un tipo de datos en el momento de su declaración.

El tipo parámetro

Es la ubicación reservada para el tipo de datos en la declaración del tipo genérico.

El tipo argumento

Es el tipo de datos que sustituye al tipo de parámetro durante la construcción de un tipo a partir de un tipo genérico.

Las restricciones

Son las condiciones que usted impone al tipo argumento que establezca.

El tipo construido

Es la clase, interfaz, estructura o procedimiento declarada a partir de un tipo genérico para el cual especificó tipos de argumento.

1. Las clases genéricas

Una clase que espera un tipo de parámetro se llama clase genérica. Usted puede generar este tipo de clases indicando a la clase genérica un tipo de argumento para cada uno de estos tipos de parámetro.

a. Definición de una clase genérica

Puede definir una clase genérica que facilite las mismas funcionalidades sobre diferentes tipos de datos. Para ello, debe facilitar uno o varios tipos de parámetro en la definición de la clase. Tomemos el ejemplo de una clase capaz de gestionar una lista de elementos con las siguientes funcionalidades:

- Añadir un elemento.
- Suprimir un elemento.
- Desplazarse al primer elemento.
- Desplazarse al último elemento.
- Desplazarse al elemento siguiente.
- Desplazarse al elemento anterior.
- Obtener el número de elementos.

Primero debemos definir la clase como una clase ordinaria.

```
class ListaGenerica
{
}
```

La transformación de esta clase en clase genérica se efectúa añadiendo un tipo de parámetro inmediatamente después del nombre de la clase.

```
class ListaGenerica<tipoDeDato>
{
}
```

Si es necesario definir varios tipos, deben separarse con comas.

Durante la definición de una clase genérica, puede aplicar restricciones a los tipos de parámetros que se pueden utilizar en el momento de usar la clase genérica. Si alguien intentase instanciar esta clase con un tipo de argumento que infringe esta restricción, el sistema lanzaría un error de compilación. Estas limitaciones, también llamadas restricciones, se ubican en el tipo de parámetro de la clase genérica. Las restricciones se especifican mediante la palabra reservada `where`. Hay seis tipos de restricciones diferentes que pueden aplicarse sobre un tipo de parámetro, con la posibilidad de combinarlas, por supuesto.

where tipoDeDato: struct

Esta restricción impone que el tipo de parámetro sea un tipo por valor, y no un tipo de referencia. Además, el tipo de parámetro no debe ser un tipo nullable.

```
public class ListaGenerica<tipoDeDato>
    where tipoDeDato:struct
{
    ...
}
```

```
static ListaGenerica<int> listaEnteros = new ListaGenerica<int>(5);
static ListaGenerica<Cliente> listaClientes = new ListaGenerica<Cliente>(5);
```

El tipo 'Contabilidad.Cliente' debe ser un tipo de valor distinto de null para poder utilizarlo como parámetro 'tipoDeDato' en el tipo o método genérico 'Contabilidad.ListaGenerica<tipoDeDato>'.

where tipoDeDato: class

Esta restricción impone que el tipo de parámetro sea un tipo de referencia: clase, interfaz, matriz o delegado.

```
public class ListaGenerica<tipoDeDato>
    where tipoDeDato:class
{
    ...
}
```

```
static ListaGenerica<int> listaEnteros = new ListaGenerica<int>(5);
static ListaGenerica<Cliente> listaClientes = new ListaGenerica<Cliente>(5);
```

El tipo 'int' debe ser un tipo de referencia para poder utilizarlo como parámetro 'tipoDeDato' en el tipo o método genérico 'Contabilidad.ListaGenerica<tipoDeDato>'.

where tipoDeDato: new()

Esta restricción impone la presencia de un constructor public y sin parámetro en el tipo de parámetro. Si se utiliza esta restricción conjuntamente con otras restricciones, debe ser en este caso la última de la lista. Las restricciones deben ir separadas con comas en la lista.

```
public class ListaGenerica<tipoDeDato>
    where tipoDeDato: class,new()
{
    ...
}
```

```
static ListaGenerica<Cliente> listaClientes = new ListaGenerica<Cliente>(5);
```

class Contabilidad.Cliente

Error:

'Contabilidad.Cliente' debe ser un tipo no abstracto con un constructor público sin parámetros para poder usarlo como parámetro 'tipoDeDato' en el tipo o método genérico 'Contabilidad.ListaGenerica<tipoDeDato>'.

where tipoDeDato: nombre de clase

Esta restricción exige que el tipo de parámetro sea la clase indicada o una de sus subclases.

```
public class ListaGenerica<tipoDeDato>
    where tipoDeDato: Cliente
{
    ...
}
```

```
}
```

where tipoDeDato: interfaz1,interfaz2...

Esta restricción exige que el tipo de parámetro implemente la interfaz o las interfaces indicadas.

```
public class ListaGenerica<tipoDeDato>
    where tipoDeDato: Comparable
{
    ...
}
```

En el código de la clase, cada miembro que debe ser del tipo del parámetro debe definirse con el tipo `tipoDeDato`, en nuestro caso. Veamos ahora el código completo de la clase.

```
public class ListaGenerica<tipoDeDato>
{
    // matriz para almacenar los elementos de la lista
    private tipoDeDato[] lista;
    // puntero de posición en la lista
    private int posicion;
    // puntero para el añadido de un nuevo elemento
    private int elementoSiguiente;
    // número de elementos de la lista
    private int numElementos;
    // dimensión de la lista
    private int tamaño;
    // indica si la lista está llena
    private bool completa = false;
    // constructor con un parámetro que permite dimensionar la lista
    public ListaGenerica(int tamaño)
    {
        lista = new tipoDeDato[tamaño];
        this.tamaño = tamaño;
    }
    public void añadido(tipoDeDato elemento)
    {
        // se verifica si la lista está completa antes
        // de añadir un elemento
        if (!completa)
        {
            lista[elementoSiguiente] = elemento;
            numElementos = numElementos + 1;
            completa = (numElementos == tamaño);
            // si la lista no está completa se posiciona el puntero
            // para el añadido del elemento siguiente
            if (!completa)
            {
                elementoSiguiente = elementoSiguiente + 1;
            }
        }
    }
    public void suprime(int index)
    {
        int i;
        // se verifica si el índice no es superior al número de elementos
        // si el índice no es inferior a 0
        if (index >= numElementos || index < 0)
```

```
{
    return;
}
// se desplazan los elementos de una posición hacia arriba
for (i = index; i <= numElementos - 2; i++)
{
    lista[i] = lista[i + 1];
}
// se posiciona el puntero para el añadido de un nuevo elemento
elementoSiguiente = elementoSiguiente - 1;
// se actualiza el número de elementos
numElementos = numElementos - 1;
}
public int tamañoLista
{
    get
    {
        return numElementos;
    }
}
public tipoDeDato primero()
{
    if (numElementos == 0)
    {
        throw new Exception("lista vacía");
    }
    // se desplaza el puntero sobre el primer elemento
    posicion = 0;
    return lista[0];
}
public tipoDeDato último()
{
    if (numElementos == 0)
    {
        throw new Exception("lista vacía");
    }
    // se desplaza el puntero sobre el último elemento
    posicion = numElementos - 1;
    return lista[posicion];
}
public tipoDeDato siguiente()
{
    if (numElementos == 0)
    {
        throw new Exception("lista vacía");
    }
    // se verifica si no estamos al final de la lista
    if (posicion == numElementos - 1)
    {
        throw new Exception("ningún elemento siguiente");
    }
    // se desplaza el puntero sobre el elemento siguiente
    posicion = posicion + 1;
    return lista[posicion];
}
public tipoDeDato anterior()
{
    if (numElementos == 0)
    {
        throw new Exception("lista vacía");
    }
}
```



```

        // se verifica si no estamos sobre el primer elemento
        if (posicion == 0)
        {
            throw new Exception("ningún elemento anterior");
        }
        // nos desplazamos sobre el elemento anterior
        posicion = posicion - 1;
        return lista[posicion];
    }
}

```

b. Utilización de una clase genérica

Para poder utilizar una clase genérica, debe generar primero una clase construida facilitando un tipo de argumento para cada uno de estos tipos de parámetro. A continuación puede instanciar la clase construida por uno de los constructores disponibles. Vamos a utilizar la clase diseñada anteriormente para trabajar con una lista de enteros.

```
static ListaGenerica<int> lista = new ListaGenerica<int>(5);
```

Esta declaración permite instanciar una lista de cinco enteros. Los métodos de la clase están entonces disponibles.

```

lista.añadido(10);
lista.añadido(11);
lista.añadido(12);
lista.añadido(13);
lista.añadido(14);
lista.añadido(15);

```

El compilador comprueba que utilizamos nuestra clase correctamente, en particular verificando los tipos de datos que le pasamos.



A continuación tenemos el código de una pequeña aplicación que permite probar el funcionamiento correcto de nuestra clase genérica:

```

static class testGenerico
{
    static ListaGenerica<int> lista = new ListaGenerica<int>(5);
    public static void main()
    {
        lista.añadido(10);
        lista.añadido(11);
        lista.añadido(12);
        lista.añadido(13);
        lista.añadido(14);
        lista.añadido(15);
    }
}

```

```

/*    lista.añadido("primera");
    lista.añadido("segunda");
    lista.añadido("tercera");
    lista.añadido("cuarta");
    lista.añadido("quinta");*/
    menu();
}
public static void menu()
{
    char eleccion='\0';
    Console.SetCursorPosition(1, 24);
    Console.WriteLine("p (primera) < (anterior) >(siguiente) d (ultima)
f (fin)");
    while (eleccion != 'f') {
        eleccion = Console.ReadKey().KeyChar;
        Console.Clear();
        Console.SetCursorPosition(1, 1);
        try
        {
            switch (eleccion)
            {
                case 'p':
                    Console.WriteLine("la primera {0}", lista.primera());
                    break;
                case '<':
                    Console.WriteLine("la anterior {0}", lista.anterior());
                    break;
                case '>':
                    Console.WriteLine("la siguiente {0}", lista.siguiente());
                    break;
                case 'd':
                    Console.WriteLine("la última {0}", lista.ultima());
                    break;
            }
        }
        catch (Exception e)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine(e.Message);
            Console.ForegroundColor = ConsoleColor.White;
        }
        Console.SetCursorPosition(1, 24);
        Console.WriteLine("p (primera) < (anterior) >(siguiente) d (ultima)
f (fin)");
    }
}
}

```

Podemos verificar también que nuestra clase funciona sin problema si le pedimos trabajar con cadenas de caracteres.

```

static ListaGenerica<String> lista = new ListaGenerica<String>(5);
public static void main()
{
    lista.añadido("primera");
    lista.añadido("segunda");
    lista.añadido("tercera");
    lista.añadido("cuarta");
    lista.añadido("quinta");
    menu();
}

```

}

2. Interfaces genéricas

De manera totalmente similar a lo que acabamos de ver respecto a las clases genéricas, también es posible diseñar interfaces genéricas. Utilizan las mismas técnicas de diseño que las clases genéricas.

a. Definición de una interfaz genérica

La definición de una interfaz genérica es similar en todo a la declaración de una interfaz normal, excepto en el hecho de que se debe especificar al menos un tipo de parámetro después del nombre de la interfaz. La interfaz Comparable definida anteriormente puede tomar, por lo tanto, la forma siguiente:

```
interface ComparableGenerica<tipoDeDatos>
{
    int compare(tipoDeDatos o1);
}
```

El tipo de parámetro se puede utilizar en la firma de los métodos exigidos por la interfaz.

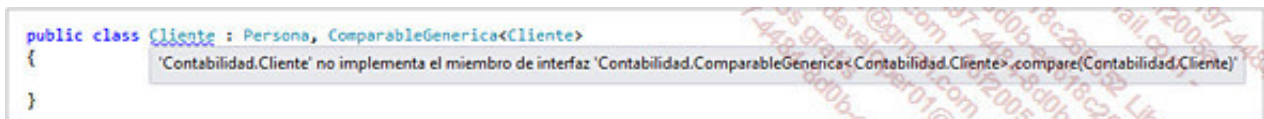
b. Utilización de una interfaz genérica

De la misma manera que para una interfaz normal, una interfaz genérica debe ser implementada por una clase. Durante la declaración de la clase, el tipo o los tipos de parámetros deben sustituirse por uno o más tipos de argumentos.

La utilización de nuestra interfaz genérica puede tomar la forma siguiente:

```
public class Cliente:Persona,ComparableGenerica<Cliente>
{
    ...
}
```

El compilador exige ahora que el método o los métodos descritos en la interfaz estén realmente disponibles en la clase.



```
public class Cliente : Persona, ComparableGenerica<Cliente>
{
    ...
}
```

También hay que observar que el compilador haya tenido en cuenta el tipo de argumento utilizado para la declaración de la clase, ya que nos reclama la presencia de una función llamada `compare` y espera como parámetro un objeto de tipo `Cliente` (el tipo de argumento especificado en el momento de la declaración de la clase).

De hecho, se puede simplificar mucho el código de la función `compare` respecto al de la versión no genérica de la clase, ya que ya no necesita efectuar una operación de conversión explícita antes de utilizar el parámetro recibido por la función.

```
public int compare(Cliente c)
{
```

```
        return elApellido.CompareTo(c.elApellido);  
    }  
}
```

3. Procedimientos y funciones genéricos

Los procedimientos o funciones genéricos son métodos definidos con al menos un tipo parámetro. Esto permite al código que llama especificar el tipo de datos que necesita a cada llamada del procedimiento o función. Sin embargo, se puede utilizar tal método sin indicar información para el tipo de argumento. En este caso, el compilador intenta determinar el tipo en función de los argumentos pasados al método. Sin embargo, se debe utilizar esta solución con precaución, ya que si el compilador no puede determinar el tipo de los argumentos, genera un error de compilación.

a. Creación de un procedimiento o función genérica

La declaración de un procedimiento o función genérica debe contener al menos un tipo de parámetro. Se define este tipo de parámetro con un identificador. Luego se utiliza este identificador en el resto del código cada vez que se necesita utilizar el tipo de parámetro.

Vamos a crear una función genérica capaz de buscar un elemento particular en una matriz de cualquier tipo. Esta función va a utilizar un tipo de parámetro indicando la naturaleza de los elementos presentes en la matriz. Para poder buscar un elemento en la matriz, deberemos compararlo con los presentes en todas las casillas de la matriz. Para garantizar que esta comparación sea posible, añadimos una restricción en el tipo de parámetro: debe implementar la interfaz `IComparable` con el fin de asegurar que el método `CompareTo` utilizado en la función esté disponible para cada elemento de la matriz. La declaración de la función toma la forma siguiente:

```
public static int busquedaGenerica<tipoDato>(tipoDato[] matriz, tipoDato  
elementoBusqueda) where tipoDato: IComparable
```

Después de haber comprobado que la matriz contiene al menos un elemento, debemos comparar el elemento buscado con aquel presente en cada casilla de la matriz. Si hay igualdad, la función devuelve el índice donde el elemento ha sido encontrado; si no, la función devuelve -1. Para efectuar la comparación, utilizaremos la función `CompareTo` de cada elemento de la matriz.

```
public static int busquedaGenerica<tipoDato>(tipoDato[] matriz, tipoDato  
elementoBusqueda) where tipoDato: IComparable  
{  
    //test si la matriz tiene más de una dimensión  
    if (matriz.Rank > 1)  
    {  
        return -1;  
    }  
    // test si la matriz está vacía  
    if (matriz.Length == 0)  
    {  
        return -1;  
    }  
    for (int i = 0; i <= matriz.GetUpperBound(0); i++)  
    {  
        if (matriz[i].CompareTo(elementoBusqueda) == 0)  
        {  
            return i;  
        }  
    }  
    return -1;  
}
```

b. Utilización de un procedimiento o función genérica

La utilización de un procedimiento o función genérica es idéntica a la de un procedimiento o función clásica, excepto por la necesidad de especificar un tipo de argumento para el tipo o los tipos parámetro.

El siguiente código permite probar el correcto funcionamiento de nuestra función.

```
public static void main()
{
    int[] t = { 12, 45, 85, 47, 62, 95, 81 };
    int resultado;
    resultado = busquedaGenerica<int>(t, 47);
    if (resultado == -1)
    {
        Console.WriteLine("valor no encontrado");
    }
    else
    {
        Console.WriteLine("valor encontrado en la posición {0}", resultado);
    }
    Console.ReadLine();
    string[] s = { "uno", "dos", "tres", "cuatro", "cinco" };
    resultado = busquedaGenerica<string>(s, "seis");
    if (resultado == -1)
    {
        Console.WriteLine("valor no encontrado");
    }
    else
    {
        Console.WriteLine("valor encontrado en la posición {0}", resultado);
    }
    Console.ReadLine();
}
```

4. Delegados genéricos

Como cualquier otro elemento, un delegado puede definir unos tipos de parámetros en su declaración. Durante la utilización del delegado, hay que facilitar tipos de argumentos para cada uno de sus tipos de parámetro. El siguiente extracto de código declara un delegado genérico.

```
public delegate int comparacion<tipoDeDatos>(tipoDeDatos
p1, tipoDeDatos p2);
```

Luego se puede utilizar este delegado en la declaración de un método facilitando un tipo de argumento para cada uno de sus parámetros.

```
public static void ordenacion(Cliente[] matriz, comparacion<Cliente> comparador)
{
    Cliente o;
    for (int i = 0; i < matriz.Length - 1; i++)
    {
        for (int j = i + 1; j < matriz.Length; j++)
        {
            if (comparador.Invoke(matriz[j], matriz[i]) < 0)
            {
                o = matriz[j];
            }
        }
    }
}
```

```

        matriz[j] = matriz[i];
        matriz[i] = o;
    }
}
}

```

Para poder llamar a esta función, hay que facilitarle ahora como primer parámetro una matriz de clientes y una función que respeta la firma del delegado como segundo parámetro.

```

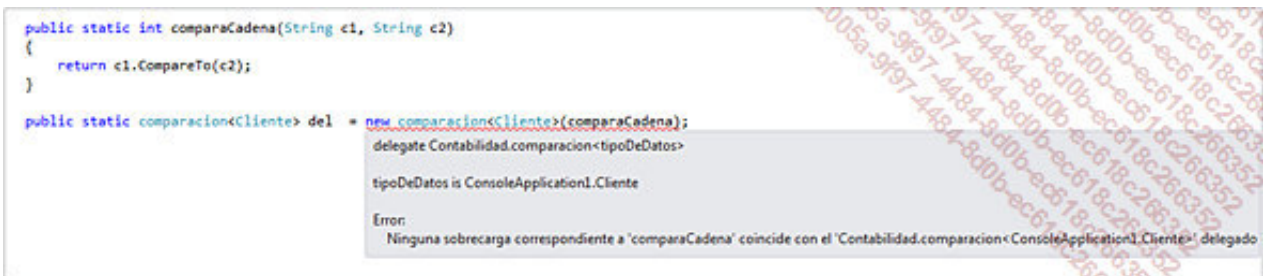
public static int compareCodigo(Cliente c1, Cliente c2)
{
    if (c1.codigo < c2.codigo)
    {
        return -1;
    }
    if (c1.codigo > c2.codigo)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

public static comparacion<Cliente> del = new comparacion<Cliente>
(compareCodigo);

ordenacion(matriz, del);

```

El compilador verifica que la firma de la función es compatible con la definición del delegado.



```

public static int comparaCadena(String c1, String c2)
{
    return c1.CompareTo(c2);
}

public static comparacion<Cliente> del = new comparacion<Cliente>(comparaCadena);
delegate Contabilidad.comparacion<tipoDeDatos>
tipoDeDatos is ConsoleApplication1.Cliente
Error:
Ninguna sobrecarga correspondiente a 'comparaCadena' coincide con el 'Contabilidad.comparacion<ConsoleApplication1.Cliente>' delegado

```

5. Varianza

En programación orientada a objetos, la varianza designa el hecho de utilizar un tipo de objetos que no corresponde exactamente al esperado. Sin embargo hay una pequeña restricción, ya que el tipo utilizado y el tipo esperado deben formar parte de la misma jerarquía de clase. Así, el tipo utilizado puede ser un supertipo del tipo esperado o un subtipo del tipo esperado. Si el tipo utilizado es un supertipo del tipo esperado (tipo menos derivado), en este caso, hablamos de contravarianza. Si el tipo utilizado es un subtipo del tipo esperado (tipo derivado), en este caso, hablamos de covarianza. Tomemos el ejemplo de una clase `Persona` y una de sus subclases, la clase `Cliente`. La covarianza consiste en utilizar la clase `Cliente` donde se espera la clase `Persona`. La contravarianza es el trámite inverso, ya que consiste en utilizar la clase `Persona` donde se espera la clase `Cliente`. Las interfaces genéricas y los delegados genéricos se encargan de estos dos mecanismos. Vamos a detallar estas dos nociones a continuación.

a. Varianza en las interfaces genéricas

Para ilustrar todo esto, utilizaremos las dos clases definidas a continuación:

```
public class Persona
{
    protected string elApellido;
    protected string elNombre;
    protected DateTime laFecha_naci;
    protected string laContraseña;
    public static int prueba = 10;

    public Persona()
    {
        elApellido = "";
        elNombre = "";
        laContraseña = "";
    }

    public Persona(string nom, string nombre, string pwd)
    {
        elApellido = apellido;
        elNombre = nombre;
        laContraseña = pwd;
    }

    public Persona(string apellido, string nombre, DateTime fNaci)
    {
        elApellido = apellido;
        elNombre = nombre;
        laFecha_naci = fNaci;
    }

    public string apellido
    {
        get
        {
            return elApellido;
        }
        set
        {
            elApellido = value.ToUpper();
        }
    }

    public string nombre
    {
        get
        {
            return elNombre;
        }
        set
        {
            elNombre = value.ToLower(); ;
        }
    }

    public DateTime fecha_naci
    {
        get
        {
            return laFecha_naci;
        }
        set
```



```
        {
            if (value.Year >= 1900)
            {
                laFecha_naci = value;
            }
        }
    }
    public int edad
    {
        get
        {
            return DateTime.Now.Year - laFecha_naci.Year;
        }
    }
    public string contraseña
    {
        set
        {
            laContraseña = value;
        }
    }
}
```

```
public class Cliente: Persona
{
    private int elNumero;

    public int numero
    {
        get
        {
            return elNumero;
        }
        set
        {
            elNumero = value;
        }
    }
}
```

Contravarianza en las interfaces genéricas

Las dos clases definidas anteriormente son completadas a continuación por la declaración de la interfaz genérica siguiente.

```
interfaz ComparadorGenerico<tipoDeDatos>
{
    int compare(tipoDeDatos o1, tipoDeDatos o2);
}
```

Las clases que implementan esta interfaz deberán contener al menos el método `compare`. Ahora vamos a crear dos clases capaces de comparar Personas o Clientes implementando la interfaz `ComparadorGenerico` con, como tipo de argumento, la clase `Persona` o la clase `Cliente`. La comparación de las personas se hará según el nombre y la comparación de los clientes, según el número.

Nuestra última etapa consiste en crear un método utilizando nuestra interfaz genérica como

```
public class ComparadorPersona: ComparadorGenerico<Persona>
{
    public int compare(Persona p1, Persona p2)
    {
        return p1.apellido.CompareTo(p2.apellido);
    }
}

public class comparadorCliente: ComparadorGenerico<Cliente>
{
    public int compare(Cliente p1, Cliente p2)
    {
        return p1.numero.CompareTo(p2.numero);
    }
}
```

parámetro. Para ello, añadimos la siguiente función, que verifica la igualdad de dos clientes en función del comparador que se le pasa como primer argumento.

```
public static void verifIgualda(ComparadorGenerico<Cliente>
c, Cliente c1, Cliente c2)
{
    if (c.compare(c1, c2) == 0)
    {
        Console.WriteLine("los dos son idénticos");
    }
    else
    {
        Console.WriteLine("los dos son diferentes");
    }
}
```

Ahora nos queda probar esto creando dos instancias de la clase `Cliente` e intentando compararlas con, como criterio, el número del cliente. Para ello, utilizaremos una instancia de la clase `ComparadorCliente`.

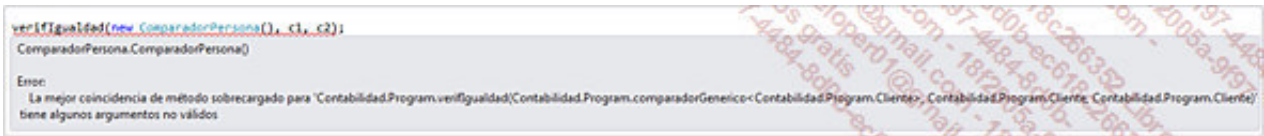
```
Cliente c1, c2;
c1 = new Cliente();
c1.numero = 10;
c1.apellido = "garcía";
c2 = new Cliente();
c2.numero = 10;
c2.apellido = "garcía";
verifIgualda(new ComparadorCliente(), c1, c2);
```

Nuestro código funciona correctamente, ya que obtenemos el siguiente mensaje en la consola.

```
los dos clientes son idénticos
```

Si ahora queremos comparar nuestros dos clientes según su apellido más que según su número, podemos utilizar la clase `ComparadorPersona`, ya que el método `compare`, definido en esta clase, espera como parámetros dos instancias de la clase `Persona`; por lo tanto, si le facilitamos dos instancias de la clase `Cliente`, funcionará de la misma manera: nuestras instancias de la clase `Cliente` disponen en efecto de un apellido debido a la relación de herencia con la clase `Persona`. Sin embargo, el compilador no tiene la misma opinión que nosotros y detecta un

error.

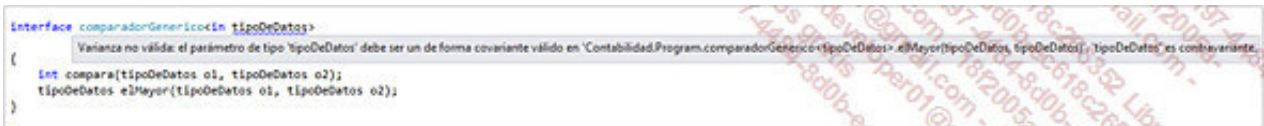


En realidad, la función `verIguualda` espera como primer parámetro un delegado capaz de trabajar con Clientes. Ahora bien, le facilitamos únicamente un delegado capaz de trabajar con Personas. Utilizamos la clase `Persona` donde se espera la clase `Cliente`; por lo tanto, estamos en presencia de contravarianza. Para que el compilador lo acepte, es indispensable añadir la palabra reservada `in` en la declaración de la interfaz genérica.

```
interfaz ComparadorGenerico<in tipoDeDatos>
{
    int compare(tipoDeDatos o1, tipoDeDatos o2);
}
```

Sin embargo, se puede declarar un tipo como contravariante en una interfaz o un delegado genérico, únicamente si se utiliza como tipo de argumentos de método. En ningún caso se puede utilizar como tipo de retorno de un método.

Si modificamos nuestra interfaz con el añadido de un método utilizando el tipo contravariante como tipo de retorno, obtenemos un error de compilación.



Covarianza en las interfaces genéricas

Para ilustrar la covarianza en las interfaces genéricas, vamos a crear una nueva interfaz que define el método `creacionInstancia`. En las clases que implementarán esta interfaz, este método deberá devolver una instancia del tipo argumento utilizado durante la implementación de la interfaz.

```
public interface IFabrica<tipoDeDato>
{
    tipoDeDato creacionInstancia();
}
```

Luego se implementa esta interfaz con la siguiente clase.

```
public class Fabrica<tipoDeDato> : IFabrica<tipoDeDato>
{
    where tipoDeDato: new()
    {
        public tipoDeDato creacionInstancia()
        {
            return new tipoDeDato();
        }
    }
}
```

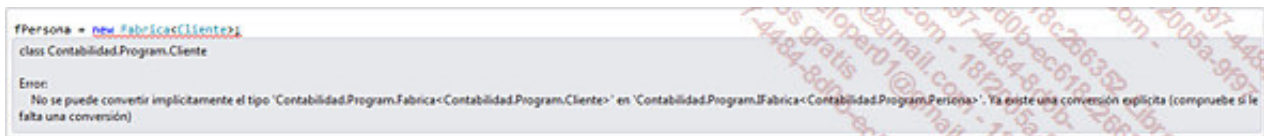
En esta clase, hay que observar que hemos añadido una restricción sobre el tipo parámetro para estar seguros de que la clase utilizada como tipo de argumento dispone correctamente de un constructor por defecto.

Ahora podemos crear una instancia de esta clase y utilizarla para producir instancias de la clase `Persona`.

```
IFabrica<Persona> fPersona = new Fabrica<Persona>();
Persona p;
p=fPersona.creacionInstancia();
```

Este código se compila sin error y nos permite obtener correctamente instancias de la clase `Persona`. Si modificamos este código para crear un objeto `Fabrica` de `Cliente`, obtenemos un error de compilación:

```
fPersona = new Fabrica<Cliente>();
```



Efectivamente, intentamos asignar a una variable de tipo `IFabrica<Persona>` una instancia de `IFabrica<Cliente>`. En realidad estamos utilizando la covarianza al especificar un tipo más derivado que aquel esperado. Para que el compilador acepte esta situación, hay que utilizar la palabra clave `out` en la declaración de la interfaz genérica.

```
public interfaz IFabrica<out tipoDeDato>
{
    tipoDeDato creacionInstancia();
}
```

Sin embargo, esta técnica comporta una limitación, ya que el tipo declarado covariante sólo se puede utilizar como tipo de retorno de una función. Si se utiliza como tipo para un parámetro de método, el compilador activa un error.



b. Varianza en los delegados genéricos

Como las interfaces genéricas, los delegados genéricos se encargan de la contravarianza y covarianza. Así, es posible utilizar para un delegado genérico un tipo más derivado o un tipo menos derivado que aquel esperado. Las limitaciones son las mismas que para las interfaces genéricas, ya que un tipo menos derivado sólo se puede utilizar como parámetro de un delegado genérico, y un tipo más derivado que el esperado sólo se puede utilizar como tipo de retorno de una función genérica.

Contravarianza en los delegados genéricos

Para ilustrar la contravarianza en los delegados genéricos, vamos a coger el ejemplo utilizado para los delegados genéricos:

```
public delegate int comparacion<tipoDeDatos>(tipoDeDatos
p1, tipoDeDatos p2);
```

```
public static void ordenacion(Cliente[] matriz, comparacion<Cliente>
comparador)
{
    Cliente o;
    for (int i = 0; i < matriz.Length - 1; i++)
    {
        for (int j = i + 1; j < matriz.Length; j++)
        {
            if (comparador.Invoke(matriz[j], matriz[i]) < 0)
            {
                o = matriz[j];
                matriz[j] = matriz[i];
                matriz[i] = o;
            }
        }
    }
}
```

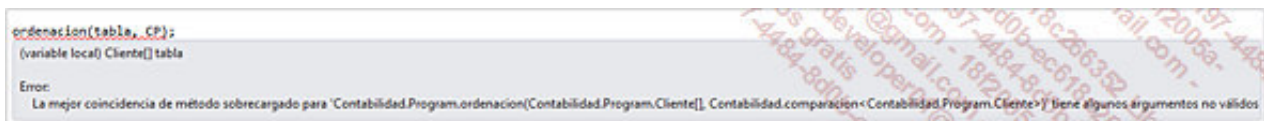
Para efectuar nuestro test, añadimos una función respetando la firma del delegado y permitiendo realizar la comparación de dos Personas según el apellido de estas personas.

```
public static int compareApellido(Persona p1, Persona p2)
{
    return p1.apellido.CompareTo(p2.apellido);
}
```

Ahora nos queda utilizar todo ello para ordenar una matriz de Clientes:

```
Cliente[] tab = new Cliente[5];
tab[0] = new Cliente("pepe2", "nombre2", new DateTime(1956, 12, 23), 2);
tab[1] = new Cliente("pepe1", "nombre1", new DateTime(1956, 12, 23), 1);
tab[2] = new Cliente("pepe5", "nombre5", new DateTime(1956, 12, 23), 5);
tab[3] = new Cliente("pepe3", "nombre3", new DateTime(1956, 12, 23), 3);
tab[4] = new Cliente("pepe4", "nombre4", new DateTime(1956, 12, 23), 4);
comparacion<Persona> CP=compareApellido;
ordenacion(tab, CP);
```

Como no hemos tomado precauciones particulares, cuando invocamos la función de ordenación pasándole como parámetro una instancia de delegado que trabaja con objetos `Persona` mientras aquella espera una instancia de delegado que trabaja con `Clientes`, el compilador genera un error.



Para que el compilador autorice la contravarianza, hay que utilizar la palabra `in` en la declaración del delegado.

```
public delegate int comparacion<in tipoDeDatos>(tipoDeDatos  
p1, tipoDeDatos p2);
```

Como para las interfaces genéricas, un tipo puede declararse contravariante únicamente si se utiliza como tipo de argumentos de método. En ningún caso se puede utilizar como tipo de retorno de un método.

Covarianza en los delegados genéricos

Para ilustrar el funcionamiento de la covarianza en los delegados genéricos, vamos a crear una función capaz de devolver una matriz rellena con instancias de una clase particular. La creación de las instancias de clase necesarias para rellenar la matriz se confiará a un delegado.

El delegado genérico correspondiente puede tener la siguiente forma:

```
public delegate tipoDeDatos construccion<out tipoDeDatos> ()  
where tipoDeDatos: new();
```

La restricción en el tipo nos impone tener un constructor por defecto en la clase correspondiente.

Ahora podemos escribir dos funciones respetando la firma del delegado.

```
public static Cliente fabricacionCliente()  
{  
    return new Cliente();  
}  
  
public static Persona fabricacionPersona()  
{  
    return new Persona();  
}
```

Ahora nos queda escribir la función que permite la creación de una matriz. Esta función espera como primer parámetro el tamaño de la matriz, y como segundo parámetro, el delegado encargado de crear las instancias de clase que sirven para rellenar la matriz. Esta función devuelve la matriz rellena.

```
public static Persona[] rellenarMatriz(int tamaño,  
construccion<Persona> cc)  
{  
    Persona[] matriz;  
    matriz=new Persona[tamaño];  
    for (int i=0;i<tamaño;i++)  
    {  
        matriz[i] = cc.Invoke();  
    }  
    return matriz;  
}
```

Ya podemos utilizar esto con las pocas líneas de código siguientes.

```
construccion<Persona> cp;  
cp = fabricacionPersona;  
Persona[] matriz= rellenarMatriz(5, cp);
```

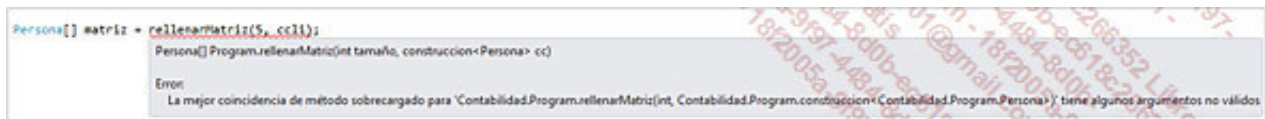
El compilador no tiene nada que decir en contra de este código.

Ahora intentamos rellenar la matriz no con instancias de la clase `Persona`, sino con instancias de la clase `Cliente`. Gracias a la relación de herencia entre estas dos clases, una casilla de la matriz se puede utilizar para referenciar una instancia de la clase `Persona`, pero también una instancia de cualquiera de estas subclases, y por lo tanto de la clase `Cliente`.

Podemos escribir con toda confianza el siguiente código:

```
construccion<Cliente> ccli;  
ccli = fabricacionCliente;  
Persona[] matriz= rellenarMatriz(5, ccli);
```

Desafortunadamente, el compilador descubre el truco, ya que facilitamos a nuestra función una instancia de delegado que utiliza un tipo más derivado que aquel esperado.



Para que el compilador acepte esta operación, hay que autorizarla añadiendo la palabra `out` en la declaración del delegado.

```
public delegate tipoDeDatos construccion<out tipoDeDatos> ()  
where tipoDeDatos: new();
```

Como para la covarianza con las interfaces genéricas, se aplica una restricción ya que el tipo covariante sólo se puede utilizar como tipo de retorno, y no como tipo para un parámetro de

Subir

Las colecciones

A menudo las aplicaciones necesitan manejar grandes cantidades de información. Hay muchas estructuras disponibles en Visual C# para facilitar la gestión de esta información. Están agrupadas bajo el término `colección`. Como en la vida corriente, hay diferentes tipos de colección. Puede haber personas que recuperan todo tipo de cosas, pero que no siguen una organización particular para guardarlas; otras que están especializadas en la colección de un tipo de objetos determinado, los maniáticos que toman todo tipo de precauciones posibles para poder encontrar con toda seguridad un objeto...

Existe en el Framework .NET clases correspondiente a cada una de estas situaciones.

1. Las colecciones predefinidas

Las diferentes clases que permiten la gestión de colecciones se reparten entre dos espacios de nombres:

- `System.Collections`
- `System.Collections.Generic`

El primero contiene las clases normales, mientras que el segundo contiene las clases genéricas equivalentes que permiten la creación de colecciones muy tipadas. Estas colecciones muy tipadas están especializadas en la gestión de un tipo determinado de datos. Aunque estas muchas clases ofrecen funcionalidades diferentes, tienen muchos puntos en común debido al hecho de que implementan las mismas interfaces. Por ejemplo, todas estas clases son capaces de facilitar un objeto `enumerator` que permite recorrer el conjunto de la colección. De hecho se trata del objeto utilizado por la instrucción `foreach` de Visual C#.

a. Array

La clase `Array` no forma parte del espacio de nombres `System.Collections`, pero se puede considerar a pesar de todo como una colección, ya que implementa la interfaz `IList`. Las matrices creadas a partir de la clase `Array` tienen un tamaño fijo. Esta clase contiene una multitud de métodos compartidos que permiten la ejecución de varias funcionalidades en matrices. Hay dos propiedades muy útiles para el uso de la clase `Array`:

- `Length`, que representa el número total de elementos en la matriz.
- `Rank`, que contiene el número de dimensiones de la matriz.

Se utiliza raramente esta clase para la creación de una matriz, ya que se prefiere utilizar la sintaxis Visual C# para ello.

b. ArrayList y List

La clase `ArrayList` o su versión genérica `List` son evoluciones de la clase `Array`. Aportan muchas mejoras respecto a esta última.

- El tamaño de un `ArrayList` es dinámico y se ajusta automáticamente a las necesidades.
- Propone métodos que permiten la adición, la inserción y la supresión de varios elementos de manera simultánea en una sola operación.

Por el contrario, en algunos puntos, la clase `ArrayList` es menos eficaz que una simple matriz:

- Los `ArrayList` sólo tienen una única dimensión.
- Una matriz de datos de un tipo específico es más eficaz que un `ArrayList` cuyos elementos son generados como `Object`. La utilización de la versión genérica (`List`) permite obtener rendimientos equivalentes.

Como cualquier clase, un `ArrayList` debe tener instancias antes de poder utilizarse. Hay dos constructores disponibles. El primero es un constructor por defecto y crea un `ArrayList` con una capacidad inicial de cero. Luego se dimensionará automáticamente durante la adición de elementos. No se aconseja esta solución, ya que la ampliación del `ArrayList` consume muchos recursos.

Si dispone de una estimación del número de elementos que hay que almacenar, es preferible utilizar el segundo constructor, que espera como parámetro la capacidad inicial del `ArrayList`. Esto evita el dimensionamiento automático durante la adición.



Hay que observar que el tamaño indicado no es definitivo y el `ArrayList` podrá contener más elementos de lo previsto inicialmente.

La propiedad `Capacity` permite conocer el número de elementos que el `ArrayList` puede contener. La propiedad `Count` indica el número actual de elementos en el `ArrayList`. Los métodos `Add` y `AddRange` añaden elementos al final de la lista. Los métodos `Insert` y `InsertRange` permiten elegir la ubicación donde efectuar el añadido. La propiedad `Item`, que es la propiedad por defecto de clase, se utiliza para alcanzar un elemento en una posición dada. La supresión de elementos se hace por el método `RemoveAt` o `RemoveRange`; el primero espera como parámetro el índice del elemento que hay que suprimir; el segundo exige además el número de elementos que hay que suprimir. El método `Clear` es más radical y suprime todos los elementos.

El siguiente código ilustra el funcionamiento de esta clase:

```
public static void main()
{
    ArrayList lista;
    Cliente c;
    lista = new ArrayList();
    Console.WriteLine("capacidad inicial de la lista {0}", lista.Capacity);
    Console.WriteLine("número de elementos de la lista {0}", lista.Count);
    Console.WriteLine("añadido de un cliente");
    c = new Cliente("cliente1", "nombre1", new DateTime(1964,12,23), 1001);
    lista.Add(c);
    Console.WriteLine("capacidad de la lista {0}", lista.Capacity);
    Console.WriteLine("número de elementos de la lista {0}", lista.Count);
    Console.WriteLine("añadido de cuatro clientes");
    c = new Cliente("cliente2", "nombre2", new DateTime(1964,12,23), 1002);
    lista.Add(c);
    c = new Cliente("cliente3", "nombre3", new DateTime(1964,12,23), 1003);
    lista.Add(c);
    c = new Cliente("cliente4", "nombre4", new DateTime(1964, 12, 23), 1004);
    lista.Add(c);
    c = new Cliente("cliente5", "nombre5", new DateTime(1964, 12, 23), 1005);
    lista.Add(c);
    Console.WriteLine("capacidad de la lista {0}",
        lista.Capacity);
    Console.WriteLine("número de elementos de la lista {0}", lista.Count);
    Console.WriteLine("visualización de la lista de los clientes");
```

```

foreach ( Cliente cl in lista)
{
    cl.visualización();
    Console.WriteLine();
}
Console.WriteLine("borrado de los clientes 1002, 1003, 1004");
lista.RemoveRange(1, 3);
Console.WriteLine("capacidad de la lista {0}", lista.Capacity);
Console.WriteLine("número de elementos de la lista {0}", lista.Count);
Console.WriteLine("visualización de la lista de los clientes");
foreach ( Cliente cl in lista)
{
    cl.visualización();
    Console.WriteLine();
}
Console.WriteLine("visualización del segundo cliente de la lista");
((Cliente)lista[1]).visualización();
Console.WriteLine();
Console.WriteLine("borrado de todos los clientes"); lista.Clear();
Console.WriteLine("capacidad de la lista {0}", lista.Capacity);
Console.WriteLine("número de elementos de la lista {0}", lista.Count);
Console.ReadLine();
}
}

```

Visualiza el resultado siguiente:

```

capacidad inicial de la lista 0
número de elementos de la lista 0
añadido de un cliente
capacidad de la lista 4
número de elementos de la lista 1
añadido de cuatro clientes
capacidad de la lista 8
número de elementos de la lista 5
visualización de la lista de los clientes
Sr cliente1 nombre1 nacido el 23/12/1964 00:00:00
Código cliente: 1001
Sr cliente2 nombre2 nacido el 23/12/1964 00:00:00
Código cliente: 1002
Sr cliente3 nombre3 nacido el 23/12/1964 00:00:00
Código cliente: 1003
Sr cliente4 nombre4 nacido el 23/12/1964 00:00:00
Código cliente: 1004
Sr cliente5 nombre5 nacido el 23/12/1964 00:00:00
Código cliente: 1005
borrado de los clientes 1002, 1003, 1004
capacidad de la lista 8
número de elementos de la lista 2
visualización de la lista de los clientes
Sr cliente1 nombre1 nacido el 23/12/1964 00:00:00
Código cliente: 1001
Sr cliente5 nombre5 nacido el 23/12/1964 00:00:00
Código cliente: 1005
visualización del segundo cliente de la lista
Sr cliente5 nombre5 nacido el 23/12/1964 00:00:00
Código cliente: 1005
borrado de todos los clientes
capacidad de la lista 8
número de elementos de la lista 0

```



La capacidad de la lista no disminuye en el momento de la supresión de un elemento, incluso cuando la lista está vacía.

c. Hashtable y Dictionary

Un `Hashtable` o su versión genérica `Dictionary` registra los datos bajo la forma de par clave-valor. El `Hashtable` se compone internamente de compartimentos que contienen los elementos de la colección. Para cada elemento de la colección, un código es generado por una función hash basada en la clave de cada elemento. Luego se utiliza el código para identificar el compartimento en el cual se almacena el elemento. Durante la búsqueda de un elemento en la colección, se efectúa la operación inversa. El código hash se genera desde la clave del elemento buscado. Luego esta clave sirve para identificar el compartimento en el cual se encuentra el elemento buscado. Para que una `Hashtable` pueda almacenar un objeto, éste debe ser capaz de facilitar su propio código hash.

d. Cola

Se utiliza este tipo de colección cuando se necesita un espacio de almacenamiento temporal. Cuando se recupera un elemento desde la colección, se suprime al mismo tiempo de la colección.

Las colecciones de tipo `Cola` están adaptadas si se requiere acceder a los datos en el mismo orden que aquel en el cual han sido almacenadas en la colección. Este tipo de gestión a veces se llama *First In - First Out* (FIFO). Las tres principales operaciones disponibles son:

- **Enqueue** para añadir un elemento al final de la cola.
- **Dequeue** para obtener el elemento más antiguo de la cola y suprimirlo.
- **Peek** para obtener el elemento más antiguo sin suprimirlo de la cola.

El ejemplo siguiente ilustra la utilización de estos tres métodos.

```
public static void main()
{
    Queue<Cliente> q;
    q = new Queue<Cliente>();
    Cliente c;
    c = new Cliente("cliente1", "nombre1", new DateTime(1964, 12, 23), 1001);
    Console.WriteLine("llegada del primer cliente:{0}", c.apellido);
    q.Enqueue(c);
    c = new Cliente("cliente2", "nombre2", new DateTime(1964, 12, 23), 1002);
    Console.WriteLine("llegada del segundo cliente:{0}", c.apellido);
    q.Enqueue(c);
    c = new Cliente("cliente3", "nombre3", new DateTime(1964, 12, 23), 1003);
    Console.WriteLine("llegada del tercer cliente:{0}", c.apellido);
    q.Enqueue(c);
    Console.WriteLine("salida del primer cliente:{0}", q.Dequeue().apellido);
    Console.WriteLine("queda {0} clientes", q.Count);
    Console.WriteLine("salida del segundo cliente:{0}", q.Dequeue().apellido);
    Console.WriteLine("queda {0} cliente", q.Count);
    Console.WriteLine("el tercer cliente se incrusta:{0}", q.Peek().apellido);
    Console.WriteLine("queda {0} cliente", q.Count);
    Console.WriteLine("salida del tercer cliente:{0}", q.Dequeue().apellido);
    Console.WriteLine("queda {0} cliente", q.Count);
    Console.ReadLine();
}
```

}

e. Stack

Las colecciones de este tipo utilizan el mismo principio que las `Cola`: cuando se recupera un elemento de la colección, se suprime de ella. La única distinción respecto a la clase `Cola` es el orden en el cual se recuperan los elementos. Este tipo de colección utiliza la técnica *Last In - First Out* (LIFO). El ejemplo clásico de este tipo de gestión es la pila de platos de su cocina. Después de fregar, apila los platos en un estante. Al día siguiente, cuando pone la mesa, el primer plato disponible es el último que se ha guardado el día anterior.

Las tres principales operaciones disponibles son:

- `Push` para añadir un elemento en la cima de la pila.
- `Pop` para obtener el elemento encima de la pila y suprimirlo.
- `Peek` para obtener el elemento encima de la pila sin suprimirlo de la pila.

2. Elegir un tipo de colección

A continuación le damos unos consejos para elegir el tipo de colección adaptado a sus necesidades.

- Si necesita acceder a los elementos de la colección con un índice: utilice un `ArrayList`.
- El acceso a los elementos debe efectuarse en el orden de la adición en la colección o en el orden inverso: utilice una `Cola` o un `Stack`.

Subir

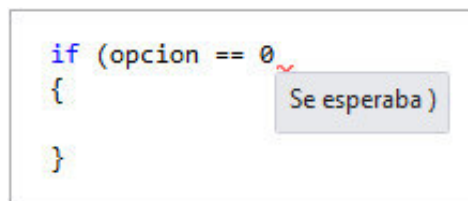
Los diferentes tipos de errores

Para un desarrollador, los errores son una de las principales fuentes de estrés. En realidad, podemos clasificar estos errores en tres categorías. Veremos cada una de ellas, así como las soluciones disponibles para resolverlas.

1. Los errores de sintaxis

Este tipo de error se produce en el momento de la compilación cuando una palabra clave del lenguaje está mal ortografiada. Muy frecuentes con las primeras herramientas de desarrollo, donde el editor de código y el compilador eran dos entidades separadas, son cada vez más raras con los entornos como Visual Studio. La mayoría de estos entornos proponen un análisis sintáctico al insertar el código. Desde este punto de vista, Visual Studio propone numerosas funcionalidades que nos permiten eliminar estos errores.

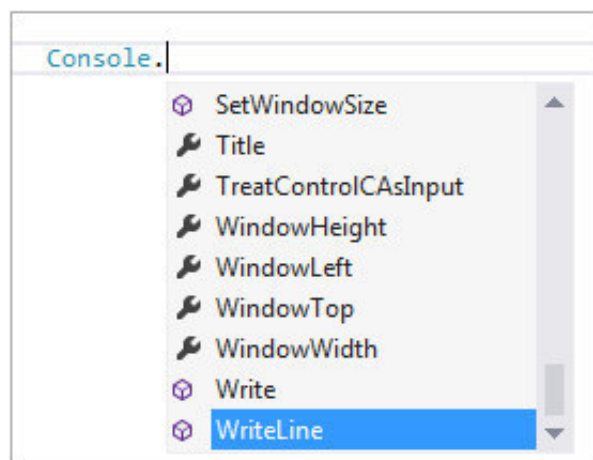
Por ejemplo, el programa comprueba que a cada paréntesis de apertura corresponda un paréntesis de cierre.



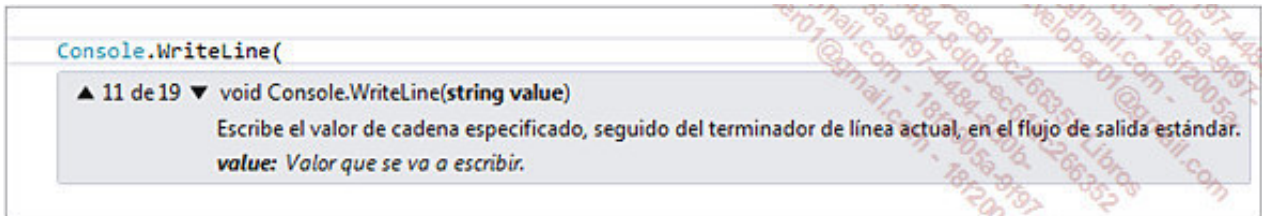
```
if (opcion == 0 {  
    {  
}
```

Por otra parte, las «faltas de ortografía» en los nombres de propiedades o métodos se eliminan fácilmente gracias a las funcionalidades IntelliSense. IntelliSense se encarga de las siguientes funciones:

- La visualización automática de la lista de los miembros disponibles:

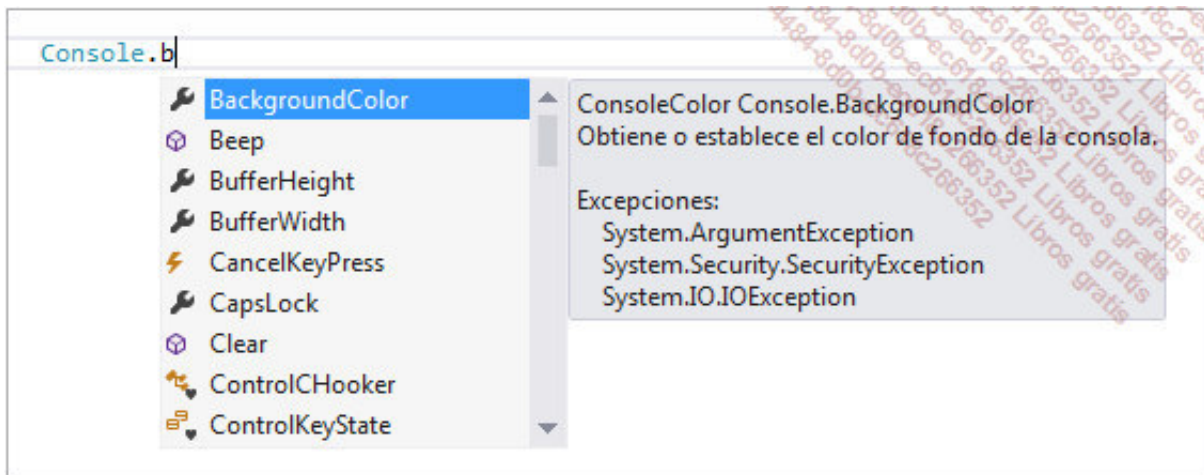


- La visualización de la lista de los parámetros que deben facilitarse al llamar a un procedimiento o función:



Si hay varias sobrecargas, IntelliSense visualiza su número y le permite recorrerlas utilizando las flechas arriba y abajo del teclado.

- La visualización de datos puntuales sobre miembros de una clase:



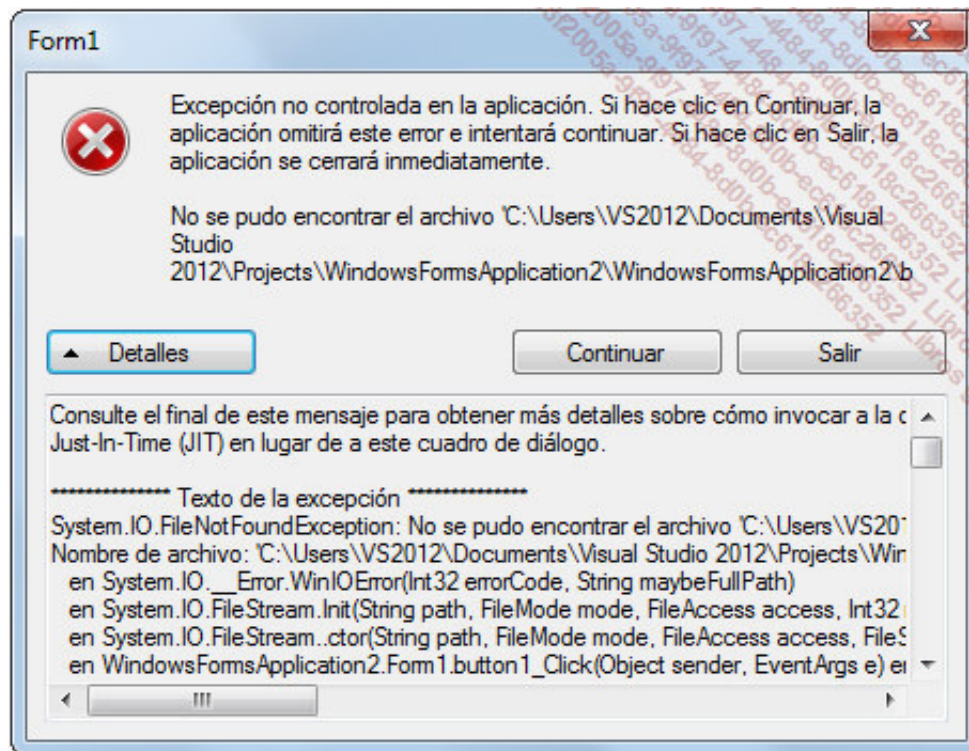
- El relleno automático de palabras: empiece a teclear un principio de palabra, luego utilice la combinación de teclas [Ctrl][Espacio] para visualizar todo lo que pueda utilizar como palabra en esta ubicación, empezando por los caracteres ya introducidos. Si sólo hay una posibilidad, se añade la palabra automáticamente; si no, selecciónela en la lista y valide con la tecla [Tab].
- La visualización de la lista de los valores posibles para una propiedad de tipo enumeración.



Con todas estas funciones, es prácticamente imposible que se produzcan errores de sintaxis en el código.

2. Los errores de ejecución

Estos errores aparecen después de la compilación, cuando usted inicia la ejecución de la aplicación. La sintaxis del código es correcto, pero el entorno de su aplicación no permite la ejecución de una instrucción utilizada en su aplicación. Por ejemplo, es el caso si intenta abrir un archivo que no existe en el disco de su máquina. Seguramente obtendrá un cuadro de diálogo de este tipo.



¡Este tipo de cuadro de diálogo no es muy simpático para el usuario!

Afortunadamente, Visual C# permite gestionar este tipo de error y evita así la visualización de este inquietante cuadro de diálogo.

Veremos esto con detalle más adelante en este capítulo.

Los errores de lógica

Subir

Tratamiento de las excepciones

1. Gestión de excepciones

La gestión de las excepciones da la posibilidad de proteger un bloque de código contra los errores de ejecución que podrían producirse. Se debe ubicar el código peligroso en un bloque `try`. Si se activa una excepción en este bloque de código, Visual C# mira las siguientes instrucciones `catch`. Si existe una capaz de tratar la excepción, se ejecuta el código correspondiente; si no, la misma excepción se puede activar para ser gestionada por un bloque `try` de más alto nivel. Una instrucción `finally` permite marcar un grupo de instrucciones, ejecutadas antes de la salida del bloque `try`, ya se haya producido un error o no.

Por lo tanto, la sintaxis general es la siguiente:

```
try
{
    ...
    Instrucciones peligrosas
    ...
}
catch (Exception e1)
{
    ...
    Código ejecutado si una excepción de tipo Excepción1 se produce
    ...
}
catch (Exception e2)
{
    ...
    Código ejecutado si una excepción de tipo Excepción2 se produce
    ...
}
finally
{
    Código ejecutado en todos los casos antes de la salida del bloque try
}
```

Esta estructura tiene un funcionamiento muy similar al `switch` ya estudiado. Se asocia cada tipo de error a una clase de excepción y cuando este error se produce, se crea una instancia de la clase `Exception` correspondiente. Podremos determinar para cada instrucción `catch` qué tipo de excepción debe tratar.

La clase básica es la clase `Exception` desde la cual se crea una multitud de subclases especializadas cada una en un tipo de error particular. A continuación, presentamos la lista de las clases que derivan directamente de la clase `Exception`.

```
Microsoft.Build.BuildEngine....InternalLoggerException
Microsoft.Build.BuildEngine....InvalidProjectFileException
Microsoft.Build.BuildEngine....InvalidToolsetDefinitionException
Microsoft.Build.BuildEngine....RemoteErrorException
Microsoft.Build.Exceptions....BuildAbortedException
Microsoft.Build.Exceptions....InternalLoggerException
Microsoft.Build.Exceptions....InvalidProjectFileException
Microsoft.Build.Exceptions....InvalidToolsetDefinitionException
Microsoft.Build.Framework....LoggerException
```

Microsoft.CSharp.RuntimeBinder....RuntimeBinderException
Microsoft.CSharp.RuntimeBinder....RuntimeBinderInternalCompilerException
Microsoft.JScript....CmdLineException
Microsoft.JScript....ParserException
Microsoft.VisualBasic.ApplicationServices....CantStartSingleInstanceException
Microsoft.VisualBasic.ApplicationServices....NoStartupFormException
Microsoft.VisualBasic.Compatibility.VB6....WebClassContainingClassNotOptional
Microsoft.VisualBasic.Compatibility.VB6....WebClassCouldNotFindEvent
Microsoft.VisualBasic.Compatibility.VB6....WebClassNextItemCannotBeCurrentWebItem
Microsoft.VisualBasic.Compatibility.VB6....WebClassNextItemRespondNotFound
Microsoft.VisualBasic.Compatibility.VB6....WebClassUserWebClassNameNotOptional
Microsoft.VisualBasic.Compatibility.VB6....WebClassWebClassFileNameNotOptional
Microsoft.VisualBasic.Compatibility.VB6....WebClassWebItemNotValid
Microsoft.VisualBasic.Compatibility.VB6....WebItemAssociatedWebClassNotOptional
Microsoft.VisualBasic.Compatibility.VB6....WebItemClosingTagNotFound
Microsoft.VisualBasic.Compatibility.VB6....WebItemCouldNotLoadEmbeddedResource
Microsoft.VisualBasic.Compatibility.VB6....WebItemCouldNotLoadTemplateFile
Microsoft.VisualBasic.Compatibility.VB6....WebItemNameNotOptional
Microsoft.VisualBasic.Compatibility.VB6....WebItemNoTemplateSpecified
Microsoft.VisualBasic.Compatibility.VB6....WebItemTooManyNestedTags
Microsoft.VisualBasic.Compatibility.VB6....WebItemUnexpectedErrorReadingTemplateFile
Microsoft.VisualBasic.CompilerServices....IncompleteInitialization
Microsoft.VisualBasic.CompilerServices....InternalErrorException
Microsoft.VisualBasic.FileIO....MalformedLineException
System.Activities.ExpressionParser....SourceExpressionException
System.Activities.Expressions....LambdaSerializationException
System.Activities....InvalidWorkflowException
System.Activities.Presentation.Metadata....AttributeTableValidationException
System.Activities.Statements....WorkflowTerminatedException
System.Activities....WorkflowApplicationException
System.AddIn.Hosting....AddInSegmentDirectoryNotFoundException
System.AddIn.Hosting....InvalidPipelineStoreException
System....AggregateException
System....ApplicationException
System.ComponentModel.Composition....CompositionContractMismatchException
System.ComponentModel.Composition....CompositionException
System.ComponentModel.Composition....ImportCardinalityMismatchException
System.ComponentModel.Composition.Primitives....ComposablePartException
System.ComponentModel.DataAnnotations....ValidationException
System.ComponentModel.Design....ExceptionCollection
System.Configuration.Provider....ProviderException
System.Configuration....SettingsPropertyIsReadOnlyException
System.Configuration....SettingsPropertyNotFoundException
System.Configuration....SettingsPropertyWrongTypeException
System.Data.Linq....ChangeConflictException
System.Diagnostics.Eventing.Reader....EventLogException
System.DirectoryServices.ActiveDirectory....ActiveDirectoryObjectExistsException
System.DirectoryServices.ActiveDirectory....ActiveDirectoryObjectNotFoundException
System.DirectoryServices.ActiveDirectory....ActiveDirectoryOperationException
System.DirectoryServices.ActiveDirectory....ActiveDirectoryServerDownException
System.DirectoryServices.Protocols....DirectoryException
System.IdentityModel.Selectors....CardSpaceException
System.IdentityModel.Selectors....IdentityValidationException
System.IdentityModel.Selectors....PolicyValidationException
System.IdentityModel.Selectors....ServiceBusyException
System.IdentityModel.Selectors....ServiceNotStartedException
System.IdentityModel.Selectors....StsCommunicationException
System.IdentityModel.Selectors....UnsupportedPolicyOptionsException
System.IdentityModel.Selectors....UntrustedRecipientException
System.IdentityModel.Selectors....UserCancellationException
System....InvalidTimeZoneException

```
System.IO.IsolatedStorage.....IsolatedStorageException
System.IO.Log.....SequenceFullException
System.Management.Instrumentation.....InstrumentationBaseException
System.Management.Instrumentation.....WmiProviderInstallationException
System.Net.Mail.....SmtpException
System.Net.PeerToPeer.....PeerToPeerException
System.Runtime.CompilerServices.....RuntimeWrappedException
System.Runtime.DurableInstancing.....InstancePersistenceException
System.Runtime.Remoting.MetadataServices.....SUDSGeneratorException
System.Runtime.Remoting.MetadataServices.....SUDSParserException
System.Runtime.Serialization.....InvalidDataContractException
System.Security.RightsManagement.....RightsManagementException
System.ServiceModel.Channels.....InvalidChannelBindingException
System.....SystemException System.Threading.....BarrierPostPhaseException
System.Threading.....LockRecursionException
System.Threading.Tasks.....TaskSchedulerException
System.....TimeZoneNotFoundException
System.Web.Query.Dynamic.....ParseException
System.Web.Security.....MembershipCreateUserException
System.Web.Security.....MembershipPasswordException
System.Web.UI.....ViewStateException
System.Web.UI.WebControls.....EntityDataSourceValidationException
System.Web.UI.WebControls.....LinqDataSourceValidationException
System.Windows.Automation.....NoClickablePointException
System.Windows.Automation.....ProxyAssemblyNotLoadedException
System.Windows.Controls.....PrintDialogException
System.Windows.Forms.....AxHost.....InvalidActiveXStateException
System.Windows.Xps.....XpsException
System.Windows.Xps.....XpsWriterException
System.Workflow.Activities.Rules.....RuleException
System.Workflow.ComponentModel.Compiler.....WorkflowValidationFailedException
System.Workflow.ComponentModel.Serialization.....WorkflowMarkupSerializationException
System.Workflow.ComponentModel.....WorkflowTerminatedException
System.Workflow.Runtime.....WorkflowOwnershipException
System.Xaml.....XamlException
```



Esta lista sólo presenta el primer nivel de la jerarquía. Cada una de estas clases tiene también numerosos descendientes.

Se utilizan todas estas clases para indicar en cada instrucción `catch` el tipo de excepción que debe gestionar.

```
public static void aperturaArchivo()
{
    try
    {
        archivo = new FileStream("a:\\data.txt", FileMode.Open);
    }
    catch (IOException e)
    {
        Console.WriteLine("error de apertura de archivo");
    }
    finally
    {
        Console.WriteLine("fin del procedimiento de apertura de archivo");
    }
}
```

Si, entre todos los `catch`, ninguno corresponde a la excepción generada, la excepción se propaga en el código de los procedimientos o funciones invocantes, a la búsqueda de una instrucción `catch` capaz de tener en cuenta esta excepción. Si no se encuentra ningún bloque, se lanza un error en tiempo de ejecución.

Si el parámetro indicado en la instrucción `catch` es una clase de «excepción general», esta instrucción `catch` será capaz de capturar todas las excepciones creadas a partir de esta clase o subclases. El siguiente código nos permite capturar todas las excepciones.

```
public static void aperturaArchivo()
{
    try
    {
        archivo = new FileStream("a:\\data.txt", FileMode.Open);
    }
    catch (Exception e)
    {
        Console.WriteLine("error de apertura de archivo");
    }
    finally
    {
        Console.WriteLine("fin del procedimiento de apertura de archivo");
    }
}
```

Las diferentes clases disponen de las siguientes propiedades, que nos permiten tener más datos sobre el origen de la excepción.

Message

Cadena de caracteres asociada a la excepción.

Source

Nombre de la aplicación que activó la excepción.

StackTrace

Lista de todos los métodos por los cuales se pasa la aplicación antes de la activación del error.

TargetSite

Nombre del método que activó la excepción.

InnerException

Obtiene la excepción original si se activan dos excepciones en cascada.

a. Creación y activación de excepciones

Ante todo, las excepciones son clases. Por lo tanto, es posible crear nuestras propias excepciones heredando de una de las numerosas clases de excepción ya disponibles. Para respetar las convenciones del Framework .NET, se aconseja conservar el término `Exception` en el nombre de la clase. Podemos, por ejemplo, escribir el siguiente código:

```
class NoFuncionaException:Exception
{
    public NoFuncionaException(): base()
    {
    }
    public NoFuncionaException(String message): base( message)
    {
    }
    public NoFuncionaException(String message,Exception inner):
base( message, inner)
    {
    }
}
```

Luego se puede utilizar esta clase para activar una excepción personalizada. El siguiente código activa una excepción personalizada en un bloque `catch`.

```
catch (Exception e)
{
    throw new NoFuncionaException("error en la aplicación", e);
}
```

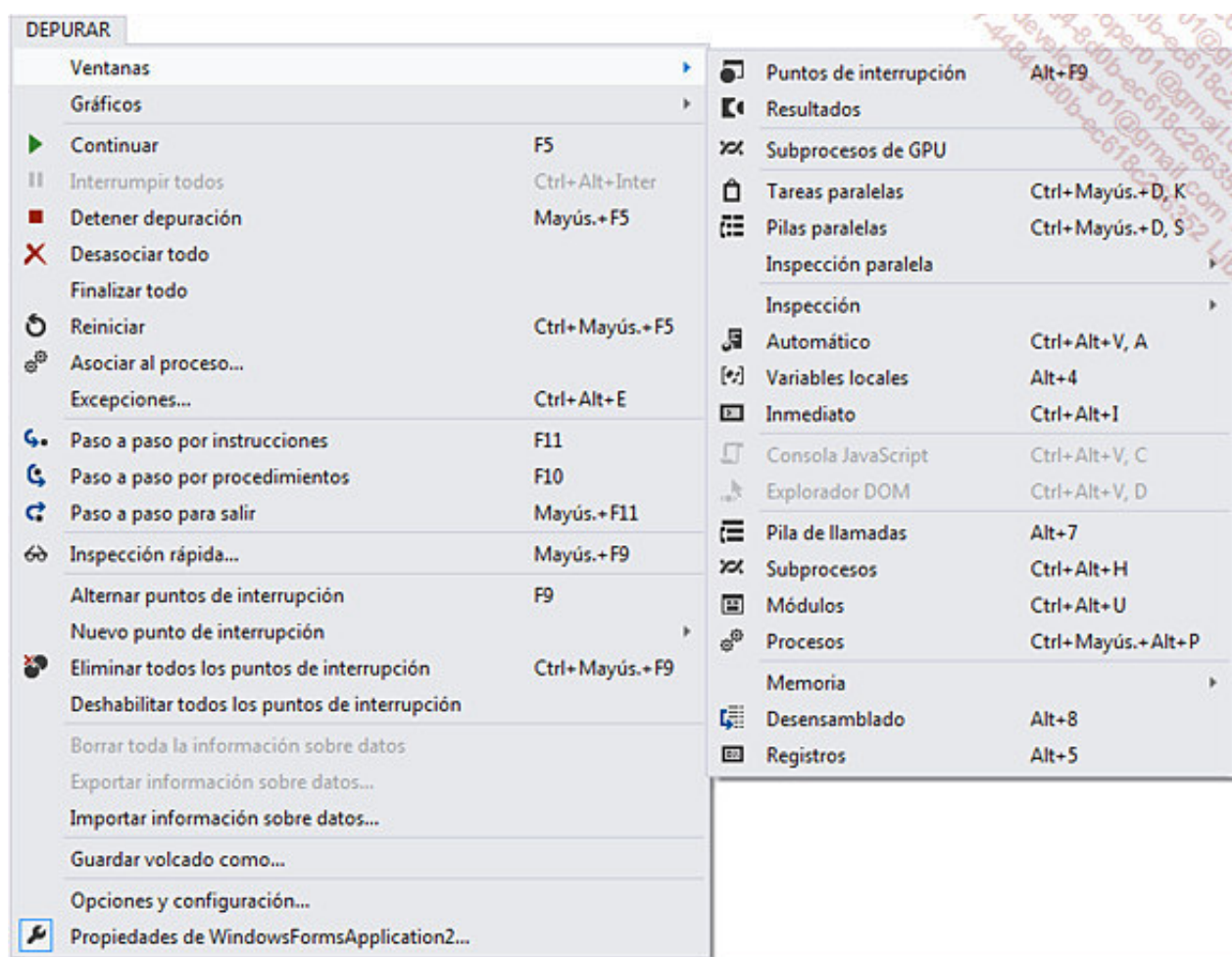
Las herramientas de depuración

En el capítulo dedicado a la gestión de los errores, hemos visto que los errores de lógica son los más difíciles de eliminar en una aplicación. Afortunadamente, Visual Studio .NET nos propone numerosas herramientas de depuración eficaces y simples de utilizar. En particular, permiten controlar el desarrollo de la ejecución de la aplicación (ubicando puntos de interrupción y haciendo ejecutar las instrucciones una por una), visualizar y modificar el contenido de las variables, visualizar el contenido de la memoria en una ubicación particular, verificar la lista de todas las funciones utilizadas, etc.

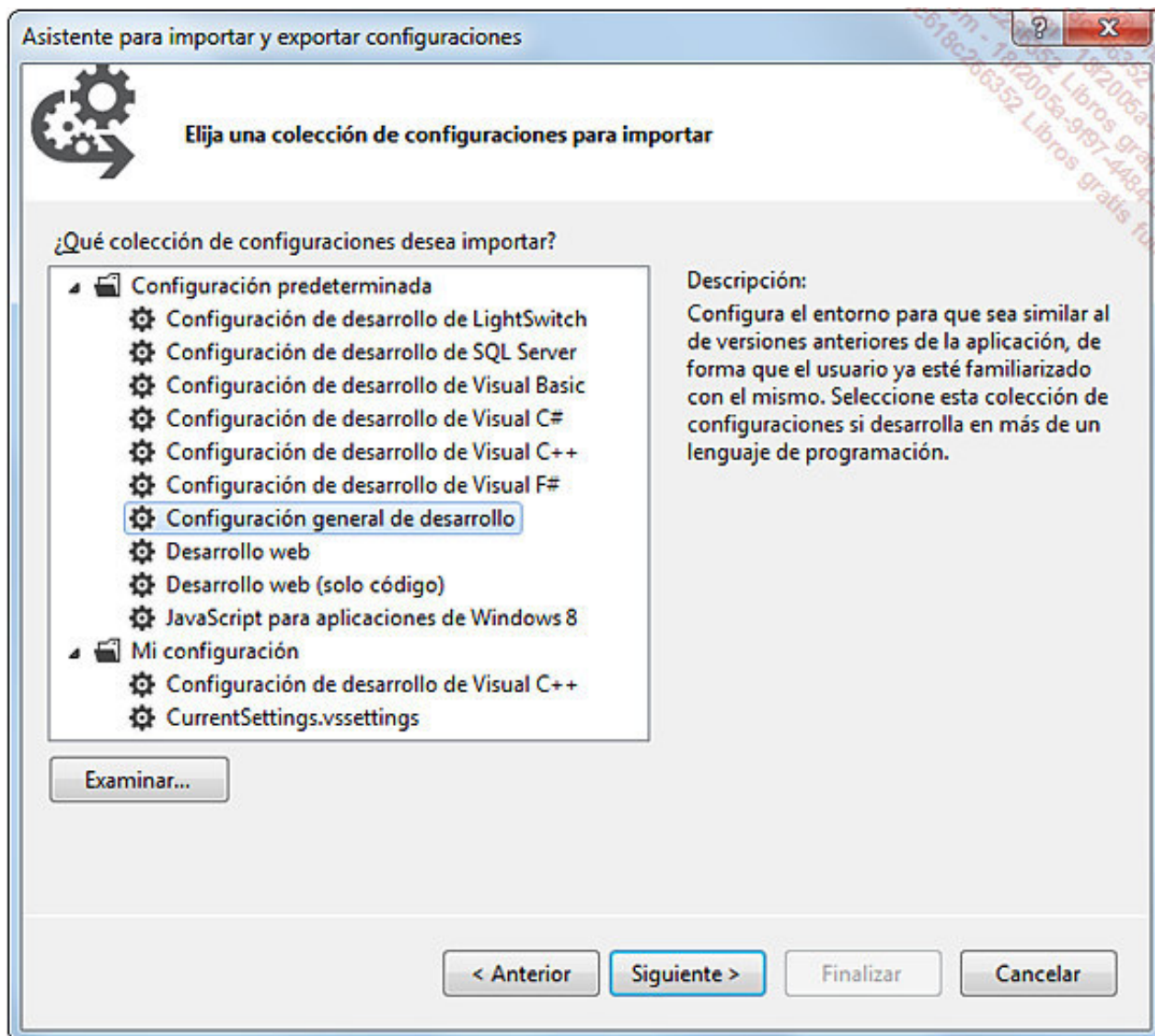
Estas diferentes herramientas son accesibles desde la barra de herramientas **Depurar**.



El menú **Depurar** facilita también el acceso a numerosas herramientas:



En función de la configuración del entorno de Visual Studio, algunas herramientas quizá no estarán disponibles. Puede volver a configurar Visual Studio para integrar estas herramientas a través del menú **Herramientas - Importar y exportar configuraciones**. Los diferentes cuadros de diálogo le proponen guardar su entorno actual antes de modificarlo, y luego elegir un entorno tipo para importar.



Entre las configuraciones disponibles, la configuración **Configuración general de desarrollo** es la que más funcionalidades propone.

Para las siguientes explicaciones de este capítulo, vamos a considerar que aquella configuración es la utilizada en Visual Studio.

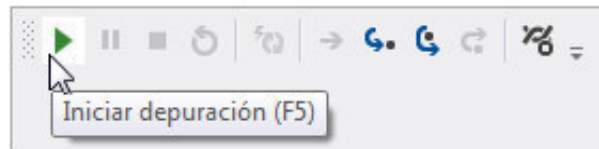
1. Control de la ejecución

a. Inicio de la solución

Un proyecto en Visual Studio puede tener tres estados distintos:

- tiempo de diseño,
- tiempo de ejecución,
- tiempo detenido (se interrumpió la ejecución).

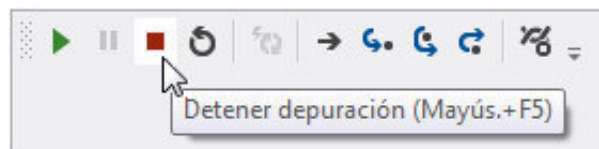
El lanzamiento de la ejecución se puede efectuar por la barra de herramientas o por la combinación de teclas [F5] o [Ctrl][F5]. Si se utiliza esta última solución, la aplicación se inicia en modo normal y no estará disponible ninguna herramienta de depuración.



Si la solución contiene varios proyectos, se debe configurar uno de ellos como proyecto de inicio para la solución. Este proyecto también debe tener un objeto de inicio configurado, se comenzará la aplicación con su ejecución.

b. Detener la solución

La detención de la aplicación puede efectuarse cerrando todas las ventanas; para una aplicación de Windows, en cuanto se cierra la última ventana, la aplicación se para, o a través de teclas [Ctrl]C para una aplicación de consola. La barra de herramientas o la combinación de teclas [Ctrl][Alt][Pausa] también permiten detener la aplicación.

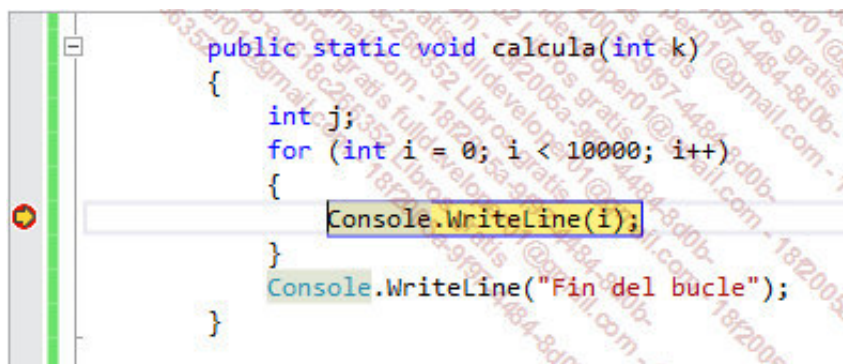


c. Interrumpir la solución

La interrupción de la ejecución se efectúa con la combinación de teclas [Ctrl][Alt][Pausa] o a través de la barra de herramientas:



La interrupción se produce sobre la instrucción siguiente a la que estuviera en curso en el momento de la parada. La ventana de código se hace de nuevo visible, con una marca al lado de la línea donde la ejecución se interrumpió.



Este método no es muy práctico, ya que hace falta tener mucha suerte para interrumpir la ejecución en un lugar preciso. Más adelante veremos que los puntos de detención son una solución mucho mejor para interrumpir la ejecución del código.

d. Proseguir con la ejecución

Una vez en modo detenido, tenemos muchas posibilidades para continuar con la ejecución de la aplicación.

La primera posibilidad permite retomar la ejecución normal de la aplicación utilizando la misma técnica que para el inicio del programa (barra de herramientas o combinación de teclas [F5]). Sin embargo, una técnica más corriente durante una depuración consiste en la ejecución paso a paso.

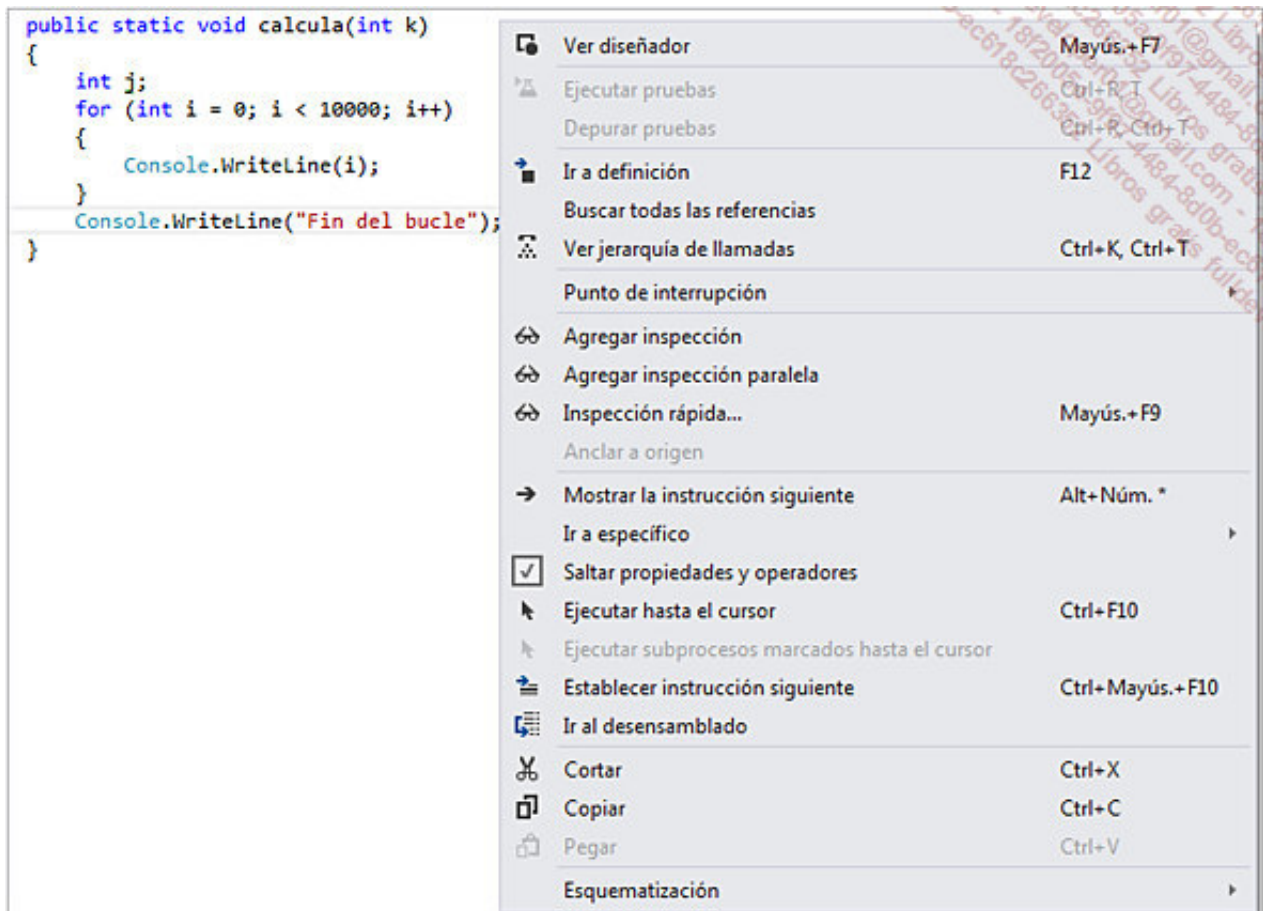
Hay tres soluciones disponibles:

- Paso a paso por instrucciones ([F11])
- Paso a paso por procedimientos ([F10])
- Paso a paso para salir ([Shift][F11])

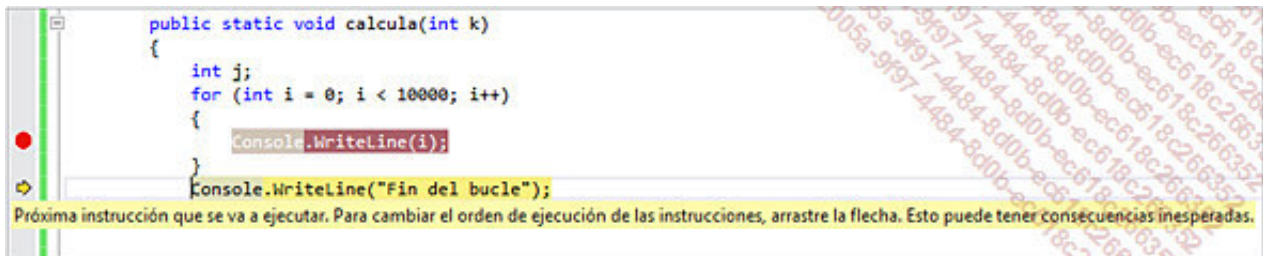
El Paso a paso por instrucciones y el Paso a paso por procedimientos difieren simplemente en su manera de gestionar las llamadas de procedimientos y funciones. Si estamos en modo detenido en una línea de código que contiene una llamada a un procedimiento o una función, el modo Paso a paso por instrucciones va a permitir entrar en el código de la función y luego iniciar la ejecución de su código línea por línea. El modo Paso a paso por procedimientos ejecutará el procedimiento o la función en una sola vez, sin que usted pueda ver lo que ocurre en el interior del procedimiento o función.

El Paso a paso para salir permite la ejecución del código hasta el final de un procedimiento o función, sin descomponer línea por línea; luego vuelva al modo detenido en la línea que sigue la llamada de la función.

Una última solución nos permite ejecutar fácilmente un bloque de código luego de pararse sobre una línea específica. Para ello, un menú contextual en la ventana de código nos ofrece la posibilidad de volver a ejecutar hasta el cursor, sin parar en todas las instrucciones entre la línea actual y la posición del cursor (muy útil para ejecutar rápidamente todas las iteraciones de un bucle).



Al revés, si usted desea ignorar la ejecución de un bloque de código o, al contrario, ejecutar de nuevo un bloque de código, es posible desplazar el punto de ejecución para designar la próxima instrucción ejecutada. Basta con desplazar la flecha amarilla que aparece en el margen, enfrente de la próxima instrucción a ejecutar.



- Como nos indica Microsoft, se debe utilizar esta funcionalidad con precaución. Hay que recordar los siguientes puntos: las instrucciones ubicadas entre el antiguo y el nuevo punto de ejecución no se ejecutarán. Desplazar el punto de ejecución hacia atrás no cancela las instrucciones ya tratadas. El punto de ejecución sólo se puede desplazar en el interior de una función o procedimiento.

2. Puntos de interrupción y TracePoint

Sólo tenemos una solución para pasar a modo detenido, que consiste en utilizar las teclas [Ctrl][Alt][Pausa]. Esta solución presenta una desventaja importante: la ejecución se para en cualquier parte. Los puntos de interrupción nos facilitan una solución más elegante gracias a la cual podemos elegir

la ubicación donde tendrá lugar la interrupción de la ejecución.

Los puntos de interrupción pueden ser condicionales. Diferentes tipos de condiciones se toman en cuenta para su activación (condición, número de paso...).

Los TracePoint son prácticamente idénticos a los puntos de interrupción, excepto que para un TracePoint debe especificar la acción que se ha de ejecutar cuando el punto se alcance. Puede ser el paso en modo detención de la aplicación y/o la visualización de un mensaje. En el entorno Visual Studio, los puntos de interrupción o los TracePoint se visualizan por una serie de iconos. Los iconos vacíos representan un elemento desactivado.

  representa un punto de interrupción normal, activado o desactivado.

  representa un punto de interrupción avanzado (condición, número de paso o filtro).

  representa un TracePoint normal, activado o desactivado.

  representa un TracePoint avanzado (condición, número de paso o filtro).

  representa un punto de interrupción o un TracePoint en error.

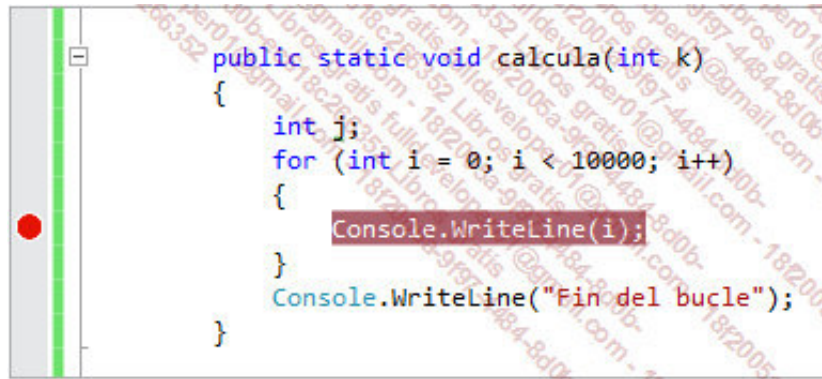
  representa una advertencia en un punto de interrupción o un TracePoint.

a. Ubicar un punto de interrupción

Para ubicar un punto de interrupción, hay muchas posibilidades disponibles:

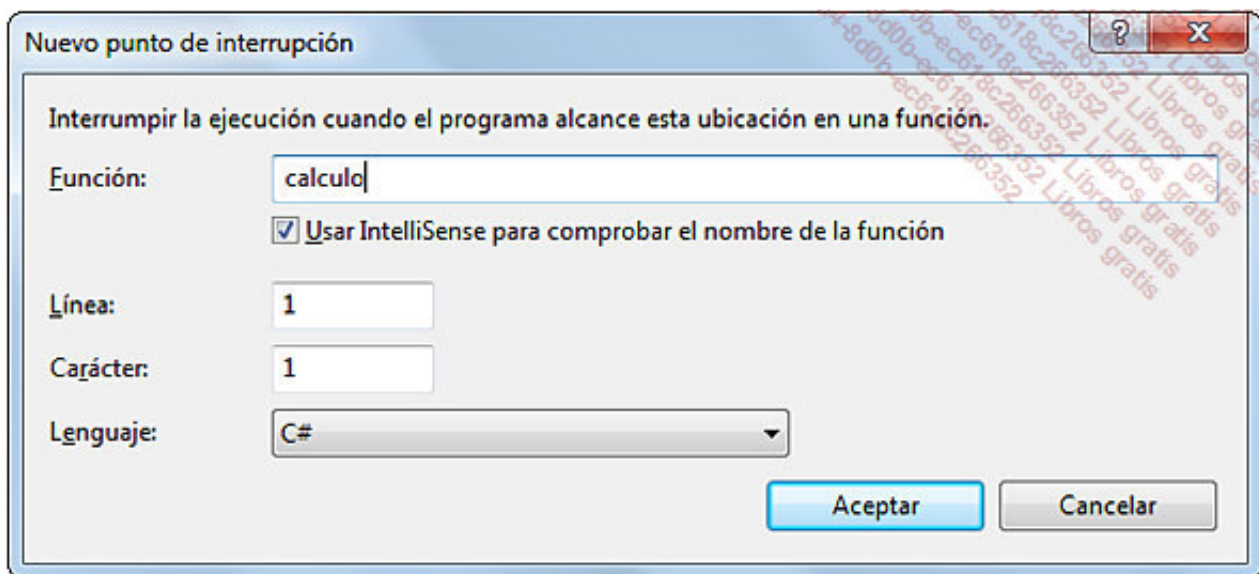
- efectuar un clic en el margen de la ventana de código,
- ubicar el cursor en la línea correspondiente y utilizar la combinación de teclas [Ctrl] **B**,
- utilizar la opción **Punto de interrupción - Insertar un punto de interrupción** del menú contextual de la ventana de código.

Todas estas técnicas insertan el punto de interrupción y materializan su ubicación con un punto rojo en el margen y el subrayado en rojo de la línea correspondiente.



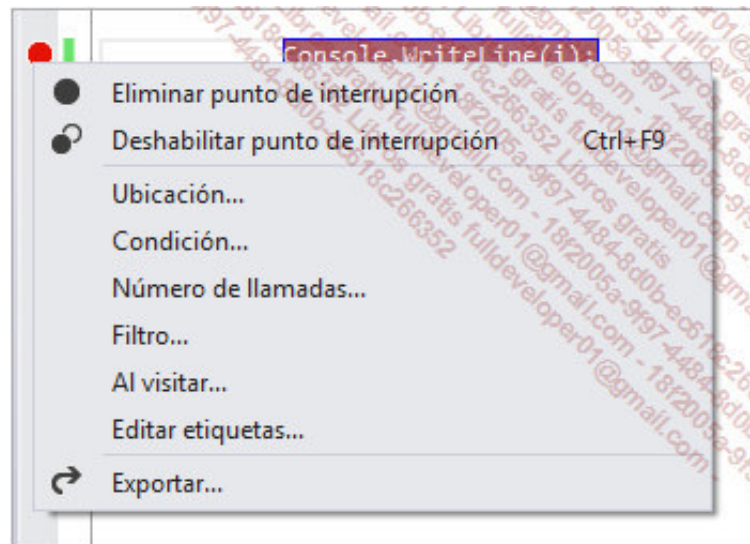
```
public static void calcula(int k)
{
    int j;
    for (int i = 0; i < 10000; i++)
    {
        Console.WriteLine(i);
    }
    Console.WriteLine("Fin del bucle");
}
```

Para todas estas soluciones, el código debe ser visible en el editor. La opción **Interrumpir en función** del menú **Depurar - Nuevo punto de interrupción** permite ubicar un punto de interrupción en un procedimiento o función sólo con teclear su nombre.



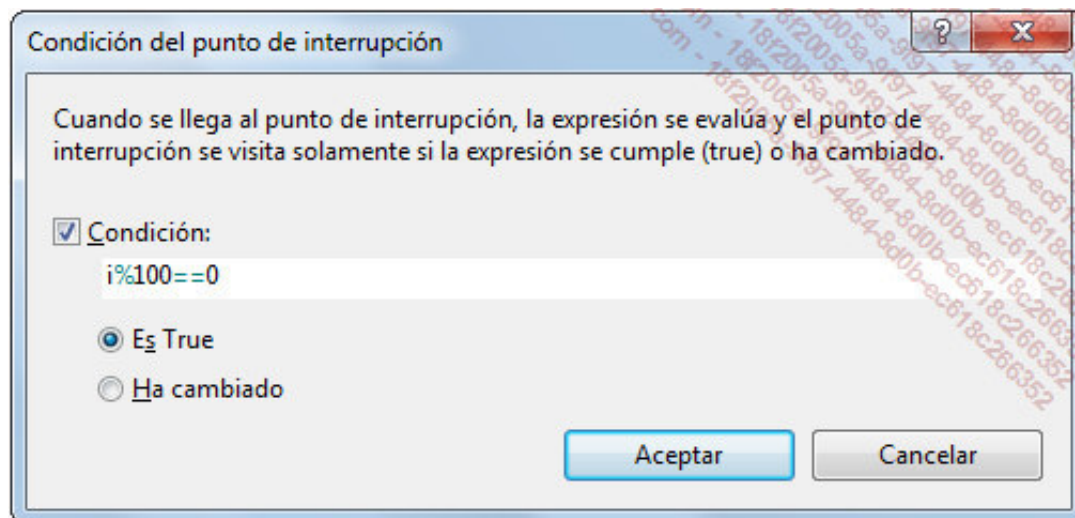
➤ Cuidado: el cuadro de diálogo le propone precisar en qué línea de la función desea ubicar un punto de interrupción, pero esta funcionalidad no está disponible para los puntos de interrupción en funciones.

Los puntos de interrupción así ubicados son incondicionales. En cuanto la ejecución llega a esta línea, la aplicación pasa a modo interrumpido. Se puede perfeccionar el funcionamiento de los puntos de interrupción añadiendo condiciones, un número de paso o transformándolos en TracePoint. Para ello, conviene modificar las propiedades del punto de interrupción a través del menú contextual disponible con un clic derecho en la línea afectada por el punto de interrupción.



Añadir una condición

Se puede someter a condición el paso a modo detenido. El siguiente cuadro de diálogo permite precisar las condiciones de ejecución del punto de interrupción.

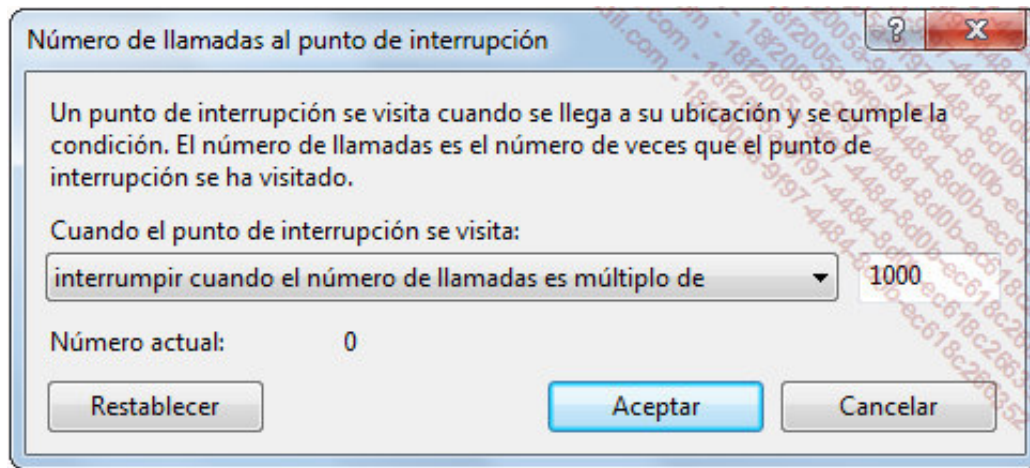


En este cuadro de diálogo debemos introducir una expresión que se evaluará a cada paso en el punto de interrupción. Entonces la ejecución se interrumpirá:

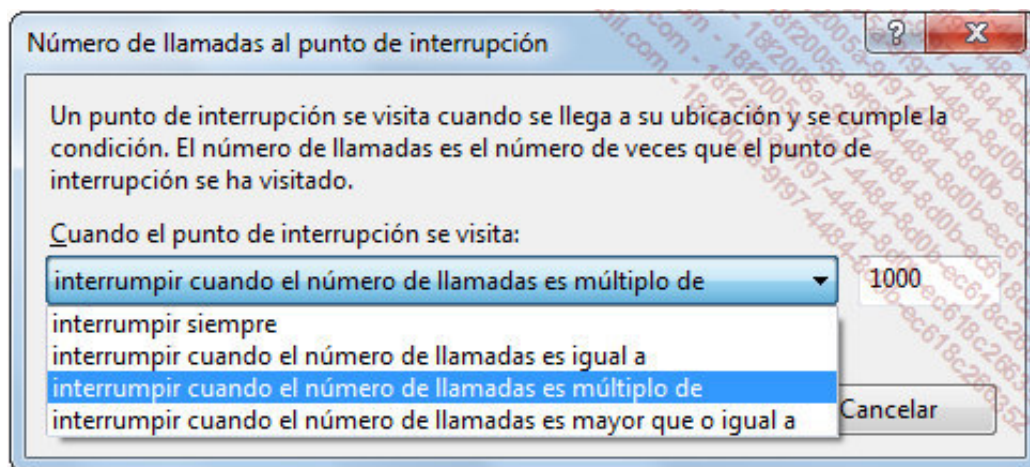
- si el resultado de la evaluación de la condición es verdadero,
- si el resultado de la evaluación de la condición ha sido modificado desde el último paso sobre este punto de interrupción. Hay que observar en este caso que al menos dos pasos son necesarios para provocar la interrupción de la aplicación (el primero sirve simplemente para guardar el resultado de la expresión).

Modificación del número de llamadas

Los puntos de interrupción son igualmente capaces contar el número de veces que se les alcanza y activarse para un número particular de llamadas.



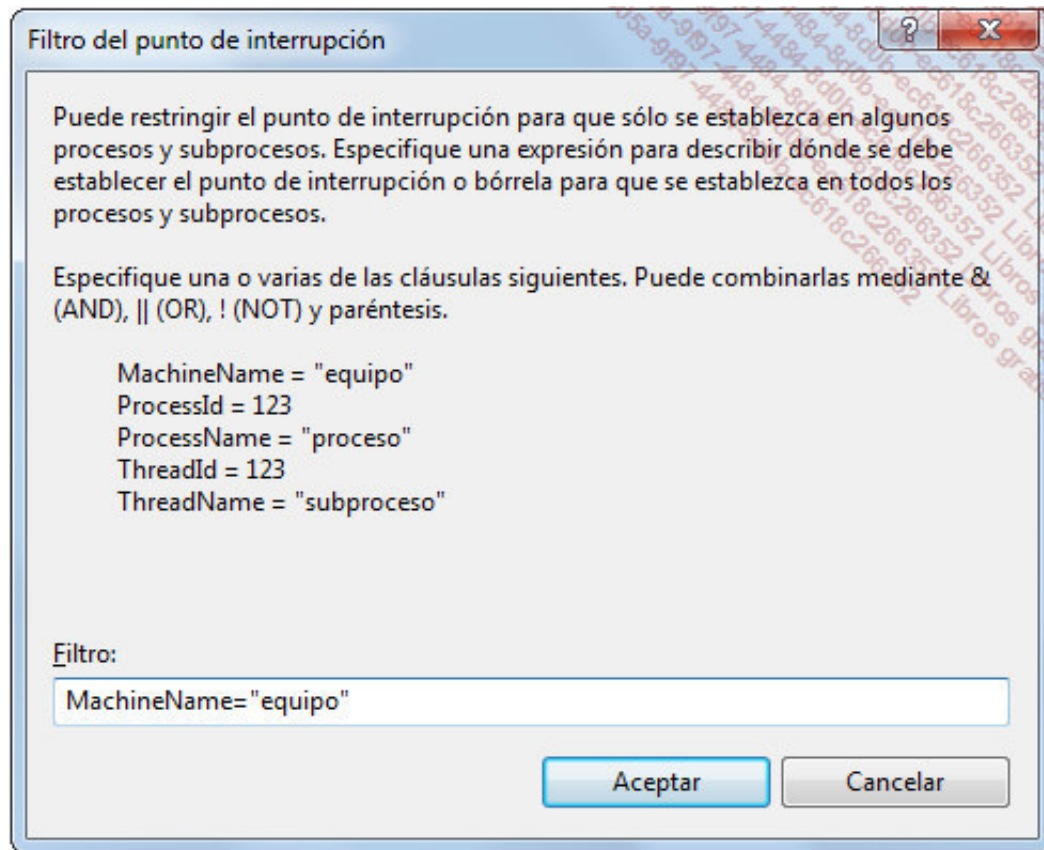
Este cuadro de diálogo nos permite definir el número de llamadas en el punto de interrupción para que éste pare efectivamente la aplicación. Hay cuatro opciones disponibles para la condición de interrupción en el número de llamadas.



Cuidado: si se indica una condición para el punto de interrupción, el número de llamadas corresponde al número de veces que la ejecución de la aplicación es pasada sobre esta línea con la condición comprobada. Con la configuración de nuestro ejemplo, pararemos en el bucle a la 100.000^o llamada (la condición será verdadera para $i = 0, 100, 200, 300, 400, 500, 600, 700, 800, 900$).

Filtrado

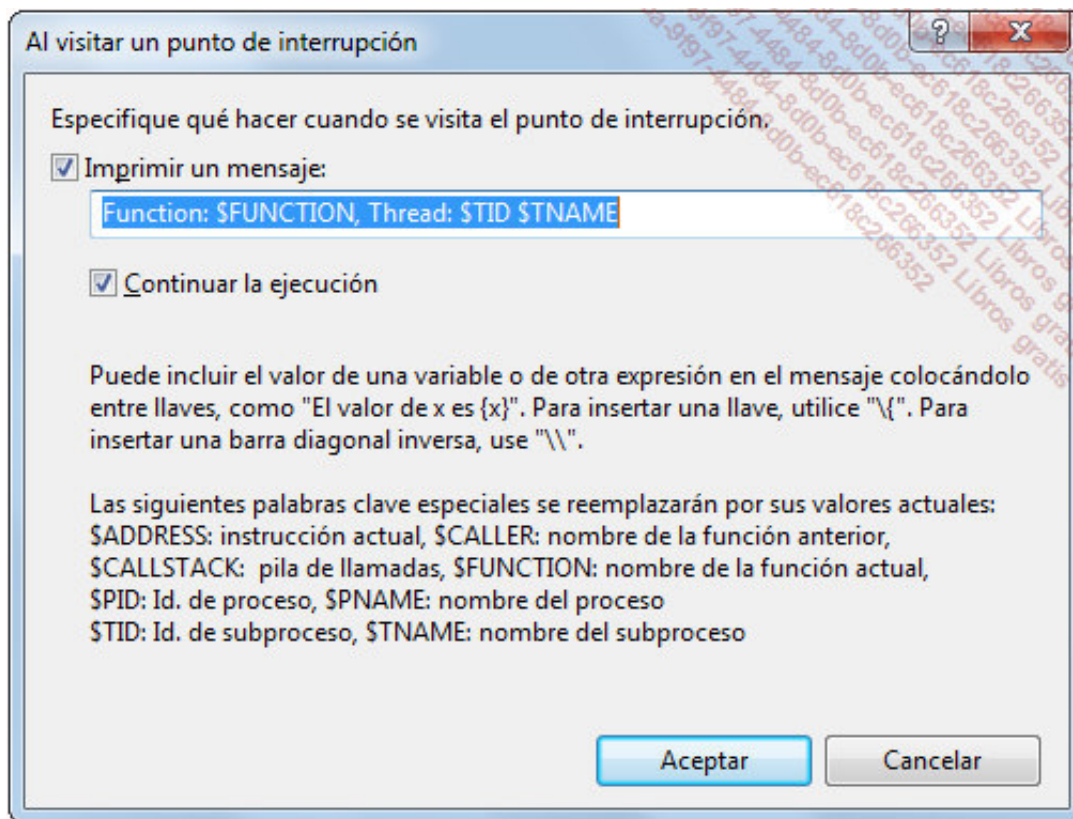
Los filtros permiten añadir criterios adicionales para la ejecución de un punto de interrupción. Estos criterios son relativos al nombre de la máquina donde se ejecuta la aplicación, así como el proceso o el thread.



Se debe expresar la condición con las palabras claves **MachineName**, **ProcessId**, **ProcessName**, **ThreadId**, **ThreadName** y los operadores **&** (y), **||** (o), **!** (not).

Transformación en TracePoint

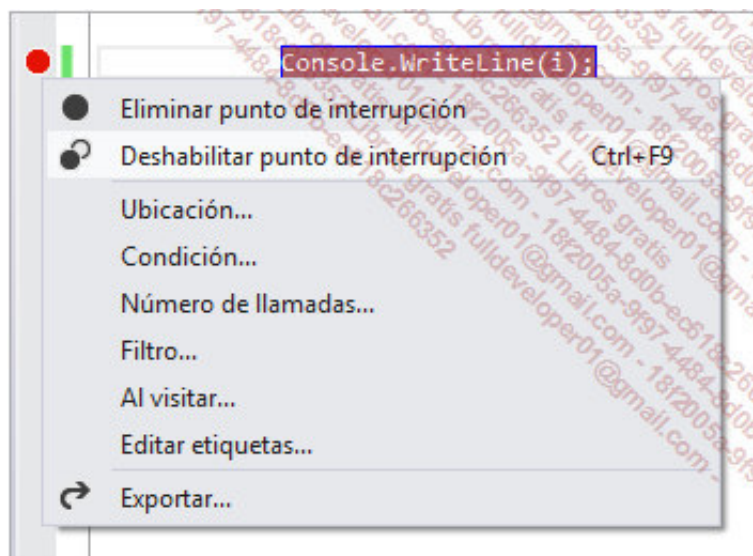
Un punto de interrupción se puede transformar en TracePoint precisando una acción particular que se debe ejecutar cuando se le alcanza.



Este cuadro de diálogo espera la formulación del mensaje visualizado en la ventana de salida cuando se alcanza el punto de interrupción. También autoriza la ejecución de una macro. Para que el punto de interrupción se transforme realmente en TracePoint, la opción **Continuar la ejecución** debe ser activada.

b. Activar, desactivar, suprimir un punto de interrupción

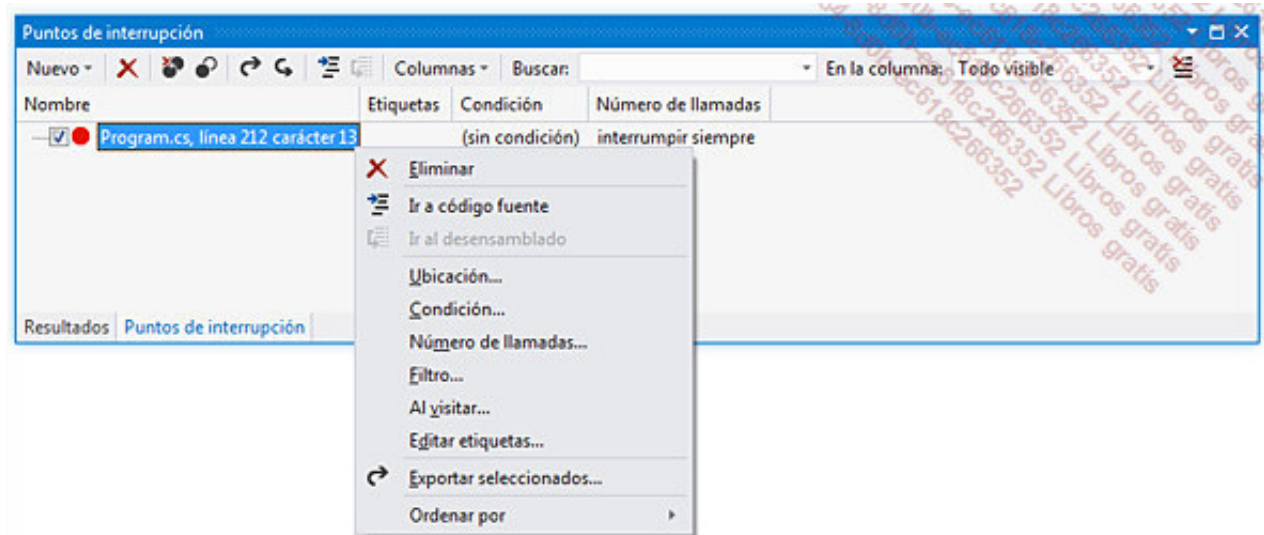
De manera momentánea se pueden desactivar los puntos de interrupción gracias al menú contextual.



Luego el punto de interrupción se puede activar de nuevo utilizando el menú contextual. Este mismo menú permite también la supresión de un punto de interrupción, pero es más rápido

efectuar un doble clic en el propio punto de interrupción. El menú **Depurar** propone también la opción **Eliminar todos los puntos de interrupción**, y evita así tener que recorrer muchas líneas de código para eliminar el conjunto de los puntos de interrupción.

Para facilitarnos la tarea durante la depuración de una aplicación, una ventana nos propone un resumen de todos los puntos de interrupción ubicados en su proyecto. Esta ventana es accesible a través del menú **Depurar - Ventanas - Puntos de interrupción**. Esta ventana propone un menú contextual que permite realizar las principales acciones sobre un punto de interrupción.



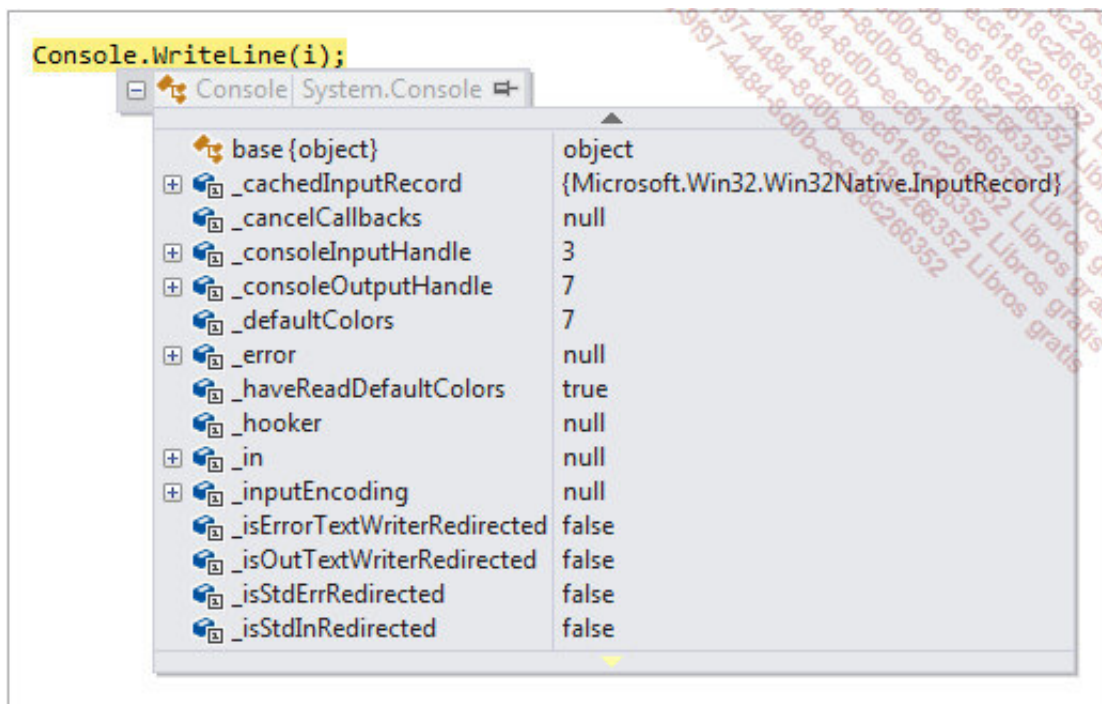
Se conservan los puntos de interrupción cuando cierra su proyecto.

3. Examen del contenido de las variables

Lo que interesa al depurador es poder seguir el funcionamiento de la aplicación durante su funcionamiento. Cuando la aplicación está en modo detenido, también es primordial poder visualizar los valores contenidos en las diferentes variables de la aplicación. Esta visualización nos permite comprobar el resultado de los tratamientos ya efectuados o anticipar los de los tratamientos efectuados en el resto del código.

a. DataTips

Los DataTips ofrecen un medio rápido para visualizar el contenido de una variable. Sólo hay que desplazar el cursor del ratón hasta el nombre y, después de un corto instante, se visualiza una ventana que presenta el contenido de la variable. Si la variable es un tipo complejo, como una instancia de clase por ejemplo, el DataTips propone un pequeño signo + que permite bajar en la estructura de la variable. Los datos visualizados también se pueden modificar directamente en el DataTips. El DataTips desaparece automáticamente cuando el ratón se aleja.

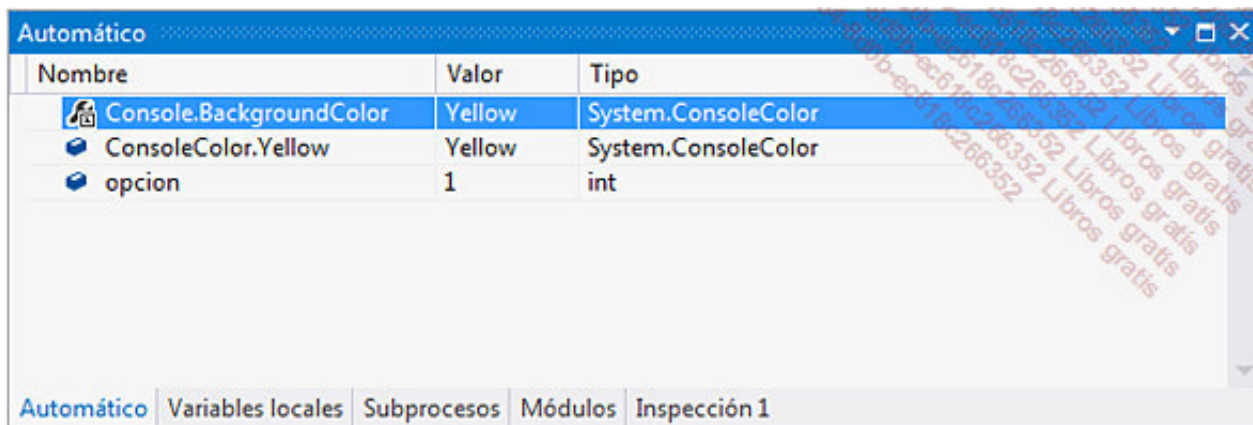


Para visualizar el resultado de un cálculo de una expresión, conviene previamente seleccionar la expresión y luego ubicar el cursor del ratón sobre la selección. El depurador evalúa la expresión y visualiza el resultado. El DataTips sólo puede visualizar las variables accesibles en el ámbito actual (variables declaradas en la función de dónde estamos detenidos o variables globales).

- Un pequeño truco: si desea visualizar el código enmascarado por el DataTips sin hacerlo desaparecer, puede utilizar la tecla [Ctrl], que lo vuelve transparente.

b. Ventana Automático

La ventana **Automático** visualiza las variables utilizadas en la instrucción corriente y en la instrucción anterior. Esta ventana es accesible a través del menú **Depurar - Ventanas - Automático**.



Esta ventana permite también la modificación del contenido de una variable haciendo doble clic en el valor en la ventana y validando la modificación, después de pulsar la tecla [Enter]. La aplicación continuará ejecutándose con este nuevo valor para la variable.

c. Ventana Variables locales

La ventana **Variables locales** es accesible por el mismo menú **Depurar - Ventanas - Variables locales** y posee un funcionamiento idéntico al de la ventana **Automático**, excepto que muestra todas las variables en el alcance actual.

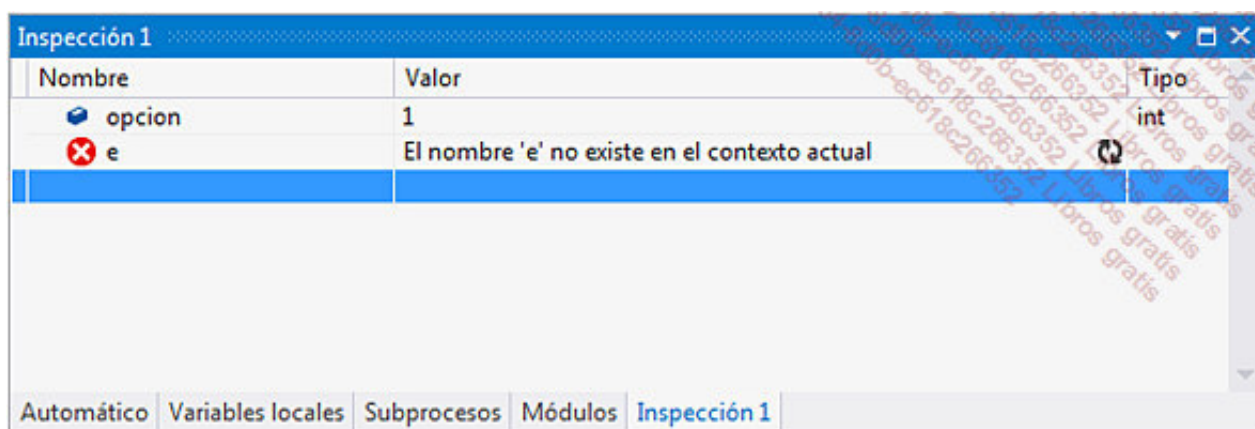
- En todas estas ventanas, no puede controlar la lista de las variables que se muestran, ya que el depurador determina la lista de ellas según el contexto en el cual se encuentra su aplicación. A veces es más práctico configurar manualmente la lista de las variables y expresiones que es preciso vigilar durante el funcionamiento de la aplicación.

d. Las ventanas Inspección

La ventana **Inspección** permite la visualización de las variables que parecen interesantes para la depuración de la aplicación. Esta ventana, o más bien estas ventanas, ya que hay cuatro ventanas **Inspección** disponibles, se puede visualizar con el menú **Depurar - Ventanas - Inspección**, luego **Inspección 1** a **Inspección 4**. A continuación debemos configurar la ventana añadiendo las variables y expresiones que deseamos visualizar. Con efectuar un doble clic en la columna **Nombre**, puede introducir lo que desea visualizar en la ventana. También puede efectuar un arrastrar-soltar desde la ventana de código. Si introduce un nombre de variable compleja (una instancia de clase por ejemplo), se visualiza el conjunto de sus propiedades en la ventana bajo la forma de árbol.

Sólo se visualizará el contenido de las variables si la aplicación está en modo detenido en una línea de código a partir de la cual se puede acceder a la variable. Por ejemplo, el contenido de las variables locales a un procedimiento o función sólo se visualiza si el código se detiene en este procedimiento o función.

En caso contrario, la ventana **Inspección** nos indica simplemente que esta variable no está declarada en la porción de código donde nos encontramos mostrándolo en caracteres en gris.



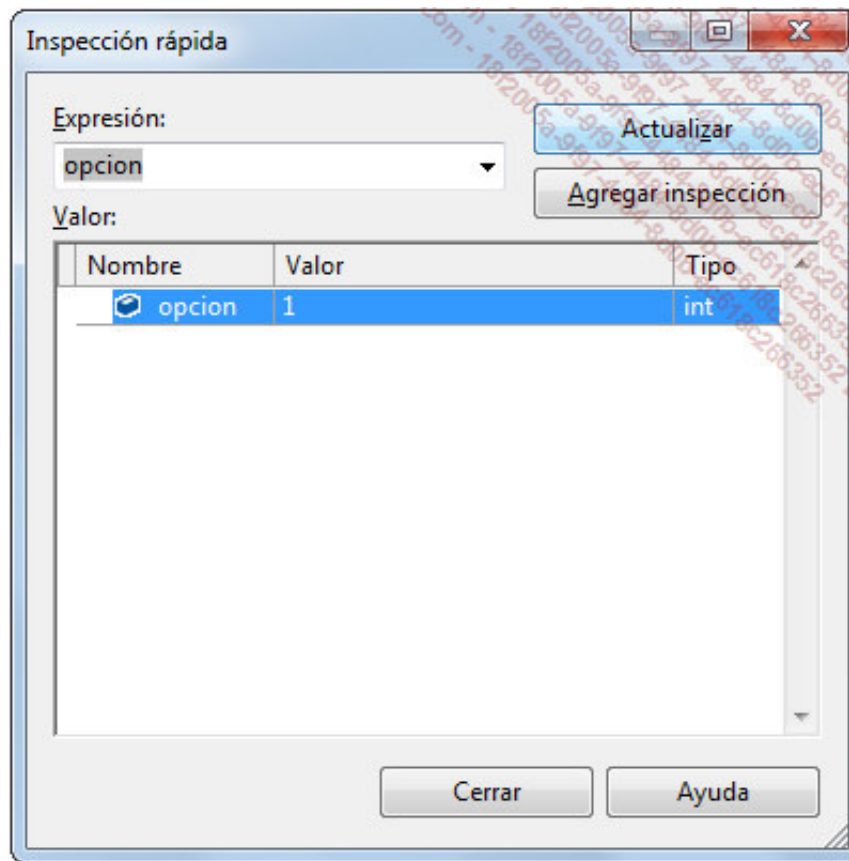
En este ejemplo se declara la variable en un procedimiento diferente a donde se encuentra el puntero de ejecución.

Como para las otras ventanas, se puede modificar el contenido de la variable haciendo doble clic encima para pasar al modo edición y confirmando la introducción con la tecla [Enter].

e. La ventana Inspección rápida

La ventana **Inspección rápida** propone el mismo principio de funcionamiento y es accesible con el

menú **Depurar - Inspección rápida**. En este caso, la variable o la expresión en la cual se encuentra el cursor se visualiza en la ventana **Inspección rápida**. Al ser la ventana modal, tendrá que cerrarla obligatoriamente antes de continuar con la depuración de su aplicación.

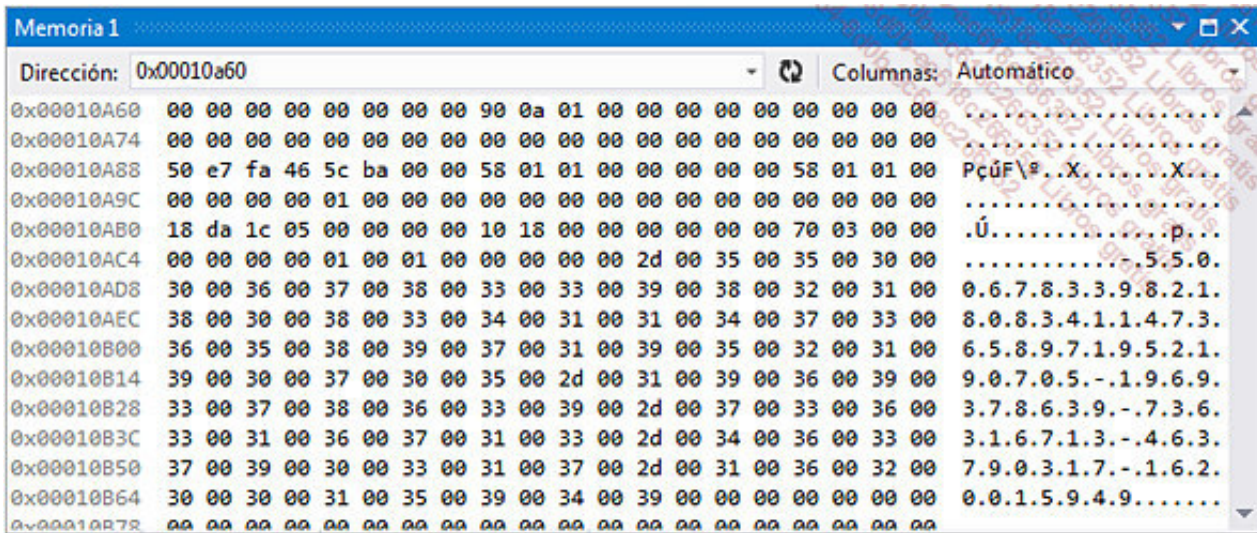


El botón **Agregar inspección** permite añadir rápidamente la expresión en la ventana **Inspección** para poder estudiarla en el resto de la depuración.

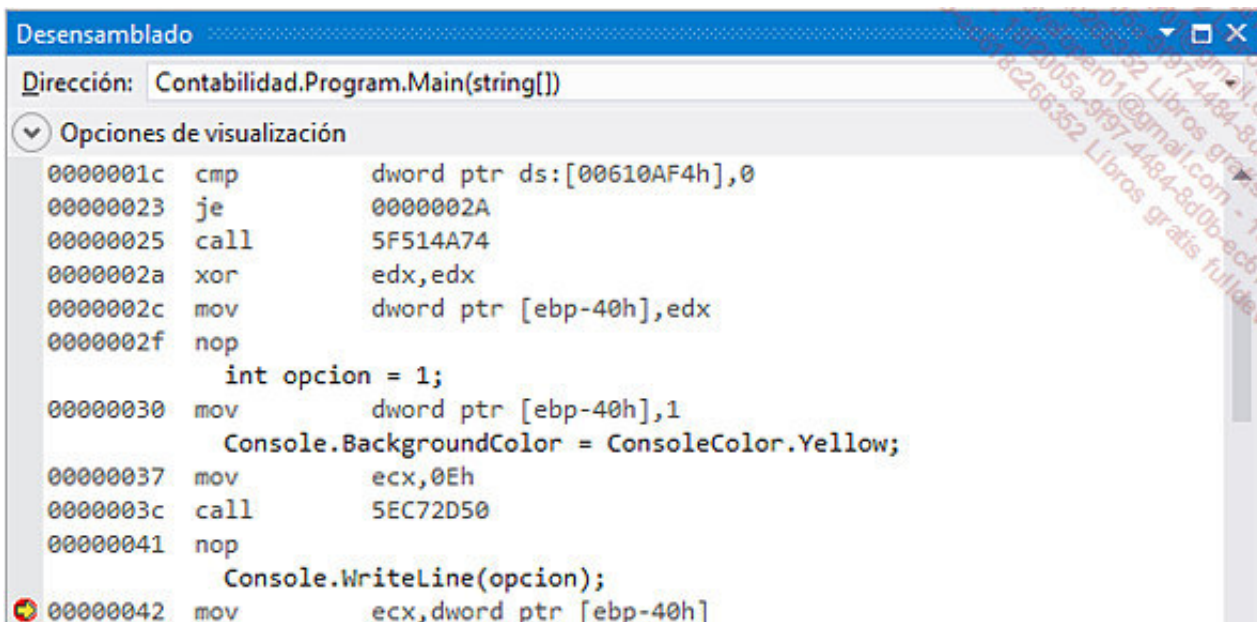
4. Las otras ventanas de depuración

Hay muchas otras ventanas disponibles para la depuración, pero algunas no son realmente útiles para el desarrollo de aplicaciones con Visual C#. Se reservan más para la prueba de aplicaciones desarrolladas con otros lenguajes, C++ por ejemplo.

Por ejemplo, es el caso de la ventana de memoria que permite la visualización del contenido de una zona de memoria de la cual conocemos la dirección.



Si lo desea, puede visualizar el código máquina correspondiente a las instrucciones Visual C#.



Subir

Otras técnicas de depuración

La compilación condicional

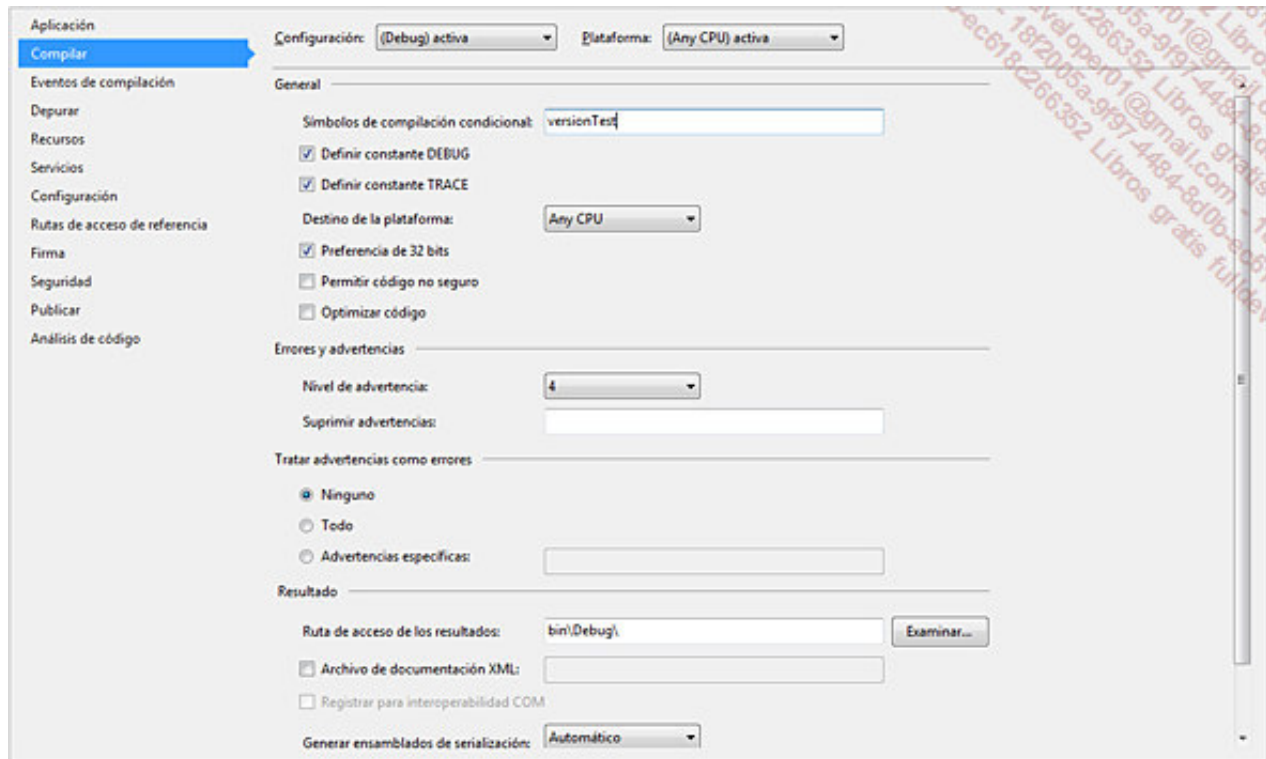
Puede utilizar la compilación condicional para especificar porciones que serán o no compiladas en función del valor de una constante que habrá definido previamente. Por ejemplo, puede probar varias soluciones para resolver un problema utilizando varios algoritmos y verificando el más eficaz de ellos.

El bloque de código cuya compilación se somete a condición se debe enmarcar con las instrucciones `#if condition` y `#endif`; en función del valor de la condición, el bloque de código será o no compilado. Por supuesto, es necesario que la variable o las variables utilizadas en la condición sea(n) inicializada(s) antes de su aparición en una instrucción `#if`.

```
#define versionTest
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.IO;
namespace depuracion
{
    class Program
    {
        public static void test()
        {
            #if versionTest
                for(int i=0;i<10000000;i++)
                {
                    Console.WriteLine(i);
                    calculo(i);
                }
            #else
                for(int i=0;i<10000000;i++)
                {
                    calculo(i);
                }
            #endif
        }
    }
}
```

Las constantes se pueden declarar con la declaración `#define`, como en el ejemplo siguiente, o aun en las propiedades del proyecto.

Si se declaran en el código, se debe hacer obligatoriamente en las primeras líneas del archivo.



- Sin embargo, hay que tener cuidado, ya que las constantes declaradas con estos dos métodos sólo se pueden utilizar para la compilación condicional y no son accesibles desde el código C#.

Subir

Condiciones generales de uso

Copyright - ©Editions ENI



Las aplicaciones de Windows

Las aplicaciones de Windows están basadas en una o varias ventanas que constituyen la interfaz entre el usuario y aquéllas. Para desarrollar este tipo de aplicación tenemos a nuestra disposición en el Framework .NET un conjunto de clases que permiten el diseño de su interfaz. A menudo estos elementos se agrupan bajo el término Tecnología Windows Forms. Una aplicación basada en los Windows Forms utilizará uno o varios formularios para construir la interfaz del usuario de la aplicación. En estos formularios (u hojas), colocaremos controles para definir exactamente el aspecto de la interfaz de la aplicación. Los formularios se crearán a partir de clases del Framework .NET que especializaremos con la inserción de funcionalidades. Dado que el formulario así creado es, en sí mismo, una clase, será posible reutilizarlo en otra aplicación añadiéndole funcionalidades adicionales mediante una relación de herencia. Los Windows Forms pueden ser creados directamente por el código, pero el entorno de desarrollo Visual Studio propone toda una gama de herramientas gráficas para facilitarnos la tarea. Utilizaremos principalmente esta técnica.

Las ventanas

Cuando comienza a diseñar una nueva aplicación con Windows Forms, el entorno de desarrollo añade automáticamente al proyecto un formulario. Este formulario sirve de punto de partida para la aplicación. Puede iniciar inmediatamente la ejecución de la solución y todo funciona. Es verdad que la aplicación no permite efectuar muchas cosas, pero dispone de todas las funcionalidades de una aplicación de Windows, y esto sin escribir una sola línea de código. En realidad, existe código correspondiente a esta aplicación, pero ha sido generado automáticamente por Visual Studio. Como este código nunca debe ser modificado manualmente, los archivos que lo contienen están ocultos en el explorador de soluciones.

Para verlos, puede utilizar el botón  de la barra de herramientas del explorador de soluciones. Así se dará cuenta de que ya hay muchos archivos en el proyecto. Todos los archivos reservados de Visual Studio tienen la extensión .designer.cs. Por supuesto, puede ver el contenido de dichos archivos.

El archivo asociado a la primera ventana de la aplicación se llama form1.designer.cs. Contendrá la descripción de todas las acciones, traducidas al código de Visual C#, que va a llevar a cabo para personalizar las características de la ventana.

Echemos un vistazo al contenido de este archivo:

```
namespace appliWindows
{
    partial class Form1
    {
        /// <summary>
        /// variable necesaria para el diseñador.
        /// </summary>
        private System.ComponentModel.IContainer components = null;
        /// <summary>
        /// Limpieza de los recursos utilizados.
        /// </summary>
        /// <param name="disposing">true si los recursos gestionados
        /// deben ser suprimidos; si no, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }
        #region Código generado por el Diseñador Windows Form
        /// <summary>
        /// Método requerido para que el Diseñador se encargue -
        /// lo modifique el contenido de este método con el editor de código.
        /// </summary>
        private void InitializeComponent()
        {
            this.components = new System.ComponentModel.Container();
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.Text = "Form1";
        }
        #endregion
    }
}
```

Contiene la definición de la clase correspondiente a la ventana que hereda de la clase `System.Windows.Forms.Form`. Esta definición conlleva una pequeña particularidad con respecto a

la definición de una clase, como hemos visto en el capítulo dedicado a la programación orientada a objetos. Se especifica la palabra clave `partial` delante del nombre de la clase. Esta palabra clave indica al compilador que el archivo sólo contiene una parte de la definición de la clase. La otra parte se encuentra en el archivo `form1.cs`. Esta técnica permite repartir los papeles:

- Visual Studio se encarga de generar en el archivo `form1.designer.cs`, el código que corresponde a la personalización de la ventana.
- Usted es el responsable del código que contiene en el archivo `form1.cs`, encargado de la personalización del funcionamiento de la ventana.

Esta solución limita los riesgos de modificación involuntaria de la parte del código reservada a Visual Studio. El elemento más importante está constituido del método `InitializeComponent`. Este método es llamado automáticamente durante la creación de una instancia de la ventana en la llamada al constructor.

Si añade un constructor sobrecargado, se hace responsable de esta llamada. Lo más sencillo en este caso consiste en llamar al constructor por defecto.

```
public Form1(int i):this()
{
}
```

➤ Piense también en respetar, en el constructor por defecto, la ubicación de sus inicializaciones particulares. Si están ubicadas antes de la llamada al método `InitializeComponent`, éste podría modificarlas. Antes de la llamada al método `InitializeComponent`, los elementos gráficos de la ventana no están disponibles porque el papel principal de este método es precisamente crearlos e inicializar algunas de sus propiedades.

También se crea un método `dispose` para poder suprimir todos los objetos instanciados por la clase. Este método comienza por suprimir los objetos creados y luego llama al método `dispose` de la clase madre.

```
protected override void Dispose(bool disposing)
{
    if (disposing && (components != null))
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}
```

Hemos acabado ya con el código generado automáticamente. Echemos un vistazo a cómo codificar la apariencia y el comportamiento de nuestra ventana gracias a estas propiedades.

1. Dimensión y posición de las ventanas

La posición de la hoja en la pantalla (o en su contenedor) es establecida por la propiedad `location`. Esta propiedad es una estructura compuesta de dos miembros que indican las coordenadas de la esquina superior izquierda de la hoja con respecto a la esquina superior izquierda de la pantalla. Se pueden modificar los miembros de esta estructura en la ventana de propiedades de Visual C#. De hecho, cuando modifica propiedades, se inserta algo de código para tomar en cuenta dichas modificaciones. Por ejemplo, si deseamos que nuestra ventana aparezca en las coordenadas 100 100, modificamos la propiedad `Location`.

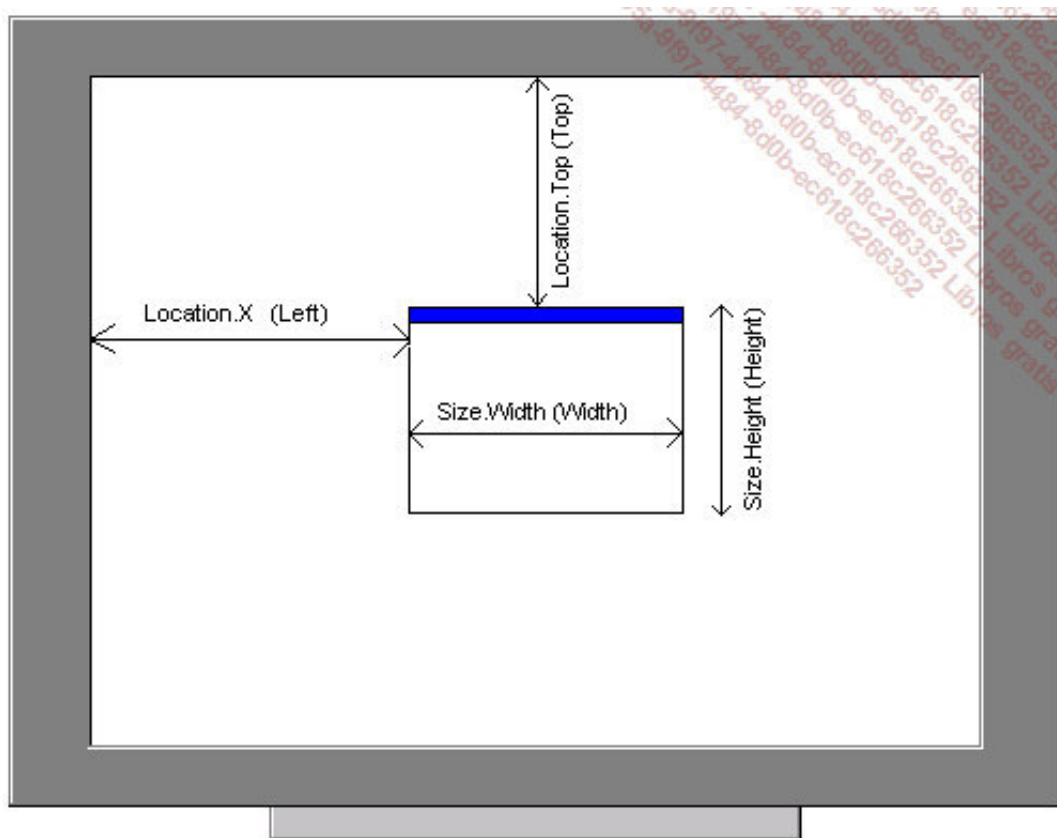
Localizable	False
☒ Location	100; 100
X	100
Y	100
Locked	False

Si echamos un vistazo por nuestro código, veremos que esta propiedad se ha visto modificada.

```
this.Location = new System.Drawing.Point(100, 100);
```

Las dimensiones de las ventanas pueden modificarse con la propiedad `size`, que contiene dos miembros `Width` y `Height` que indican la anchura y la altura de la ventana.

Resumimos todo esto mediante este pequeño esquema:



Las unidades son píxeles para todas las propiedades relativas a las posiciones y dimensiones del objeto. Las propiedades `Left`, `Top`, `Height` y `Width` están disponibles en el código, pero no en la ventana de propiedades. La correspondencia con las propiedades `Location` y `Size` de estas propiedades está indicada entre paréntesis sobre el esquema.

Se actualizan dichas propiedades durante la ejecución de la aplicación si la ventana se desplaza o se redimensiona. Son accesibles por el código de la aplicación.

La anchura y la altura de la ventana pueden evolucionar entre los límites fijados por las propiedades `MinimumSize` y `MaximumSize`. Por defecto se inicializan estas dos propiedades a cero. En este caso, cero indica que no hay límite fijado para el tamaño de la ventana.

Hay otras dos propiedades que establecen el comportamiento de la ventana durante el inicio de la

aplicación.

La propiedad `StartPosition` permite imponer una posición a la ventana, al inicio de la aplicación. La tabla siguiente resume los posibles valores:

Valor de la propiedad	Efecto sobre la ventana
Manual	Se utilizan las propiedades <code>Location</code> y <code>Size</code> para visualizar la ventana.
CenterParent	La ventana está centrada con respecto a la ventana madre.
CenterScreen	La ventana está centrada con respecto a la pantalla.
WindowsDefaultLocation	El sistema ubica automáticamente las ventanas partiendo de la esquina superior izquierda de la pantalla. Las ventanas se desplazan hacia la parte inferior derecha de la pantalla con cada visualización de una nueva ventana. El tamaño de cada ventana queda especificado por la propiedad <code>Size</code> .
WindowsDefaultBounds	Mismo principio que anteriormente, pero el tamaño viene determinado por el sistema al mostrar la ventana.

La propiedad `WindowState` indica el estado de la hoja. Hay tres valores posibles:

Valor de la propiedad	Estado de la ventana
Normal	Tamaño definido por la propiedad <code>Size</code> .
Minimized	Ventana como icono en la barra de tareas.
Maximized	Ventana a tamaño completo.

Por supuesto, se pueden modificar todas estas propiedades por el código de la aplicación. En contraposición, es más eficaz utilizar los métodos `SetLocation` y `SetSize`, que permiten dimensionar y posicionar la hoja directamente. La utilización de estos métodos o la manipulación directa de las propiedades inicia los eventos `Resize` y `Move` de la hoja correspondiente.

Para añadir el código de gestión de estos eventos, debe crear un método que repete la firma de los delegados y luego asociar cada método al evento. Visual Studio facilita muchísimo este trabajo gracias

a la ventana de propiedades. El botón  de esta última permite obtener la lista de los eventos

disponibles para el elemento seleccionado. Sólo hace falta efectuar un doble clic en el nombre del evento que nos interesa para que Visual Studio genere un esqueleto de método y asocie automáticamente dicho método al evento. Se realiza esta asociación en el método `InitializeComponent`. Presentamos un ejemplo de código generado:

```
private void InitializeComponent()
{
    this.SuspendLayout();
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleModeMode.Font;
    this.ClientSize = new System.Drawing.Size(292, 266);
    this.Location = new System.Drawing.Point(100, 100);
```

```
this.Name = "Form1";
this.Text = "Form1";
this.Move += new System.EventHandler(this.Form1_Move);
this.Resize += new System.EventHandler(this.Form1_Resize);
this.ResumeLayout(false);
}
private void Form1_Resize(object sender, EventArgs e)
{
}
private void Form1_Move(object sender, EventArgs e)
{
}
```

Sólo nos queda el código de los métodos generados automáticamente.

```
private void Form1_Resize(object sender, EventArgs e)
{
    Console.WriteLine("Mi anchura: {0}", this.Size.Width);
    Console.WriteLine("Mi altura: {0}", this.Size.Height);
}
private void Form1_Move(object sender, EventArgs e)
{
    Console.WriteLine("Estoy en la posición x : {0}", this.Location.X);
    Console.WriteLine("Estoy en la posición y : {0}", this.Location.Y);
}
```

Obtenemos la siguiente información:

```
Estoy en la posición X :263
Estoy en la posición Y :311
Mi anchura: 364
Mi altura: 122
```

Una pequeña curiosidad para terminar con el tamaño y posición de las ventanas: si reducimos nuestra ventana hasta convertirla en icono, haciendo clic en el botón **Minimizar**  o modificando la propiedad `WindowState`, obtenemos los siguientes valores:

```
Estoy en la posición X :-32000
Estoy en la posición Y :-32000
Mi anchura: 160
Mi altura: 24
```

Las posiciones X e Y de la hoja son, efectivamente, valores negativos. En realidad, usted puede utilizar valores comprendidos en el límite de los enteros. Sólo la parte de su ventana comprendida entre cero y la anchura y la altura de su pantalla será visible. Puede usar el método `GetBounds` del objeto `Screen` para obtener el tamaño de la pantalla.

```
private void Form1_Move(object sender, EventArgs e)
{
    Console.WriteLine("Estoy en la posición x: {0}", this.Location.X);
    Console.WriteLine("Estoy en la posición y: {0}", this.Location.Y);
    Console.WriteLine(" en una pantalla de {0} por {1}",
        Screen.GetBounds(this).Width,
        Screen.GetBounds(this).Height);
}
```

Este código nos permite conocer la dimensión de la pantalla que visualiza la aplicación.

Estoy en la posición X: 115
Estoy en la posición Y: 203
en una pantalla de 1280 por 800

Para que el usuario pueda desplazar o modificar el tamaño de una ventana, debe disponer de las herramientas necesarias:

- Una barra de título para poder coger la ventana y desplazarla.
- Un borde para poder dimensionarla.
- Botones para poder maximizarla, minimizarla y restaurar su tamaño normal.

Para poder redimensionar la ventana, ésta debe disponer de un borde de tipo «sizable» asignado a su propiedad `FormBorderStyle`.

Para ser desplazada, una ventana debe poseer una barra de título. Se puede ocultar esta barra con la propiedad `ControlBox` colocada en `false`. En este caso, incluso el título de la ventana especificado por la propiedad `Text` deja de verse. En caso de mostrar la barra de título, los diferentes botones que aparecen encima pueden ser controlados por las siguientes propiedades:

<code>MinimizeBox</code>	Visualización o no del botón de minimizar la ventana.
<code>MaximizeBox</code>	Visualización o no del botón de maximizar la ventana.
<code>HelpButton</code>	Visualización del botón de ayuda. Visible sólo si los dos botones anteriores no están a la vista.

2. Colores y fuentes utilizados en las ventanas

La propiedad `BackColor` indica el color de fondo utilizado en la ventana. También se utilizará este color para todos los controles que luego se colocarán en la hoja. La propiedad `ForeColor` indica el color de los elementos que se dibujarán directamente sobre la hoja o el color de la leyenda de los controles ubicados en dicha hoja. Hay cuatro posibilidades para asignar un valor a estas propiedades de color:

- por la ventana de propiedades, seleccionando un color en la pestaña **Personalizar**.
- por la ventana de propiedades, seleccionando un color Web. Estos colores corresponden a los colores disponibles en el lenguaje HTML.
- por la ventana de propiedades, seleccionando un color del sistema. En este caso, su aplicación se adaptará automáticamente al entorno del puesto de trabajo en el cual está instalada. Si el usuario ha configurado su puesto para tener botones de color rosa fosforito, encontrará la misma apariencia en su aplicación.
- fabricando usted mismo el color con un poco de rojo, un poco de verde, un poco de azul. Para mezclar todo eso y obtener el color final, utilice el método `FromARGB`, que toma como parámetro la cantidad de rojo, verde y azul y proporciona el color resultante. Las cantidades de cada color son valores incluidos entre 0 y 255. Este último valor corresponde al color puro.

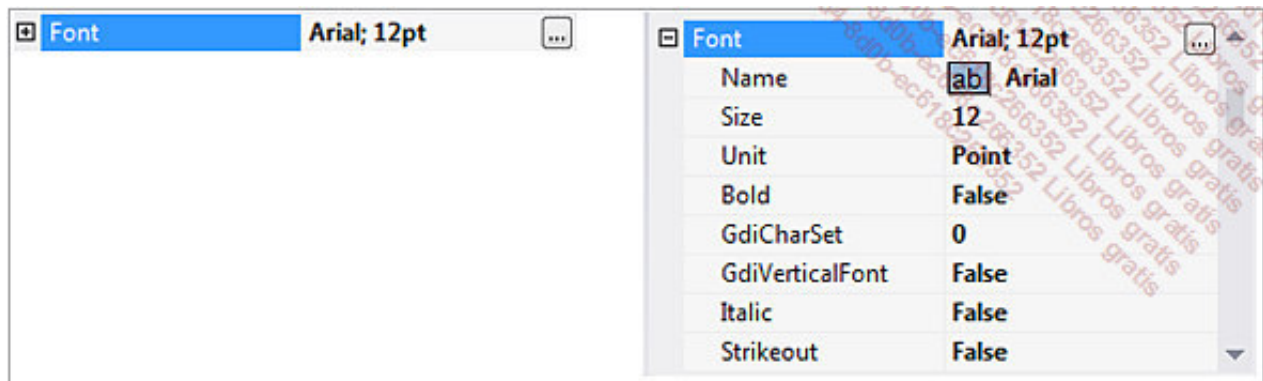
La propiedad `Opacity` permite ajustar la transparencia de su hoja. El valor debe estar comprendido entre cero (ventana transparente) y uno (ventana opaca). Puede igualmente tapizar la ventana indicando una imagen de fondo para su ventana con la propiedad `BackgroundImage`. Si la imagen no es bastante grande para tapar la ventana, se reproduce en mosaico.

De la misma manera, puede especificar que un color se considere transparente en su ventana. Para ello, debe asignar a la propiedad `TransparencyKey` el valor de este color.

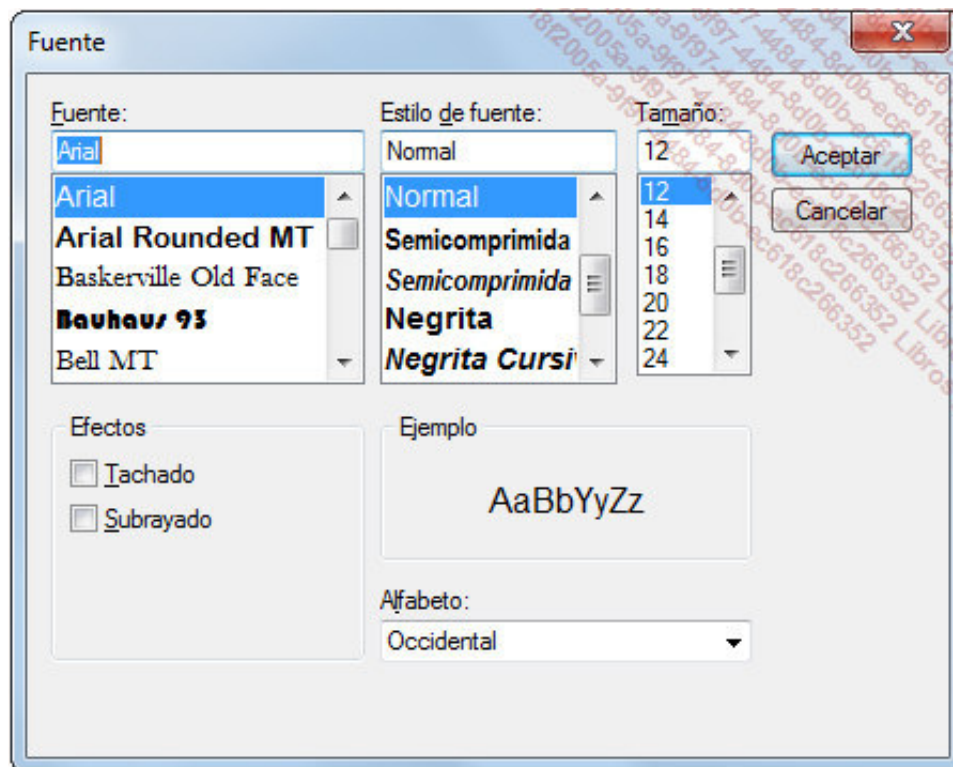
Para ilustrar la utilización poco obvia de esta propiedad, hemos indicado en la ventana siguiente que el color blanco sea transparente (vemos una parte de la ventana de propiedades a través de la zona de texto).



La propiedad `Font` permite especificar las características de la fuente de caracteres, utilizada para la visualización de texto directamente en la ventana. También se utilizará esta fuente por defecto para todos los controles que ubicaremos en la ventana. Puede modificar las propiedades directamente en la ventana de propiedades, desplegando la propiedad `Font` mediante un clic en el signo más (+) que se muestra al lado de la propiedad.



También podrá modificar las características de la fuente con el cuadro de diálogo estándar de selección de fuente. Éste se visualiza utilizando el botón , a la derecha de la propiedad `Font` en la ventana de propiedades.



3. Las ventanas MDI

Las aplicaciones están constituidas por dos tipos de hojas:

- las hojas madre,
- las hojas hija.

En Visual C#, se utiliza la misma clase básica para los dos tipos de ventana. En el primer caso, se indicará simplemente el hecho de que la ventana es una ventana madre MDI poniendo a **True** su propiedad `IsMdiContainer`. Luego, para añadir una ventana hija, conviene primero crear la ventana y después asociarla a una ventana madre por su propiedad `MdiParent`.

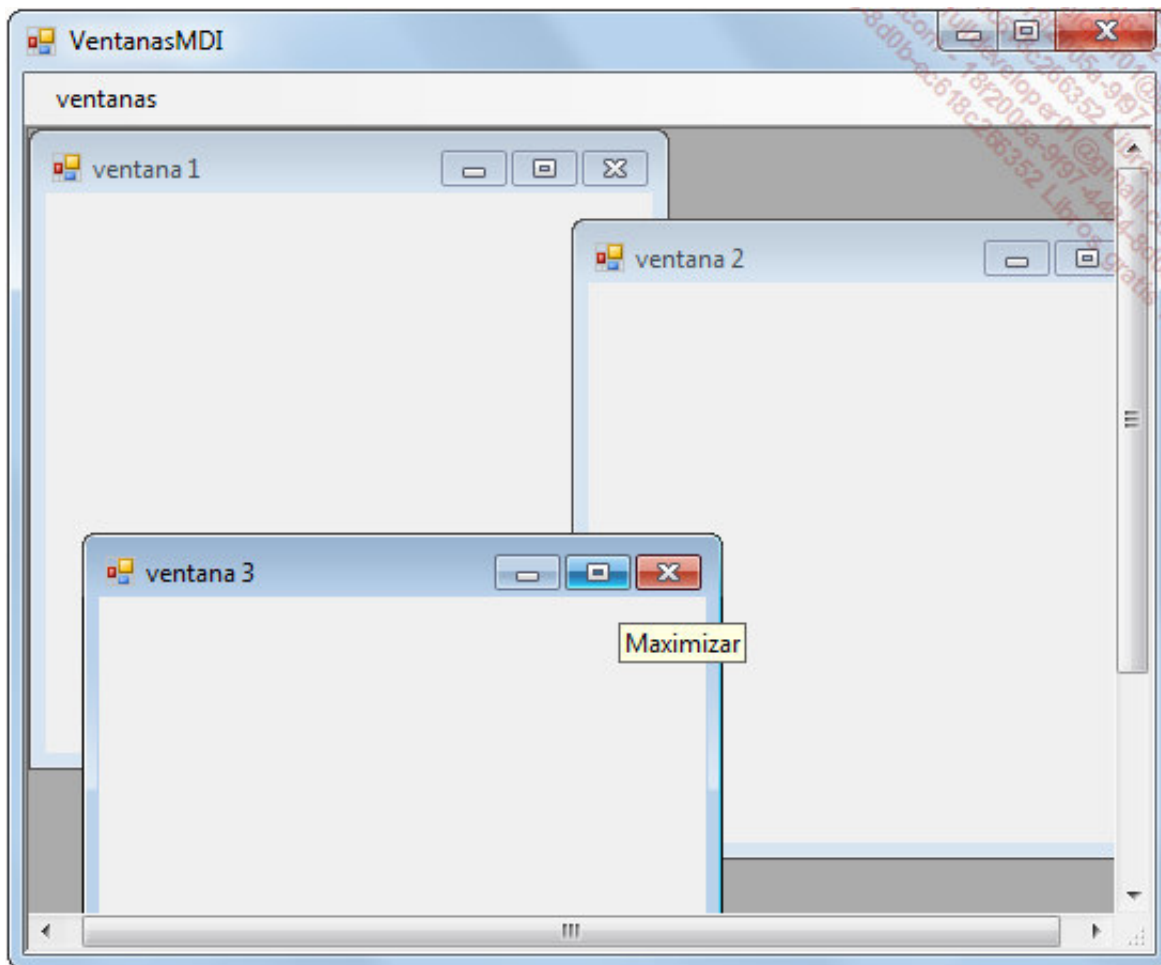
A continuación presentamos un código que crea tres ventanas y las transforma en ventanas hija MDI:

```
public partial class VentanasMDI : Form
{
    public VentanasMDI()
    {
        InitializeComponent();
        Form ventana1, ventana2, ventana3;
        ventana1 = new Form();
        ventana1.Text="ventana 1";
        ventana1.MdiParent=this;
        ventana1.Show();
        ventana2 = new Form();
        ventana2.Text = "ventana 2";
        ventana2.MdiParent = this;
        ventana2.Show();
        ventana3 = new Form();
        ventana3.Text = "ventana 3";
        ventana3.MdiParent = this;
    }
}
```

```
ventana3.Show();
```

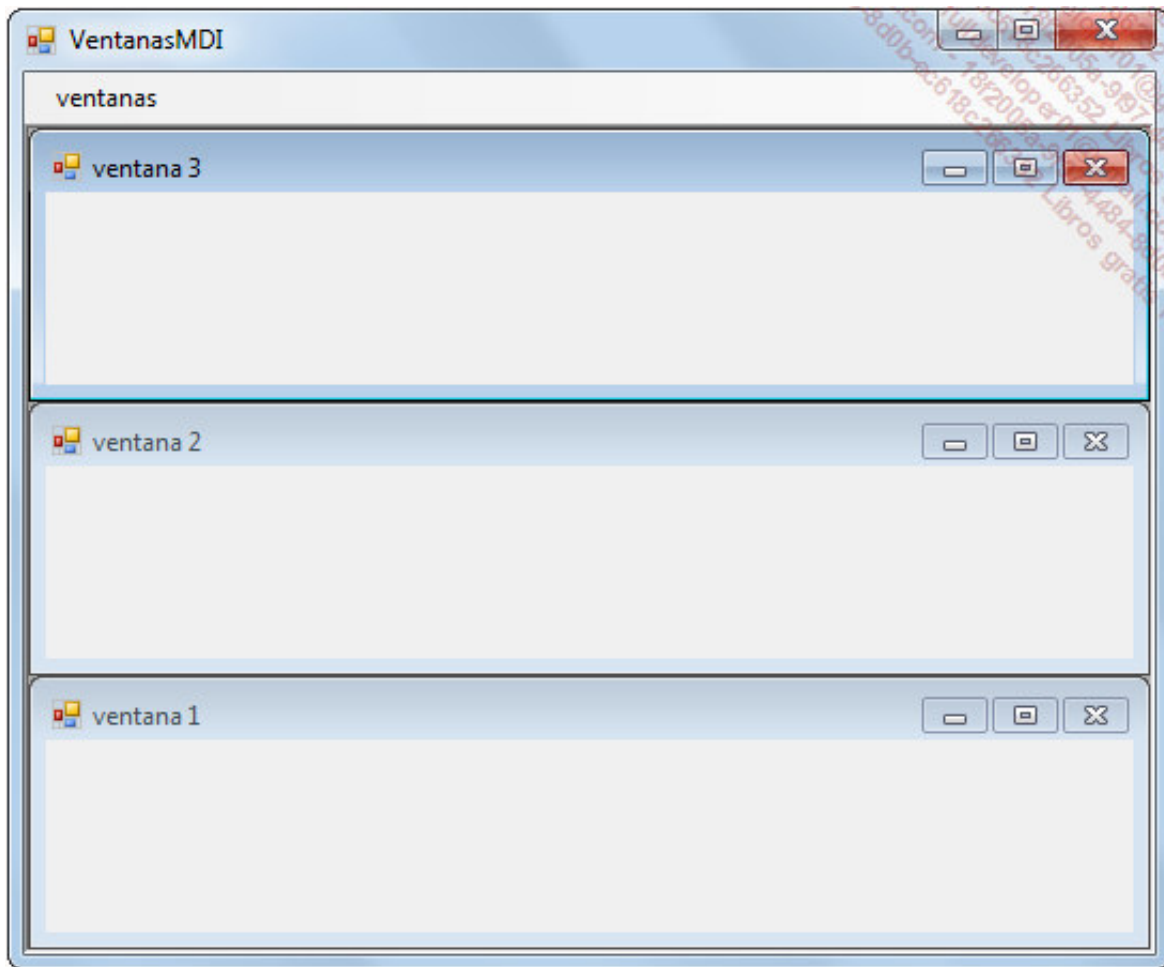
```
}
```

```
}
```

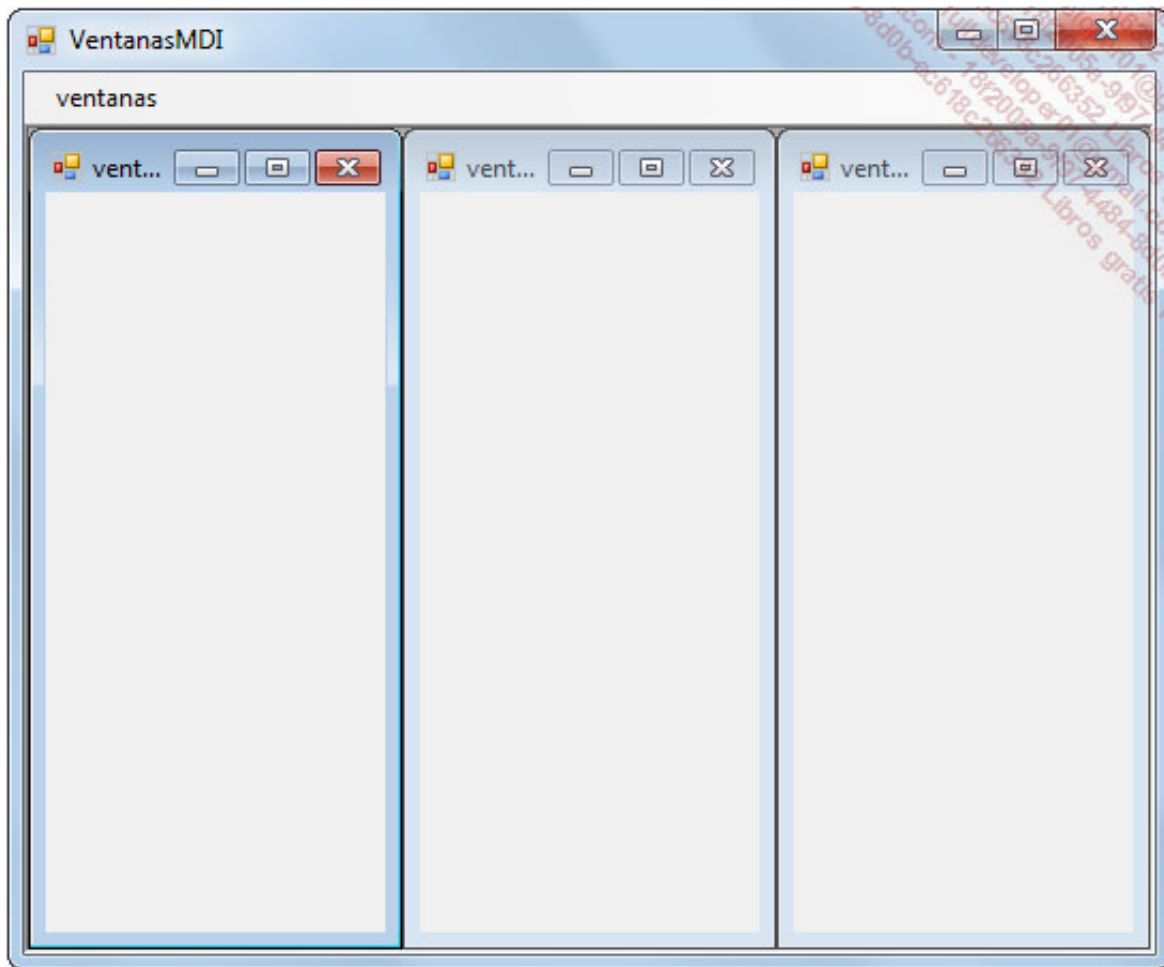


Para obtener ventanas hijas bien ordenadas en su ventana madre, debe llamar al método `LayoutMdi` pasándole como parámetro una de las constantes predefinidas de la enumeración `MdiLayout`:

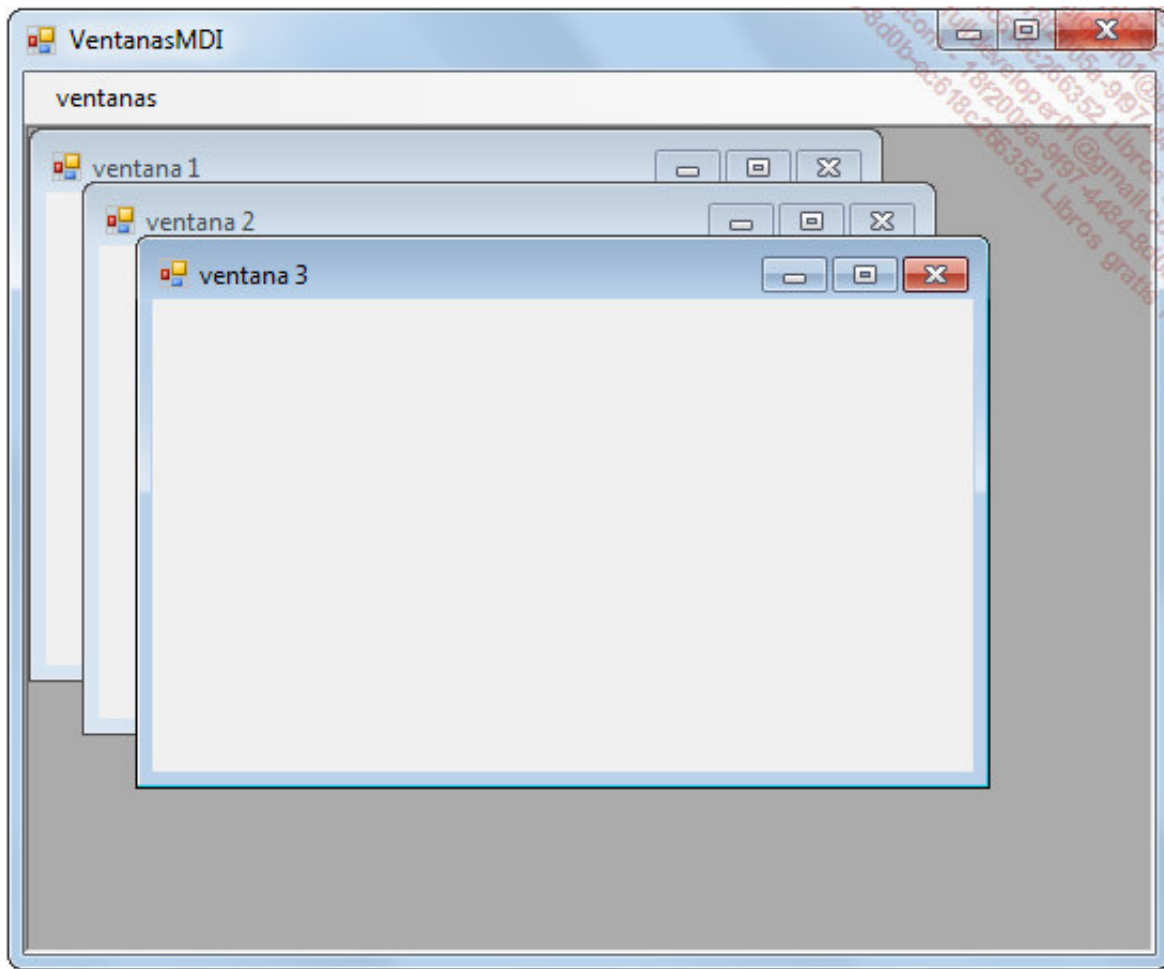
```
this.LayoutMdi(MdiLayout.TileHorizontal);
```



```
this.LayoutMdi (MdiLayout.TileVertical);
```

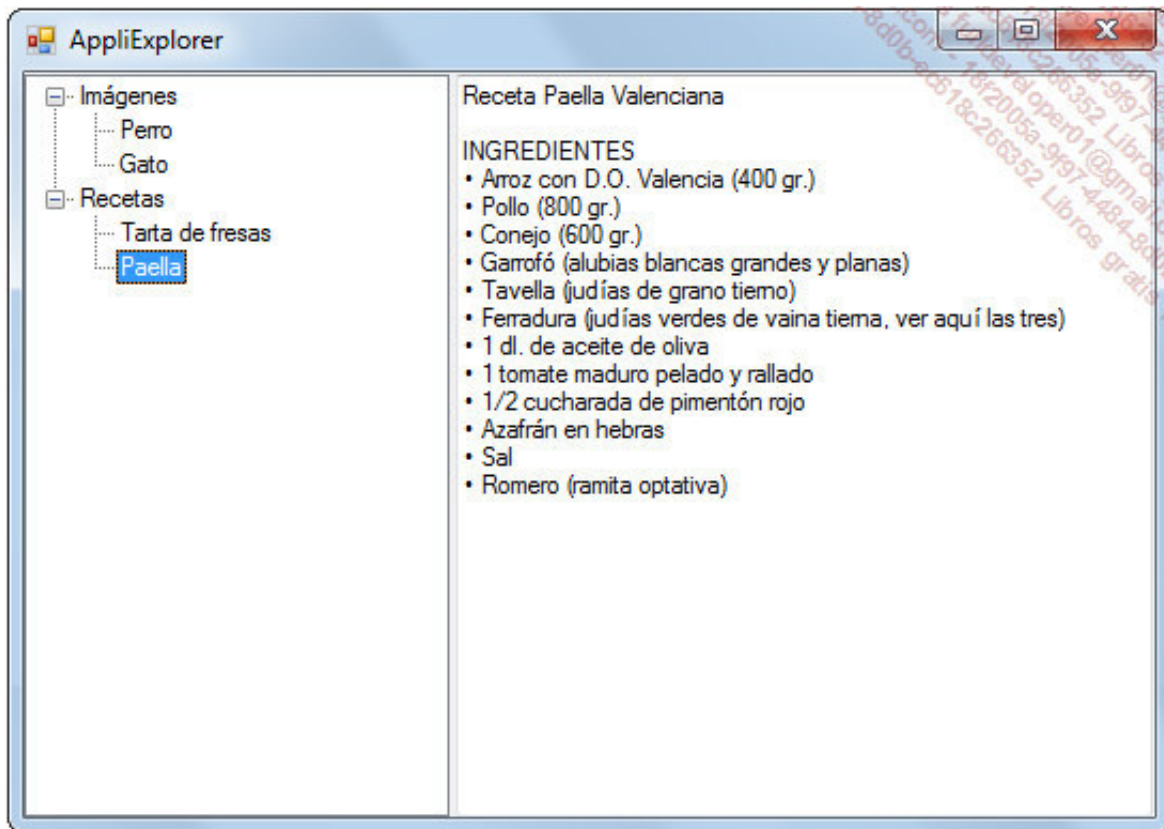



```
this.LayoutMdi (MdiLayout.Cascade) ;
```



- Se suele llamar a estos métodos a través de un menú de la aplicación que proporciona la lista de las ventanas abiertas en la aplicación. Veremos cómo poner esto en práctica en la sección dedicada a los menús.

Para ilustrar la operación alternativa de ventanas MDI, las vamos a utilizar para realizar una aplicación de tipo explorador. A continuación vemos el aspecto general de la aplicación.



En la parte izquierda, siempre visible, tenemos en forma de árbol los documentos disponibles en la aplicación. Según la selección hecha en este árbol, la zona derecha se adapta para visualizar o la imagen o el texto de una receta. Por ello necesitamos tres ventanas diferentes:

- la ventana principal que va a contener el control `TreeView`, y luego las ventanas encargadas de mostrar los documentos,
- una ventana para visualizar imágenes,
- una ventana para mostrar texto.

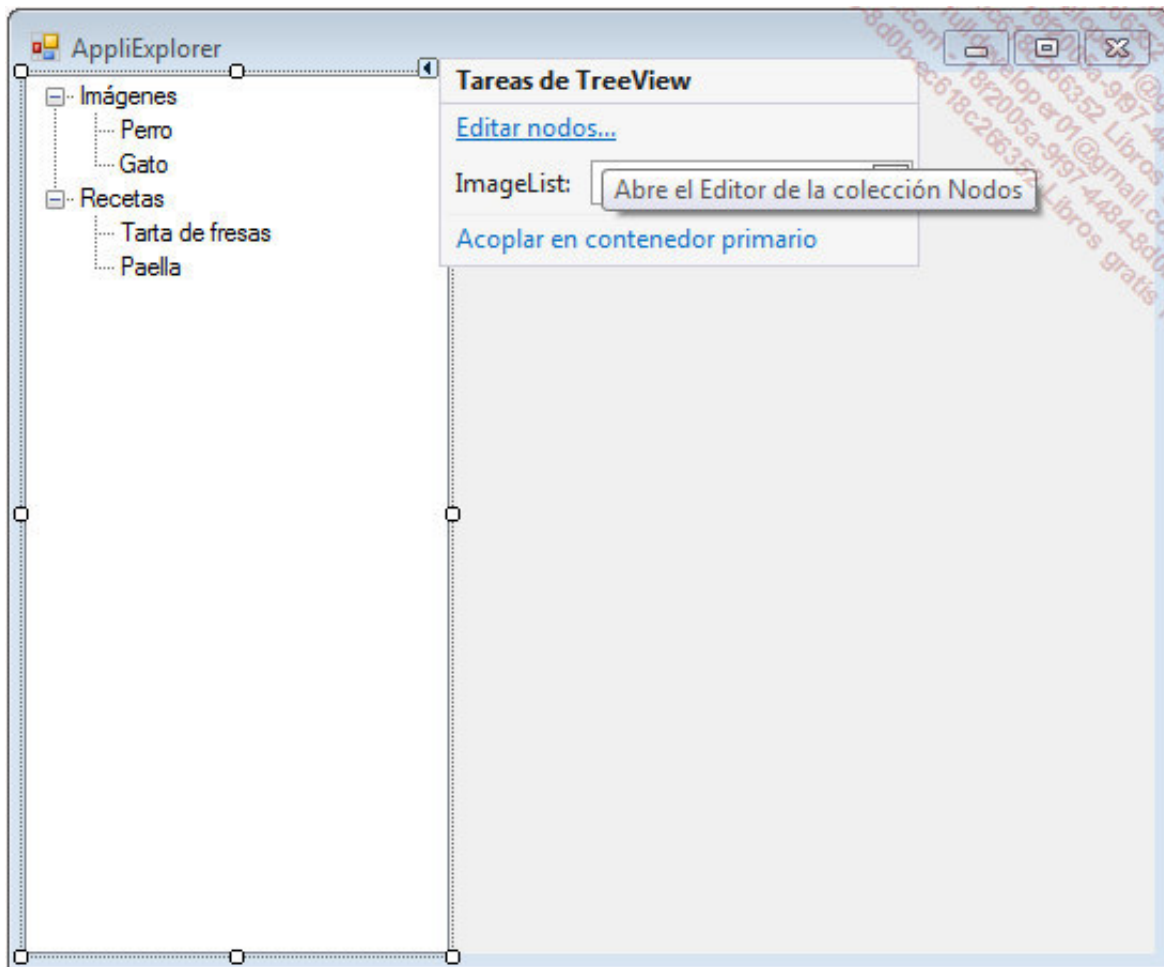
Preparemos la ventana principal:

Modifique la propiedad `IsMdiContainer` a `True` para activar la funcionalidad de la ventana madre MDI.

Añada un control `TreeView`.

Modifique la propiedad `Dock` del control `TreeView` a `Left` para que quede anclado al borde izquierdo de la ventana.

Añada los elementos al control `TreeView` ayudándose del gestor de nodos.



- Para nuestra aplicación, se utiliza la propiedad `Name` de los nodos raíz para determinar el tipo de documento (tx para archivos de texto, gr para archivos gráficos). Para los demás nodos del árbol, la aplicación guarda el nombre del archivo implicado.

Las ventanas hijas son todas igual de simples de configurar.

Para la ventana de gráficos:

Modifique la propiedad `BorderStyle` a `none`.

Añada un control `PictureBox`.

Modifique la propiedad `dock` de este control a `Fill` para que ocupe toda la superficie disponible de la ventana.

Modifique la propiedad `SizeMode` de este control `StretchImage` para que la imagen se adapte al tamaño del control (y, por lo tanto, de la ventana).

Para la ventana de texto:

Modifique la propiedad `BorderStyle` a `none`.

Añada un control `RichTextBox`.

Modifique la propiedad `dock` de este control a `Fill` para que ocupe toda la superficie disponible de la ventana.

Ahora sólo nos queda escribir algunas líneas de código para visualizar la ventana correcta durante una selección en el control `TreeView`. Veamos a continuación estas líneas de código:

```
private void treeView1_AfterSelect(object sender, TreeViewEventArgs e)
{
    if (e.Node.Parent != null)
    {
        foreach (Form f in this.MdiChildren)
        {
            f.Close();
        }
    }
    switch ( e.Node.Parent.Name)
    {
        case "gr":
            Grafico fGr;
            fGr=new Grafico();
            fGr.MdiParent=this;
            fGr.Show();
            fGr.Dock=DockStyle.Fill;
            fGr.pictureBox1.Image=Image.FromFile("../.." + e.Node.Name);
            break;
        case "tx":
            Texto fTx;
            fTx=new Texto();
            fTx.MdiParent=this;
            fTx.Show();
            fTx.Dock=DockStyle.Fill;
            fTx.richTextBox1.LoadFile("../.." + e.Node.Name);
            break;
    }
}
```

La totalidad del código se encuentra en el procedimiento de evento `TreeView1_AfterSelect` invocado automáticamente tras la selección por el usuario de un elemento en el control `TreeView`. Nuestro primer trabajo consiste en probar si acabamos de seleccionar un nodo hijo. Si es el caso, cerramos todas las ventanas hijas presentes («todas» es una palabra excesiva, ya que con este mecanismo nunca tendremos más de una ventana hija visualizada a la vez). Luego, probamos el tipo de documento requerido verificando la propiedad `Name` del nodo paterno del elemento seleccionado (`gr` o `tx`). En función del resultado, creamos una instancia de la ventana adaptada a la situación. Establecemos su vínculo de parentesco con la ventana principal (propiedad `MdiParent`). Luego se visualiza la ventana ocupando toda la superficie libre de la ventana madre `Mdi` (propiedad `Dock=DockStyle.Fill`). Última etapa: visualizamos el documento en el control adaptado `RichTextBox` o `PictureBox`. También hay que observar que, para que estos dos controles sean accesibles desde fuera de la clase en la cual han sido declarados, debe modificar su propiedad `Modifiers` con el valor `Public`. Esta propiedad determina la visibilidad de la variable utilizada para referenciar el control.

```
}  
private void txtSource_MouseMove(object sender, MouseEventArgs e)  
{  
    if (e.Button == MouseButtons.Left)  
    {  
        txtSource.DoDragDrop(txtSource.Text, DragDropEffects.Move |  
                               DragDropEffects.Copy);  
        ctrlSource = txtSource;  
    }  
}  
private void txtDestination_DragEnter(object sender, DragEventArgs e)  
{  
    if (e.Data.GetDataPresent(DataFormats.Text))  
    {  
        if ((e.KeyState & 8) == 8)  
        {  
            e.Effect = DragDropEffects.Copy;  
        }  
        else  
        {  
            e.Effect = DragDropEffects.Move;  
        }  
    }  
}  
private void txtDestination_DragDrop(object sender, DragEventArgs e)  
{  
    txtDestination.Text = (String)e.Data.GetData(DataFormats.Text);  
    if ((e.KeyState & 8) != 8)  
    {  
        ctrlSource.Clear();  
    }  
}  
}
```

Subir

Cuadros de diálogo

Los cuadros de diálogo son ventanas que tienen una función especial en una aplicación. Se suelen utilizar para pedir al usuario la inserción de datos. Para asegurarse de que estos datos se introducen correctamente antes de continuar con la ejecución de la aplicación, los cuadros de diálogo se muestran a menudo en modo modal, es decir, que el resto de la aplicación está bloqueado mientras el cuadro de diálogo esté abierto. Ocurre a menudo en una aplicación que se necesiten los mismos datos: un nombre de archivo que hay que abrir, una fuente de caracteres que hay que elegir, etc. Para no tener que volver a crear cada vez un nuevo cuadro de diálogo, disponemos de una serie de cuadros de diálogo predefinidos.

1. El cuadro de mensaje

Los cuadros de mensaje permiten pasar al usuario información y le dan la posibilidad de contestar mediante botones de comando del cuadro de mensaje.

El cuadro de mensaje se muestra invocando al método `show` de la clase `MessageBox`. Este método toma muchos parámetros para configurar el cuadro de diálogo. El primero de ellos corresponde al mensaje que se va a mostrar. El siguiente especifica el título del cuadro del mensaje. Los siguientes deben ser elegidos entre las constantes predefinidas para indicar respectivamente:

- Los botones mostrados en el cuadro de mensaje.
- El icono visualizado en el cuadro de mensaje.
- El botón seleccionado por defecto al visualizar el cuadro de mensaje.

Las constantes disponibles son:

- para determinar los botones:

Constante	Significado
<code>MessageBoxButtons.OK</code>	Botón OK únicamente
<code>MessageBoxButtons.OKCancel</code>	Botones OK y Cancelar
<code>MessageBoxButtons.AbortRetryIgnore</code>	Botones Salir, Reintentar y Omitir
<code>MessageBoxButtons.YesNoCancel</code>	Botones Sí, No y Cancelar
<code>MessageBoxButtons.YesNo</code>	Botones Sí y No
<code>MessageBoxButtons.RetryCancel</code>	Botones Reintentar y Cancelar

para determinar los iconos:

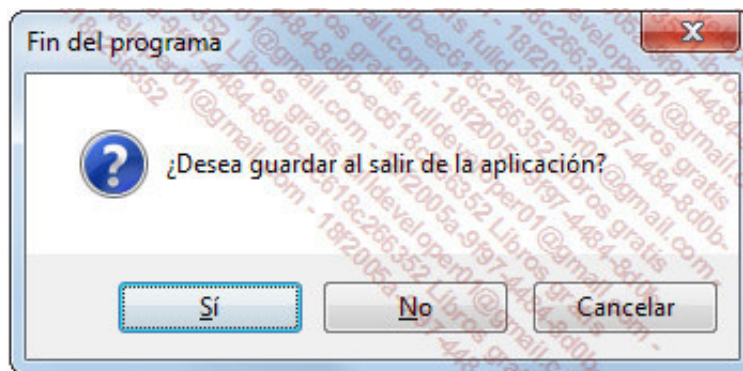
Constante	Significado
<code>MessageBoxIcon.Information</code>	
<code>MessageBoxIcon.Exclamation</code>	
<code>MessageBoxIcon.Error</code>	

	
MessageBoxIcon.Question	

para determinar el botón por defecto:

Constante	Significado
MessageBoxDefaultButton.Button1	Primer botón
MessageBoxDefaultButton.Button2	Segundo botón
MessageBoxDefaultButton.Button3	Tercer botón

Para obtener el siguiente cuadro de mensaje



utilizaremos el siguiente código:

```
DialogResult respuesta;
respuesta=MessageBox.Show("¿Desea guardar al salir de la aplicación?"
, "Fin del programa",
    MessageBoxButtons.YesNoCancel,
    MessageBoxIcon.Question,
    MessageBoxDefaultButton.Button1);
```

Como planteamos una pregunta al usuario, debemos recuperar su respuesta para decidir el comportamiento que hay que adoptar en la aplicación. Para ello, el método `Show` devuelve un valor que indica el botón utilizado para cerrar el cuadro de mensaje. Para esto, también hay definidas una serie de constantes que identifican cada caso.

Valor devuelto	Botón utilizado
DialogResult.Ok	Botón Ok
DialogResult.Cancel	Botón Cancelar
DialogResult.Abort	Botón Salir
DialogResult.Retry	Botón Reintentar
DialogResult.Ignore	Botón Omitir

A

DialogResult.Yes	Botón Sí
DialogResult.No	Botón No

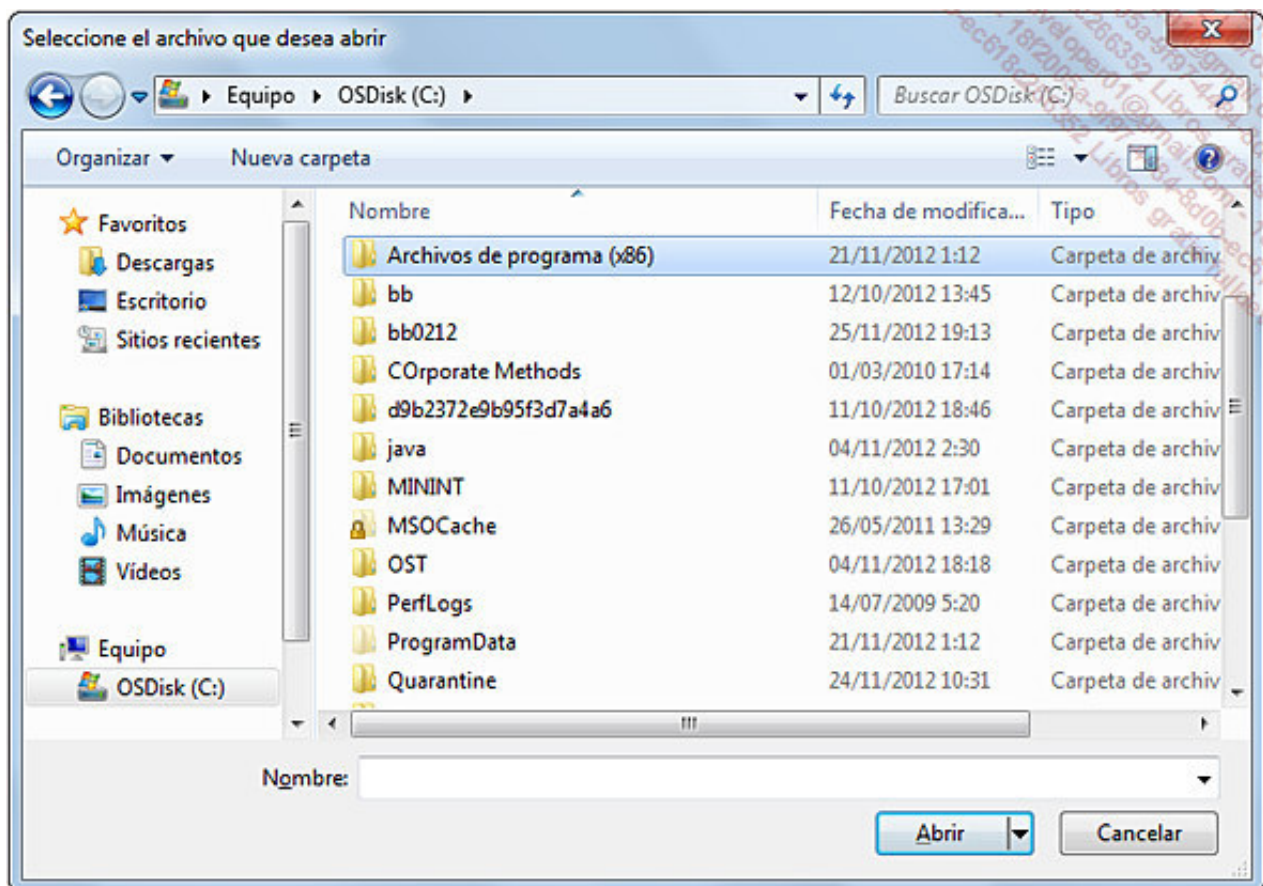
continuación podemos comprobar la respuesta:

```
switch (respuesta)
{
    case DialogResult.Yes:
        ...
        break;
    case DialogResult.No:
        ...
        break;
    case DialogResult.Cancel:
        ...
        break;
}
```

2. Los cuadros de diálogo de Windows

Ya hay muchos cuadros de diálogo definidos por el propio sistema. Para poder utilizarlos en nuestras aplicaciones disponemos de una serie de clases. Veamos cómo configurarlos y utilizarlos en una aplicación.

a. Diálogo de apertura de archivo



Este cuadro de diálogo nos ofrece la posibilidad de seleccionar uno o más nombres de archivos con la posibilidad añadida de desplazarse por el árbol de la máquina. Se utiliza la clase `OpenFileDialog`.

Por lo tanto, debemos crear una instancia en nuestra aplicación.

```
OpenFileDialog dlgAbrir;  
    dlgAbrir = new OpenFileDialog();
```

También conviene configurar nuestro cuadro de diálogo. La propiedad `InitialDirectory` indica el directorio en el que se encuentra el cuadro de diálogo en el momento de su apertura. Es posible mostrar sólo algunos de los archivos en los directorios examinados. Por lo tanto, hay que configurar mediante la propiedad `Filter` las correspondencias entre la descripción del contenido y la extensión asociada. La propiedad `Filter` almacena información en forma de cadena de caracteres. La descripción y la extensión están separadas en la cadena por el carácter | ([AltGr] 6). Si hay varias extensiones establecidas para una misma extensión, deben venir separadas por un punto y coma en la cadena.

También podría indicar si se debería añadir alguna extensión a los nombres de archivo introducidos manualmente en caso de que éstos no la contuviesen.

La propiedad `DefaultExt` contiene la extensión que hay que añadir y `AddExtension` indica si se añadió esta extensión automáticamente. Dado que se le permite al usuario introducir manualmente la ruta y el nombre del archivo que hay que abrir, puede encargar al cuadro de diálogo que verifique que la ruta de acceso y el nombre son correctos. Las propiedades `CheckFileExists` y `CheckPathExists` gestionan dichas verificaciones. Puede autorizar igualmente las selecciones múltiples mediante la propiedad `Multiselect`.

Finalmente, para mostrar el cuadro de diálogo se invoca al método `ShowDialog`:

```
dlgAbrir.Title = "Selección del archivo que hay que abrir";  
    dlgAbrir.Filter = "todos|*.*|Imágenes|*.bmp;*.gif;*.jpg|texto|*.txt";  
    dlgAbrir.DefaultExt = "pepe";  
    dlgAbrir.AddExtension = true;  
    dlgAbrir.CheckFileExists = false;  
    dlgAbrir.Multiselect = true;  
    dlgAbrir.ShowDialog();
```

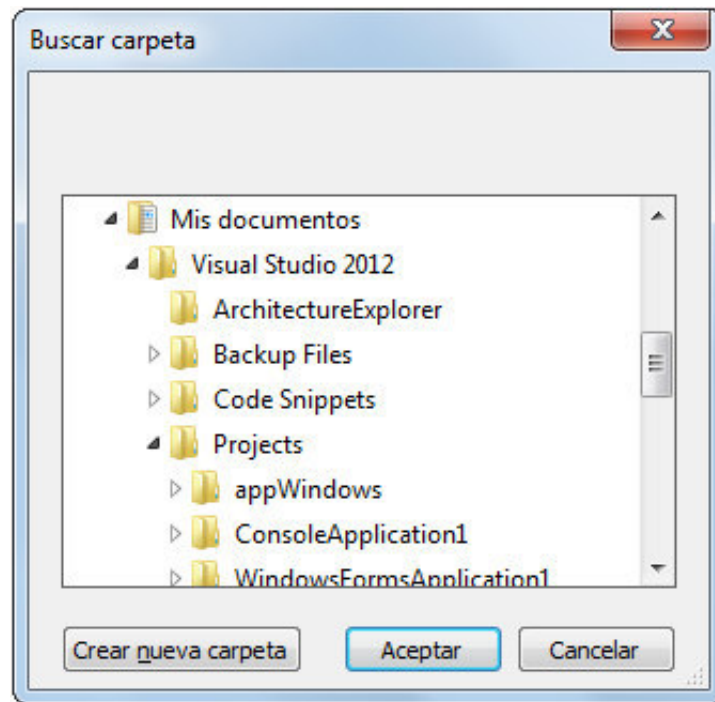
Los nombres del archivo o de los archivos seleccionados están disponibles en la propiedad `FileNames` para una selección única o en la propiedad `FileNames` para las selecciones múltiples. Esta propiedad `FileNames` es una matriz de cadenas de caracteres con el nombre completo de uno de los archivos seleccionados en cada posición de dicha matriz.

```
foreach (String nombreArchivo in dlgAbrir.FileNames)  
{  
    Console.WriteLine(nombreArchivo);  
}
```

b. Diálogo de guardar archivo

El cuadro de diálogo de guardar archivo es similar al anterior, aparte de la propiedad `Multiselect`, que desaparece, y de las propiedades `CreatePrompt` y `OverwritePrompt`, que permiten mostrar un mensaje de alerta si el nombre del archivo introducido no existe o, al contrario, si ya existe.

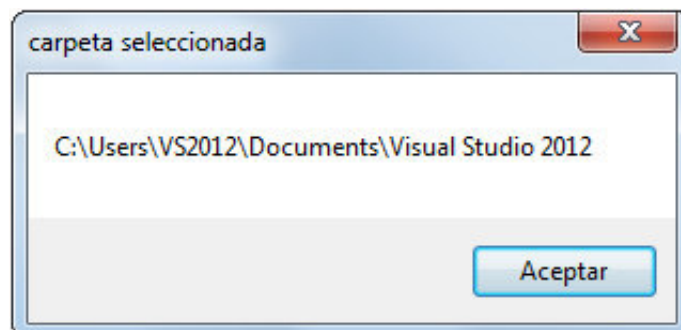
c. Diálogo de selección de directorio



Se utiliza este cuadro de diálogo para la selección o creación de un directorio. Se crea a partir de la clase `FolderBrowserDialog`. Esta última contiene muy pocas propiedades. La más utilizada ciertamente es la propiedad `SelectedPath`, que permite la recuperación de la ruta de acceso al directorio seleccionado. El directorio raíz del cuadro de diálogo viene marcado por la propiedad `RootFolder`. Esta propiedad recibe uno de los valores de la enumeración `Environment.SpecialFolder`, que representa los principales directorios característicos del sistema como, por ejemplo, el directorio **Mis documentos**. Si se utiliza esta propiedad, sólo se podrá hacer la selección en un subdirectorio del directorio raíz. Se puede autorizar la inserción de un botón que permita la creación de un nuevo directorio modificando la propiedad `ShowNewFolderButton`. La visualización del cuadro de diálogo se hace de forma clásica con el método `ShowDialog`:

```
FolderBrowserDialog dlgSelecDir;  
dlgSelecDir = new FolderBrowserDialog();  
dlgSelecDir.RootFolder = Environment.SpecialFolder.MyDocuments;  
dlgSelecDir.ShowDialog();  
MessageBox.Show(dlgSelecDir.SelectedPath, "directorio seleccionado");
```

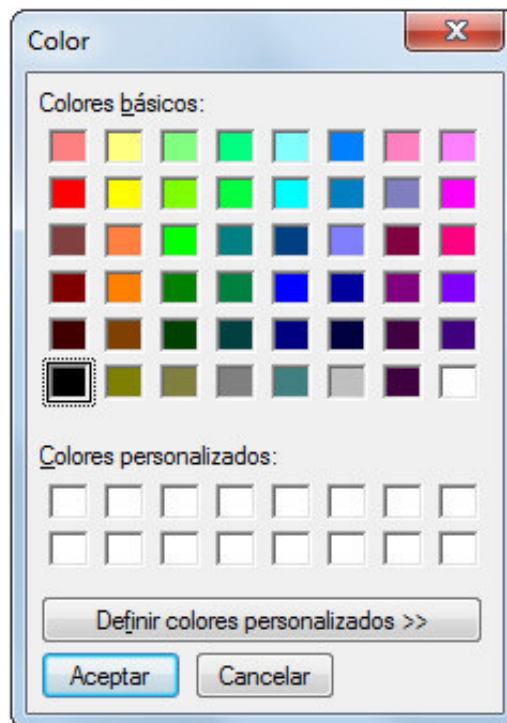
También hay que destacar que la ruta de acceso devuelta por este cuadro de diálogo es una ruta absoluta, como muestra el siguiente ejemplo:



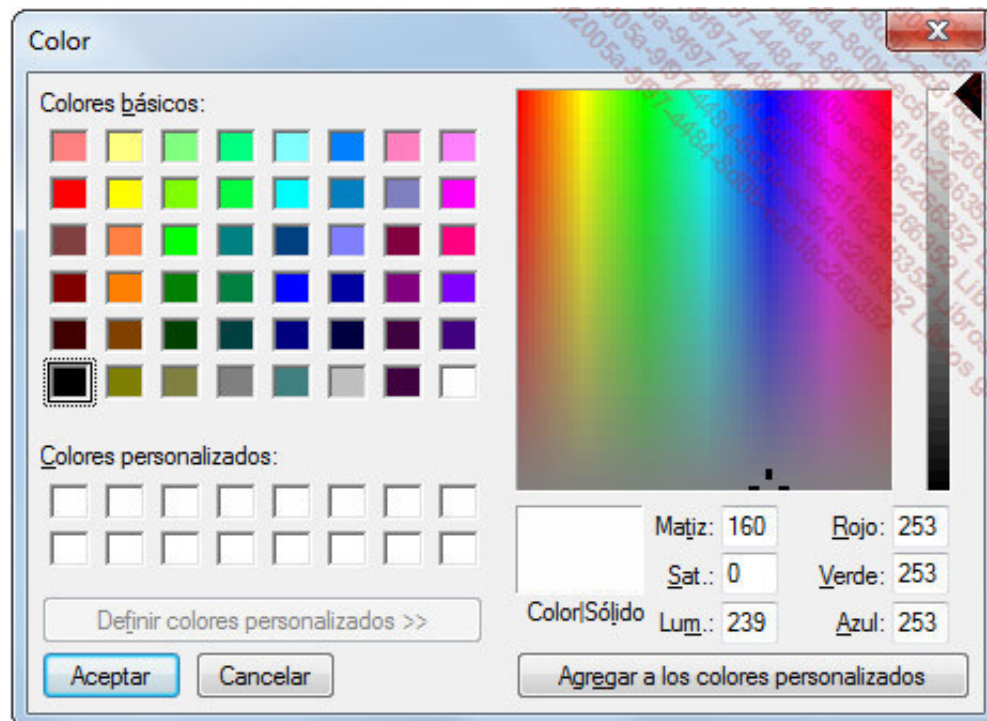
d. Diálogo de selección de color

El cuadro de diálogo de selección de color creado a partir de la clase `ColorDialog` ofrece dos configuraciones diferentes.

Una versión «simple», donde sólo se muestran los colores básicos:



Una versión completa en la que el usuario podrá crear colores personalizados:



La propiedad `Color` permite inicializar el cuadro de diálogo antes de su visualización y luego recuperar el color seleccionado por el usuario. Usted puede prohibir el uso de los colores

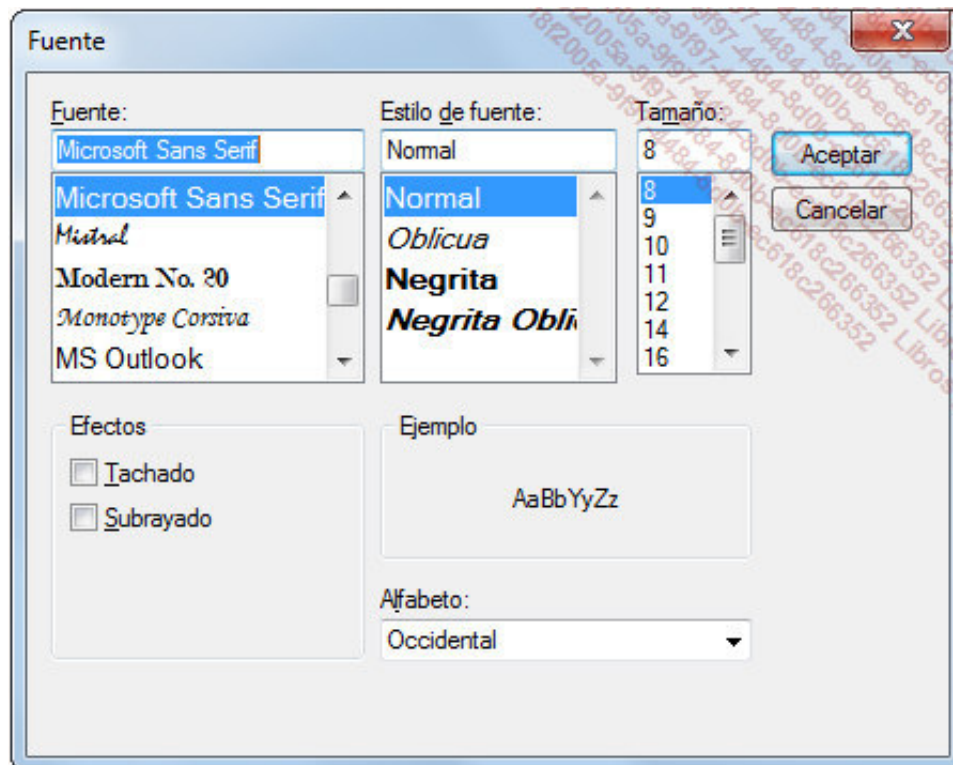
personalizados o, al contrario, mostrar el cuadro de diálogo completo desde el inicio. Para prohibir la visualización de los colores personalizados, se modifica la propiedad `AllowFullOpen`. Para forzar la visualización completa, se utiliza la propiedad `FullOpen`.

La visualización del cuadro de diálogo se lleva a cabo siempre con el método `ShowDialog`. Para conservar una calidad de visualización correcta, puede autorizar únicamente el uso de colores puros (los colores obtenidos por yuxtaposición de diferentes píxeles se eliminarán de las posibles elecciones). Se debe utilizar esta opción si se dispone de una tarjeta gráfica de 256 colores.

Este ejemplo modifica el color de fondo de nuestra hoja.

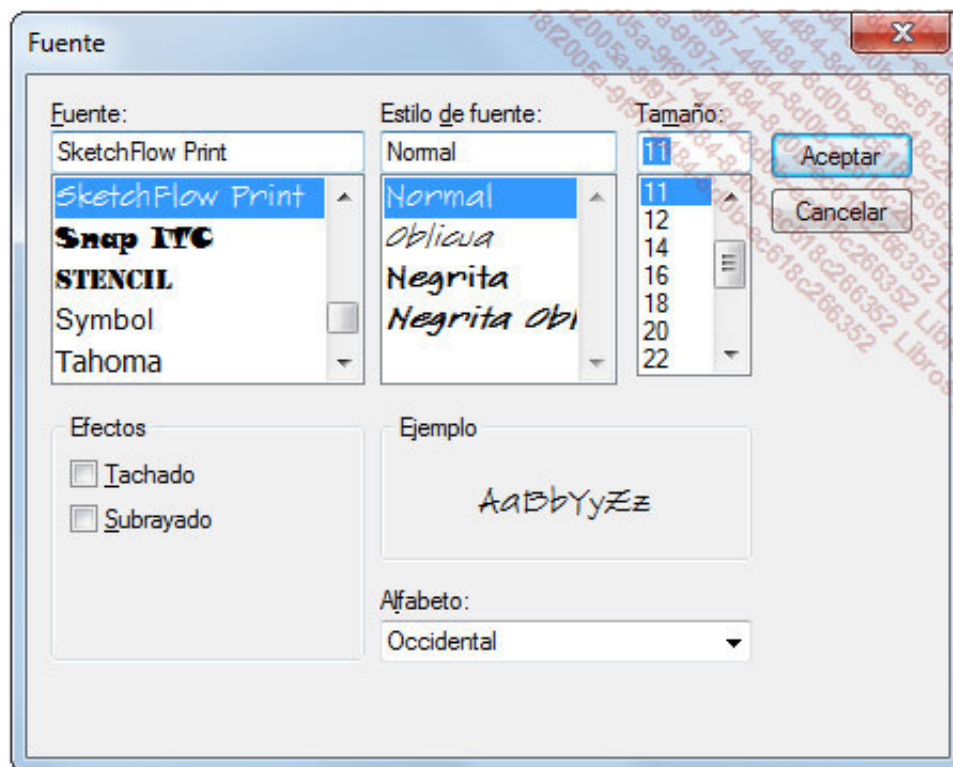
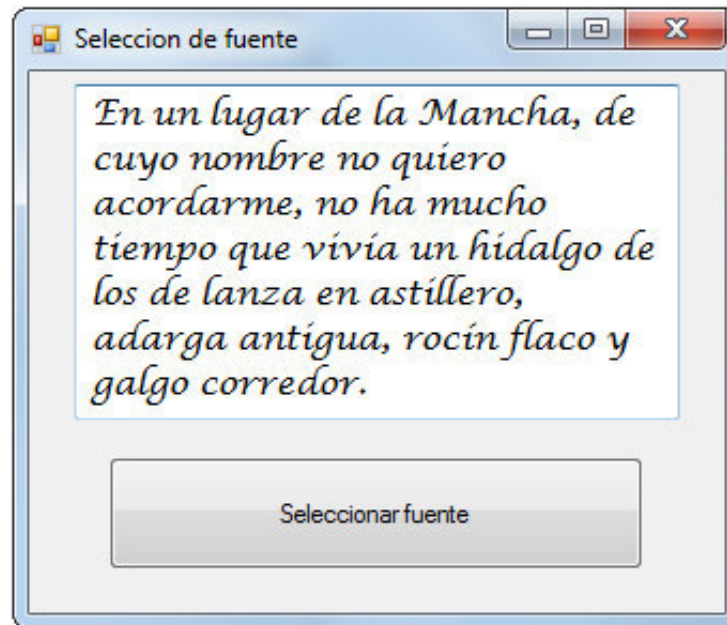
```
ColorDialog dlgColor;  
dlgColor = new ColorDialog();  
dlgColor.FullOpen = true;  
dlgColor.SolidColorOnly = true;  
dlgColor.Color = this.BackColor;  
dlgColor.ShowDialog();  
this.BackColor = dlgColor.Color;
```

e. Diálogo de selección de fuente



La clase básica utilizada para la selección de una fuente es `FontDialog`. La propiedad `Font` permite definir la fuente de caracteres utilizada para inicializar el cuadro de diálogo o, después de su cierre, recuperar la fuente seleccionada. También puede visualizar un cuadro de diálogo simplificado sin las opciones de color o efectos. Para ello, las propiedades `ShowColor` y `ShowEffects` controlan la visualización de estos parámetros en el cuadro de diálogo. A fin de garantizar que los parámetros seleccionados corresponden efectivamente a una fuente existente en la máquina, puede hacer uso de la propiedad `FontMustExist`. Esta propiedad obligará al cuadro de diálogo a comprobar la existencia de una fuente correspondiente en el sistema antes de cerrarse. Algunas fuentes cuentan con varios juegos de caracteres. Puede autorizar a los usuarios a elegir uno de estos juegos de caracteres modificando la propiedad `AllowScriptChange`. El tamaño de la fuente seleccionada también puede limitarse mediante las propiedades `MaxSize` y `MinSize`.

Para que se dé cuenta del efecto que produce la fuente seleccionada, existe una previsualización para algunos caracteres. Si esta previsualización no fuera suficiente, podría mostrar un botón **Aplicar** en su cuadro de diálogo mediante de la propiedad `ShowApply`. Este botón lanza un evento `Apply` en el cuadro de diálogo. En la gestión de este evento, puede utilizar la propiedad `Font` del cuadro de diálogo para muestra el efecto de la fuente actualmente seleccionada en su texto. Se debe declarar la variable que hace referencia al cuadro de diálogo con la palabra clave `WithEvents`, es decir, fuera de un procedimiento. Veamos a continuación un pequeño ejemplo que muestra el uso de estas propiedades:



```
dlgFont = new FontDialog();  
dlgFont.ShowApply = true;  
dlgFont.ShowColor = true;
```



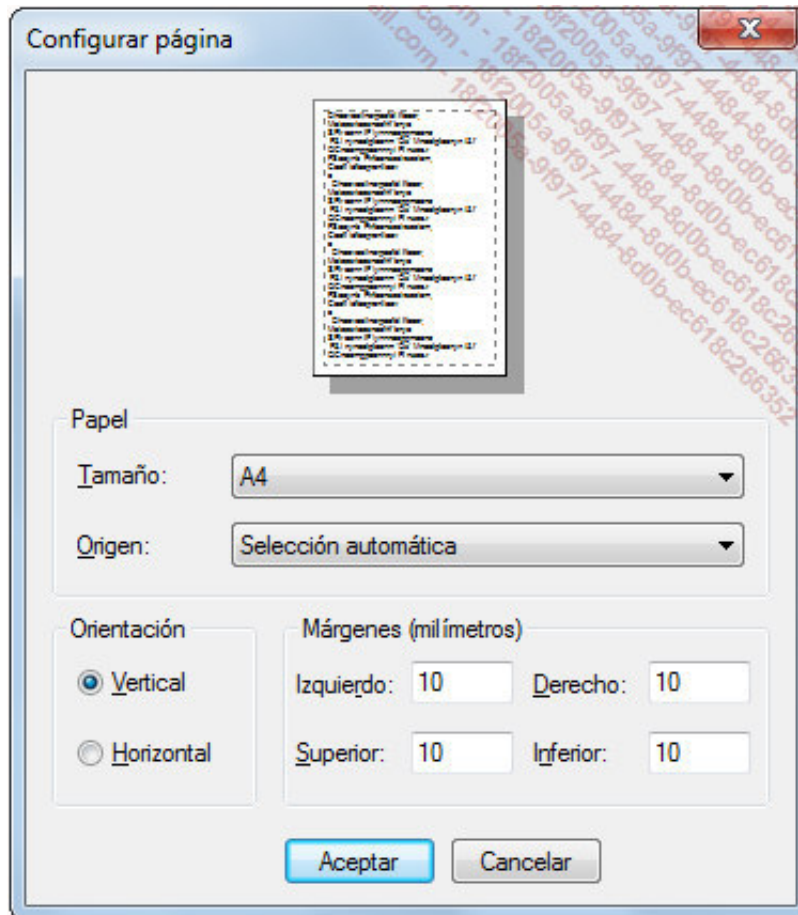
```

dlgFont.ShowEffects = true;
dlgFont.MaxSize = 20;
dlgFont.MinSize = 12;
dlgFont.FontMustExist = true;
dlgFont.AllowScriptChange = true;
dlgFont.Apply += dlgFont_Apply;
dlgFont.ShowDialog();
txtMuestra.Font = dlgFont.Font;

```

f. Diálogo de configuración de página

Por medio de este cuadro de diálogo, podrá configurar los parámetros de su documento (márgenes, orientación...).



Se crea este cuadro de diálogo a partir de la clase `PageSetupDialog`. Para poder trabajar, esta clase necesita dos clases auxiliares: la clase `PageSettings` sirve para almacenar la configuración de la página, la clase `PrinterSettings` almacena la configuración de la impresora seleccionada. Hay que crear una instancia de estas dos clases y asociarlas a las propiedades `PageSettings` y `PrinterSettings` del cuadro de diálogo. Se verá obligado a importar el espacio de nombres `System.Drawing.Printing` para poder utilizar esas dos clases.

Las siguientes propiedades pueden prohibir el uso de las diferentes secciones del cuadro de diálogo:

- `AllowMargins` para la modificación de los márgenes.
- `AllowOrientation` para la modificación de la orientación.
- `AllowPaper` para la selección del papel.
- `AllowPrinter` para la selección de impresora.

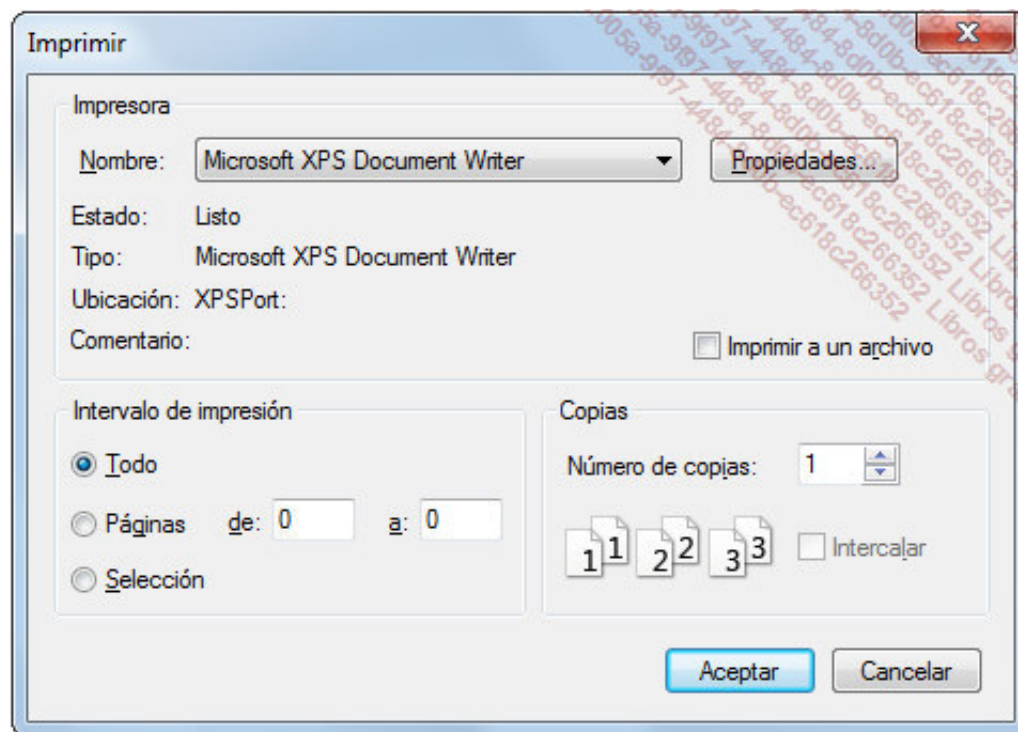
Luego se podrá recuperar la selección del usuario mediante las propiedades `PageSettings` y `PrinterSettings` del cuadro de diálogo.

A continuación, un ejemplo de su uso:

```
PageSetupDialog dlgPgSetup=null;
PageSettings configPg;
PrinterSettings configPrt;
dlgPgSetup = new PageSetupDialog();
configPg = new PageSettings();
configPrt = new PrinterSettings();
dlgPgSetup.PageSettings = configPg;
dlgPgSetup.AllowPrinter = true;
dlgPgSetup.PrinterSettings = configPrt;
dlgPgSetup.ShowDialog();
MessageBox.Show("Usted ha seleccionado imprimir con la impresora " +
dlgPgSetup.PrinterSettings.PrinterName + " en papel " +
dlgPgSetup.PageSettings.PaperSize.PaperName + " con el formato " +
((dlgPgSetup.PageSettings.Landscape ? "Horizontal" : "Vertical")));
```

g. Diálogo de configuración de la impresión

Con este cuadro de diálogo, puede configurar los parámetros de impresión de su documento. Se creará a partir de la clase `PrintDialog`.



Como en el caso del cuadro de configuración de página, el cuadro de diálogo de configuración de impresión requiere una instancia de la clase `PrinterSettings` para almacenar la información de configuración de la impresora.

Las siguientes propiedades pueden prohibir el uso de las diferentes secciones:

- `AllowSelection` autoriza la utilización del botón **Selección**. Este botón suele ser accesible únicamente si hay algo seleccionado en el documento que quiere imprimir.
- `AllowSomePages` autoriza la selección de una página de principio y otra de fin para la

impresión del documento. Este botón debe estar disponible si el documento contiene varias páginas.

- `AllowPrintToFile` indica si la casilla de selección **Imprimir a un archivo** está disponible. Esta funcionalidad permite recuperar un archivo con el formato PostScript para exportarlo a otra aplicación.

Tras el cierre del cuadro de diálogo, el resultado de todas estas opciones queda reflejado en la propiedad `PrinterSettings`.

A continuación, un nuevo ejemplo de este cuadro de diálogo.

```
PrinterSettings configPrt;
PrintDialog dlgprinter;
configPrt = new PrinterSettings();
dlgprinter = new PrintDialog();
dlgprinter.PrinterSettings = configPrt;
dlgprinter.AllowSomePages = true;
dlgprinter.AllowSelection = true;
dlgprinter.ShowDialog();
switch (dlgprinter.PrinterSettings.PrintRange)
{
    case PrintRange.AllPages:
        MessageBox.Show("Usted ha pedido la impresión de todo el
                        documento");
        break;
    case PrintRange.SomePages:
        MessageBox.Show("Usted ha pedido la impresión de la página " +
                        dlgprinter.PrinterSettings.FromPage + " a la página " +
                        dlgprinter.PrinterSettings.ToPage);
        break;
    case PrintRange.Selection:
        MessageBox.Show("Usted ha pedido la impresión de la selección");
        break;
}
```

3. Cuadro de diálogo personalizado

Después de este breve vistazo a los cuadros de diálogo predefinidos, vamos a ver cómo crear nuestros propios cuadros de diálogo. La base de creación de un cuadro de diálogo es una ventana clásica cuyas siguientes propiedades se modifican:

- El estilo del borde, para tener una ventana que no se pueda redimensionar.
- La propiedad `ShowInTaskBar`, que se establece en **False** para que la ventana no aparezca en la barra de tareas.
- También hay que prever un botón de validación y otro de cancelación para el cierre del cuadro de diálogo.

La visualización del cuadro de diálogo se hará invocando el método `ShowDialog` en lugar del método `Show`, ya que el método `ShowDialog` visualiza la ventana en modo modal (nuestro cuadro de diálogo será la única parte en uso de nuestra aplicación mientras permanezca abierto).

Tras el cierre del cuadro de diálogo, debemos poder determinar qué botón provocó el cierre del cuadro de diálogo. En realidad es el método `ShowDialog` el que nos proporciona la solución. Nos devuelve uno de los valores de la enumeración `System.Windows.Forms.DialogResult`. Por supuesto, el valor devuelto no se escoge al azar. Por lo tanto, en el momento de diseñar el cuadro de diálogo tiene la obligación de facilitar el valor que hay que devolver para cada uno de los botones que provocan el

cierre del cuadro de diálogo. Usted puede hacer esto modificando la propiedad `DialogResult` del cuadro de diálogo en el evento `Click` de cada uno de los botones, o modificando la propiedad `DialogResult` de los botones implicados en el cierre del cuadro de diálogo. Debe tener en cuenta que en este caso no hace falta gestionar el evento `Click` del botón para provocar el cierre del cuadro de diálogo. Si se utilizan las dos soluciones de forma simultánea, la propiedad `DialogResult` del cuadro de diálogo será prioritaria para determinar el valor devuelto por el método `ShowDialog`.

Ahora que sabemos cómo configurar y visualizar un cuadro de diálogo, nos queda lo más difícil: crear la interfaz visual del cuadro de diálogo.

Utilización de controles

Los controles nos van a permitir crear la interfaz entre la aplicación y su usuario. Gracias a ellos el usuario podrá actuar sobre el funcionamiento de la aplicación insertando texto, seleccionando opciones, iniciando la ejecución de una parte específica de nuestra aplicación, etc.

Los controles estarán disponibles en Visual C# mediante una serie de clases para las que se deberán definir instancias durante la ejecución de la aplicación.

Estas clases proceden de una jerarquía que comienza con la clase base `Control`. Esta clase asegura las funciones elementales de los controles (posiciones, dimensiones...). Y luego, una clase derivada añade funcionalidades adicionales y así sucesivamente hasta la clase final de la jerarquía.

1. Añadir controles

Se pueden cambiar los controles de una ventana de dos maneras diferentes. La más simple, y también la más rápida, consiste en utilizar el cuadro de herramientas. Aquí también hay tres posibilidades para añadir controles:

- Haga un doble clic sobre el control en el cuadro de herramientas. Este método permite ubicar en el centro de la ventana un ejemplar con un tamaño por defecto.
- Arrastre y suelte el control desde el cuadro de herramientas hasta la ventana. Cuando pase por encima de la hoja, el cursor del ratón le indicará, mediante un pequeño signo de más (+), que va a añadir algo en la hoja. La posición en la cual suelte el ratón corresponderá a la posición de la esquina superior izquierda de su control. Se dimensionará con los valores por defecto.
- Seleccione el control en el cuadro de herramientas y luego haga clic en la ventana, en el lugar exacto donde quiere colocar la esquina superior izquierda de su control. Luego, sin soltar el botón del ratón, maximice el rectángulo de su control hasta el tamaño deseado.

Si desea colocar varios ejemplares del mismo control en su ventana, es posible bloquear la selección en el cuadro de herramientas utilizando la tecla `[Ctrl]` cuando selecciona el control en el cuadro de herramientas. Entonces, podrá colocar varios ejemplares del mismo control sin tener que volver a seleccionarlo en el cuadro de herramientas manteniendo la tecla `[Ctrl]` pulsada.

Algunos controles no disponen de una interfaz visible durante el diseño de la ventana. Para evitar sobrecargar la superficie de la ventana, se ubican en una zona situada debajo de la zona de diseño gráfico. Es el caso, por ejemplo, de los controles `ImageList` y `Timer`, que veremos más adelante en este capítulo. Es posible añadir controles al cuadro de herramientas. Estos controles pueden ser del tipo `.NET` o `ActiveX`. El uso de controles `ActiveX` implicará inconvenientes para su aplicación. El código de su aplicación será menos eficaz (será necesario llevar a cabo algunas operaciones adicionales para acceder al control `ActiveX`).

El despliegue de su aplicación requerirá modificaciones en la base de registro de las máquinas para guardar los controles `ActiveX`.

Visual Studio nombra los controles agregados automáticamente a medida que se añaden. En cambio, los nombres utilizados por defecto no son muy explícitos.

El siguiente código no le va a parecer tan claro.

```
Button1.Enabled = false;  
TextBox1.Clear();  
CheckBox1.Checked = true;  
RadioButton1.Checked = false;
```

```
RadioButton2.Checked = true;
```

Por lo tanto es primordial para la legibilidad del código volver a nombrar los controles de preferencia en el momento de la creación o, como muy tarde, antes de utilizarlos en el código. Sólo hace falta cambiar la propiedad `name` de cada uno de ellos mediante la ventana de propiedades. No hay una regla absoluta que respetar para sus nombres. Una solución a menudo utilizada consiste en asociar un prefijo representativo del tipo de control a un nombre explícito para la aplicación. Los prefijos no están normalizados.

Prefijo	Control
cbo	ComboBox
lst	Listbox
chk	CheckBox
opt	RadioButton
cmd	Button
txt	TextBox
lbl	Label

Respetando estas convenciones y con un poco de sentido común, el código se hace mucho más claro:

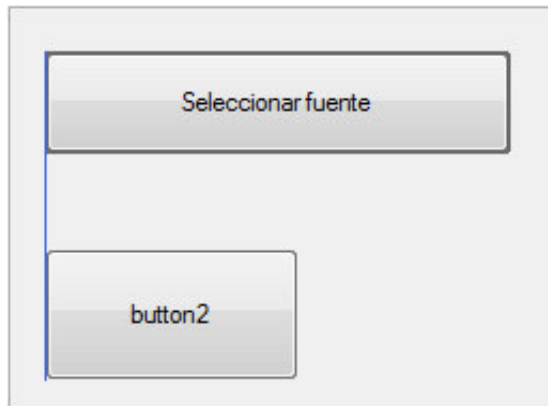
```
cmdValidacion.Enabled = false;  
txtNombre.Clear();  
chkCursiva.Checked = true;  
optAzul.Checked = false;  
optVerde.Checked = true;
```

2. Posición y dimensión de los controles

Después de haber colocado los controles en la ventana, es posible volver a ubicarlos o redimensionarlos. Cuando desplaza el ratón sobre un control, el cursor cambia de apariencia para indicar la posibilidad de mover el control.



Basta con hacer clic en el control y luego desplazar el control. El control sigue al cursor del ratón representando así la futura posición de su control. Se visualizan líneas guía durante el desplazamiento del control para facilitar su alineamiento con los otros controles ya ubicados en la ventana. Las líneas azules representan los alineamientos posibles sobre los bordes de otros controles. Las líneas rosas representan los alineamientos posibles sobre el nombre de los controles. Efectivamente, el control se desplazará en el momento que suelte el botón de su ratón.



También es posible usar las flechas de dirección del teclado, lo que aporta más precisión durante el desplazamiento.

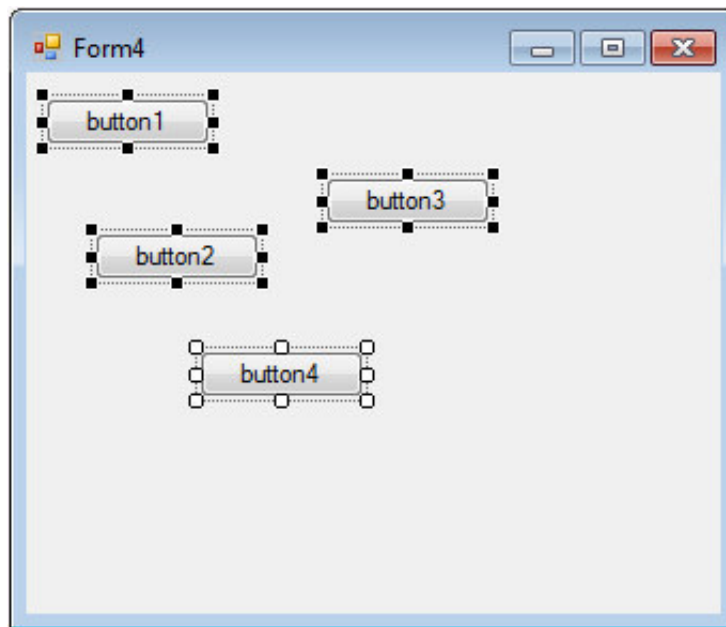
Puede modificar la posición de un control mediante su propiedad `Location` en la ventana de propiedades. De hecho, esta propiedad se modifica cuando desplaza el control con ratón o el teclado.

Finalmente, la última posibilidad consiste en modificar las propiedades `Left` y `Top` del control mediante el código. El siguiente extracto de código permite desplazar el botón de comando a una posición aleatoria cada vez que usted hace clic en él.

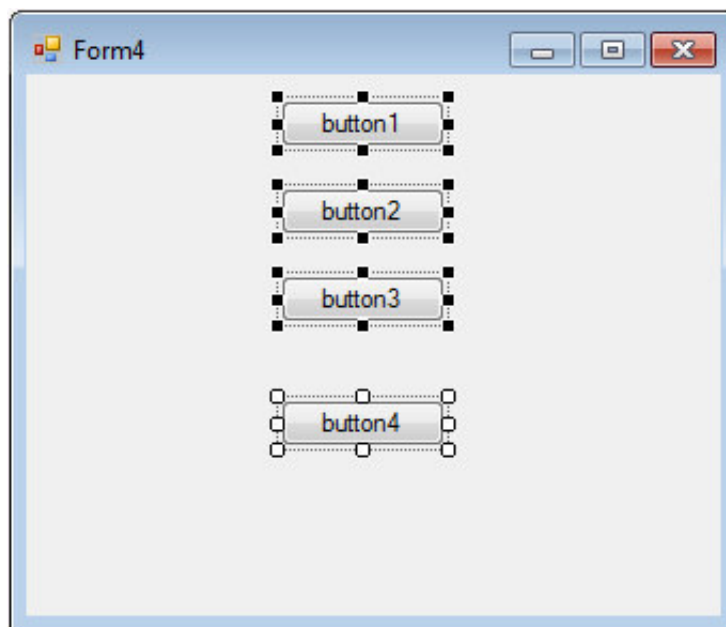
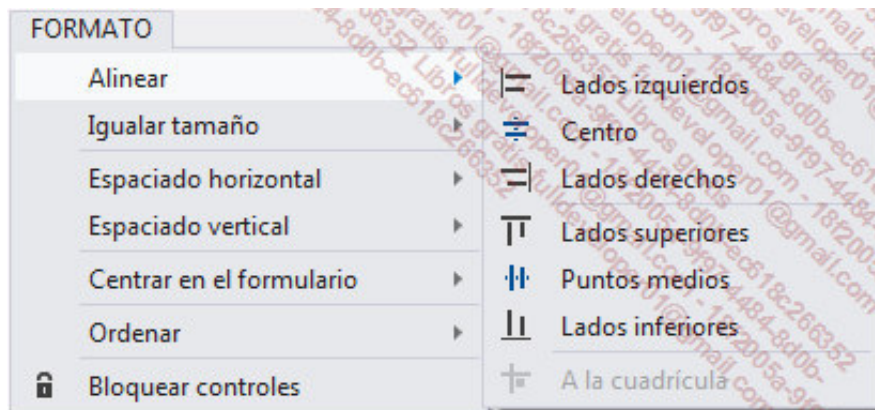
```
private void cmdTest_Click(object sender, EventArgs e)
{
    cmdTest.Left = new Random().Next(0, (this.ClientSize.Width -
                                         cmdTest.Size.Width));
    cmdTest.Top = new Random().Next(0, (this.ClientSize.Height -
                                         cmdTest.Size.Height));
}
```

Algunas funcionalidades más evolucionadas permiten la ubicación de los controles los unos respecto a los otros. Para poder utilizarlas es necesario seleccionar previamente un grupo de controles. Hay dos soluciones posibles en este caso:

- Dibujar un rectángulo de selección con el ratón alrededor de los controles.
- Hacer clic en un control tras otro, manteniendo la tecla `[Ctrl]` pulsada. El primer control seleccionado aparece con un recuadro blanco.

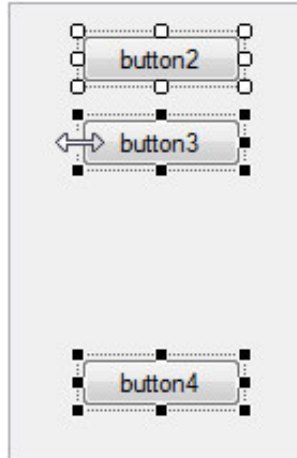


Las opciones del menú **Formato** están activadas y le proporcionan muchas opciones para ubicar los controles. El control que aparece en la selección rodeado de un recuadro blanco se considera la referencia para el alineamiento.



Hay muchas otras opciones a su disposición para organizar la ubicación de los controles en su hoja.

También el redimensionamiento de los controles es muy simple de aplicar, ya que sólo es necesario seleccionar el control o los controles que va a redimensionar y ubicar el cursor del ratón en uno de los recuadros de selección para que aparezca una flecha indicándole en qué dimensión puede redimensionar el control. Entonces hay que hacer clic en el cuadro correspondiente y desplazar el ratón hasta que el control haya alcanzado el tamaño deseado.



También puede hacer uso de las flechas del teclado asociadas a la tecla [Shift] para dimensionar los controles.

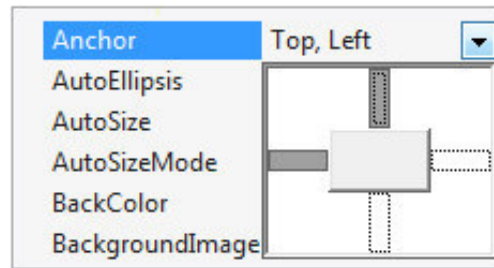
El redimensionamiento mediante código utiliza el método `SetBounds`, que permite fijar tanto la posición como el tamaño del control. El siguiente código reduce el tamaño del control cada vez que se hace clic en él.

```
private void cmdReducir_Click(object sender, EventArgs e)
{
    cmdReducir.SetBounds(cmdReducir.Left, cmdReducir.Top,
                        cmdReducir.Width - 5, cmdReducir.Height - 5);
}
```

Después de muchos esfuerzos para ubicar y dimensionar los controles, sería una pena que un error de manejo lo fastidiase todo. Para evitar esto, es posible bloquear los controles de la hoja, por el menú **Formato - Bloquear controles**. Este comando bloquea el desplazamiento y el redimensionamiento de todos los controles presentes en la hoja, así como el redimensionamiento de la propia hoja. Luego se pueden desbloquear los controles con la misma opción del menú. Igualmente podrá desbloquear los controles individualmente mediante la propiedad `locked`.

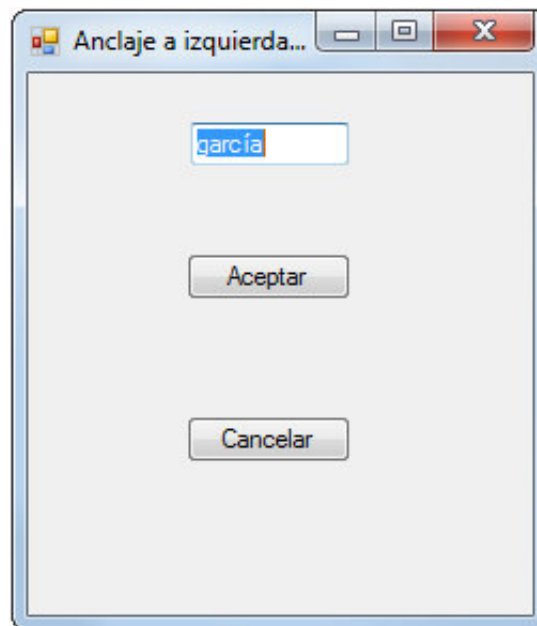
Si diseña una aplicación en la cual el usuario puede redimensionar la ventana en tiempo de ejecución, los controles deben seguir las modificaciones de tamaño de la ventana. Para autorizar el redimensionamiento automático de un control, puede utilizar la propiedad `Anchor` del control. Por medio de esta propiedad, indica que la distancia entre los bordes del control y las posiciones de anclaje se respetarán durante el redimensionamiento de la ventana.

En el momento de la creación, se ancla los controles a los bordes superior e izquierdo de la hoja. La modificación de esta propiedad se efectúa mediante un asistente disponible en la ventana de propiedades.

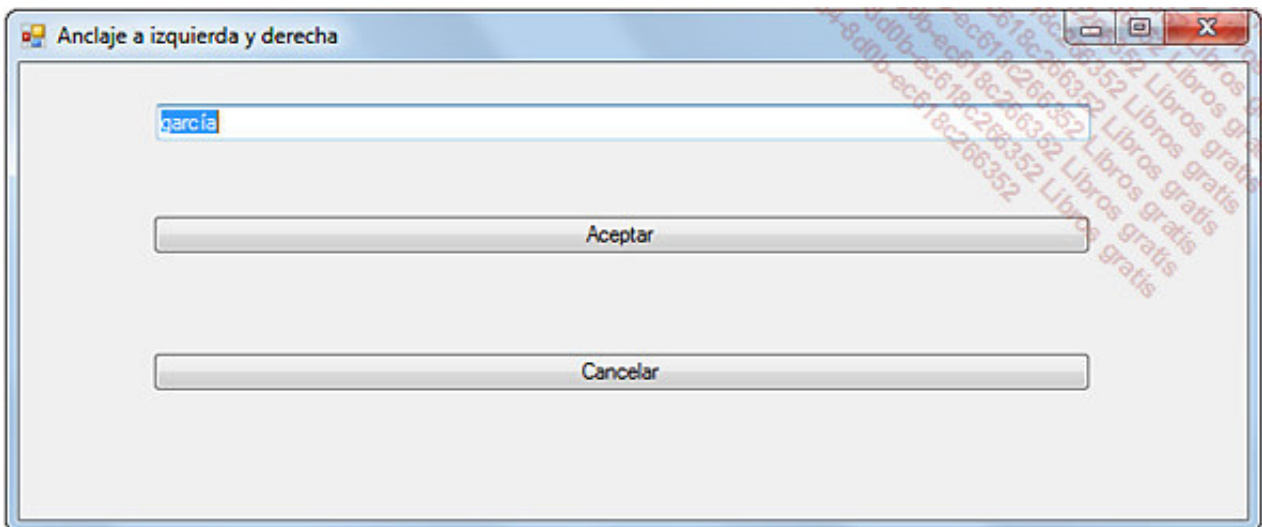


Para modificar la propiedad `anchor`, seleccione el brazo de la estrella que corresponde al lado con el cual quiere realizar un anclaje o bien suprimir uno existente.

Por ejemplo, en la ventana siguiente, los controles están anclados a la izquierda y a la derecha.

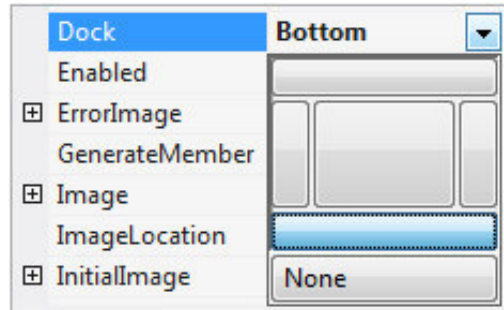


Si redimensionamos la ventana, los controles siguen la ampliación horizontal de la hoja.

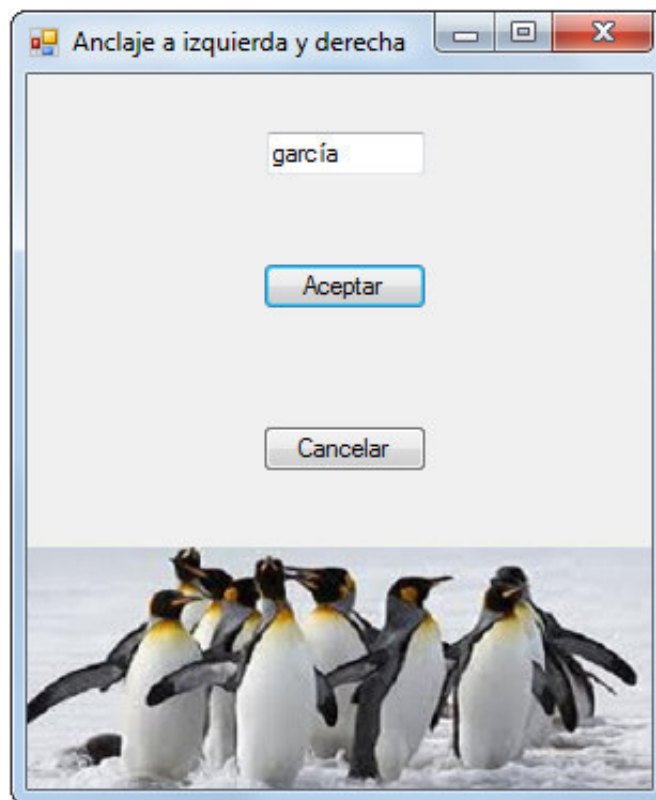


También puede indicar que un control debe adaptar una o varias de estas dimensiones a la de su contenedor. Para ello utilice la propiedad `Dock` del control indicando a qué borde de su contenedor va a adaptar su control una de sus dimensiones.

Por ejemplo, podemos ubicar un control `PictureBox` exigiendo que se amarre al borde inferior de la ventana.



Nuestra `PictureBox` se adapta automáticamente al ancho de la ventana y se queda pegada a su borde inferior.



3. Paso del foco entre controles

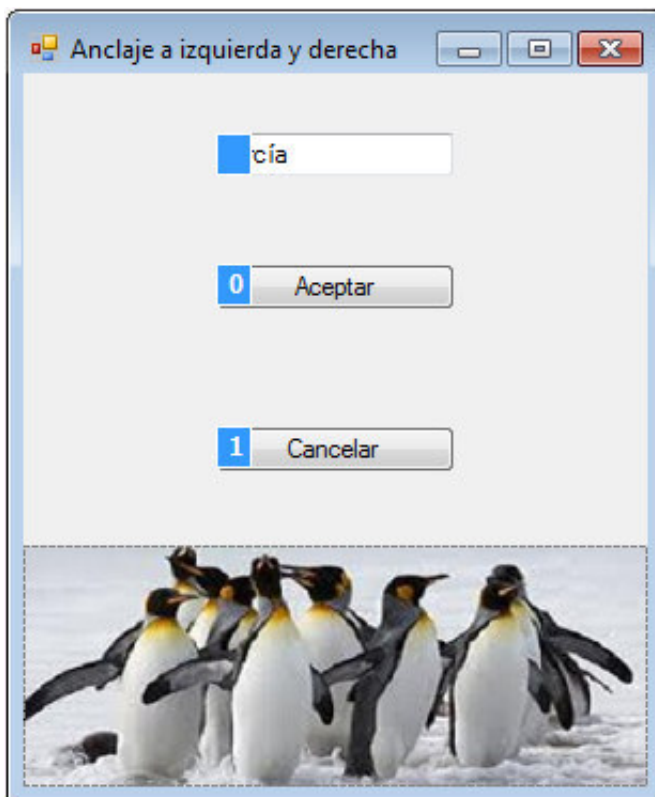
Cuando diseña la aplicación, debe tener en cuenta a las personas contrarias a la utilización del ratón y permitirles a pesar de todo, usar su aplicación. Por lo tanto, conviene diseñar la aplicación para que pueda funcionar con el teclado (isin teclado ni ratón sería mucho más difícil!).

En una aplicación de Windows, se dice que un control tiene el foco cuando está listo para recibir la introducción de datos del usuario. El foco se puede desplazar de control en control utilizando la tecla [Tab]. Dos propiedades de los controles ajustan el paso del foco mediante la tecla [Tab].

- La propiedad `TabStop` indica si un control podrá recibir el foco mediante el uso de la tecla [Tab].
- La propiedad `TabIndex` indica el orden en el cual el foco se pasará entre los controles.

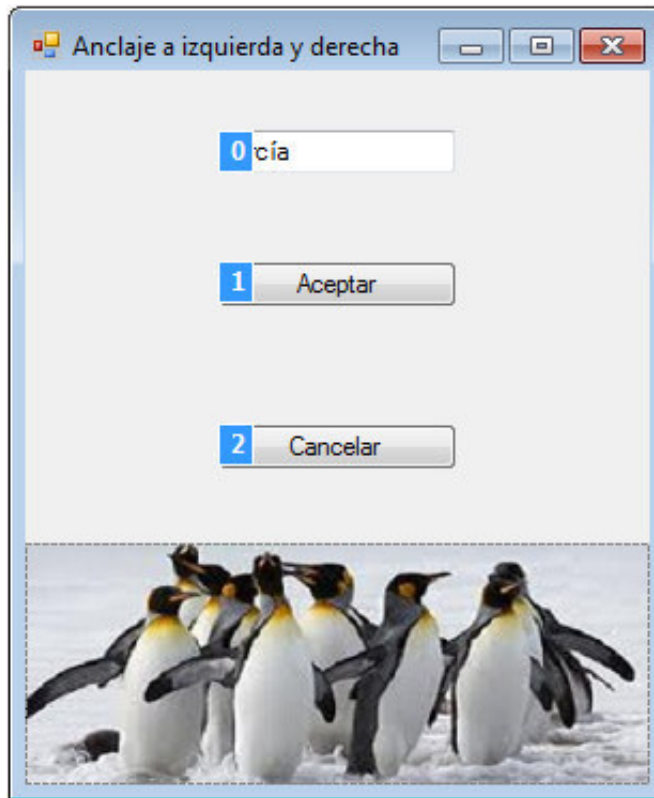
Por defecto, las propiedades `TabIndex` están numeradas en el orden en el que se crean los controles.

Para modificar este orden, puede cambiar directamente la propiedad `TabIndex` de cada control o utilizar el menú **Ver - Orden de tabulación**. Entonces se muestran los controles con el valor de su propiedad `TabIndex` en su esquina superior izquierda.



Luego debería hacer clic en los controles en el orden en el cual quiere que pase el foco.

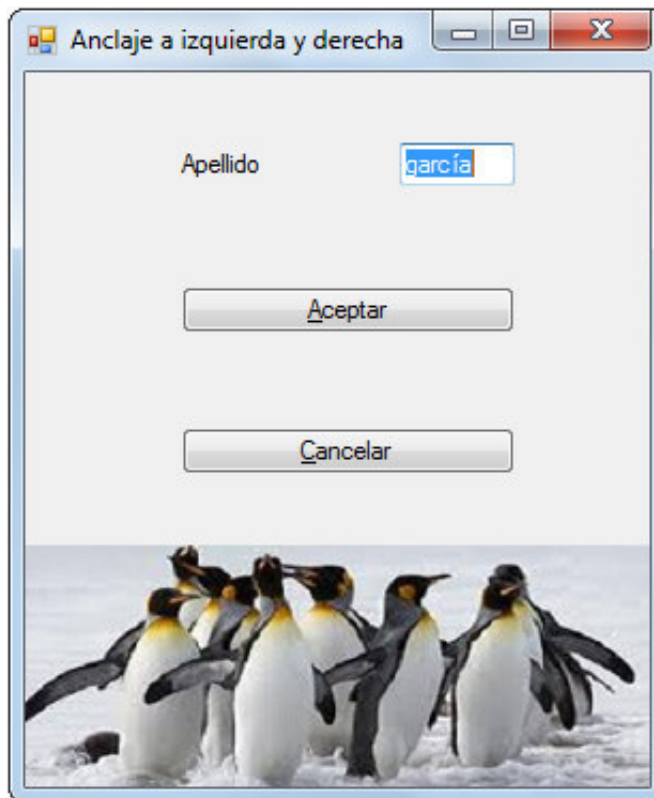
El orden siguiente parece mucho más lógico para este cuadro de diálogo.



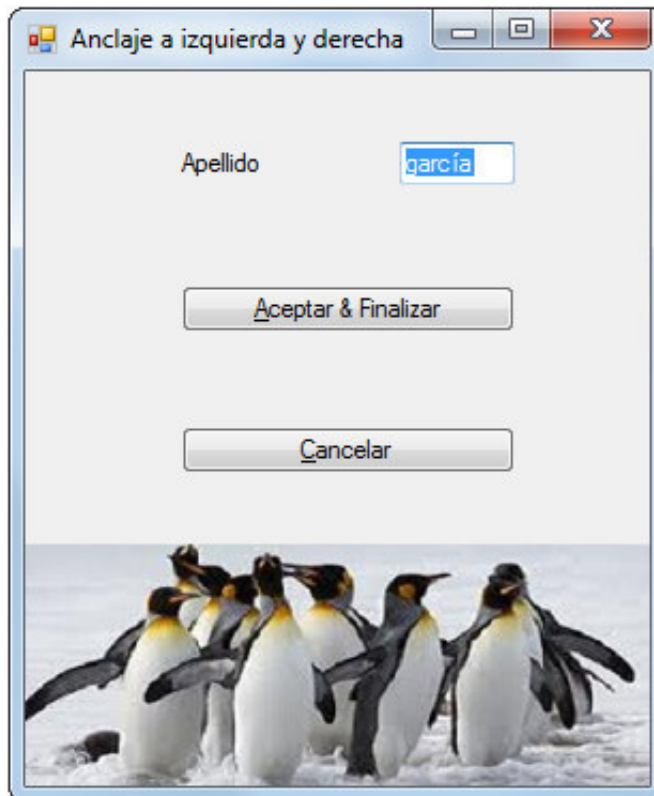
Más tarde puede volver al orden normal utilizando el menú **Ver - Orden de tabulación** o utilizando la tecla [Esc].

4. Atajos de teclado

Algunos usuarios con prisa desean poder desplazarse directamente sobre un control particular sin tener que pasar el foco sobre todos los que le preceden en el orden de tabulación. Para ello, puede añadir un atajo de teclado que se activará mediante la tecla [Alt] y un carácter. Para especificar el carácter que se va a utilizar con objeto de activar el control, hay que añadir en la propiedad `Text` del control un ampersand & delante del carácter utilizado para el atajo del teclado asociado al control. Esto provoca la activación del atajo y el subrayado del carácter en el texto que aparece sobre el control.



Si en cambio, quiere insertar un carácter ampersand &, hay que repetirlo dos veces en su propiedadText.



En el caso de algunos controles (botones, casilla de selección, botones de opción...), la utilización del atajo equivale a un clic de ratón e inicia la acción correspondiente. Para los otros, el atajo del teclado simplemente pone el foco.

Subir

[Condiciones generales de uso](#)

Copyright - ©Editions ENI



Los controles

Cada control utilizable en Visual C# está representado por una clase de la cual vamos a poder crear instancias para diseñar la interfaz de la aplicación. La mayoría de los controles derivan de la clase `Control` y por ello heredan una buena cantidad de sus propiedades, métodos y eventos.

Vamos ahora a estudiar los elementos más útiles de la clase `Control`.

1. La clase Control

a. Dimensiones y posición

Las propiedades `Left`, `Top`, `Width`, `Height` permiten ubicar controles. Se pueden modificar estas propiedades de forma individual. Aceptan valores de tipo `Integer`.

Por lo tanto, es posible utilizar en nuestro código la siguiente sintaxis:

```
TextBoxNombre.Left = 100;  
    TextBoxNom.Top = 50;  
    TextBoxNom.Width = 150;  
    TextBoxNom.Height = 50;
```

Otras dos propiedades permiten trabajar con la posición y el tamaño de un control: la propiedad `Location` acepta un objeto de tipo punto gracias al cual podemos especificar la posición de nuestro control. De igual manera, la propiedad `Size`, que acepta un objeto de tipo `Size`, gestiona las dimensiones del control. Las líneas anteriores se pueden sustituir por:

```
TextBoxNombre.Location = New Point(100, 50);  
    TextBoxNombre.Size = New Size(150, 50);
```

en las cuales construimos una instancia de `Point` y de `Size`, antes de asociarlas a las propiedades correspondientes.

Un tercera posibilidad nos permite manejar a la vez la posición y el tamaño de los controles: la propiedad `Bounds` espera una instancia de clase `Rectangle`, para definir las características del control. Así nuestro código se resume en una única línea:

```
TextBoxNombre.Bounds = New Rectangle(100, 50, 150, 50);
```

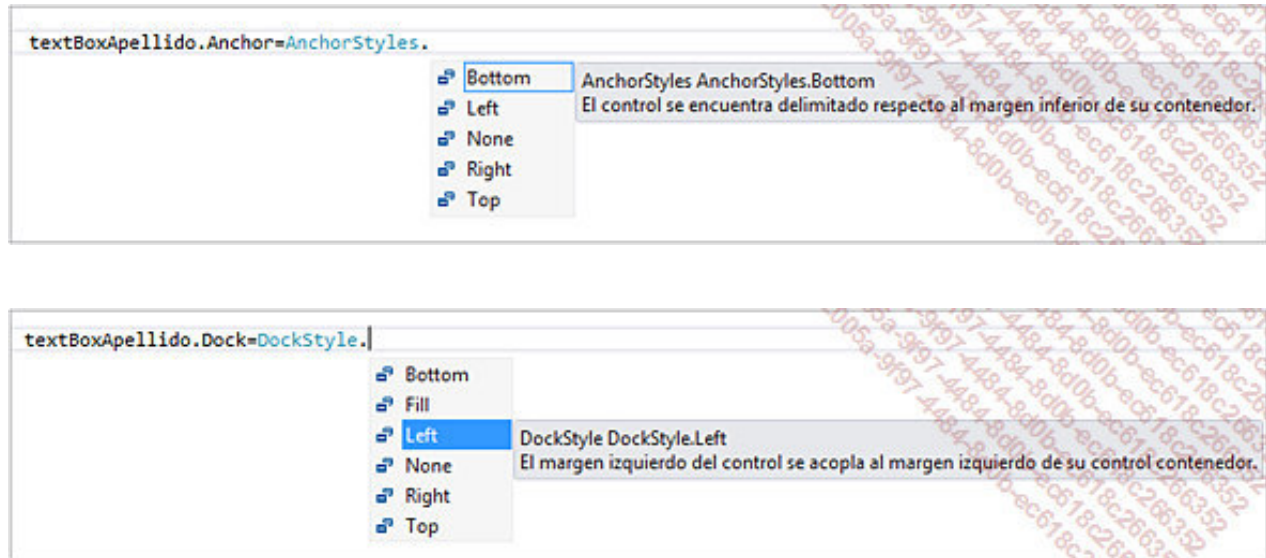
El método `SetBounds` permite también modificar las posiciones y dimensiones de los controles sin tener que crear una nueva instancia de la clase `Rectangle`, sino modificándola ya una vez asociada al control.

```
TextBoxNombre.SetBounds(100, 50, 150, 50);
```

La modificación de estas propiedades implica el lanzamiento de los eventos `Resize` y `Move` sobre el control. Por supuesto, estos eventos se inician cuando el valor de las propiedades se modifica en el código, pero también, por ejemplo, la modificación del tamaño de la ventana implica una reubicación o redimensionamiento del control.

El comportamiento de los controles cuando se redimensiona la ventana es especificado por las propiedades `Anchor` y `Dock`. Ya hemos visto cómo modificarlas en la ventana de propiedades. Para

modificarlas mediante código, basta asignarles alguno de los valores definidos en las enumeraciones `Anchor-Styles` y `DockStyle`.



Hasta ahora, las posiciones con las cuales hemos trabajado eran posiciones expresadas con respecto a la esquina superior izquierda del contenedor del control. En algunos casos, puede ser útil obtener las coordenadas de un punto del control no tanto con respecto a la esquina superior izquierda del control, sino con respecto a la de la pantalla. El método `PointToScreen` permite esta conversión. Espera como parámetro una instancia de la clase `Point` con las coordenadas expresadas con respecto al control y devuelve una nueva instancia de la clase `Point` con las coordenadas expresadas con respecto a la pantalla.

El siguiente código convierte en coordenadas de pantalla la posición superior izquierda de un control `TextBox`:

```
private void button1_Click(object sender, EventArgs e)
{
    Console.WriteLine("Control/Ventana");
    Console.WriteLine(button1.Location);
    Point p=new Point(0,0);
    p=button1.PointToScreen(button1.Location);
    Console.WriteLine("Control/Pantalla");
    Console.WriteLine(p);
}
```

Resultado:

```
Control/Ventana:{X=107,Y=72}
Control/Pantalla:{X=306,Y=255}
```

Se puede realizar la operación inversa con el método `pointToClient`, que toma como parámetro un punto en coordenadas de pantalla y devuelve un punto expresado en coordenadas relativas al control. Si se efectúa la operación inversa, es decir, a partir de las coordenadas de la pantalla, se obtiene efectivamente el mismo valor:

```
Console.WriteLine("Control/Ventana a partir de la pantalla" +
    button1.PointToClient(p).ToString());
```

Resultado:

Control/Ventana a partir de la pantalla{X=107,Y=72}

b. Apariencia de los controles

El color de fondo se puede modificar con la propiedad `BackColor`, mientras que se puede modificar el color de texto del control con la propiedad `ForeColor`.

Se puede asignar a dichas propiedades valores definidos en el espacio de nombres `System.Drawing.Color` para obtener colores predefinidos en Visual Basic.

```
TextBoxNombre.BackColor = System.Drawing.Color.Yellow;
```

También se pueden utilizar las constantes definidas en el espacio de nombres `System.Drawing.SystemColors` para utilizar uno de los colores definidos a nivel del propio sistema. El interés en este caso es que su aplicación se adaptará en función de la configuración de la máquina sobre la que se instale.

```
TextBoxNombre.BackColor=System.Drawing.SystemColors.InactiveCaptionText;
```

Una tercera solución consiste en que efectúe la mezcla de color usted mismo, utilizando la función `FromArgb` y especificando como parámetro la cantidad de cada uno de los colores básicos (rojo, verde, azul).

```
TextBoxNombre.BackColor = System.Drawing.Color.FromArgb (127, 0, 127);
```

Se puede modificar la fuente con la propiedad `Font` del control. Para ello se puede crear una nueva instancia de la clase `Font` y asignarla al control. Hay trece constructores diferentes para la clase `Font` y, por lo tanto, trece maneras diferentes de crear una fuente de carácter. Utilizaremos la más simple indicando el tipo de fuente y el tamaño.

```
TextBoxNombre.Font =  
New Font(System.Drawing.FontFamily.GenericMonospace, 16);
```

Tras haber efectuado modificaciones en estas propiedades es posible volver a una configuración normal llamando a los métodos `ResetBackColor`, `ResetForeColor`, `ResetFont`. Las propiedades correspondientes se reinician con los valores definidos para el contenedor del control.

La propiedad `BackgroundImage` permite especificar una imagen que se utilizará como fondo para el control. Si la imagen no tiene suficiente tamaño para cubrir el control, se representará en forma de mosaico:

```
BtnValidar.BackgroundImage = New Bitmap("cut.bmp");
```

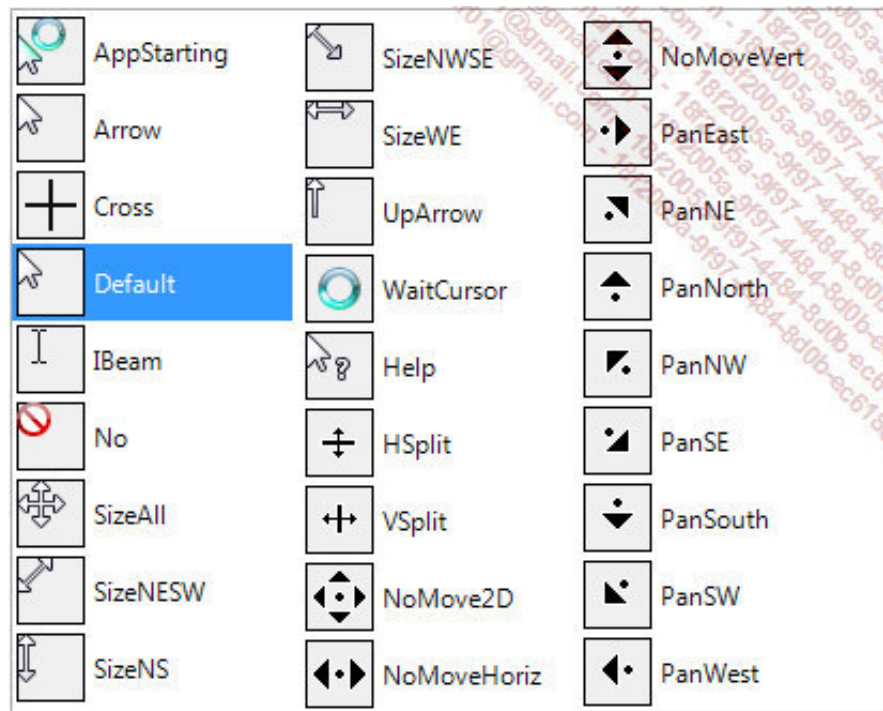


El resultado es sorprendente.

Para regresar a algo más clásico:

```
BtnValidar.BackgroundImage = null;
```

La propiedad `Cursor` permite elegir la apariencia de un cursor cuando el ratón se encuentra sobre la superficie de un control. Hay varios cursores predefinidos en Windows.



Estos cursores están almacenados en una colección **Cursors** y puede utilizarlos directamente asignándolos a la propiedad `Cursor` del control.

```
BtnValidar.Cursor = Cursors.WaitCursor;
```

Si, entre éstos, ninguno le conviene, puede utilizar un cursor personalizado creando una instancia de la clase `Cursor` y asignándola a la propiedad `Cursor` del control.

```
BtnValidar.Cursor = New Cursor("h_nodrop.cur");
```

El propio control gestiona automáticamente la detección de la entrada y la salida del ratón sobre él, así como la modificación del propio cursor como consecuencia de ello.

- Como en el caso de la fuente de caracteres, es posible restaurar el cursor por defecto llamando al método `ResetCursor`.

La modificación de la mayoría de las propiedades de los controles inicia un evento. Estos eventos son identificados por el nombre de la propiedad seguido del sufijo `Changed`. Se pueden utilizar para guardar las preferencias del usuario cuando personaliza la aplicación.

c. Comportamiento de los controles

Se pueden ocultar los controles ubicados en la hoja modificando la propiedad `Visible` o desactivarlos modificando la propiedad `Enabled`. En este caso, el control sigue visible, pero aparece con un aspecto gris para indicar al usuario que de momento está inactivo.

```
BtnValidar.Enabled = False;
```



Por supuesto, los controles en este estado no pueden recibir el foco en la aplicación. Puede comprobar este término examinando la propiedad `CanFocus`, que devuelve un booleano. También puede comprobar si un control tiene actualmente el foco verificando la propiedad `Focused` o la propiedad `ContainsFocus`. Esta última se debe utilizar con los controles contenedores (es decir, los controles que pueden contener otros controles). En este caso, esta propiedad vale **True** si uno de los controles ubicados en el interior del contenedor tiene el foco.

Se puede ubicar el foco en un control sin la intervención del usuario llamando al método `Focus` del control.

```
BtnValidar.Focus();
```

Para vigilar el paso del foco de un control a otro, hay cuatro eventos disponibles:

- `Enter` indica que el foco ha llegado a uno de los controles de un contenedor.
- `GotFocus` indica que un control particular ha recibido el foco.
- `LostFocus` indica que un control ha perdido el foco.
- `Leave` indica que el foco ya no está en uno de los controles del contenedor.

Por ejemplo, para visualizar correctamente que un control tiene el foco, se puede usar el siguiente código, que modifica el control del texto cuando el control recibe o pierde el foco:

```
void txtNombre_LostFocus(object sender, EventArgs e)
{
    txtNombre.ResetForeColor(); ;
}
void txtNombre_GotFocus(object sender, EventArgs e)
{
    txtNombre.ForeColor = Color.Green;
}
```

En algunos casos, es mejor comprobar la introducción de datos del usuario en un formulario antes de continuar en la aplicación. Se puede efectuar esta comprobación al cerrar el formulario o mientras el usuario introduce datos en los diferentes controles del formulario. Se puede configurar cada control para permitir la verificación de los datos introducidos modificando la propiedad `CausesValidation` a **True**. Justo antes de que el control pierda el foco, se lanza el evento `Validating` para permitir verificar la introducción de datos del usuario. Si la inserción de datos no es correcta (en función de criterios que hemos fijado), podemos bloquear el paso del foco hacia otro control modificando la propiedad `Cancel` del objeto `CancelEventArgs`, que es pasado como parámetro. En este caso, el foco se queda en el control en el que la introducción de datos no es correcta. En cambio, si la introducción de datos es correcta, el evento `Validated` es lanzado sobre el control y el foco se desplaza al siguiente control.

Por ejemplo, para introducir un número de teléfono, podemos verificar que únicamente se han tecleado valores numéricos. En caso de error generamos un bip, modificamos el color del texto y bloqueamos el paso del foco a otro control.

```
private void txtTel_Validating(object sender, CancelEventArgs e)
{
    bool resultado;
    long i;
    resultado = long.TryParse(txtTel.Text, out i);
    if (!resultado)
    {
        SystemSounds.Beep.Play();
        txtTel.ForeColor = Color.Red;
        e.Cancel = true;
    }
}
private void txtTel_Validated(object sender, EventArgs e)
{
    txtTel.ResetForeColor();
}
```

A veces pueden ser útiles dos propiedades cuando trabajamos con controles contenedores. La propiedad `HasChildren` nos permite saber si hay controles ubicados en nuestro contenedor. Si es el caso, la colección `Controls` contiene la lista de todos estos controles. Podemos modificar, por ejemplo, el color del texto de todos los controles de un contenedor cuando el foco esté ubicado en uno de ellos.

```
private void grBoxIdent_Enter(object sender, EventArgs e)
{
    if (grBoxIdent.HasChildren)
    {
        foreach (Control c in grBoxIdent.Controls)
        {
            c.ForeColor = Color.YellowGreen;
        }
    }
}
private void grBoxIdent_Leave(object sender, EventArgs e)
{
    if (grBoxIdent.HasChildren)
    {
        foreach (Control c in grBoxIdent.Controls)
        {
            c.ResetForeColor();
        }
    }
}
```

La operación inversa también es posible. Es decir que, a partir de un control, podremos recuperar las propiedades de su contenedor. La propiedad `Parent` proporciona una referencia hacia el contenedor del control. Por ejemplo, podemos hacer de tal manera que el color de fondo de cada control cambie al mismo tiempo que el de su contenedor.

```
private void grBoxIdent_BackColorChanged(object sender, EventArgs e)
{
    foreach (Control c in grBoxIdent.Controls)
    {
        c.BackColor = c.Parent.BackColor;
    }
}
```


Ahora que hemos examinado las propiedades comunes a los diferentes controles disponibles, los vamos a estudiar uno por uno explorando sus particularidades.

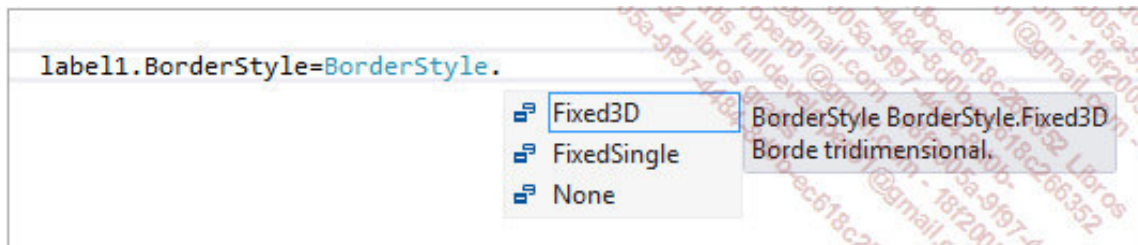
2. Los controles que visualizan información

a. El control Label

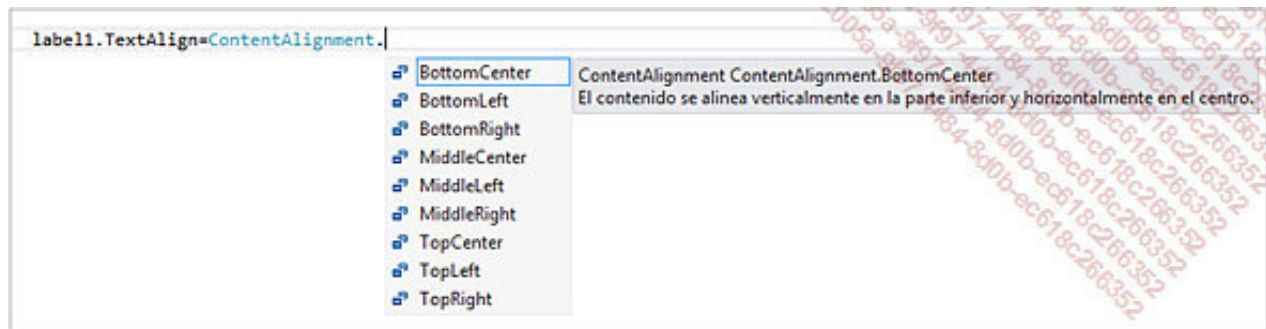
El control `Label` se utiliza para mostrar en un formulario un texto que no podrá ser editado por el usuario. Sirve esencialmente para proporcionar una etiqueta a controles que no disponen de ella (cuadros de texto, desplegables...). En estos casos también va a permitir facilitar un atajo de teclado para llegar al control.

El texto visualizado por el control está indicado por la propiedad `Text`. Naturalmente esta propiedad podrá ser modificada por el código de la aplicación. Sin embargo, hay que ser prudente, ya que, por defecto, el control conservará el tamaño que usted le dio en tiempo de diseño. Si la nueva cadena de caracteres asignada a la propiedad `Text` es mayor que la especificada en el momento de diseñar, sólo la primera parte será visible. Para evitar este problema, hay que pedir al control `Label` que adapte su anchura en función del texto que se debe visualizar poniendo la propiedad `AutoSize` a **True**.

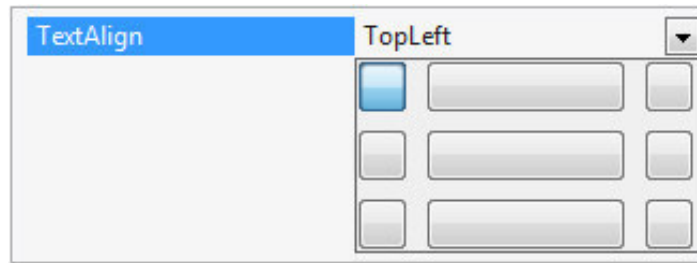
Por defecto, el control `Label` no dispone de borde. Puede añadir uno modificando la propiedad `BorderStyle`, utilizando alguno de los tres valores disponibles.



También tiene la posibilidad de indicar la posición del texto en el control mediante la propiedad `TextAlign`. En el código, podrá usar cualquiera de las constantes predefinidas.



A través de la ventana de propiedades, sólo hace falta hacer clic en la posición deseada para el texto en el interior de su control.



- Tenga en cuenta, sin embargo, que la propiedad `TextAlign` modificará la posición del texto únicamente si la propiedad `AutoSize` está en **False**.

Los controles `Label` también pueden mostrar imágenes. Puede indicar la imagen que desea que se muestre usando la propiedad `Image`. Otra solución consiste en utilizar un control `ImageList` que servirá, en cierta manera, como almacén de imágenes de la aplicación. En tal caso, usted indicará, mediante la propiedad `ImageList`, en qué control buscará la imagen, y gracias a la propiedad `ImageIndex` qué posición ocupa en el control `ImageList`. Si utiliza un control `ImageList`, la propiedad `Image` de su control se ignorará. De la misma forma que en el caso del texto, puede modificar la posición de la imagen en el control mediante la propiedad `ImageAlign` con las mismas constantes que para la propiedad `TextAlign`.

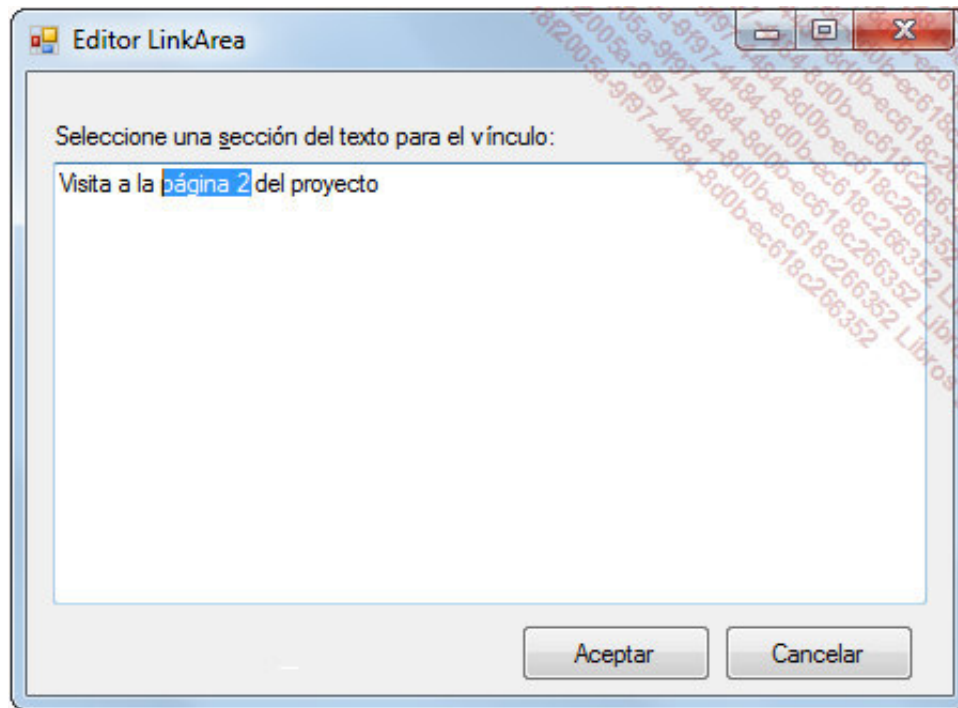
Hemos indicado que el control `Label` se puede utilizar como atajo de teclado por otro control. Para ello, tome las siguientes tres precauciones.

- Como para los otros controles, añada un `&` en la propiedad `Text` para el carácter utilizado en el atajo.
- Indique al control `Label` su papel de gestor de atajos de teclado modificando la propiedad `UseMnemonic` a **True**.
- Verifique que el control, que debe recibir el foco, se encuentra inmediatamente después del control `Label` en el orden de tabulación (propiedad `TabIndex`).

b. El control `LinkLabel`

El control `LinkLabel` hereda todas las características del control `Label` y simplemente añade funcionalidades de enlace tipo `Web`. Las propiedades adicionales respecto al control `Label` gestionan los diferentes parámetros del enlace.

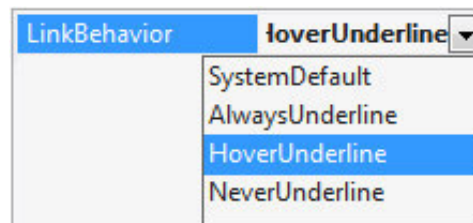
La propiedad `LinkArea` indica qué porción del texto activará el enlace. Se puede modificar esta propiedad mediante la ventana de propiedades, con una pequeña herramienta con la que podrá seleccionar la porción del texto que forma el enlace.



Se pueden establecer los colores usados por el enlace gracias a tres propiedades:

<code>LinkColor</code>	color del enlace en estado normal.
<code>VisitedLinkColor</code>	color del enlace después de un primer uso.
<code>ActiveLinkColor</code>	color del enlace en el momento se activa.

Se puede modificar la apariencia mediante la propiedad `LinkBehavior`.



Los cuatro valores posibles permiten respectivamente:

- Utilizar la misma configuración del navegador por defecto para los enlaces.
- Tener los enlaces siempre subrayados.
- Tener los enlaces subrayados cuando el ratón pasa por ellos.
- No tener enlaces subrayados.

Cuando el usuario hace clic en el enlace, se lanza el evento `LinkClicked` en la aplicación. Le corresponde a usted escribir código para ejecutar una acción en su aplicación.

También debe modificar la propiedad `LinkVisited` poniéndola a **True**, para indicar que este enlace ya ha sido utilizado en la aplicación.

La acción puede ser la apertura de una página en el sitio Web en el navegador por defecto, como en el siguiente ejemplo:

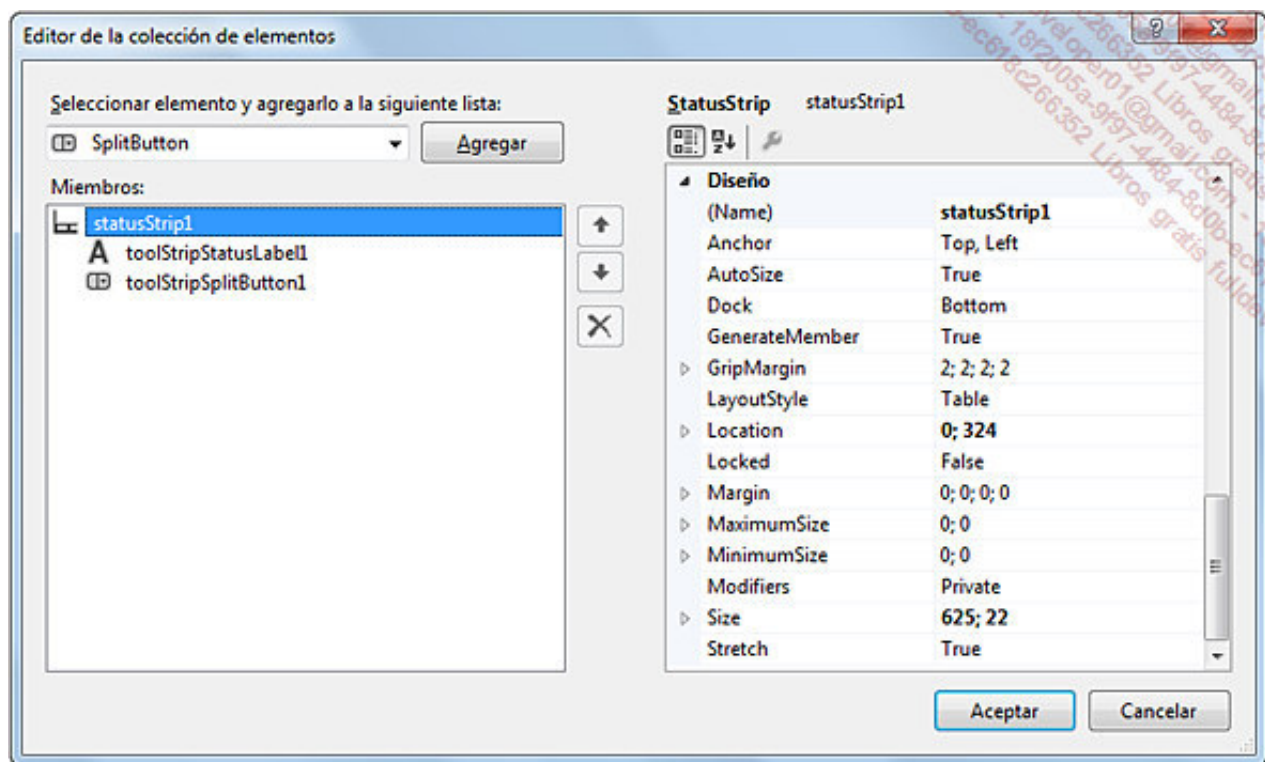
```
private void linkLabel1_LinkClicked(object sender,
                                   LinkLabelLinkClickedEventArgs e)
{
    System.Diagnostics.Process.Start("http://www.microsoft.com");
    linkLabel1.LinkVisited = true;
}
```

O, también, la acción podría ser la visualización de una nueva hoja en nuestra aplicación, como en el siguiente ejemplo:

```
private void linkLabel2_LinkClicked(object sender,
                                   LinkLabelLinkClickedEventArgs e)
{
    testDialogos d;
    d = new testDialogos();
    d.ShowDialog();
    linkLabel2.LinkVisited = true;
}
```

c. El control StatusStrip

Se suele utilizar el control `StatusStrip` para presentar información al usuario relativa al funcionamiento de la aplicación. Puede mostrar la información en varios tipos de áreas. Se puede visualizar esa información en forma de texto, de barra de progreso, de menú o de botón de comando asociado a un menú. Un editor específico accesible mediante la propiedad `Items` del control permite su configuración.

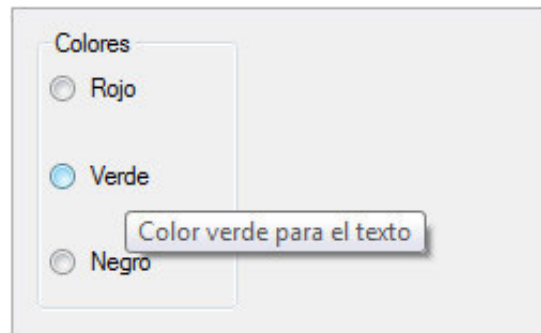


A continuación, cada elemento insertado en el control `StatusStrip` debe configurarse individualmente. Las propiedades de los elementos que se pueden utilizar para la construcción de un `StatusStrip` son muy similares a las de los controles normales. Por ejemplo, el elemento `ToolStripStatusLabel` es casi idéntico al control `LinkLabel`.

d. El control ToolTip

Este control permite la visualización de una etiqueta de ayuda asociada a un control. Este control no tiene interfaz visible; por lo tanto, estará ubicado en la zona situada debajo de la ventana de diseño. Efectúa mucho trabajo sin ningún esfuerzo de programación. Por ejemplo, vigila permanentemente dónde está el ratón; si éste está en un control, comprueba si hay una etiqueta de información asociada al control. Si es el caso, visualiza esta etiqueta durante el tiempo establecido en la propiedad `AutoPopDelay`.

Para poder funcionar, el control `ToolTip` debe asociar una cadena de caracteres a cada uno de los controles de la interfaz. Para ello, en cuanto un control `ToolTip` está disponible en una hoja, se añade una propiedad `ToolTip` a cada uno de los controles, lo que permite especificar el texto de la etiqueta de información asociada al control.



También se pueden indicar las cadenas de caracteres asociadas a cada control mediante algo de código llamando al método `SetToolTip` e indicando como parámetro el nombre del control y la cadena de caracteres que tiene asociada.

```
toolTip1.SetToolTip(radioButton1, "Color rojo para el texto");
```

Esta técnica permite conservar leyendas relativamente cortas para los controles proporcionando a la vez bastante información sobre el uso de la aplicación.

e. El Control ErrorProvider

Este control permite indicar fácilmente al usuario problemas relativos a los datos que ha introducido en un formulario. Suele intervenir durante la fase de validación de los datos del formulario, visualizando frente cada control un pequeño icono  con el fin de atraer la atención del usuario. Se pueden facilitar datos adicionales mediante una etiqueta de información asociada al control `ErrorProvider`.

Un mismo control `ErrorProvider` puede utilizarse para todos los controles de un formulario.

La activación del control `ErrorProvider` se puede efectuar al cerrar el formulario cuando el usuario hace clic en el botón **Aceptar**. Pero también es posible supervisar la inserción de datos a medida que ésta se efectúa gestionando por ejemplo los eventos `Validating`. Este evento es activado por un control cuando éste pierde el foco. Así podemos verificar inmediatamente el valor introducido en el control y reaccionar en consecuencia visualizando nuestro control `ErrorProvider`. Para ello llamamos al método `SetError` especificando el nombre del control que nos da problemas y la cadena de caracteres visualizada en la etiqueta de información asociada al control. Si no hay error, hay que reinicializar la cadena para hacer desaparecer el icono del control `ErrorProvider`.

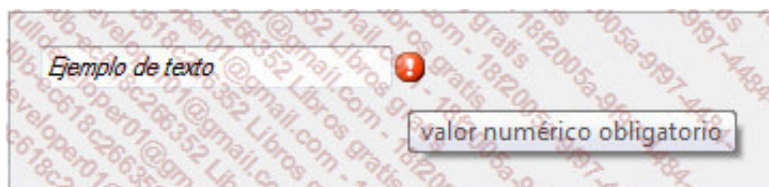
```
private void txtTel_Validating(object sender, CancelEventArgs e)
```

```

{
    bool resultado;
    long i;
    resultado = long.TryParse(txtTel.Text, out i);
    if (!resultado)
    {
        SystemSounds.Beep.Play();
        txtTel.ForeColor = Color.Red;
        e.Cancel = true;
        errorProvider1.SetError(txtTel, "valor numérico obligatorio");
    }
}

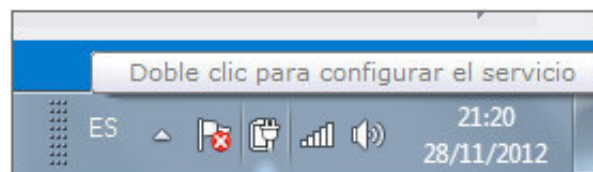
private void txtTel_Validated(object sender, EventArgs e)
{
    txtTel.ResetForeColor();
    errorProvider1.SetError(txtTel, "");
}

```



f. El control NotifyIcon

Se utiliza principalmente este control para mostrar información relativa al funcionamiento de un proceso que se ejecuta en segundo plano en la aplicación. Se visualiza en la zona de status del sistema. La propiedad `Icon` del control determina el icono mostrado. La propiedad `Text` representa la leyenda visualizada cuando el ratón pasa por encima del control.



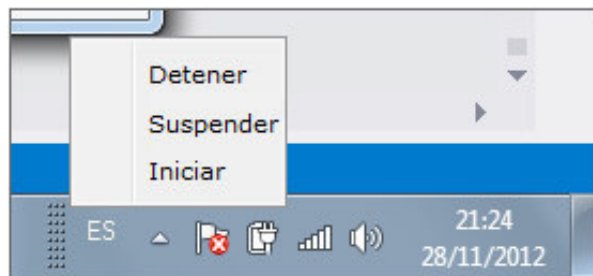
A través de la gestión del evento `DoubleClick` del control, puede visualizar un cuadro de diálogo que permite la configuración del proceso asociado al control.

```

private void IconService_DoubleClick(object sender, EventArgs e)
{
    DialogoConfig d;
    d = new DialogoConfig();
    d.ShowDialog();
}

```

Es igualmente posible asociar un menú contextual, indicando la propiedad `ContextMenuStrip`. Este menú puede controlar, por ejemplo, el funcionamiento del proceso al cual se asocia el control.



g. El control HelpProvider

El control `HelpProvider` asegura el vínculo entre un archivo de ayuda y la aplicación. Se debe generar el archivo de ayuda con la herramienta `Html Help Workshop`, disponible para descargar desde el sitio de Microsoft. Para nuestro ejemplo, utilizaremos un archivo de ayuda existente en el sistema: `C:\WINDOWS\Help\charmap.chm`, que corresponde a la herramienta Tabla de caracteres. Este archivo debe estar asociado al control por la propiedad `HelpNamespace`. La presencia de un control `HelpProvider` en una ventana añade automáticamente tres propiedades a cada control presente en la ventana:

`HelpKeyword`

Indica la palabra clave asociada al control en el archivo de ayuda.

`HelpNavigator`

Indica la acción ejecutada durante la visualización de la ayuda.

`HelpString`

Contiene la cadena de caracteres visualizada durante el uso de un botón  de un cuadro de diálogo. Para que este botón esté disponible en el cuadro de diálogo, hay que modificar la propiedad `HelpButton` de la ventana a **True** y ocultar los botones de maximizar y minimizar de la ventana, modificando las propiedades `MaximizeBox` y `MinimizeBox` a **False**.

El siguiente ejemplo asocia al botón de comando `CmdOk` la sección de ayuda **Vista general de la tabla de caracteres** del archivo `charmap.chm` y configura el sistema de ayuda para que se pueda mostrar dicha sección automáticamente al pulsar la tecla [F1].

<code>HelpKeyword</code> en <code>helpProvider1</code>	Vista de conjunto de la tabla de caracteres
<code>HelpNavigator</code> en <code>helpProvider1</code>	<code>AssociateIndex</code>
<code>HelpString</code> en <code>helpProvider1</code>	cadena que se muestra cuando se utiliza el botón ?

h. El control ProgressBar

Este control se utiliza para informar al usuario sobre el progreso de una acción iniciada en la aplicación. Muestra esta información en forma de una zona rectangular que estará más o menos llena en función del estado de avance de la acción ejecutada. El aspecto del `ProgressBar` se controla por su propiedad **Style**. Hay tres valores posibles:

`Continuous`

El progreso se visualiza mediante una barra azul llena.

Blocks

El progreso se visualiza mediante una serie de pequeños rectángulos.

Marquee

Esta presentación es idéntica a la anterior añadiendo un deslizamiento al `ProgressBar`.

La posición de la barra de progreso está controlada por la propiedad `Value`. Esta propiedad puede evolucionar entre los dos extremos marcados por las propiedades `Minimum` y `Maximum`.

Hay tres técnicas posibles para hacer progresar la barra:

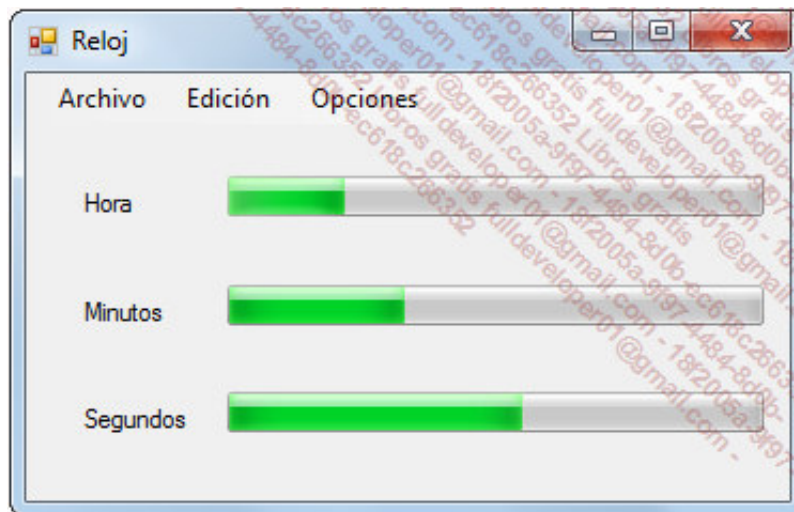
- Modificar directamente la propiedad `Value` del control. Tenga en cuenta que, en este caso, si el valor de esta propiedad supera los límites, se lanza una excepción.
- Utilizar el método `PerformStep`, que incrementa en cada llamada la propiedad `Value` del valor fijado en la propiedad `Step`. En este caso, el control verifica el valor de la propiedad `Value` y se asegura de que no superará los límites.
- Utilizar el método `Increment` indicando como parámetro el valor utilizado como incremento para la propiedad `Value`. También se puede comprobar el valor de la propiedad `Value` durante la ejecución de este método.



Si el `ProgressBar` tiene el estilo `Marquee`, la propiedad `Value` no tiene ningún efecto sobre el tamaño de la barra de progreso y no se deberían utilizar los métodos `PerformStep` e `Increment`, porque si no se produce una excepción.

El siguiente ejemplo presenta un reloj original donde la hora está visualizada por tres `ProgressBar`:

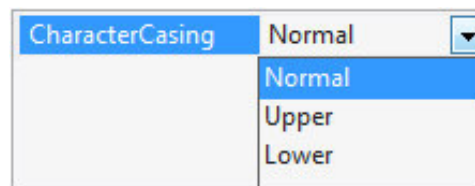
```
public partial class Reloj : Form
{
    public Reloj()
    {
        InitializeComponent();
        pgbHora.Minimum = 0;
        pgbHora.Maximum = 23;
        pgbHora.Style = ProgressBarStyle.Continuous;
        pgbMinuto.Minimum = 0;
        pgbMinuto.Maximum = 59;
        pgbMinuto.Style = ProgressBarStyle.Continuous;
        pgbSegundo.Minimum = 0;
        pgbSegundo.Maximum = 59;
        pgbSegundo.Style = ProgressBarStyle.Continuous;
        Timer1.Interval = 10;
        Timer1.Enabled = true;
    }
    private void Timer1_Tick(object sender, System.EventArgs e)
    {
        pgbHora.Value = DateTime.Now.Hour;
        pgbMinuto.Value = DateTime.Now.Minute;
        pgbSegundo.Value = DateTime.Now.Second;
    }
}
```



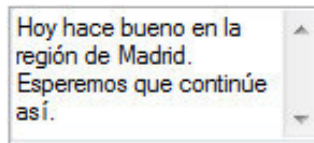
3. Los controles de edición de texto

a. El control TextBox

Se utiliza el control `TextBox` para permitir al usuario introducir datos. Se puede configurar el control para insertar texto en una o varias líneas. El tamaño máximo del texto varía de 2.000 a 32.000 caracteres, según la configuración del control (línea simple o líneas múltiples). También el control es capaz de gestionar la selección de texto y las operaciones con el portapapeles. Hay muchas propiedades y métodos disponibles para trabajar con este control. El texto mostrado en el control puede modificarse o recuperarse mediante la propiedad `Text`. Es posible modificar el formato de visualización del texto mediante distintas propiedades. La propiedad `Autosize` permite pedir al control `TextBox` redimensionarse en función del tamaño de la fuente de caracteres. Esta propiedad está colocada casi siempre en **True**. La propiedad `CharacterCasing` autoriza al control a modificar todos los caracteres introducidos a minúsculas o mayúsculas.



La propiedad `Lines` permite recuperar el texto introducido, línea a línea. Esta propiedad es una matriz de cadenas de caracteres que contiene tantas cajas como líneas, y sólo tiene interés si el control está configurado para aceptar la introducción de datos en varias líneas con la propiedad `Multiline` puesta a **True**. En este caso, también hay que prever la posibilidad de poder desplazar el texto añadiendo barras de desplazamiento con la propiedad `ScrollBars`. Las distintas posibilidades permitirán disponer de una barra de desplazamiento horizontal, vertical o ambas. Cuidado, sin embargo, ya que la barra de desplazamiento vertical sólo será visible si la propiedad `WordWrap` está en **False**. En caso contrario el control gestiona por sí mismo el salto de línea cuando la longitud de la línea supera la anchura del control. En contraposición, en este caso, los retornos de carro añadidos automáticamente no se insertan en el texto.



En este ejemplo, la propiedad `Lines` contendrá dos elementos, ya que el primer retorno de carro es añadido por el control simplemente para la visualización.

`Lines(0)`-> Hoy hace bueno en la región de Madrid

`Lines(1)`-> Esperemos que continúe así

La longitud máxima del texto del control es fijada por la propiedad `MaxLength`. Hay que tener en cuenta que, en el caso de un control de líneas múltiples, los caracteres de retorno de carro y salto de línea también cuentan. Se utiliza esta propiedad a menudo cuando se hace uso del control `TextBox` para introducir una contraseña. En este caso, la propiedad `PasswordChar` indica el carácter utilizado durante la visualización para ocultar la inserción del usuario. Se suele utilizar el carácter `*` o `#`. Esta propiedad, por supuesto, sólo influye en la visualización. Los caracteres introducidos por el usuario se pueden recuperar con la propiedad `Text`.

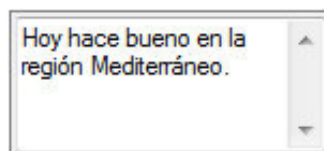
La gestión de la selección del texto la realiza el control de forma automática. La propiedad `SelectedText` permite recuperar la cadena de caracteres actualmente seleccionada en el control. Las propiedades `SelectionStart` y `SelectionLength` indican respectivamente el carácter del inicio de la selección (el primer carácter de índice 0) y el número de caracteres de la selección. También se utilizan estas propiedades para insertar texto en el control: la propiedad `SelectionStart` indica en este caso el punto de inserción y la propiedad `SelectedText`, el texto que se ha de insertar. Para añadir texto después del ya existente en el control, resulta más práctico emplear el método `AppendText` pasándole como parámetro la cadena de caracteres que hay que añadir.

La sustitución de una porción de texto en el control `TextBox` se ejecuta en dos etapas. Primero hay que seleccionar el texto que se desea sustituir usando las propiedades `SelectionStart` y `SelectionLength`. Y luego hay que indicar el texto de sustitución con la propiedad `SelectedText`. El texto sustituido y el de sustitución no tienen por qué disponer del mismo tamaño.

```
TextBox1.SelectionStart = 28;  
TextBox1.SelectionLength = 10;  
TextBox1.SelectedText = "Mediterráneo";
```

- La selección de texto también puede efectuarse con el método `Select`, indicando el carácter de inicio de la selección y el número de caracteres de la selección.

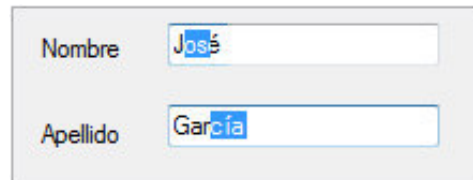
```
TextBox1.Select(28,10);  
TextBox1.SelectedText = "Mediterráneo";
```



De la selección de la totalidad del texto se encarga el método `SelectAll`. Por ejemplo, se puede forzar la selección de todo el texto cuando el control recibe el foco.

```
void textBox1_GotFocus(object sender, System.EventArgs e)
{
    textBox1.SelectAll();
}
```

De manera clásica, en cuanto un control pierde el foco, la selección de texto que estaba en el interior del texto ya no es visible. La propiedad `HideSelection` colocada en **False** permite conservar la selección visible, incluso si el control ya no tiene el foco.



Para la gestión del portapapeles, el control `TextBox` dispone de un menú contextual que permite efectuar las operaciones corrientes. Sin embargo, tiene la posibilidad de llamar los métodos `copy`, `cut` y `paste` para gestionar las operaciones de copiar y pegar de otra manera, por ejemplo un menú de la aplicación. Las operaciones cortar y pegar no serán posibles si el control `TextBox` está configurado en sólo lectura con la propiedad `ReadOnly` a **True**; la modificación del texto por el usuario es imposible en este caso.

Como todo el mundo se puede equivocar, el control `TextBox` nos propone el método `Undo`, que permite cancelar la última modificación de texto efectuada en el control. Este método ya se puede utilizar con la opción **Deshacer** del menú contextual del control `TextBox` o con el atajo de teclado **[Ctrl] Z**. También se le puede llamar gracias a otro menú de su aplicación. Sólo hay un nivel de "Undo". ¡No podrá volver al texto que introdujo hace dos horas!

Este control cuenta con el evento `TextChanged`. Se lanza cuando la propiedad `Text` del control ha sido modificada (por el código de la aplicación o por el usuario).

b. El control `MaskedTextBox`

Este control representa una mejora respecto al control `TextBox`, ya que permite verificar automáticamente que los datos introducidos corresponden a lo esperado por la aplicación. La propiedad `Mask` determina el formato de los datos que se pueden introducir en el control. El editor al que se accede por la ventana de propiedades permite elegir una máscara existente o configurar su propia máscara.

Máscara de entrada

Seleccione una descripción de máscara predefinida de la siguiente lista o seleccione Personalizada para definir una máscara personalizada.

Descripción de la máscara	Formato de fecha	Tipo de validación
Fecha	26/10/2005	DateTime
Fecha y hora	26/10/2005 14:30	DateTime
Hora	08:30	DateTime
Númérico	12345	Int32
Número de teléfono	(012)345-6789	(ninguno)
Número de teléfono móvil	000-123-4567	(ninguno)
Número de teléfono sin c...	123-4567	(ninguno)
Número del seguro social	123-45-6789	(ninguno)
<Personalizada>		(ninguno)

Máscara: 000-000-0000 ☒ Utilizar ValidatingType

Vista previa: _-_-_-

Aceptar Cancelar

Para la propiedad `Mask`, algunos caracteres tienen un significado particular:

0 representa una cifra obligatoria (0 a 9).

9 representa una cifra o un espacio en opción.

L representa una letra obligatoria (de la a la z o de la A a la Z).

? representa una letra opcional.

C representa un carácter cualquiera.

. representa el separador decimal.

, representa el separador de los miles.

: representa el separador horario.

/ representa el separador de fecha.

\$ representa el símbolo monetario.

< los siguientes caracteres se transformarán en minúsculas.

> los siguientes caracteres se transformarán en mayúsculas.

| cancela el efecto de dos caracteres > y <.

\ carácter de escape que hace perder su significado especial al siguiente carácter.

Todos los demás caracteres se visualizan como en el control.

La siguiente máscara, por ejemplo, puede utilizarse para la introducción de una dirección IP:

000\ .000\ .000\ .000

c. El control RichTextBox

El control `RichTextBox` permite la visualización, la introducción y el manejo de texto con configuración del formato. Posee las mismas funcionalidades que el control `TextBox`, pero es capaz de gestionar fuentes de caracteres diferentes, colores diferentes, imágenes, etc. Propone en realidad todas las funciones básicas de una aplicación de tratamiento de texto. Por lo tanto, vamos a detallar estas principales funciones.

Cargar y guardar un archivo

Los métodos `LoadFile` y `SaveFile` permiten la carga y el volcado desde un archivo o hacia él. El único parámetro obligatorio para estas dos funciones representa la ruta de acceso completa hacia el archivo que hay que cargar o guardar. El formato de archivo utilizado por defecto para estas dos funciones es el formato `rtf` (*Rich Text Format*). Si hubiera otros formatos que utilizar, deberíamos especificarlo con un segundo parámetro que es una constante de la enumeración `RichTextBoxStreamType`. En el caso de una lectura de archivo es importante que los datos contenidos en el archivo concuerden con la constante utilizada.

Por ejemplo, la lectura de un archivo de texto normal con la línea de código siguiente activará una excepción.

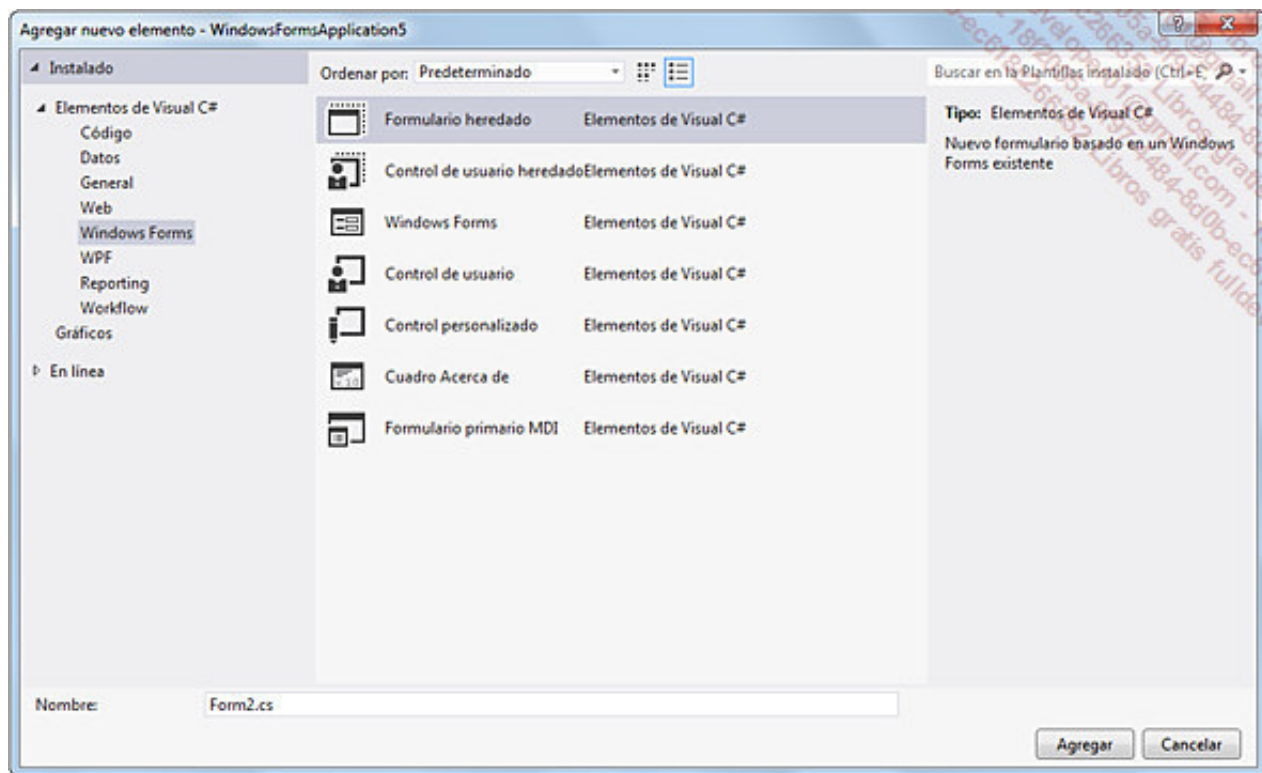
```
rtb.LoadFile(dlgAbrir.FileName, RichTextBoxStreamType.RichText);
```

En cambio, no hay problema para guardar, ya que es el control `RichTextBox` el que gestiona el formato de los datos incluidos en el archivo. El único riesgo co

La herencia de formularios

A veces puede necesitar que un proyecto llame a un formulario similar a otro que ya ha creado en otro proyecto. Además, puede crear un formulario básico que contenga parámetros tales como un segundo plano estático o una presentación particular de los controles que piensa utilizar varias veces en un proyecto, ya que cada nueva versión contiene modificaciones respecto al modelo original. La herencia de formularios le permite crear un formulario básico y luego heredarlo para personalizar las nuevas versiones así creadas.

Para poder crear un formulario heredado previamente, hace falta diseñar el formulario básico. Para que la herencia de formulario esté accesible, el proyecto que contiene el formulario básico debe haber sido compilado obligatoriamente. La inserción de un formulario heredado se realiza mediante el cuadro de diálogo clásico de inserción de elementos en un proyecto seleccionando la opción **Formulario heredado**.



A continuación, dele un nombre a su nuevo formulario y haga clic en el botón **Agregar**. El cuadro de diálogo **Selector de herencia** se abre y, si el proyecto actual ya contiene formularios, se muestran en este cuadro de diálogo. Para heredar de un formulario disponible en otro ensamblado, haga clic en el botón **Examinar** y seleccione el archivo (.exe o .dll) que contiene el formulario básico, y luego valide su elección con el botón **Aceptar**. Así el nuevo formulario se añade a su proyecto. En este formulario, los controles heredados vienen marcados por el símbolo .

La propiedad `Modifiers` de cada control del formulario básico determina las posibles acciones sobre estos controles en un formulario heredado. Se aplican las reglas estándar de la herencia. A continuación se resumen estas reglas de visibilidad:

- **Public:** se pueden redimensionar y desplazar los controles. El control es accesible internamente por la clase que lo declara y externamente por las demás clases.
- **Protected:** se pueden redimensionar y desplazar los controles. El control es accesible internamente por la clase que lo declara y por cualquier clase que hereda de la clase madre. Pero no es accesible por clases externas.
- **Protected Internal:** se pueden redimensionar y desplazar los controles. El control es accesible

internamente por la clase que lo declara y por cualquier clase que hereda de la clase madre, y por otros miembros del ensamblado que los contiene.

- **Internal:** todos los aspectos del control se consideran accesibles en modo de sólo lectura. No los podrá desplazar ni redimensionar ni modificar sus propiedades. El control es accesible únicamente por otros miembros del ensamblado que lo contiene.
- **Private:** todos los aspectos del control son considerados accesibles en modo de sólo lectura. No los podrá desplazar ni redimensionar ni modificar sus propiedades. El control sólo es accesible desde la clase que lo declara.

Por supuesto, se pueden añadir otros controles al formulario heredado para personalizar su aspecto. Si se modifica el formulario básico después de haber establecido una relación de herencia, las modificaciones se propagan a los formularios heredados durante la compilación del formulario básico.

Principio del funcionamiento de una base de datos

Las bases de datos se han convertido en elementos ineludibles en la mayoría de las aplicaciones. Sustituyen la utilización de archivos gestionados por el propio desarrollador. Esta aportación permite ganar mucha productividad durante el desarrollo, y mejora de manera significativa las prestaciones de las aplicaciones. También facilitan compartir información entre usuarios. Para poder utilizar una base de datos, debe familiarizarse mínimamente con el vocabulario relacionado con esta tecnología.

1. Terminología

En el contexto de las bases de datos, los siguientes términos se utilizan muy a menudo:

Base de datos relacional

Una base de datos relacional es un tipo de base de datos que utiliza tablas para el almacenamiento de la información. Usa valores procedentes de dos tablas para asociar los datos de una tabla a los datos de otra. Por regla general, en una base de datos relacional, se almacena la información sólo una vez.

Tabla

Una tabla es un componente de una base de datos que almacena la información en registros (filas) y campos (columnas). En general la información de una base de datos se agrupa a nivel de tabla. Por ejemplo, tenemos la tabla de los Clientes, de los Productos o de los Pedidos.

Registro

El registro es el conjunto de la información relativa a un elemento de una tabla. Los registros son los equivalentes a nivel lógico de las filas de una tabla. Por ejemplo, un registro de la tabla **Clientes** contiene las características de un cliente particular.

Campo

Un registro se compone de varios campos. Cada campo de un registro contiene una sola información relativa al registro. Por ejemplo, un registro Cliente puede contener los campos **CodigoCliente**, **Apellido**, **Nombre**...

Clave primaria

Una clave primaria se utiliza para identificar de manera única cada fila de una tabla. La clave primaria es un campo o una combinación de campos cuyo valor es único en la tabla. Por ejemplo, el campo **CodigoCliente** es la clave primaria de la tabla **Cliente**. No puede haber dos clientes con el mismo código.

Clave foránea

Una clave foránea representa uno o varios campos de una tabla que hacen referencia a los campos de la clave primaria de otra tabla. Las claves foráneas indican la manera según la cual se relacionan las tablas.

Relación

Una relación es una asociación establecida entre campos comunes en dos tablas. Una relación puede ser de uno a uno, de uno a varios o de varios a varios. Gracias a las relaciones, los resultados de consultas pueden contener datos provenientes de varias tablas. Una relación de uno a varios entre la tabla **Cliente** y la tabla **Pedido** permite a una consulta devolver todos los pedidos correspondientes a un cliente.

2. El lenguaje SQL

Antes de poder escribir una aplicación Visual C# que utiliza datos, se debe familiarizar con el lenguaje SQL (*Structured Query Language*). Este lenguaje permite dialogar con la base de datos. Existen diferentes versiones del lenguaje SQL según la base de datos utilizada. Sin embargo, SQL dispone también de una sintaxis básica, normalizada e independiente de todas las bases de datos.

a. Búsqueda de información

El lenguaje SQL permite especificar los registros que hay que extraer, así como el orden en el cual desea extraerlos. Puede crear una instrucción SQL que extraiga información de varias tablas simultáneamente, o crear una instrucción que extraiga únicamente un registro específico.

La instrucción `SELECT` se utiliza para devolver campos específicos de una o varias tablas de la base de datos.

La siguiente instrucción devuelve la lista de los apellidos y nombres de todos los registros de la tabla **Cliente**:

```
SELECT Apellido,Nombre FROM Cliente
```

Puede utilizar el símbolo `*` en lugar de la lista de los campos para los cuales desea el valor:

```
SELECT * FROM Cliente
```

Puede limitar el número de registros seleccionados utilizando uno o varios campos para filtrar el resultado de la consulta. Hay diferentes cláusulas disponibles para ejecutar este filtro.

Cláusula WHERE

Esta cláusula permite especificar la lista de las condiciones que tienen que cumplir los registros para formar parte de los resultados devueltos. El siguiente ejemplo permite encontrar todos los clientes habitantes de Barcelona:

```
SELECT * FROM Cliente WHERE Ciudad='Barcelona'
```



La sintaxis de esta cláusula requiere la utilización de comillas simples para la delimitación de las cadenas de caracteres.

Cláusula WHERE ... IN

Puede utilizar la cláusula `WHERE ... IN` para devolver todos los registros que cumplen con una lista de criterios. Por ejemplo, puede buscar todos los clientes que viven en Francia o en España:

```
SELECT * FROM Cliente WHERE Pais IN ('Francia','España')
```

Cláusula WHERE ... BETWEEN

También puede devolver una selección de registros que se sitúan entre dos criterios especificados. La siguiente consulta permite recuperar la lista de los pedidos pasados en el mes de noviembre del 2005:

```
SELECT * from Pedidos WHERE FechaPedido BETWEEN '01/11/05' AND '30/11/05'
```

Cláusula WHERE ... LIKE

Puede utilizar la cláusula WHERE ... LIKE para devolver todos los registros en los que existe una condición particular para un campo dado. Por ejemplo, la siguiente sintaxis selecciona todos los clientes cuyo apellido empieza con una d:

```
SELECT * FROM Cliente WHERE Apellido LIKE 'd%'
```



En esta instrucción, el símbolo % se utiliza para reemplazar una secuencia de caracteres cualquiera.

Cláusula ORDER BY ...

Puede utilizar la cláusula ORDER BY para devolver los registros en un orden particular. La opción ASC indica un orden ascendente, la opción DESC indica un orden descendente. Varios campos se pueden especificar como criterio de ordenación. Se analizan desde la izquierda hacia la derecha. En caso de igualdad en el valor de un campo, se utiliza el siguiente campo:

```
SELECT * FROM Cliente ORDER BY Apellido DESC,Nombre ASC
```



Esta instrucción devuelve los clientes ordenados de forma descendente según el apellido, y en caso de igualdad, por orden ascendente según el nombre.

b. Añadir información

La creación de registros en una tabla se efectúa por el comando INSERT INTO. Usted debe indicar la tabla en la cual desea insertar una fila, la lista de los campos para los cuales especifica un valor y, para terminar, la lista de los valores correspondientes. Por lo tanto, la sintaxis completa es la siguiente:

```
INSERT INTO cliente (codigoCliente,apellido,nombre) VALUES (1000,'García','Pedro')
```

Durante la adición de este nuevo cliente, sólo el apellido y el nombre están indicados en la tabla. Los otros campos tomarán el valor **NULL**. Si la lista de los campos no está indicada, la instrucción insert exige que usted especifique un valor para todos los campos de la tabla. Por lo tanto, está obligado utilizar la palabra clave **NULL** para indicar que, para un campo particular, no hay información. Si la tabla **Cliente** está compuesta por cinco campos (codigoCliente, apellido, nombre, direccion, pais), la instrucción anterior se puede escribir con la siguiente sintaxis:

```
INSERT INTO cliente VALUES (1000,'García','Pedro',NULL,NULL)
```



En este caso, las dos palabras claves NULL son obligatorias para los campos dirección y país.

c. Actualizar información

La modificación de los campos para registros existentes se efectúa con la instrucción `UPDATE`. Esta instrucción puede actualizar varios campos de varios registros de una tabla a partir de las expresiones que se le facilitan. Para ello, debe facilitar el nombre de la tabla que se debe actualizar, así como el valor que hay que asignar a los diferentes campos. La lista se indica con la palabra clave **SET** seguidas de la asignación del nuevo valor a los diferentes campos. Si desea que las modificaciones sólo afecten a un conjunto limitado de registros, debe especificar la cláusula `WHERE`, con el fin de limitar el alcance de la actualización. Si no se indica ninguna cláusula `WHERE`, la modificación se hará sobre el conjunto de los registros de la tabla.

Por ejemplo, para modificar la dirección de un cliente particular, puede utilizar la siguiente instrucción:

```
UPDATE Cliente SET direccion= 'calle de Madrid, 08000 Barcelona' WHERE  
codigoCliente=1000
```

Si la sentencia debe modificar todos los registros de una tabla, la cláusula `WHERE` es superflua. Por ejemplo, si desea aumentar el precio unitario de todos sus artículos, puede utilizar la siguiente instrucción:

```
UPDATE CATALOGO SET precioUnitario=precioUnitario*1.1
```

d. Suprimir información

La instrucción `DELETE FROM` permite suprimir uno o varios registros de una tabla. Como mínimo, debe facilitar el nombre de la tabla en la cual se va a efectuar la supresión. Si no indica más precisiones, todas las filas de la tabla se suprimirán. En general, se añade una cláusula `WHERE` para limitar la extensión de la supresión. La siguiente sentencia borra todos los registros de la tabla **Cliente**:

```
DELETE FROM Cliente
```

El siguiente comando es menos radical y sólo suprime un registro particular:

```
DELETE FROM Cliente WHERE codigoCliente=1000
```

Por supuesto, el lenguaje SQL es mucho más completo que eso, y no se limita a estas cinco instrucciones. Sin embargo, son suficientes para el manejo de datos a partir de Visual C#. Si desea profundizar en el aprendizaje del lenguaje SQL, consulte uno de los libros disponibles en esta misma colección que tratan este tema de manera más avanzada.

Presentación de ADO.NET

ADO.NET es un conjunto de clases, interfaces, estructuras y enumeraciones que permiten el manejo de los datos. Los diferentes componentes de ADO.NET permiten separar el acceso a los datos de su manejo. ADO.NET facilita también la utilización del lenguaje XML, al permitir la conversión de datos relacionales al formato XML o la importación de datos a los formatos XML en un modelo relacional. Hay dos modos de funcionamiento disponibles en ADO.NET:

- el modo conectado;
- el modo no conectado.

1. Modo conectado

En un entorno conectado, la aplicación o el usuario está permanentemente conectado a la fuente de datos. Desde los principios de la informática, ha sido el único modo disponible. Este modo presenta algunas ventajas en su funcionamiento:

- Es fácil de gestionar: la conexión se hace al principio de la aplicación y luego se corta al cierre de ésta.
- El acceso concurrente es más fácil de controlar: como todos los usuarios están conectados de forma permanente, es más fácil controlar cuál trabaja con los datos.
- Los datos están actualizados: siempre gracias a la conexión permanente a los datos, es fácil avisar a todas las aplicaciones que utilizan los datos de que se acaban de producir algunas modificaciones.

Por el contrario, ciertos inconvenientes vienen a oscurecer el panorama:

- Se debe mantener la conexión de red constantemente: en caso de utilización de la aplicación en un ordenador portátil, el acceso a la red se arriesga a no estar disponible permanentemente.
- Se corre el riesgo de infrautilizar los recursos del servidor: en el momento de establecer una conexión entre una aplicación cliente y un servidor, se reservan recursos del servidor para la gestión de esta conexión. Estos recursos siguen monopolizados por la conexión, incluso aunque ninguna información transite por ella.

Sin embargo, en ciertas situaciones, la utilización de un modo conectado es ineludible. Es el caso, por ejemplo, de las aplicaciones que realizan procesos en tiempo real.

2. Modo no conectado

Un modo no conectado significa que una aplicación o un usuario no está conectado constantemente a una fuente de datos. Las aplicaciones de Internet utilizan a menudo este modo de funcionamiento. Se abre la conexión a los datos, se obtienen los datos y luego se cierra la conexión. El usuario trabaja con los datos a partir de su navegador, y se reabre la conexión para la actualización de la fuente de datos o la obtención de otros datos. Los usuarios que trabajan en ordenadores portátiles también son los principales usuarios de entornos desconectados. Un médico, por ejemplo, puede cargar por la mañana los historiales de salud de los pacientes que va a visitar durante el día, luego, por la tarde, actualizar las modificaciones en la base de datos. Las ventajas de un entorno no conectado son las siguientes:

- Se utilizan las conexiones durante la duración más corta posible. De esta manera, un pequeño número de conexiones disponibles en un servidor bastan para muchos usuarios.
- Un entorno desconectado mejora la escalabilidad y las prestaciones de una aplicación al

optimizar la disponibilidad de las conexiones.

Sin embargo, el entorno desconectado comporta algunos inconvenientes:

- Los datos disponibles en la aplicación no siempre están actualizados. Por ejemplo, en el caso de nuestro médico, si su secretaria añade resultados de análisis después de que él haya recuperado los historiales médicos de estos pacientes, no podrá disponer inmediatamente de la información.
- En ocasiones pueden surgir conflictos durante la actualización de la información en la base. Durante el desarrollo de la aplicación, se debe asumir este tipo de problemas. Hay diferentes planteamientos disponibles para la gestión de estos conflictos:
 - Autorizar la prevalencia de las actualizaciones más recientes, sobrescribiendo los datos ya presentes en la base.
 - Autorizar la prevalencia de las actualizaciones más antiguas rechazando las nuevas actualizaciones.
 - Prever código que permite al usuario elegir lo que desea hacer en caso de conflicto durante la actualización.

3. Arquitectura de ADO.NET

La meta de ADO.NET consiste en facilitar un conjunto de clases que permite el acceso a las bases de datos. Hay dos tipos de componentes disponibles:

- Los proveedores de datos específicos a un tipo de base de datos. Aseguran la comunicación con un tipo específico de base de datos y permiten el trabajo con los datos directamente en la base en modo conectado. Sin embargo, las posibilidades son limitadas, ya que sólo se dispone de un acceso en modo lectura.
- Las clases de manejo de los datos, independientes del tipo de base de datos, incluso utilizables sin base de datos, permiten el manejo local de los datos en la aplicación.

4. Los proveedores de datos

Los proveedores de datos sirven de pasarela entre una aplicación y una base de datos. Se utilizan para recuperar la información a partir de la base de datos y transferir los cambios efectuados en los datos por la aplicación hacia la base de datos. Hay cuatro proveedores de datos disponibles en el Framework .NET:

- el proveedor para SQL Server;
- el proveedor para OLE DB;
- el proveedor para ODBC;
- el proveedor para Oracle.

Todos proponen la implementación de cuatro clases, básicas, necesarias para el diálogo con la base de datos:

- La clase **Connection** permite establecer una conexión con el servidor de base de datos.
- La clase **Command** permite pedir la ejecución de una instrucción o de un conjunto de instrucciones de SQL a un servidor.
- La clase **DataReader** facilita un acceso a los datos sólo en modo lectura. Al igual que en el

caso de los archivos, este acceso es sólo secuencial y, por lo tanto, el conjunto de datos es recorrido sólo una vez y de atrás adelante.

- La clase **DataAdapter** se utiliza para asegurar la transferencia de los datos hacia un sistema de caché local a la aplicación llamado `DataSet` y para actualizar la base de datos, en función de las modificaciones efectuadas localmente en el `DataSet`.

Hay otras clases que están especializadas en la gestión de las transacciones o el paso de parámetros a una instrucción SQL.

a. SQL Server

El proveedor de datos para SQL Server utiliza un protocolo nativo para dialogar con el servidor de base de datos. Además, como accede al servidor sin hacer uso de capas de software adicional (OLE DB u ODBC), consume muy pocos recursos. Se puede utilizar con SQL Server a partir de la versión 7. Todas las clases de este proveedor de datos están disponibles en el espacio de nombres **System.Data.SqlClient**. En este espacio de nombres, el nombre de cada clase viene prefijado por `Sql`. Así, la clase que permite conectarse a un servidor SQL Server se llama `SqlConnection`.

b. OLE DB

El proveedor OLE DB utiliza la capa de software OLE DB para comunicarse con el servidor de base de datos. Puede utilizar este proveedor para dialogar con una base de datos que no dispone de proveedores específicos, pero que cuenta con compatibilidad OLE DB. Con esta solución, el proveedor no contacta con el servidor directamente, sino que usa un driver OLE DB para comunicarse. Para que esta comunicación sea posible, el driver debe implementar algunas interfaces. Todas las clases están disponibles en el espacio de nombres **System.Data.OleDb**. Los nombres de clase de este espacio de nombres vienen prefijados con `OleDb`. Para poder funcionar correctamente, este proveedor exige la instalación de MDAC 2.6 en la máquina (*Microsoft Data Access Components*) o una versión posterior.

c. ODBC

El proveedor ODBC utiliza un driver ODBC nativo para comunicarse con el servidor de base de datos. Este proveedor utiliza un driver ODBC nativo para la comunicación. El principio es idéntico al utilizado para el proveedor OLE DB. Todas las clases están disponibles en el espacio de nombres **System.Data.Odbc**. Los nombres de clases vienen prefijados con `Odbc`. Para poder funcionar correctamente, este proveedor exige la instalación de MDAC 2.6 en la máquina (*Microsoft Data Access Components*) o una versión posterior.

d. Oracle

El proveedor para Oracle permite la conexión a una fuente de datos Oracle. Las clases están localizadas en el espacio de nombres **System.Data.OracleClient** y utilizan Oracle como prefijo de nombre.

5. Buscar los proveedores disponibles

Para asegurar el buen funcionamiento de una aplicación que utiliza un acceso a los datos, los proveedores deben estar instalados en el puesto cliente. La clase `DbProviderFactories` propone el método compartido `GetFactoryClasses`, que permite enumerar los proveedores de datos disponibles en el puesto. El ejemplo de código siguiente muestra el nombre, la descripción y el espacio de nombres raíz de cada uno de los proveedores instalados en el puesto de trabajo.

```

public static void listaProveedores()
{
    DataTable resultado;
    //recuperacion de la lista de los proveedores en una dataTable
    resultado = DbProviderFactories.GetFactoryClasses();
    //recorrido de las columnas de la dataTable y visualización
del nombre
    foreach (DataColumn columna in resultado.Columns)
    {
        Console.Write(columna.ColumnName + "\t");
    }
    Console.WriteLine();
    // recorrido de la dataTable y visualización de cada fila
    foreach (DataRow linea in resultado.Rows)
    {
        // recorrido de cada fila y visualización de cada campo
        foreach (DataColumn columna in resultado.Columns)
        {
            Console.Write(linea[columna.ColumnName] + "\t");
        }
        Console.WriteLine();
    }
    Console.ReadLine();
}

```

6. Compatibilidad del código

En función del proveedor utilizado, debe importar el espacio de nombres correspondiente para tener un acceso fácil a las clases del proveedor. Sin embargo, como las clases de cada uno de los proveedores no llevan el mismo nombre, su código será específico para un tipo de proveedor. Sin embargo, es posible escribir código prácticamente independiente del tipo de proveedor. Para ello, en vez de utilizar las clases específicas a cada uno de los proveedores, puede utilizar como tipo de datos las interfaces que implementan. La utilización de una clase específica sólo es indispensable para la creación de la conexión. Una vez creada la conexión, puede trabajar únicamente con interfaces. El siguiente ejemplo de código hace la lista del contenido de una tabla de una base SQL Server usando únicamente interfaces.

```

public static void accesParInterfaces()
{
    IDbConnection ctn;
    ctn=new SqlConnection();
    ctn=new SqlConnection("Data Source=localhost;Initial Catalog=
Northwind;Integrated Security=True");
    IDbCommand cmd;
    cmd=ctn.CreateCommand();
    ctn.Open();
    cmd.CommandText="select * from products";
    IDataReader lector;
    lector=cmd.ExecuteReader();
    Console.WriteLine("lectura de los datos en una base SQL server");
    while (lector.Read())
    {
        Console.WriteLine("numero : {0} nombre producto: {1}",
lector.GetInt32(0),lector.GetString(1));
    }
}

```

La ejecución de este código muestra el siguiente resultado:

```
lectura de los datos en una base SQL server
numero: 1 nombre producto: Chai
numero: 2 nombre producto: Chang
numero: 3 nombre producto: Aniseed Syrup
numero: 4 nombre producto: Chef Anton's Cajun Seasoning
numero: 5 nombre producto: Chef Anton's Gumbo Mix
numero: 6 nombre producto: Grandma's Boysenberry Spread
numero: 7 nombre producto: Uncle Bob's Organic Dried Pears
numero: 8 nombre producto: Northwoods Cranberry Sauce
numero: 9 nombre producto: Mishi Kobe Niku
numero: 10 nombre producto: Ikura
numero: 11 nombre producto: Queso Cabrales
numero: 12 nombre producto: Queso Manchego La Pastora
numero: 13 nombre producto: Konbu
numero: 14 nombre producto: Tofu
numero: 15 nombre producto: Genen Shouyu
numero: 16 nombre producto: Pavlova
```

Si esta aplicación debe migrar luego hacia otro tipo de base de datos, sólo hay que modificar la fila relativa a la conexión. Si ahora los datos están disponibles en una base de Access, la creación de la conexión toma la siguiente forma:

```
ctn = new OleDbConnection("Provider=Microsoft.Jet.OLEDB.4.0;
Data Source=C:\\Documents and Settings\\tgroussard\\
Mis documentos\\libro c sharp 2008\\capítulo 8\\NWIND.mdb");
```

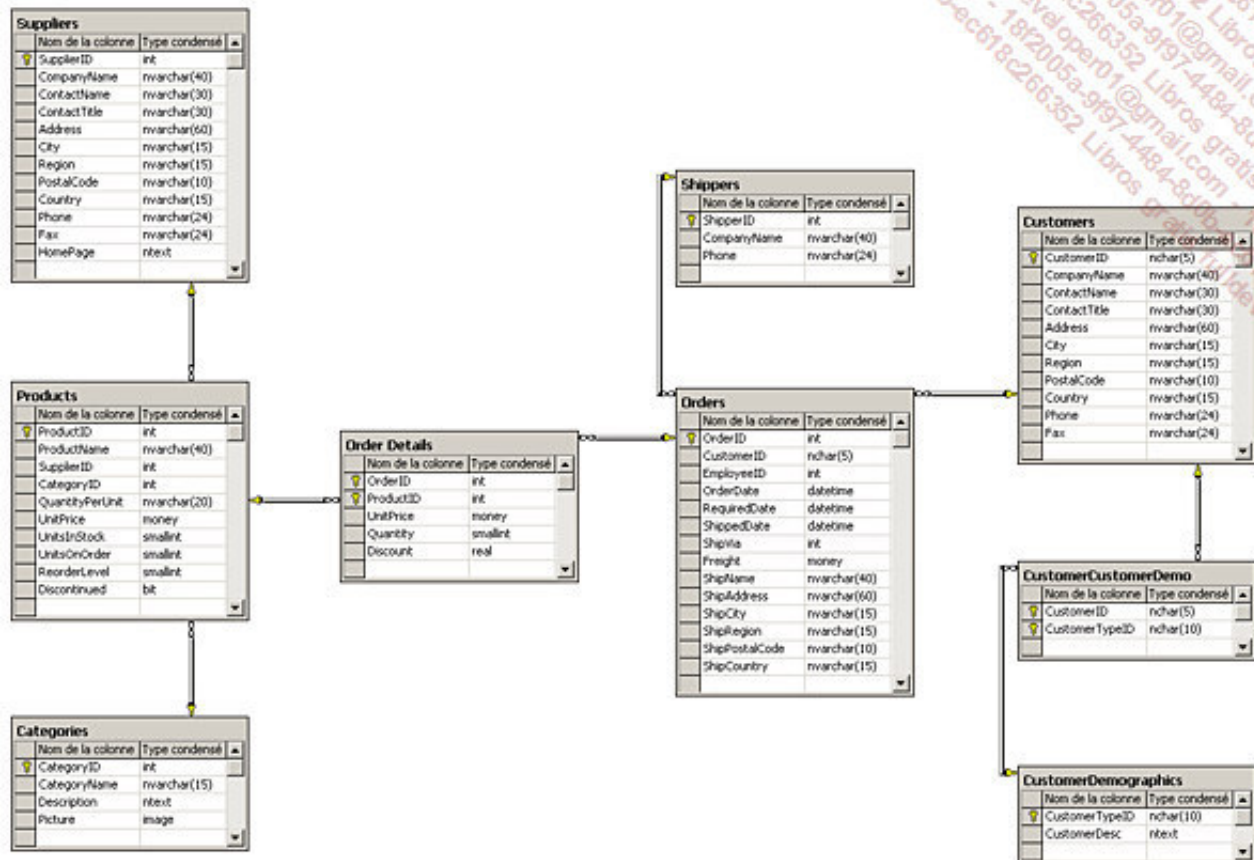
La ejecución del código así modificado genera, efectivamente, el mismo resultado:

```
lectura de los datos en una base SQL server
numero: 1 nombre producto: Chai
numero: 2 nombre producto: Chang
numero: 3 nombre producto: Aniseed Syrup
numero: 4 nombre producto: Chef Anton's Cajun Seasoning
numero: 5 nombre producto: Chef Anton's Gumbo Mix
numero: 6 nombre producto: Grandma's Boysenberry Spread
numero: 7 nombre producto: Uncle Bob's Organic Dried Pears
numero: 8 nombre producto: Northwoods Cranberry Sauce
numero: 9 nombre producto: Mishi Kobe Niku
numero: 10 nombre producto: Ikura
numero: 11 nombre producto: Queso Cabrales
numero: 12 nombre producto: Queso Manchego La Pastora
numero: 13 nombre producto: Konbu
numero: 14 nombre producto: Tofu
numero: 15 nombre producto: Genen Shouyu
numero: 16 nombre producto: Pavlova
```

Por el contrario, conviene ser prudente y no utilizar instrucciones SQL específicas a un tipo de base de datos particular. Para facilitar la corrección del código, es preferible agrupar todas las instrucciones SQL en forma de constantes de tipo cadena de caracteres al principio de cada módulo. Con esta técnica, no tendrá que buscar instrucciones SQL en mitad de centenas de filas de código de Visual C#. También conviene ser prudente durante la utilización de parámetros en una instrucción SQL. El proveedor para SQL Server utiliza parámetros con nombre; por lo tanto el orden de creación de los parámetros no tiene importancia. El proveedor para OLE DB utiliza la posición de los parámetros en la instrucción SQL para los reemplazos durante la ejecución. El orden de la creación de los parámetros es, pues, en este caso, capital para el funcionamiento correcto de la instrucción

Utilización del modo conectado

En este capítulo, vamos a tratar las operaciones que pueden ejecutarse en una base de datos usando el modo conectado. Ciertas nociones estudiadas en este capítulo también serán útiles para el funcionamiento en modo desconectado. Para probar las diferentes funcionalidades estudiadas en este capítulo, utilizaremos un servidor SQL Server 2008. La base de datos usada será la base Northwind, que se puede instalar gracias al script instrund.sql disponible en los archivos que se descargan. El siguiente esquema muestra una parte de la estructura de la base.



1. Conexión a una base

Para poder trabajar con un servidor de base de datos, una aplicación debe establecer una conexión de red con el servidor. La clase `SqlConnection` es capaz de gestionar una conexión hacia un servidor SQL Server versión 7.0 o posterior. Como para cualquier objeto, en primer lugar debemos declarar una variable.

```
SqlConnection ctn;
```

Luego, debemos crear la instancia de la clase e inicializarla llamando a un constructor. La inicialización va a consistir esencialmente en indicar los parámetros utilizados para establecer la conexión con el servidor. Estos parámetros se definen en forma de cadena de caracteres. Pueden ser indicados durante la llamada del constructor o modificados luego por la propiedad `ConnectionString`.

a. Cadena de conexión

El formato estándar de una cadena de conexión está constituido por una serie de pares clave/valor

separados con punto y coma. El signo = se usa para la asignación de un valor a una palabra clave. El análisis de la cadena se efectúa durante la asignación de la cadena a la propiedad `ConnectionString`. A continuación se extraen los valores asociados a las palabras clave y se asignan las diferentes propiedades de la conexión. Si se encuentra un error de sintaxis, entonces se genera una excepción inmediatamente y no se modifica ninguna propiedad. Por el contrario, sólo se pondrán controlar algunas propiedades durante la apertura de la conexión. Sólo en este momento se activará una excepción si la cadena de conexión contiene un error. Únicamente se puede modificar la cadena de conexión si la conexión está cerrada. Las siguientes palabras están disponibles para una cadena de conexión:

Connect Timeout

Tiempo en segundos durante el cual la aplicación esperará una respuesta del servidor a su petición de conexión. Pasado este plazo, se activa una excepción.

Data Source

Nombre o dirección de red del servidor hacia el cual se establece la conexión. El número del puerto se puede especificar después del nombre o de la dirección de red. Si no está indicado, el número de puerto es igual a 1433.

Initial Catalog

Nombre de la base de datos a la que se desea conectar.

Integrated Security

Si este valor es **false**, entonces se debe facilitar un nombre de usuario y una contraseña en la cadena de conexión. Si es **true**, la cuenta local Windows del usuario se usará como autenticación.

Persist Security Info

Si se coloca este valor en **true**, entonces el nombre del usuario y su contraseña serán accesibles por la conexión. Por razones de seguridad, se debe colocar este valor en **false**. De hecho, es el caso si usted no indica nada en su cadena de conexión.

Pwd

Contraseña asociada a la cuenta SQL Server utilizada para la conexión. Si no existe contraseña asociada a una cuenta, se puede omitir esta información en la cadena de conexión.

User ID

Nombre de la cuenta SQL Server utilizada para la conexión.

Connection LifeTime

Indica la duración de vida de una conexión en un pool de conexiones. Un valor igual a cero indica una duración ilimitada.

Connection Reset

Indica si la conexión se reinicializa al ser devuelta al pool.

Max Pool Size

Número máximo de conexiones en el pool.

Min Pool Size

Número mínimo de conexiones en el pool.

Pooling

Indica si es posible extraer una conexión de un pool de conexiones.

Una cadena de conexión toma así la forma mínima siguiente:

```
ctn.ConnectionString= "Data Source=localhost;Initial Catalog=Northwind;  
Integrated Security=true";
```

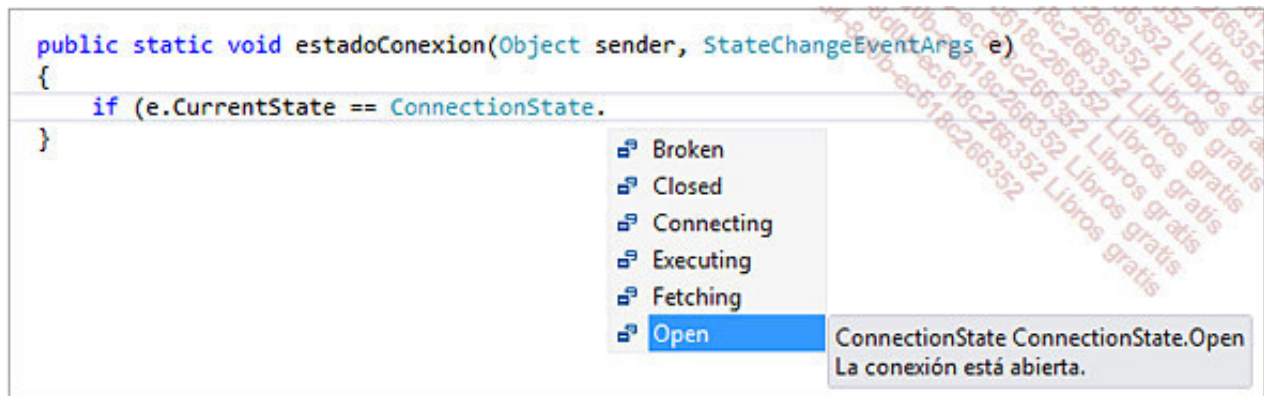
b. Pool de conexiones

Los pools de conexiones permiten mejorar las prestaciones de una aplicación evitando la creación de conexiones adicionales. Cuando una conexión está abierta, se crea un pool de conexiones utilizando un algoritmo basado en la cadena de conexión. Así, cada pool está asociado a una cadena de conexión particular. Si se abre una nueva conexión y no existe pool que corresponda exactamente a su cadena de conexión, entonces se crea un nuevo pool. Los pools de conexiones así creados existirán hasta el final de la aplicación. Durante la creación del pool, otras conexiones pueden crearse automáticamente para satisfacer el valor `Min Pool Size` indicado en la cadena de conexión. Se podrán añadir otras conexiones al pool hasta alcanzar el valor `Max Pool Size` de la cadena de conexión. Cuando se requiere una conexión, ésta se puede obtener desde un pool de conexión (si existe uno que corresponda exactamente a las características de la conexión requerida). Por supuesto, hace falta que el pool contenga una disponible y activa.

Si se alcanza el número máximo de conexión en el pool, la petición se colocará en cola hasta que haya una conexión libre. Se devuelve una conexión al pool tras su cierre o invocando el método `Dispose` sobre la conexión. Por esta razón, se recomienda cerrar explícitamente las conexiones cuando ya no se utilizan en la aplicación. Se retira una conexión del pool cuando el sistema detecta que no hay más conexiones desde hace un cierto tiempo, indicado por el valor `ConnectionLifeTime` de la cadena de conexión. También se retira del pool si detecta que la conexión con el servidor se ha interrumpido.

c. Eventos de conexión

La clase `SqlConnection` propone dos eventos que le permiten recibir una advertencia cuando el estado de la conexión cambia o cuando un mensaje de información se envía a través del servidor. El evento `StateChanged` se activa durante un cambio de estado de la conexión. El gestor de este evento recibe un parámetro de tipo `StateChangeEventArgs`, que permite obtener, con la propiedad `CurrentState`, el estado actual de la conexión, y con la propiedad `OriginalState`, el estado de la conexión antes de la desactivación del evento. Para probar el valor de estas dos propiedades, puede utilizar la enumeración `ConnectionState`.



El evento `InfoMessage` se activa cuando el servidor le informa de una situación anormal, pero que no justifica la activación de una excepción (gravedad del mensaje inferior a 10). El gestor de eventos asociado recibe un parámetro de tipo `InfoMessageEventArgs`. Por la propiedad `Errors` de este parámetro, tiene acceso a objetos `SqlErrors` que corresponden a la información enviada por el servidor. El siguiente código muestra en la consola los mensajes de información que provienen del servidor.

```
public static void ctn_InfoMessage(object sender,
System.Data.SqlClient.SqlInfoMessageEventArgs e)
{
    foreach ( SqlError info in e.Errors)
    {
        Console.WriteLine(info.Message);
    }
}
```

2. Ejecución de un comando

Después de haber establecido una conexión hacia un servidor de base de datos, usted le puede transmitir instrucciones SQL. Se utiliza la clase `SqlCommand` para pedir al servidor la ejecución de un comando SQL. Esta clase contiene varios métodos que permiten la ejecución de diferentes tipos de consultas SQL. Se puede instanciar la clase `SqlCommand` de manera clásica usando uno de sus constructores o se puede obtener una instancia con el método `CreateCommand` de la conexión.

a. Creación de un comando

La primera posibilidad para crear un `SqlCommand` consiste en utilizar uno de los constructores de la clase. La utilización del constructor por defecto le obliga a utilizar luego diferentes propiedades para facilitar la información relativa a la instrucción SQL que se ha de ejecutar.

La propiedad `CommandText` contiene el texto de la instrucción SQL que se ha de ejecutar. La propiedad `Connection` debe hacer referencia a una conexión válida hacia el servidor de base de datos. El siguiente código resume estas operaciones:

```
SqlCommand cmd;
cmd = new SqlCommand();
cmd.Connection = ctn;
cmd.CommandText = "select * from products";
```

La segunda solución consiste en utilizar un constructor sobrecargado aceptando como parámetros la instrucción SQL con la forma de una cadena de caracteres y la conexión utilizada por

esta `SqlCommand`. Así se puede resumir el código anterior a la siguiente fila:

```
cmd = new SqlCommand("select * from products", ctn);
```

La tercera solución consiste utilizar el método `CreateCommand` de la conexión. En este caso, sólo se tiene que especificar a continuación la instrucción SQL con la propiedad `CommandText`.

```
SqlCommand cmd;  
cmd = ctn.CreateCommand();  
cmd.CommandText = "select * from products";
```

b. Lectura de información

A menudo, la instrucción SQL de un `SqlCommand` selecciona un conjunto de registros en la base, o eventualmente un valor único que es resultado de un cálculo efectuado sobre valores contenidos en la base. Una instrucción SQL, que devuelve un conjunto de registros, debe ser ejecutada por el método `ExecuteReader`. Este método devuelve un objeto `DataReader` que va a permitir luego la lectura de la información que proviene de la base de datos. Si la instrucción sólo devuelve un valor único, el método `ExecuteScalar` se encarga de la ejecución y devuelve él mismo el valor que proviene de la base de datos.

El siguiente código permite la recuperación del número de pedidos que ha hecho un cliente:

```
SqlConnection ctn;  
SqlCommand cmd;  
ctn = new SqlConnection();  
ctn = new SqlConnection("Data Source=localhost;Initial  
                        Catalog=Northwind;Integrated Security=True");  
ctn.Open();  
cmd = ctn.CreateCommand();  
cmd.CommandText = "select count(orderid) from orders where  
                  customerid='FRANK'";  
Console.WriteLine("el cliente ALFREDO ha pasado {0} pedido(s)",  
cmd.ExecuteScalar());
```

El caso de instrucciones que devuelven varios registros es un poco más complejo. Después de haber ejecutado la instrucción con el método `ExecuteReader` y recuperado el objeto `DataReader`, puede utilizar este último para recorrer los resultados devueltos. El método `Read` de la clase `DataReader` permite el desplazamiento en el conjunto de los registros devueltos. Este método devuelve un booleano que indica si queda aún un registro. El desplazamiento sólo es posible desde el primero al último registro. Este tipo de desplazamiento se llama `Forward Only`. La información contenida en el registro corriente está accesible por uno de los métodos `Get...` de la clase `DataReader`. Estos métodos permiten extraer los datos del registro y convertirlos en un tipo de datos .NET. Existe una versión para cada tipo de datos del Framework .NET. Por supuesto, hace falta que la información presente en el registro se pueda convertir en el tipo correspondiente. Si la conversión es imposible, se activa una excepción. Los métodos `Get...` esperan como parámetro el número del campo a partir de la cual se recupera la información. Por defecto, también puede utilizar la propiedad del `DataReader` indicando el nombre del campo en cuestión. En este caso, no hay conversión y el valor devuelto es de tipo `Object`.

El siguiente código visualiza la lista de todas las categorías de productos disponibles:

```
public static void testDataReader()  
{  
    SqlCommand cmd;
```

```

SqlConnection ctn;
SqlDataReader lector;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                          Catalog=Northwind;Integrated Security=true";

    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = " select * from categories";
    lector = cmd.ExecuteReader();
    while (lector.Read())
    {
        Console.WriteLine("numero de la categoria:{0}" + "\t" +
"Nombre:{1}", lector.GetInt32(0), lector["CategoryName"]);
    }
    lector.Close();
    ctn.Close();
}

```

La utilización de una conexión por un `DataReader` se efectúa de manera exclusiva. Para que la conexión esté de nuevo disponible para otro comando, debe cerrar obligatoriamente `DataReader` después de su utilización.

c. Modificaciones de la información

La modificación de la información en una base de datos se efectúa principalmente con las instrucciones SQL `INSERT`, `UPDATE`, `DELETE`. Estas instrucciones no devuelven registros de la base de datos. Para utilizar estas instrucciones, debe crear un `SqlCommand` y luego pedir la ejecución de este pedido a través del método `ExecuteNonQuery`. Este método devuelve el número de registros a los que afecta la ejecución de la instrucción SQL contenida en el `SqlCommand`. Si la propiedad `CommandText` contiene varias instrucciones SQL, entonces el valor devuelto por el método `ExecuteNonQuery` corresponde al número total de filas a las que afectan todas las instrucciones SQL del `SqlCommand`.

El siguiente código añade una nueva empresa de entrega en la tabla **Shippers**:

```

public static void TestExecuteNonQuery()
{
    SqlCommand cmd;
    SqlConnection ctn;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                          Catalog=Northwind;Integrated Security=true";

    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = "Insert into shippers (companyname,phone)
                      values ('DHL','02 40 41 42 43')";
    Console.WriteLine("{0} linea(s) añadida(s) en la tabla",
                      cmd.ExecuteNonQuery());
    ctn.Close();
}

```

d. Utilización de parámetros

El manejo de instrucciones SQL puede resultar más fácil si se crean parámetros. Éstos permiten construir instrucciones SQL genéricas que se pueden reutilizar fácilmente. El principio de

funcionamiento es similar al de los procedimientos y funciones de Visual C#. Una alternativa a la utilización de parámetros podría ser la construcción dinámica de una instrucción SQL por concatenación de cadenas de caracteres.

A continuación, un ejemplo que utiliza esta técnica y que permite la búsqueda de un cliente según su código (luego veremos cómo mejorar este código usando parámetros):

```
public static void TestRequeteConcat()
{
    SqlCommand cmd;
    SqlConnection ctn;
    SqlDataReader lector;
    string codigoCliente;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
        Catalog=Northwind;Integrated Security=true";
    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    Console.Write("introducir el código del cliente buscado:");
    codigoCliente = Console.ReadLine();
    cmd.CommandText = " SELECT * from Customers WHERE CustomerID =
        '" + codigoCliente + "'";
    lector = cmd.ExecuteReader();
    while (lector.Read())
    {
        Console.WriteLine("apellido del cliente:{0}",
            lector["ContactName"]);
    }
    lector.Close();
    ctn.Close();
}
```

La parte importante de este código corresponde al momento en el que se asigna un valor a la propiedad `CommandText`. Una instrucción SQL correcta debe construirse por concatenación de cadenas de caracteres. En nuestro caso, es relativamente simple, ya que sólo hay un valor variable en la instrucción SQL, pero si fueran varios, hay una multitud de concatenaciones que realizar. Los errores clásicos en estas concatenaciones son:

- el olvido de un espacio,
- el olvido de los caracteres `'` para enmarcar un valor de tipo cadena de caracteres,
- un número de caracteres `'` impar.

Todos estos errores tienen el mismo efecto: la creación de una instrucción SQL no válida que será rechazada por el servidor durante la ejecución.

La utilización de los parámetros simplifica considerablemente la escritura de este tipo de consulta. Se utilizan los parámetros para marcar una ubicación en una consulta donde estará colocado, en el momento de la ejecución, un valor literal de cadena de caracteres o numérico. Los parámetros pueden ser nominados o anónimos. Un parámetro anónimo es introducido en una consulta por el carácter `?`. Los parámetros nombrados se especifican mediante el carácter `@` seguido del parámetro.

La consulta de nuestro ejemplo anterior puede tomar las siguientes formas:

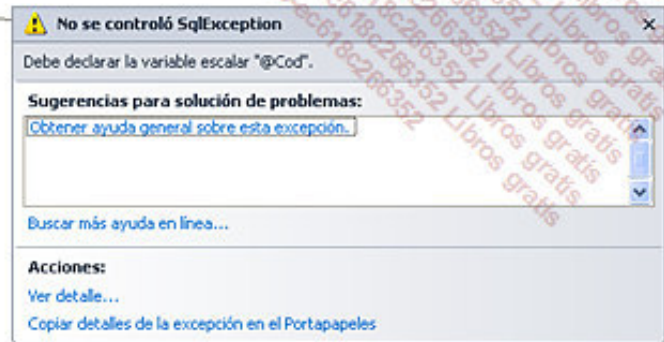
```
cmd.CommandText = " SELECT * from Customers WHERE CustomerID = ?";
```

```
cmd.CommandText = " SELECT * from Customers WHERE CustomerID = @Cod";
```

La ejecución del `SqlCommand` fracasa ahora si no se facilita información alguna para el parámetro o los parámetros.

```
cmd.CommandText = " SELECT * from Customers WHERE CustomerID = @Cod";
lector = cmd.ExecuteReader();
while (lector.Read())
{
    Console.WriteLine("apellido del cliente:{0}",
        lector["ContactName"]);
}
lector.Close();

ctn.Close();
```



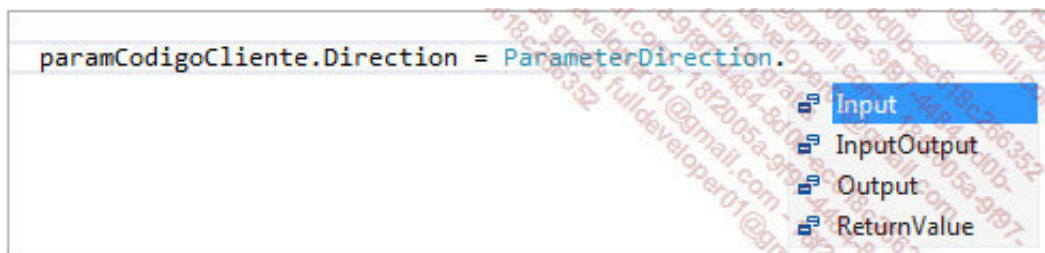
El `SqlCommand` debe tener una lista de valores utilizados para reemplazar los parámetros en el momento de la ejecución. Se almacena esta lista en la colección **Parameters** del `SqlCommand`. Antes de la ejecución del `SqlCommand`, hace falta crear los objetos `SqlParameter` y añadirlos a la colección. Para cada `SqlParameter`, hay que proveer:

- el nombre del parámetro;
- el valor del parámetro;
- la dirección de utilización del parámetro.

Se indica las dos primeras informaciones durante la construcción del objeto:

```
SqlParameter paramCodigoCliente;
paramCodigoCliente = new SqlParameter("@code", codigoCliente);
```

La dirección de utilización indica si la información contenida en el parámetro se pasa al código SQL para su ejecución (Input) o si le corresponde a la ejecución del código SQL modificar el valor del parámetro (Output), o ambas cosas (InputOutput). La propiedad `Direction` de la clase `SqlParameter` indica el modo de utilización del parámetro.



El parámetro está ahora preparado para añadirse a la colección **Parameters**. A este nivel, conviene estar al tanto si la consulta utiliza parámetros anónimos, ya que se deben añadir a la colección dichos parámetros obligatoriamente en el orden de su aparición en la consulta. Si se utilizan los parámetros nominados, no es necesario respetar esta regla, pero es prudente atenerse a ella, por si un día el código SQL se modifica y deja de utilizar los parámetros nominados. Podría ser el caso si usted debe

cambiar de tipo de proveedor de datos y el nuevo no acepta los parámetros nominados en una instrucción SQL. Ahora el `SqlCommand` está listo para la ejecución. Hay que observar que, con esta solución, no tenemos que preocuparnos del tipo de valor esperado por la instrucción SQL para saber si debemos enmarcarlo con caracteres '. Si se usan parámetros en la salida de la instrucción SQL, sólo estarán disponibles después del cierre del `DataReader`. El siguiente ejemplo muestra, además del nombre del cliente, el número de pedidos ya pasados:

```
public static void TestConsultaParam()
{
    SqlCommand cmd;
    SqlConnection ctn;
    SqlDataReader lector;
    string codigoCliente;
    SqlParameter paramCodigoCliente;
    SqlParameter paramNumPedidos;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
        Catalog=Northwind;Integrated Security=true";
    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    Console.WriteLine("introducir el código del cliente buscado:");
    codigoCliente = Console.ReadLine();
    cmd.CommandText = " SELECT * from Customers WHERE CustomerID =
        @Code;select @nbCmd=count(orderid) from
        orders where customerid=@code";
    paramCodigoCliente = new SqlParameter("@Code", codigoCliente);
    paramCodigoCliente.Direction = ParameterDirection.Input;
    cmd.Parameters.Add(paramCodigoCliente);
    paramNumPedidos = new SqlParameter("@nbCmd", null);
    paramNumPedidos.Direccion = ParameterDirection.Output;
    paramNumPedidos.SqlDbType=SqlDbType.Int;
    cmd.Parameters.Add(paramNumPedidos);
    lector = cmd.ExecuteReader();
    while (lector.Read())
    {
        Console.WriteLine("apellido del cliente:{0}",
            lector["ContactName"]);
    }
    lector.Close();
    Console.WriteLine("este cliente ha pasado {0} pedido(s)",
        cmd.Parameters["@nbCmd"].Value);
    ctn.Close();
}
```

e. Ejecución de procedimientos almacenados

Los procedimientos almacenados son componentes de una base de datos que corresponden a un conjunto de instrucciones SQL, los cuales pueden ejecutarse simplemente invocando su nombre. Son verdaderos programas SQL que pueden recibir parámetros y devolver valores. Además, los procedimientos almacenados se registran en la memoria caché del servidor en forma compilada durante su primera ejecución, lo que aumenta las prestaciones para las ejecuciones siguientes. Otra ventaja de los procedimientos almacenados es que centralizan en el servidor de base de datos todo el código SQL de una aplicación. Si se deben aportar modificaciones en las instrucciones SQL, sólo tendrá que efectuar modificaciones en el servidor, sin necesidad de retomar el código de la aplicación ni tener que volver a generar y desplegar la aplicación.

La llamada a un procedimiento almacenado a partir de Visual C# es prácticamente similar a la ejecución de una instrucción SQL. La propiedad `CommandText` contiene el nombre del procedimiento

almacenado. También puede modificar la propiedad `CommandType` con el valor `CommandType.StoredProcedure` para indicar que la propiedad `CommandText` contiene el nombre de un procedimiento almacenado. Como para una instrucción SQL, un procedimiento almacenado puede utilizar parámetros de entrada o salida. Hay un tercer tipo de parámetro disponible para los procedimientos almacenados en el tipo `ReturnValue`. Este tipo de parámetro sirve para recuperar el valor devuelto por la instrucción `Return` del procedimiento almacenado (mismo principio que una función Visual C#). Para probar estas nuevas nociones, vamos a utilizar el procedimiento almacenado siguiente, que devuelve el importe total de todos los pedidos hechos por un cliente.

```
CREATE PROCEDURE TotalCliente @code nchar(5) AS
declare @total money
select @total=sum(UnitPrice*Quantity*(1-Discount)) from Orders,[Order Details]
where customerid=@code and Orders.orderid=[order details].orderid
return @total
GO
```

A nivel de código Visual C#, debemos indicar que se trata de la ejecución de un procedimiento almacenado y añadir un parámetro para recuperar el valor de retorno del procedimiento almacenado. Este parámetro debe llamarse `RETURN_VALUE`.

```
public static void TestProcedimientoAlmacenado()
{
    SqlCommand cmd;
    SqlConnection ctn;
    SqlParameter paramCodigoCliente;
    SqlParameter paramImporte;
    string codigoCliente;
    Console.WriteLine("introducir el código del cliente buscado:");
    codigoCliente = Console.ReadLine();
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
        Catalog=Northwind;Integrated Security=true";
    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = "TotalCliente";
    cmd.CommandType = CommandType.StoredProcedure;
    paramCodigoCliente = new SqlParameter("@Code", codigoCliente);
    paramCodigoCliente.Direccion = ParameterDirection.Input;
    cmd.Parameters.Add(paramCodigoCliente);
    paramImporte = new SqlParameter("RETURN_VALUE",
        SqlDbType.Decimal);
    paramImporte.Direccion = ParameterDirection.ReturnValue;
    cmd.Parameters.Add(paramImporte);
    cmd.ExecuteNonQuery();
    Console.WriteLine("Este cliente ha efectuado pedidos por importe
        de {0} Euros", paramImporte.Value);
    ctn.Close();
}
```


Utilización del modo no conectado

En un modo no conectado, el enlace con el servidor de base de datos no es permanente. Hay que conservar de forma local los datos con los cuales se desea trabajar. La idea es volver a crear, con la ayuda de diferentes clases, una organización similar a la de una base de datos. Las principales clases vienen representadas en el siguiente esquema:

`DataSet`

Es el contenedor de mayor nivel, desempeña el mismo papel que la base de datos.

`DataTable`

Como su nombre indica, es el equivalente de una tabla de la base de datos.

`DataRow`

Esta clase desempeña el papel de un registro (fila).

`DataColumn`

Esta clase reemplaza un campo (columna) de una tabla.

`UniqueConstraint`

Es el equivalente de la clave primaria de una tabla.

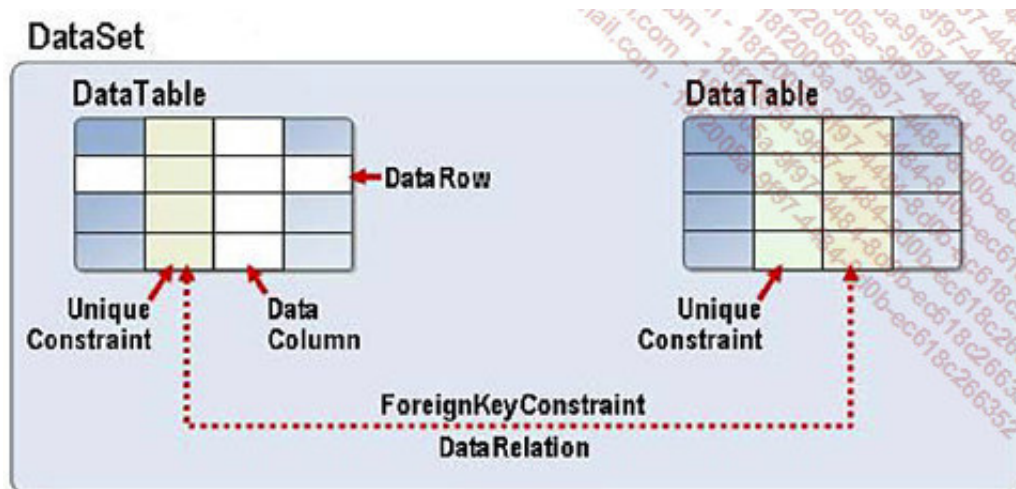
`ForeignKeyConstraint`

Es el equivalente de la clave foránea.

`DataRelation`

Representa un enlace padre/hijo entre dos `DataTable`.

El esquema siguiente representa esta organización.



Ahora vamos a ver cómo crear y manejar todas estas clases.

1. Rellenar un DataSet a partir de una base de datos

Para poder trabajar localmente con los datos, debemos volcarlos desde la base de datos en un DataSet. Cada proveedor de datos facilita una clase Data-Adapter, que asegura el diálogo entre la base de datos y un DataSet. Todos los intercambios se hacen por el medio de esta clase, tanto desde la base hacia el DataSet como desde el DataSet hacia la base para la actualización de los datos. El DataAdapter utilizará una conexión para contactar con el servidor y uno o varios comandos para el tratamiento de los datos.

a. Utilización de un DataAdapter

La primera tarea que debe hacerse consiste en crear una instancia de la clase SqlDataAdapter. Luego debemos configurar el DataAdapter con el fin de indicarle qué datos deseamos volcar desde la base de datos. La propiedad SelectCommand debe referenciar un objeto Command, que contiene la instrucción SQL encargada de seleccionar los datos. El objeto Command utilizado también puede llamar un procedimiento almacenado. La única restricción es que la instrucción SQL ejecutada por el objeto Command sea una instrucción SELECT. La clase DataAdapter contiene también las propiedades InsertCommand, DeleteCommand, y UpdateCommand, que hacen referencia a los objetos Command, utilizados durante la actualización de la base de datos.

Mientras no deseemos efectuar una actualización de la base, estas propiedades son opcionales. Se estudiarán más en detalle en la sección Utilización del modo no conectado - Actualización de la base de datos, en este capítulo.

El método Fill de la clase DataAdapter se utiliza para rellenar el DataSet con el resultado de la ejecución del comando SelectCommand. Este método espera como parámetro el DataSet que debe rellenar y un objeto DataTable o una cadena de caracteres que se usa para nombrar el DataTable en el DataSet. El DataAdapter utiliza, internamente, un objeto DataReader para obtener el nombre de los campos y el tipo de los campos con objeto de crear el DataTable en el DataSet y luego rellenarlo con los datos. El DataTable y los DataColumn se crean sólo si no existen anteriormente. En caso de que sí, el método Fill utiliza dicha estructura existente. Si se crea un DataTable, se añade a la colección **Tablas** del DataSet. El tipo de datos de los DataColumn se define en función de los mapeos previstos por el proveedor de datos, entre los tipos de la base de datos y los tipos .NET. El siguiente ejemplo rellena un DataSet con el código, el apellido, la dirección y la ciudad de los clientes.

```
public static void TestDataSet1()
{
    SqlCommand cmd;
    SqlConnection ctn;
    DataSet ds;
    SqlDataAdapter da;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                          Catalog=Northwind;Integrated Security=true";
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = " SELECT CustomerId,ContactName,Address,city from
                      Customers";
    ds = new DataSet();
    da = new SqlDataAdapter();
    da.SelectCommand = cmd;
```

```
da.Fill(ds, "Customers");  
}
```

En este código, la conexión no ha sido ni abierta ni cerrada explícitamente. En efecto, el método `Fill` abre la conexión si ya no está abierta y, en este caso, la vuelve a cerrar también al final de su ejecución. Sin embargo, si necesita utilizar varias veces el método `Fill`, es más eficaz que gestione por sí mismo la apertura y el cierre de conexión. En todos los casos, el método `Fill` deja la conexión en el estado en el que la ha encontrado.

Por supuesto, un `DataSet` puede contener varios `DataTable` creados a partir de `DataAdapter` diferentes. Los datos pueden provenir de bases de datos diferentes, incluso de tipos de servidores diferentes.

Cuando el `DataAdapter` construye el `DataTable`, los nombres de los campos de la base se utilizan para nombrar los `DataColumn`. Es posible personalizar estos nombres creando objetos `DataTableMapping` y añadiéndolos a la colección **TableMappings** del `DataAdapter`. Estos objetos `DataTableMapping` contienen ellos mismos objetos `DataColumnMapping` utilizados por el método `Fill`, como traductores entre los nombres de los campos en la base y los nombres de los `DataColumn` en el `DataSet`. En este caso, durante la llamada del método `Fill`, debemos indicarle el nombre del `DataTableMapping` que se ha de utilizar. Si para uno o varios campos no hay mapeo disponible, entonces el nombre del campo en la base se utiliza como nombre para el `DataColumn` correspondiente. Por ejemplo, podemos utilizar esta técnica para traducir los campos de la base **Northwind**.

El siguiente código efectúa esta traducción y visualiza el nombre de los `DataColumn` del `DataTable` creado:

```
public static void TestTableMapping()  
{  
    SqlCommand cmd;  
    SqlConnection ctn;  
    DataSet ds;  
    SqlDataAdapter da;  
    DataTableMapping mapeo;  
    ctn = new SqlConnection();  
    ctn.ConnectionString = "Data Source=localhost;Initial  
                          Catalog=Northwind;Integrated Security=true";  
    cmd = new SqlCommand();  
    cmd.Connection = ctn;  
    cmd.CommandText = " SELECT CustomerId,ContactName,Address,city from  
                      Customers";  
    ds = new DataSet();  
    da = new SqlDataAdapter();  
    da.SelectCommand = cmd;  
    mapeo = new DataTableMapping("Customers", "Clientes");  
    mapeo.ColumnMappings.Add("CustomerId", "CodigoCliente");  
    mapeo.ColumnMappings.Add("ContactName", "Apellido");  
    mapeo.ColumnMappings.Add("Address", "Dirección");  
    mapeo.ColumnMappings.Add("city", "Ciudad");  
    da.TableMappings.Add(mapeo);  
    da.Fill(ds, "Customers");  
    foreach ( DataColumn dc in ds.Tables["Clientes"].Columns)  
    {  
        Console.WriteLine(dc.ColumnName + "\t");  
    }  
}
```

Esto es lo que visualizamos:

b. Añadir restricciones a un DataSet

El método `Fill` sólo transfiere hacia el `DataSet` los datos que provienen de la base. Cuando la tabla que trae el `DataAdapter` de la base de datos presenta restricciones, éstas no se vuelcan en el `DataSet`. Para poder recuperar estas restricciones en el `DataSet`, hay dos soluciones posibles:

- Modificar la propiedad `MissingSchemaAction` del `DataAdapter` con el valor `MissingSchemaAction.AddWithKey`.

```
da.MissingSchemaAction = MissingSchemaAction.AddWithKey;
```

Proceder en dos etapas llamando primero el método `FillSchema` del `DataAdapter` para crear la estructura completa del `DataTable`, y luego llamar el método `Fill` para rellenar el `DataTable` con los datos.

```
da.FillSchema(ds, SchemaType.Mapped, "Customers");  
da.Fill(ds, "Customers");
```

El segundo parámetro del método `FillSchema` indica si se debe tener en cuenta el mapeo o si se utiliza la información proveniente de la base de datos.

Es importante añadir las restricciones de claves primarias, ya que el método `Fill` se va comportar de manera diferente según existan o no.

Si existen a nivel del `DataSet`, cuando el método `Fill` importa un registro desde la base, verifica si ya no existe una fila con el valor de clave primaria en el `DataTable`. Si es el caso, sólo actualiza los campos de la fila existente. Si, por el contrario, no hay una fila con un valor de clave primaria idéntica, entonces se crea la fila en el `DataTable`.

Si no hay restricción de clave primaria en el `DataTable`, el método `Fill` añade todos los registros procedentes de la base de datos. En este caso, puede que haya duplicados en el `DataTable`. Eso es particularmente importante cuando se debe llamar el método `Fill` varias veces para, por ejemplo, obtener los datos modificados por otra conexión a la base de datos.

2. Configurar un DataSet sin base de datos

No es necesario disponer de una base de datos para poder utilizar un `DataSet`; puede servir de alternativa a la utilización de tablas para la gestión interna de los datos de una aplicación. En este caso, todas las operaciones efectuadas automáticamente por el `DataAdapter` deberán realizarse manualmente mediante el código. Esto incluye en particular la creación de los `DataTable` con sus `DataColumn`. La primera operación que se ha de realizar consiste en crear una instancia de la clase `DataTable`. El constructor espera como parámetro el nombre de la `DataTable`. Luego se utiliza este nombre para identificar el `DataTable` en la colección `Tables` del `DataSet`. Después de su creación, el `DataTable` no contiene estructura alguna. Por lo tanto, debemos crear uno o varios `DataColumn` y añadirlos a la colección **Columns** del `DataTable`.

Se pueden crear los `DataColumn` haciendo uso de los constructores de la clase o automáticamente durante la adición a la colección **Columns**. La primera solución aporta más flexibilidad, ya que permite la configuración de numerosas propiedades del `DataColumn` en el momento de su creación. Debe, como mínimo, indicar un nombre y un tipo de datos para el `DataColumn`.

```
col = new DataColumn("Bruto", Type.GetType("decimal"));
table.Columns.Add(col);
table.Columns.Add("Iva", Type.GetType("decimal"));
```

Un `DataColumn` también se puede construir como una expresión basada en uno o varios otros `DataColumn`. En este caso, debe indicar durante la creación del `DataColumn` la expresión que sirve para calcular su valor. Por supuesto el tipo de datos generado por la expresión debe ser compatible con el tipo de datos del `DataColumn`. También debe tener cuidado con el diseño de la expresión, respetar las mayúsculas o minúsculas y vigilar con no crear referencias circulares entre los `DataColumn`.

```
table.Columns.Add("Neto", Type.GetType("decimal"), "Bruto *
(1 + Iva / 100)");
```

Para asegurar la unicidad de los valores de un `DataColumn`, es posible utilizar un tipo de `DataColumn` autoincrementado. La propiedad `AutoIncrement` de este `DataColumn` se debe colocar en **true**. También puede modificar el paso de incremento con la propiedad `AutoIncrementStep` y el valor de salida con la propiedad `AutoIncrementSeed`. El valor que contiene este `DataColumn` se calcula automáticamente durante la inserción de una fila en el `DataTable`, en función de estas propiedades y filas ya existentes en la `DataTable`.

Este tipo de `DataColumn` se suele utilizar como clave primaria de una `DataTable`. Usted tiene la posibilidad de definir la clave primaria de un `DataTable` facilitando la propiedad `PrimaryKey` de una tabla que contenga los diferentes `DataColumn` que deben componer la clave primaria. Los `DataColumn` implicados verán que algunas de sus propiedades se modifican automáticamente. La propiedad `Unique` se colocará en **true**, y la propiedad `AllowDBNull`, en **false**. Si la clave primaria está constituida de varias `DataColumn`, sólo se modificará la propiedad `AllowDBNull` en estos `DataColumn`.

```
col = new DataColumn("Numero", Type.GetType("int"));
col.AutoIncrement = true;
col.AutoIncrementSeed = 1000;
col.AutoIncrementStep = 1;
table.Columns.Add(col);
table.PrimaryKey=new DataColumn[] {col};
```

3. Manejar los datos en un DataSet

Sea cual sea el método utilizado para rellenar un `DataSet`, el objetivo de cualquier aplicación consiste en manejar los datos presentes en el `DataSet`. La clase `DataTable` contiene muchas propiedades y métodos que facilitan el manejo de los datos.

a. Lectura de los datos

La lectura de los datos es la operación más frecuente realizada en un `DataSet`. Primero hay que obtener una referencia sobre la `DataTable` que contiene los datos: luego podemos recorrer la colección **Rows** del `DataTable`. Esta colección es una instancia de la clase `DataRowCollection`. Por defecto, dispone de la propiedad `Item`, que permite el acceso a una fila particular por un índice. La propiedad `count` permite conocer el número de filas disponibles. En un `DataTable`, no hay noción de puntero de registro, de registro corriente, de métodos de desplazamiento en el juego de resultados. Si quiere gestionar todas estas nociones, debe administrarlas explícitamente en su código. El método `GetEnumerator` pone a nuestra disposición una instancia de clase que implementa la interfaz `IEnumerator`. Gracias a esta instancia de clase, tenemos acceso a los métodos `MoveNext` y `Reset`, así como a la propiedad `Current`. Estos tres elementos permiten

recorrer fácilmente todas las filas del `DataTable`. Cada fila corresponde a una instancia de la clase `DataRow`. Esta clase posee también una propiedad `Item` por defecto que aporta un acceso a los diferentes campos del `DataRow`. Se puede obtener cada campo gracias a su nombre o su índice.

El siguiente código ilustra estas nociones mostrando la lista de los clientes:

```
public static void TestLecturaDataTable()
{
    SqlCommand cmd;
    SqlConnection ctn;
    DataSet ds;
    SqlDataAdapter da;
    IEnumerator en;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                          Catalog=Northwind;Integrated Security=true";
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = " SELECT ContactTitle,ContactName from Customers";
    ds = new DataSet();
    da = new SqlDataAdapter();
    da.SelectCommand = cmd;
    da.Fill(ds, "Customers");
    // se recupera el enumerador en las filas+ de la DataTable
    en = ds.Tables["Customers"].Rows.GetEnumerator();
    // nos volvemos a situar al principio de la tabla (por seguridad)
    en.Reset();
    // bucleamos hasta que el método MoveNext nos indica que queda filas
    while (en.MoveNext())
    {
        // accedemos a los campos por el nombre
        Console.WriteLine(((DataRow)en.Current) ["ContactName"] + "\t");
        // o por el numero
        Console.WriteLine(((DataRow)en.Current) [0]);
    }
    Console.ReadLine();
}
```

b. Creación de restricciones sobre una DataTable

Puede utilizar restricciones para activar limitaciones sobre los datos presentes en un `DataTable`. Las restricciones constituyen reglas que se aplican a un `DataColumn` o a sus `DataColumn` relacionadas. Determinan las acciones efectuadas cuando se modifica el valor contenido en una fila. Sólo se tienen en cuenta para un `DataSet` si su propiedad `EnforceConstraints` se coloca en **true**.

Se pueden utilizar dos tipos de restricciones:

UniqueConstraint

Este tipo de restricción va a garantizar que el valor o los valores presentes en un `DataColumn` o un grupo de `DataColumn` sean únicos. La instalación de una restricción única se efectúa al crear una instancia de la clase `UniqueConstraint` con la lista de los `DataColumn` afectados por la restricción. Luego, esta `UniqueConstraint` se debe añadir a la colección **Constraints** del `DataTable`.

```
table.Constraints.Add(new UniqueConstraint(new DataColumn[] { col }));
```

Si la restricción sólo se refiere a un `DataColumn`, también es posible modificar simplemente la

propiedad `Unique` de este `DataColumn` a **true**, para crear una restricción única. Hay que observar también que la creación de una clave primaria genera automáticamente una restricción única; sin embargo, lo contrario no es cierto. La violación de la restricción tras una modificación de una fila desencadena una excepción.

ForeignKeyConstraint

Las `ForeignKeyConstraint` controlan cómo van a comportarse los `DataTable` relacionados durante la modificación o la supresión de un valor en el `DataTable` principal. Se puede considerar una acción diferente para una supresión y para una modificación. La clase `ForeignKeyConstraint` dispone de las propiedades `DeleteRule` y `UpdateRule`, que indican el comportamiento durante la supresión o la modificación. Son posibles los valores siguientes:

Cascade

La supresión o modificación se propaga a la fila o las filas relacionadas.

SetNull

El valor se modifica a `DBNull` en las filas relacionadas.

SetDefault

El valor por defecto se toma en las filas relacionadas.

None

No se lleva a cabo ninguna acción sobre las filas relacionadas.

La adición de una `ForeignKeyConstraint` se hace por la creación de una instancia indicando los `DataColumn` del `DataTable` padre y los `DataColumn` de la tabla hijo. Si varios `DataColumn` forman parte de la restricción, se facilitan en forma de tabla.

El siguiente código añade una restricción entre el `DataTable` **Facturas** y el `DataTable` **LineasFactura**, para que la supresión de una factura conlleve la supresión de todas sus filas.

```
var fkFact_LineasFact = new ForeignKeyConstraint("FK_FACT_LINEASFACT",
        ds.Tables["Facturas"].Columns["Numero"],
        ds.Tables["LineasFactura"].Columns["NumFact"]);
fkFact_LineasFact.AcceptRejectRule = AcceptRejectRule.Cascade;
fkFact_LineasFact.DeleteRule = Rule.Cascade;
ds.EnforceConstraints = true;
```

c. Creación de relaciones entre las DataTables

En un `DataSet` que contiene varios `DataTable`, puede añadir relaciones entre los `DataTable`. Estas relaciones permiten la navegación entre las filas de los diferentes `DataTable`. Debe crear una instancia de la clase `DataRelation` y añadirla a la colección `Relations` del `DataSet`. La creación se puede hacer directamente con el método `Add` de la colección `Relations`. La información que hay que facilitar es:

- El nombre de la relación que permite encontrar a continuación la `DataRelation` en la colección.

- El DataColumn o los DataColumn padres bajo la forma de una tabla de DataColumn si hay varias.
- El DataColumn o los DataColumn hijas bajo la forma de una tabla si hay varias.

El código siguiente añade una relación entre la tabla **Customers** y la tabla **Orders**:

```
ds.Relations.Add("Cliente_Pedidos",  
                ds.Tables["Customers"].Columns["CustomerId"],  
                ds.Tables["Orders"].Columns["CustomersId"]);
```

Hay que observar que las DataRelation funcionan en paralelo con las ForeignKeyConstraint y las UniqueConstraint. Por defecto, la creación de la relación va a colocar una UniqueConstraint en la tabla padre y una ForeignKeyConstraint en la tabla hijo. Si no desea que estas restricciones se agreguen automáticamente en caso de que no existan, debe añadir un booleano **false** como cuarto parámetro durante la adición de la DataRelation.

d. Recorrer las relaciones

El objetivo principal de las relaciones consiste en permitir la navegación de un DataTable hacia otro en el interior de un DataSet. Así podemos obtener todos los objetos DataRow de un DataTable vinculados con un DataRow de otro DataTable. Por ejemplo, después de haber cargado las tablas **Customers** y **Orders** en el DataSet y establecido una relación entre estas dos tablas, podemos, desde una fila del DataTable **Customers**, obtener del DataTable **Orders** todos los pedidos de este cliente. El método GetChildRows devuelve en forma de una tabla de DataRow todas las filas que contienen los pedidos de este cliente.

Este método toma como parámetro el nombre de la DataRelation utilizada para seguir el enlace. El siguiente ejemplo de código aplica esto: muestra, para cada cliente, el número y la fecha de sus pedidos:

```
public static void TestRelations()  
{  
    SqlCommand cmdCustomers;  
    SqlCommand cmdOrders;  
    SqlConnection ctn;  
    DataSet ds;  
    SqlDataAdapter daCustomers;  
    SqlDataAdapter daOrders;  
    ds = new DataSet();  
    ctn = new SqlConnection();  
    ctn.ConnectionString = "Data Source=localhost;Initial  
                          Catalog=Northwind;Integrated Security=true";  
    cmdCustomers = new SqlCommand();  
    cmdCustomers.Connection = ctn;  
    cmdCustomers.CommandText = " SELECT * from Customers";  
    daCustomers = new SqlDataAdapter();  
    daCustomers.SelectCommand = cmdCustomers;  
    daCustomers.Fill(ds, "Customers");  
    cmdOrders = new SqlCommand();  
    cmdOrders.Connection = ctn;  
    cmdOrders.CommandText = " SELECT * from Orders";  
    daOrders = new SqlDataAdapter();  
    daOrders.SelectCommand = cmdOrders;  
    daOrders.Fill(ds, "Orders");  
    ds.Relations.Add("Cliente_Pedidos",
```

```
        ds.Tables["Customers"].Columns["CustomerId"],
        ds.Tables["Orders"].Columns["CustomerId"]);
foreach ( DataRow lineaCliente in ds.Tables["Customers"].Rows)
{
    Console.WriteLine(lineaCliente["ContactName"]);
    foreach ( DataRow lineaPedidos in
        lineaCliente.GetChildRows("Cliente_Pedidos"))
    {
        Console.WriteLine("\t" + "pedido N {0} de {1}",
            lineaPedidos["OrderId"],
            lineaPedidos["OrderDate"]);
    }
}
```

La navegación de una fila hijo hacia una fila padre también es posible usando el método `GetParentRow`, que también espera como parámetro el nombre de la relación utilizada como enlace.

La parte del código siguiente muestra, para cada pedido, el nombre del cliente que lo ha hecho:

```
foreach (DataRow l in ds.Tables["Orders"].Rows)
{
    Console.WriteLine("el pedido {0} ha sido pasado por {1}",
        l["OrderId"], l.GetParentRow("Cliente_Pedidos")["ContactName"]);
}
```

e. Estado y versiones de una DataRow

La clase `DataRow` es capaz de monitorizar las diferentes modificaciones aplicadas a los datos que contiene. La propiedad `RowState` permite controlar las modificaciones aportadas a la fila.

Son posibles cinco valores definidos en una enumeración para esta propiedad:

Unchanged

La fila no ha cambiado desde que se llenó el `DataSet` con el método `Fill` o desde que se aceptaron las modificaciones con el método `AcceptChanges`.

Added

La fila se ha añadido, pero las modificaciones aún no han sido aceptadas por el método `AcceptChanges`.

Modified

Uno o varios campos de la fila se han modificado.

Deleted

La fila se ha borrado, pero las modificaciones aún no han sido aceptadas por el método `AcceptChanges`.

Detached

La fila se ha creado pero aún no forma parte de la colección **Rows** de un `DataTable`.

También se dispone de las diferentes versiones de una fila. Cuando acceda a los valores contenidos en una fila, puede especificar la versión que le interesa.

Para ello, la enumeración `DataRowVersion` propone cuatro valores:

Current

Versión actual de la fila. Esta versión no existe para una fila cuyo estado es `Deleted`.

Default

Versión por defecto de la fila. Para una fila cuyo estado es `Added`, `Modified`, `Unchanged`, esta versión es equivalente a la versión `Current`. Para una fila cuyo estado es `Deleted`, esta versión es equivalente a la versión `Original`. Para una fila cuyo estado es `Detached`, esta versión es igual a la versión `Proposed`.

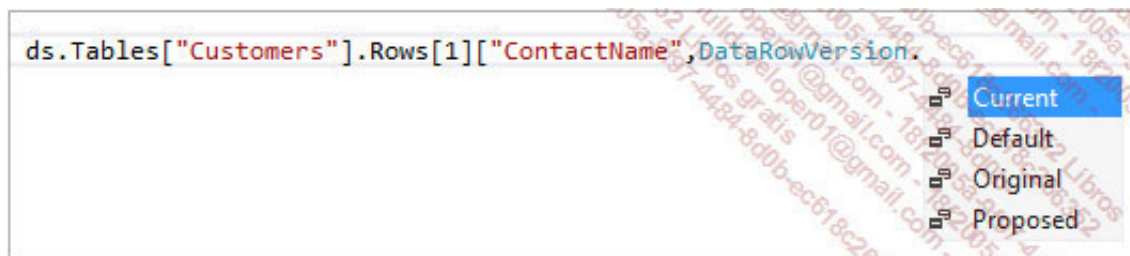
Original

Versión de origen de la fila. Para una fila cuyo estado es `Added`, esta versión no existe.

Proposed

Versión transitoria disponible durante una operación de modificación de la fila o para una fila que no forma parte de la colección **Rows** de un `DataTable`.

Se debe especificar la versión deseada durante el acceso a un campo particular de un `DataRow`. Para ello, hay que utilizar una de las constantes anteriores a continuación del nombre o del índice del campo durante la utilización de la propiedad `Item`, por defecto, del `DataRow`.



Se utilizarán estas diferentes versiones durante la actualización de la base de datos para gestionar los accesos concurrentes por ejemplo.

f. Adición de datos

La adición de una fila a un `DataTable` se efectúa simplemente al añadir un `DataRow` a la colección **Rows** de un `DataTable`. Previamente hace falta crear una instancia de la clase `DataRow`. Es a este nivel cuando encontramos un problema.

```
nuevaFila = new DataRow();
```

'System.Data.DataRow.DataRow(System.Data.DataRowBuilder)' no es accesible debido a su nivel de protección

No hay constructor disponible para la clase `DataRow`. Tranquilícese, no es un error sino algo realmente voluntario que no haya constructor para esta clase. En efecto cuando necesitamos una nueva instancia de un `DataRow`, no queremos un `DataRow` cualquiera sino un `DataRow` específico al esquema de nuestro `DataTable`. Por esta razón se le confía a él la tarea de crear la instancia que necesitamos por medio del método `NewRow`.

```
DataRow nuevaLinea;  
nuevaLinea = ds.Tables["Customers"].NewRow();
```

El estado de esta fila es de momento `Detached`. Luego podemos añadir datos en esta nueva fila.

```
nuevaLinea["ContactName"] = "García";
```

Después de ello, nos queda añadir la fila de la colección **Rows** del `DataTable`.

```
ds.Tables["Customers"].Rows.Add(nuevaLinea);
```

El estado de esta nueva fila es ahora `Added`.

g. Modificación de datos

Se realiza la modificación de los datos contenidos en una fila simplemente asignando a los campos correspondientes los valores deseados. Estos valores están almacenados en la versión `Current` de la fila. El estado de la fila es entonces `Modified`. Esta solución presenta una pequeña desventaja. Si se modifican simultáneamente varios campos de una fila, puede haber estados transitorios que violen restricciones colocadas en el `DataTable`. Por ejemplo, es el caso si existe en el `DataTable`, una restricción de clave primaria colocada en dos `DataColumn`. Esto tiene como efecto activar una excepción.

Para paliar este problema, podemos pedir que se ignore temporalmente la verificación de las restricciones para esta fila. El método `BeginEdit` pasa la fila en modo edición y suspende entonces la verificación de las restricciones para esta fila. Los valores asignados a los campos no están almacenados en la versión `Current` de la fila, sino en la versión `Proposed`. Cuando haya finalizado con las modificaciones de la fila, las puede validar o cancelar llamando respectivamente al método `EndEdit` o el método `CancelEdit`. También, puede verificar los valores gestionando el evento `ColumnChanged` del `DataTable`.

En el gestor de eventos, recibe un argumento de tipo `DataColumnChangeEventArgs` que permite saber qué `DataColumn` ha sido modificado (`args.Column.ColumnName`), el valor propuesto para este `DataColumn` (`args.ProposedValue`) y que permite cancelar las modificaciones (`args.row.CancelEdit`). En caso de validación con el método `EndEdit`, la versión `Proposed` de la fila se copia en la versión `Current` y el estado de la fila se convierte en `Modified`. Si, por el contrario, cancela las modificaciones con el método `CancelEdit`, la versión `Current` no se modifica y el estado de la fila es `unchanged`. En todos los casos, después de la llamada de uno de estos dos métodos, se reactiva la verificación de las restricciones.

El siguiente ejemplo permite la modificación del código postal de un cliente verificando que éste es efectivamente numérico:

```

public static void TestModificacionFila ()
{
    SqlCommand cmd;
    SqlConnection ctn;
    string codigoCliente;
    string codigoPostal;
    SqlParameter paramCodigoCliente;
    DataSet ds;
    SqlDataAdapter da;
    DataTable table;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                          Catalog=Northwind;Integrated Security=true";

    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    Console.Write("introducir el código del cliente a modificar:");
    codigoCliente = Console.ReadLine();
    cmd.CommandText = " SELECT * from Customers WHERE CustomerID = @Code";
    paramCodigoCliente = new SqlParameter("@Code", codigoCliente);
    paramCodigoCliente.Direction = ParameterDirection.Input;
    cmd.Parameters.Add(paramCodigoCliente);
    ds = new DataSet();
    da = new SqlDataAdapter(cmd);
    da.Fill(ds, "Clientes");
    table = ds.Tables["Clientes"];
    table.ColumnChanged += table_ColumnChanged;
    table.Rows[0].BeginEdit();
    Console.Write("introducir el nuevo código postal del cliente:");
    codigoPostal = Console.ReadLine();
    table.Rows[0]["PostalCode"] = codigoPostal;
    table.Rows[0].EndEdit();
    Console.WriteLine("el nuevo código postal es: {0}",
                     table.Rows[0]["PostalCode"]);
    Console.ReadLine();
}

public static void table_ColumnChanged(object sender,
                                     System.Data.DataColumnChangeEventArgs e)
{
    int cp;
    if (e.Column.ColumnName == "PostalCode")
    {
        if (int.TryParse(((string)e.ProposedValue), out cp))
        {
            e.Row.CancelEdit();
        }
    }
}
}

```

h. Supresión de datos

Dispone de dos soluciones diferentes. Puede borrar una fila o suprimir una fila. El matiz entre estas dos soluciones es sutil:

La supresión de una fila se hace con el método `Remove`, que retira definitivamente la `DataRow` de la colección **Rows** del `DataTable`. Esta supresión es definitiva.

El método `Deleted` sólo marca la fila para suprimirla posteriormente. El estado de la fila pasa a `Deleted` y sólo en el momento de validar las modificaciones se suprime realmente la fila de la

colección **Rows** del `DataTable`. Si se cancelan las modificaciones, la fila se queda en la colección **Rows**.

El método `Remove` es un método de la colección **Rows** (actúa directamente sobre su contenido), el método `Delete` es un método de la clase `DataRow` (sólo hace cambiar una propiedad de la fila).

```
// borra la línea
ds.Tables["Customers"].Rows[1].Delete();
// suprime la línea
ds.Tables["Customers"].Rows.Remove(ds.Tables["Customers"].Rows[1]);
```

i. Validar o cancelar las modificaciones

Hasta ahora, las modificaciones efectuadas en una fila son temporales, todavía es posible volver a la versión anterior, o por el contrario, validar de manera definitiva las modificaciones en las filas (pero todavía aún no en la base). Los métodos `AcceptChanges` o `RejectChanges` permiten respectivamente la validación o la anulación de las modificaciones. Se pueden aplicar sobre un `DataRow` individual, un `DataTable` o un `DataSet` entero.

Cuando se invoca al método `AcceptChanges`, se desencadenan las siguientes acciones:

- El método `EndEdit` se llama implícitamente para la fila.
- Si el estado de la fila era `Added` o `Modified`, se convierte en `Unchanged` y la versión `Current` se considera la versión `Original`.
- Si el estado de la fila era `Deleted`, entonces se suprime la fila.

El método `RejectChanges` ejecuta las siguientes acciones:

- El método `CancelEdit` se llama implícitamente para la fila.
- Si el estado de la fila era `Deleted` o `Modified`, se convierte en `Unchanged` y la versión `Original` es considerada la versión `Current`.
- Si el estado de la fila era `Added`, entonces se suprime la fila.

Si existen restricciones de clave foránea, la acción del método `AcceptChanges` o `RejectChanges` se propaga a las filas hijos en función de la propiedad `AcceptRejectRule` de la restricción.

j. Filtrar y ordenar datos

Es frecuente necesitar limitar la cantidad de datos visibles en un `DataTable` o aun modificar el orden de las filas. La primera solución que viene a la mente consiste en recrear una consulta SQL con una restricción o una cláusula `ORDER BY`. Pero eso implica olvidar que estamos en un modo de funcionamiento desconectado y que es deseable limitar los accesos a la base o, incluso peor, que la base no esté disponible. Por lo tanto, sólo debemos utilizar los datos disponibles teniendo cuidado de no perderlos. La clase `DataView` nos va a ser muy útil para solucionar nuestros problemas. Esta clase nos va a servir para modificar la visión de los datos en el `DataTable` sin riesgo para los propios datos. Puede haber varios `DataView` para un mismo `DataTable`; corresponden a puntos de vista diferentes del `DataTable`. Prácticamente todas las operaciones realizables en un `DataTable` también lo son mediante un `DataView`.

Hay dos soluciones disponibles para obtener un `DataView`:

- Crear una instancia gracias a uno de los constructores.
- Utilizar la instancia facilitada por defecto por la propiedad `DefaultView`.

El primer constructor utilizable espera simplemente como parámetro el `DataTable` a partir del cual se genera el `DataGridView`. En este caso, no hay ningún filtro ni tampoco ordenación efectuada sobre los datos visibles por el `DataGridView`. Se obtiene un resultado equivalente utilizando la propiedad `DefaultView` de un `DataTable`.

El segundo constructor permite especificar un filtro, un criterio de ordenación y la versión de las filas implicadas. Para ser visibles en el `DataGridView`, las filas deberán corresponder a todos estos criterios. También se pueden modificar los diferentes criterios con tres propiedades.

RowFilter

Esta propiedad acepta una cadena de caracteres que representa la condición que se debe completar para que una fila sea visible. Esta condición tiene una sintaxis totalmente similar a las condiciones de una cláusula `WHERE`. Se pueden utilizar los operadores `And` y `Or` para asociar varias condiciones.

El siguiente ejemplo muestra el nombre de los clientes comerciales o directores de venta en España:

```
public static void TestDataGridView ()
{
    SqlCommand cmd;
    SqlConnection ctn;
    DataSet ds;
    SqlDataAdapter da;
    DataTable table;
    ctn = new SqlConnection();
    ctn.ConnectionString = "Data Source=localhost;Initial
                        Catalog=Northwind;Integrated Security=true";

    ctn.Open();
    cmd = new SqlCommand();
    cmd.Connection = ctn;
    cmd.CommandText = " SELECT * from Customers";
    ds = new DataSet();
    da = new SqlDataAdapter(cmd);
    da.Fill(ds, "Clientes");
    table = ds.Tables["Clientes"];
    table.DefaultView.RowFilter = "Country='España' and (contactTitle=
                                'Sales Agent' or contactTitle='Sales Manager')";
    foreach ( DataGridView fila in table.DefaultView)
    {
        Console.WriteLine("apellido: {0}", fila["ContactName"]);
    }
}
```



Se puede cancelar un filtro asignándole una cadena vacía a la propiedad `RowFilter`.

Sort

Esta propiedad acepta también una cadena de caracteres que representa el criterio o los criterios utilizados para la ordenación. La sintaxis es equivalente a la de la cláusula `ORDER BY`.

El siguiente ejemplo muestra los clientes ordenados primero por país y luego por apellido para un mismo país:

```
//se cancela el filtro anterior
table.DefaultView.RowFilter = "";
```



```
//todas las filas son visibles ahora
//se añade un criterio de ordenación
table.DefaultView.Sort="Country ASC,ContactName ASC";
foreach(DataRowView fila in table.DefaultView)
{
    Console.WriteLine("País: {0} \t apellido:{1}"
                      ,fila["Country"],fila["ContactName"]);
}
```

RowStateFilter

Esta propiedad determina el estado de las filas y qué versión de la fila muestra en el `DataGridView`. Hay ocho posibilidades disponibles:

CurrentRows

Presenta la versión `Current` de todas las filas añadidas, modificadas o sin cambios.

Added

Presenta la versión `Current` de todas las filas añadidas.

Deleted

Presenta la versión `Original` de todas las filas borradas.

ModifiedCurrent

Presenta la versión `Current` de todas las filas modificadas.

ModifiedOriginal

Presenta la versión `Original` de todas las filas modificadas.

None

Ninguna fila.

OriginalRows

Presenta la versión `Original` de todas las filas modificadas, suprimidas o sin cambios.

Unchanged

Presenta la versión `Current` de todas las filas sin cambios.

En el siguiente ejemplo se suprimen dos filas, que pueden visualizarse por medio de un filtro:

```
// se suprimen dos filas
table.Rows[2].Delete();
table.Rows[5].Delete();
// se cancela el filtro
```

```
table.DefaultView.RowFilter = "";  
// se muestra la versión original de las filas suprimidas  
table.DefaultView.RowStateFilter = DataRowState.Deleted;  
foreach (DataRowView fila in table.DefaultView)  
{  
    Console.WriteLine("País: {0} \t apellidos: {1}", fila["Country"],  
        fila["ContactName"]);  
}
```

k. Buscar datos

La búsqueda de datos se puede realizar con los siguientes dos métodos: `Find` y `FindRows`. Para que funcionen estos dos métodos es imperativo haber ordenado previamente los datos con la propiedad `Sort`.

Find

Este método devuelve el índice de la primera fila que corresponde al criterio de búsqueda. Si no encuentra ninguna fila, devuelve -1. Espera como parámetro el valor que se ha de encontrar. Este valor es buscado por el campo que se utiliza como criterio de ordenación. Si el criterio de ordenación se compone de varios campos, hay que pasar al método `Find` una matriz de objetos que contenga los valores buscados para cada campo del criterio de ordenación en el orden de aparición de la propiedad `Sort`.

Este método se utiliza a menudo para buscar una fila a partir de la clave primaria.</p>
</div>
<div data-bbox="38 965 403 980" data-label="Page-Footer>
http://www.eni-training.com/client_net/mediabook.aspx?idR=69406
</div>
<div data-bbox="406 972 588 990" data-label="Page-Footer>
www.FreeLibros.me
</div>
<div data-bbox="926 965 969 979" data-label="Page-Footer>
16/16
</div>

Presentación de LINQ

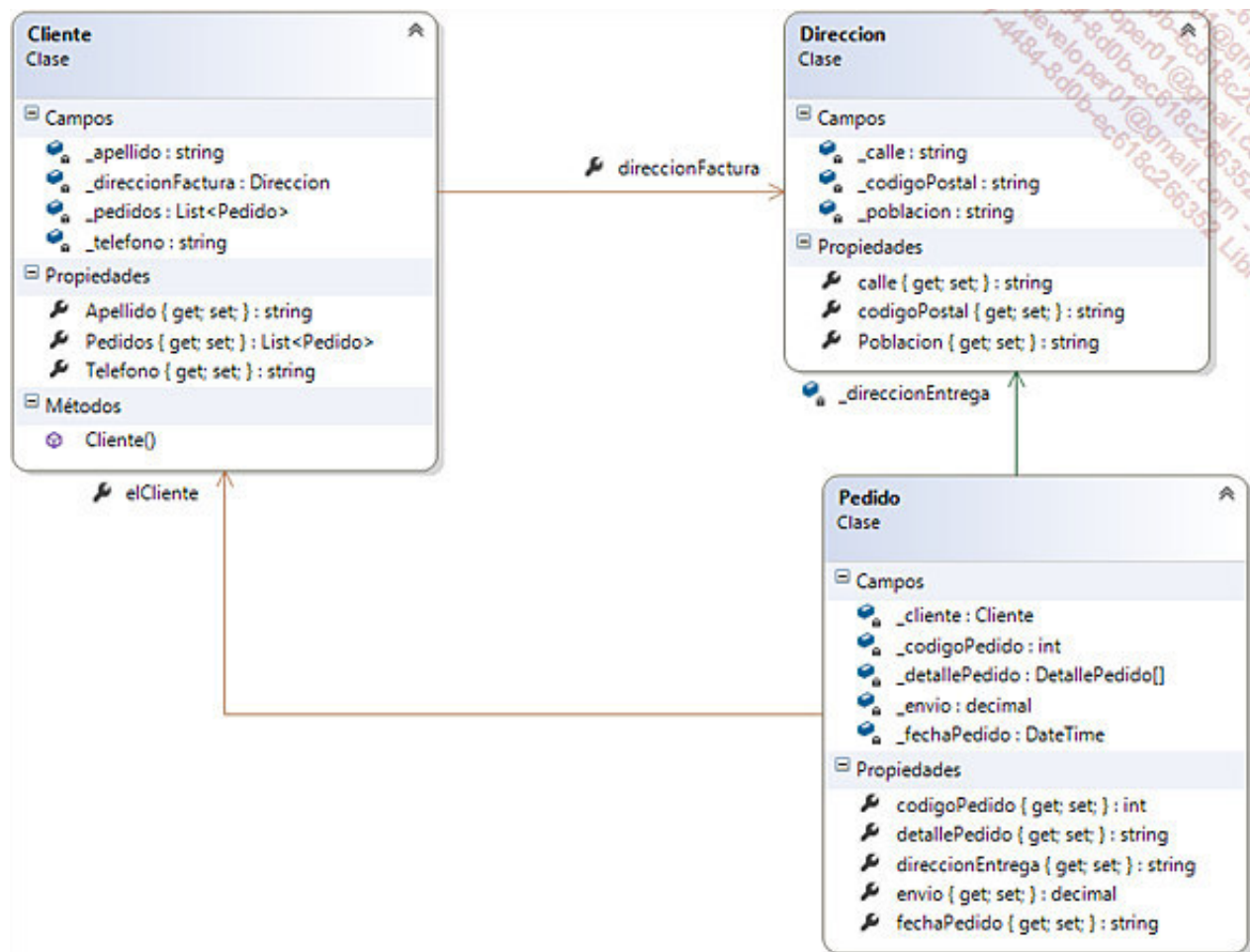
Después de muchos años de evolución, los lenguajes orientados a objetos se han hecho ineludibles en los desarrollos informáticos. De manera paralela, los sistemas de almacenamiento también han evolucionado sobre dos ejes: las bases de datos y los archivos XML. La combinación de conceptos orientados a objetos y datos se ha solucionado añadiendo a estos lenguajes módulos para dialogar con los datos. Esta solución no es del toda satisfactoria, ya que presenta las desventajas siguientes:

- El lenguaje utilizado para manejar datos suele ser muy específico de un tipo de fuente de datos.
- Las palabras claves de este lenguaje son desconocidas por el lenguaje de programación, que las considera como simples cadenas de caracteres, y por lo tanto no realiza verificación sintáctica alguna antes de la ejecución.
- El cambio de tipo fuente de datos conlleva importantes modificaciones del código y el desarrollador se ve obligado a aprenderlo.
- Los tipos de datos a veces son incompatibles entre el lenguaje de programación y la fuente de datos. En este caso, hay que realizar conversiones que a menudo requieren mucho tiempo y que incluso pueden resultar peligrosas.

Con LINQ todo esto queda superado. Pero ¿qué se esconde detrás de estas cuatro letras, *Language Integrated Query* o lenguaje de consulta integrado? De momento se trata de un lenguaje de consulta que permite consular fuentes de datos. Pero ¿qué más ofrece con respecto a SQL? La clave del misterio se sitúa en el término «integrado». En efecto, a diferencia de otros métodos utilizados para interrogar fuentes de datos (SQL, XPATH...), LINQ forma parte del lenguaje en el cual se desarrolla la aplicación (VB, C#...). Otro punto muy importante relativo a LINQ reside en la propia sintaxis del lenguaje. Ésta será idéntica sea cual sea el tipo de fuente de datos interrogado: tabla, colección, base de datos, archivo XML, dataset... El último punto importante de esta representación de LINQ hace referencia a los datos que se manejan. Su aplicación está desarrollada con un lenguaje orientado a objetos, y resulta que LINQ también maneja objetos. Así, no es necesario realizar manualmente las operaciones de conversión. Si son necesarias, LINQ las realizará automáticamente. Después de este breve vistazo, veamos ahora la sintaxis de LINQ.

Sintaxis del lenguaje LINQ

Antes de detallar la sintaxis de LINQ, vamos a estudiar un primer ejemplo muy simple. En los ejemplos de este párrafo, utilizaremos como fuente de datos dos listas completadas con instancias de las clases del siguiente diagrama.



Los datos utilizados para crear estas instancias de clase están extraídos de la base de datos Northwind. Están ubicados en un archivo de texto que luego es leído por el código para volver a crear las instancias de clase en memoria. A continuación mostramos el extracto de código que permite realizar estas operaciones.

```

public static void rellenar()
{
    Pedido co=null;
    Cliente cl=null;
    StreamReader f;
    String linea;
    String[] col;
    String nombre="";
    listaPedido=new List<Pedido>();
    listaClientes=new List<Cliente>();
    f = new StreamReader("c:\\data.txt");
    do
    {
        linea = f.ReadLine();
        if (linea != null)
    
```

```
{
    col = línea.Split(new char[] { '\t' });
    if (!nombre.Equals(col[0]))
    {
        nombre = col[0];
        cl = new Cliente { nombre = col[0], direccionFactura = new Direccion {
            calle = col[1], ciudad = col[2], codigoPostal = col[3] },
            telefono=col[4] };
        listaClientes.Add(cl);
    }
    co = new Pedido { codigoPedido = int.Parse(col[5]), fechaPedido =
        DateTime.Parse(col[6]), port = decimal.Parse(col[7]), direccionEntrega =
        new Direccion { calle = col[8], ciudad = col[0], codigoPostal = col[10] } };
    listaPedido.Add(co);
    co.ElCliente = cl;
    cl.LosPedidos.Add(co);
}
} while (línea != null);
f.Close();
}
```

En los siguientes ejemplos, supondremos que esta porción de código ya ha sido ejecutada para rellenar las dos listas. Este boceto de proyecto está disponible para su descarga en la página Información.

1. Primeras consultas LINQ

La utilización de una consulta LINQ está constituida por tres acciones:

- la obtención de los datos;
- la creación de la propia consulta;
- la ejecución de la consulta.

La primera etapa es muy simple, ya que para poder ser utilizada como fuente de datos, una clase debe implementar simplemente la interfaz genérica `IEnumerable<T>`. Es el caso de muchas clases del Framework .NET que se utilizan directamente en consultas LINQ.

Para las fuentes de datos que no implementan esta interfaz, como por ejemplo un documento XML, hay disponibles varias clases auxiliares que permiten hacerlas compatibles con LINQ.

La mayor parte del trabajo está constituido por la segunda etapa: la creación de la propia consulta.

En la consulta, vamos a indicar qué datos deseamos obtener de la fuente de datos, cómo se ordenarán, agruparán o estructurarán.

La consulta contiene tres cláusulas:

- `from`: indica el origen de los datos.
- `where`: especifica el filtro o los filtros que hay que aplicar en los valores devueltos.
- `select`: indica qué información es devuelta por la fuente de datos.

Así, presentamos a continuación nuestra primera consulta LINQ.

```
var consulta = from unCliente in listaClientes where
                unCliente.nombre.StartsWith("A") select unCliente;
```

Se suele almacenar una consulta LINQ en una variable y luego se ejecuta posteriormente. Es importante recordar que la variable que contiene la consulta no ejecuta ninguna acción y no devuelve ningún dato. Simplemente almacena la definición de la consulta. La ejecución de la consulta sólo tiene lugar cuando uno se interesa en los datos que devuelve. Es el caso del siguiente ejemplo, donde recorreremos los resultados en un bucle `foreach`.

```
foreach (var unCliente in consulta)
{
    Console.WriteLine(unCliente.apellido);
}
```

Este mecanismo permite ejecutar varias veces la misma consulta sin tener que volver a definirla a cada ejecución.

Sin embargo, en algunos casos, la variable contiene el resultado de la consulta, y no la propia consulta. Esto sucede, por ejemplo, cuando hay un cálculo de agregado en la consulta. En el siguiente ejemplo, buscamos cuántos clientes hay cuyo apellido empiece con la letra «A». En este caso, el resultado de la consulta es un simple entero calculado en el momento de la definición de la consulta.

```
var numClientes=(from unCliente in listaClientes where
unCliente.apellido.StartsWith("A") select unCliente).Count();
Console.WriteLine(numClientes);
```

También puede utilizar varias fuentes de datos en la cláusula `from`. Para ello, la palabra clave `join` permite combinar los datos de las diferentes fuentes. El siguiente ejemplo busca los clientes cuyo apellido empieza por una «A» y para cada uno recupera la fecha de todos los pedidos.

```
var consulta3 = from unCliente in listaClientes
                join unPedido in listaPedido
                on unCliente.apellido equals unPedido.ElCliente.apellido
                where unCliente.apellido.StartsWith("A")
                select new {apellidoCli = unCliente.apellido, fechaPdo =
                           unPedido.fechaPedido};
foreach (var r in consulta3)
{
    Console.WriteLine(r.apellidoCli + " " + r.fechaPdo);
}
```

Este código merece un pequeño comentario adicional en cuanto a la cláusula `select`. Deseamos obtener el apellido del cliente y la fecha de los pedidos, o sea, una cadena de caracteres y una fecha. Es inútil utilizar una instancia de la clase `Cliente` y una instancia de la clase `Pedido` sólo para estos dos datos. El compilador genera, por lo tanto, una clase anónima para estas dos informaciones con una propiedad `apellidoCli` y una propiedad `fechaPdo`. El tipo de la variable utilizada en el bucle `foreach` para recorrer el resultado de la consulta vendrá determinado implícitamente para corresponder al tipo anónimo creado por el compilador.

2. Los operadores de consulta

Los operadores que permiten la creación de una consulta LINQ se pueden clasificar en siete categorías:

- Ordenación de datos.
- Operaciones sobre conjuntos de datos.
- Filtrado.

- Proyección.
- Particionamiento.
- Uniones, agrupaciones.
- Agregación.

Para escribir consultas LINQ eficaces, conviene conocer correctamente estos operadores. Por lo tanto, vamos a detallarlos con muchos ejemplos.

a. Ordenación de datos

Es muy fácil obtener los resultados de una consulta ordenada según uno o varios criterios. Usando el operador `orderby`, debemos indicar la propiedad sobre la cual se realizará la ordenación. La siguiente consulta ordena los clientes según su número de pedidos.

```
var consultaOrdenacion1 = from unCliente in listaClientes
                          orderby unCliente.LosPedidos.Count
                          select unCliente;
foreach (var uncliente in consultaOrdenacion1)
{
    Console.WriteLine(uncliente.apellido + "num pedidos: " +
                      uncliente.LosPedidos.Count);
}
```

Por defecto, la ordenación se hace por orden ascendente. Para obtener los mejores clientes al principio de la lista, es preferible utilizar la palabra clave `descending` después del criterio de ordenación.

```
var consultaOrdenacion2 = from unCliente in listaClientes
                          orderby unCliente.LosPedidos.Count descending
                          select
unCliente;
foreach (var uncliente in consultaOrdenacion2)
{
    Console.WriteLine(uncliente.apellido + "num pedidos: " +
                      uncliente.LosPedidos.Count);
}
```

Se pueden indicar varios criterios de ordenación para evitar ambigüedades cuando dos propiedades tienen el mismo valor. Se deben separar con comas los criterios de ordenación en la consulta. La consulta ordena los clientes según el número de pedidos en orden descendente y luego según el apellido del cliente en orden ascendente en caso de igualdad del número de pedidos.

```
var consultaOrdenacion3 = from unCliente in listaClientes
                          orderby unCliente.LosPedidos.Count descending,
unCliente.apellido ascending
                          select unCliente;
foreach (var uncliente in consultaOrdenacion3)
{
    Console.WriteLine(uncliente.apellido + "num pedidos: " +
                      uncliente.LosPedidos.Count);
}
```

b. Operaciones sobre conjuntos de datos

El único operador disponible en esta categoría permite la eliminación de los duplicados durante la búsqueda de información. La palabra clave `distinct` ubicada al final de la cláusula `select` indica que se eliminarán los duplicados. Se tiene en cuenta el conjunto de los elementos de la cláusula `select` para eliminar los duplicados. La siguiente consulta determina las diferentes ciudades donde tenemos clientes. Si tenemos varios clientes en la misma ciudad, ésta sólo se listará una vez.

```
var consultaConjunto = (from unCliente in listaClientes
                        orderby unCliente.direccionFactura.ciudad
                        select
(unCliente.direccionFactura.ciudad)).Distinct();
foreach (var ciudad in consultaConjunto)
{
    Console.WriteLine(ciudad);
}
```

c. Filtrado de datos

El filtrado consiste en reducir el número de elementos devueltos por la consulta. Se añaden una o varias expresiones a la consulta mediante la cláusula `where`. Éstas deben facilitar un booleano durante la evaluación de la consulta. Se pueden utilizar los operadores de comparación estándar de Visual C# en el interior de la expresión. La utilización de cadenas de caracteres en una cláusula `where` merece una pequeña precisión. Aunque se pueda utilizar el operador `'=='` para un criterio de filtrado que trata sobre una cadena de caracteres, la utilización del método `Equals` ofrece muchas más funcionalidades. En efecto, con el operador `'=='` debe haber una estricta igualdad, incluyendo la distinción entre minúsculas y mayúsculas durante el proceso de evaluación. El método `Equals` es más flexible, ya que permite indicar cómo se debe hacer la comparación y si es preciso ignorar la distinción entre minúsculas y mayúsculas, como en el siguiente ejemplo.

```
var consultaFiltrado=from unCliente in listaClientes
                     where unCliente.direccionFactura.ciudad.Equals
                           ("barcelona",StringComparison.OrdinalIgnoreCase )
                     select unCliente;
foreach (var uncliente in consultaFiltrado)
{
    Console.WriteLine(uncliente.apellido);
}
```

d. Proyecciones

Una operación de proyección corresponde a la transformación de un objeto en una nueva forma. Esta nueva forma está constituida por el conjunto de las propiedades del objeto especificado en la cláusula `select`. Al utilizar la proyección, se puede crear automáticamente un nuevo tipo construido a partir de cada proyecto. Usted puede proyectar una propiedad directamente, o bien ejecutar una función que toma como parámetro la propiedad. En este caso, se utiliza el resultado de la función como valor para la propiedad del objeto creado. También puede proyectar el objeto original sin modificarlo.

```
var consultaProyeccion = from unCliente in listaClientes
                          select new { apellidoCli = unCliente.nom.ToUpper(),
                                      ciudadCli = unCliente.direccionFactura.ciudad.ToLower() };
foreach (var r in consultaProyeccion)
{
    Console.WriteLine(c.apellidoCli + " " + c.ciudadCli);
}
```

Las proyecciones también pueden llevarse a cabo indicando varias cláusulas `from` en la consulta. En

este caso, el resultado de la consulta hace corresponder cada objeto de cada una de las fuentes de datos con todos los objetos de otras fuentes.

```
var consultaProyeccion1 = from unCliente in listaClientes
                        from unPedido in listaPedido
                        select new { cli = unCliente, pdo = unPedido };
foreach (var r in consultaProyeccion1)
{
    Console.WriteLine(c.cli.apellido + " " + c.pdo.fechaPedido);
}
```

Este tipo de operación conduce muy rápidamente a una explosión combinatoria. En efecto, el número de objetos en el resultado de la consulta es igual al producto de los números de objetos en cada una de las fuentes de datos. Un filtrado que permita restringir el número de objetos en el resultado suele ser recomendable en este tipo de proyección.

```
var consultaProyeccion2 = from unCliente in listaClientes
                        from unPedido in listaPedido
                        where unCliente.Equals(unPedido.ElCliente)
                        select new { cli = unCliente, pdo = unPedido };
foreach (var r in consultaProyeccion2)
{
    Console.WriteLine(c.cli.apellido + " " + c.pdo.fechaPedido);
}
```

e. Particionamiento

El particionamiento consiste en recortar en dos partes un conjunto de datos y en devolver una de las dos partes. El límite de la partición puede ser absoluto, y en este caso expresado en número de objetos, o condicional. Se utilizan dos cláusulas para el particionamiento:

- **Skip** indica que se desea obtener la segunda parte de la lista (en realidad se «salta» los objetos ubicados al principio);
- **Take** indica que se desea obtener el principio de la lista sin tener en cuenta los registros del final de lista.

Veamos cómo utilizar estos dos operadores con la sintaxis absoluta y la sintaxis condicional.

La siguiente consulta permite obtener la lista de los diez peores clientes (basándose en el número de pedidos).

```
var consultaParticion = from uncliente in listaClientes
                        orderby uncliente.LosPedidos.Count
                        select uncliente;
foreach (var unCliente in consultaParticion.Skip(listaClientes.Count-10))
{
    Console.WriteLine(unCliente.apellido);
}
```

Para ilustrar la sintaxis condicional, buscamos ahora todos los clientes que tienen un número de pedidos inferior o igual a 5.

```
Console.WriteLine("*****");
var consultaParticion1 = from uncliente in listaClientes
                        orderby uncliente.LosPedidos.Count descending
                        select uncliente;
```

```
foreach (var unCliente in consultaParticion1.SkipWhile
                                     (test =>test.LosPedidos.Count>5))
{
    Console.WriteLine(unCliente.apellido);
}
```

Para recompensar a nuestros clientes fieles, buscamos ahora los diez mejores.

```
var consultaParticion2 = from uncliente in listaClientes
                        orderby uncliente.LosPedidos.Count descending
                        select uncliente;
foreach (var unCliente in consultaParticion2.Take(10))
{
    Console.WriteLine(unCliente.apellido);
}
```

Para terminar, buscamos los clientes que han hecho, al menos, diez pedidos.

```
var consultaParticion3 = from uncliente in listaClientes
                        orderby uncliente.LosPedidos.Count descending
                        select uncliente;
foreach (var unCliente in consultaParticion3.TakeWhile
                                     (unCliente=> unCliente.LosPedidos.Count>=10))
{
    Console.WriteLine(unCliente.apellido);
}
```

f. Uniones y agrupaciones

La unión de dos fuentes de datos corresponde a la asociación de una de las fuentes de datos con los objetos de la otra fuente de datos que tiene una propiedad común. En programación orientada a objetos, las uniones permiten reemplazar asociaciones incompletas. En el ejemplo que utilizamos desde el principio de este capítulo, la clase `Cliente` contiene una propiedad que permite obtener la lista de los pedidos de un cliente (`LosPedidos`) y la clase `Pedido` contiene un atributo que permite referenciar el cliente que ha hecho el pedido (`ElCliente`).

En nuestro caso, la asociación es bidireccional. Si por motivos de ahorro o por olvido no existiera la propiedad `LosPedidos` de la clase `Cliente`, haría falta recorrer en este caso la lista de los pedidos y probar cada uno de ellos para encontrar todos los pedidos de un cliente preciso. Le corresponde a la unión realizar este trabajo. La siguiente consulta obtiene los pedidos de cada cliente.

```
var consultaUnion = from uncliente in listaClientes
                  join unPedido in listaPedido on uncliente
                  equals unPedido.ElCliente
                  select new { uncliente, unPedido };
foreach (var c in consultaUnion)
{
    Console.WriteLine(c.uncliente.apellido + " " + c.unPedido.fechaPedido);
}
```

Para cada pedido se repiten los datos relativos al cliente. Una solución más eficaz consiste en ejecutar la consulta para que añada a cada cliente la lista de sus pedidos. La cláusula `join` permite realizar esta agrupación. En este caso, también es necesario especificar con la cláusula `into` el nombre de la propiedad utilizada para acceder a la agrupación: en nuestro caso, la lista de los pedidos del cliente. Cabe observar que para nosotros esta propiedad duplicará la que ya teníamos prevista en nuestra clase `Client`.

```

var consultaUnion2 =from unCliente in listaClientes
                    join unPedido in listaPedido on unCliente
                    equals unPedido.ElCliente
                    into PedidosDelCliente
                    select new { unCliente, PedidosDelCliente };
foreach (var r in consultaUnion2)
{
    Console.WriteLine(c.unCliente.apellido);
    foreach (var pdo in c.PedidosDelCliente)
    {
        Console.WriteLine("\t" + pdo.fechaPedido); }
}

```

También se puede realizar una agrupación sin por ello hacer una unión entre dos fuentes de datos. Por ejemplo, podemos buscar para cada ciudad la lista de los clientes que residen en ella. Para ello, la cláusula `group by into` es idónea. Basta indicar que deseamos agrupar los clientes usando como clave de agrupación su ciudad de residencia e indicar el nombre de la propiedad que se generará para contener la agrupación.

La consulta genera, durante su ejecución, una lista de instancias de clases que contiene dos propiedades:

- El nombre de la ciudad.
- La lista de los clientes gracias a la propiedad indicada en la consulta.

```

var consultaAgrup = from unClvar consultaAgrup = from unCliente in listaClientes
group unCliente by unCliente.direccionFactura.ciudad into ClientesPorCiudad
select new {ciudad=ClientesPorCiudad.Key,ClientesPorCiudad};
foreach (var v in consultaAgrup)
{
    Console.WriteLine(v.ciudad);
    foreach (var c in v.ClientesPorCiudad)
    {
        Console.WriteLine("\t" + c.apellido);
    }
}

```

g. Agregaciones

Se utilizan las operaciones de agregación para el cálculo de un valor único desde valores contenidos en una lista de elementos. Las operaciones más corrientes son:

- el cálculo de la media;
- la búsqueda de un máximo;
- la búsqueda de un mínimo;
- el cálculo de un total.

El siguiente ejemplo aplica estos cuatro operadores a los gastos de envío de todos los pedidos.

```

var mediaEnvio = listaPedido.Average(c => c.envio);
Console.WriteLine("media de los gastos de envío: {0}",medioEnvio);
var maxiEnvio = listaPedido.Max(c => c.envio);
Console.WriteLine("máximo de los gastos de envío: {0}",maxiEnvio);
var miniEnvio = listaPedido.Min(c => c.envio);
Console.WriteLine("mínimo de los gastos de envío: {0}",miniEnvio);

```

```
var totalEnvio = listaPedido.Som(c => c.envio);  
Console.WriteLine("total de los gastos de envío: {0}", totalEnvio);
```

Este código también es un buen ejemplo de consulta ejecutada inmediatamente, ya que para obtener el resultado se debe recorrer de forma obligatoria la lista desde el primero hasta el último elemento.

Después de haber estudiado la sintaxis del lenguaje, vamos a ver ahora cómo utilizarlo en asociación con una base de datos.

LINQ to SQL

Como hemos visto en los párrafos anteriores, el mundo predilecto de LINQ es el de los objetos. Sabe manejar perfectamente las listas, los objetos y las propiedades de estos objetos. Sin embargo, estos elementos presentan un grave inconveniente: desaparecen de manera inexorable en cuanto termina la aplicación. La solución más utilizada para paliar este problema consiste en confiar a una base de datos el trabajo de asegurar la persistencia de la información cuando la aplicación se encuentra detenida. Para el diálogo con una base de datos, lo que se usa con mayor frecuencia es el lenguaje SQL. Aunque las principales palabras claves son idénticas, las sintaxis de estos lenguajes no son compatibles. Las consultas LINQ se transforman automáticamente en sus homólogas SQL para realizar los tratamientos. Además, se debe tener en cuenta otro problema. Hasta ahora hemos manejado, gracias a consultas LINQ, objetos y sólo objetos. El concepto objeto es totalmente extraño a una base de datos. Por lo tanto, hay que encontrar una solución para que LINQ pueda acceder a la información. La clave del enigma consiste simplemente en crear clases para representar en la aplicación los datos que hay en la base de datos. Esta técnica se llama mapeo de objeto relacional. Debe ser la primera etapa en la utilización de LINQ con una base de datos; por lo tanto, vamos a ver cómo crear estas clases.

1. El mapeo de objeto relacional

Existen tres soluciones para generar las clases que representan los datos almacenados en la base de datos.

- Crear las clases manualmente, como cualquier otra clase de su aplicación, usando un editor de código. Esta solución es muy fastidiosa y suele emplearse sólo para las modificaciones mínimas de clases existentes.
- Utilizar la herramienta en línea de comando SQLMetal.
- Utilizar el Diseñador Objeto/Relacional en modo gráfico.

a. SQLMetal

Esta herramienta está disponible en una de las ventanas de comando del entorno de Visual Studio. Las opciones indicadas en la línea de comando permiten configurar su funcionamiento. Las opciones disponibles tratan de:

- La generación a partir de una base de datos del código fuente de las clases y de los atributos de mapeo.
- La generación a partir de una base de datos de un archivo intermedio de mapeo (.dbml).
- La generación a partir de un archivo de mapeo de las clases y de los atributos de mapeo.

Cada opción debe venir precedida de un carácter '/' y seguida del carácter ':' y del valor de la opción si es necesario.

Las opciones de conexión:

/server: <nombre del servidor>

Indica el nombre o la dirección IP del servidor de base de datos.

/database: <nombre de la base de datos>

Indica el nombre de la base de datos a partir de la cual debe efectuarse la generación.

/user: <nombre de conexión>

Indica la cuenta usuario con la cual la conexión hacia la base de datos estará abierta. Si no se especifica esta opción, se utilizará la autenticación de Windows.

/password: <contraseña>

Indica la contraseña asociada a la cuenta utilizada para establecer la conexión.

/conn: <cadena conexión>

Se puede utilizar en lugar de las cuatro opciones anteriores para facilitar los datos relativos a la conexión hacia el servidor de base de datos.

timeout: <segundos>

Indica la duración máxima durante la cual SqlMetal intenta establecer la conexión hacia la base de datos. Un valor igual a cero indica una duración ilimitada.

Las opciones de salida:

/dbml: <nombre del archivo>

Genera un archivo de mapeo.

/code: <nombre del archivo>

Genera el código fuente de las clases en el archivo indicado.

Las opciones de generación:

/lenguaje: <vb o csharp>

Indica el lenguaje en el que el código se generará. Las dos opciones válidas son vb para Visual Basic y csharp para C#.

/namespace: <nombre>

Indica el espacio de nombres en que se generarán las clases. Por defecto, no hay espacio de nombres.

/context: <nombre>

Especifica el nombre del data context generado. Por defecto, se deduce este nombre del nombre de la base de datos.

/entitybase: <nombre>

Especifica la clase de base de las clases generadas. Por defecto, las clases generadas no tienen clase básica.

Para terminar, la última información que hay que facilitar corresponde al nombre del archivo de mapeo

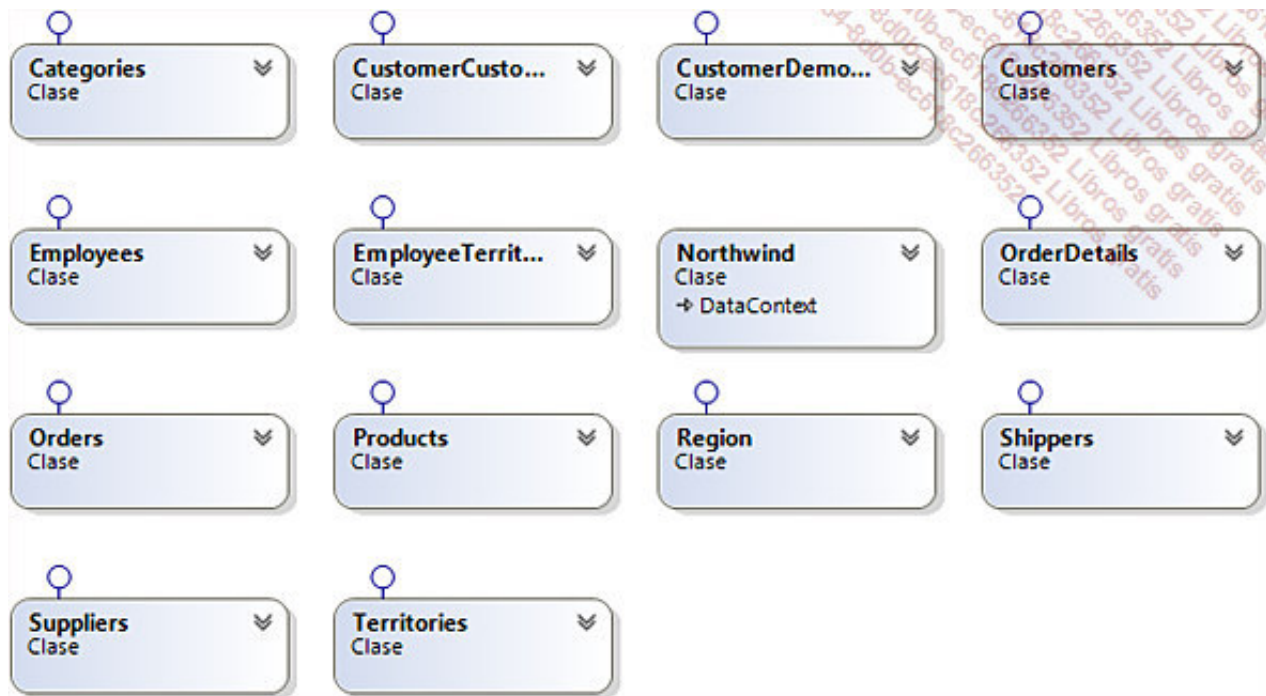
a partir del cual se realizará la generación de las clases. Esta información es inútil si la generación se ejecuta directamente desde la base de datos.

Le presentamos a continuación algunos de los usos más corrientes de esta herramienta.

Generación en Visual C# de las clases de la base Northwind situada en el ordenador local:

```
SqlMetal /server:localhost /database:northwind /language:csharp
/code:nw.cs
```

Como el código generado es demasiado voluminoso para listarlo aquí (unas 3.500 líneas), veamos el diagrama siguiente de las clases generadas.



Tenemos la clase `Northwind`, que hereda de la clase `DataContext` y que rápidamente nos va a servir para que LINQ pueda dialogar con la base de datos. También tenemos una clase generada para cada una de las tablas de la base de datos. Son instancias de estas clases lo que manejaremos en la aplicación.

Generación del archivo de mapeo de la base Northwind situada en el ordenador local:

```
SqlMetal /server:localhost /database:northwind /dbml:nw.dbml
```

Este comando genera un archivo xml del cual vemos un extracto a continuación:

```
<Table Name="dbo.Customers" Member="Customers">
  <Type Name="Customers">
    <Column Name="CustomerID" Type="System.String" DbType="NChar(5)
NOT NULL"
IsPrimaryKey="true" CanBeNull="false" />
    <Column Name="CompanyName" Type="System.String" DbType="NVarChar(40)
NOT NULL" CanBeNull="false" />
    <Column Name="ContactName" Type="System.String" DbType="NVarChar(30) "
CanBeNull="true" />
    <Column Name="ContactTitle" Type="System.String" DbType="NVarChar(30) "
CanBeNull="true" />
```

```

        <Column Name="Address" Type="System.String" DbType="NVarChar(60)"
CanBeNull="true" />
        <Column Name="City" Type="System.String" DbType="NVarChar(15)"
CanBeNull="true" />
        <Column Name="Region" Type="System.String" DbType="NVarChar(15)"
CanBeNull="true" />
        <Column Name="PostalCode" Type="System.String" DbType="NVarChar(10)"
CanBeNull="true" />
        <Column Name="Country" Type="System.String" DbType="NVarChar(15)"
CanBeNull="true" />
        <Column Name="Phone" Type="System.String" DbType="NVarChar(24)"
CanBeNull="true" />
        <Column Name="Fax" Type="System.String" DbType="NVarChar(24)"
CanBeNull="true" />
        <Association Name="FK_CustomerCustomerDemo_Customers" Member=
"CustomerCustomerDemo" OtherKey="CustomerID" Type="CustomerCustomerDemo"
DeleteRule="NO ACTION" />
        <Association Name="FK_Orders_Customers" Member="Orders" OtherKey=
"CustomerID" Type="Orders" DeleteRule="NO ACTION" />
    </Type>
</Table>

```

Se puede modificar este archivo para cambiar, por ejemplo, el nombre de las clases y de las propiedades asociadas a la información proveniente de la base de datos. En el siguiente ejemplo, hemos castellanizado los nombres.

```

<Table Name="dbo.Customers" Member="Customers">
    <Type Name="Customers">
        <Column Name="CustomerID" Member="código de cliente"
Storage="_CustomerID"
Type="System.String" DbType="NChar(5) NOT NULL" IsPrimaryKey="true"
CanBeNull="false" />
        <Column Name="CompanyName" Member="nombre de la empresa"
Storage="_CompanyName"
Type="System.String" DbType="NVarChar(40) NOT NULL" CanBeNull="false" />
        <Column Name="ContactName" Member="nombre de contacto"
Storage="_ContactName"
Type="System.String" DbType="NVarChar(30)" CanBeNull="true" />
        <Column Name="ContactTitle" Member="función" Storage="_ConctatTitle"
Type="System.String" DbType="NVarChar(30)" CanBeNull="true" />
        <Column Name="Address" Member="dirección" Storage="_Address"
Type="System.String" DbType="NVarChar(60)" CanBeNull="true" />
        <Column Name="City" Member="ciudad" Storage="_City"
Type="System.String"
DbType="NVarChar(15)" CanBeNull="true" />
        <Column Name="Region" Type="System.String" DbType="NVarChar(15)"
CanBeNull="true" />
        <Column Name="PostalCode" Member="código postal" Storage="_PostalCode"
Type="System.String" DbType="NVarChar(10)" CanBeNull="true" />
        <Column Name="Country" Member="País" Storage="_Country"
Type="System.String" DbType="NVarChar(15)" CanBeNull="true" />
        <Column Name="Phone" Member="teléfono" Storage="_Phone"
Type="System.String"
DbType="NVarChar(24)" CanBeNull="true" />
        <Column Name="Fax" Type="System.String" DbType="NVarChar(24)"
CanBeNull="true" />
        <Association Name="FK_CustomerCustomerDemo_Customers" Member=
"CustomerCustomerDemo" Thiskey="código de cliente" OtherKey="CustomerID"
Type="CustomerCustomerDemo"
        <Association Name="Clients_Orders" Member="Orders" ThisKey=

```

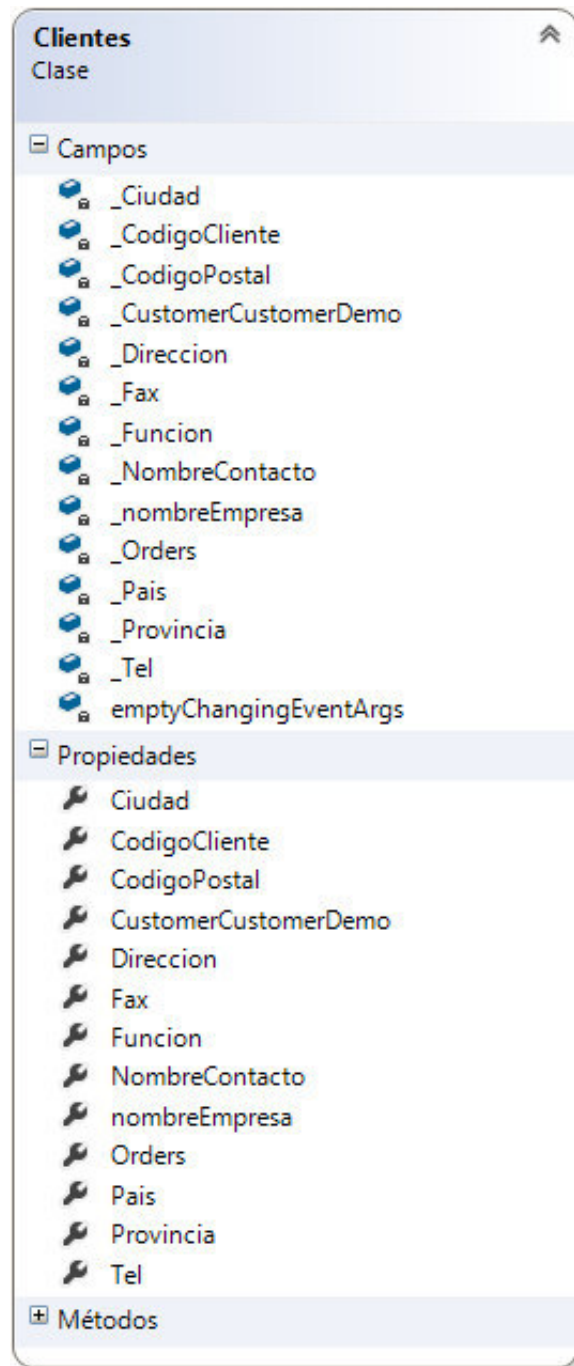
```
"código de cliente"  
OtherKey= "CustomerID" Type="Orders"  
    </Type>  
</Table>
```

En este ejemplo, para poder utilizar nombres de propiedades diferentes de los nombres de las columnas en la base de datos, hemos añadido el atributo `Member` a cada etiqueta `<Column>` para especificar el nombre de la propiedad y el atributo `Storage` para indicar el nombre de la variable interna de la clase que contendrá la información.

Ahora podemos generar el código a partir del archivo de mapeo modificado con el siguiente comando.

```
SqlMetal /code:nw.cs /language:csharp nw.dbml
```

Nos queda visualizar la clase generada para comprobar que nuestras modificaciones se han tomado en cuenta.



Esta herramienta es muy fácil de utilizar, pero presenta el pequeño inconveniente que puede generar las clases sólo para la totalidad de una base de datos. Además, las posibles modificaciones se deben hacer manualmente, sea en el código fuente generado, sea en el archivo de mapeo intermedio. Para la generación y la personalización de algunas clases, es preferible utilizar el diseñador Objeto/Relacional integrado en Visual Studio.

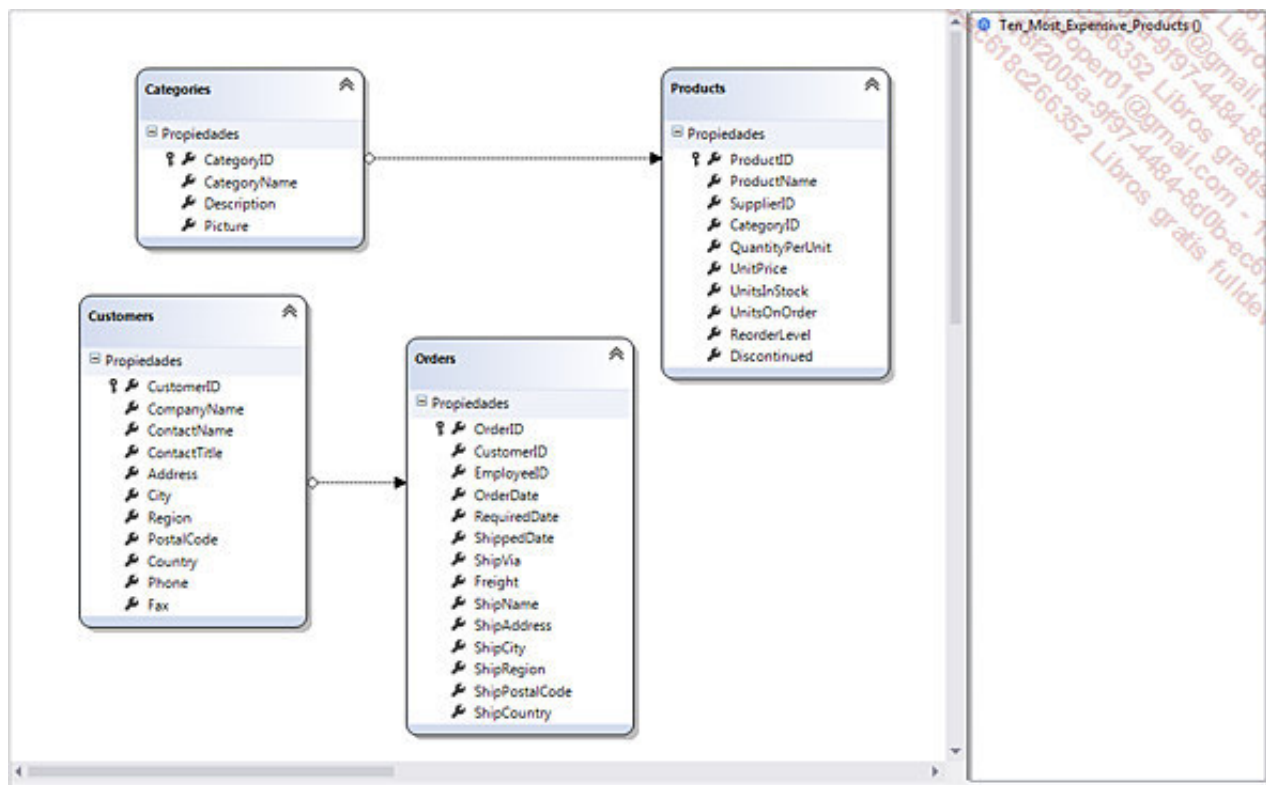
b. Diseñador Objeto/Relacional

El diseñador Objeto/Relacional ofrece una solución muy práctica para crear el modelo de objeto de una aplicación que representa la información disponible en una base de datos. También permite la creación de procedimientos y funciones que autorizan la utilización de los procedimientos almacenados presentes en la base de datos. Sin embargo, adolece de algunas limitaciones:

- Sólo las bases de datos SQL Server 2000 o versiones superiores son compatibles.

- El mapeo sólo es posible entre una clase y una tabla. Eso significa que no es posible crear una clase para representar el resultado de una unión entre varias tablas.
- El diseñador funciona en «sentido único», ya que sólo las modificaciones efectuadas en el diseñador se vuelcan en el código generado. Si se modifica el código manualmente, el diseñador no toma en cuenta las modificaciones. O peor, si se hacen modificaciones en el diseñador después de las modificaciones manuales del código, estas modificaciones se pierden durante el volcado del diseñador, ya que, en este caso, se genera código automáticamente. La solución consiste en crear una clase parcial en un archivo independiente del manejo por el diseñador.

El diseñador Objeto/Relacional se inicia automáticamente durante la adición de un elemento de tipo LINQ to SQL Classes. La adición a un proyecto de un archivo .dbml provoca también la apertura de esta herramienta. En el momento de la apertura, la superficie del diseñador se separa en dos partes. La zona de la izquierda va a acoger las clases asociadas a las tablas, mientras que la zona de la derecha va a acoger los procedimientos y funciones asociados a los procedimientos almacenados. El conjunto representa el DataContext generado.

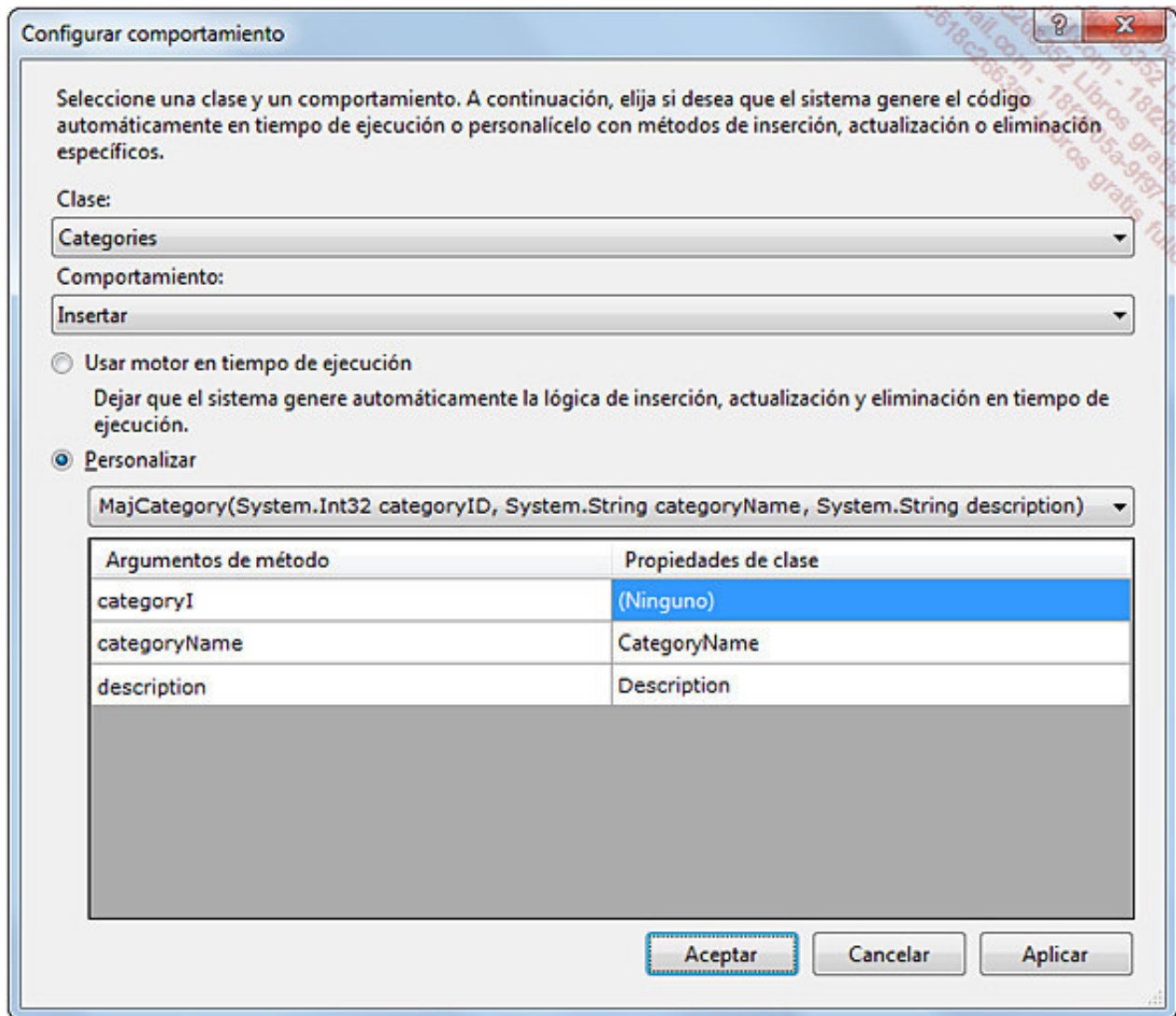


Adición de clases

Usted puede crear las clases que representan las tablas de una base de datos arrastrando y soltando una o varias tablas desde el explorador de servidores hasta la parte izquierda del diseñador. El primer elemento añadido al diseñador Objeto/Relacional también se utiliza para configurar las propiedades de conexión del DataContext. En caso de que se añada otro elemento que provenga de otra base de datos, un cuadro de diálogo le pregunte si desea sustituir la conexión existente. Si acepta la modificación, las clases ya presentes en el diseñador no se podrán utilizar. Añadir una tabla genera el código necesario para que el DataContext inicialice las propiedades de una instancia de la clase a partir de la información de una fila de la base de datos. También añade el código necesario para que las modificaciones aportadas a las propiedades de la instancia se puedan proyectar en la base de datos. El diseñador se basa en la estructura de la tabla y en la clave primaria para efectuar las actualizaciones. También puede indicar usted mismo cómo se efectuarán las actualizaciones. Para ello, cada clase posee tres propiedades Insert, Update, Delete. Por defecto se

inician estas propiedades con el valor utilizar el runtime para indicar que el código encargado de las actualizaciones se genera automáticamente.

Para modificar este comportamiento, puede asignar procedimientos almacenados a estas propiedades. Un cuadro de diálogo permite la configuración de estas propiedades.

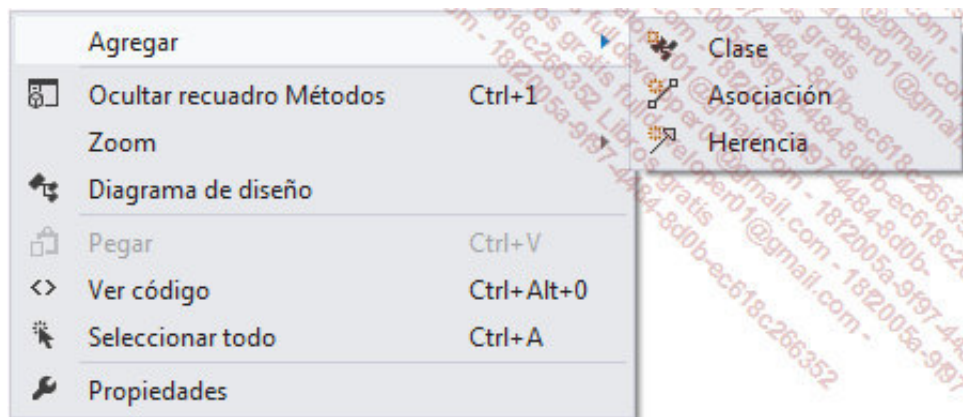


Después de la selección del procedimiento almacenado, debe señalar cómo se indican los parámetros que se esperan en entrada por el procedimiento almacenado. Los valores disponibles corresponden a las diferentes propiedades de la clase. Para cada propiedad, está disponible la versión actual o de origen.

Agregar asociaciones

Después de haber depositado varias tablas en el diseñador, es posible crear asociaciones entre algunas de ellas. Las asociaciones son totalmente similares a las relaciones entre tablas en una base de datos. De hecho, si existe en la base de datos una relación de clave extranjera entre dos tablas, se creará automáticamente una asociación cuando estas dos tablas estén en el diseñador.

Para agregar manualmente una asociación, debe dirigirse al menú contextual del diseñador Objeto/Relacional.



Un cuadro de diálogo le propone entonces configurar la relación. Debe elegir la clase padre y la clase hijo de la relación. La clase padre es la clase que se encuentra en el extremo «uno» de una relación uno a varios; la clase hijo representa el extremo «varios» de la relación. Por ejemplo, en la asociación entre las clases `Product` y `Category`, la clase `Category` representa el lado 'uno' de la relación y la clase `Product`, el lado 'varios'. En efecto, un producto pertenece a una categoría y una categoría contiene varios productos. Luego usted debe indicar, para cada una de las clases, la propiedad o las propiedades que van a participar en la relación. Hay que tener cuidado con que las propiedades que participan en la asociación sean del mismo tipo en cada extremo de ésta. Después de la creación de la asociación, puede configurar algunas propiedades que no estén disponibles en el momento de la creación.

- **Cardinalidad:** determina si la asociación es de tipo uno a varios (one-to-many) o de tipo uno a uno (one-to-one).
- **Propiedad hijo:** indica si una propiedad debe crearse en la clase padre para referenciar la información de la clase hijo. El tipo de esta propiedad está determinado por el tipo de la clase hijo y la cardinalidad. Si la cardinalidad es uno a uno, la propiedad es una simple referencia hacia una instancia de la clase correspondiente. Si la cardinalidad es uno a varios, la propiedad es una colección de instancias de la clase correspondiente.
- Las propiedades `Nombre` permiten identificar las propiedades creadas para realizar la asociación.

Adición de métodos

Los procedimientos almacenados y las funciones se pueden añadir al diseñador Objeto/Relacional para ser transformados luego en métodos del `DataContext`. La llamada de estos métodos provocará la ejecución del procedimiento almacenado o de la función por el servidor de base de datos. Si se esperan parámetros en entrada por el procedimiento almacenado, se les deberá facilitar al método durante su ejecución. La adición de un método al `DataContext` se realiza simplemente arrastrando y soltando entre el explorador de servidores y el diseñador Objeto/Relacional. Sin embargo, debe tener cuidado cuando añada un procedimiento almacenado o una función, ya que la ubicación donde tendrá lugar el desplazamiento determina el tipo de retorno del método generado. Si se desplaza el elemento hacia la zona de derecha del diseñador, el tipo de retorno se generará automáticamente.

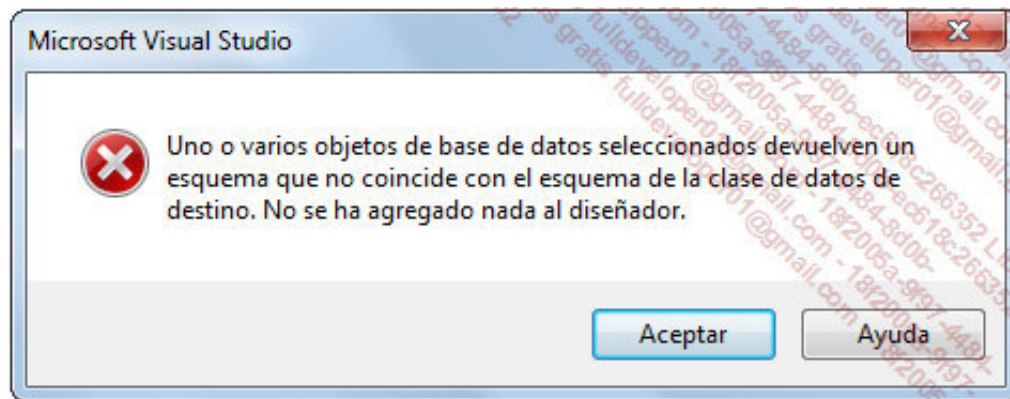
En cambio, si se desplaza el elemento hacia una clase existente del diseñador, el tipo de retorno corresponderá a esta clase siempre y cuando la información devuelta por el procedimiento almacenado sea compatible con esta clase. Vamos a trabajar con el procedimiento almacenado `[Ten Most Expensive Products]`, cuyo código es el siguiente:

```
set ANSI_NULLS ON
set QUOTED_IDENTIFIER ON
```



```
go
CREATE procedure [dbo].[Ten Most Expensive Products] AS
SET ROWCOUNT 10
SELECT Products.ProductName AS TenMostExpensiveProducts,
Products.UnitPrice
FROM Products
ORDER BY Products.UnitPrice DESC
```

Este procedimiento devuelve el nombre y el precio de los diez productos más caros. Si intentamos añadir este procedimiento al DataContext efectuando un arrastrar-soltar sobre la superficie de la clase Product, obtenemos el siguiente mensaje:

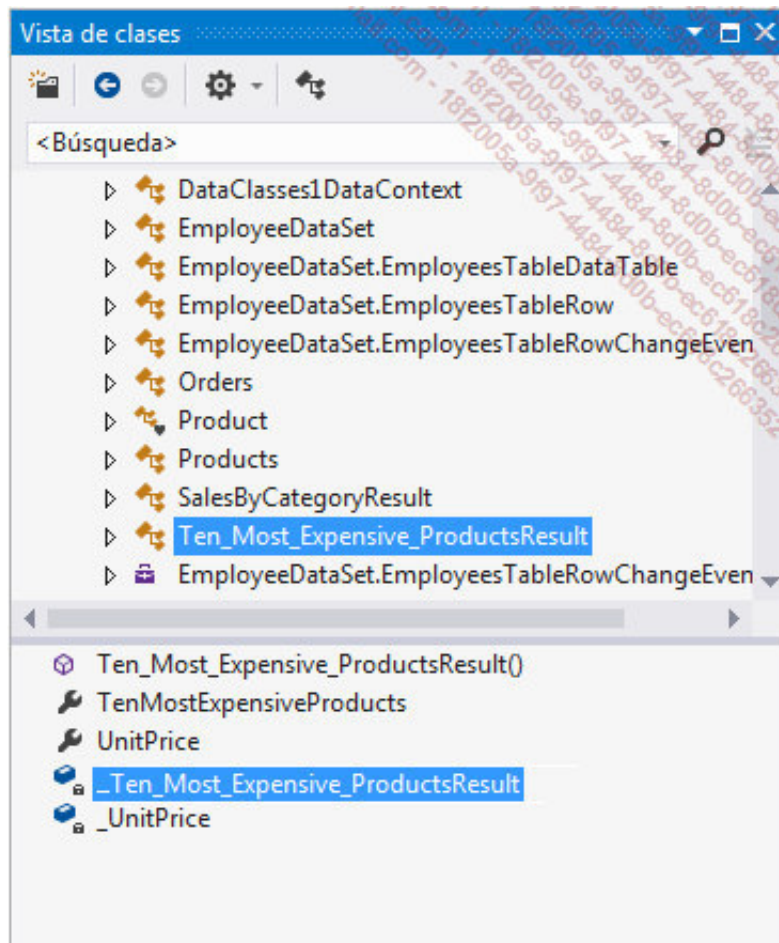


En efecto, los elementos devueltos por el procedimiento almacenado no son productos, sino simplemente el nombre y el precio del producto.

Por el contrario, si arrastramos y soltamos hacia la zona derecha del diseñador, la operación se realiza sin problema. La siguiente función se añade al DataContext.

```
[Function(Name="dbo.[Ten Most Expensive Products]")]
public ISingleResult<Ten_Most_Expensive_ProductsResult>
Ten_Most_Expensive_Products()
{
    IExecuteResult result = this.ExecuteMethodCall(this,
        ((MethodInfo) (MethodInfo.GetCurrentMethod())));
    return ((ISingleResult<Ten_Most_Expensive_ProductsResult>)
        (result.ReturnValue));
}
```

Esta función no devuelve una lista de productos, sino una lista de Ten_Most_Expensive_ProductsResult, que corresponde a una clase generada automáticamente en función de los datos devueltos por el procedimiento almacenado que tiene la estructura siguiente.



Para que la clase `Product` pueda usarse como tipo de retorno para la función, estamos obligados a modificar ligeramente el procedimiento almacenado de modo que devuelva los productos, y no solamente el nombre y el precio del producto.

```
set ANSI_NULLS ON
set QUOTED_IDENTIFIER ON
go
ALTER procedimiento [dbo].[Ten Most Expensive Products] AS
SET ROWCOUNT 10
SELECT *
FROM Products
ORDER BY Products.UnitPrice DESC
```

Ahora podemos volver a hacer la misma operación y obtener la siguiente función, que esta vez devuelve una lista de productos.

```
[Function(Name="dbo.[Ten Most Expensive Products]")]
public ISingleResult<Product> Ten_Most_Expensive_Products()
{
    IExecuteResult result = this.ExecuteMethodCall(this,
        ((MethodInfo) (MethodInfo.GetCurrentMethod())));
    return ((ISingleResult<Product>)
        (result.ReturnValue));
}
```

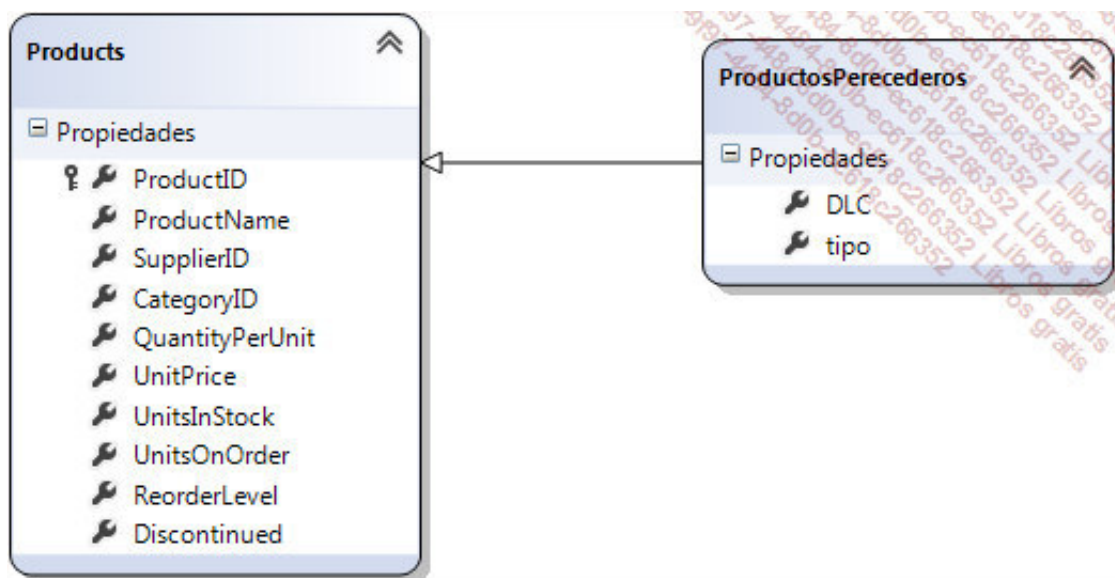
Herencia de clases

Como cualquier clase, las clases generadas por el diseñador Objeto/Relacional pueden utilizar la herencia. Sin embargo, para las bases de datos se trata de una noción perfectamente desconocida. Hay que recurrir a algunos trucos para simular esta técnica en una base de datos. La solución más utilizada consiste en crear una tabla única que contendrá a la vez los datos de los objetos de la clase base y la información de la subclase. Se añade una columna adicional a la tabla que sirva de discriminador. En función del valor de esta columna, es fácil determinar si se debe representar la línea con una instancia de la clase base o una instancia base. Vamos a modificar la tabla `Products` para poder aplicar esta técnica. Añadimos a la tabla una columna llamada `Perecedero`, de tipo entero. Esta columna nos va a servir de discriminador. Si el valor que contiene es igual a 1, se trata de un producto no perecedero. Si el valor es igual a 2, se trata de un producto perecedero. En este caso, la segunda columna, llamada `DLC`, de tipo fecha, contiene la fecha límite de consumo del producto. Para los productos no perecederos, esta columna no contiene ningún valor (null). Modifique la información de algunos productos para que se conviertan en productos perecederos (Manchego Pierrot, Caracoles de Burgoña, Mascarpone Fabioli...).

Ahora que la base de datos está lista, podemos añadir las clases al `DataContext`. La primera etapa consiste en añadir la tabla que constituye la clase básica. Luego añada un segundo ejemplar de esta tabla y vuelva a nombrar la clase correspondiente, que se convertirá en la clase derivada. Luego añada a partir del cuadro de herramientas una relación de herencia entre las dos clases dibujándola desde la clase hija hasta la clase padre. En cada una de las clases, suprima las propiedades inútiles. Por ejemplo, conservando sólo en la clase derivada la propiedad `DLC` y suprimiéndola en la clase base. Después de haber seleccionado la relación de herencia en el diagrama, debe modificar sus propiedades.

- Indique, por medio de la propiedad `Discriminator Property`, la propiedad que sirve para hacer la distinción entre una instancia de la clase base y una instancia de la subclase.
- Luego configure las propiedades `Base Class Discriminator Value` y `Derived Class Discriminator Value` con los valores de la propiedad configurada anteriormente que representa una instancia de la clase básica y una instancia de la subclase.
- La propiedad `Inheritance Default` indica qué clase se utilizará si el discriminador contiene un valor desconocido.

Obtenemos las siguientes clases:



Ahora que nuestras clases están disponibles, veamos cómo utilizarlas por medio de consultas LINQ.

c. Utilización de consultas LINQ to SQL

Las consultas LINQ to SQL utilizan de manera rigurosa la misma sintaxis que las que hemos estudiado en la sección Sintaxis del lenguaje LINQ. La única pequeña distinción proviene de los datos que, en este caso, son extraídos de la base de datos y transformados en instancias de clases a partir de los datos de mapeo. El `DataContext` se encarga totalmente del diálogo con la base de datos. Por lo tanto, hace falta crear una instancia `DataContext`; gracias a ella los datos estarán preparados para la ejecución de la consulta LINQ. A continuación presentamos nuestra primera consulta LINQ hacia la base de datos.

```
NorthWind dc;
dc = new NorthWind();
var consulta = from unCliente in dc.Customers
               where unCliente.ContactName.StartsWith("A")
               select unCliente;
foreach (var c in consulta)
{
    Console.WriteLine(c.ContactName);
}
```

En este código, la clase `NorthWind` corresponde al `DataContext` y gracias a ella los datos están disponibles para la consulta LINQ. Pero ¿cómo se seleccionan los datos?

En realidad, el método más natural para obtener datos provenientes de una base de datos consiste en pedir a ésta que ejecute una consulta SQL. Efectivamente, esta solución es la utilizada por LINQ. Para comprobarlo, podemos pedir al `DataContext` (`NorthWind` en nuestro caso) que visualice en la consola el código SQL que genera automáticamente. Para ello, basta simplemente inicializar la propiedad `Log` del `DataContext` en dirección de la consola antes de crear la consulta LINQ.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
var consulta = from unCliente in dc.Customers
               where unCliente.ContactName.StartsWith("A")
               select unCliente;
foreach (var c in consulta)
{
    Console.WriteLine(c.ContactName);
}
```

Durante la ejecución, obtenemos lo siguiente:

```
SELECT [t0].[CustomerID], [t0].[CompanyName], [t0].[ContactName],
[t0].[Contact Title], [t0].[Address], [t0].[City], [t0].[Region],
[t0].[Postal Code], [t0].[Country], [t0].[Phone], [t0].[Fax]
FROM [dbo].[Customers] AS [t0]
WHERE [t0].[ContactName] LIKE @p0
-- @p0: Input NVarChar (Size = 2; Prec = 0; Scale = 0) [A%]
-- Context: SqlProvider(Sql2005) Model: AttributedMetaModel
Build: 3.5.21022.8
Ana Trujillo
Antonio Moreno
Ann Devon
Aria Cruz
André Fonseca
Annette Roulet
Alexander Feuer
Alejandra Camino
Art Braunschweiger
```

Efectivamente, tenemos una consulta SQL con parámetros creada de forma automática por elDataContext. Esta consulta no es muy complicada y se hubiese escrito con facilidad usando directamente ADO.NET. Intentemos ejecutar otra consulta retomando la que nos permitía obtener las fechas de pedidos de cada uno de los clientes.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
var consultaUnion3 = from unCliente in dc.Customers
                      join unPedido in dc.Orders on unCliente.CustomerID
                      equals unPedido.CustomerID into PedidosDelCliente
                      select new { unCliente, PedidosDelCliente };
foreach (var r in consultaUnion3)
{
    Console.WriteLine(r.unCliente.ContactName);
    foreach (var pdo in c.PedidosDelCliente)
    {
        Console.WriteLine("\t" + pdo.OrderDate);
    }
}
```

Ahí tenemos el código SQL generado para la ejecución de esta consulta:

```
SELECT [t0].[CustomerID], [t0].[CompanyName], [t0].[ContactName],
[t0].[ContactTitle], [t0].[Address], [t0].[City], [t0].[Region],
[t0].[PostalCode], [t0].[Country], [t0].[Phone], [t0].[Fax],
[t1].[OrderID], [t1].[CustomerID] AS [CustomerID2], [t1].[EmployeeID],
[t1].[OrderDate], [t1].[RequiredDate], [t1].[ShippedDate], [t1].[ShipVia],
[t1].[Freight], [t1].[ShipName], [t1].[ShipAddress], [t1].[ShipCity],
[t1].[ShipRegion], [t1].[ShipPostalCode], [t1].[ShipCountry], (
    SELECT COUNT(*)
    FROM [dbo].[Orders] AS [t2]
    WHERE [t0].[CustomerID] = [t2].[CustomerID]
) AS [value]
FROM [dbo].[Customers] AS [t0]
LEFT OUTER JOIN [dbo].[Orders] AS [t1] ON [t0].[CustomerID] = [t1].[CustomerID]
ORDER BY [t0].[CustomerID], [t1].[OrderID]
-- Context: SqlProvider(Sql2005) Model: AttributedMetaModel Build: 3.5.21022.8
```

Empieza a complicarse de manera seria. La escritura de una consulta así directamente en SQL exigiría con toda seguridad un buen dominio de este lenguaje, mientras que la sintaxis LINQ sigue resultando muy simple. Efectivamente, la potencia de LINQ to SQL reside en este nivel.

Esta facilidad no se limita a la extracción de datos desde la base de datos, ya que LINQ to SQL también es capaz de gestionar las actualizaciones de los datos hacia la base de datos.

d. Actualización de los datos

La actualización de la base de datos se realiza también de forma muy sencilla, sólo manejando objetos y sin escribir ni una línea de SQL.

Modificación de datos existentes

La primera etapa consiste en obtener los datos que se desea modificar ejecutando una consulta de

selección ordinaria. Una vez que los datos están disponibles en forma de instancias de clases, podemos modificar simplemente las propiedades de estas instancias. Para transferir las modificaciones en la base de datos, basta simplemente pedir al DataContext que propague las modificaciones hacia la base de datos. Vamos a probar esta técnica moviendo a nuestros clientes barceloneses hacia Valencia.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
var clientesBarceloneses = from unCliente in dc.Customers
                           where unCliente.City=="Barcelona"
                           select unCliente;
foreach (var unCliente in clientesBarceloneses)
{
    unCliente.City = "Valencia";
    unCliente.PostalCode = "44800";
}
dc.SubmitChanges();
```

En este código, le corresponde a la instrucción SubmitChanges del DataContext provocar la actualización de la base de datos ejecutando automáticamente la siguiente consulta SQL Update para cada objeto que haya sido modificado.

```
UPDATE [dbo].[Customers]
SET [City] = @p10, [PostalCode] = @p11
WHERE ([CustomerID] = @p0) AND ([CompanyName] = @p1) AND ([ContactName] = @p2) A
ND ([ContactTitle] = @p3) AND ([Address] = @p4) AND ([City] = @p5) AND ([Region]
IS NULL) AND ([PostalCode] = @p6) AND ([Country] = @p7) AND ([Phone] = @p8) AND
([Fax] = @p9)
-- @p0: Input NChar (Size = 5; Prec = 0; Scale = 0) [FRANR]
-- @p1: Input NVarChar (Size = 19; Prec = 0; Scale = 0) [España restauración]
-- @p2: Input NVarChar (Size = 14; Prec = 0; Scale = 0) [Carine Schmitt]
-- @p3: Input NVarChar (Size = 17; Prec = 0; Scale = 0) [Marketing Manager]
-- @p4: Input NVarChar (Size = 14; Prec = 0; Scale = 0) [calle real 54]
-- @p5: Input NVarChar (Size = 6; Prec = 0; Scale = 0) [Barcelona]
-- @p6: Input NVarChar (Size = 5; Prec = 0; Scale = 0) [08000]
-- @p7: Input NVarChar (Size = 6; Prec = 0; Scale = 0) [España]
-- @p8: Input NVarChar (Size = 11; Prec = 0; Scale = 0) [40.32.21.21]
-- @p9: Input NVarChar (Size = 11; Prec = 0; Scale = 0) [40.32.21.20]
-- @p10: Input NVarChar (Size = 14; Prec = 0; Scale = 0) [Valencia]
-- @p11: Input NVarChar (Size = 5; Prec = 0; Scale = 0) [46000]
-- Context: SqlProvider(Sql2005) Model: AttributedMetaModel Build: 3.5.21022.8
```

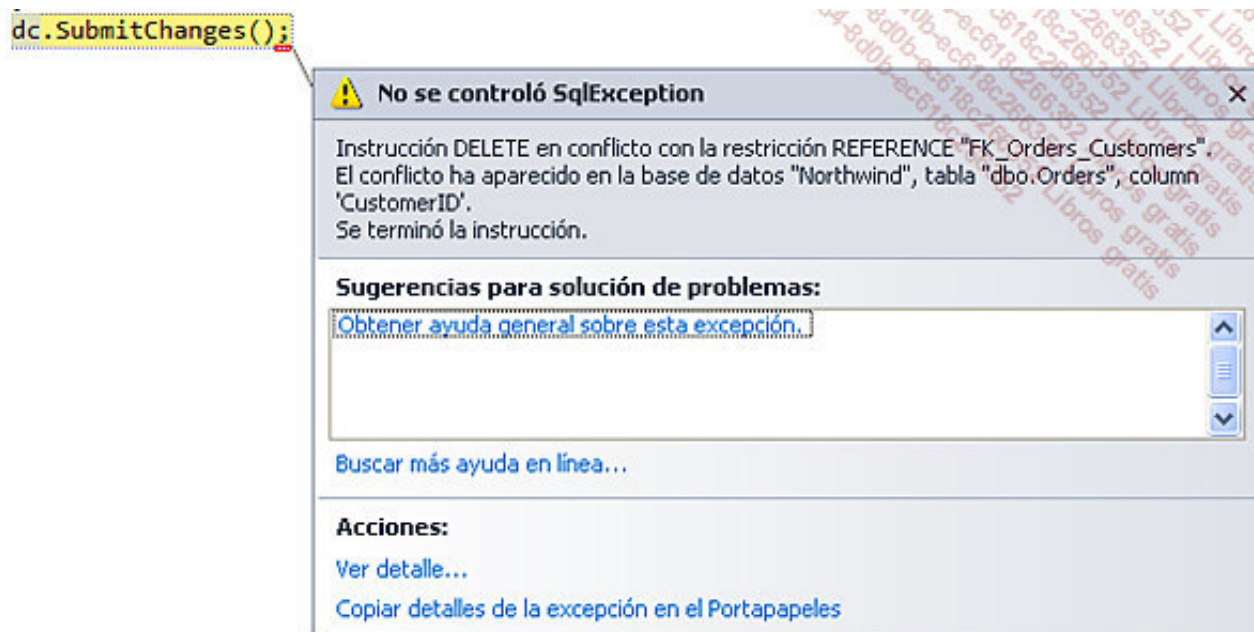
Supresión de datos

Al igual que con la modificación, debemos obtener previamente los elementos que deseamos suprimir ejecutando una consulta de selección, y luego indicar para cada uno de ellos que deseamos suprimirlos. Para ello, llamamos al método DeleteOnSubmit de la tabla a la cual pertenece el elemento, pasándole como parámetro el objeto que se debe suprimir. Para validar las supresiones, debemos llamar luego al método SubmitChanges del DataContext. Vamos a probar esto suprimiendo los clientes brasileños de la base de datos.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
var supresionCliente = from unCliente in dc.Customers
```

```
        where unCliente.Country == "Brazil"
        select unCliente;
foreach (var unCliente in supresionCliente)
{
    dc.Customers.DeleteOnSubmit(unCliente);
}
dc.SubmitChanges();
```

Cuando se ejecute este código, obtenemos esta magnífica excepción.



Con la prisa, hemos olvidado un pequeño detalle. En la base de datos, las tablas Customer, Order y OrderDetail están vinculadas por restricciones de clave foránea. Por lo tanto, es imposible suprimir un cliente si posee todavía pedidos, y de la misma manera es imposible suprimir un pedido si contiene todavía detalles. El problema viene del hecho de que LINQ no es capaz de gestionar las supresiones en cascada. Para resolver nuestro problema, tenemos dos soluciones:

- Activar la regla ON DELETE CASCADE en las restricciones de clave foránea.
- Gestionar nosotros mismos la supresión de los objetos antes de la supresión de los objetos padres.

Esta última solución es la que vamos a utilizar. Ya que nuestro modelo de objeto ha sido diseñado correctamente, esta solución es muy fácil de poner en práctica. En efecto, en la clase Customers, tenemos la colección Orders, que representa los pedidos del cliente. Del mismo modo, en la clase Orders, tenemos la colección Order_Details, que representa todos los detalles del pedido. Sólo hace falta ejecutar tres bucles anidados que supriman los detalles de cada pedido, los pedidos de cada cliente y luego los propios clientes.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
var supresionClient= from unCliente in dc.Customers
                     where unCliente.Country == "Brazil"
                     select unCliente;
foreach (var unCliente in supresionCliente)
{
    foreach (var unPedido in unCliente.Orders)
```

```
{
    foreach (var unaFila in unPedido.Order_Details)
    {
        dc.Order_Details.DeleteOnSubmit(unaFila);
    }
    dc.Orders.DeleteOnSubmit(unPedido);
}
dc.Customers.DeleteOnSubmit(unCliente);
}
dc.SubmitChanges();
```

Con esta solución, ya no hay problemas y nuestros clientes brasileños están borrados correctamente de la base de datos.

Inserción de datos

La inserción de datos se realiza en tres etapas. Primero hace falta crear una instancia de la clase que representa los datos que se van a insertar en la base de datos. Luego se inicializan las propiedades de estas instancias con los valores que se desea añadir en la base de datos. El objeto así configurado debe insertarse luego en la tabla del `DataContext`. Finalmente, las modificaciones se transfieren hacia la base de datos. Para ilustrar estas etapas, vamos a añadir un nuevo cliente en la base de datos.

```
NorthWind dc;
dc = new NorthWind();
dc.Log = Console.Out;
Customer nuevoCliente;
nuevoCliente = new Customer();
nuevoCliente.CustomerID = "JGARC";
nuevoCliente.ContactName = "José García";
nuevoCliente.CompanyName = "ENI";
nuevoCliente.ContactTitle = "Formador";
nuevoCliente.Country = "España";
nuevoCliente.City = "Valencia";
nuevoCliente.Address = "calle Benjamin Franklin";
nuevoCliente.Fax = "916.090.900";
nuevoCliente.Phone = "916.090.901";
nuevoCliente.PostalCode = "46070";
dc.Customers.InsertOnSubmit(nuevoClient);
dc.SubmitChanges();
```

e. Conflictos de las actualizaciones

Ocurre a menudo que varios usuarios trabajan simultáneamente en la misma base de datos. Se puede producir conflictos cuando los mismos registros de la base de datos son actualizados por varios usuarios. LINQ propone un mecanismo que permite tratar este problema. Este mecanismo se descompone en cuatro etapas:

- Configurar para qué datos vigilará los conflictos la base de datos.
- Detectar la aparición de un conflicto.
- Obtener información relativa al conflicto.
- Resolver el conflicto.

Configuración de las clases para la detección de los conflictos

Durante la creación de las clases con el diseñador Objeto/Relacional, podemos indicar para cada

propiedad si debe ser incluida en el mecanismo de detección de los conflictos. Cada miembro de la clase generada posee una propiedad `Verificación de las actualizaciones` a la cual podemos asignar tres valores diferentes:

- **Siempre:** la detección de los conflictos está siempre activa para este elemento.
- **WhenChanged:** activa la detección únicamente si el valor ha sido modificado.
- **Nunca:** no se tiene en cuenta este elemento para la detección de los conflictos.

Por defecto, todas las propiedades se utilizan para la detección de los conflictos.

Detección de los conflictos

Los conflictos surgen durante el traslado de la información hacia la base de datos. Por lo tanto, debemos intervenir a este nivel. Para ello, durante la llamada del método `SubmitChanges` del `DataContext`, indicamos pasando un parámetro a este método cómo se debe comportar el mecanismo de detección de los conflictos. Dos soluciones son posibles:

- **FailOnFirstConflict:** señala el problema en cuanto interviene el primer conflicto.
- **ContinueOnConflict:** intenta efectuar todas las actualizaciones y señala al final si ha habido algún conflicto.

Si se detectase un conflicto, se activaría una excepción tipo `ChangeConflictException`. Por lo tanto, se debe ubicar la llamada del método `SubmitChanges` en un bloque try catch para recuperar la excepción.

Obtener la información relativa a los conflictos

En el bloque catch, podemos obtener información relativa a los conflictos recorriendo la colección `ChangeConflicts` del `DataContext`. Esta colección contiene uno o varios objetos de tipo `ObjectChangeConflict`. La propiedad `Object` nos permite obtener una referencia sobre el elemento situado en el origen del problema. La propiedad `MemberConflicts` facilita la lista de todos los miembros de este objeto que se hallan en el origen del problema.

Para cada uno, tenemos a nuestra disposición el valor de origen en el momento de crear la instancia de la clase a partir de la información de la base de datos, el valor actual para la instancia de la clase, el valor actual en la base de datos.

Resolver los conflictos

Para resolver los conflictos ocurridos durante la actualización de la base de datos, se pueden considerar tres hipótesis.

- Reemplazar los valores de las propiedades en conflicto con la información presente en la base de datos. Para esto, se debe llamar al método `Resolve` del objeto `ObjectChangeConflict` pasándole la constante `Overwrite CurrentValues`.
- Reemplazar los valores de la base de datos por la información contenida en las propiedades del objeto. De la misma manera que para la solución anterior, debemos llamar al método `Resolve` del objeto `ObjectChangeConflict` pasándole esta vez la constante `KeepCurrentValues`.
- Fusionar las propiedades del objeto con la información de la base de datos. Se modifica la información de la base de datos sólo si la propiedad correspondiente al objeto ha sido modificada. En este caso, se debe llamar al método `Resolve` con la constante `KeepChanges`.

El siguiente código le permite probar estas soluciones simplemente reemplazando la constante

durante la llamada al método `Resolve`.

```
Customer cl=null;
NorthWind dc;
dc = new NorthWind();
var cns = from unCliente in dc.Customers
where unCliente.CustomerID == "BOLID"
select unCliente;
foreach (var c in cns)
{
    c.City = "Barcelona";
    cl = c;
}
Console.WriteLine("modificar el codigo postal del cliente BOLID
en la base luego apretar una tecla");
Console.ReadLine();
try
{
    dc.SubmitChanges(ConflictMode.FailOnFirstConflict);
}
catch (ChangeConflictException ex)
{
    foreach(var o in dc.ChangeConflicts)
    {
        o.Resolve(RefreshMode.KeepChanges);
        // o.Resolve(RefreshMode.KeepCurrentValues);
        // o.Resolve(RefreshMode.OverwriteCurrentValues);
    }
}
Console.WriteLine("el objeto cliente:");
Console.WriteLine("city:" + cl.City);
Console.WriteLine("postalCode:" + cl.PostalCode);
foreach (var cli in cns)
{
    Console.WriteLine("el cliente en la base:");
    Console.WriteLine("city:" + cli.City);
    Console.WriteLine("postalCode:" + cli.PostalCode);
}
```

Presentación

El lenguaje XML (*eXtensible Markup Language*) es un lenguaje que permite la representación de datos. Permite encapsular cualquier tipo de datos y representarlos en forma de árbol. Éstos están escritos entre etiquetas o como atributos. Este formato permite escribir datos, pero no permite darles formato ni tampoco utilizarlos. Se usa principalmente para permitir el intercambio de datos entre aplicaciones e incluso entre sistemas diferentes. También se utiliza a menudo como formato de almacenamiento para los parámetros de configuración de una aplicación. Visual Studio y Windows lo utilizan con este objetivo de manera corriente. Este lenguaje ha sido diseñado por el W3C (*World Wide Web Consortium*); en el sitio <http://www.w3.org/XML> podrá obtener más detalles de las especificaciones de este lenguaje.

El lenguaje XML se confunde a menudo con el lenguaje HTML. Aunque comparten similitudes, estos dos lenguajes no tienen la misma vocación. A continuación presentamos los puntos comunes entre los lenguajes XML y HTML:

- Estos dos lenguajes se presentan en forma de «texto plano».
- Se representa el contenido de los documentos a través de etiquetas.
- Estas etiquetas pueden tener atributos.
- Se pueden anidar las etiquetas unas en el interior de otras.
- Estos dos lenguajes provienen de una misma base: el SGML (*Standard Generalized Markup Language*).

El lenguaje XML se distingue del lenguaje HTML en los puntos siguientes:

- El lenguaje XML autoriza la creación de sus propias etiquetas.
- Las herramientas encargadas del tratamiento gestionan la sintaxis de manera más rigurosa.
- HTML es un lenguaje diseñado para la presentación de los datos. Por su parte, XML se utiliza para la descripción de los datos.

Para poder trabajar con ellos fácilmente, los datos XML deben confiarse a un procesador XML.

Un procesador XML es un módulo software especialmente escrito para manejar XML. El uso de un módulo externo para el tratamiento XML se explica por la complejidad que representa el desarrollo de un procesador XML totalmente funcional. Efectivamente, para que un procesador XML pueda considerarse totalmente funcional, su funcionamiento debe seguir las evoluciones del lenguaje definidas por el W3C. Por lo tanto, es importante visitar regularmente el sitio de Microsoft para verificar si existe una versión más reciente del procesador XML que la instalada en su máquina.

Estructura de un documento XML

Antes de trabajar con los documentos XML usando Visual C#, es importante entender correctamente la estructura de este tipo de documento. Los siguientes párrafos van a presentar las nociones elementales que se deben conocer antes de lanzarse a la utilización de documentos XML.

1. Partes constituyentes de un documento XML

Un documento XML se puede constituir a partir de los siguientes elementos:

Instrucción de tratamiento

Las intrucciones de tratamiento permiten incorporar en un documento XML información destinada al procesador XML o a otras aplicaciones antes de editar el documento. Se utilizan estas instrucciones para facilitar una instrucción especial a una aplicación que trabaja en el documento.

Se inserta la instrucción de tratamiento en el documento con la siguiente sintaxis:

```
< ?nombreAplicacion instruccion ?>
```

La primera parte es el nombre de la aplicación a la cual se destina esta instrucción. La segunda parte es el texto de la instrucción.

Un documento XML contiene, en general, una instrucción de tratamiento especial para definir la versión de XML que conforma el documento y la codificación de los caracteres utilizada por el documento.

```
<?xml version="1.0" encoding="utf-8" ?>
```

Comentarios

Los comentarios sirven para incluir en el documento la información destinada a los usuarios de éste. Son ignorados por el procesador XML o por las aplicaciones que usan el documento. No se deben incorporar a una etiqueta.

La siguiente sintaxis se debe utilizar para ubicar un comentario en el documento.

```
<!--esto es un comentario-->
```

En el interior del comentario, la utilización de los caracteres -- está prohibida:

```
<!--esto es -- un comentario-->
```

Caracteres reservados

Algunos caracteres quedan reservados para el lenguaje XML, como por ejemplo el carácter & del siguiente ejemplo:

```
<menu>Queso & Postres </menu>
```

Para poder utilizar estos caracteres en un documento XML, debe sustituirlos por la siguiente sintaxis:

Carácter	Utilizar en lugar de
&	&
<	<
>	>
'	'
"	"

Por lo tanto, la sintaxis correcta es:

```
<menu>queso &amp; postre</menu>
```

Secuencias de caracteres más largas se pueden incorporar usando una sección CDATA. La sintaxis es la siguiente:

```
<![CDATA[{ Select * from postres where precio < 10} and calorias > 500]]>
```



Con esta sintaxis se puede utilizar cualquier carácter sin necesidad de tomar más precauciones.

Elementos XML

Un elemento XML es un contenedor que acoge datos y otros elementos. Se compone de una etiqueta de principio y de una etiqueta de fin. La sintaxis de un elemento XML es la siguiente:

```
<NombreElemento>contenido</NombreElemento>
```

Los elementos deben respetar algunas reglas relativas a su forma:

- Los nombres de elementos no pueden contener espacios.
- No pueden empezar por un número o un signo de puntuación.
- No pueden empezar con xml (sea cual sea la caja de la letra).
- Deben empezar justo después del signo >, sin espacio.
- Las etiquetas de principio y fin deben usar de forma idéntica las mayúsculas y las minúsculas.
- Un documento XML debe contener al menos un elemento: el elemento raíz.
- Todos los elementos que siguen el elemento raíz deben anidarse en éste.
- Si un elemento no tiene contenido, puede estar constituido únicamente por una etiqueta de fin.
- Sólo el elemento raíz debe tener una etiqueta de principio y una de fin, incluso aunque no tenga contenido.

Ejemplo:

```
<?xml version="1.0" encoding="utf-8" ?>
```

```
<restaurante>
<menu>
  <entradas>
    <nombre>rábanos</nombre>
    <nombre>pasta</nombre>
    <nombre>salchichón</nombre>
  </entradas>
  <platos>
    <nombre>paella</nombre>
    <nombre>fideuá</nombre>
    <nombre>cuscús</nombre>
  </platos>
  <quesos>
    <nombre>manchego</nombre>
    <nombre>tetilla</nombre>
    <nombre>cabrales</nombre>
  </quesos>
  <postres>
    <nombre>helado</nombre>
    <nombre>tarta</nombre>
    <nombre>crema catalana</nombre>
  </postres>
</menu>
</restaurante>
```

Atributos de elementos

Se utilizan los atributos de elementos para calificar un elemento. Se ubican en la etiqueta de principio del elemento. Como los elementos, deben seguir unas reglas:

- Un atributo se compone de un nombre y de una asignación de valor.
- Un elemento puede contener un número cualquiera de atributos.
- Los nombres de atributos están separados por espacios.
- Un nombre de atributo sólo puede aparecer una vez en un elemento.
- Un nombre de atributo puede aparecer en varios elementos.
- Un nombre de atributo no puede contener espacios.
- La asignación de un valor a un atributo se hace con el signo igual seguido del valor, rodeado de comillas dobles.

Ejemplo:

```
<?xml version="1.0" encoding="utf-8" ?>
<restaurante>
<menu type="gastronomico">
  <entradas>
    <nombre calorías="50">rábanos</nombre>
    <nombre calorías="300">pasta</nombre>
    <nombre calorías="350">salchichón</nombre>
  </entradas>
  <platos>
    <nombre calorías="1000">paella</nombre>
    <nombre calorías="2000">fideuá</nombre>
    <nombre calorías="1700">cuscús</nombre>
  </platos>
  <quesos>
```

```

    <nombre calorías="240">manchego</nombre>
    <nombre calorías="300">tetilla</nombre>
    <nombre calorías="120">cabrales</nombre>
  </quesos>
  <postres>
    <nombre calorías="340" sabor="chocolate">helado</nombre>
    <nombre calorías="250" frutas="manzanas">tarta</nombre>
    <nombre calorías="400">crema catalana</nombre>
  </postres>
</menu>
</restaurante>

```

Espacios de nombres

Un espacio de nombres es un conjunto de nombres de elementos identificados por una referencia única. Permiten evitar las confusiones cuando unos datos XML se fusionan a partir de diferentes fuentes.

Tomemos el siguiente ejemplo, que podría ser un archivo de configuración de una aplicación:

```

<?xml version="1.0" encoding="utf-8" ?>
<aplicacion>
  <menu nombre="archivo">
    <entradas>nuevo</entradas>
    <entradas>abrir</entradas>
    <entradas>cerrar</entradas>
  </menu>
  <menu nombre="edicion">
    <entradas>copiar</entradas>
    <entradas>cortar</entradas>
    <entradas>pegar</entradas>
  </menu>
</aplicacion>

```

En este archivo, tenemos elementos que ya habíamos definido en otro archivo. Es obvio que los elementos **menu** y **entradas** no tienen el mismo significado que en el archivo utilizado anteriormente. Para evitar toda ambigüedad, hay que añadir en cada uno de los archivos una definición de espacio de nombres que convierta cada elemento en único. La definición de un espacio de nombres se efectúa con el atributo `xmlns` seguido de un prefijo y del identificador del espacio de nombres.

La sintaxis es la siguiente para cada uno de nuestros dos archivos:

```

<restaurante xmlns:restaurante="http://www.eni-escuela.es/restaurante">
<aplicacion xmlns:appli="http://www.eni-escuela.es/configappli">

```

Es muy importante que los identificadores de espacios de nombres sean únicos si se desea intercambiar información con otras personas. Por eso, es habitual utilizar el nombre de dominio de la empresa en el identificador (se supone que éste es único). Con esta modificación, podemos usar en el mismo archivo elementos **menu** y **entradas**, añadiendo delante el prefijo del espacio nombres en el cual tienen un significado.

```

<fusion xmlns:appli="http://www.eni-escuela.es/configappli"
xmlns:restaurante="http://www.eni-escuela.es/restaurante">
  <appli:menu nombre="archivo">
    <appli:entradas>registrar</appli:entradas>
  </appli:menu>
  <restaurante:menu nombre="economico">

```

```
<restaurante:entradas>aguacate</restaurante:entradas>  
</restaurante:menu>  
</fusion>
```

2. Documento bien formado y documento válido

Gracias al lenguaje XML, tenemos la posibilidad de crear fácilmente documentos estructurados y comprensibles. Existen dos nociones que permiten verificar la calidad de un documento XML: un documento puede estar bien formado y un documento puede ser válido.

a. Documento bien formado

Un documento está bien formado si obedece a las reglas sintácticas del lenguaje XML. Estas reglas son mucho menos estrictas que las reglas de validez. Gestionan las atribuciones de nombres, las creaciones y las relaciones entre elementos. Para poder ser tratado por un procesador XML, un documento debe estar bien formado. Si el procesador detecta un error, detiene inmediatamente el tratamiento del documento.

b. Documento válido

Un documento válido es un documento XML que tiene vinculada una DTD o un esquema XSD (definición del tipo de documento) y respeta todas las reglas de construcción definidas en esta última. Cuando un procesador XML analiza el documento, busca en la DTD o en el esquema XSD una definición para cualquier elemento, atributo, entidad de este documento. En cuanto encuentra un error, detiene el tratamiento.

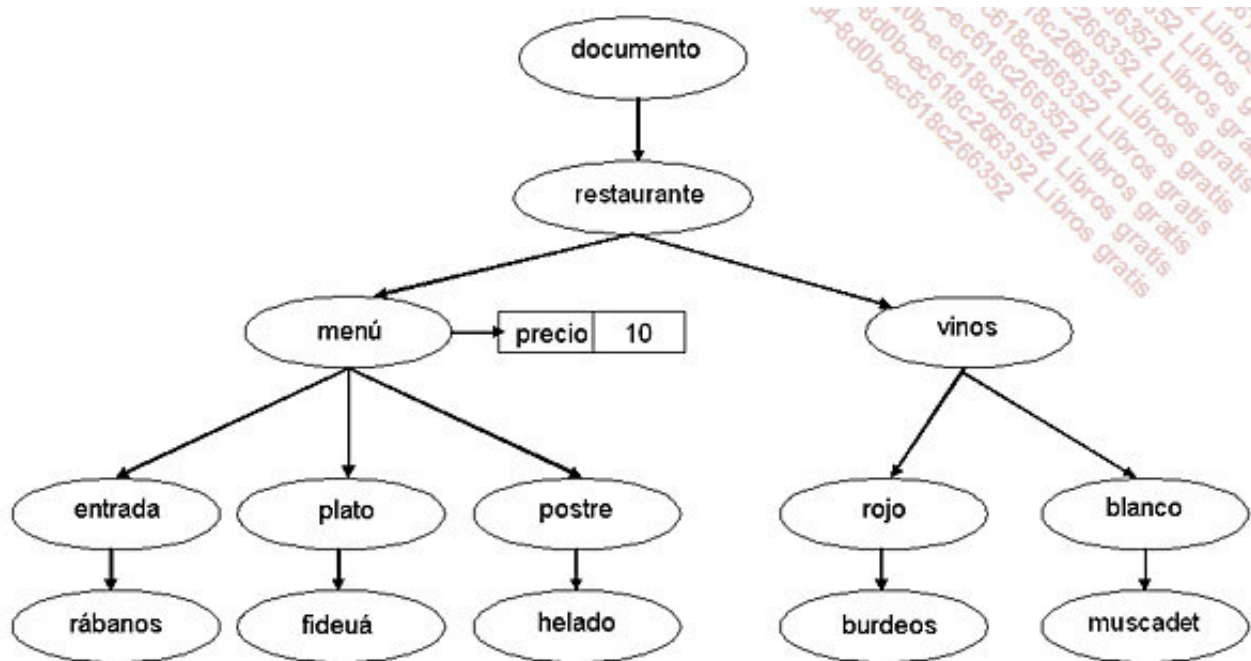
Manejo de un documento XML

El manejo de un documento XML en una aplicación C# viene facilitada por la utilización de DOM (*Document Object Model*). El DOM le permite leer, trabajar y modificar un documento XML por programa. Este último gobierna la representación en memoria de los datos XML, aunque los verdaderos datos XML estén almacenados de manera lineal cuando se encuentran en un archivo o provienen de otro objeto.

Por ejemplo, el siguiente documento:

```
<?xml version="1.0"?>
<restaurante>
  <menu precio="10">
    <entrada>rábanos</entrada>
    <plato>fideuá</plato>
    <postre>helado</postre>
  </menu>
  <vinos>
    <tinto>burdeos</tinto>
    <blanco>muscadet</blanco>
  </vinos>
</restaurante>
```

Se representa con esta forma en memoria en una aplicación:



En la estructura de un documento XML, cada círculo de esta ilustración representa un nodo, llamado objeto `XmlNode`, que es el objeto básico del árbol DOM. La clase `XmlDocument` se encarga de los métodos destinados a ejecutar las operaciones sobre el documento en su conjunto, por ejemplo para cargarlo en memoria o grabarlo en forma de archivo. Los objetos `XmlNode` comportan un conjunto de métodos y de propiedades, así como características básicas bien definidas. A continuación, presentamos algunas de estas características:

- Un nodo sólo puede poseer un nodo padre, que es el nodo situado justo encima de él.
- El único nodo que no tiene padre es la raíz del documento, ya que se trata del nodo de primer nivel que contiene el propio documento y los fragmentos de documento.

- La mayoría de los nodos pueden comportar varios nodos hijos, que son los nodos situados directamente bajo ellos.
- Los nodos situados en el mismo nivel, representados en el diagrama por los nodos menú y vinos, son nodos hermanos.

Una de las características del DOM es su manera de gestionar los atributos. Los atributos no son nodos que formen parte de las relaciones padre-hijo ni hermano. Se consideran una propiedad del nodo y están formados por un par, compuesto de un nombre y de un valor.

En nuestro ejemplo, `precio="10"` asociado al elemento `menú`, la palabra `precio` corresponde al nombre y el valor del atributo `precio` es 10. Para extraer el atributo `precio="10"` del nodo `menú`, se llama al método `GetAttribute` cuando el cursor se encuentra en el nodo `menú`.

Para los ejemplos siguientes, utilizaremos este documento XML:

```
<?xml version="1.0" encoding="utf-8" ?>
<restaurante>
  <menu tipo="gastronomico">
    <entradas>
      <nombre calorias="50">rábanos</nombre>
      <nombre calorias="300">pasta</nombre>
      <nombre calorias="350">salchichón</nombre>
    </entradas>
    <platos>
      <nombre calorias="1000">paella</nombre>
      <nombre calorias="2000">fideuá</nombre>
      <nombre calorias="1700">cuscús</nombre>
    </platos>
    <quesos>
      <nombre calorias="240">manchego</nombre>
      <nombre calorias="300">tetilla</nombre>
      <nombre calorias="120">cabrales</nombre>
    </quesos>
    <postres>
      <nombre calorias="340" sabor="chocolate">helado</nombre>
      <nombre calorias="250" frutas="manzanas">tarta</nombre>
      <nombre calorias="400">crema catalana</nombre>
    </postres>
  </menu>
  <menu tipo="economico">
    <entradas>
      <nombre calorias="50">pan</nombre>
    </entradas>
    <platos>
      <nombre calorias="1700">jamón</nombre>
    </platos>
    <quesos>
      <nombre caloria="240">manchego</nombre>
    </quesos>
    <postres>
      <nombre calorias="340" sabor="fresa">helado</nombre>
    </postres>
  </menu>
</restaurante>
```

1. Utilización de DOM

La primera etapa durante la utilización de DOM consiste en cargar el documento XML en un árbol de nodos DOM. Para ello, debe declarar un objeto `XmlDocument` y luego utilizar el método `Load` con la

finalidad de rellenar este objeto a partir de un archivo XML.

```
XmlDocument doc;
doc = new XmlDocument();
doc.Load("restaurante.xml");
```

También es posible cargar datos XML a partir de una cadena de caracteres. En este caso, debe utilizar el método `LoadXML` y facilitar la cadena de caracteres que contiene los datos XML.

Una vez los datos XML estén cargados en el árbol, puede localizar nodos particulares con el fin de someterlos a operaciones de tratamiento o modificación. El método `GetElementsByTagName` permite obtener un objeto `XmlNodeList`, que contiene los nodos afectados. Entonces puede obtener los atributos del nodo usando la propiedad `Attributes` o comprobar si posee nodos hijos con la propiedad `HasChildNodes`. Si es el caso, tiene acceso a estos nodos a través de la propiedad `ChildNodes` en forma de un objeto `XmlNodeList`.

El siguiente ejemplo busca los nodos menú en el árbol y visualiza el atributo `type`.

```
XmlNodeList menus;
menus=doc.GetElementsByTagName("menu");
foreach( XmlNode unMenu in menus)
{
    Console.WriteLine(unMenu.Attributes["type"].Value);
}
```

También se pueden modificar las características de los nodos añadiéndoles un atributo. Los nodos pueden recibir, por ejemplo, un atributo **precio**.

```
menus = doc.GetElementsByTagName("menu");
XmlAttribute att;
foreach ( XmlNode unMenu in menus)
{
    if (unMenu.Attributes["type"].Value == "gastronomico")
    {
        att = doc.CreateAttribute("precio");
        att.Value = "50€";
        unMenu.Attributes.Append(att);
    }
    if (unMenu.Attributes["type"].Value == "economico")
    {
        att = doc.CreateAttribute("precio");
        att.Value = "15€";
        unMenu.Attributes.Append(att);
    }
}
```

También es posible añadir nodos hijos a nodos que existen en el árbol, creando instancias de la clase `XmlNode` y uniéndolos a su nodo padre. El siguiente ejemplo añade un **digestivo** al menú **gastronomico**.

```
menus = doc.GetElementsByTagName("menu");
XmlAttribute att;
foreach ( XmlElement unMenu in menus)
{
    if (unMenu.Attributes["tipo"].Value == "gastronomico")
    {
```

```

        XmlNode n1;
        XmlNode n2;
        XmlNode n3;
        n1 = doc.CreateNode(XmlNodeType.Element, "digestivo", "");
        n2 = doc.CreateNode(XmlNodeType.Element, "nombre", "");
        n3 = doc.CreateNode(XmlNodeType.Text, "", "");
        n3.Value = "Cognac";
        n2.AppendChild(n3);
        n1.AppendChild(n2);
        unMenu.AppendChild(n1);
    }
}

```

Después de la ejecución de los dos ejemplos anteriores, el documento XML debe presentar la siguiente forma:

```

<?xml version="1.0" encoding="Windows-1252"?>
<restaurante>
  <menu tipo="gastronomico" precio="50€">
    <entradas>
      <nombre calorías="50">rábanos</nombre>
      <nombre calorías="300">pasta</nombre>
      <nombre calorías="350">salchichón</nombre>
    </entradas>
    <platos> <nombre calorías="1000">paella</nombre>
      <nombre calorías="2000">fideuá</nombre>
      <nombre calorías="1700">cuscús</nombre>
    </platos>
    <quesos>
      <nombre calorías="240">manchego</nombre>
      <nombre calorías="300">tetilla</nombre>
      <nombre calorías="120">cabrales</nombre>
    </quesos>
    <postres>
      <nombre calorías="340" sabor="chocolate">helado</nombre>
      <nombre calorías="250" frutas="manzanas">tarta</nombre>
      <nombre calorías="400">crema catalana</nombre>
    </postres>
    <digestivo>
      <nombre>Cognac</nombre>
    </digestivo>
  </menu>
  <menu tipo="economico" precio="15€">
    <entradas>
      <nombre calorías="50">pan</nombre>
    </entradas>
    <platos>
      <nombre calorías="1700">jamón</nombre>
    </platos>
    <quesos>
      <nombre calorías="240">manchego</nombre>
    </quesos>
    <postres>
      <nombre calorías="340" sabor="fresa">helado</nombre>
    </postres>
  </menu>
</restaurante>

```

En realidad, sólo se modifica la representación en memoria del documento XML. Si desea conservar las

modificaciones, debe registrar el documento en un archivo para asegurar la persistencia de los datos. Para ello, debe utilizar el método `save` de la clase `XmlDocument`, y facilitar el nombre del archivo en el cual desea efectuar la copia de seguridad.

```
doc.Save("restaurante2.xml");
```

2. Utilización de XPath

El principal objetivo de XPath consiste en definir la manera de dirigirse a partes de un documento XML. El nombre XPath viene de la utilización de una escritura de tipo «path», como en los shells DOS y UNIX. El objetivo consiste en desplazarse en el interior de la estructura jerárquica de un documento XML como si se tratase de un árbol de directorios. Para darse cuenta del interés de XPath, podríamos decir que es el equivalente del lenguaje SQL para un documento XML. La comparación debe detenerse aquí, ya que la sintaxis de ambas no tiene nada que ver entre sí!

a. Búsqueda en un documento XML

Para buscar un elemento en un documento XML, la primera etapa consiste en crear una instancia de la clase `XPathNavigator`. Esta instancia de clase debe conocer el documento en el cual tendrá que hacer búsquedas. Por eso, el propio documento, por medio del método `CreateNavigator`, va a facilitar esta instancia de clase.

```
XPathNavigator navegador;  
navegador = doc.CreateNavigator();
```

A partir de esta instancia, vamos a poder iniciar búsquedas en el documento usando el método `Select`. Este método utiliza como parámetro una cadena de caracteres que contiene la ruta XPath de búsqueda. Después de la ejecución, obtenemos un objeto `XPathNodeIterator`, que permite recorrer la lista de los nodos encontrados.

El siguiente ejemplo busca en el documento **restaurante.xml** las entradas disponibles en los diferentes menús:

```
XmlDocument document = new XmlDocument();  
doc.Load("restaurante.xml");  
XPathNavigator navegador = doc.CreateNavigator();  
XPathNodeIterator nodos = navegador.Select("/restaurante/menu/entradas");  
while (nodos.MoveNext())  
{  
    Console.WriteLine(nodos.Current.OuterXml);  
    Console.WriteLine();  
}
```

Obtenemos el siguiente resultado:

```
<entradas>  
  <nombre calorías="50">rábanos</nombre>  
  <nombre calorías="300">pasta</nombre>  
  <nombre calorías="350">salchichón</nombre>  
</entradas>  
<entradas>  
  <nombre calorías="50">pan</nombre>  
</entradas>
```

También es posible añadir a la petición XPath los criterios de selección sobre el valor de ciertos atributos.

El siguiente ejemplo busca los postres del menú **gastronomico** con menos de **350** calorías.

```
XmlDocument doc = new XmlDocument();
doc.Load("restaurante.xml");
XPathNavigator navegador = doc.CreateNavigator();
XPathNodeIterator nodos = navegador.Select(
    "/restaurante/menu[@tipo='gastronomico']/postres/nombre[@calorias<350]");
while (nodos.MoveNext())
{
    Console.WriteLine(nodos.Current.Value);
    Console.WriteLine();
}
```

b. Modificación de los datos de un documento XML

Después de haber encontrado un elemento en el árbol de un documento, es posible modificar su valor.

El siguiente ejemplo disminuye un **50%** las calorías de cada postre del menú **gastronomico**.

```
doc = new XmlDocument();
doc.Load("restaurante.xml");
XPathNavigator navegador = doc.CreateNavigator();
XPathNodeIterator nodos = navegador.Select(
    "/restaurante/menu[@tipo='gastronomico']/postres/nombre");
while (nodos.MoveNext())
{
    nodos.Current.MoveToAttribute("calorias", "");
    nodos.Current.SetValue(String.Format("{0:####}",
        (double.Parse(nodos.Current.Value) * 0.5)));
}
doc.Save("restaurante.xml");
```

A continuación presentamos el contenido del archivo después de la ejecución del código anterior.

```
<?xml version="1.0" encoding="utf-8" ?>
<restaurante>
  <menu tipo="gastronomico">
    <entradas>
      <nombre calorias="50">rábanos</nombre>
      <nombre calorias="300">pasta</nombre>
      <nombre calorias="350">salchichón</nombre>
    </entradas>
    <platos>
      <nombre calorias="1000">chucrut</nombre>
      <nombre calorias="2000">cocido</nombre>
      <nombre calorias="1700">cuscus</nombre>
    </platos>
    <quesos>
      <nombre calorias="240">manchego</nombre>
      <nombre calorias="300">tetilla</nombre>
      <nombre calorias="120">cabrales</nombre>
    </quesos>
    <postres>
      <nombre calorias="170" sabor="chocolate">helado</nombre>
```

```

        <nombre calorías="125" frutas="manzanas">tarta</nombre>
        <nombre calorías="200">crema catalana</nombre>
    </postres>
</menu>
<menu type="economico">
    <entradas>
        <nombre calorías="50">pan</nombre>
    </entradas>
    <platos>
        <nombre calorías="1700">jamón</nombre>
    </platos>
    <quesos>
        <nombre calorías="240">manchego</nombre>
    </quesos>
    <postres>
        <nombre calorías="340" sabor="fresa">helado</nombre>
    </postres>
</menu>
</restaurante>

```

c. Añadir un nodo a un documento XML

Después de buscar un nodo en un documento, es posible añadirle nodos hijos y nodos hermanos. Los métodos `InsertAfter` y `InsertBefore` añaden un nodo hermano después o antes del nodo actual. El método `AppendChild` añade un nodo hijo al nodo actual.

El siguiente ejemplo añade un nuevo postre al menú **gastronomico**.

```

XmlDocument doc;
doc = new XmlDocument();
doc.Load("restaurante.xml");
XPathNavigator navegador = doc.CreateNavigator();
XPathNodeIterator nodos = navegador.Select(
    "/restaurante/menu[@tipo='gastronomico']/postres");
nodos.MoveNext();
nodos.Current.AppendChild("<nombre calorías='800'>crepes</nombre>");
doc.Save("restaurante.xml");

```

Después de la ejecución de este código, el documento se convierte en:

```

<?xml version="1.0" encoding="utf-8" ?>
<restaurante>
  <menu tipo="gastronomico">
    <entradas>
      <nombre calorías="50">rábanos</nombre>
      <nombre calorías="300">pasta</nombre>
      <nombre calorías="350">salchichón</nombre>
    </entradas>
    <platos>
      <nombre calorías="1000">paella</nombre>
      <nombre calorías="2000">cocido</nombre>
      <nombre calorías="1700">cusús</nombre>
    </platos>
    <quesos>
      <nombre calorías="240">manchego</nombre>
      <nombre calorías="300">tetilla</nombre>
      <nombre calorías="120">cabrales</nombre>
    </quesos>
    <postres>

```

```
<nombre calorias="340" sabor="chocolate">helado</nombre>
<nombre calorias="250" frutas="manzanas">tarta</nombre>
<nombre calorias="400">crema catalana</nombre>
<nombre calorias="800">crepes</nombre>
</postres>
</menu>
<menu tipo="economico">
  <entradas>
    <nombre calorias="50">pan</nombre>
  </entradas>
  <platos>
    <nombre calorias="1700">jamón</nombre>
  </platos>
  <quesos>
    <nombre calorias="240">manchego</nombre>
  </quesos>
  <postres>
    <nombre calorias="340" sabor="fresa">helado</nombre>
  </postres>
</menu>
</restaurante>
```


Introducción

Ahora que su aplicación está terminada, probada, depurada y, por lo tanto, funciona sin problemas, es el momento de pensar en la manera de ponerla a disposición de los usuarios. Existen dos soluciones para ello:

- La tecnología de despliegue Windows Installer. La aplicación se empaqueta en uno o varios archivos que luego se distribuyen a los usuarios. Éstos ejecutan el archivo Setup.exe para instalar la aplicación.
- El despliegue ClickOnce. Con esta solución, la publicación de los archivos de la aplicación se hace en una ubicación centralizada y el usuario instala o ejecuta la aplicación a partir de esta ubicación.

Vamos a detallar cada una de estas dos técnicas de despliegue.

Despliegue con Windows Installer

Microsoft Windows Installer es un servicio de instalación y configuración de aplicación disponible en todos los sistemas operativos de Microsoft. El principio básico del funcionamiento de Windows Installer es la agrupación en un único elemento de todos los datos e instrucciones necesarios para el despliegue de una aplicación. Representa una evolución importante respecto a los procedimientos de instalaciones clásicos, que consistían principalmente en facilitar el conjunto de los archivos necesarios para el funcionamiento correcto de la aplicación y un script encargado de copiar estos archivos en el disco duro de la máquina.

Con Windows Installer, el sistema conserva un registro de todas las operaciones efectuadas durante la instalación: directorios creados, archivos copiados, entradas de la base de registro modificadas, etc. Luego, estos datos se utilizan durante la desinstalación de la aplicación. Windows Installer efectúa las operaciones inversas durante la desinstalación de la aplicación. Sin embargo, se realiza un control para garantizar que ninguna otra aplicación necesite un archivo, una clave de registro o un componente que está a punto de ser suprimido. Esta verificación permite asegurar que la supresión de una aplicación no conlleva problemas de funcionamiento en otra aplicación.

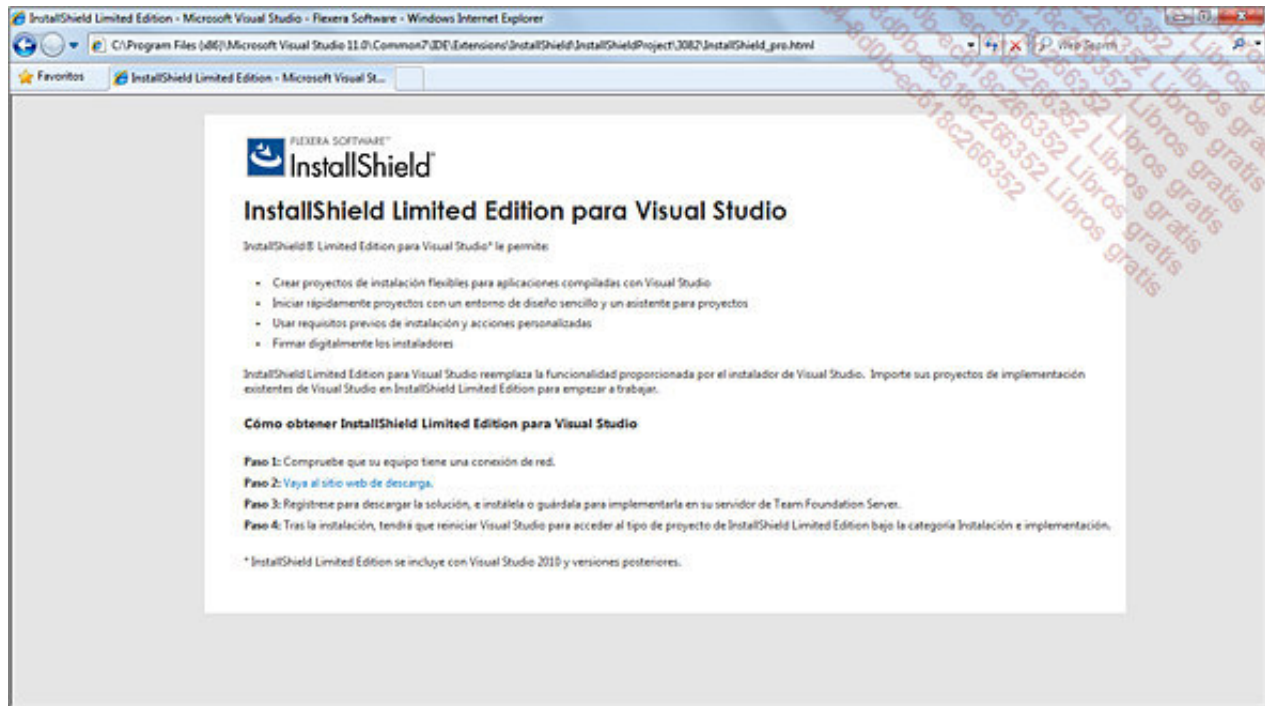
Windows Installer también gestiona la reparación de una aplicación y reinstala automáticamente los archivos que faltan que hubieran podido suprimirse, debido a un error del usuario.

El procedimiento de instalación se efectúa en el interior de una transacción para garantizar que la aplicación quedará instalada completamente o que, en caso de fracaso durante la instalación, el sistema volverá a su estado inicial.

En realidad, los procedimientos de instalaciones son verdaderas aplicaciones. De hecho, son gestionadas por Visual Studio como si se tratase de cualquier otro proyecto.

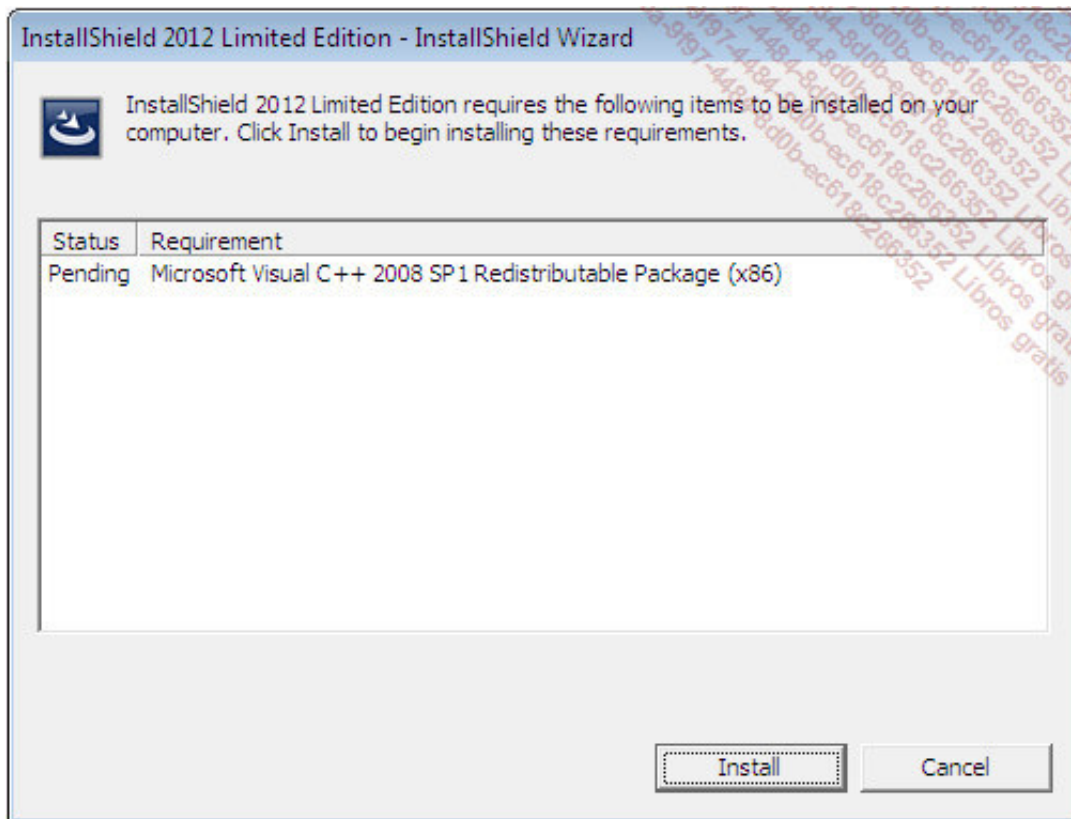
1. Instalación de InstallShield Limited Edition

La primera etapa consiste en descargar el programa de instalación del producto. En el menú **Archivo- Nuevo - Proyecto** existe un enlace a la página web que le permite realizar esta descarga; a continuación, a través de la opción **Instalado - Plantillas - Otros tipos de proyecto - Instalación e implementación - Habilitar InstallShield Limited Edition**, se le redirige a la siguiente página.

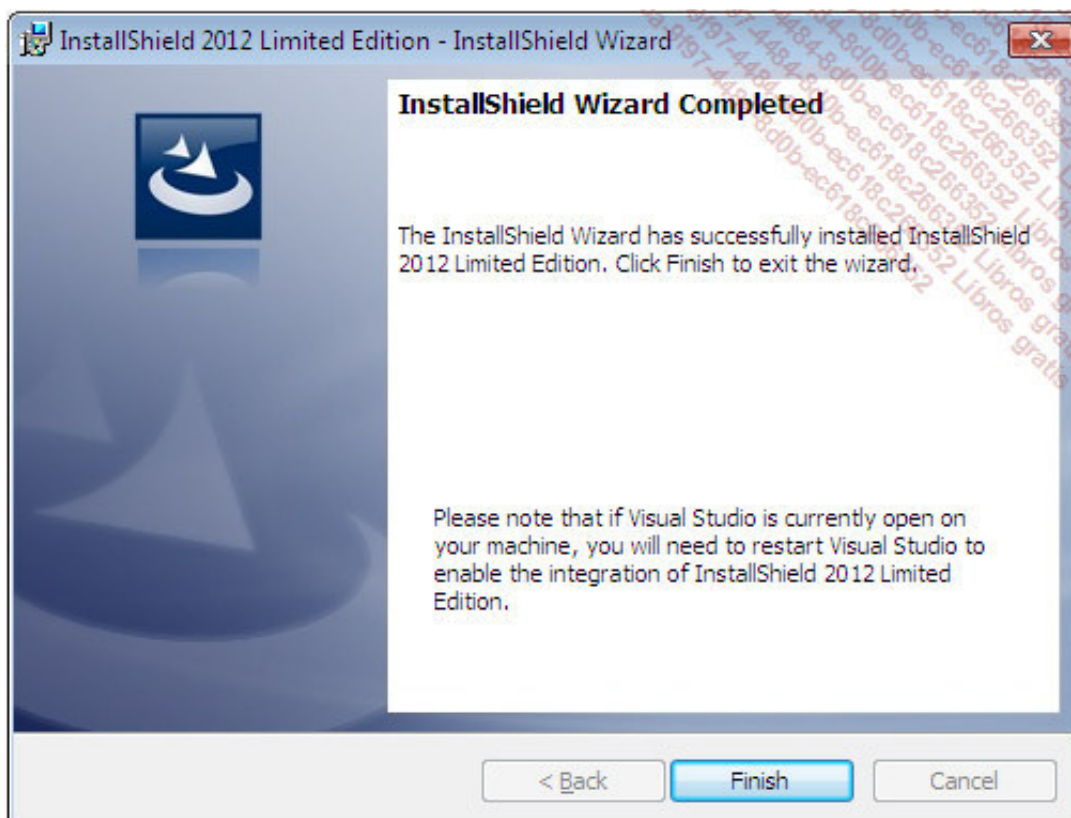


El enlace **Vaya al sitio web de descarga** le permite acceder al sitio web de la empresa FLEXERA Software, que distribuye el producto. Debe rellenar el formulario de inscripción antes de poder iniciar la descarga del producto. Es obligatorio proveer una dirección de correo electrónico válida puesto que se le enviará un código a dicha dirección con el objetivo de activar el producto antes de su primer uso.

Puede iniciar la instalación directamente desde el sitio de descarga o copiar de forma local el archivo y, a continuación, ejecutar la instalación a partir de la copia local. Esta última opción es preferible, puesto que facilita la posibilidad de retomar la instalación en caso de que ocurra algún incidente sin tener que descargar de nuevo el archivo. Es preferible que Visual Studio esté cerrado durante el procedimiento de instalación, pues si no habrá que reiniciarlo. El programa de instalación realiza la actualización de su sistema con el objetivo de integrar los componentes requeridos por InstallShield.

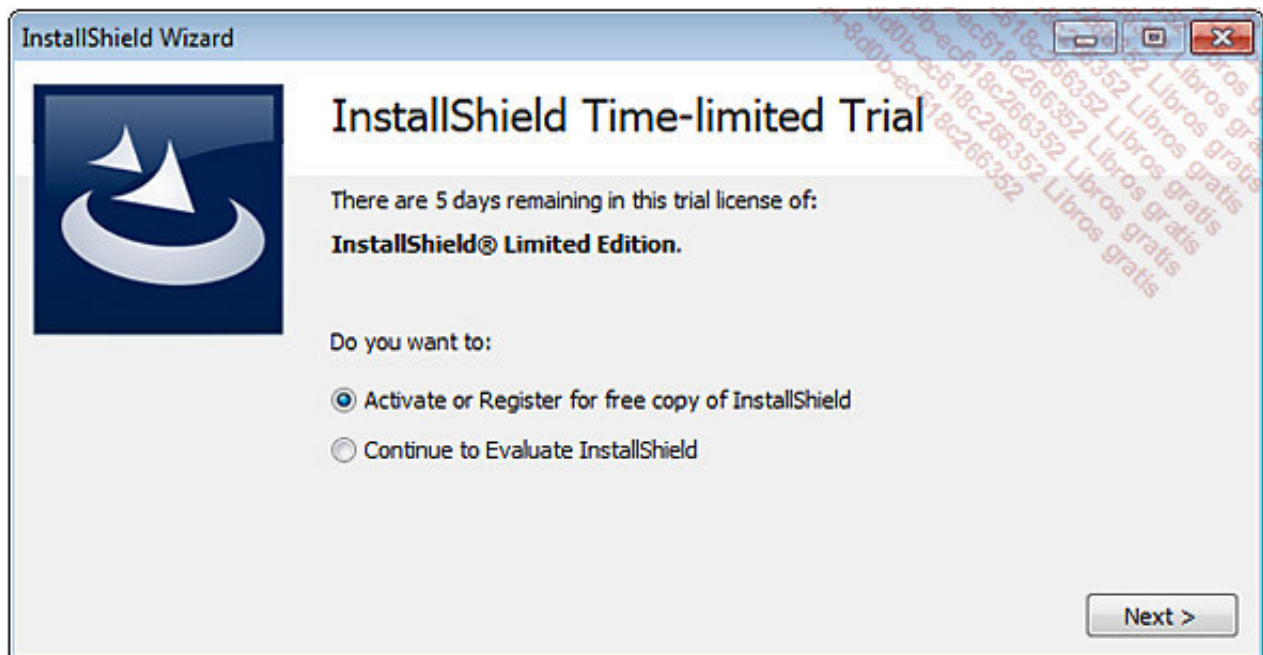


A continuación, debe aceptar el contrato de licencia y seleccionar la carpeta en la que se instalará InstallShield, antes de poder ejecutar la etapa de recogida de archivos. Al final de la copia, la siguiente pantalla le informa del éxito de la instalación.

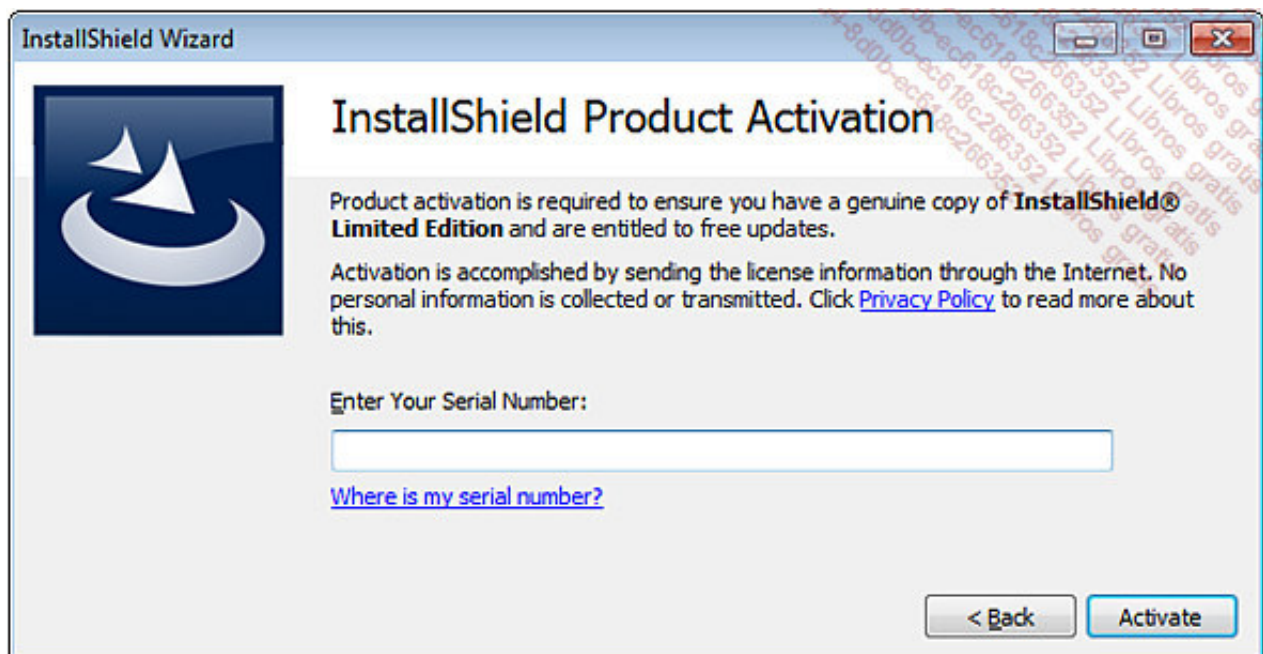


De hecho, la instalación no está completamente terminada puesto que queda la etapa de activación del

producto que se realizará durante la creación del primer proyecto InstallShield Limited Edition. Entonces, se le invitará a que inserte el código de activación o a continuar trabajando con una versión de evaluación limitada en el tiempo.



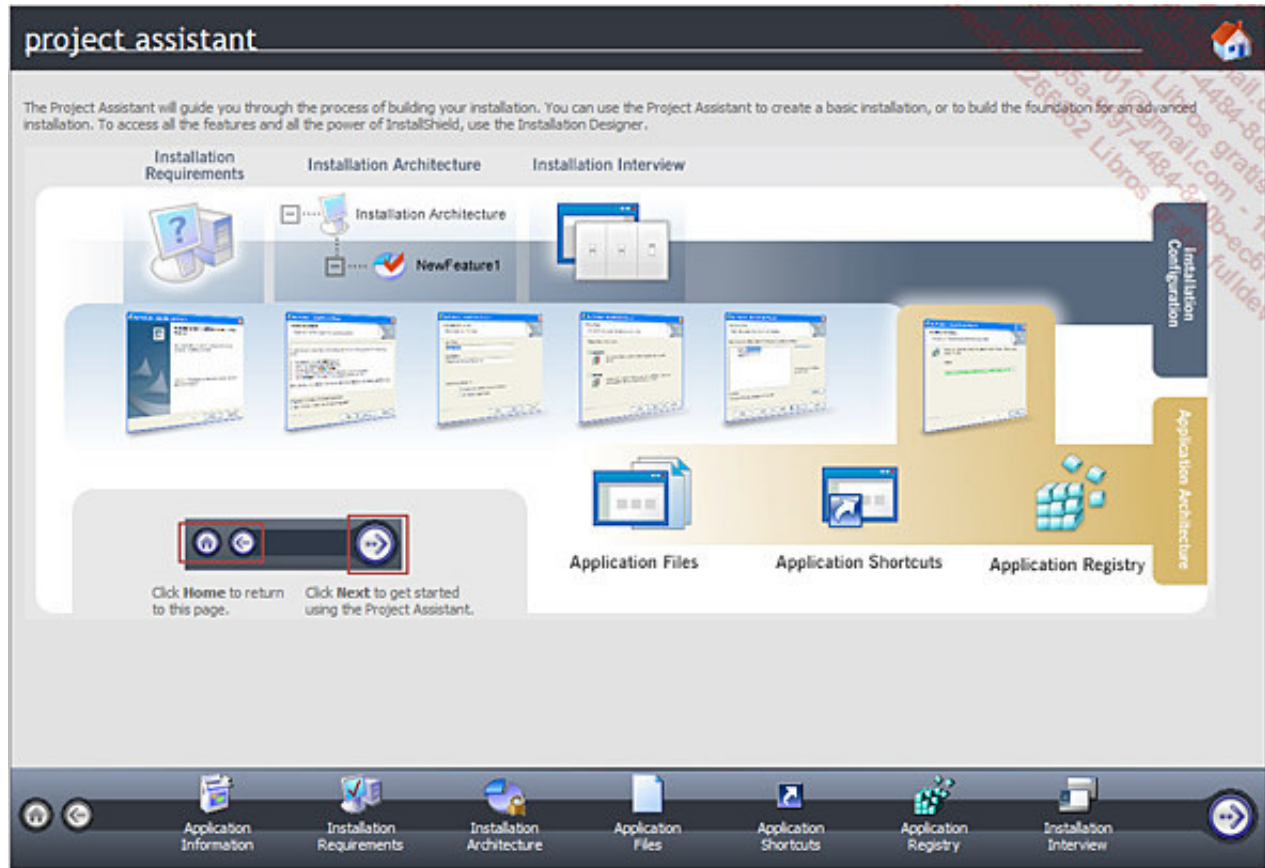
Si elige la opción de activar el producto, la siguiente pantalla le permite introducir la clave de producto que se le ha enviado por correo electrónico durante su inscripción para la descarga del producto.



Esta operación de activación requiere una conexión a Internet activa. Finaliza así la instalación de InstallShield Limited Edition, sólo nos falta descubrir cómo se utiliza, mediante la creación de un proyecto de despliegue.

2. Creación de un proyecto de instalación

El método de creación de un proyecto de instalación es idéntico al utilizado por cualquier otro tipo de proyecto de Visual Studio. En el menú **Archivo**, seleccione **Agregar** y, a continuación, **Nuevo proyecto**. En la ventana para agregar un proyecto, seleccione **Otros tipos de proyectos** y, a continuación, **InstallShield Limited Edition**. El proyecto se agrega a la solución actual. El proyecto para el que quiere crear un programa de instalación y el proyecto InstallShield deben formar parte, obligatoriamente, de la misma solución de Visual Studio. Un asistente le permite completar las distintas etapas de configuración del proyecto de instalación.



Cada etapa cubre un aspecto particular del funcionamiento del programa de instalación. Con esta versión Limited Edition de InstallShield, ciertas funcionalidades o ciertas opciones están bloqueadas. Están señaladas en el asistente mediante este icono  .

a. Información general

La primera etapa del asistente recoge la información general asociada a la aplicación.

Application Information
Specify general application information.

The Application Information page assists you in requesting general information about your application. This information is used from within the installation as well as from the Add or Remove Programs in the Windows Control Panel.

More Options
Edit the default installation location.

Other Places
General Information
Update Notifications

Help Links
Tell me more about automatic update notification.
Learn more about Add/Remove Programs.
Learn about how the company name and product name are used by the installation.
Tell me more about the installation life cycle.

Specify your company name:
Edicoes ENI

Specify your application name:
Telepantalla

Specify your application version:
1.00.007

Specify your company Web address:
http://www.edicoes-eni.com/

Select the icon to display with your application in Add or Remove Programs.
C:\Users\VS2012\Documents\Visual Studio 2012\Projects\appTelepantalla\cono.ico
Browse...

Application Information | Installation Requirements | Installation Architecture | Application Files | Application Shortcuts | Application Registry | Installation Interview

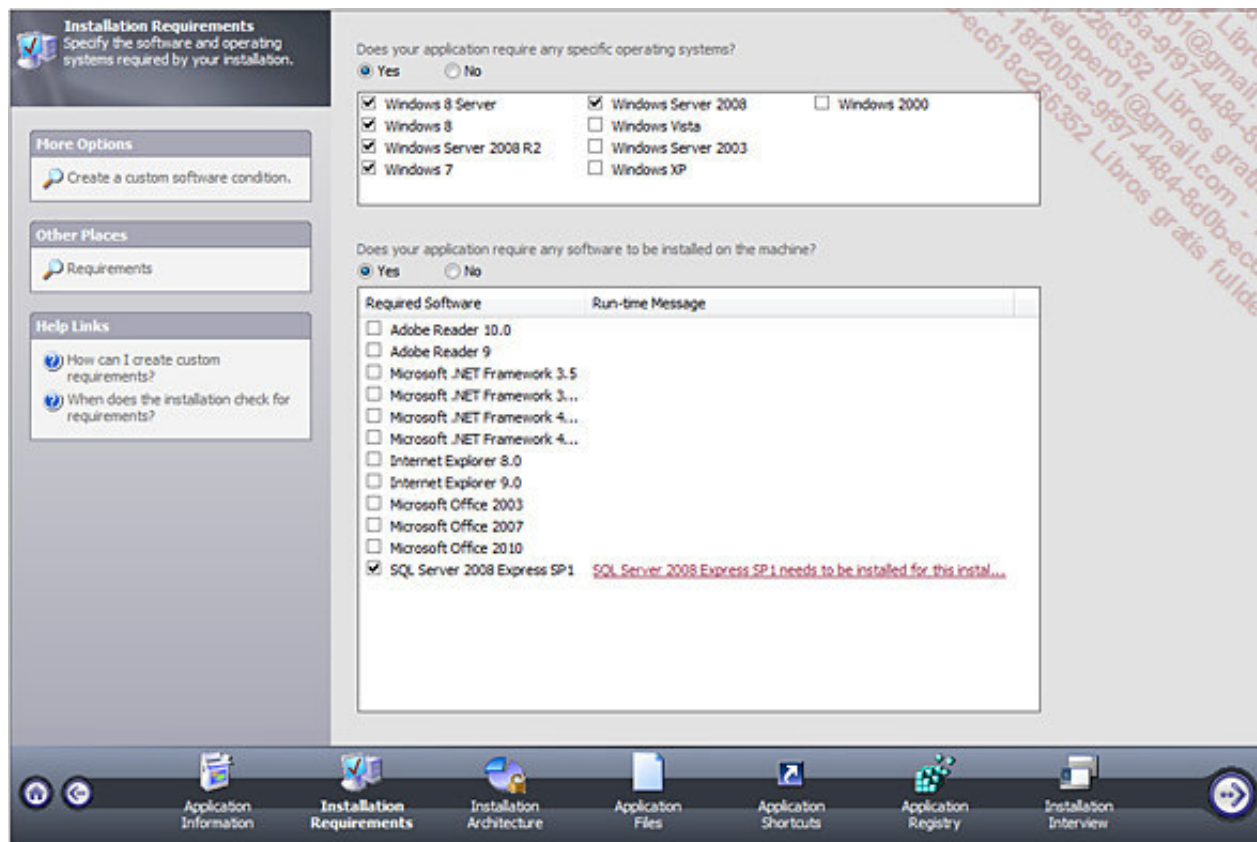
En esta pantalla debe indicar:

- El nombre de su empresa.
- El nombre que se quiere dar a la aplicación.
- El número de versión de la aplicación.
- El sitio de Internet donde está disponible la información complementaria asociada a la aplicación.
- El icono asociado a la aplicación.

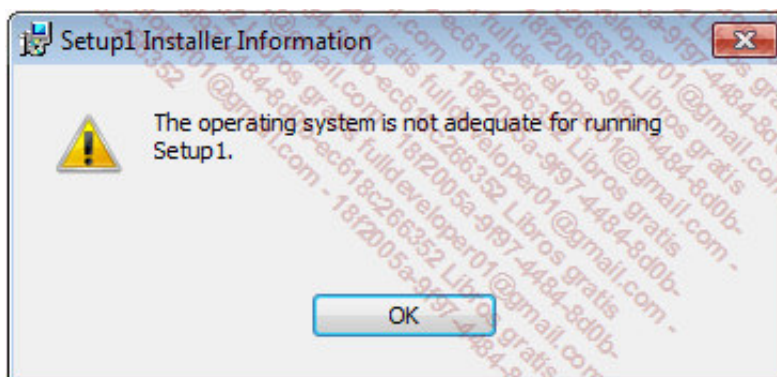
Esta información se utiliza durante la fase de instalación de la aplicación y también para el funcionamiento de la opción **Desinstalar o modificar un programa** del panel de control de Windows.

b. Requisitos previos de instalación

La siguiente etapa le permite especificar las exigencias de la aplicación en lo que respecta al puesto de trabajo sobre el que se va a instalar.



Puede especificar distintas versiones de Windows para poder instalar la aplicación. Si durante la instalación de la aplicación no se detecta en el equipo alguno de los sistemas operativos exigidos, aparece el siguiente mensaje y se detiene la instalación.

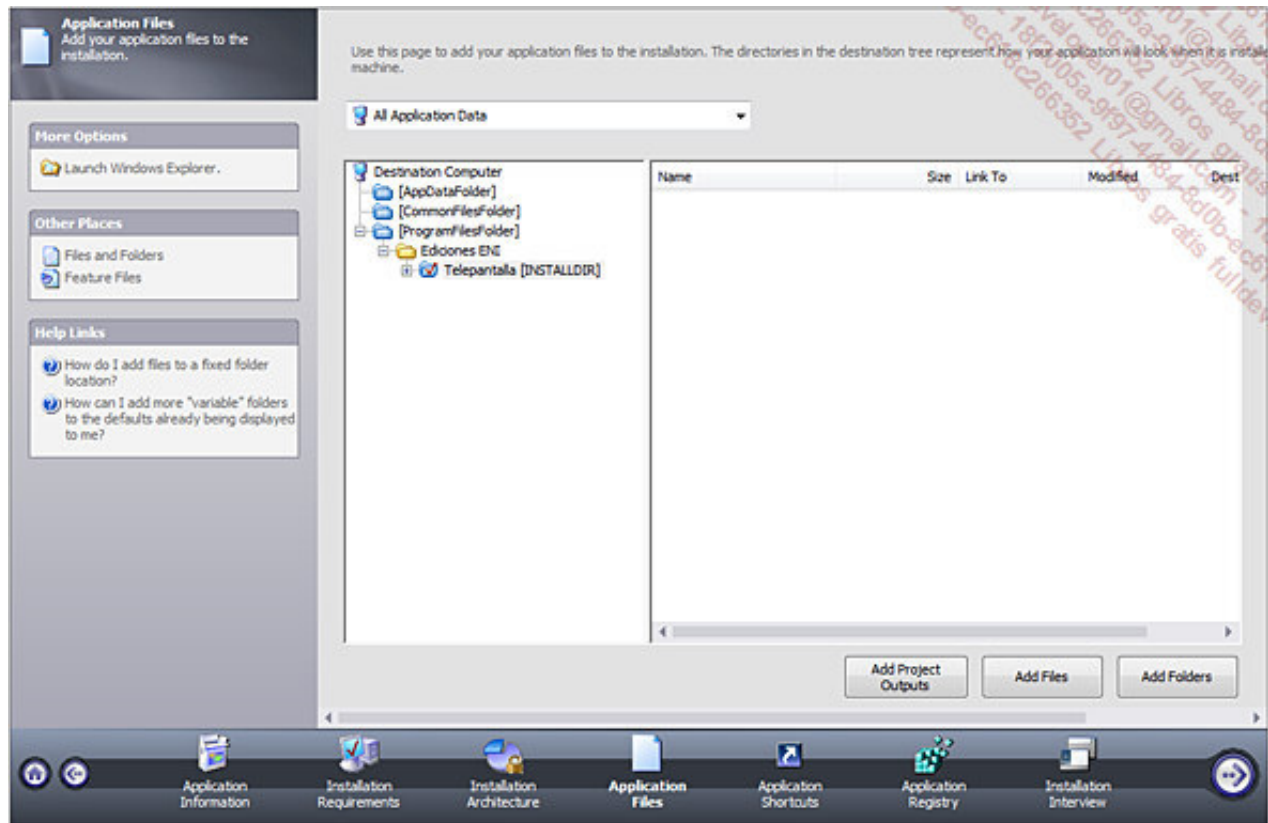


Si se satisface la primera condición respecto a la presencia de una versión concreta, se realiza una segunda verificación para controlar la presencia de una o varias aplicaciones en el puesto de instalación. Como ocurre con la primera verificación, si alguno de los elementos requeridos no está presente, aparece un mensaje y la instalación finaliza justo después de cerrar el cuadro de diálogo. En este caso, no se realiza ninguna modificación en el equipo de trabajo.

La siguiente etapa no puede configurarse con la versión Limited Edition de InstallShield. Pasamos, por tanto, a la cuarta etapa que es, realmente, la más importante del asistente.

c. Archivos de la aplicación

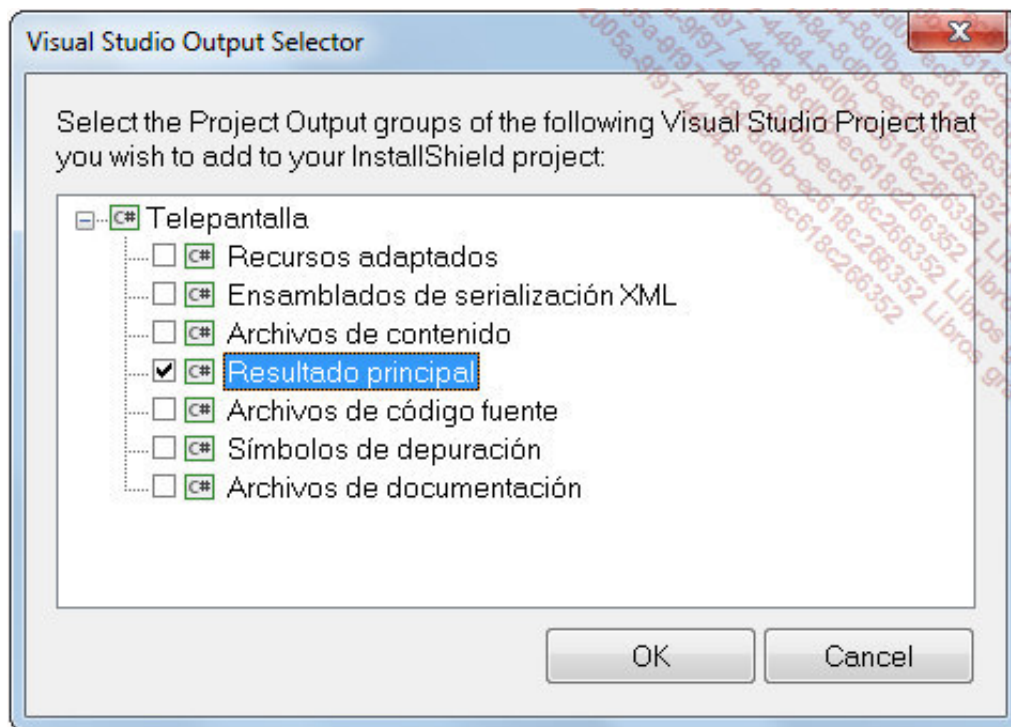
Esta etapa es, sin duda, primordial para el buen funcionamiento de la aplicación, pues define lo que se va a instalar y dónde se va a instalar.



Esta pantalla está compuesta por dos áreas:

- Un área de navegación en la parte izquierda.
- Un área de información en la parte derecha.

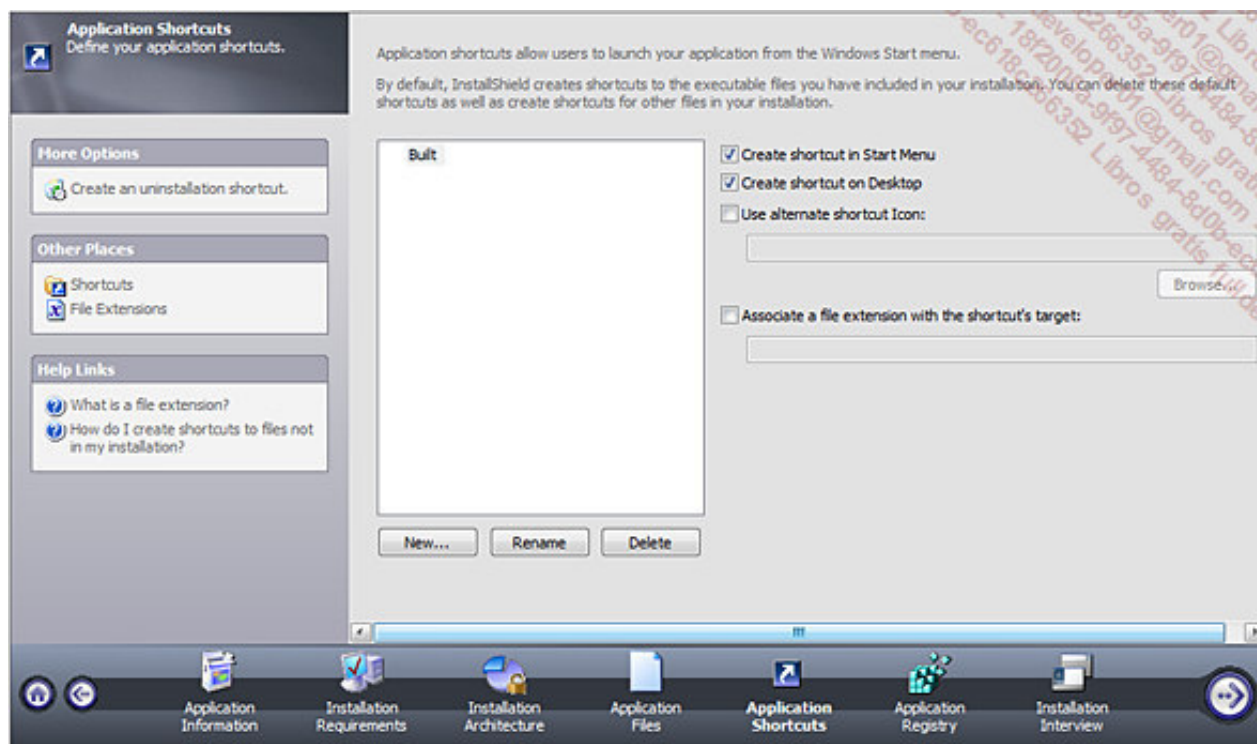
El área de navegación contiene la lista jerárquica de carpetas correspondientes al sistema de archivos del equipo de instalación. Los nombres de las carpetas corresponden con carpetas Windows estándar. Los tres botones del área derecha permiten agregar distintos elementos. En la carpeta de aplicación, puede, por ejemplo, agregar una carpeta, un archivo o, con mayor frecuencia, una salida de proyecto (los archivos creados por la compilación de un proyecto). Es, por tanto, gracias a esta etapa como puede indicar el proyecto que quiere instalar. El siguiente cuadro de diálogo le permite indicar el proyecto que quiere desplegar y qué elementos quiere desplegar en el puesto del usuario.



Para que la aplicación pueda ejecutarse en el puesto del usuario, es preciso seleccionar, como mínimo, la opción **Resultado principal**. Por el contrario, la opción **Archivos de código fuente** se utiliza en raras ocasiones.

d. Accesos directos a la aplicación

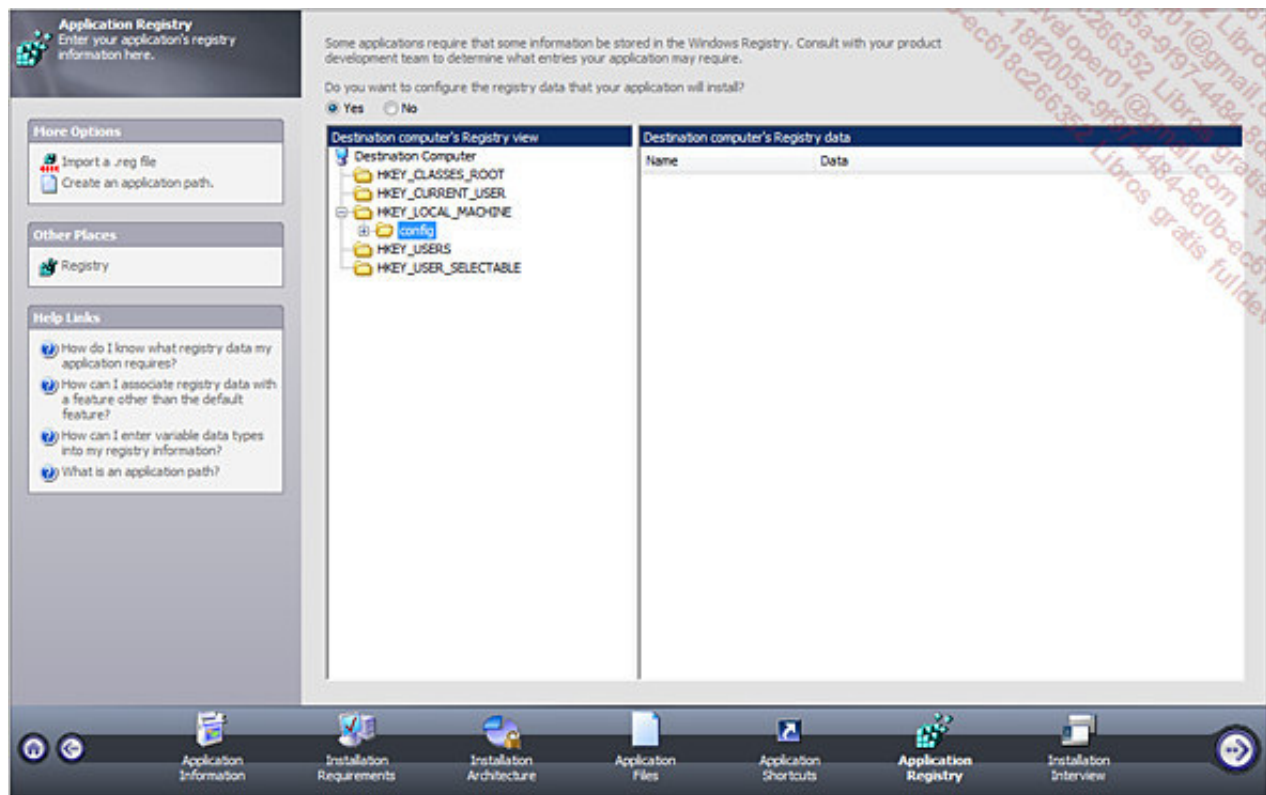
Para facilitar la ejecución de la aplicación en el puesto del usuario, es deseable poder proveer accesos directos que eviten al usuario tener que buscar el ejecutable en el árbol de carpetas del sistema de archivos. Esto es lo que le permite la siguiente pantalla del asistente.



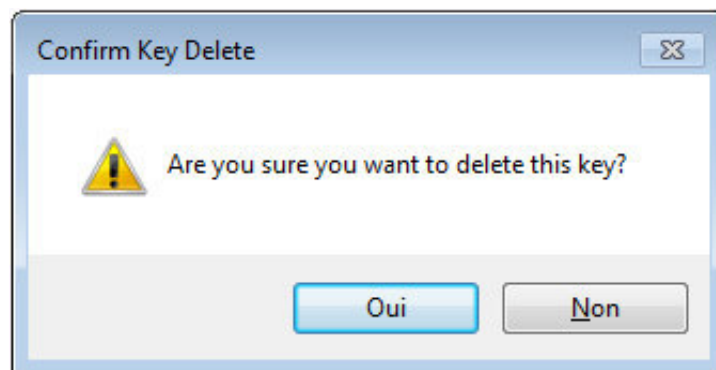
Es preciso, como mínimo, definir un acceso directo hacia la salida principal de la aplicación en el menú **Inicio**. Esto se representa por el elemento **Built** en la lista. Por cada acceso directo, puede escoger situarlo en el menú **Inicio** o en el escritorio (o ambos). Cabe destacar también que si se agrega una carpeta con los archivos ejecutables en la etapa anterior, se crean accesos directos automáticamente apuntando a dichos ejecutables. Se trata de herramientas complementarias a la aplicación, de modo que puede ser preferible eliminarlas para que no estén accesibles de manera directa.

e. Información en el registro de Windows

La penúltima pantalla le permite especificar las modificaciones que debe aportar la aplicación en el registro durante la instalación de la aplicación. Si no existe una clave en el registro del equipo a la hora de realizar el despliegue, se agrega durante la instalación. Es posible agregar claves sobre cualquier clave de nivel superior en el editor de registro.



Para agregar una clave en el registro debe, previamente, seleccionar un nodo de nivel superior o una subclave y a continuación, con ayuda del menú contextual, utilizar la opción **New - Key**. A continuación es preciso renombrar la clave. Es posible borrar una clave con la opción **Delete** del menú contextual. Hay que ser prudente, pues la supresión de una clave entraña la supresión de todas las subclaves y valores contenidos en ella. Aparece un mensaje de advertencia que le avisa de esta situación peligrosa y le solicita confirmar su elección.

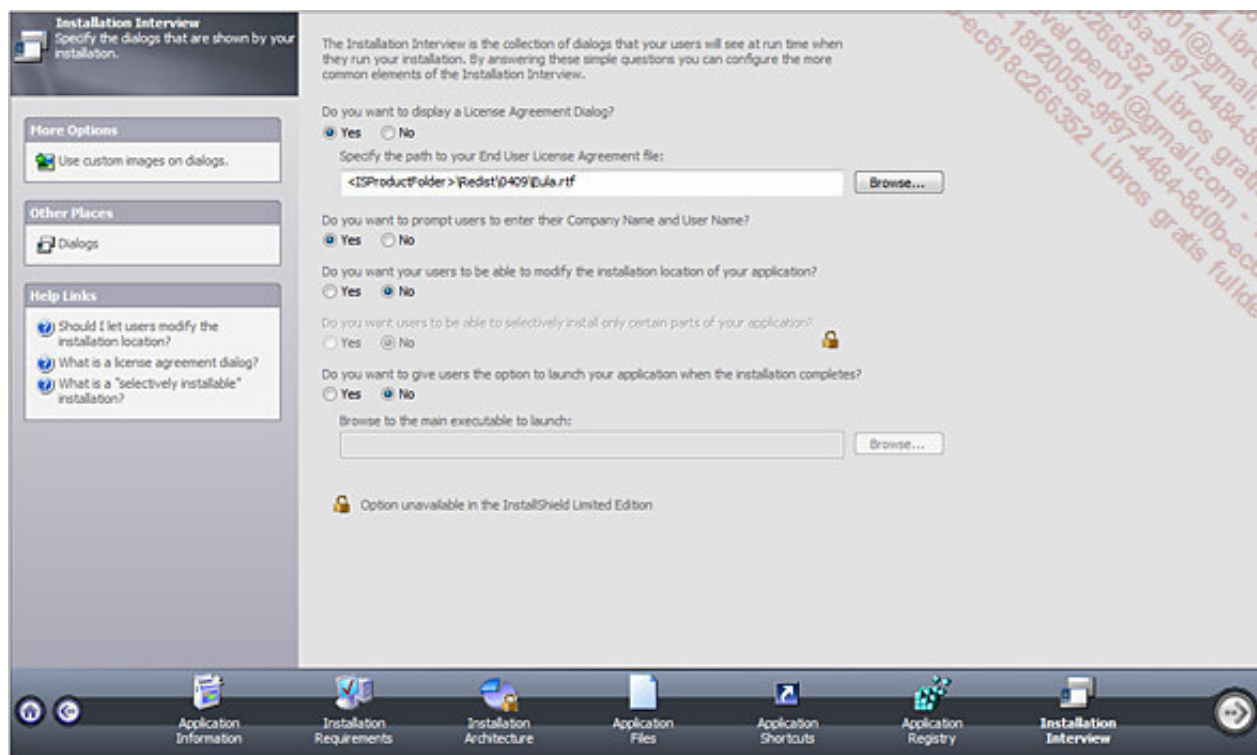


También es posible especificar valores para las nuevas claves o modificar los valores de las claves existentes. Puede agregar valores de tipo cadena, binario, DWORD, Multi String y Expandable String. Durante la instalación, estos valores se escriben en el registro; los valores existentes se ven remplazados por los valores especificados en el programa de instalación.

Es posible agregar claves y valores de registro a un proyecto de despliegue importando un archivo de registro (.reg). Esto le permite copiar una sección completa de un registro existente de una sola vez, para ganar tiempo. Los archivos de registro pueden crearse mediante herramientas tales como el editor de registro de Windows (regedit.exe). Esta solución es muy práctica para transferir en el puesto de los usuarios una porción del registro recuperado en el puesto utilizado para el despliegue de la aplicación. El enlace importado como archivo .reg permite realizar esta operación. Simplemente debe escoger el archivo (.reg) que contiene la información que quiere importar.

f. Configuración de los cuadros de diálogo

Durante la instalación de la aplicación, aparece una serie de cuadros de diálogo para recoger la información necesaria para instalar la aplicación. Esta última etapa nos va a permitir configurar el aspecto de los distintos cuadros de diálogo.



Para ciertos cuadros de diálogo será, posiblemente, necesario proveer un archivo externo que

contenga la información mostrada en el cuadro de diálogo. Es el caso, por ejemplo, de la opción que controla la visualización del contrato de licencia de la aplicación, que necesita un archivo con formato .rtf que contenga el contrato de licencia.

Durante las distintas etapas, el proyecto de despliegue debe configurarse correctamente. Tan solo queda generar el proyecto o la solución completa.

Despliegue con ClickOnce

ClickOnce es una tecnología de despliegue que permite crear aplicaciones de Windows cuya actualización puede efectuarse automáticamente. La instalación de este tipo de aplicación se realiza con un mínimo de intervención de parte del usuario. Esta técnica simplifica la etapa de despliegue que a veces llega a ser un auténtico rompecabezas. Nos encontramos a menudo con los siguientes problemas durante el despliegue de una aplicación.

Actualización de la aplicación

Con un método de despliegue clásico, cuando una nueva versión de la aplicación está disponible, el usuario debe reinstalar la aplicación en general para aprovechar las actualizaciones. La tecnología ClickOnce es capaz de facilitar las actualizaciones automáticamente. En este caso, sólo las partes de la aplicación que han cambiado se descargan, y luego la aplicación completa actualizada se reinstala automáticamente a partir de un nuevo archivo.

Componentes compartidos

Las aplicaciones dependen a menudo de componentes compartidos, de ahí la existencia de un riesgo potencial de conflicto de versiones. En el caso de un despliegue con ClickOnce, cada aplicación es autónoma y no puede interferir en las otras aplicaciones.

Autorización de seguridad

En general, la instalación de una aplicación con un método clásico exige autorizaciones administrativas en el puesto de trabajo donde se efectúa la instalación. El despliegue con ClickOnce autoriza a los usuarios que no tienen privilegios administrativos a efectuar la instalación y sólo atribuye las autorizaciones de seguridad de acceso del código necesarias para el buen funcionamiento de la aplicación.

A veces, todas estas restricciones han conducido a los desarrolladores a elegir una tecnología Web en lugar de aplicaciones de Windows clásicas simplemente para obtener las facilidades de despliegue de este tipo de aplicaciones. La contrapartida a esta elección se nota en el menor rendimiento de la aplicación y en una interfaz de usuario menos elaborada. La tecnología ClickOnce convierte el despliegue de aplicaciones de Windows en algo tan simple como el despliegue de aplicaciones Web. Cualquier aplicación consola o Windows Forms se puede publicar con ClickOnce. Hay tres técnicas de publicación disponibles:

- a partir de una página Web;
- a partir de una compartición de archivos de red;
- a partir de un soporte como un CD-Rom o DVD.

La ejecución de la aplicación dispone también de dos variantes. Se puede instalar la aplicación en el ordenador de un usuario y ejecutarla incluso si el ordenador está fuera de conexión. También se puede ejecutar únicamente en modo en línea sin instalar ningún elemento de manera permanente en el ordenador. Las aplicaciones ClickOnce están aisladas unas de otras, y la instalación o la ejecución de una aplicación no puede interrumpir aplicaciones existentes. Por defecto, las aplicaciones ClickOnce se ejecutan en las zonas de seguridad de Internet o de la intranet. En función de las necesidades, la aplicación puede pedir autorizaciones de seguridad más elevadas.

Las actualizaciones de la aplicación también pueden tener varios modos de funcionamiento. Pueden ser automáticas, y en este caso la aplicación verifica cada vez que se inicia si hay actualizaciones disponibles, y luego las instala automáticamente. El usuario puede comprobar manualmente la existencia de una

actualización y decidir o no su instalación. El administrador puede convertir en obligatoria la instalación de una actualización.

1. Principio de funcionamiento de ClickOnce

El mecanismo de despliegue de ClickOnce se basa en dos archivos XML llamados manifiestos:

- Un manifiesto de aplicación.
- Un manifiesto de despliegue.

El manifiesto de aplicación describe la propia aplicación, los ensamblados y los archivos que la componen, las dependencias, las autorizaciones requeridas para la ejecución y la ubicación donde hay actualizaciones disponibles.

El manifiesto de despliegue describe cómo se despliega la aplicación, incluso la ubicación del manifiesto de aplicación y la versión de la aplicación que deben ejecutar los clientes. Estos archivos son generados por Visual Studio.

El manifiesto de despliegue se copia en la ubicación de despliegue. Esta ubicación puede ser un servidor Web, un directorio compartido en la red o soportes removibles, como un CD-Rom. El manifiesto de aplicación y todos los archivos de la aplicación también se copian en una ubicación de despliegue especificado en el manifiesto de despliegue. Se pueden copiar estos archivos en la misma ubicación o en dos ubicaciones distintas. Visual Studio también se encarga de las copias de estos archivos.

Después del despliegue de la aplicación en la ubicación de despliegue, los usuarios pueden descargar e instalar la aplicación haciendo clic en el icono que representa el archivo manifiesto de despliegue disponible en una página Web o en un archivo. El usuario sólo ve un simple cuadro de diálogo que le pide confirmar la instalación. Después de la validación, la instalación continúa y se inicia la aplicación sin otra intervención. Si la aplicación requiere autorizaciones de ejecución más elevadas, el cuadro de diálogo pide al usuario conceder las autorizaciones para que la instalación pueda continuarse.

Se añade la aplicación al menú **Arrancar** del usuario y a la sección **Añadir/Suprimir programas del Panel de control**. A diferencia de otras tecnologías de despliegue, nada se añade al directorio **Program Files** en el registro o en el escritorio. Además, no es necesario disponer de ningún derecho de administración para hacer la instalación.

Cuando crea una versión actualizada de la aplicación, también debe generar un nuevo manifiesto de aplicación y copiar los archivos hacia una ubicación de despliegue, en general un archivo hermano del archivo de despliegue de origen. También se debe actualizar el manifiesto para que apunte hacia la ubicación de la nueva versión de la aplicación.

2. Los diferentes métodos de despliegue

Para desplegar una aplicación ClickOnce, hay tres estrategias posibles. La estrategia que elija depende principalmente del tipo de aplicación que despliega. Las tres estrategias de despliegue son las siguientes:

- Instalación desde la Web o una red compartida.
- Instalación desde un CD-Rom.
- Arranque de la aplicación desde la Web o una red compartida.

Instalación desde la Web o una red compartida

Esta estrategia permite desplegar su aplicación en un servidor Web o una partición de archivos en red.

Cuando un usuario final desea instalar la aplicación, hace clic en un icono de una página Web o doble clic en un icono de la partición de archivos. Luego se descarga, instala y arranca la aplicación en el ordenador del usuario. Unos elementos se añaden al menú **Inicio** y al grupo **Añadir/Suprimir programas** en el **Panel de control**.

Puesto que esta estrategia depende de la conexión de red, funciona de manera óptima para aplicaciones desplegadas si los usuarios tienen acceso a una red local o una conexión a Internet rápida.

Instalación desde un CD-Rom

Esta estrategia permite desplegar su aplicación en un soporte removible, como un CD-Rom o un DVD. Como para la opción anterior, cuando el usuario elige instalar la aplicación, esta última se instala, se inicia y unos elementos se añaden al menú **Inicio** y al grupo **Añadir/Suprimir programas** en el **Panel de control**.

Esta estrategia funciona mejor en el caso de aplicaciones desplegadas en los ordenadores de usuarios que no tienen una conectividad de red persistente o que tienen conexiones de poca banda ancha. Como la aplicación está instalada a partir de un soporte removible, ninguna conexión es necesaria para la instalación; sin embargo, la conectividad de red es necesaria para la comprobación de las actualizaciones de la aplicación.

Arranque de la aplicación desde la Web o una red compartida

Esta estrategia es similar a la primera, excepto por que la aplicación actúa como una aplicación Web. Cuando el usuario hace clic en un enlace de una página Web (o doble clic en un icono de la partición de archivos), la aplicación arranca. Cuando los usuarios cierran la aplicación, esta última ya no está disponible en el ordenador local. Ningún elemento se añade al menú **Inicio** ni al grupo **Añadir/Suprimir programas** en el **Panel de control**. Técnicamente, la aplicación se descarga e instala en un caché de aplicación del ordenador local, de la misma manera que una aplicación Web se descarga hacia el caché Web. Como para el caché Web, los archivos se limpian del caché de aplicación al finalizar su utilización. Sin embargo, el usuario tiene la impresión de que la aplicación se ejecuta desde la Web o la partición de archivos.

Se tiene que dar prioridad a esta estrategia para las aplicaciones poco utilizadas.

3. Las actualizaciones de la aplicación

ClickOnce puede facilitar automáticamente las actualizaciones de la aplicación. Una aplicación ClickOnce lee periódicamente su archivo manifiesto de despliegue para comprobar si las actualizaciones de la aplicación están disponibles. Si lo están, la nueva versión de la aplicación se descarga y ejecuta. Por razones de eficacia, sólo se descargan los archivos modificados.

Hay tres estrategias básicas posibles para las actualizaciones:

- La verificación de las actualizaciones durante el arranque de la aplicación.
- La verificación de las actualizaciones después del arranque de la aplicación (ejecutada por un proceso en segundo plano).
- La presentación de una interfaz de usuario destinada a las actualizaciones.

También puede determinar la frecuencia de verificación de las actualizaciones efectuada por la aplicación o configurar una actualización obligatoria. Las actualizaciones de aplicación exigen una conexión a la red. En ausencia de una conexión a la red, la aplicación se ejecuta sin verificar las actualizaciones sea cual sea la estrategia de actualización elegida.

Verificación de las actualizaciones después del arranque

Por defecto, la aplicación intenta localizar y leer el archivo manifiesto de despliegue en segundo plano durante su ejecución. Si una actualización está disponible durante la próxima ejecución, se invitará al usuario a descargar e instalar la actualización.

Esta estrategia se adapta particularmente a las conexiones de red de banda estrecha o a las aplicaciones voluminosas que puedan necesitar largas descargas.

Verificación de las actualizaciones en el arranque

Con esta estrategia, la aplicación intenta localizar y leer el archivo manifiesto de despliegue a cada lanzamiento. Si una actualización está disponible, se descargará y ejecutará. En caso contrario, se ejecutará la versión existente de la aplicación.

Esta estrategia se adapta bien a las conexiones de red de banda ancha. El plazo necesario para el arranque de la aplicación puede ser inaceptable en conexiones de banda más restringida en razón de la descarga de las actualizaciones.

Actualizaciones obligatorias

A veces es deseable obligar a los usuarios a ejecutar una versión actualizada de la aplicación, si, por ejemplo, ha modificado un recurso que puede cambiar el funcionamiento de la antigua versión de la aplicación. En este caso, puede marcar la actualización como obligatoria y, por lo tanto, impedir la ejecución de una versión más antigua de la aplicación. Se debe asociar esta estrategia con la verificación de las actualizaciones durante el arranque.

Intervalos de actualización

En el marco de las actualizaciones automáticas, puede especificar la frecuencia de verificación de las actualizaciones. Por ejemplo, puede desear una verificación a cada ejecución de la aplicación, una vez a la semana o una vez al mes. Si no hay conexión de red disponible en el momento especificado para la verificación, ésta se efectúa durante la próxima ejecución de la aplicación.

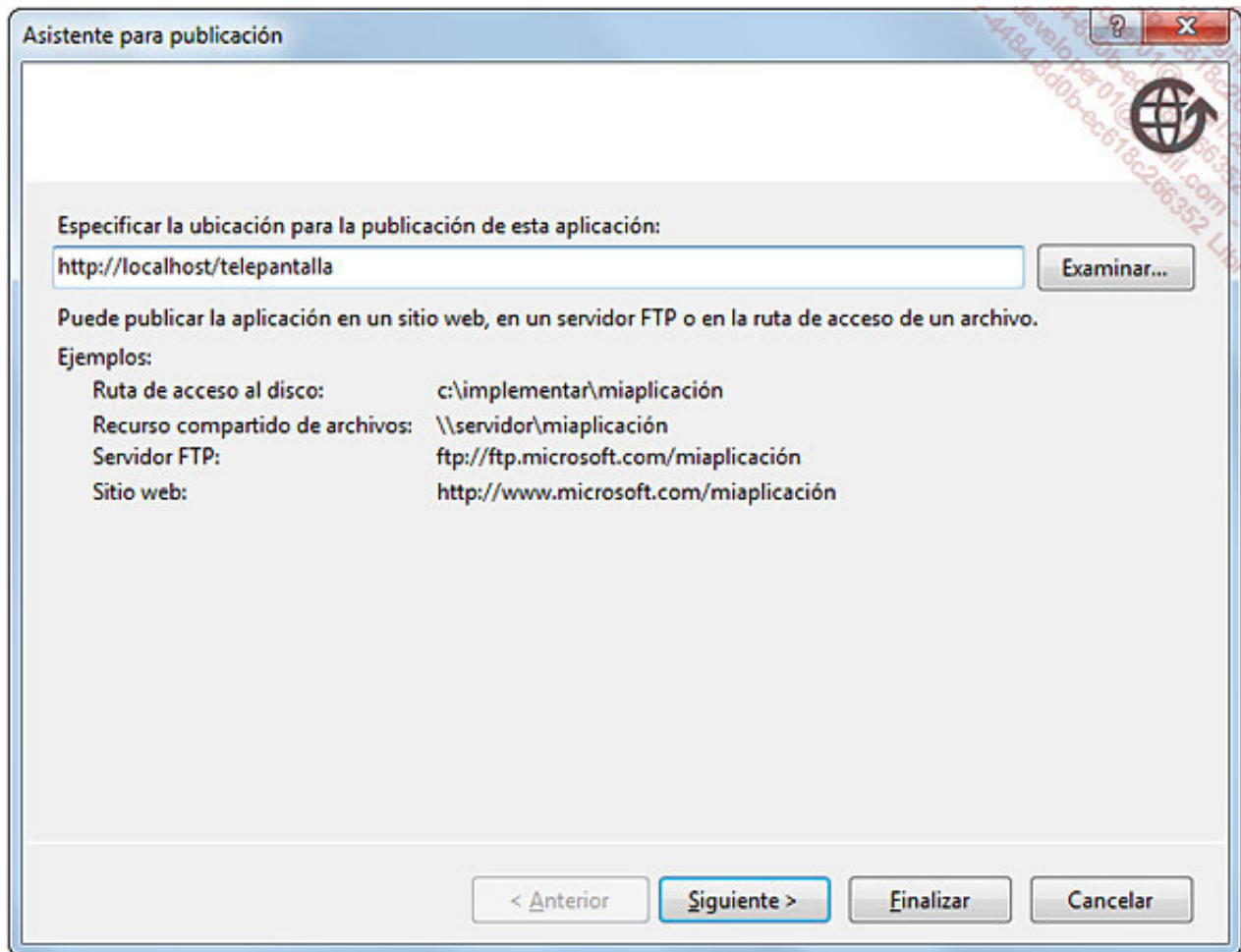
Bloqueo de las actualizaciones

También es posible hacer de tal manera que su aplicación nunca verifique las actualizaciones. Por ejemplo, puede desplegar una aplicación simple que no se actualice, al mismo tiempo que se beneficia de la facilidad de instalación de ClickOnce.

4. Puesta en marcha de la publicación ClickOnce

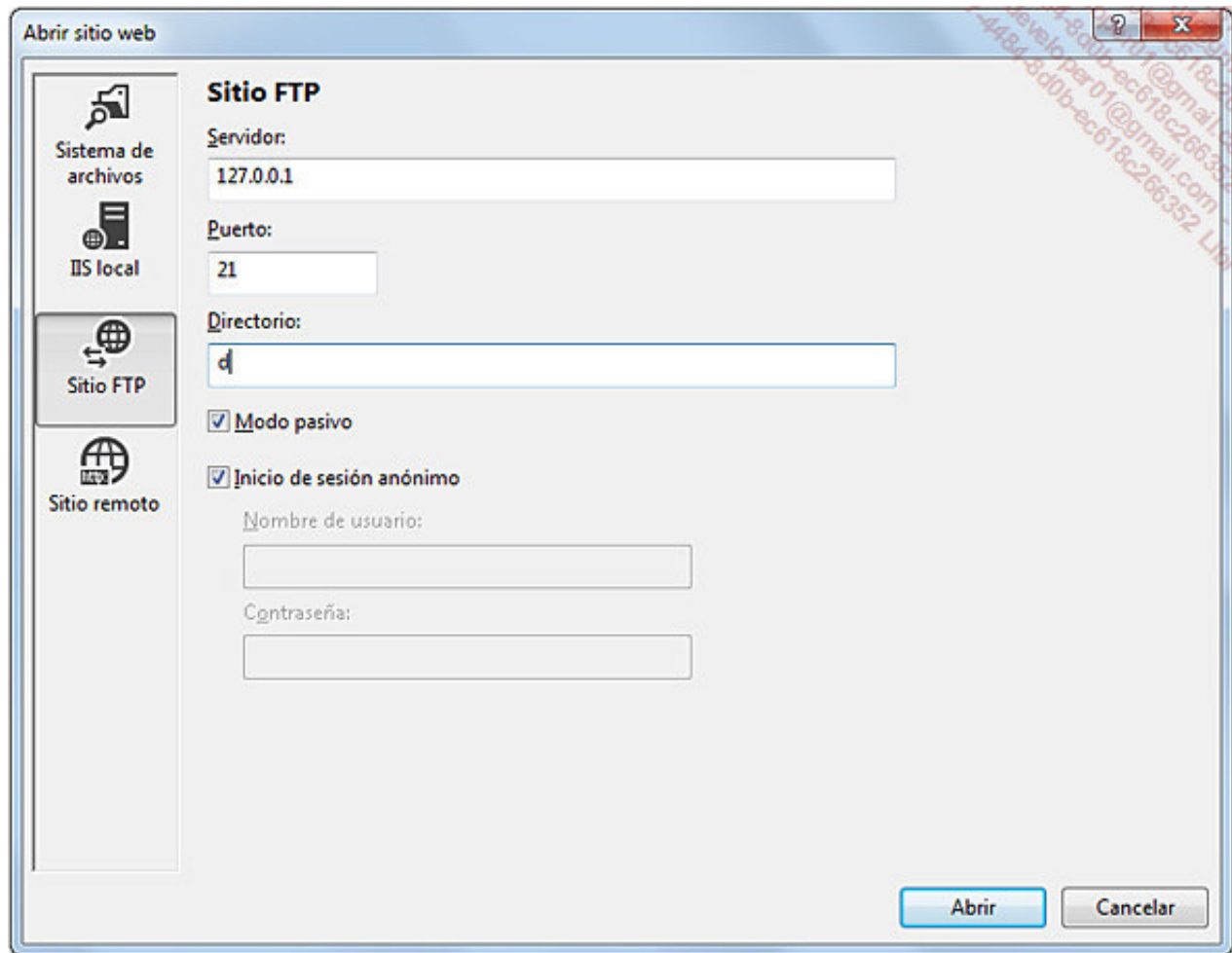
La publicación de una aplicación con la tecnología ClickOnce es facilitada por un asistente que permite recoger la mayoría de los datos necesarios para su despliegue. Este asistente está disponible eligiendo la opción **Publicar** del menú contextual, accesible sobre el proyecto que se ha de desplegar en el explorador de soluciones. Sin embargo, algunas opciones de despliegue no son gestionadas por este asistente y deben configurarse manualmente a través del cuadro de diálogo de propiedades del proyecto.

La primera etapa del asistente consiste en configurar la ubicación donde se debe hacer la publicación.



Esta ubicación puede ser:

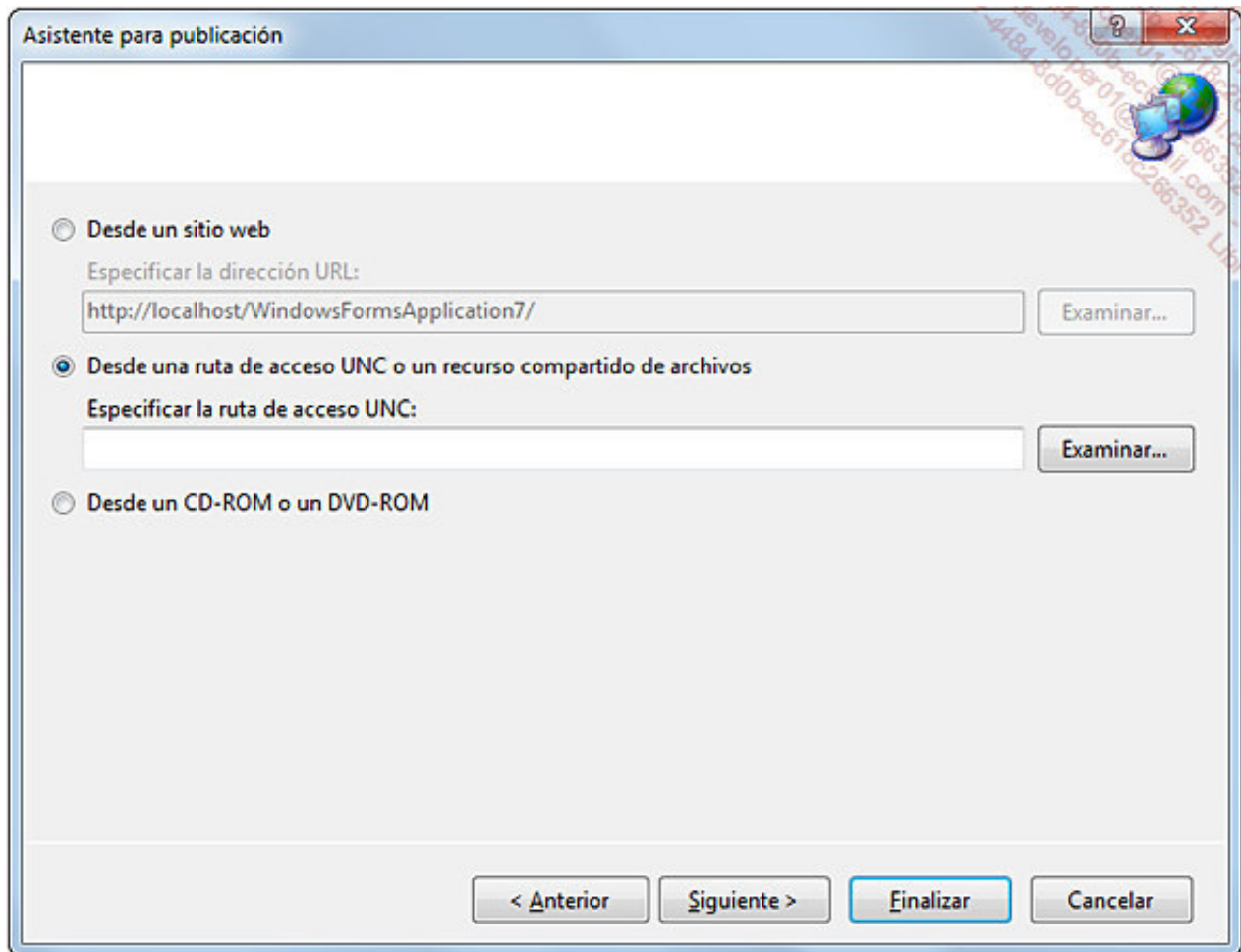
- Un directorio de la máquina.
- Un directorio compartido en otra máquina indicando una ruta UNC de la siguiente manera \\nombre de la máquina\nombre del directorio. Usted debe tener permisos de escritura sobre la partición para que la publicación se pueda realizar.
- El servidor Web IIS de la máquina en la cual usted debe haber añadido previamente un directorio virtual para alojar los archivos.
- Un servidor FTP cuyos datos de conexión debe facilitar en el cuadro de diálogo siguiente:



Debe indicar:

- La dirección IP o el nombre del servidor FTP.
- El número del puerto utilizado para contactar con el servidor (en general, 21).
- El directorio del servidor en el cual se efectuará la copia de los archivos. Debe tener la autorización de escritura en este directorio.
- Si está situado detrás de un cortafuegos activando la opción **Modo pasivo**.
- Si se conecta de manera anónima o en caso contrario, el nombre de usuario y la contraseña utilizados para la conexión.

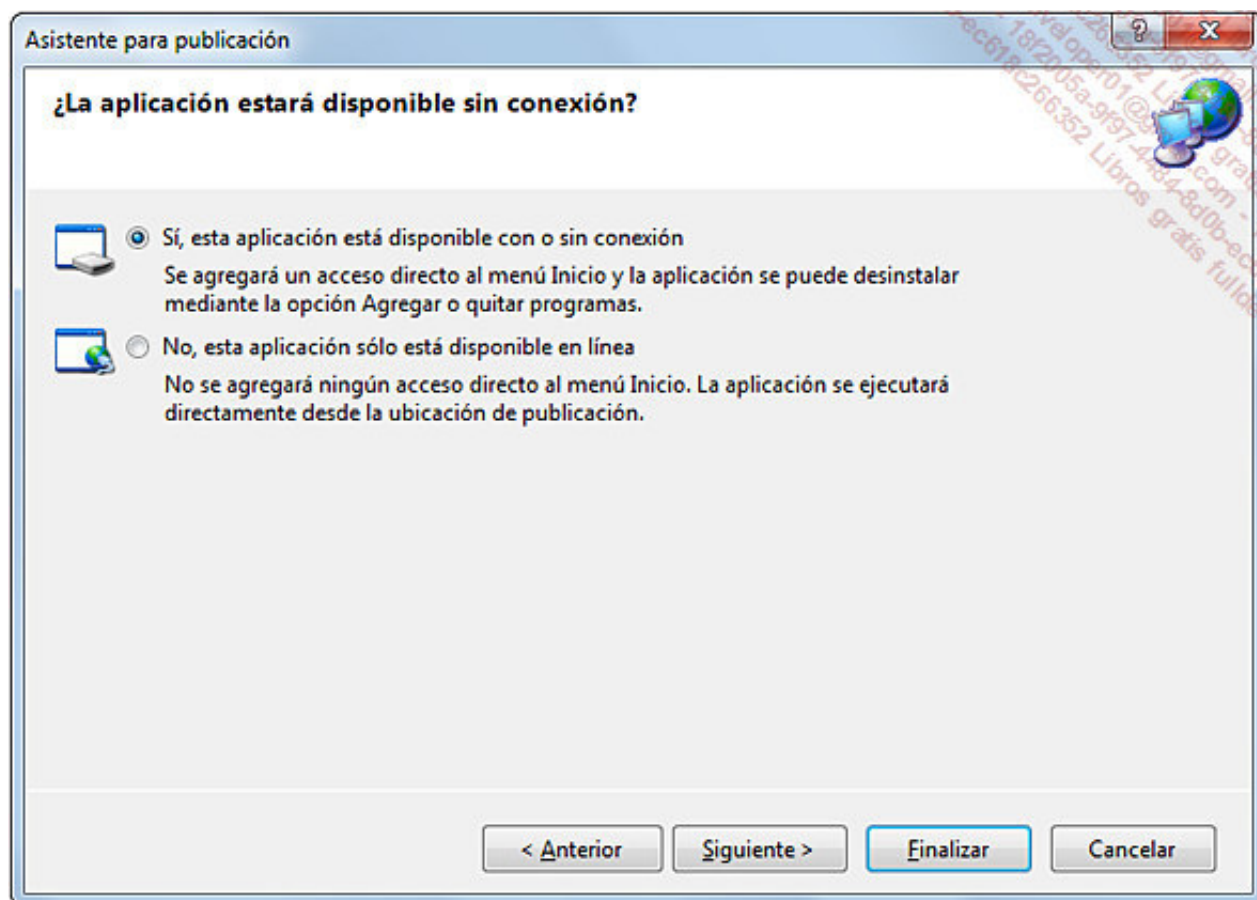
La segunda etapa determina cómo los usuarios van a instalar la aplicación.



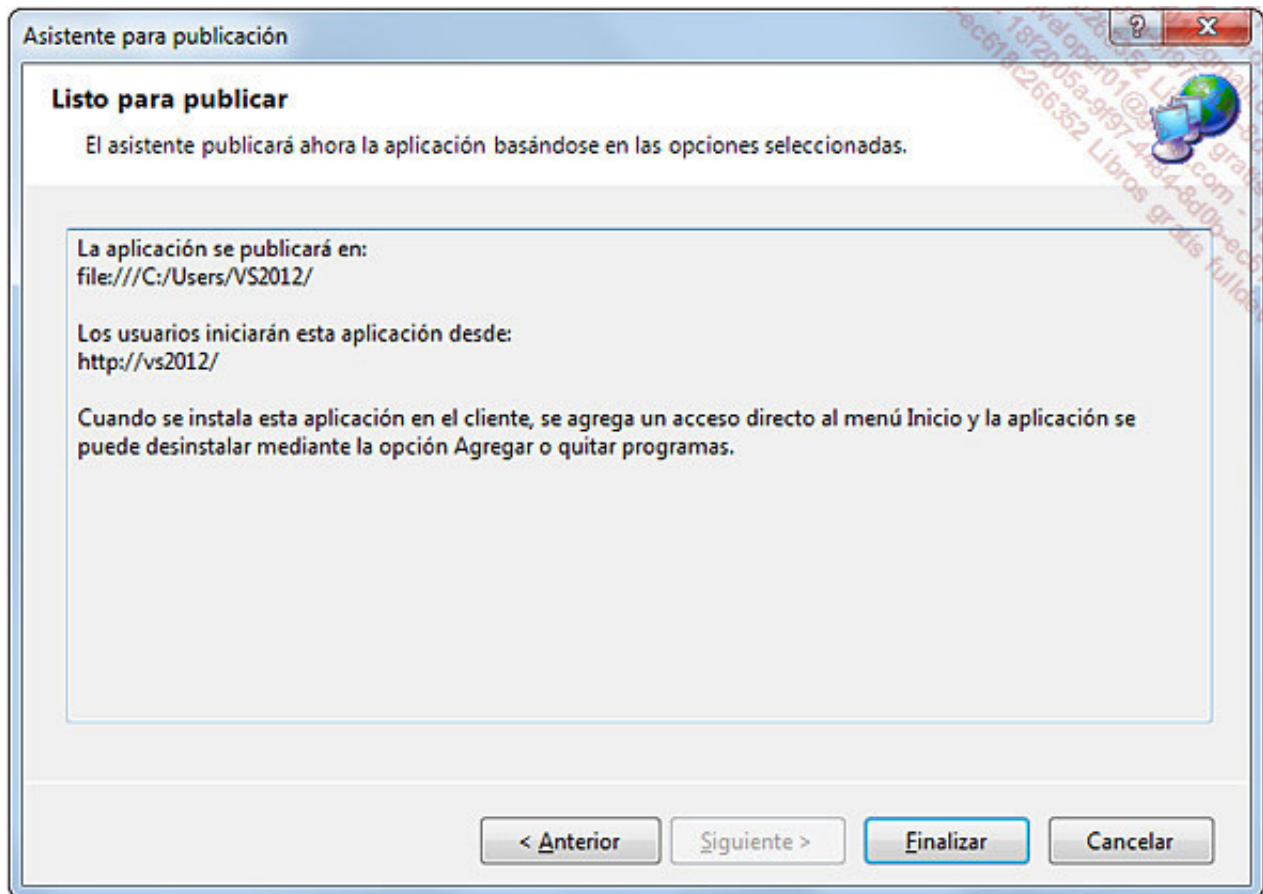
Las opciones posibles son:

- Desde un sitio Web del cual indica la URL.
- Desde una partición de red de la cual especifica la ruta UNC. Por supuesto, los usuarios deberán tener el permiso de lectura sobre la partición. El permiso de escritura no es obligatorio e, incluso, muy desaconsejado.
- Desde un CD-Rom o DVD que usted proporcionará. La creación de este soporte no lo realiza el asistente y se debe efectuar con una aplicación de grabación externa.

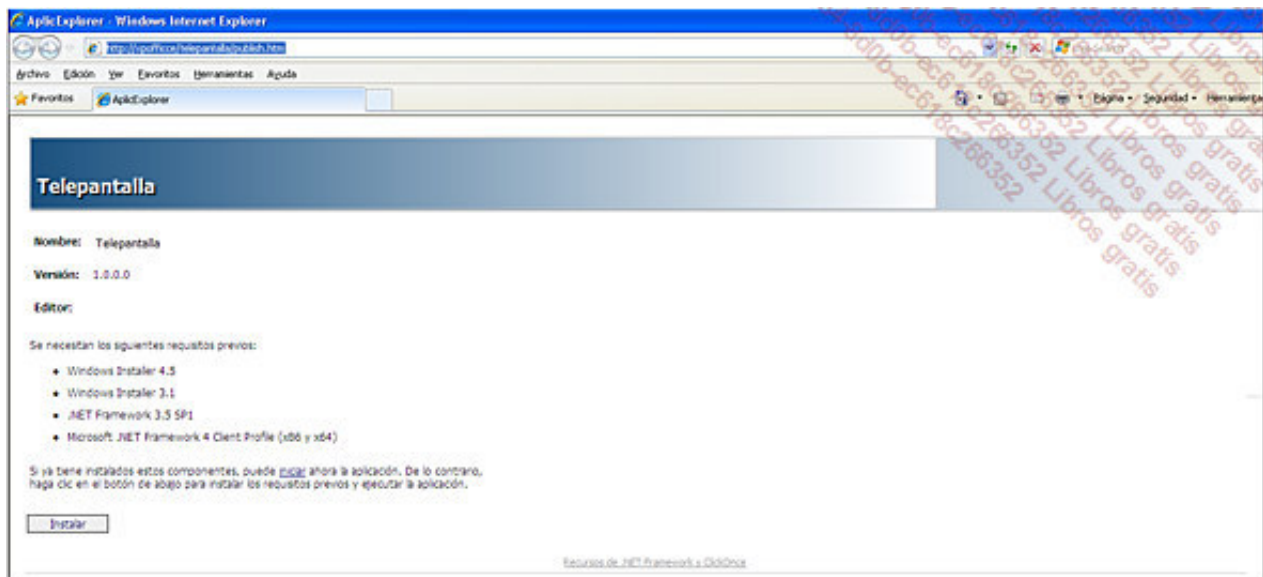
El asistente le propone a continuación configurar la estrategia de ejecución de la aplicación.



La última etapa muestra un resumen de los datos seleccionados y permite iniciar la publicación con el botón **Finalizar**.



Al final de la instalación, una página html se abre en la ubicación utilizada durante la publicación y permite el arranque de la instalación o la ejecución de la aplicación.



Las opciones de despliegue más específicas se deben configurar en la sección **Publicar** propiedades del proyecto. Este cuadro de diálogo retoma las propiedades configuradas por el asistente de publicación.

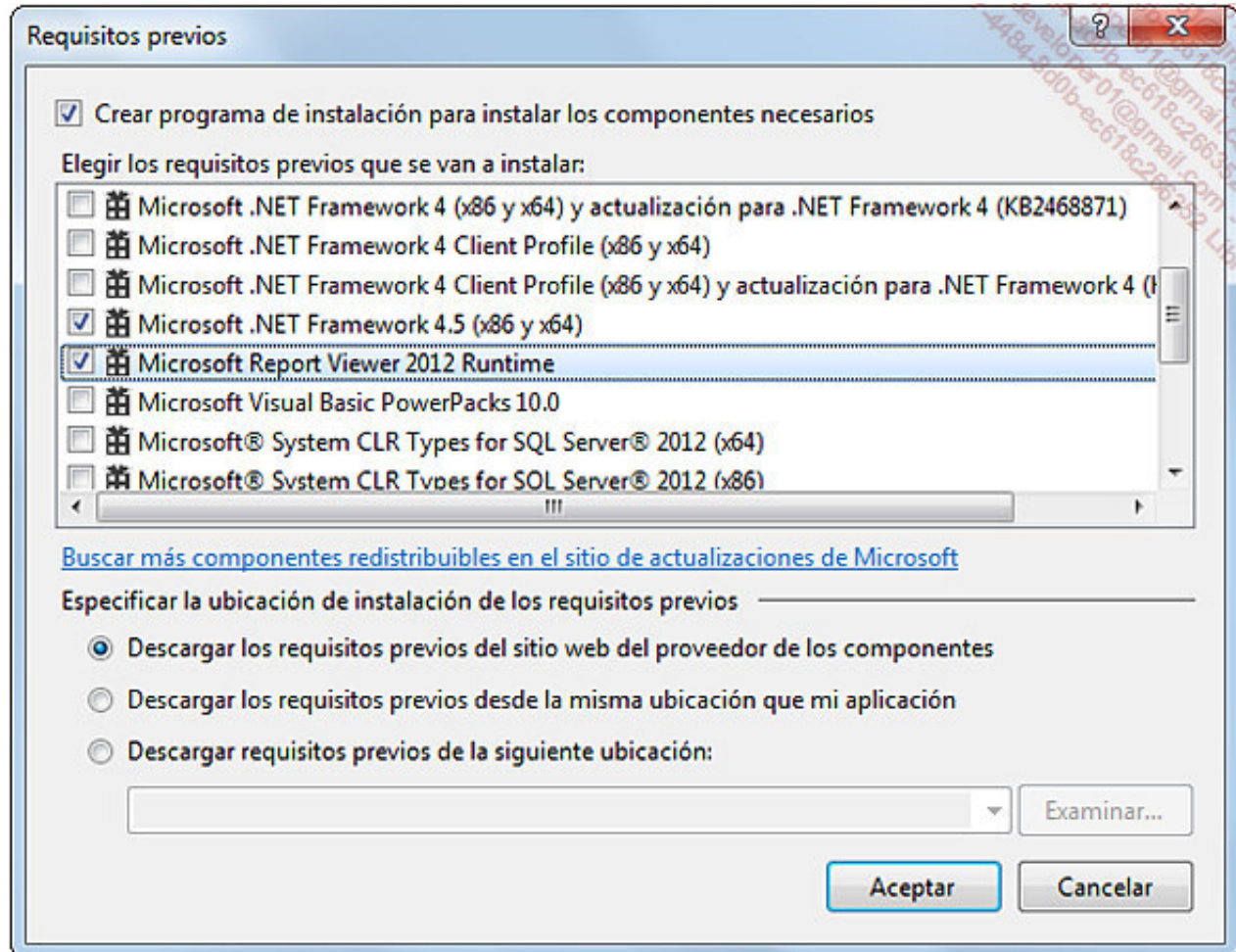
Los botones **Archivos de aplicación**, **Requisitos previos**, **Actualizaciones** y **Opciones** permiten dar el último retoque a estas configuraciones.

El botón **Archivos de aplicación** muestra el cuadro de diálogo siguiente relativo a los archivos que constituyen la aplicación.

Nombre de archivo	Estado de la publicación	Grupo de descarga	Hash
Telepantalla.exe	Incluir (automático)	(Requerido)	Incluir
Telepantalla.exe.config	Incluir (automático)	(Requerido)	Incluir
Telepantalla.exe.manifest	Incluir (automático)	(Requerido)	Incluir

El estado de la publicación de cada archivo se puede configurar con tres valores diferentes:

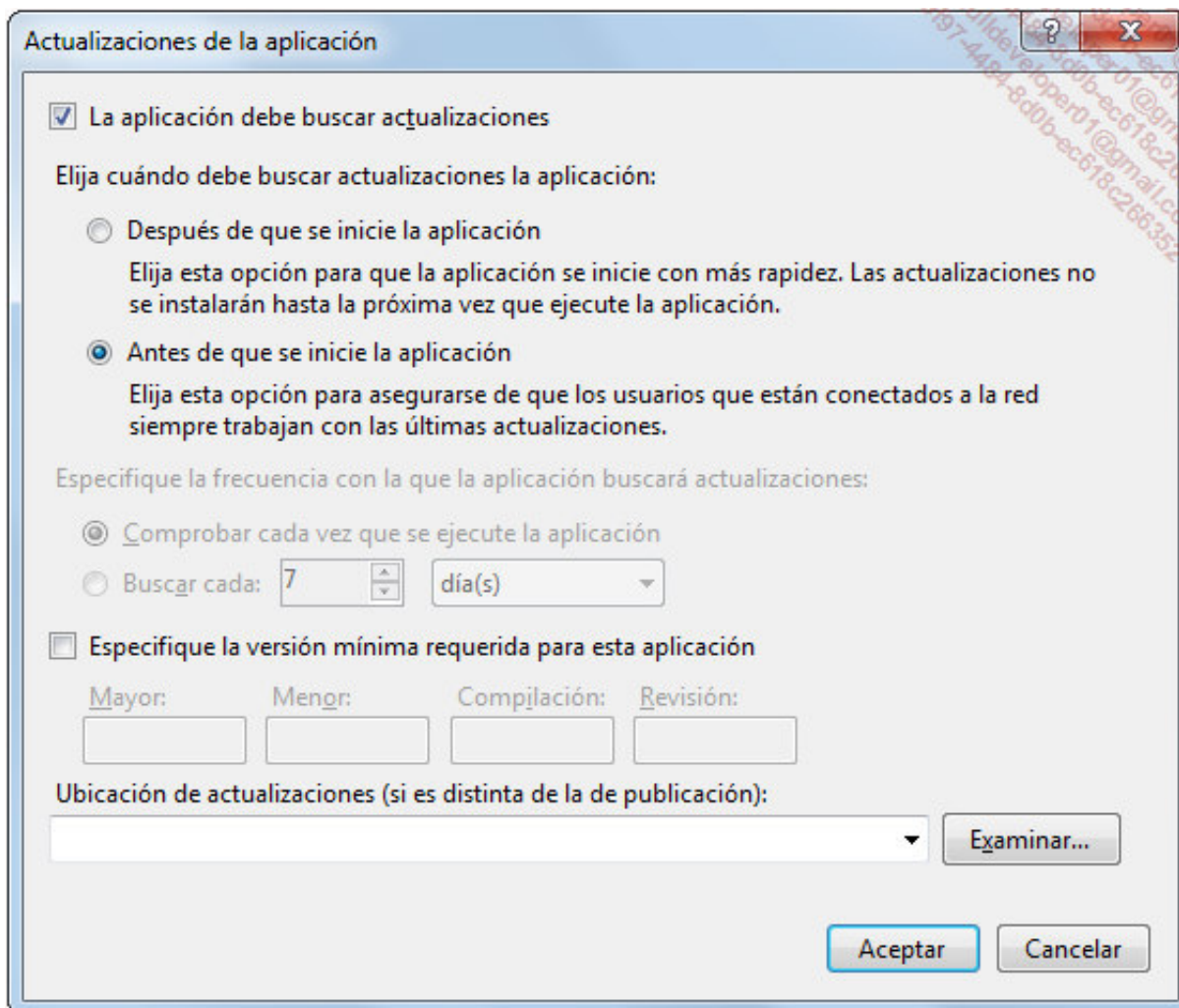
- **Incluir:** el archivo estará disponible para los usuarios en el soporte de despliegue.
- **Excluir:** el archivo no se copiará de nuevo en el soporte de despliegue.
- **Archivo de datos:** el archivo contiene datos necesarios para el correcto funcionamiento de la aplicación y se incluirá en la publicación. El botón **Requisitos previos** se utiliza para configurar los elementos necesarios.



Puede elegir crear un programa de instalación para los componentes necesarios para el funcionamiento de la aplicación marcando la casilla **Crear programa de instalación para instalar los componentes necesarios**. Se deben elegir los componentes concernientes en la lista presentada. También debe indicar desde qué ubicación se instalarán estos componentes. Hay tres opciones posibles:

- desde el sitio Web del proveedor del componente;
- desde la misma ubicación que la utilizada para instalar la aplicación;
- desde la ubicación indicada.

La configuración de las actualizaciones prevista durante la utilización del asistente puede modificarse con el botón **Actualizaciones**.



La casilla de verificación **La aplicación debe buscar actualizaciones** especifica que la aplicación debe verificar la disponibilidad de actualizaciones en el momento de su instalación. Si selecciona esta opción, las otras opciones pasan a estar disponibles. Permiten elegir el momento en que tendrá lugar la verificación de la disponibilidad de una actualización. La opción **Antes de que se inicie la aplicación** indica que la aplicación debe verificar la disponibilidad de las actualizaciones antes del arranque. Esto garantiza que los usuarios conectados a la red siempre disponen de la versión más reciente de la aplicación. Esta opción puede ralentizar el arranque de la aplicación en caso de que haya actualizaciones disponibles. La opción **Después de que se inicie la aplicación** planifica la ejecución de la actualización durante el próximo arranque de la aplicación. La frecuencia de las actualizaciones también se puede indicar con un número de horas, días o semanas, o bien ser ejecutada a cada arranque de la aplicación. También puede indicar la ubicación desde la cual las actualizaciones están disponibles, si ésta es diferente de la ubicación de instalación.

El último botón servirá para configurar varias opciones de despliegue.

Las opciones siguientes están disponibles:

Idioma de publicación

Especifica el idioma (y los parámetros regionales) en el cual se publica la aplicación.

Nombre del editor

Especifica el nombre del editor de la aplicación. Si esta zona está vacía, se usará el valor de la propiedad `RegisteredOrganization` del ordenador. Si este valor es nulo, se utiliza el

nombre del proyecto utilizado.

Nombre de la suite

Especifique el nombre la carpeta del menú **Inicio** en el que se instalará la aplicación.

Nombre del producto

Especifica el nombre del producto de la aplicación. Si el nombre de producto está vacío, se utiliza el nombre del ensamblado.

URL del soporte técnico

Especifica un sitio Web que contiene datos de soporte para su aplicación. La especificación de esta URL es facultativa. Si se utiliza, esta URL aparece en la entrada **Añadir/Suprimir programas** para su aplicación en el **Panel de control** de Windows.

Página Web de despliegue

Especifica un nombre para la página Web de despliegue. El nombre de archivo por defecto es Publish.htm.

Generar automáticamente la página Web de despliegue después de cada publicación

Si esta opción está seleccionada, el proceso de publicación genera una página Web de despliegue a cada publicación. Esta opción sólo está disponible si se especifica una página Web de despliegue.

Abrir la página Web de despliegue después de la publicación

Si se selecciona esta opción, la página Web de despliegue generada automáticamente se abre después de la publicación.

Bloquear la activación de la aplicación a través de una URL

Si esta opción está desactivada, la aplicación se ejecuta automáticamente después de la instalación. Si está activada, el usuario deberá arrancar la aplicación desde el acceso directo del programa en el Menú **Inicio**.

Utilizar la extensión de archivo «.deploy»

Si esta opción está seleccionada, el archivo de despliegue utiliza la extensión .deploy. Algunos servidores Web están configurados para bloquear, por razón de seguridad, los archivos que no suelen estar presentes en un contenido Web. Por ejemplo, los archivos que llevan las extensiones siguientes se pueden bloquear: .dll, .config, .mdf. Las aplicaciones de Windows suelen contener archivos con algunas de estas extensiones. Si un usuario intenta ejecutar una aplicación ClickOnce que accede a un archivo bloqueado en un servidor Web, se produce un error. En vez de desbloquear todas las extensiones de archivo, se publica cada archivo de aplicación por defecto con una extensión de archivo «.deploy». Si se hace uso de esta opción, el servidor Web sólo debe configurarse para desbloquear las tres extensiones de archivos siguientes:

- .application
- .manifest
- .deploy

Autorizar la transferencia de los parámetros de URL hacia la aplicación

Por defecto esta opción está desactivada. Si esta opción está activada, la aplicación será capaz de acceder y tratar los datos de parámetros de la URL.

Para las instalaciones desde un CD-ROM, arrancar automáticamente la instalación en el momento de la inserción del CD-ROM

Si esta opción está seleccionada, añade un archivo Autorun.inf a la raíz del soporte para las aplicaciones ClickOnce que están instaladas a partir de un CD-Rom o DVD-Rom.

Verificar los archivos tranferidos a un servidor Web

Si esta opción está activada, el proceso de publicación descarga cada archivo para verificar que efectivamente se pueden descargar. Se le informará de los archivos que no se pueden descargar.

Usar el manifiesto de la aplicación para la información de confianza

Cuando esta opción está seleccionada, puede firmar de nuevo el manifiesto de la aplicación con la ayuda de un certificado que contiene sus propios datos.